

OpenFOAM

A little User-Manual

Gerhard Holzinger^{*†}

23rd March 2020

Abstract

This document is a collection of my own experience on learning and using OpenFOAM. Herein, knowledge and background information is assembled which may be useful to others when learning to use OpenFOAM.

WARNING:

During the assembly of this manual OpenFOAM and other tools, e.g. *pyFoam*, have been continuously updated. This manual was started with OpenFOAM-2.0.x installed and over the time the author has worked with all major point releases of OpenFOAM and the development versions. Consequently it is possible that some parts may be outdated by the time you read this. Furthermore, functionalities may have been extended, modified or superseded. Nevertheless, this manual is intended to cast some light on the inner workings of OpenFOAM and explain the usage in a rather practical way.

Furthermore, this document is, and will always be, *work in progress*. As it is extended whenever something interesting is encountered or learned, parts of the document will always be fragmentary. All errors and omissions are the sole product of the author.

All information contained in this manual can be found in the internet (<http://www.openfoam.org>, <http://www.cfd-online.com/Forums/openfoam/>); in the source code of OpenFOAM, or it was gathered by trial and error (*What happens if ...? Why did that happen?!*). All errors in this manual are the solely the product of the author. The interested reader is invited to contact the author to point out any discovered errors.

This offering is not approved or endorsed by ESI® Group, ESI-OpenCFD® or the OpenFOAM® Foundation, the producer of the OpenFOAM® software and owner of the OpenFOAM® trademark.

^{*}currently K1MET GmbH, Linz, Austria, <http://www.k1-met.com>

[†]formerly Particulate Flow Modelling, Johannes Kepler University, Linz, Austria, <http://www.jku.at/pfm/>

Contents

1	Getting help	14
2	Lessons learned	15
2.1	Philosophy	15
2.2	Learning by using OpenFOAM	16
2.3	Learning by tinkering with OpenFOAM	17
	I Installation	18
3	Install OpenFOAM	18
3.1	Prerequisites	18
3.2	Download the sources	18
3.3	Compile the sources	19
3.4	Install paraView	19
3.5	Remove OpenFOAM	19
3.6	Install several versions of OpenFOAM	20
4	Updating the repository release of OpenFOAM	20
4.1	Version management	20
4.2	Check for updates	21
4.3	Check for updates only	21
4.4	Install updates	22
4.5	Problems with updates	22
5	Updating OpenFOAM-6 source installation	23
5.1	Motivation	23
5.2	Make the OpenFOAM-6 source pack able to being updated	24
5.3	Recompile the source pack	25
6	Maintaining your OpenFOAM installation	26
6.1	Dealing with OS updates	26
6.2	Dealing with OS upgrades	26
7	Install third-party software	27
7.1	Install <i>pyFoam</i>	27
7.2	Install swak4foam	27
7.3	Compile external libraries	27
8	Setting up the environment	27
8.1	Sourcing OpenFOAM	27
8.2	Useful helpers	28
	II General Remarks about OpenFOAM	29
9	Units and dimensions	29
9.1	Unit inspection	29
9.2	Dimensions	31
9.3	Kinematic viscosity vs. dynamic viscosity	32
9.4	Pitfall: pressure vs. pressure	32
10	Files and directories	33
10.1	Required directories	33
10.2	Supplemental directories	34
10.3	Files in <i>system</i>	34

11 Controlling OpenFOAM	35
11.1 The means of exerting control	35
11.2 Syntax of the dictionaries	36
11.3 The <code>controlDict</code>	38
11.4 Run-time modifications of dictionaries	44
11.5 The <code>fvSolution</code> dictionary	44
11.6 Command line arguments	44
12 Usage of OpenFOAM	45
12.1 Use OpenFOAM	45
12.2 Abort an OpenFOAM simulation	47
12.3 Terminate an OpenFOAM simulation	48
12.4 Continue a simulation	51
12.5 Do parallel simulations with OpenFOAM	51
12.6 Using tools	55
12.7 Using OpenFOAM on a local machine and a cluster	56
III dynamicCode	56
IV Pre-processing	58
13 Mesh basics	58
13.1 Basics of the mesh	58
13.2 General advice on meshing and mesh/case manipulation	60
14 Geometry creation & other pre-processing software	60
14.1 <i>blockMesh</i>	60
14.2 CAD software	60
14.3 Salome	61
14.4 GMSH	62
15 <i>blockMesh</i>	62
15.1 The block	62
15.2 The <code>blockMeshDict</code>	62
15.3 Create multiple blocks	73
15.4 <i>Grading</i>	75
15.5 Parametric meshes by the help of <i>m4</i> and <i>blockMesh</i>	78
15.6 Trouble-shooting	83
16 <i>snappyHexMesh</i>	84
16.1 Documentation	84
16.2 Work flow	85
16.3 Example: Bath Tub	85
17 <i>foamyHexMesh</i>	88
17.1 Crude comparison between a snappy and a foamy bath tub	88
18 <i>cfMesh</i>	90
18.1 Usage	90
19 <i>checkMesh</i>	92
19.1 Definitions	93
19.2 Pitfalls	98
19.3 Useful output	102
20 <i>extrudeMesh</i>	103
20.1 Control	103
21 <i>polyDualMesh</i>	107
22 <i>combinePatchFaces</i>	109

23 Salome	110
23.1 Export & Conversion	110
23.2 Combined operations: blockMesh & salome	111
24 Gmsh	113
25 enGrid	114
26 Mesh converters	114
26.1 <i>fluentMeshToFoam</i> and <i>fluent3DMeshToFoam</i>	115
26.2 ideasUnvToFoam	115
26.3 Pitfall: length units	115
27 Other mesh manipulation tools	116
27.1 <i>transformPoints</i>	116
27.2 <i>topoSet</i>	116
27.3 <i>setsToZones</i>	118
27.4 <i>refineMesh</i>	118
27.5 <i>refineWallLayer</i>	118
27.6 <i>renumberMesh</i>	121
27.7 <i>subsetMesh</i>	124
27.8 <i>createPatch</i>	125
27.9 <i>stitchMesh</i>	125
27.10 <i>tetDecomposition</i>	125
27.11 <i>decomposePar</i>	126
27.12 <i>mirrorMesh</i>	126
28 Surface mesh manipulation tools	128
28.1 <i>surfaceAdd</i>	128
28.2 <i>surfaceSubset</i>	128
28.3 <i>surfaceFeatureExtract</i>	128
28.4 <i>surfaceFeatureConvert</i>	128
28.5 <i>surfaceTransformPoints</i>	128
28.6 Third party surface manipulation tools	128
28.7 The Linux command line	129
29 Initialize Fields	130
29.1 Basics	130
29.2 <i>setFields</i>	131
29.3 <i>mapFields</i>	133
30 Case manipulation	136
30.1 <i>changeDictionary</i>	137
30.2 The allmighty Linux Terminal	139

V Modelling 143

31 Solution dimensions	143
31.1 Basic rules for simulations with reduced solution dimensions	143
32 Turbulence-Models	144
32.1 Organisation	144
32.2 Reynolds averaged models (RAS)	150
32.3 Categories	151
32.4 RAS-Models	151
32.5 LES-Models	153
32.6 Pitfalls	153

33 Thermophysical modelling	156
33.1 The modelling framework	156
33.2 Model selection	160
33.3 transport models	161
33.4 thermo	163
33.5 equationOfState	164
33.6 Thermophysical properties	165
33.7 Mixture	166
33.8 Model combination	167
33.9 Solids : Thermophysical modelling	167
34 Radiation modelling	170
34.1 Discrete ordinates - fvDOM	171
34.2 P1	172
34.3 View factor	172
35 Eulerian multiphase modelling	175
35.1 Phase model class	176
35.2 Phase system classes	181
35.3 Turbulence modelling	183
35.4 Interfacial momentum exchange	183
35.5 Diameter models	184
35.6 Thermophysical models	186
36 Boundary conditions	187
36.1 <i>Base types</i>	187
36.2 <i>Primitive types</i>	188
36.3 <i>Derived types</i>	188
36.4 Pitfalls	188
36.5 Time-variant boundary conditions	189
37 Mesh interfaces: AMI and ACMI	190
37.1 AMI and ACMI in brevity	190
37.2 Arbitrary Mesh Interface - AMI	190
37.3 Arbitrary Coupled Mesh Interface - ACMI	192
37.4 Avoiding errors	193
38 The MRF method	193
38.1 Usage	193
38.2 Avoiding errors	194
39 The fvOption framework	195
39.1 Controlling space & time	197
39.2 Types of options	197
39.3 Sources	198
40 The Lagrangian world	200
40.1 Background	200
40.2 Libraries	201
40.3 Cloudy, with a chance of particles	202
40.4 Cloudy Templates	205
40.5 Run-time post-processing	207
40.6 Times of Use	207
40.7 Sub models	207
VI Solver	210
41 Solution Algorithms	210
41.1 SIMPLE	210
41.2 PISO	212
41.3 PIMPLE	212
41.4 Block-coupled solution	212

42	<i>pimpleFoam</i>	213
42.1	Governing equations	213
42.2	The PIMPLE Algorithm – or, what’s under the hood?	215
43	<i>rhoPimpleFoam</i>	220
43.1	General remarks	220
43.2	Solution algorithm	220
43.3	Governing equations	220
44	<i>twoPhaseEulerFoam</i>	221
44.1	General remarks	221
44.2	Solver algorithm	222
44.3	Momentum exchange between the phases	224
44.4	Kinetic Theory	227
45	<i>twoPhaseEulerFoam-2.3</i>	227
45.1	Physics	227
45.2	Naming scheme	227
45.3	Solver capabilities	228
45.4	Turbulence models	228
45.5	Energy equation	235
45.6	Momentum equation	236
45.7	Interfacial interaction	238
45.8	Interfacial momentum exchange	241
45.9	MRF method - avoiding errors	247
46	<i>reactingTwoPhaseEulerFoam</i>	248
46.1	Solver basics	248
46.2	Phase modelling	248
46.3	Turbulence modelling	251
46.4	Interfacial momentum exchange	251
47	<i>multiphaseEulerFoam</i>	251
47.1	Fields	251
47.2	Momentum exchange	251
48	<i>driftFluxFoam</i>	252
48.1	Governing equations	252
48.2	<i>incompressibleTwoPhaseInteractingMixture</i>	255
48.3	Mixture viscosity models	255
48.4	Relative velocity models - hindered settling	256
48.5	<i>settlingFoam</i>	258
	VII Postprocessing	260
49	<i>functions</i>	260
49.1	Stay up to date	260
49.2	Definition	261
49.3	Control	262
49.4	<i>probes</i>	263
49.5	<i>fieldAverage</i>	265
49.6	<i>faceSource</i>	265
49.7	<i>cellSource</i>	267
49.8	<i>readFields</i>	267
49.9	<i>writeObjects</i>	268
49.10	Execute C++ code as <i>functionObject</i>	268
49.11	<i>wallHeatFlux</i>	269
49.12	Execute <i>functions</i> after a simulation has finished	270
50	<i>sample</i>	271
50.1	Usage	271
50.2	<i>sampleDict</i>	271
50.3	Update OpenFOAM-4	274

51	<i>ParaView</i>	274
51.1	Reader's choice	274
51.2	View the mesh	277
51.3	Saving animations	279
52	postProcess	281
52.1	Usage	281
VIII External Tools		282
53	<i>pyFoam</i>	282
53.1	Installation	282
53.2	<i>pyFoamPlotRunner</i>	282
53.3	<i>pyFoamPlotWatcher</i>	282
53.4	<i>pyFoamClearCase</i>	287
53.5	<i>pyFoamCloneCase</i>	287
53.6	<i>pyFoamDecompose</i>	287
53.7	<i>pyFoamDisplayBlockMesh</i>	288
53.8	<i>pyFoamCaseReport</i>	289
54	<i>swak4foam</i>	289
54.1	Installation	289
54.2	<i>simpleSwakFunctionObjects</i>	290
55	<i>blockMeshDG</i>	291
55.1	Installation	291
55.2	Usage	291
55.3	Pitfalls	291
IX Source Code & Programming		293
56	Understanding some C and C++	293
56.1	Definition vs. Declaration	293
56.2	Namespaces	293
56.3	<code>const</code> correctness	294
56.4	Function inlining	295
56.5	Constructor (de)construction	296
56.6	Object orientation	298
56.7	Templates	298
57	Under the hood of OpenFOAM	300
57.1	Solver algorithms	300
57.2	Namespaces	300
57.3	Keyword lookup from dictionary	301
57.4	OpenFOAM specific datatypes	303
57.5	OpenFOAM specific macros for convenient programming	311
57.6	Time management	312
57.7	The registry	321
57.8	I/O - input & output	325
57.9	Making an argument – passing arguments	329
57.10	Turbulence models	330
57.11	Debugging mechanism	333
57.12	A glance behind the run-time selection and debugging magic	334
57.13	Notes on running OpenFOAM in parallel	338
57.14	Math-like syntax in OpenFOAM	340

58 General remarks on OpenFOAM programming	341
58.1 Preparatory tasks	341
58.2 Start from existing code	341
58.3 Create the source code from scratch	343
58.4 Using a user-created libraries	343
58.5 Pitfalls	344
58.6 Tips	345
 X Theory	 346
59 Discretization	346
59.1 Temporal discretization	346
59.2 Spatial discretization	346
59.3 Continuity error correction	346
60 Momentum diffusion in an incompressible fluid	349
60.1 Governing equations	349
60.2 Implementation	349
61 The incompressible k-ϵ turbulence model	350
61.1 The k- ϵ turbulence model in literature	350
61.2 The k- ϵ turbulence model in OpenFOAM	351
61.3 The k- ϵ turbulence model in <i>bubbleFoam</i> and <i>twoPhaseEulerFoam</i>	353
61.4 Modelling the production of turbulent kinetic energy	354
62 Some theory behind the scenes of LES	358
62.1 LES model hierarchy	358
62.2 Eddy viscosity models	359
63 The use of phi	363
63.1 The question	363
63.2 Implementation	363
63.3 The math	365
63.4 Summary	366
64 Derivation of the IATE diameter model	366
64.1 Number density transport equation	367
64.2 Interfacial area transport equation	367
64.3 Interfacial curvature transport equation	369
64.4 Interaction models	371
64.5 Appendix	375
65 Derivation of the MRF approach	377
65.1 Preliminary observations	377
65.2 Mass conservation equation	377
65.3 Momentum conservation equation	378
65.4 Notes on the implementation of the MRF Approach	379
 XI Appendix	 382
66 Useful Linux commands	382
66.1 Getting help	382
66.2 Finding files	382
66.3 Find files and scan them	383
66.4 Scan a log file	383
66.5 Running in scripts	384
66.6 diff	385
66.7 Case setup	386
66.8 Miscellaneous	386
67 Archive data	387

Bibliography	390
List of Abbreviations	393

List of Figures

1	Initializing a new git repository.	24
2	Pointing the new git repository to the proper remote repository.	24
3	Updating the new git repository by pulling from the remote repository.	25
4	Updating the local files of our OpenFOAM-6 installation.	25
5	The top face of the generic block of Figure 7	59
6	The STL mesh of a circular area generated by OpenSCAD	61
7	The generic block	63
8	Creating a wedge geometry for 2D, axisymmetric domains. Reproduced after [50].	65
9	A block with a poly-line at the left side. The red line indicates the poly-line. This figure makes it obvious that edges defines in the <code>blockMeshDict</code> serve to compute the locations of the block's internal nodes. The block itself however, does not obey the poly-line.	68
10	The initial velocity field depending on the order of the <code>wall</code> and <code>banana</code> . Left: Setting as in Listing 105. Right: <code>wall</code> and <code>banana</code> have changed places.	71
11	The mesh of two merged blocks	73
12	The mesh of two merged blocks.	73
13	Two connected blocks	74
14	Two unconnected blocks	75
15	The mesh of a stirred tank with a Rushton impeller, stator baffles and an aeration device.	81
16	The blocks of a parametric mesh consisting of nine blocks.	84
17	A bath tub. The outlet patch is marked grey at the very bottom of the drain tube.	85
18	A badly chosen <code>featureAngle</code> causes snappy to add incomplete boundary layers.	86
19	The boundary layers added by snappy. On the left, layer addition went as we intended it to do; on the right, we see the effect of the (missing) keyword <code>slipFeatureAngle</code> of the <code>addLayersControls</code> dictionary of <code>snappyHexMeshDict</code>	87
20	A collapsing boundary layer. Maybe we did not want the mesh that way, however, we told <i>snappy</i> to create it exactly that way.	88
21	A bath tub with a background mesh enclosing the STL-surface of the bath tub.	89
22	SnappyBathTub	89
23	FoamyBathTub	90
24	Poor feature edge resolution caused by not providing information on feature edges. Note, the whole geometry is bounded by a single patch.	91
25	Resolved feature edge of the bath tub. In this case, the boundary consists of two patches: the top surface and the rest.	92
26	Definition of non-orthogonality for internal faces	93
27	Definition of non-orthogonality for boundary faces	94
28	Definition of skewness of internal faces	95
29	Definition of skewness of boundary faces	97
30	Face warpage	98
31	A distorted mesh	99
32	The cells with two internal faces in an all-tet, 3D-mesh.	101
33	The under-determined cells, which were found by <i>checkMesh</i> in the mesh of an axi-symmetric simulation. These cells are on a far corner of the 2-D domain, and grading towards the more interesting regions led them to have a high aspect ratio. In fact, these cells have the highest aspect ratio of the whole mesh. The proximity to the lower wall results in these cells to be quite fine in the <i>y</i> -direction. Since, these cells are furthest from the axis of symmetry, they are relatively large in radial and tangential direction.	102
34	Sets created by <i>checkMesh</i> in the <code>sets</code> directory.	103
35	Extrude the wall patch from a cylinder mesh. left: <code>constructFrom mesh</code> . right: <code>constructFrom patch</code>	104
36	The mesh for a 2D study generated from an STL surface.	105
37	A cheap 90° pipe bend. The outlet patch of the original mesh was extruded along the sector of a circle.	105
38	Subsequent mesh extrusions: <code>sector</code> , <code>linearNormal</code> and <code>linearDirection</code>	106
39	Grow a wall! The <i>walls</i> patch of the pipe mesh was extruded using the <code>linearNormal</code> model.	106
40	Extruding a slice. Left: a 1 degree sector extrusion, Right: a 5 degrees sector extrusion.	107

41	Connecting the centers of the circumcircles produces the Voronoi diagram (in red). Source https://commons.wikimedia.org/wiki/File:Delaunay_Voronoi.svg .	107
42	The mesh of the <i>elbow</i> tutorial case; before and after the application of <code>polyDualMesh</code> .	108
43	The mesh of the <i>bubble column</i> tutorial case; before and after the application of <code>polyDualMesh</code> .	108
44	The dual mesh of a single tetrahedron: the original tet-cell is outlined in blue, the face-decomposition is outlined in black, and one of the resulting cells is shown in grey.	109
45	The dual mesh of a single hexahedron: the original hex-cell is outlined in blue, the face-decomposition is outlined in black, and one of the resulting cells is shown in grey.	109
46	The dual mesh of the <i>elbow</i> tutorial case; before and after the application of <code>combinePatchFaces</code> .	110
47	Mesh export issue in Salome with the UNV format.	110
48	A big simulation domain with a quite small geometric feature: the geometry.	111
49	A big simulation domain with a quite small geometric feature: the <i>blockMesh</i> -only mesh.	112
50	The mesh of the cut-out block in salome.	112
51	Meshing of a block in salome.	113
52	Combining blockMesh and salome: the meshed block was re-combined with the initial mesh.	113
53	An extruded 2D mesh of quad elements created with Gmsh.	114
54	Meshes by enGrid: left : tet-mesh with prismatic boundary layer, right : polyhedral mesh with boundary layer.	114
55	A faulty cell set definition. The red cells are part of the cell set. All other cells are blue.	117
56	An example of a refined mesh. The refined region is marked in red.	118
57	The base mesh for the wall layer refinement.	119
58	Applying the wall layer refinement once.	119
59	Applying the wall layer refinement a second time.	119
60	Applying the wall layer refinement twice on the horizontal patch at a concave edge.	120
61	Applying the wall layer refinement twice on both patches of a convex edge.	120
62	Applying the wall layer refinement twice on one patch of a convex edge.	120
63	Applying the wall layer refinement successively on two patches of a convex edge. This approach leads to a different outcome than the one shown in Figure 61.	121
64	A simple mesh with 8 cells and different cell labelling schemes.	122
65	The connectivity graph of our mesh.	122
66	The matrix structure of the connectivity graph of Figure 65	122
67	Scrambled cell sets caused by mesh renumbering	123
68	Renumbering the solution of the cavity case: the case was run, and all time steps prior to 0.3 were deleted. Then <code>renumberMesh -overwrite</code> was run. As 0.3 was the first time step, the fields in the time step 0.3 were renumbered along with the mesh. The later time steps, however, were left untouched.	124
69	A tet-decomposed tetrahedron: the original tet-cell is outlined in blue, the face-decomposition is outlined in black, and one of the resulting sub-tets is shown in grey.	125
70	A tet-decomposed hexahedron: the original hex-cell is outlined in blue, the face-decomposition is outlined in black, and one of the resulting sub-tets is shown in grey.	126
71	The cell distribution of a multi-region case, with 2 regions and 4 sub-domains for parallel processing. The small region outlined in white is the solid region, the surrounding larger region is the fluid region of this case. Each sub-domain (colour-coded from 0 to 3) is assigned a chunk of each region.	126
72	A cellSet after running <code>mirrorMesh</code> to mirror the mesh using the $x - y$ plane.	127
73	The mapped field	136
74	The unmapped fields	137
75	Established flow and modified boundary condition	139
76	The class hierarchy of the basis of the old turbulence model framework.	145
77	The class hierarchy of the basis of the new turbulence model framework.	147
78	The (templated) class hierarchy of the new turbulence model framework.	148
79	The class hierarchy of the elementary turbulence models of the new turbulence model framework.	149
80	The class hierarchy of a selection of turbulence models of the new turbulence model framework.	150
81	The class hierarchy of the basis of the thermophysical modelling framework in OpenFOAM-7.	157
82	The class hierarchy of the templated classes of the thermophysical modelling framework in OpenFOAM-7.	160

83	The dynamic viscosity of nitrogen: the model coefficients were fitted to data reported in [14], the model is compared to values reported in [2].	161
84	The thermal conductivity of nitrogen: the model is using a modified Eucken correlation with the viscosity model, which was fitted to viscosity data reported in [14], the model is compared to values of thermal conductivity reported in [2, 14].	162
85	Specific heat capacity C_p of air computed with the JANAF model compared to data from [62].	164
86	The thermal conductivity of nickel: data from literature [27], and the available, fitted models of OpenFOAM.	169
87	The result of <code>faceAgglomerate</code> and <code>viewFactorGen</code> applied to a slightly refined cavity case. The patches <i>movingWall</i> and <i>fixedWalls</i> have both been agglomerated, i.e. for the purposes of computing the view factor, several faces of a patch form a coarsened face. This is evident from the visualisation of the rays created by <code>viewFactorGen</code> , each bundle of rays starts from the center of such a coarsened face. The colour of the rays is a mere ID, not the actual view factor.	173
88	One of the aggressively agglomerated faces: On the right boundary the temperature is 300 K, yet the radiative heat-flux computed by the mean temperature cools the solid down to 165 K, which is very unphysical for a case that computes a heated solid interacting with a body of gas. No cooling should occur at all.	175
89	Modelling approach on the example of a gas-liquid two-phase system.	176
90	Modelling approach on the example of a gas-liquid two-phase system.	184
91	Our simulation domain. The block structure of outer domain, in blue, does not match the block structure of the inner domain, in red. However, by keeping the two regions separate, we can mesh each region individually, which is fairly easy, and use AMI to essentially connect the two regions. After meshing, we can rotate the inner domain with respect to the outer domain to change the orientation of the triangle in the center of the inner domain.	191
92	Varying the orientation of the investigated body. Since the body-fitting mesh of the inner domain is in an over-all cylindrical shape, we simply can rotate the inner domain with respect to the outer domain, to change the orientation of the investigated body with respect to the incident flow. After creating the mesh, we can use <code>moveMesh</code> to rotate the inner domain. With appropriate settings for the angular velocity and the write interval, we can get a mesh for any orientation of the triangular prism.	191
93	A 2D simulation of the triangular, prismatic body in laminar cross-flow using AMI to connect the body fitted mesh of the inner region with the mesh of the outer domain.	192
94	A simulation domain with two unconnected regions. Note that the inner region only partially fills the void of the outer region.	192
95	The inner domain in more detail. The block-structure of the inner domain is clearly recognizable. The remaining void is the body under investigation, in this case a semi-cylinder.	193
96	A 2D simulation of the semi-cylinder in laminar cross-flow using ACMI to connect the mesh of the inner region with the mesh of the outer domain.	193
97	A baffled stirred tank with a Rushton impeller. The stator patch is shown in grey and the rotor patch is shown in red. The white wireframe shows the boundary of the rotor zone. For all cells of the rotor zone the MRF method is applied.	194
98	Half a baffled stirred tank with a Rushton impeller. The <i>cyclic</i> -type patches are shown as coloured wireframes, and all other patches are shown as coloured surfaces.	195
99	Schematic diagrams of doubly-linked lists.	204
100	The class hierarchy needed for intrusive lists of objects of type <code>T</code> ;	204
101	The class hierarchy of the class <code>basicKinematicCloud</code>	206
102	A set of polygons has been defined to count and remove traversing particles. In this case of a cylinder in laminar cross-flow, particles are inserted through the inlet patch. The <code>ParticleCollector</code> cloud function object was set to remove all counted particles, which is clearly visible in this snapshot.	209
103	Flow chart of the SIMPLE algorithm	211
104	Flow chart of the PISO algorithm	212
105	Flow chart of the PIMPLE algorithm	216
106	Flow chart of the main loop of <i>twoPhaseEulerFoam</i>	223
107	Flow chart of the operations in <code>alphaEqn.H</code>	225
108	Air volume fraction of the bubble column. Initial field (left) and solution at $t = 10$ s (right).	234
109	Linear blending: <code>f1</code> over α	241
110	Hyperbolic blending: <code>f1</code> over α	241

111	Velocity vectors of the gaseous phase at the inlet boundary (red vectors) in an aerated stirred tank. That the gas inlet boundary lies within the MRF zone. On the left, we see the initial condition and on the right we see the boundary condition after the constraints by the MRF method have been applied.	248
112	A part of the directory tree after the simulation ended	264
113	The content of the <code>postProcessing</code> folder	267
114	Directory tree after compilation of a coded functionObject	269
115	Launching ParaView with OpenFOAM's reader for OpenFOAM cases.	275
116	Launching ParaView with its native reader for OpenFOAM cases.	275
117	Using ParaView with its native reader to read a multi-region OpenFOAM case.	276
118	The case directories of a decomposed, multi-region case with the initial mesh still present. This directory tree was generated by calling the following command from the case directory: <code>tree -L 3 -d</code>	277
119	Select the proper representation to view the mesh	277
120	Viewing a polyhedral mesh with ParaView's standard settings.	278
121	Viewing a polyhedral mesh with adjusted settings.	278
122	Viewing a polyhedral mesh with ParaView's native OpenFOAM reader.	279
123	Selecting the file name for the animation.	280
124	The Courant number plotted with <i>pyFoamPlotWatcher</i>	283
125	The Courant number based on the relative velocity plotted with <i>pyFoamPlotWatcher</i>	284
126	The average volume fraction plotted with <i>pyFoamPlotWatcher</i> and a custom regular expression	286
127	The execution time plotted over time with <i>pyFoamPlotWatcher</i>	287
128	Screenshot of <i>pyFoamDisplayBlockMesh</i>	289
129	Double grading problem	292
130	Class hierarchy of some injection models for Lagrangian particles. An intermediate base class is used to reduce code duplication from closely related, yet different injection models.	303
131	The three arguments of Eq. (164) plotted over x	315
132	A partial view of the class hierarchy involving <code>regIOobject</code> ;	321
133	The base classes of the class <code>objectRegistry</code> ;	322
134	Graphic representation of inheritance of the turbulence model classes.	331
135	Inheritance of RAS turbulence models	332
136	First layer of the class hierarchy of the LES models of OpenFOAM	359
137	Class hierarchy of the eddy viscosity models in OpenFOAM	360
138	A screenshot of <i>Meld</i>	386

List of Tables

1	Run-time <i>cavity</i> test case	46
2	Comparison of hard disk space consumption	47
3	Valid and invalid face definitions	59
4	Overview of diameter modelling in Eulerian multiphase solvers	184
5	Levels of coupling between Lagrangian particles and (Eulerian) flow	200
6	Turbulence model combinations for phase-inversion cases.	235
7	Comparison of the eddy viscosity models of OpenFOAM	360
8	Comparison of disk space reduction	387
9	Comparison of disk space reduction	387
10	Comparing the resulting file size of the mesh archive file for various conditions/treatments. All file or folder sizes were determined with the Linux command <code>du -sh FILE</code> . The mesh was compressed using the LZMA algorithm at maximum compression: <code>tar -cv constant/polyMesh lzma -9 > polyMesh.tar.xz</code>	388

1 Getting help

Apart from this manual, there are lots of resources on the internet to find help on OpenFOAM.

- The OpenFOAM User Guide
<http://www.openfoam.org/docs/user/>
- The CFD Online Forum
<http://www.cfd-online.com/Forums/openfoam/>
- The OpenFOAM Wiki
http://openfoamwiki.net/index.php/Main_Page
The OpenFOAM Wiki is maintained by a community of developers behind the OpenFOAM-extend project. This wiki covers not only the OpenFOAM but also tools that developed for OpenFOAM, e.g. *pyFoam* or *swak4foam*.
- The OpenFOAM tutorial collection
<https://wiki.openfoam.com>
This is a community driven effort to collect all the various sources of information, which are all over the internets. This tutorial collection is more of a register of tutorials from various sources. There are two curated lists of tutorials, the first glimpse series and the more in-depth three-weeks series which span a range of topics from the general introduction to quite specific topics. Furthermore, there are also lists of tutorials sorted by topic or by contributor.
- The CoCoons Project
<http://www.cocoons-project.org/>
This is a community driven effort to create a documentation on solvers, utilities and modelling.
- The materials of the course CFD with open source software of Chalmers University
http://www.tfd.chalmers.se/~hani/kurser/OS_CFD/
- The CAELinux Wiki
<http://caelinux.org/wiki/index.php/Doc:OpenFOAM>
CAELinux is a collection of open source CAE software including several CFD codes (OpenFOAM, Code_Saturne, Gerris, Elmer).
- Q&A on the internets
You can find questions – and hopefully answers – on the various Q&A sites on the internets, such as StackExchange (<http://stackexchange.com/>), which is a collection of Q&A site specific to a topic or region of interest.
Currently, OpenFOAM questions tend to get posted on the Computational Science Q&A site.
<http://scicomp.stackexchange.com/>
- Word of mouth
https://github.com/ParticulateFlow/OSCCAR-doc/blob/master/openFoamUserManual_PFM.pdf
This is where this manual is hosted.

2 Lessons learned

- For production-use we strongly recommend to use the point-releases of OpenFOAM. As the development versions of OpenFOAM continuously get updated, OpenFOAM's behaviour might change. Thus, users are advised to base their work entirely on point-releases of OpenFOAM. That way, once your simulation cases run, they will run indefinitely, or as long as you are able to install the respective version of OpenFOAM on a computer.
- Keep an eye on developments in OpenFOAM. A more recent version might provide some functionality or feature you desperately need. Even if you added this feature yourself to e.g. your custom solver or model, the developers of OpenFOAM might provide a cleaner or more powerful implementation of that feature. As it is easily possible to install several versions of OpenFOAM side by side on a computer, play around with the latest version.
- Build the source-code documentation of your local installation. It is located e.g. in `$HOME/OpenFOAM/OpenFOAM-2.3.x/doc/Doxygen` if you installed OpenFOAM in your home directory. This makes you independent of being online and the doxygen gives you e.g. a very well-structured overview of a classes methods and members.
- Study the code. Even as *"the documentation is in the code"* does not sound helpful at all, the code in fact tells you what is going on provided you are able to make sense of the C++ syntax. Become familiar with basic concepts of *object-oriented* (OO) software design.
- The more I used and tinkered with OpenFOAM, the more I am convinced that its design is really ingenious. However, it takes time and effort to come to this conclusion. It is also probably a matter of taste.
- Document your own work and stuff you tried. There is no need to create hundreds of pages, but paper or dead electrons have a longer memory as mere mortal humans. Furthermore, the fact *"I have already tried X at some point in the past, and I wrote it down at Y"* is more likely to be remembered than *"I tried X, and that's how it went in all detail"*.

2.1 Philosophy

OpenFOAM is largely following the general rules of the UNIX philosophy – see e.g. Eric S. Raymond [20] or <http://www.catb.org/esr/writings/taoup/html/ch01s06.html> – by accident, by design or by law.

1. Rule of Modularity: *Write simple parts connected by clean interfaces.*
We see this rule in action, when we take a look at all the small pre- and post-processing
2. Rule of Clarity: Clarity is better than cleverness.
3. Rule of Composition: *Design programs to be connected to other programs.*
OpenFOAM's extensive use of text files can be interpreted as a consequence of the Rule of Composition. The structured, textual formal makes it easy to define and interpret OpenFOAM's in- and output.
4. Rule of Separation: Separate policy from mechanism; separate interfaces from engines.
5. Rule of Simplicity: Design for simplicity; add complexity only where you must.
6. Rule of Parsimony: *Write a big program only when it is clear by demonstration that nothing else will do.*
Again, OpenFOAM is a large collection of specialized tools, rather than a big monolithic – one size fits nobody – monster.
7. Rule of Transparency: *Design for visibility to make inspection and debugging easier.*
Here, we quote Eric S. Raymond¹: "A software system is transparent when you can look at it and immediately understand what it is doing and how." CFD is admittedly very complex, however, the close-to-mathematical notation of OpenFOAM's high-level code, can be seen as an example of OpenFOAM's obedience to the Rule of Transparency.
8. Rule of Robustness: Robustness is the child of transparency and simplicity.

¹<http://www.catb.org/esr/writings/taoup/html/ch01s06.html>

9. Rule of Representation: *Fold knowledge into data so program logic can be stupid and robust.*
Although this rule was stated without object-orientation in mind, we can observe, that OpenFOAM's data structures and classes absorb much of the complexity. Thus, the top level solver source code looks quite unspectacular.
10. Rule of Least Surprise: *In interface design, always do the least surprising thing.*
We see this rule in action, when we look at all the shared command line options. All tools that support time selection offer common options, such as `latestTime` or `noZero`.
11. Rule of Silence: *When a program has nothing surprising to say, it should say nothing.*
This rule is obeyed by most function objects, which provide the user with the choice of deactivating writing to the Terminal. This output may be useful during testing. As soon as the case is properly set up, however, it is sufficient for the function object to write its output to the corresponding file in the folder `postProcessing`.
12. Rule of Repair: *When you must fail, fail noisily and as soon as possible.*
Ever noticed the `FOAM FATAL ERROR` messages?
13. Rule of Economy: *Programmer time is expensive; conserve it in preference to machine time.*
If we allow ourselves a very broad view of this rule, we might postulate, that OpenFOAM's mechanism to specify default values for keywords² is one example for following this rule from a user's perspective, i.e. it is the user's time which is conserved.
14. Rule of Generation: *Avoid hand-hacking; write programs to write programs when you can.*
We can see the heavy use of templates as an example of OpenFOAM following the Rule of Generation. The `TurbulenceModels` framework³ is an example of a modelling framework, which is coded once and applied in several different incarnations.
However, this applies only in a wider sense, since this rule was stated not with C++'s templates in mind.
15. Rule of Optimization: Prototype before polishing. Get it working before you optimize it.
16. Rule of Diversity: *Distrust all claims for "one true way".*
OpenFOAM offers the user plenty of choice such as the solvers to use, the solution algorithms, and discretisation and interpolation schemes.
17. Rule of Extensibility: *Design for the future, because it will be here sooner than you think.*
OpenFOAM sometimes exhibits a different behaviour based on its version, or the format of the input files. See Section 36.4.1 for an example on differences in the input syntax of `fixedValue` boundary conditions. The important lesson in this case is to allow for evolution of the code without breaking compatibility.

2.2 Learning by using OpenFOAM

- Numerical errors can ruin your day in CFD. Not every simulation crash is the fault of some bug in OpenFOAM. The numerics of CFD is also keen to crash simulations.
- Never deactivate the unit checking of OpenFOAM.
- Many classes provide optional debug information. Debug flags can be controlled via a global `controlDict` as well as the case's `controlDict`.
- Play around! A great part of learning is trial and error. Although many of us regard themselves as scientists or aspire to become scientists, never disregard the value of plain trial and error.

²See Section 57.3.2

³See Section 32.

2.3 Learning by tinkering with OpenFOAM

2.3.1 *I learned something today.*

- Have a look at the `test` directory in the `applications` folder of your installation, e.g. in `$HOME/OpenFOAM/OpenFOAM-2.3.x/applications/test`. There, you find examples of how to use certain data structures, which may be exactly what you need when implementing something.
- Create your own test application, if you are about to implement something new. With a test application, you can keep the problem nearly primitive, thus, allowing yourself more mental freedom to explore and to learn. Later, you might be more likely to implement your solver / library with less bugs and errors.
- OpenFOAM makes heavy use of C++'s language features and other smart moves in OO software design. Thus, make sure you understand the basics of the following concepts / language features before you try to study / modify the code of OpenFOAM. Your life gets easier if you do.

inheritance virtually everything of OpenFOAM is described and implemented using the concept of classes. Classes can be derived from other classes to implement an *is a* relationship, i.e. every cat is an animal but not vice versa.

Note: C++ support multiple inheritance, i.e. a class can be derived from a number of classes, not just one. Other programming languages are (slightly) different in this aspect, e.g. Java allows you to derive only from one class, however, you can implement interfaces.

poly-morphism is a wider concept, however it applies also to inheritance and classes.

templates allow the user to write code for as-of-yet unspecified data types. Container classes are the prime example for the use of templates (or generics as this concept is called in Java).

Examples of the excellent use of the aforementioned concepts is the turbulence modelling framework discussed in Section 32.1.2, or the Lagrangian modelling framework discussed in Section 40.2.

2.3.2 Trouble with the code?

it does not compile

- Due to the heavy use of templates the syntax and the compiler error messages are quite lengthy and often hard to read. However, the compiler error message might contain exactly the information you need to track down the error, e.g. a data-type mismatch. Familiarize yourself with C++'s syntax if you haven't already.
If you are baffled by the very lengthy error messages, take special care to the top and the bottom of the error message, as it is there where you might find the most useful clues.

it does not run

- Spurious crashes (e.g. caused by floating point errors) may be an indication of class members being un-initialized.
- No offence, but it's most probably your fault.

Part I

Installation

3 Install OpenFOAM

3.1 Prerequisites

OpenFOAM is easily installed by following the instructions from this website: <http://www.openfoam.org/download/git.php>.

First of all, you need to make sure all required packages are installed on your system. This is easily done via the package management software. OpenFOAM is a software made primarily for Linux systems. It can also be installed on Mac or Windows platforms. However, the authors uses a Ubuntu-Linux system, therefore this manual will be based on the assumption that a Linux system is used.

```
sudo apt-get install git-core
sudo apt-get install build-essential flex bison cmake zlib1g-dev qt4-dev-tools libqt4-dev
gnuplot libreadline-dev libxt-dev
sudo apt-get install libscotch-dev libopenmpi-dev
```

Listing 1: Installation of required packages

If OpenFOAM is to be used by a single user, then the User Manual suggests to install OpenFOAM in the `$HOME/OpenFOAM` directory.

3.2 Download the sources

First of all the source files need to be downloaded. This is done with the version control software *Git*. Afterwards we change into the new directory and check for updates. All steps to perform the described operations are listed in Listing 2.

```
cd $HOME
mkdir OpenFOAM
cd OpenFOAM
git clone git://github.com/OpenFOAM/OpenFOAM-2.1.x.git
cd OpenFOAM-2.1.x
git pull
```

Listing 2: Installation von *openFOAM*

Prior to compiling the sources some environment variables have to be defined. In order to do that a line (see Listing 3) has to be added to the file `$HOME/.bashrc`.

```
source $HOME/OpenFOAM/OpenFOAM-2.1.x/etc/bashrc
```

Listing 3: Addition to *.bashrc*

When the command `source $HOME/.bashrc` is issued or when a new Terminal is opened this change is effective. Now with the defined environment variables OpenFOAM can be installed on the system. Before compiling a system check can be made by running *foamSystemCheck*.

```
user@host:~/OpenFOAM/OpenFOAM-2.1.x$ foamSystemCheck
Checking basic system... -----
Shell:      /bin/bash
Host:       host
OS:         Linux version 2.6.32-39-generic
User:       user

System check: PASS
=====
Continue OpenFOAM installation.
```

3.3 Compile the sources

If the system check produced to error messages then OpenFOAM can be compiled. This is done by executing `./Allwmake`. This is an installation script that takes care of all required operations. Compiling OpenFOAM can be done by using more than one processor to save time. In order to do this, an environment variable needs to be set before invoking `./Allwmake`. Listing 5 shows how to compile OpenFOAM using 4 processors.

```
export WM_NCOMPPROCS=4
./Allwmake
```

Listing 5: Parallel compilation using 4 processes.

For working with OpenFOAM a user directory needs to be created. The name of this directory consists of the username and the version number of OpenFOAM. With version 2.1.x this folder needs to be named like this: `user-2.1.x`

3.4 Install paraView

paraView is a post processing tool, see <http://www.paraview.org/>. The OpenFOAM Foundation distributes *paraView* from its homepage and recommends to use this version. The source code can be downloaded from <http://www.openfoam.org/> in an archive, e.g. `ThirdParty-2.1.0.tgz`. This archive has to be unpacked into a folder named correspondingly to the OpenFOAM directory, e.g. `ThirdParty-2.1.x` when `OpenFOAM-2.1.x` is used. This naming scheme is mandatory because there is an environment variable that points to the location of *paraView*. As there is no development of *paraView* by the OpenFOAM developers, there is no repository release of third-party tools.

Subsequently *paraView* can be compiled by the use of an installation script. Afterwards some *plug-ins* for *paraView* need to be compiled.

```
cd $WM_THIRD_PARTY_DIR
./makeParaView

cd $FOAM_UTILITIES/postProcessing/graphics/PV3Readers
wmSET
./Allwclean
./Allwmake
```

Listing 6: Installation of *paraView*

3.5 Remove OpenFOAM

If OpenFOAM is to be removed from the system, then a few simple operations do the job⁴, provided the installation was done following the installation guidelines of OpenFOAM⁵.

Listing 7 shows how OpenFOAM can be removed from the system. We assume, we want to remove an installation of OpenFOAM-2.0.1. The first line changes the working directory to the installation directory of OpenFOAM. This folder contains all files of the OpenFOAM installation. Listing 8 shows the content of the `~/OpenFOAM`. In this example, two versions of OpenFOAM are installed.

The second line removes all files of OpenFOAM and the third line removes the files of the user related to OpenFOAM. The last line of Listing 7 removes a hidden folder. If there are several versions of OpenFOAM installed, then this folder should not be removed.

⁴<http://www.cfd-online.com/Forums/openfoam-installation/57512-completely-remove-openfoam-start-fresh.html>

⁵<http://www.openfoam.org/download/git.php>

```
cd ~/OpenFOAM
rm -rf OpenFOAM-2.0.1
rm -rf user-2.0.1
cd
rm -rf ~/.OpenFOAM
```

Listing 7: Removing *OpenFOAM*

```
cd ~/OpenFOAM
ls -l
user-2.0.x
user-2.1.x
OpenFOAM-2.0.x
OpenFOAM-2.1.x
ThirdParty-2.0.x
ThirdParty-2.1.x
```

Listing 8: Content of *~/OpenFOAM*

Another thing to remove is the entry in the `.bashrc` file in the home directory. Delete the line shown in Listing 3.

3.6 Install several versions of OpenFOAM

It is possible to install several versions of OpenFOAM on the same machine. However due to the fact that OpenFOAM relies on some environment variables some precaution is needed. See <http://www.cfd-online.com/Forums/blogs/wyldckat/931-advanced-tips-working-openfoam-shell-environment.html> for detailed information about OpenFOAM and the Linux shell.

The most important fact about installing several versions of OpenFOAM is to keep the separated.

4 Updating the repository release of OpenFOAM

4.1 Version management

OpenFOAM is distributed in two different ways. There is the *repository release* that can be downloaded using the *Git repository*. The version number of the repository release is marked by the appended x, e.g. OpenFOAM 2.1.x. This release is updated regularly and is in some ways a development release. Changes and updates are released quickly, however, there is a larger possibility of bugs in this release. Because this release is updated frequently an OpenFOAM installation of version 2.1.x on one system may or will be different to another installation of version 2.1.x on an other system. Therefore, each installation has an additional information to mark different builds of OpenFOAM. The version number is accompanied by a hash code to uniquely identify the various builds of the repository release, see Listing 9. Whenever OpenFOAM is updated and compiled anew, this hash code gets changed. Two OpenFOAM installations are on an equal level, if the build is equal.

```
Build : 2.1.x-9d344f6ac6af
```

Listing 9: Complete version identification of *repository releases*

Apart from the repository release there are also *pack releases*. These are updated periodically in longer intervals than the repository release. The version number of a pack release contains no x, e.g. OpenFOAM 2.1.1. In contrast to the repository release all installations of the same version number are equal. Due to the longer release cycle the pack release is regarded to be less prone to software bugs.

There are several types of those releases. There are precompiled packages for widely used Linux distributions (Ubuntu, SuSE and Fedora) and also a source pack. The source pack can be installed on any system on which the source codes compile (usually all kinds of Linux running computers, e.g. high performance computing clusters, or even computers running other operation systems, e.g. Mac OSX⁶ or even Windows⁷).

⁶See http://openfoamwiki.net/index.php/Howto_install_OpenFOAM_v21_Mac

⁷See http://openfoamwiki.net/index.php/Tip_Cross_Compiling_OpenFOAM_in_Linux_For_Windows_with_MinGW

4.2 Check for updates

If OpenFOAM was installed from the repository release, updating is rather simple. To update OpenFOAM simply use *Git* to check if there are newer source files available. Change in the Terminal to the root directory of the OpenFOAM installation and execute `git pull`.

If there are newer files in the repository *Git* will download them and display a summary of the changed files.

```
user@host:~$ cd $FOAM_INST_DIR
user@host:~/OpenFOAM$ cd OpenFOAM-2.1.x
user@host:~/OpenFOAM/OpenFOAM-2.1.x$ git pull
remote: Counting objects: 67, done.
remote: Compressing objects: 100% (13/13), done.
remote: Total 44 (delta 32), reused 43 (delta 31)
Unpacking objects: 100% (44/44), done.
From git://github.com/OpenFOAM/OpenFOAM-2.1.x
 72f00f7..21ed37f master -> origin/master
Updating 72f00f7..21ed37f
Fast-forward
.../extrude/extrudeToRegionMesh/createShellMesh.C | 10 +-
.../extrude/extrudeToRegionMesh/createShellMesh.H | 7 +-
.../extrudeToRegionMesh/extrudeToRegionMesh.C | 157 ++++++-----
.../Templates/KinematicCloud/KinematicCloud.H | 6 +-
.../Templates/KinematicCloud/KinematicCloudI.H | 7 +
.../baseClasses/kinematicCloud/kinematicCloud.H | 47 +++++-
6 files changed, 193 insertions(+), 41 deletions(-)
```

Listing 10: There are updates available

If OpenFOAM is up to date, then *Git* will output a corresponding message.

```
user@host:~/OpenFOAM/OpenFOAM-2.1.x$ git pull
Already up-to-date.
```

Listing 11: OpenFOAM is up to date

4.3 Check for updates only

If you want to check for updates only, without actually making an update, *Git* can be invoked using a special option (see Listings 12 and 13). In this case *Git* only checks the repository and displays its findings without actually making any changes. The option responsible for this is `--dry-run`. Notice, that `git fetch` is called instead of `git pull`⁸.

```
user@host:~$ cd OpenFOAM/OpenFOAM-2.0.x/
user@host:~/OpenFOAM/OpenFOAM-2.0.x$ git fetch --dry-run -v
remote: Counting objects: 189, done.
remote: Compressing objects: 100% (57/57), done.
remote: Total 120 (delta 89), reused 93 (delta 62)
Receiving objects: 100% (120/120), 17.05 KiB, done.
Resolving deltas: 100% (89/89), completed with 56 local objects.
From git://github.com/OpenFOAM/OpenFOAM-2.0.x
 5ae2802..97cf67d master -> origin/master
user@host:~/OpenFOAM/OpenFOAM-2.0.x$
```

Listing 12: Check for updates only – updates available

```
user@host:~$ cd OpenFOAM/OpenFOAM-2.1.x/
user@host:~/OpenFOAM/OpenFOAM-2.1.x$ git fetch --dry-run -v
From git://github.com/OpenFOAM/OpenFOAM-2.1.x
 = [up to date] master -> origin/master
user@host:~/OpenFOAM/OpenFOAM-2.1.x$
```

Listing 13: Check for updates only – up to date

⁸`git pull` calls `git fetch` to download the remote files and then calls `git merge` to merge the retrieved files with the local files. So checking for updates is actually done by `git fetch`.

4.4 Install updates

After updates have been downloaded by `git pull` the changed source files need to be compiled in order to update the executables. This is done the same way as is it done when installing OpenFOAM. Simply call `./Allwmake` to compile. This script recognises changes, so unchanged files will not be compiled again. So, compiling after an update takes less time than compiling when installing OpenFOAM.

4.4.1 Workflow

Listing 14 shows the necessary commands to update an existing OpenFOAM installation. However this applies only for repository releases (e.g. OpenFOAM-2.1.x). The point releases (every version of OpenFOAM without an x in the version number) are not updated in the same sense as the repository releases. For simplicity an update of a point release (OpenFOAM-2.1.0 → OpenFOAM-2.1.1) can be treated like a complete new installation, see Section 3.6.

The first two commands in Listing 14 change to the directory of the OpenFOAM installation. Then the latest source files are downloaded by invoking `git pull`.

The statement in red can be omitted. However if the compilation ends with some errors, this command usually does the trick, see Section 4.5.2. The last statement causes the source files to be compiled. If `wclean all` was not called before, then only the files that did change are compiled. If `wclean all` was invoked then everything is compiled. This may or will take much longer.

If there is enough time for the update (e.g. overnight), then `wclean all` should be called before compiling. This will in most cases make sure that compilation of the updated sources succeeds.

```
cd $FOAM_INST_DIR
cd OpenFOAM-2.1.x
git pull
wclean all
./Allwmake
```

Listing 14: Update an existing OpenFOAM installation. The complete workflow

4.4.2 Trouble-shooting

If compilation reports some errors it is helpful to call `./Allwmake` again. This reduces the output of the successful operations considerably and the actual error messages of the compiler are easier to find.

4.5 Problems with updates

4.5.1 Missing packages

If there has been an upgrade of the operating system⁹ it can happen, that some relevant packages have been removed in the course of the update (e.g. if these packages are only needed to compile OpenFOAM and the OS 'thinks' that these packages aren't in use). Consequently, if recompiling OpenFOAM fails after an OS upgrade, missing packages can be the cause.

4.5.2 Updated Libraries

When libraries have been updated, they have to be recompiled. Otherwise solvers would call functions that are not (yet) implemented. In order to avoid this problem the corresponding library has to be recompiled.

```
wclean all
```

Listing 15: Prepare recompilation with *wclean*

The brute force variant would be, to recompile OpenFOAM as a whole, instead of recompiling a updated library.

⁹An *upgrade* of an OS is indicated by a higher version number of the same (Ubuntu 11.04 → Ubuntu 11.10). An *update* leaves the version number unchanged.

4.5.3 Updated sources fail to compile

In some cases, e.g. when there were changes in the organisation of the source files, the sources fail to compile right away. Or, if there is any other reason the sources won't compile and the cause is not found, then a complete recompilation of OpenFOAM may be the solution of choice. Although compiling OpenFOAM takes its time, this may take less time than tracking down all errors.

To recompile OpenFOAM the sources need to be reset. Instead of deleting OpenFOAM and installing it again, there is a simple command that takes care of this.

```
git clean -dfx
```

Listing 16: Reset the sources using *git*

The command listed in Listing 16 causes *git* to erase all files *git* does not track. That means all files that are not part of the *git*-repository are deleted. In this case, this is the official *git*-repository of OpenFOAM. *git clean* removes all files that are not under version control recursively starting from the current directory. The option `-d` means that also untracked folders are removed.

After the command from Listing 16 is executed, the sources have to be compiled as described in Section 3.3.

4.5.4 Own code fails to run

Updating your repository release of OpenFOAM leads to interesting effects. When libraries of OpenFOAM are updated, their implementation might change. Even if the updated code is fully compatible with the previous one, the compiled libraries might look different after the update. Thus, even if the update maintains code-compatibility¹⁰, the update might break binary compatibility. Thus, a recompilation of your own code following the update of the underlying OpenFOAM installation is required.

Lost binary compatibility after an update of OpenFOAM leads to segmentation faults when loading a library with lost binary compatibility. This happens because our own solvers dynamically load the required libraries of OpenFOAM at start-up and the memory layout of certain objects of the library has changed since the update.

See the following resources for further information on this topic:

- https://community.kde.org/Policies/Binary_Compatibility_Issues_With_C%2B%2B
- https://en.wikipedia.org/wiki/Binary_code_compatibility
- https://en.wikipedia.org/wiki/Source_code_compatibility

Losing binary compatibility happens not after every update, and it also does not happen to every library. Thus, you may encounter such problems long after the update, and after you successfully used other solvers and libraries of your creation. Thus, the source of the issues described in this Section may not be immediately clear to the user. Thus, if your code suddenly fails to run properly for no good reason, recompile and see what happens.

5 Updating OpenFOAM-6 source installation

5.1 Motivation

OpenFOAM-6 is kind-of rolling point-release, i.e. the version number remains unchanged, yet there are occasional patch-releases. According to the OpenFOAM Foundation¹¹:

Version 6 is a snapshot of the OpenFOAM development version which, through sustainable development, is always-releasable. It provides new functionality and major improvements to existing code, with strict demands on usability, robustness and extensibility.

The initial installation of OpenFOAM-6 follows the steps of any other installation from-source. However, when a subsequent patch-release is released, we're left with an OpenFOAM-6 installation that is not equal in capabilities, features and bugs, than the current OpenFOAM-6 sources.

¹⁰This is the general behaviour of an update. In an ideal world only newer versions are allowed to introduce incompatibility.

¹¹<https://openfoam.org/release/6/>

```

gerhard@gerhardWork:~/software/OpenFOAM-6$ git init
Initialisiertes leeres Git-Repository in /home/gerhard/software/OpenFOAM-6/.git/
gerhard@gerhardWork:~/software/OpenFOAM-6$ ls -al
Insgesamt 92
drwxrwxr-x 11 gerhard gerhard 4096 Apr 19 14:45 .
drwxrwxr-x 18 gerhard gerhard 4096 Apr 19 14:45 ..
-rwxrwxr-x 1 gerhard gerhard 1109 Jan 8 17:54 Allwmake
drwxrwxr-x 5 gerhard gerhard 4096 Jan 8 17:54 applications
drwxrwxr-x 3 gerhard gerhard 4096 Jan 8 17:54 bin
-rw-rw-r-- 1 gerhard gerhard 35646 Jan 8 17:54 COPYING
drwxrwxr-x 5 gerhard gerhard 4096 Jan 8 17:54 doc
drwxrwxr-x 8 gerhard gerhard 4096 Jan 8 17:54 etc
drwxrwxr-x 7 gerhard gerhard 4096 Apr 19 14:45 .git
-rw-rw-r-- 1 gerhard gerhard 1349 Jan 8 17:54 .gitignore
-rw-rw-r-- 1 gerhard gerhard 1626 Jan 8 17:54 README.org
drwxrwxr-x 41 gerhard gerhard 4096 Jan 8 17:54 src
drwxrwxr-x 3 gerhard gerhard 4096 Jan 8 17:54 test
drwxrwxr-x 17 gerhard gerhard 4096 Jan 8 17:54 tutorials
drwxrwxr-x 6 gerhard gerhard 4096 Jan 8 17:54 wmake
gerhard@gerhardWork:~/software/OpenFOAM-6$

```

Figure 1: Initializing a new git repository.

```

gerhard@gerhardWork:~/software/OpenFOAM-6$ git remote add origin git@github.com:OpenFOAM/OpenFOAM-6.git
gerhard@gerhardWork:~/software/OpenFOAM-6$

```

Figure 2: Pointing the new git repository to the proper remote repository.

In ye olden days, we would have been gifted with an OpenFOAM-X.Y++. However, as OpenFOAM-6 is OpenFOAM-6, which sort of isn't OpenFOAM-6¹², we need a way to properly update our source-pack installation of OpenFOAM-6.

5.2 Make the OpenFOAM-6 source pack able to being updated

Since the OpenFOAM-6 source pack is a snapshot of the OpenFOAM-6 git repository¹³, updating the source pack is pretty easy. The basic steps are:

1. Initialize a new git repository within the source pack, using `git init`
2. Point the remote-URL of the newly created git repository to the remote repository of OpenFOAM-6, using `git remote`
3. Download the updates, using `git pull`
4. And finally update the local state of your source pack, using `git checkout`

Alternatively, we could simply remove the current installation of OpenFOAM-6, download the latest source pack and re-install afresh. Yet, this would be the less elegant brute-force method of updating our OpenFOAM-6 installation. Connoisseurs, read on.

5.2.1 Initialization

With `git init`, we tell git to create a new git repository. If we run this command in a directory with files being already present, this does not change anyone of them.

After running `git init`, we will find that a new folder has been created: `.git`. The file `.gitignore` has been part of the source pack.

5.2.2 Pointing to the remote repository

Next, we need to tell git where to find the updates. We do this, by calling `git remote add origin git@github.com:OpenFOAM/OpenFOAM-6.git`

¹²The reader may apologize the rambling babble.

¹³<https://github.com/OpenFOAM/OpenFOAM-6>

```

gerhard@gerhardWork:~/software/OpenFOAM-6$ git pull
remote: Enumerating objects: 104, done.
remote: Counting objects: 100% (104/104), done.
remote: Compressing objects: 100% (84/84), done.
remote: Total 144740 (delta 35), reused 50 (delta 19), pack-reused 144636
Empfange Objekte: 100% (144740/144740), 80.58 MiB | 16.01 MiB/s, Fertig.
Löse Unterschiede auf: 100% (103494/103494), Fertig.
Von github.com:OpenFOAM/OpenFOAM-6
* [neuer Branch] master -> origin/master
* [neues Tag] 20180710 -> 20180710
* [neues Tag] 20180805 -> 20180805
* [neues Tag] 20181130 -> 20181130
* [neues Tag] 20181221 -> 20181221
* [neues Tag] 20190108 -> 20190108
* [neues Tag] 20190304 -> 20190304
* [neues Tag] version-3.0.0 -> version-3.0.0
* [neues Tag] version-4.0 -> version-4.0
* [neues Tag] version-5.0 -> version-5.0
* [neues Tag] version-6 -> version-6
Es gibt keine Tracking-Informationen für den aktuellen Branch.
Bitte geben Sie den Branch an, welchen Sie zusammenführen möchten.
Siehe git-pull(1) für weitere Details.

git pull <remote> <branch>

Wenn Sie Tracking-Informationen für diesen Branch setzen möchten, können Sie
dies tun mit:

git branch --set-upstream-to=origin/<Branch> master
gerhard@gerhardWork:~/software/OpenFOAM-6$

```

Figure 3: Updating the new git repository by pulling from the remote repository.

```

gerhard@gerhardWork:~/software/OpenFOAM-6$ git checkout -t -f -b master origin/master
Branch master konfiguriert zum Folgen von Remote-Branch master von origin.
Bereits auf 'master'
gerhard@gerhardWork:~/software/OpenFOAM-6$ git pull
Already up-to-date.
gerhard@gerhardWork:~/software/OpenFOAM-6$

```

Figure 4: Updating the local files of our OpenFOAM-6 installation.

5.2.3 Pulling the updates

After, our repository has been made aware, where to get updates, it is time to download them. This is done with `git pull`.

5.2.4 Updating the local repository

After, we have downloaded the updates to our repository, we need to update the local, working files of our repository. Right now, the updates are present locally, yet the working files – the files we can open and read with our text editor have not been updated yet. The local, working files are updated by applying the changes we just downloaded by running `git checkout -t -f -b master origin/master`.

The checkout command now applies all the changes that have been made to the remote master branch to our local master branch. Now, all the local files have been updated, and we can recompile the source files.

Note, that we need to update the third-party source-code directory in the same way as we updated the OpenFOAM source-code directory.

5.3 Recompile the source pack

The last step to update any installation from source, is to recompile the sources. This can be conveniently done by running

```
./Allwmake -update
```

Listing 17: Recompile an updated source pack

6 Maintaining your OpenFOAM installation

Due to the ease of having multiple versions of OpenFOAM installed side-by-side on your system, there is the issue of long-term maintaining of your OpenFOAM installations. Over the years OpenFOAM installations may accumulate on your system. Furthermore, the operating system as well receives attention from its developers in the form of updates and upgrades.

6.1 Dealing with OS updates

Over time, the operating system you are running will receive lots of updates, unless you only work on machines that are not connected to the internet. Usually, updates to the operating system do not affect your OpenFOAM installation. If your OpenFOAM installation ceases to work properly following an OS update, a version conflict is the most likely culprit. In such a case, a total rebuild from source, as outlined in Listing 18 should solve the problem.

Trouble-shooting tip: reboot your machine and try again, this may actually help

OS updates can cause weird errors. Your trusted author encountered a situation in which `mpirun` ceased to work, and the error message, contained a stack trace which featured words such as memory mapping and such. This problem was most likely caused by OS updates which updated the MPI library and/or kernel updates. However, the problems disappeared when the machine was rebooted.

Having a long uptime, i.e. is has been a very long time since the computer last rebooted, is only recommended for computing server, which do not receive updates frequently, and are best not connected to the general internet. A normal workstation, which is frequently updated, should be rebooted regularly. The author has experienced multiple occasions, when the workstation after several kernel updates (these are ones that require a reboot to take effect) with no reboot started to behave weirdly.

6.2 Dealing with OS upgrades

Upgrading the operating system nearly always required a total rebuild from source of your OpenFOAM installation. An upgrade to your OS is very likely to update many or even all libraries and packages OpenFOAM depends on. Thus, trying to run OpenFOAM applications after an upgrade to the OS might result in odd errors, e.g. OpenFOAM failing to launch due to its inability to load shared libraries such as `libmpi.so.XX`, with `XX` being a version number.

In the case of the updated OS, `libmpi.so.YY` was present at the system, with `YY` being a different version number than `XX`. This slight oddity of the error: OpenFOAM reporting a missing `libmpi` library, with a `libmpi` library being present at the system; indicates a version conflict due to the OS upgrade. OpenFOAM tries to load the version of the library when it was installed, yet due to the intervening OS update a newer version of the library replaced the older one.

Hence, after an OS upgrade total rebuild from source is nearly in all cases warranted. Especially when the underlying OS is some sort of long-term release, such as the LTS versions of Ubuntu, which are released every two years. With a long OS release cycle, we are almost guaranteed to run into version problems after the OS upgrade.

Thus, after upgrading the OS, we run a total rebuild from source as shown in Listing 18. First, we enter the installation directory, clear the installation with `wclean all` and finally run `Allwmake`. Note, that this needs to be done with all your OpenFOAM installations, that were installed prior to the upgrade.

```
cd $WM_PROJECT_DIR
wclean all
./Allwmake
```

Listing 18: Performing a total rebuild from source

Running old versions of OpenFOAM is generally not recommended, as they might or will contain bugs which have long been fixed since. However, for checking on old simulations or re-running them to compare them with current runs, it is quite handy to have a functional installation at hand.

7 Install third-party software

The software presented in this section is optional. Without this software OpenFOAM is complete and perfectly useable. However, the software mentioned in this section can be very useful for specific tasks.

7.1 Install *pyFoam*

See http://openfoamwiki.net/index.php/Contrib_PyFoam#Installation for the instructions on the installation of *pyFoam*.

7.2 Install *swak4foam*

See <http://openfoamwiki.net/index.php/Contrib/swak4Foam> for instructions on installing *swak4foam*.

7.3 Compile external libraries

There is the possibility to extend the functionality of OpenFOAM with additional external libraries, i.e. libraries for OpenFOAM from other sources than the developers of OpenFOAM. One example of such an external library is a large eddy turbulence model from <https://github.com/AlbertoPa/dynamicSmagorinsky>. The source code is stored in `OpenFOAM/AlbertoPa/`.

Such a library is compiled with `wmake libso`. This is also the case when libraries of OpenFOAM have been modified. The reason why typing `wmake libso` is sufficient is because all information *wmake* requires is stored in the files `Make/files` and `Make/options`. These files tell *wmake* – and therefore also the compiler – where to find necessary libraries and where to put the executable. A more detailed description of these two files can be found in Section 58.2.2.

To use an external library the solver needs to be told so. See Section 11.3.3.

```
cd OpenFOAM/AlbertoPa/dynamicSmagorinsky
wmake libso
```

Listing 19: Compilation of a library

8 Setting up the environment

8.1 Sourcing OpenFOAM

OpenFOAM makes use of plenty of environment variables, see Section 11.1.1 for a brief discussion. In order to use OpenFOAM, we need to assign values to the variables. Another task enabling the convenient use of OpenFOAM is to add the directories in which OpenFOAM executables are located to the system's `$PATH` variable.

The name of this section stems from the Linux command `source`, which is used in setting up the proper environment for using OpenFOAM. Setting up the environment for using OpenFOAM can be done in two ways, which are discussed below. Each of these variants involves editing a `.bashrc` file¹⁴. This `.bashrc` file can be either a systemwide one for systemwide installations, or belonging to the user who installed OpenFOAM in his/her home directory.

Once the OpenFOAM environment has been sourced in a Terminal, OpenFOAM is ready to use as long as the Terminal is open.

8.1.1 Permanently sourcing OpenFOAM

If we only use one OpenFOAM installation, we could permanently source OpenFOAM. In this case, once this is set up, OpenFOAM is ready for use without any further user action. To achieve this, we add the following line to the appropriate `.bashrc` file. In the case of a single user's installation, this would be the file `$HOME/.bashrc`. The `$HOME/.bashrc` file is loaded every time a Terminal is opened. Thus, if we add the command of Listing 20 to the `$HOME/.bashrc` file, then OpenFOAM is ready to use, whenever the user opens a Terminal. This also

¹⁴If you for some reason unthinkable to the author do not want to edit any `.bashrc` file, you can simply enter the instruction shown in Listing 20 into the Terminal whenever you want to use OpenFOAM.

applies to login shells, thus remote connections via SSH or systems without any graphical desktop are covered as well.

```
source $HOME/OpenFOAM/OpenFOAM-4.0/etc/bashrc
```

Listing 20: Permanently sourcing OpenFOAM

8.1.2 Sourcing OpenFOAM on demand

Permanently sourcing OpenFOAM is impossible if we want to use several OpenFOAM versions alongside each other. If we have OpenFOAM-3.0 and OpenFOAM-4.1 installed on our system, where should/does `$FOAM_SRC` point to?

In this case, we need a solution to set up the OpenFOAM environment on demand for a specific version of OpenFOAM. Again, we need to add instructions to the `.bashrc` file. However, now we add definitions for *aliases*. An *alias* is a placeholder for a set of instructions, which are to be executed only on demand. Since we add the *alias* definitions to the `.bashrc` file, the *aliases* we defined are available in every Terminal. However, in contrast to sourcing OpenFOAM permanently, the OpenFOAM environment is set up only when we invoke the *alias*. An *alias* is a convenient way to save on typing effort, since we can assign one or several commands of arbitrary length¹⁵ to a rather short *alias*. We are free to choose the *alias*' name, as long as this name does not collide with an existing command¹⁶.

In Listing 21 two *aliases* are shown for enabling OpenFOAM-3.0 and OpenFOAM-4.1. If we want to use OpenFOAM-3.0, we simply type `of30` into the Terminal, this will source the environment for OpenFOAM-3.0. The use of these four letter *aliases*, which include the major and minor version number of OpenFOAM, saved us from typing a 46 character command to enable the OpenFOAM environment.

```
alias of30='source $HOME/OpenFOAM/OpenFOAM-3.0/etc/bashrc'
alias of41='source $HOME/OpenFOAM/OpenFOAM-4.1/etc/bashrc'
```

Listing 21: Sourcing OpenFOAM on demand by using an *alias*

8.2 Useful helpers

¹⁵There is a limit on how long a single command can be. On the author's Linux system, this is north of 2 million bytes.

¹⁶We can, in fact, define an *alias* which has the same name as an existing command. In this case, the *alias* “shadows” the corresponding command. This is used in Ubuntu Linux to add some eye candy, e.g. for `ls`, which is shadowed by `alias ls='ls -color=auto'`. In this case, the Terminal expands the *alias*, whenever a user types `ls`.

Part II

General Remarks about OpenFOAM

9 Units and dimensions

This section discusses the treatment of physical units (e.g. meter, second, etc.) and dimensions (scalar, vector, etc.) in OpenFOAM. In OpenFOAM physical units are referred to as *dimensions* and they are covered by the class `dimensionSet`. The dimensionality (a quantity being a scalar or a vector) is treated implicitly by the data types. The data types `scalar` or `vector` do not need any further specification of their dimensionality.

9.1 Unit inspection

Basically, OpenFOAM uses the International System of Units, short: SI units. Nevertheless, also other units can be used. In that case it is important to remember, that some physical constant, e.g. the universal gas constant, are stored in SI units. Consequently the values need to be adapted if other units than SI should be used.

OpenFOAM performs in addition to its calculations also a inspection of the physical units of all involved variables and constants. For fields, like the velocity, or constants, like viscosity, the unit has to be specified. The unit is defined in the *dimension set*. Units in the International System of Units are defined as products of powers of the SI base units.

$$[Q] = \text{kg}^\alpha \text{m}^\beta \text{s}^\gamma \text{K}^\delta \text{mol}^\epsilon \text{A}^\zeta \text{cd}^\eta \quad (1)$$

A dimension set contains the exponents of (1) that define the desired unit. With the dimension set OpenFOAM is able to perform unit checks.

```
dimensions      [0 1 -2 0 0 0 0];
```

Listing 22: False *dimensions* for U

```
--> FOAM FATAL ERROR:
incompatible dimensions for operation
  [U[0 1 -3 0 0 0 0] ] + [U[0 1 -4 0 0 0 0] ]

From function checkMethod(const fvMatrix<Type>&, const fvMatrix<Type>&)
in file /home/user/OpenFOAM/OpenFOAM-2.1.x/src/finiteVolume/lnInclude/fvMatrix.C at line
1316.

FOAM aborting
```

Listing 23: Incompatible *dimensions* for summation

Listing 22 shows an incorrect definition of the dimension of the velocity, e.g. in the file 0/U. m/s^2 has been defined instead of m/s . OpenFOAM recognises this false definition, because mathematical operations do not work out anymore. Listing 23 shows a corresponding error message produced by two summands having different units. Therefore, OpenFOAM aborts and displays an error message.

9.1.1 An important note on the base units

The order in which the base units are specified differs between OpenFOAM and many publications dealing with SI units, compare (2) and (3). The order of the base units as it is used by OpenFOAM swaps the first two base units. As the list of base units in [4, 3] starts with the metre followed by the kilogram, OpenFOAM reverses this order and begins with the kilogram followed by the metre. Also the fourth, fifth and sixth base units appear in a different position.

$$[Q]_{\text{OpenFOAM}} = \text{kg}^\alpha \text{m}^\beta \text{s}^\gamma \text{K}^\delta \text{mol}^\epsilon \text{A}^\zeta \text{cd}^\eta \quad (2)$$

$$[Q]_{\text{SI}} = \text{m}^\alpha \text{kg}^\beta \text{s}^\gamma \text{A}^\delta \text{K}^\epsilon \text{mol}^\zeta \text{cd}^\eta \quad (3)$$

Eq. (2) is based on the source code of OpenFOAM, see Listing 24. Eq. (3) is based on [4, 3].

```

1  //- Define an enumeration for the names of the dimension exponents
2  enum dimensionType
3  {
4      MASS,           // kilogram    kg
5      LENGTH,        // metre      m
6      TIME,           // second    s
7      TEMPERATURE,   // Kelvin    K
8      MOLES,          // mole      mol
9      CURRENT,        // Ampere    A
10     LUMINOUS_INTENSITY // Candela   Cd
11 };

```

Listing 24: The definition of the order of the base units in the file `dimensionSet.H`

The reason for changing the order of the base units may be motivated from a CFD based point of view. For fluid dynamics involving compressible flows as well as reactive flows and combustion the first five units of OpenFOAM's set of base units suffice.

9.1.2 Input syntax of units

Listing 25 shows the definition of a phase in a two-phase problem. Notice the difference between the first two definitions and the third one. The unit of `d` is defined by the full set of seven exponents, whereas the other two units (`rho` and `nu`) are defined only by five exponents. Apparently it is allowed to omit the last two exponents (defining candela and ampere).

Defining units with five entries (for kilogram, metre, second, kelvin and mol) seems to be perfectly appropriate. Neither the OpenFOAM User Guide [50] or the OpenFOAM Programmer's Guide [49] mention this behaviour. Defining a unit with an other number of values than five or seven leads to an error (see Listing 26).

```

phaseb
{
    rho          rho [ 1 -3 0 0 0 ] 1000;
    nu           nu [ 0 2 -1 0 0 ] 1e-06;
    d            d [ 0 1 0 0 0 0 0 ] 0.00048;
}

```

Listing 25: Definition of the unit

```

--> FOAM FATAL IO ERROR:
wrong token type - expected Scalar, found on line 22 the punctuation token ']'

file: /home/user/OpenFOAM/user-2.1.x/run/twoPhaseEulerFoam/bed/constant/transportProperties::
phaseb::nu at line 22.

From function operator>>(Istream&, Scalar&)
in file lnInclude/Scalar.C at line 91.

FOAM exiting

```

Listing 26: Erroneous definition of units

9.1.3 Programming syntax of units

Single numbers or entire fields in OpenFOAM are not only read from file, they are also calculated from existing ones or they created completely new, independent of existing quantities. Let's take a look on how to create dimensioned quantities from the programming point of view. In OpenFOAM there are dimensioned and undimensioned data types, e.g. there are the data types `scalar` and `dimensionedScalar`. The type `dimensionedScalar` is basically a `scalar` with an additional `dimensionSet`.

Calculating dimensioned quantities

Calculated fields inherit their dimension set from the involved operations and operands. Listing 27 shows the creation of the kinetic energy field `K` from the square of the velocity fields¹⁷. The newly created field, bears the name `K`, as this is passed as an argument to the constructor. The dimension set of the field `K` is derived from the constructor's second argument. Since all mathematical operations on numeric types are mirrored for dimensions, any mathematical operation on a dimensioned type not only yields a numerical result, it also yields a resulting dimension. In this case the resulting dimension is square metre per square second.

```
1 Info<< "Creating field kinetic energy K\n" << endl;
2 volScalarField K("K", 0.5*magSqr(U));
```

Listing 27: Computing the kinetic energy fields

Creating dimensioned quantities

When creating a dimensioned quantity from scratch, the dimension set needs to be stated explicitly. In Listing 28 the dimension set is explicitly passed to the constructor of `dimensionedScalar` as the second argument. Note the use of the five argument constructor of `dimensionSet`. As the last two SI units (for current and luminous intensity) are scarcely needed in fluid dynamics, the five argument constructor is a convenience feature of this data type.

```
1 dimensionedScalar foo("foo", dimensionSet(0, 3, 0, 0, 0), scalar(1.0))
```

Listing 28: Create a new variable of the `dimensionedScalar` datatype

Always explicitly stating the dimension set with its 5 or 7 exponents would seriously bloat the code for no benefit. Thus, there are a number of global constants of the data type `dimensionSet`. These constants define the most common dimension sets and offer a very convenient short-hand notation¹⁸ as seen in Listing 29.

```
1 dimensionedScalar bar("bar", dimless, scalar(0.0))
```

Listing 29: Create a new variable of the `dimensionedScalar` datatype

Since all mathematical operations performed on the numeric part of a dimensioned quantity are also performed on the dimension set, the class `dimensionSet` implements mathematical operations. We can use these and the global short-hands to define the dimension set of our new dimensioned quantity. In Listing 30, we needlessly compute the dimension set for a velocity, however, this Listing demonstrates the use of mathematical operations on dimension sets.

```
1 dimensionedScalar baz("baz", dimLength/dimTime, 42.0)
```

Listing 30: Create a new variable of the `dimensionedScalar` datatype

9.2 Dimensions

Fields in fluid mechanics can be scalars, vectors or tensors. There are in OpenFOAM different data types to distinguish between quantities of different dimensions.

volScalarField A scalar field throughout the whole computaional domain, e.g. pressure.
volScalarField p

volVectorField A vector field throughout the whole domain, e.g. velocity.
volVectorField U

¹⁷The kinetic energy is defined in textbooks as $k = \frac{1}{2}\rho u^2$, which involves the fluid density ρ , which the definition in Listing 27 is lacking. However, the fluid density field `rho` enters the scene as second argument in the terms of the energy transport equation, as can be seen in the temporal derivative and convective terms of `rhoPimpleFoam`'s energy equation: `fvc::ddt(rho, K) + fvc::div(phi, K)`. Thus, OpenFOAM's kinetic energy `K` is in fact a specific kinetic energy.

¹⁸You can find these definitions in `$FOAM_SRC/OpenFOAM/dimensionSet/dimensionSets.C`

volTensorField A tensor field throughout the whole domain, e.g. Reynolds stresses.

volTensorField Rca

surfaceScalarField A scalar field, defined on surfaces (surfaces of the finite volumes), e.g. flux.

surfaceScalarField phi

dimensionedScalar A scalar constant throughout the whole domain (i.e. no field quantity).

dimensionedScalar nu

9.2.1 Dimension check

The data type defines also, as described before, the dimension of a quantity. The dimension of a quantity defines the syntax how quantities have to be entered.

Listing 32 shows the error message OpenFOAM displays when the value of a scalar quantity is entered as a vector (Listing 31).

```
dimensions      [ 0 0 0 0 0 0 0 ];
internalField    uniform ( 0 0 0 );
boundaryField
{
    inlet
    {
        type      fixedValue;
        value      uniform 0;
    }
}
```

Listing 31: Erroneous definition of α

```
--> FOAM FATAL IO ERROR:
wrong token type - expected Scalar, found on line 19 the punctuation token '('

file: /home/user/OpenFOAM/user-2.1.x/run/twoPhaseEulerFoam/bed/0/alpha::internalField at line
19.

From function operator>>(Istream&, Scalar&)
in file lnInclude/Scalar.C at line 91.

FOAM exiting
```

Listing 32: Error message caused by invalid dimension

9.3 Kinematic viscosity vs. dynamic viscosity

To determine if OpenFOAM uses the kinematic viscosity [$\text{Ns}/\text{m}^2 = \text{Pas}$] or the dynamic viscosity [m^2/s] one has simply to take a look on the dimension.

```
nu      nu [ 0 2 -1 0 0 0 0 ] 0.01;
```

Listing 33: *dimensions* of the viscosity

The type of viscosity is primarily determined by the used solver, e.g. compressible or incompressible.

9.4 Pitfall: pressure vs. pressure

The definition of pressure in OpenFOAM differs between the compressible and incompressible solvers. Compressible solvers work with the pressure itself. Incompressible solvers use a modified pressure. The reason for this is, because of $\rho = \text{const}$ the incompressible equations are divided by the density and to eliminate density entirely the modified pressure is introduced into the pressure term.

$$\hat{p} = \frac{p}{\rho} \quad (4)$$

For this reason the entries in the 0/p files differ depending on the solver in use. This is visible by the unit of pressure.

9.4.1 Incompressible

The unit of the pressure in an incompressible solver is defined by (4)

$$[\hat{p}] = \frac{\text{N}}{\text{m}^2} \cdot \frac{\text{m}^3}{\text{kg}} = \text{N} \frac{\text{m}}{\text{kg}} = \frac{\text{kgm}}{\text{s}^2} \cdot \frac{\text{m}}{\text{kg}} = \frac{\text{m}^2}{\text{s}^2} \quad (5)$$

```
dimensions      [ 0 2 -2 0 0 0 0 ];
```

Listing 34: Unit of pressure - incompressible

9.4.2 Compressible

The unit of the pressure in a compressible solver is the physical unit of pressure.

$$[p] = \frac{\text{N}}{\text{m}^2} = \frac{\frac{\text{kgm}}{\text{s}^2}}{\text{m}^2} = \frac{\text{kg}}{\text{ms}^2} \quad (6)$$

```
dimensions      [ 1 -1 -2 0 0 0 0 ];
```

Listing 35: Unit of pressure - compressible

9.4.3 Pitfall: Pressure in incompressible multi-phase problems

When solving a multi-phase problem in an Eulerian-Eulerian fashion, for each phase a momentum equation is solved. In most cases it is assumed that the pressure is equal in all phases. For this reason the incompressible equations can not be divided by the density, because each phase has a different density and therefore, the modified pressure would be different for each phase. To avoid this issue, incompressible Euler-Euler solvers, like *bubbleFoam*, *twoPhaseEulerFoam* or *multiPhaseEulerFoam*, use the physical pressure like compressible solvers do.

10 Files and directories

OpenFOAM saves its data not in a single file, like Fluent does, it uses several different files. Depending on its purpose a specific file is located in one of several folders.

10.1 Required directories

An OpenFOAM case has a minimal set of files and directories. The directory that contains those folders is called the root directory of the case or case directory. Listing 36 shows the output of the commands `pwd` and `ls` when they are invoked from a case directory. The first command returns the absolute path of the current working directory. The second command prints the contents of the current folder. When `ls` is invoked without any options it returns the names of all non-hidden files and folders. In this case there are three subdirectories (*0*, *constant* and *system*). The fact that these three items are directories and not files is indicated by a different color. If `ls` is called with the option `-l` a more detailed list is printed. This detailed list indicates if an entry is a file or a directory.

```
user@host:~/OpenFOAM/user-2.1.x/run/icoFoam/cavity$ pwd
/home/user/OpenFOAM/user-2.1.x/run/icoFoam/cavity
user@host:~/OpenFOAM/user-2.1.x/run/icoFoam/cavity$ ls
0  constant  system
user@host:~/OpenFOAM/user-2.1.x/run/icoFoam/cavity$ ls -l
insgesamt 12
drwxrwxr-x 2 user group 4096 0kt  2 14:53 0
drwxrwxr-x 3 user group 4096 0kt  2 14:53 constant
drwxrwxr-x 2 user group 4096 0kt  2 14:53 system
```

Listing 36: Case directory

0 This is the first of the time-directories. It contains the initial and boundary conditions of all variable quantities. A case does not have to start at time $t = 0$. However, if there is no specific reason for a case to start at another time than $t = 0$, a case will always begin at time $t = 0$. The name of a time-directory is simply the number of elapsed seconds.

constant This folder contains all files dealing with constant quantities as well as the mesh.

polymesh This is a subdirectory of *constant*. In this folder all files defining the mesh reside.

system In this folder all files that control the solver or other tools are located

In the course of computing the case two kinds of folders are created. First of all, at defined times all information is written to the harddisk. A new time-directory is created with the number of elapsed seconds in its name. In this folder all kinds of files are saved. The number of files is equal or larger than in the *0*-directory containing the initial conditions.

The second category of directory subsumes all kinds of folders created for all kind of reasons or by all kind of tools, see Section 10.2 for a brief introduction to some of the more common of them.

10.2 Supplemental directories

Directories described in this Section may be created in the course of a computation.

10.2.1 *processor**

If a case is solved in parallel, i.e. the case is computed using more than one processor at the time. In this case the computational domain has to be decomposed into several parts, to divide the problem between the involved parallel processes. The tool that is used to decompose the case created the *processor**-directories. The *** stands for a consecutive number starting with 0. So, if a case is to be solved using 4 parallel processes, then the domain has to be split into 4 parts. Therefore, the folders *processor0* to *processor3* are created.

Every one of the *parallel**-directories contains a *0*- and also a *constant*-directory containing only the mesh. The *system*-directory remains in the case folder. See Section 12.5 for more information about conducting parallel calculations.

10.2.2 *functions*

functions or functionObjects perform all kind of operations during the computation. Each function creates a folder of the same name to save its data in. See Section 49 for more information about functions.

10.2.3 *sets*

If the tool *sample* has been used, then all data generated by *sample* is stored in a folder named *sets*. See Section 50 for more information about *sample*.

10.3 Files in *system*

In the directory named *system* there are three files for controlling the solver. These files are necessary to run a simulation. Besides them there may also be additional files controlling other tools.

10.3.1 The main files

These files have to be present in the system folder to be able to run a calculation

controlDict This file contains the controls related to time steps, output interval, etc.

fvSchemes In this file the finite volume discretisation schemes are defined

fvSolution This file contains controls related to the mathematical solver, solver algorithms and tolerances.

10.3.2 Additional files

This list contains a selection of the most common files to be found in the system-directory.

probesDict Alternative to the use of the file *probesDict*, *probes* can also be defined in the file *controlDict*.

decomposeParDict Used by *decomposePar*. In this file the number of subdomains and the method of decomposition are defined.

setFieldsDict Necessary for the tool *setFields* to initialise field quantities.

sampleDict Definitions for the post-processing tool *sample*.

11 Controlling OpenFOAM

11.1 The means of exerting control

Classical UNIX applications know several means of controlling their configuration [20]:

- *System-wide run-control files*

An example for these are files in `/etc` on Linux or UNIX systems. For OpenFOAM, such system-wide run-control files are located in `$FOAM_ETC`, which might be `home/user/OpenFOAM/OpenFOAM-3.0.0/etc`. There, we can find the global `controlDict`, controlling OpenFOAM's behaviour installation-wide.

- *System-wide environment variables*

Such a system-wide variable on a Linux system is `$HOSTNAME`, which is the name associated to identify the computer within a network. This name is the same for all users logged in at a certain machine, and it can and should not be changed by a user. For OpenFOAM such system-wide environment variables are `$FOAM_ETC`, `$FOAM_INST_DIR` or `$WM_THIRD_PARTY_DIR`. These variables are equal for all users of a certain installation.

The distinction between system-wide and user-defined settings blurs, when we install OpenFOAM in our home directory, then we are the administrator and the single user of our installation. This distinction was made for clusters, which provide one installation to many users.

- *User-defined run-control files*

A perfect example of a user-defined run-controlled file is the file `.bashrc` in the user's home directory. This file contains user-specific settings. During the installation process of OpenFOAM, this file needs to be edited to make the OpenFOAM installation available to the user.

- *User-set environment variables*

These aren't quite common. On a Linux or UNIX system, a user might set the `$EDITOR` variable, then applications, which might call an editor can simply query this variable to call the preferred editor of the user.

- *Switches and arguments passed on the command line*

These are very common. A widely known example are the command line arguments `-h`, `-help` or `--help` for displaying a summary of the application usage.

The order of the above listed means of control is descending from the system-level down to the per-execution level. With the freedom to choose between five mechanisms to control the behaviour of an application comes great responsibility to the software developer to choose wisely. Nobody wants to pass the same, never-changing command line arguments every time an application is run. Otherwise, users often do not have the possibility to edit system-wide run-control files, so these might be a bad location for settings which change on a daily basis.

11.1.1 Variables

Variables are the best place to store information, which is repeatedly needed. E.g. it would make no sense to specify the installation directory of OpenFOAM in every run-control file which needs to know where OpenFOAM is installed on the system, instead a variable `$FOAM_INST_DIR` is defined in one of OpenFOAM's global run-control files. In all other run-control files, which need to know the installation path, this variable is used. Thus, information redundancy is avoided. Imagine the poor cluster administrators, if some information were stored

in multiple places, and this information were to change. Good luck finding and updating ALL occurrences of this data.

Variables offer the freedom to use the same name (i.e. the variable) regardless of what the actual information is. OpenFOAM is always installed at `$FOAM_INST_DIR`, whether that is `/home/user/OpenFOAM`, `/opt/OpenFOAM` or `/home/user/Desktop/important_softWare`.

11.1.2 Dictionaries

Dictionaries are the run-control files of OpenFOAM. Most of the controls of OpenFOAM are set in so called *dictionaries*. An important *dictionary* is the file *controlDict*. Dictionaries offer a convenient way to store structured information of arbitrary size, which would be rather impossible using variables or command line arguments. Imagine typing all contents of *controlDict* every time you run a solver.

The distinction between global and local dictionaries saves ourselves from messing up the OpenFOAM installation when fiddling with a case's set-up.

11.1.3 Command line arguments

Besides the dictionaries, there are also command-line arguments to control certain aspects of OpenFOAM's solvers and utilities. Command line arguments are the best way to pass information to an application that might change from one run to the other, even when the case is the same.

An example is the `-parallel` command line switch. Regardless of whether we run a case with a single process or in parallel, the case's settings are unchanged. Thus, it would be inconsistent to tell the solver to run in parallel via a case file.

Command line switches are command line arguments, which do not need any additional information. Adding `-help` to a solver name is sufficient to make the solver display its usage summary. A command line argument, on the other hand, needs additional information. An example is the `-time` argument used to tell post-processing tools on which time steps to act upon. Passing `-time` alone without any further information leaves the tool clueless and it will issue an error message.

11.2 Syntax of the dictionaries

The dictionaries need to comply a certain format. The OpenFOAM User Guide states, that the dictionaries follow a syntax similar to the C++ syntax.

The file format follows some general principles of C++ source code.

The most basic format to enter data in a dictionary is the key-value pair. The value of a key-value pair can be any sort of data, e.g. a number, a list or a dictionary.

11.2.1 Keywords - the banana test

As OpenFOAM offers no graphical menus, in some cases allowed entries are not visible at a glance. If a key expects a value of a finite set of data, then the user can enter a value that is definitely not applicable, e.g. *banana*. Then, OpenFOAM produces an error message with a list of allowed entries.

```
--> FOAM FATAL IO ERROR:
expected startTime, firstTime or latestTime found 'banana'
```

Listing 37: Wrong keyword, or the banana test

Listing 37 shows the error message that is displayed when the value *banana* is assigned to the key `startFrom` that controls at which time a simulation should start. The error message contains a note that is formatted in this way: *expected X, Y or Z found ABC*.

If in a dictionary several key-value pairs are erroneous, only the first one produces an error, as OpenFOAM aborts all further operations.

Pitfall: assumptions & default values

In some cases the banana test behaves differently than expected. Listing 38 shows the warning message OpenFOAM returns, when the banana test is used with the control *compression* of *controlDict*. See Section 11.3.2 for a description of this control. In this case, OpenFOAM does not abort but continues to run the case. Instead of returning an error message and exiting, OpenFOAM simply assumes a value in place of the invalid entry.

```
--> FOAM Warning :  
    From function IOstream::compressionEnum(const word&)  
    in file db/IOstreams/IOstreams/IOstream.C at line 80  
    bad compression specifier 'banana', using 'uncompressed'
```

Listing 38: Failed banana test

11.2.2 Mandatory and optional settings

Some settings are expected by the solver to be made. If they are not present, OpenFOAM will return an error message. Other settings have a default value, which is used if the user does not specify a value. In this sense, settings can be divided into mandatory and optional ones.

As mandatory settings causes an error if they are not set, a simulation can be run only if all mandatory settings were made.

About errors

- There will be an error when mandatory settings were not made.
- There is no error message if an optional setting (that is necessary) was omitted. All optional controls have a default value and will be in place.
- There is no error message if a setting was made and that setting is not needed. The solver simply ignores it. Consequently the definition of a variable time step in *controlDict* does not necessarily mean, that the simulation is performed with variable time steps, e.g. if *icoFoam* (a fixed time step solver) is used.
- Sometimes an error message points to the setting of a keyword that is actually not faulty. See Section 11.2.3.

See Section 57.3 for a detailed discussion – including a thorough look at some source code – about reading keywords from dictionaries.

11.2.3 Pitfall: semicolon (;)

Similar to C++, lines are terminated by a semicolon. Listing 39 shows the content of the file *U1* in the *0*-directory. The line defining the boundary condition (BC) for the outlet was not terminated properly. Listing 40 shows the provoked error message. This error message does not mention *outlet*, but rather *walls* – *keyword walls is undefined*. The definition of the boundary condition for the walls comes after the outlet definition. One reason for this may be, that OpenFOAM terminates reading the file after the missing semicolon causes a syntax error, and therefore the boundary condition for the walls remain undefined.

This example demonstrates that the error messages are sometimes not very meaningful if they are taken literally. The error was made at the definition of the BC for the outlet. If only the definition of the BC of the walls is examined, the cause for the error message will remain unclear, because the BC definition of the walls is perfectly correct.

```
dimensions      [0 1 -1 0 0 0 0];  
  
internalField    uniform (0 0 0);  
  
boundaryField  
{  
    inlet  
    {  
        type      fixedValue;
```

```

    value            uniform (0 0 0.03704);
}

outlet
{
    type            zeroGradient
}

walls
{
    type            fixedValue;
    value            uniform (0 0 0);
}
}

```

Listing 39: Missing semicolon in the definition of the BC

```

--> FOAM FATAL IO ERROR:
keyword walls is undefined in dictionary "/home/user/OpenFOAM/user-2.1.x/run/twoPhaseEulerFoam
/case/0/U1::boundaryField"

file: /home/user/OpenFOAM/user-2.1.x/run/twoPhaseEulerFoam/case/0/U1::boundaryField from line
25 to line 47.

From function dictionary::subDict(const word& keyword) const
in file db/dictionary/dictionary.C at line 461.

FOAM exiting

```

Listing 40: Error message caused by missing semicolon

11.2.4 Switches

Besides key-value pairs there are switches. These enable or disable a function or a feature. Consequently, they only can have a logical value.

Allowed values are: *on/off*, *true/false* or *yes/no*. See Section 57.4.1 for a detailed discussion about valid entries.

11.3 The controlDict

In this *dictionary* controls regarding time step, simulation time or writing data to hard disk are located.

The settings in the **controlDict** are not only read by the solvers but also by all kinds of utilities. E.g. some mesh modification utilities obey the settings of the keywords **startFrom** and **startTime**. This has to be kept in mind when using a number of utilities for pre-processing.

11.3.1 Time control

In this Section the most important controls with respect to time step and simulation time are listed. This list makes no claim of completeness.

startFrom controls the start time of the simulation. There are three possible options for this keyword.

firstTime the simulation starts from the earliest time step from the set of time directories.

startTime the simulation starts from the time specified by the **startTime** keyword entry.

latestTime the simulation starts from the latest time step from the set of time directories.

startTime start time from which the simulation starts. Only relevant if **startFrom** **startTime** has been specified. Otherwise this entry is completely ignored¹⁹.

stopAt controls the end of the simulation. Possible values are *{endTime, nextWrite, noWriteNow, writeNow}*.

endTime the simulation stops when a specified time is reached.

¹⁹If the simulation is set to start from *firstTime* or *latestTime*, this keyword can be omitted or the value of this keyword can be anything – **startTime** **banana** does not lead to an error, what would be the case if the simulation started from a specific start time.

writeNow the simulation stops after the current time step is completed and the current solution is written to disk.

endTime end time for the simulation

deltaT time step of the simulation if the simulation uses fixed time steps. In a variable time step simulation this value defines the initial time step.

adjustTimeStep controls whether time steps are of fixed or variable length.²⁰ If this keyword is omitted, a fixed time step is assumed by default. When an adjustable time step is used, then the time step length Δt is controlled via a Courant number criterion. See Section 57.6.4 for more on the Courant number and its influence on the time step.

runTimeModifiable controls whether or not OpenFOAM should read certain dictionaries (e.g. *controlDict*) at the beginning of each time step. If this option is enabled, a simulation can be stopped by using setting **stopAt** to one of these values *{nextWrite, noWriteNow, writeNow}*, see Section 12.2.

maxCo when the simulation is run with an adjustable time step, then we can specify the maximum Courant number, which is used to impose an upper boundary on the time step size.

maxDeltaT when we run a simulation with an adjustable time step, we can provide a manual, hard upper boundary for the maximum time step. This setting limits the time step size, regardless of other conditions allowing for a larger time step, e.g. the Courant number criterion.

In Section 57.6 a couple of aspects regarding time step control are discussed.

11.3.2 Data writing

In *controlDict* the controls regarding data writing can be found. Often, it is not necessary to save every time step of a simulation. OpenFOAM offers several ways to define how and when the data is to be written to the hard disk.

writeControl controls the timing of writing data to file. Allowed values are *{adjustableRunTime, clockTime, cpuTime, runTime, timeStep}*.

runTime when this option is chosen, then every **writeInterval** seconds the data is written. This option has no influence on the time step. Hence, the interval at which data is written may/does not exactly match the entry in **writeInterval**, i.e. for a 1s interval the data may be written at $t = 1.0012, 2.0005, \dots$ s.

adjustableRunTime this option allows the solver to adjust the time step, so that every **writeInterval** seconds the data can be written. This option imposes an upper boundary upon the time step. There must be at least 5 time steps between two instances at which data is written, i.e. $\Delta t \leq 0.2 * \text{writeInterval}$.

timeStep the data is written every **writeInterval** time steps. In this case, **writeInterval** is an integer, not a time.

writeInterval a value that controls the interval of data writing. This value gets its meaning from the value assigned to *writeControl*.

writeFormat controls how the data is written to hard disk. It is possible to write text files or binary files. Consequently, the options are *{ascii, binary}*.

writePrecision controls the precision of the values written to the hard disk.

writeCompression controls whether to compress the written files or not. By default compression is disabled. When it is activated, all written files are compressed using *gzip*.

timeFormat controls the format that is used to write the time step folders.

²⁰This keyword is important only for solvers featuring variable time stepping. A fixed time step solver simply ignores this control without displaying any warning or error message.

timePrecision specifies the number of digits after the decimal point. The default value is 6.

purgeWrite this setting control whether to clear out old time steps. The default value is 0, which means that no clearing out will be conducted. For enabling clearing out old time steps, valid values are positive integer numbers. If enabled with a non-zero value N , only the last N time steps will be retained. Once the simulation has written N time steps to disk, for every new time step saved, the oldest one will be deleted. The initial time step is not affected and will always remain in the case²¹.

Pitfall: timePrecision

OpenFOAM is able to automatically increase the value of **timePrecision** parameter if need arises, e.g. due to a reduction in (dynamic) time step size²². This is typically the case when a simulation diverges and the (dynamic) time step gets decreased by orders of magnitudes. However, simulations that do not diverge may also create the need for an increase in time precision.

Increased the timePrecision from 6 to 7 to distinguish between timeNames at time 4.70884

Listing 41: Exemplary solver output in the case of an automatic increase of the **timePrecision** value.

If a simulation that increased its time precision is to be restarted or continued from the latest time step, then the chosen time precision may not be sufficient to represent the present time step values, i.e. a **timePrecision** of 3 is not sufficient to represent the latest time step at $t = 0.1023$ s. OpenFOAM will apply rounding to the reach the selected number of digits behind the comma. Consequently, OpenFOAM will fail to find files at time $t = 0.102$ s.

This behaviour is hard to detect for an unaware user. The only clue for detection lies in this case in the fourth digit behind the comma, which is present in only in the name of the time step directory but not in the **timeName** that is looked up by OpenFOAM. Listing 42 shows the according error message and a directory listing of the case directory. It is up to the reader to decide whether this is an easy to spot error. The author took some time, which motivated him to elaborate on this issue in this little collection of errors and misbehaviour.

```
--> FOAM FATAL IO ERROR: cannot find file

file: /home/user/OpenFOAM/user-2.3.x/run/icoFoam/cavity/0.102/p at line 0.

    From function regIOobject::readStream()
    in file db/regIOobject/regIOobjectRead.C at line 73.

FOAM exiting

user@host:~/OpenFOAM/user-2.3.x/run/icoFoam/cavity$ ls
0 0.1023 constant system
user@host:~/OpenFOAM/user-2.3.x/run/icoFoam/cavity$
```

Listing 42: Exemple of an error caused by an automatic increase of the **timePrecision** value in the previous simulation run. We fail to restart the simulation as OpenFOAM is not able to find the correct time step.

Non-pitfall: time step control & writeInterval control of functionObjects

Note, that the general settings in the file **controlDict** are not the only settings controlling the time step size. When we use **functionObjects** to extract data from a running simulation, we can control the interval at which the data from the **functionObjects** is written independently from the general write interval of the simulation. The **functionObjects** allow to use the setting **adjustableRunTime** in the same manner as we can use it for the simulation in general.

²¹In the file **TimeIO.C** we see, that each time we reach write-time, the current time step is added to a FIFO stack. Subsequently, the stack's size is checked against the **purgeWrite** value. If the stack is larger, then one item will be removed.

²²A dynamic increase of the **timePrecision** value in simulations with fixed time steps indicates a setting in which the time precision is not sufficient to adequately represent the time step. This leads to a automatic increase of time precision after the first time step is written to disk. I.e. if Δt can't be represented with **timePrecision** number of digits after the comma, then $t_1 + \Delta t$ also can't be represented. Thus, t_1 and $t_1 + \Delta t$ would get the same time name and would consequently be indistinguishable. See Section 57.6.3 on more implementation details on this matter.

Thus in such a case, the time step of the simulation is controlled by the `functionObject` in addition to the general simulation settings. This can lead to a situation, when the write interval of the `functionObject` is sufficiently small, that the maximum allowed Courant number is not reached when running the simulation. This is normal behaviour, since there are multiple criteria at work to determine the time step size, and figuratively spoken: the lowest bidder wins.

Somewhat-non-pitfall: time step control & `writeInterval`

The OpenFOAM manual says, that when we select `runTime` as write-control, then the data is written every `writeInterval` seconds to disk. This does however, allow for the somewhat strange case of a write-interval which is actually (much) larger than the `writeInterval`.

Since, we selected `runTime` as write-control, the `writeInterval` has no effect on the time step size. If the time step controls allow for a time step larger than `writeInterval`, e.g. via the Courant criterion, then OpenFOAM will oblige. In this case, after every time step OpenFOAM notices, that the time elapsed since the last write-to-disk is larger than the `writeInterval`. Thus, it is time to write to disk again.

If we do not want this kind of behaviour, then we need to select `adjustableRunTime` as our write-control, because then the time step will be checked against the `writeInterval`.

More an observation than a pitfall: time step control & `writeInterval`

If we select `adjustableRunTime` as our write-control, then the time step is adjusted to conform the interval set by `writeInterval`. It has been observed, that in this case the time step adjustment adheres to the following boundary:

$$\Delta t \leq \frac{\text{writeInterval}}{5}$$

Thus, OpenFOAM performs at least 5 time steps between writing data to disk. This behaviour might be confusing, and it is not explained in the documentation. Also, your trusted author has not been able to infer this behaviour from OpenFOAM's source code.

11.3.3 Loading additional Libraries

Additional libraries can be loaded with an instruction in `controlDict`. Listing 43 shows how an external library (in this case a turbulence model that is not included in OpenFOAM) is included. This model can be found at <https://github.com/AlbertoPa/dynamicSmagorinsky/>.

```
libs ( "libdynamicSmagorinskyModel.so" ) ;
```

Listing 43: Load additional libraries; *controlDict* entry

Note that the line in Listing 43 is a keyword (`libs`) followed by a list-type entry. Thus, the space between the keyword and the opening parenthesis is vital, since whitespace is used to separate a keyword from its value.

11.3.4 *functions*

functions, or *functionObjects* as they are called in OpenFOAM, offer a wide variety of extra functionality, e.g. probing values or run-time post-processing. See Section 49.

functions can be enabled or disabled at run-time.

11.3.5 Outsourcing a *dictionary*

Some definitions can be outsourced in a separate *dictionary*, e.g. the definition of a *probe-functionObject*.

All inclusive

In this case the *probe* is defined completely in *controlDict*.

```

functions
{
    probes1
    {
        type probes;
        functionObjectLibs ("libsampling.so");

        fields
        (
            P
            U
        );
        outputControl    outputTime;
        outputInterval    0.01;

        probeLocations
        (
            (0.5 0.5 0.05)
        );
    }
}

```

Listing 44: Definition of a *probe* in *controlDict*

Seperate *probesDict*

In this case the definition of the *probe* is done in a seperate file – the *probesDict*. In *controlDict* the name of this dictionary is assigned to the keyword *dictionary*. This dictionary has be located in the *system-directory* of the case. It is not possible to assign the path of this dictionary to this keyword.

```

functions
{
    probes1
    {
        type probes;
        functionObjectLibs ("libsampling.so");

        dictionary probesDict;
    }
}

```

Listing 45: External definition of *probes*; Entry in *controlDict*

```

fields
(
    P
    U
);

outputControl    outputTime;
outputInterval    0.01;

probeLocations
(
    (20.5 0.5 0.05)
);

```

Listing 46: Definition of *probes* in the file *probesDict*

Everything external

There is also the possibility to move the whole definition of a *functionObject* into a seperate file. In this case the macro `#include` is used. This macro is similar to the pre-processor macro if C++.

```

functions
{
    #include "cuttingPlane"
}

```

Listing 47: Completely external definition of a *functionObject*; Entry in `controlDict`

```

cuttingPlane
{
    type                surfaces;
    functionObjectLibs ("libsampling.so");
    outputControl        outputTime;

    surfaceFormat        raw;
    fields                ( alpha1 );

    interpolationScheme   cellPoint;

    surfaces
    (
        yNormal
        {
            type                cuttingPlane;
            planeType            pointAndNormal;
            pointAndNormalDict
            {
                basePoint        (0 0.1 0);
                normalVector      (0 1 0);
            }

            interpolate          true;
        }
    );
}

```

Listing 48: Definition of a *cuttingPlane functionObject* in a separate file named `cuttingPlane`

11.3.6 Pitfalls

timePrecision

If the time precision is not sufficient, then OpenFOAM issues a warning message and increases the time precision without aborting a running simulation.

Listing 49 shows such a warning message. The simulation time exceeded 100s and OpenFOAM figured that the time precision was not sufficient anymore.

```

--> FOAM Warning :
    From function Time::operator++()
    in file db/Time/Time.C at line 1024
    Increased the timePrecision from 6 to 13 to distinguish between timeNames at time 100.001

```

Listing 49: Warning message: automatic increase of time precision

A side effect of this increase in time precision was a slight offset in simulation time. The time step of this simulation was 0.001s and the time steps were written every 0.5s. As it is clearly visible in Listing 50, the names of the time step folders indicate this offset. This effect on the time step folder names was the reason, the automatic increase of time precision was noticed by the author.

However, automatic increase of time precision has no negative effect on a simulation. This purpose of this section is to explain the cause for this effect.

```

101.50000000002
101.00000000002
100.50000000002
100
99.5

```

Listing 50: Time step folders after increase of time precision

11.4 Run-time modifications of dictionaries

If the switch *runTimeModifiable* is set *true*, *on* or *yes*; certain files (e.g. *controlDict* or *fvSolution*) are read anew, if a file has changed. In this way, e.g. the write interval can be changed during the simulation. If OpenFOAM detects a run-time modification it issues a message on the Terminal.

```
regIOobject::readIfModified() :  
  Re-reading object controlDict from file "/home/user/OpenFOAM/user-2.1.x/run/  
  multiphaseEulerFoam/bubbleColumn/system/controlDict"
```

Listing 51: Detected modification of *controlDict* at run-time of the solver

11.5 The fvSolution dictionary

The file *fvSolution* contains all settings controlling the solvers and the solution algorithm. This file must contain two dictionaries. The first controls the solvers and the second controls the solution algorithm.

11.5.1 Solver control

The *solvers* dictionary contains settings that determine the work of the solvers (e.g. solution methods, tolerances, etc.).

11.5.2 Solution algorithm control

The dictionary controlling the solution algorithm is named after the solution algorithm itself. I.e. the name of the dictionary controlling the PIMPLE algorithm is PIMPLE. Note, that the name of this dictionary is in upper case letters unlike most other dictionaries.

Listing 52 shows an example of a PIMPLE dictionary. See Section 42.2 for a detailed discussion on the PIMPLE algorithm.

```
PIMPLE  
{  
  nOuterCorrectors 1;  
  nCorrectors      2;  
  nNonOrthogonalCorrectors 0;  
  pRefCell         0;  
  pRefValue        0;  
}
```

Listing 52: The PIMPLE dictionary

11.6 Command line arguments

OpenFOAM's solvers and utilities can be controlled by a set of command line arguments. Some of them are common to all or many executables, some might be special to a certain tool.

11.6.1 Getting help: -help

The most important command line argument is **-help**. This is common to all solvers and tools of OpenFOAM and it displays a summary of the respective tool.

11.6.2 Getting in control: `-dict`

Certain tools expect to find a specific dictionary containing necessary information. With the `-dict` option, the user can tell the executable, where to look for the dictionary. To the authors knowledge, all tools expecting a dictionary assume a default location and filename. E.g. in older versions of OpenFOAM *blockMesh* expected to find a dictionary named `blockMeshDict` in the `constant/polyMesh` sub-directory of the case's root, in newer versions it checks also the `system` directory. If the user chooses to put the dictionary containing into a different folder, he or she can do so, however, the path to the dictionary now needs to be passed using the `-dict` command line argument.

no control dict

The help summary displayed by `-help`, in some cases, describes the `-dict` options as follows: *read control dictionary from specified location*. However, the dictionary specified with the `-dict` option is not the `controlDict`. Thus, all entries that go into `controlDict` need to go into `controlDict`. For some tools the description of the `-dict` option seems a little ambiguous. What is meant by control dictionary in this case is the dictionary controlling this specific tool, such as `blockMeshDict` controls *blockMesh* or `snappyHexMeshDict` controls *snappyHexMesh*.

12 Usage of OpenFOAM

12.1 Use OpenFOAM

In the most simple case, Listing 53 represents a complete simulation-run.

```
blockMesh
checkMesh
icoFoam
paraFoam
```

Listing 53: Compute a simple simulation case

The first command, *blockMesh*, creates the mesh. The geometry has to be defined in *blockMeshDict*. *checkMesh* performs, as the name suggests, checks on the mesh. The third command is also the name of the solver. All solvers of OpenFOAM are invoked simply by their name. The last command opens the post-processing tool ParaView.

There are additional tasks that extend the sequence of commands shown in Listing 53. These can be

- Convert a mesh created by an other meshing tool, e.g. import a Fluent mesh
- Initialise fields
- Set up an parallel simulation; see Section 12.5

12.1.1 Redirect output and save time

The solver output can be printed to the Terminal or redirected to a file. Listing 54 shows how the solver output is redirected to a file named *foamRun.log*.

```
mpirun -np N icoFoam -parallel > foamRun.log
```

Listing 54: Redirect output to a file

Redirecting the solver output does not only create a log file, it also save the time that is needed to print the output to the Terminal. In some cases this can reduce simulation time drastically. However, writing to hard disk also takes its time.

Time steps	Cells	Print to Terminal		Redirect to file	
		executionTime	clockTime	executionTime	clockTime
5000	400	6,36	9	4,6	6
10000	400	12,71	18	9,22	10
12500	400	15,8	23	11,54	12
25000	400	32,33	47	22,99	23
5000	1600	9,74	11	9,3	10
5000	6400	282,19	283	282,83	283

Table 1: Run-time *cavity* test case

executionTime is the time the processor takes to calculate the solution of the case. *clockTime* is the time that elapses between start and end of the simulation, this is the time the wall clock indicates. The value of the *clockTime* is always larger than the value of the *executionTime*, because computing the solution is not the only task the processor of the system performs. Consequently, the value of the *ClockTime* depends on external factors, e.g. the system load.

Redirect output to nowhere

If the output of a program is of no interest it can be redirected to virtually nowhere to prevent it from being displayed on the Terminal. Listing 55 shows how this is done. `/dev/null` is a special file on unix-like systems that discards all data written to it.

```
mpirun -np N icoFoam -parallel > /dev/null
```

Listing 55: Redirect output to nowhere

12.1.2 Run OpenFOAM in the background, redirect output and read log

In Section 12.1.1 the redirection of the solver output was explained. To monitor the progress of running calculation the end of the log can be read with the *tail* command.

Listing 56 shows how a simulation with *icoFoam* is started and the solver output is redirected. The `&` at the end of the line causes the invoked command to be executed in the background. The Terminal remains therefore available. Otherwise the Terminal would be waiting for *icoFoam* to finish before executing any further commands.

The second command invoked in Listing 56 prints the last 5 lines of the log file to the Terminal. *tail* returns the last lines of a text file. Without the parameter `-n` *tail* returns by default the last 10 lines.

```
user@host:~/OpenFOAM/user-2.1.x/run/icoFoam/cavity$ icoFoam > foamRun.log &
[1] 10416
user@host:~/OpenFOAM/user-2.1.x/run/icoFoam/cavity$ tail foamRun.log -n 5
ExecutionTime = 0.74 s  ClockTime = 1 s

Time = 1.12

Courant Number mean: 0.444329 max: 1.70427
user@host:~/OpenFOAM/user-2.1.x/run/icoFoam/cavity$
```

Listing 56: Read redirected output from log file while the solver is running

12.1.3 Save hard disk space

OpenFOAM saves the data of the solution in intervals in time directories. The name of a time directory represents the time of the simulation. Listing 57 shows the content of a case directory after the simulation has finished. Besides the three folders that define the case (*0*, *constant* and *system*) there are more time directories and a *probes1*-folder present.

```

user@host:~/OpenFOAM/user-2.1.x/run/icoFoam/cavity$ ls
0  0.1  0.2  0.3  0.4  0.5  0.6  0.7  0.8  0.9  1  constant  probes1  system
user@host:~/OpenFOAM/user-2.1.x/run/icoFoam/cavity$

```

Listing 57: List folder contents

The *probes1*-directory contains the data generated by the functionObject named probes1. The time-directories contain the solution data of the whole computational domain. Listing 58 shows the contents of the *0*- and the *0.1*-directory. Typically, time-directories generated in the course of the computation contain more data than the *0*-directory defining the initial conditions.

```

user@host:~/OpenFOAM/user-2.1.x/run/icoFoam/cavityBinary$ ls 0
p  U
user@host:~/OpenFOAM/user-2.1.x/run/icoFoam/cavityBinary$ ls 0.1
p  phi  U  uniform
user@host:~/OpenFOAM/user-2.1.x/run/icoFoam/cavityBinary$

```

Listing 58: List folder contents

Using binary files or compressing files

In general the time-directories use the majority of the hard disk space a completed case takes. If the time-directories are saved in binary instead of ascii format, these use generally a little less space. Another advantage of storing time step data in binary format, the time step data has full precision.

OpenFOAM also offers the possibility to compress all files in the time step directories. For compression OpenFOAM uses *gzip*, this is indicated by the files names in the time step directories, i.e. **alpha1.gz** instead **alpha1**.

Table 2 shows a comparison of hard disk use. The most reduction is achieved by compressing ascii data files. However, storing the time step data in ascii has the disadvantage that the numerical precision is limited to the number of digits stated with the **writePrecision** keyword in the **controlDict**. In this case **writePrecision** was set to 6, i.e. numbers have up to 6 significant digits. Compressing the binary files shows less effect than compressing the ascii files, which indicates that the binary files contain less redundant bytes.

Write settings	Used space	reduction	
ascii	45.5 MB		
ascii, compressed	16.7 MB	28.8 MB	-63.3 %
binary	33.8 MB	11.7 MB	-25.7 %
binary, compressed	28.8 MB	16.7 MB	-36.7 %

Table 2: Comparison of hard disk space consumption

Make sure to avoid unnecessary output

Disk space can easily be wasted by writing everything to disk. Not only writing too many time steps to disk can waste space, *functionObjects* can be the culprit too. See 49.6.3.

12.2 Abort an OpenFOAM simulation

An OpenFOAM simulation ends when the simulation time reaches the value specified with the **endTime** keyword in **controlDict**. However, we also need to be able to stop a simulation prematurely. This section explains how to end a simulation in a controlled manner, i.e. the current state of the solution is written to the harddisk in order to be able to continue the simulation at a later time.

As a prerequisite, the **runTimeModifiable** flag has to be enabled in **controlDict**. This keyword controls whether **controlDict** is monitored for changes during the run-time of the simulation. This is necessary for this method to work. Otherwise, the simulation will stop at **endTime**.

To abort a simulation we simply need to change the value of the **stopAt** entry in **controlDict** from **endTime** to **writeNow**. When OpenFOAM detects the change and re-reads **controlDict**, this causes OpenFOAM to finish its current time step and write the state of the solution to disk before ending the run.

12.3 Terminate an OpenFOAM simulation

This section describes how to terminate a running OpenFOAM simulation. See Section 12.2 on how to abort a simulation in a controlled manner, i.e. saving the current solution and stop the simulation.

This section explains how to terminate a running simulation immediately and without saving the current solution. Use this approach when you wouldn't use the solution anyway, e.g. because you chose incorrect settings.

12.3.1 Terminate a process in the foreground

If a command is executed in the Terminal without any additional parameters the process runs in the foreground. The Terminal is therefore busy and can not be used until the process is finished. When a process is running in the foreground it can easily be terminated by pressing `CTRL`+`C`. Listing 59 features the GNU command `sleep`. The only function of this command is to pause for a specified amount of time. With this command the premature termination of a process can be tried.

```
user@host:~$ sleep 3
user@host:~$
```

Listing 59: Keep the Terminal busy

12.3.2 Terminate a background process

If a process runs in the background, the Terminal is free to be used for further tasks while the process is running. In this case, the background process can not be terminated by pressing `CTRL`+`C` because the Operating System can not tell which background process the user wants to terminate.

Identify the process

On UNIX based systems every process is identified by a unique number. This is the PID, the **p**rocess **i**dentifier. The PID is equivalent to a licence plate for a car. During run-time this number is unique. However, after a process has finished the PID of this process is available for other, later processes.

To find out which processes are currently running, invoke the command `ps`. This lists all running processes. Without any further parameters only the processes that were executed from the current Terminal are listed. Listing 60 shows the result if a new Terminal is opened and `ps` is called. The first entry – `bash` – is the Terminal itself. The second entry – `ps` – is the only other process active at the time `ps` looks for all running processes. The PID is listed in the first column of Listing 60. Depending on the parameters passed to `ps` the output can be formatted differently.

```
user@host:~$ ps
  PID TTY          TIME CMD
 13490 pts/1    00:00:00 bash
 13714 pts/1    00:00:00 ps
user@host:~$
```

Listing 60: List processes in a fresh Terminal

The output of 60 is rather dull. However, there are lots of parameters telling `ps` what to do. The option `-e` makes `ps` list all systemwide running processes. The output of such a call can be quite long, because `ps` lists all processes started by the users as well as all system processes²³.

The option `-F` controls the output format of `ps`. In this case `-F` stands for *extra full*. This means the output contains a lot of information. Another option to display much information is `-l`. This option truncates the names of the processes to 15 characters, whereas `-F` displays not only the full name of the process, it also displays the parameters with which the processes were called.

```
ps -eF
```

Listing 61: List all running processes of the system

`ps` displays much information about a process. For terminating a process only the PID is necessary.

²³System processes are processes run by the Operating System itself.

Search in the list of processes

The output of `ps` is a list which can be quite long. To terminate a certain process its PID has to be known. Searching a number in a list of numbers can be quite painful and errorprone. Therefore it would be handy to search in the list `ps` has returned for the desired process.

Before all else, `grep` does the trick. And now for something more detailed. `grep` is a program that searches the lines of its input for a certain pattern. `grep` can use a file or the standard input as its input. As it is unpractical to redirect the output of `ps` into a file only for `grep` to read it, we directly redirect the output of `ps` to the input of `grep`. This is achieved by the use of a pipe.

Listing 62 shows how this is done. The first part of the command invoked – `ps -eF` – calls `ps` to list all processes currently running in great detail. The option `-F` is used to make sure long process names can be distinguished, e.g. to tell ***buoyantBoussinesqPimpleFoam*** apart from ***buoyantBoussinesqSimpleFoam***. Both are standard solvers of OpenFOAM. The bold part are the first 15 characters of the solver's name. If the option `-F` was omitted and both solvers were running, the results of `ps` would be ambiguous.

The second part of the command invoked in Listing 62 shows the call of `grep`. `grep` can be called with one or two arguments. If only one argument is passed to `grep`, `grep` uses the standard input as input. If `grep` is called with two parameters, the second argument has to specify the file from which `grep` has to read. As `grep` is called with only one argument, it reads from the standard input.

Because it would be even more boring to type the list returned by `ps` we redirect the output of `ps` to the standard input of `grep`. This is done by the pipe. The character `|` marks the connection of two processes in the Terminal. The command left of the `|` passes its output directly to the command specified right of the `|`.

Now we can read and interpret Listing 62. It shows the output of the search for all running processes containing the pattern **Foam**. In this case a parallel computation is going on. The first line of the result is *mpirun*. This process controls the parallel running solvers. The next four lines are the four instances of the solver. How parallel simulation works is explained in Section 12.5. The second last entry of the result is `grep` waiting for input²⁴. The last line of the result is the pdf viewer which displays this document at that time. This example shows that is important to choose the pattern wisely, the search may return unexpected results.

```
user@host:~$ ps -ef | grep Foam
user  11005  5117  0 17:11 pts/2    00:00:05 mpirun -np 4 twoPhaseEulerFoam -parallel
user  11006  11005 99 17:11 pts/2    00:40:27 twoPhaseEulerFoam -parallel
user  11007  11005 99 17:11 pts/2    00:40:28 twoPhaseEulerFoam -parallel
user  11008  11005 99 17:11 pts/2    00:40:27 twoPhaseEulerFoam -parallel
user  11009  11005 99 17:11 pts/2    00:40:26 twoPhaseEulerFoam -parallel
user  11673  11116  0 17:52 pts/12   00:00:00 grep --color=auto Foam
user  32041      1  0 Aug01  ?        00:00:31 evince /tmp/lyx_tmpdir.J18462/lyx_tmpbuf0/open
FoamUserManual_CDLv2.pdf
user@host:~$
```

Listing 62: Search for processes

List only specified processes

You can tell `ps` directly in which processes you are interested. The option `-C` of `ps` makes `ps` list only those processes that stem from a certain command. Listing 63 shows the output when `ps -C twoPhaseEulerFoam` is typed into the Terminal. In this case also there are four parallel processes running. Notice, that only the processes directly related to the solvers are shown. No other results are displayed unlike in Listing 62.

One has to bear in mind, that `ps -C` does not search for patterns. If the command name passed to `ps` as an argument is misspelled, `ps` will not display the desired result. Listing 64 shows the effect of typos in this case. The truncation of the process name in the list does not affect the search if the passed command name is equal or longer than the truncated process name. The first two commands issued in Listing 64 result in a list of all running instances of the solver. If the passed argument is shorter than the truncated process name – the third command – `ps` does not output any results. Also if there is a typo in the passed argument, `ps` does not find anything.

```
user@host:~$ ps -C twoPhaseEulerFoam
  PID TTY          TIME CMD
 11005 pts/2    00:00 mpirun
 11006 pts/2    00:40 twoPhaseEulerFoam
 11007 pts/2    00:40 twoPhaseEulerFoam
 11008 pts/2    00:40 twoPhaseEulerFoam
 11009 pts/2    00:40 twoPhaseEulerFoam
```

²⁴On most Unix-like systems processes connected by a pipe are started at the same time. For this reason `grep` is already running while `ps` is listing all running processes.

```

11006 pts/2      00:47:44 twoPhaseEulerFo
11007 pts/2      00:47:44 twoPhaseEulerFo
11008 pts/2      00:47:44 twoPhaseEulerFo
11009 pts/2      00:47:43 twoPhaseEulerFo
user@host:~$

```

Listing 63: List all instances of *twoPhaseEulerFoam*

```

user@host:~$ ps -C twoPhaseEulerFoa
  PID TTY          TIME CMD
 12741 pts/0      00:00:34 twoPhaseEulerFo
 12742 pts/0      00:00:34 twoPhaseEulerFo
 12743 pts/0      00:00:34 twoPhaseEulerFo
 12744 pts/0      00:00:34 twoPhaseEulerFo
user@host:~$ ps -C twoPhaseEulerFo
  PID TTY          TIME CMD
 12741 pts/0      00:00:36 twoPhaseEulerFo
 12742 pts/0      00:00:36 twoPhaseEulerFo
 12743 pts/0      00:00:36 twoPhaseEulerFo
 12744 pts/0      00:00:36 twoPhaseEulerFo
user@host:~$ ps -C twoPhaseEulerF
  PID TTY          TIME CMD
user@host:~$ ps -C twPhaseEulerFoa
  PID TTY          TIME CMD

```

Listing 64: List all instances of *twoPhaseEulerFoam* – the effect of typos

Terminate

The operating system interacts with running processes using signals. The user can also send signals to processes using the command *kill*. *kill* sends by default the termination signal. To identify the process to which the signal is to be sent, the PID of this process has to be passed as an argument.

Listing 65 shows how the program *sleep* is executed, all running processes are listed, the running instance of *sleep* is terminated and the running processes are listed again. When *ps* was executed the second time, a message is displayed stating the process has been terminated²⁵. If the process would not have been terminated the message at the “natural” end of the process would be like in Listing 66²⁶.

```

user@host:~$ sleep 20 &
[1] 13063
user@host:~$ ps
  PID TTY          TIME CMD
 12372 pts/0      00:00:00 bash
 13063 pts/0      00:00:00 sleep
 13064 pts/0      00:00:00 ps
user@host:~$ kill 13063
user@host:~$ ps
  PID TTY          TIME CMD
 12372 pts/0      00:00:00 bash
 13065 pts/0      00:00:00 ps
[1]+  Beendet                  sleep 20
user@host:~$

```

Listing 65: Terminate a process using *kill*

```

user@host:~$ sleep 1 &
[1] 13126
user@host:~$ ps
  PID TTY          TIME CMD
 12372 pts/0      00:00:00 bash
 13127 pts/0      00:00:00 ps
[1]+  Fertig                  sleep 1
user@host:~$

```

²⁵On other systems this message is displayed immediately – see Listing 67. In this case the procedure was tried on the local computing cluster.

²⁶A system with English language setting the message would read **Terminated** if the process would have been terminated and **Done** if the process would have been allowed to finish.

Listing 66: The natural end of a process

```
user@cluster user> sleep 10 &
[1] 31406
user@cluster user> kill 31406
user@cluster user>
[1] Terminated sleep 10
user@cluster user>
```

Listing 67: Terminate a process using *kill* on a different machine

12.4 Continue a simulation

If a simulation has ended at the end time or if it has been aborted there may be the need to continue the simulation. The most important setting to enable a simulation to be continued has to be made in the file `controlDict`. There, the keyword `startFrom` controls from which time the simulation will be started.

The easiest way to continue a simulation is to set the `startFrom` parameter to `latestTime`. Then, if necessary, the value of `endTime` needs to be adjusted. After this changes, the simulation can be continued by simply invoking the solver in the Terminal.

12.5 Do parallel simulations with OpenFOAM

OpenFOAM is able to do parallel simulations. There is no great difference between calculating a case with one single process or using many parallel processes. The only obvious additional task is to split the computation domain into several pieces. This step is called *domain decomposition*. After the domain is decomposed several instances of the solver are running the case on a subdomain each. Additionally, the invocation of the solver differs from the single process case.

In fact, not only solvers are capable to be run in parallel, also many pre- and post-processing tools can be similarly applied in parallel.

12.5.1 Starting a parallel simulation

To enable a simulation using several parallel instances of a solver, OpenFOAM uses the MPI standard in the implementation of OpenMPI. OpenMPI ensures that all parallel instances of the solver run synchronously. Otherwise the simulation would generate no meaningful results. In order to be able to manage all parallel processes the simulation has to started using the command *mpirun*.

Listing 68 shows how a parallel simulation using 4 parallel processes is started. The solver outputs are redirected into a file called `> foamRun.log` and the simulation runs in the background of the Terminal. So the same Terminal can be used to monitor the progress of the calculation. See Section 12.1.2 for a discussion about running a process in the background.

The output message in the Listing shows the PID of the running instance of *mpirun*. This PID can be used to terminate the parallel calculation, like it is explained in Section 12.3.2.

```
user@host:~$ mpirun -np 4 icoFoam -parallel > foamRun.log &
[1] 11099
user@host:~$
```

Listing 68: Run OpenFOAM with 4 processes

The number of processes, in this case 4, has to be equal the number of *processor** folders. These folders are created by *decomposePar* and their number is defined in *decomposeParDict*. See Section 12.5.2 for information about domain decomposition.

If this numbers – the number of *processor** folders and the number of parallel processes with which *mpirun* is invoked – are not equal OpenFOAM issues an error message similar to Listing 69. In this case the domain was decomposed into 4 subdomains and it was tried to start the parallel simulation with 2 processes. If the parallel simulation is called with too many processes, OpenFOAM issues an error message like in Listing 70. The first example shows, that OpenFOAM reacts differently whether the parallel job was started with too little or too many processes.

```
[0] --> FOAM FATAL ERROR:
[0] "/home/user/OpenFOAM/user-2.1.x/run/icoFoam/cavity/system/decomposeParDict" specifies 4
    processors but job was started with 2 processors.
```

Listing 69: Run OpenFOAM with too little parallel processes

```
[0] --> FOAM FATAL ERROR:
[0] number of processor directories = 4 is not equal to the number of processors = 8
```

Listing 70: Run OpenFOAM with too many parallel processes

Pitfall: `-parallel`

The parameter `-parallel` is important. If this parameter is omitted, the solver will be executed n times. Listing 71 shows the output of the command `ls` when it is run with `mpirun` with two processes. In this case `ls` is simply run twice.

If the parameter `-parallel` is missing, the same happens as in the case of `ls`. The simulation is run by n processes at roughly the same time. Listing 72 shows the first lines of output of a situation where the `-parallel` parameter was omitted. All solvers start the calculation of the whole case and write their output to the Terminal. The output appears on the Terminal in the order as it is generated by the solvers – in other words, the output on the Terminal is completely disarranged. If the `-parallel` parameter is missing, there is also no check if the `processor*` folders are present.

```
user@host:~/OpenFOAM/user-2.1.x/run/icoFoam/cavity$ mpirun -np 2 ls
0 constant system
0 constant system
user@host:~/OpenFOAM/user-2.1.x/run/icoFoam/cavity$
```

Listing 71: Run `ls` using 2 processes

```
user@host:~/OpenFOAM/user-2.1.x/run/icoFoam/cavity$ mpirun -np 4 icoFoam
/*-----*\
| ===== |
|  \ \      /  F ield      | OpenFOAM: The Open Source CFD Toolbox |
|  \ \      /  O peration   | Version:  2.1.x                     |
|   \ \      /  A nd        | Web:      www.OpenFOAM.org          |
|    \ \      /  M anipulation |
|     \ \      /
/*-----*\
Build   : 2.1.x-6e89ba0bcd15
Exec    : icoFoam
Date    : Jan 29 2013
Time    : 10:51:12
Host    : "host"
PID     : 25622
/*-----*\
| ===== |
|  \ \      /  F ield      | OpenFOAM: The Open Source CFD Toolbox |
|  \ \      /  O peration   | Version:  2.1.x                     |
|   \ \      /  A nd        | Web:      www.OpenFOAM.org          |
|    \ \      /  M anipulation |
|     \ \      /
/*-----*\
Build   : 2.1.x-6e89ba0bcd15
Exec    : icoFoam
```

Listing 72: Run `icoFoam` without the `-parallel` parameter

Pitfall: domain decomposition

If there was no domain decomposition prior to starting a parallel simulation, OpenFOAM will issue an corresponding error message.

```

[0] --> FOAM FATAL ERROR:
[0] twoPhaseEulerFoam: cannot open case directory "/home/user/OpenFOAM/user-2.1.x/run/
    twoPhaseEulerFoam/testColumn/processor0"
[0]
[0] FOAM parallel run exiting

```

Listing 73: Missing *domain decomposition*

Pitfall: domain reconstruction

After a parallel simulation has ended, all data is residing in the *processor** folders. If *paraView* is started – without prior domain reconstruction – *paraView* will only find the data of the 0 directory.

12.5.2 Domain decomposition

Before a parallel simulation can be started the domain has to be decomposed into the correct number of subdomains – one for each parallel process. The parallel processes calculate on their own subdomain and exchange data of the border regions at the end of each time step. This is also the reason why the parallel processes have to be synchronous. Otherwise, processes with a lower computational load would overtake other processes and they would exchange data from different times.

Just before starting the simulation the domain has to be decomposed. The tool *decomposePar* is used for this purpose. Other operations, e.g. initialising fields using *setFields* have to take place before the domain decomposition. *decomposePar* reads from *decomposeParDict* in the *system* directory. This file has to contain at least the number of subdomains and the decomposition method.

decomposePar creates the *processor** directories in the case directory. Inside the *processor** folders a *0* and a *constant* folder are created. The *0* folder contains the initial and boundary conditions of the subdomain and the *constant* folder contains a *polyMesh* folder containing the mesh of the subdomain.

All parallel processes read from the same *system* directory, as the information stored there is not affected by the domain decomposition. Also the files in the *constant* directory are not altered.

Pitfall: Existing decomposition

If the domain has already been decomposed and *decomposePar* is called again, e.g. because the number of subdomains has been changed or some fields have been reinitialised, OpenFOAM issues an error message. Listing 74 shows an example. In this case the domain has already been decomposed into 2 subdomains and the attempt is made to decompose it again. OpenFOAM always issues an error message, whether the number of subdomains has changes or not.

The resulting error message proposes two possible solutions. The first is to invoke *decomposePar* with the *-force* option to make *decomposePar* remove the *processor** folders before doing its job. The second proposed solution is to manually remove the *processor** folders. In this case the error message contains the proper command to do so. The user can retype the command or copy and paste it into the Terminal.

```

--> FOAM FATAL ERROR: Case is already decomposed with 2 domains, use the -force option or
    manually
remove processor directories before decomposing. e.g.,
    rm -rf /home/user/OpenFOAM/user-2.1.x/run/icoFoam/cavity/processor*

```

Listing 74: Already decomposed domain

Time management with *decomposePar*

In the course of an update of OpenFOAM *decompose* gained the option *-time*. This enhancement took place between the release of OpenFOAM 2.1.0 and OpenFOAM 2.1.1. Such enhancements typically first appear in the repository release OpenFOAM 2.1.x. So, it may be, that some installations of OpenFOAM 2.1.x contain this feature and some not depending on the time of installation or the time of the last update.

The option *time* lets the user specify a time from which or a time range in which the domain is to be decomposed. Listing 75 shows some examples of how this option works.

The option `-latestTime` makes *decomposePar* use the latest time step as starting time step for the subdomains.

```

user@host:~/OpenFOAM/user-2.1.x/run/icoFoam/cavity$ ls
0 0.1 0.2 constant probes1 processor0 processor1 processor2 system
user@host:~/OpenFOAM/user-2.1.x/run/icoFoam/cavity$ decomposePar -time 0.1:0.2 -force > /dev/
null
user@host:~/OpenFOAM/user-2.1.x/run/icoFoam/cavity$ ls processor0
0.1 0.2 constant
user@host:~/OpenFOAM/user-2.1.x/run/icoFoam/cavity$ decomposePar -time 0.2 -force > /dev/null
user@host:~/OpenFOAM/user-2.1.x/run/icoFoam/cavity$ ls processor0
0.2 constant
user@host:~/OpenFOAM/user-2.1.x/run/icoFoam/cavity$

```

Listing 75: Time management with *decomposePar*

12.5.3 Domain reconstruction

To be able to look at the results the data has to be reassembled again. This job is done by *reconstructPar*. This tool collects all data of the *processor** folders and reconstructs the original domain using all the generated time step data. After *reconstructPar* has finished the data of the whole domain resides in the case directory and the data of the subdomains resides in the *processor** folders.

Listing 76 shows the content of the case directory after a parallel simulation has finished. The first command is a simple call of `ls` to display the contents of the case directory. This is not different from the situation before the parallel simulation was started with the exception of the log file. However, this log file could be from a previous run. So, listing the contents after a parallel simulation has finished carries no real information.

The second command lists the contents of the *processor0* directory. In this directory – as well as in all other *processor** folders – there is time step data. The third command reconstructs the domain. After this tool has finished, the case directory also contains time step data. The last command lists the contents of the *processor0* folder again. This data has not been removed. So, a finished parallel case stores its time step data twice and therefore uses a lot of space.

```

user@host:~/OpenFOAM/user-2.1.x/run/icoFoam/cavity$ ls
0 constant foamRun.log probes1 processor0 processor1 processor2 processor3 system
user@host:~/OpenFOAM/user-2.1.x/run/icoFoam/cavity$ ls processor0
0 0.1 0.2 0.3 0.4 0.5 constant
user@host:~/OpenFOAM/user-2.1.x/run/icoFoam/cavity$ reconstructPar > foamReconstruct.log &
[1] 26269
user@host:~/OpenFOAM/user-2.1.x/run/icoFoam/cavity$ ls
0 0.1 0.2 0.3 0.4 0.5 constant foamReconstruct.log foamRun.log probes1 processor0
processor1 processor2 processor3 system
[1]+  Fertig reconstructPar > foamReconstruct.log
user@host:~/OpenFOAM/user-2.1.x/run/icoFoam/cavity$ ls processor0
0 0.1 0.2 0.3 0.4 0.5 constant
user@host:~/OpenFOAM/user-2.1.x/run/icoFoam/cavity$

```

Listing 76: A finished parallel simulation

Time management

If a simulation has been started from $t = t_1$ the domain has to be reconstructed for times $t > t_1$. Calling *reconstructPar* without any options regarding time, the program starts reconstructing the domain at the earliest time. To prevent the tool from reconstructing already reconstructed time steps the `-time` option can be used. Listing 77 shows how simulation results are reconstructed for $t \leq 60$ s.

```

reconstructPar -time 60:

```

Listing 77: Zeitparameter für *reconstructPar*

Another option to reconstruct only the new time steps is the command line option `-newTimes`. By using this option the proper time span to reconstruct is automatically determined.

12.5.4 Run large studies on computing clusters

Simulating parallel on a machine brings some advantages and enables the user to run even large simulations on a workstation. However, if the cases is very large, or parametric studies are to be conducted, using the workstation can be counter productive. Therefore, simulating on a computing cluster is the method of choice for large scale calculations. The user can follow a two step method.

1. Set up the case and run some test simulations, e.g. for a small number of time steps, on the workstation to ensure the simulation runs
2. Do the actual simulation on the cluster

The fact, that OpenFOAM runs on a great number of platforms enables the user to do simulations on the workstation as well as on a big cluster with tens or hundreds of processors.

Run OpenFOAM using a script

Section 66.5 explains how to set up a script that runs multiple cases.

12.5.5 Weird MPI behaviour

Parallel simulation silently fails when a network interface is disconnected

Some weird behaviour was observed by your trusted author with regards to running parallel simulations. Although, not the fault of OpenFOAM, it still is of interest, as this behaviour can be quite annoying and mysterious. Furthermore, when OpenFOAM simulations silently fail, then the first route of investigation for most users is most probably related to OpenFOAM, as can be seen here²⁷ in the OpenFOAM forums.

It was observed that parallel simulations on a laptop simply stopped without error message, when its fragile wifi connection dropped. Apparently, MPI uses all interfaces it can find for communication between its child processes. If at the start of a simulation, wifi is available, MPI will use the wifi network interface. If the wifi connection fails, then `mpirun` silently stops.

This behaviour was reported in OpenFOAM's bug reporting platform²⁸, yet the reason is caused by MPI itself. Thus, OpenFOAM is not at fault here. This behaviour was also reported with respect to general MPI use²⁹.

Thus, if a network connection causes problems, `mpirun` can be explicitly told not to use a specific interface. Listing 78 shows how to keep `mpirun` from using the wifi interface. What is important to note in the listing, is the twofold exclusion of the wifi interface. Both, OOB (out of band messaging) and BTL (MPI point-to-point Byte Transfer Layer, used for MPI point-to-point messages on some types of networks) need to be told not to use wifi.

```
mpirun --mca oob_tcp_if_exclude wlp3s0 --mca btl_tcp_if_exclude wlp3s0 -np 2
someFoamApplication -parallel > logFile.log &
```

Listing 78: Excluding the wifi interface (named `wlp3s0` in this case) from use by `mpirun`

Alternatively, this setting can also be made permanent via an environment variables³⁰.

12.6 Using tools

OpenFOAM consists besides of solvers of a great collection of tools. These tools are used for all kind of operations.

All solvers and tools of OpenFOAM³¹ assume that they are called from the case directory. If an executable is to be called from another directory the path to the case directory has to be specified. Then the option `-case` has to be used to specify this path.

Listing 79 shows the error message displayed by the tool *fluentMeshToFoam* as it was executed from the *polyMesh* directory. The tool added the relative path `system/controlDict` to the current working directory. This resulted in an invalid path to *controlDict* as the error message tells the user. Actually, the error message

²⁷<https://www.cfd-online.com/Forums/openfoam-bugs/191087-openfoam-stops-when-there-no-internet.html>

²⁸<https://bugs.openfoam.org/view.php?id=2821>

²⁹<https://stackoverflow.com/questions/32162317/mpirun-hangs-when-connected-to-wifi/49662866>

³⁰<https://www.open-mpi.org/faq/?category=tuning#setting-mca-params>

³¹No exception known to the author.

states that the file could not be found. This does not solely imply an invalid path. The file could simply be missing.

```
--> FOAM FATAL IO ERROR:
cannot find file

file: /home/user/OpenFOAM/user-2.1.x/run/icoFoam/testCase/constant/polyMesh/system/controlDict
at line 0.

From function regIOobject::readStream()
in file db/regIOobject/regIOobjectRead.C at line 73.

FOAM exiting
```

Listing 79: Wrong path

The correct usage of the `-case` option is shown in Listing 80. There the correct path to the case directory – two levels upwards – is specified using `../..`³²

```
user@host:~/OpenFOAM/user-2.1.x/run/icoFoam/testCase/constant/polyMesh$ fluent3DMeshToFoam -
case ../.. caseMesh.msh
```

Listing 80: Specify the correct path to the case

12.7 Using OpenFOAM on a local machine and a cluster

If you are using OpenFOAM, or learning to use OpenFOAM, there is a big chance that you will use OpenFOAM not only on your local machine, the computer you sit in front of, but also on a remote computing cluster.

12.7.1 General tips

Use the same versions locally and remotely

In order to avoid obscure problems stemming from version conflicts, make sure to use the same version on the local machine as on the cluster. Usually, a computing cluster is managed by an IT department, which is responsible to maintain the cluster. Ideally, the cluster has the point-release(es) of OpenFOAM installed. This ensures, that the behaviour of OpenFOAM will be clearly defined.

Thus, we can only influence which versions of OpenFOAM are installed on our local machine. Installing several versions of OpenFOAM side-by-side is no problem. Thus, there should be no reason not to install the same version(s) of OpenFOAM, which are installed on the cluster, on the local machine. This way you can make sure that a case will run on the cluster by testing it on the local machine first.

12.7.2 Pitfalls

Delete dynamicCode folder

A very common workflow, when using OpenFOAM on a remote computing cluster, involves setting-up the case on your local machine, and then pass the case to the number crunching cluster. Set-up the case, and subsequent trouble-shooting is best done locally, since generally time on clusters is a limited resource, which is best not wasted with trouble-shooting.

However, when tinkering with a case with dynamic code, e.g. coded boundary conditions or coded function objects, we need to delete the `dynamicCode` directory from the case, before passing the case on to the cluster. The local machine and the cluster, are most probably not completely binary compatible, causing the parts relying on dynamic code to cause OpenFOAM to abort. By deleting the

³²On most Linux or Unix systems `.` refers to the current directory and `..` refers to the directory above the current one. To change in the Terminal one directory upwards on Linux `cd ..` does the job and on MS-DOS or Windows `cd..` is the proper command.

Also, on Linux systems the tilda `~` refers to the home directory of the current user.

Part III

dynamicCode

folder, we ensure that the OpenFOAM installation compiles all dynamic code related parts of the case by itself.

A common nuisance related to this topic occurs when dealing with cases featuring coded boundary conditions, such as `codedFixedValue`. Operations on the local machine, e.g. parallel decomposition, cause the local OpenFOAM installation to compile the coded boundary condition. Later, when starting the parallel run of the case on the remote machine, the coded boundary condition is not recognized by the remote OpenFOAM installation, and thus (remote) OpenFOAM aborts with an error message saying that the specified boundary condition does not exist.

Part IV

Pre-processing

This part of the document deals with all issues related to pre-processing, i.e. all the tasks necessary to set-up a valid simulation case. The majority of the following sections deal with the mesh. The final sections of this part deal with general case manipulation and initialisation.

13 Mesh basics

13.1 Basics of the mesh

13.1.1 Files

A mesh is defined by OpenFOAM using several files. All of these files reside in `constant/polyMesh/`. The names of these files are rather self explanatory, the rest is explained in the OpenFOAM User Guide [50].

boundary contains a list of all faces forming the boundary patches. This file is always written in ASCII format.

faces contains the definition of all faces. A face is defined by the points that form the face.

neighbour contains a list of the neighbouring cells of the faces

owner contains a list of the owning cells of the faces

points contains a list of the coordinates of all points

The description of a mesh is based on the faces. The geometry is discretised into finite volumes – the cells. Each cell is delimited by a number of faces, e.g. a hexahedron has 6 faces. The faces can be divided into two groups. Boundary faces border only one cell. These faces make up the boundary patches. All other faces can be seen as the connection between two cells and are called internal faces. A face bordering more than two cells is not possible. An internal face is, by definition, owned by one cell and neighboured by the other one. So, the two cells connected by a face can be destincted.

This five files are absolutely necessary to describe a mesh regardless of how the mesh was created in the first place. However, some ways of creating a mesh produce additional files. Listing 81 shows a list of all files created with Gambit and converted by *fluentMeshToFoam*.

```
user@host:~/OpenFOAM/user-2.1.x/run/twoPhaseEulerFoam/columnCase$ ls constant/polyMesh/  
boundary  cellZones  faces      faceZones  neighbour  owner      points     pointZones
```

Listing 81: Content of `constant/polyMesh`

13.1.2 Definitions

Face

A face is defined by the vertices or points that are part of the face. The points need to be stated in an order which is defined by the face normal vector pointing to the outside of the cell or the block. The way faces are defined is the same for cells of the mesh or for blocks of the geometry.

To elaborate this further we look at the top face of the generic block of Figure 7 in Figure 5. The vertices with the numbers 4, 5, 6 and 7 are part of the face. The face normal vector – denoted by **n** in Figure 5 – that points outwards of the block is parallel to the local *z* axis. Therefore we need to specify the vertices defining the face in counter-clockwise circular order, when we look at the block from the top. The direction of rotation is marked in Figure 5 with the + sign. The starting vertex is arbitrary but it must not appear twice in the list.

While the definition of the face normal of boundary faces is straight forwards, as discussed above, the face normal of internal faces is not as obvious. Internal faces connect two adjacent cells. While internal faces are created by OpenFOAM's meshing utilities, and no valid use-case for manually specifying internal faces comes to mind, the face-normal orientation of internal faces may be important when specifying sets of internal faces

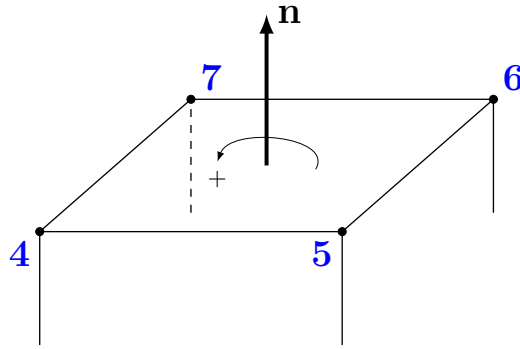


Figure 5: The top face of the generic block of Figure 7

Correct definitions			
(4 5 6 7)	(7 4 5 6)	(6 7 4 5)	(5 6 7 4)
Wrong direction of rotation			
(7 6 5 4)	(4 7 6 5)	(5 4 7 6)	(6 5 4 7)
Non-circular	Starting point repeated		
(7 5 6 4)	(4 5 6 7 4)		

Table 3: Valid and invalid face definitions

for run-time post-processing. The user-guide³³ states, that the orientation of the face-normal of internal faces is such, that the face normal points from the cell with the lower cell label to the cell with the higher cell label. As all cells are consecutively numbered from 0 to N , with N being the number of cells, the stated rule provides a clear rule for the face-normal direction of internal faces.

13.1.3 Some notes on practical use

Use the binary writing format or sufficient writePrecision

Whenever, there is no specific reason to use ascii format, one should always use the binary write format. As the `points` file contains a list of coordinates, we avoid losing information, i.e. geometric precision, by using the binary format. If you need to write in ascii, use a large setting for the `writePrecision`, at least for mesh generation and manipulation.

The binary format might not be compatible across various OpenFOAM variants

If we run `checkMesh` of foam-extend on a mesh which was written in binary format by OpenFOAM-6, then an error as in Listing 82 occurs. Thus, make sure to pass the mesh between OpenFOAM variants in ASCII format. See Section 27.6.4 for a discussion on how to convert mesh files from the binary to the ASCII format.

```
--> FOAM FATAL IO ERROR:
unexpected class name faceCompactList expected faceList
while reading object faces

file: /home/user/foam/run-4.0/meshTesting/constant/polyMesh/faces at line 15.

From function regIOobject::readStream(const word&)
in file db/regIOobject/regIOobjectRead.C at line 108.

FOAM exiting
```

Listing 82: A possible error when reading a binary mesh with a different OpenFOAM variant.

³³<https://cfd.direct/openfoam/user-guide/v6-mesh-description/>

13.2 General advice on meshing and mesh/case manipulation

13.2.1 Division of labour: keep mesh creation/manipulation separate from field definition

Since, OpenFOAM's toolbox contains lots of tool, which are specifically created for one job, OpenFOAM lends itself well for creating meshing cases, which are cases that run a sequence of meshing and mesh-manipulation tools to create the intended mesh. The most basic of such a mesh-creation & mesh-manipulation sequence would be to first run *blockMesh* and subsequently run *renumberMesh*.

Thus, we should aspire to organize our cases in a way to achieve a separation of separated tasks, i.e. creating the mesh, running the case and post-processing the case. Such a separation can be translated into an actual division of labour by creating sub-scripts for each task. Listing 83 shows a pattern for an *Allrun* script, which embodies this separation of tasks. Instead of calling tools and solvers directly, we call sub-scripts which we created for those specific tasks.

```
#!/bin/sh

# this script contains all steps for mesh-creation and mesh-manipulation
./createMesh

# now, that the mesh is in its final form, we instate the initial time step 0
cp -r 0.org 0

# beyond this point, calls to initialization tools, such as setFields,
# or case-decomposition for parallel running, are the only remaining pre-processing steps
./runCase

# run post-processing
./runPostProcessing
```

Listing 83: Division of labour in an exemplary *Allrun* script.

If we introduced our fields too early, then missing – yet to be created – patches may cause some pre-processing tools to fail. By eliminating the fields from the mesh-creation stage, we eliminate one source of error. Furthermore, dragging the fields through the mesh-generation process, with creation of patches, adding or removing cells, any many other non-trivial mesh-modifications, is a useless waste of computational effort, and a needless source of error. Thus, it is advised to follow this rough guideline:

1. Create the mesh, without involving the fields
2. Set up the fields for the final form of the mesh

14 Geometry creation & other pre-processing software

There are many ways to create a geometry. There is a great number of CAD software, there is a number of CFD pre-processors capable of creating geometries and there is the good old *blockMeshDict*.

This section is about the different ways to generate the geometry for creating a finite volume mesh.

14.1 *blockMesh*

blockMesh is one of OpenFOAM's own pre-processing tools. It is able to create the domain geometry and the corresponding mesh. See Section 15 for a discussion on *blockMesh*. For the reason of simplicity all aspects of *blockMesh* – geometry creation as well as meshing – are covered in Section 15.

14.2 CAD software

There is a great number of CAD software around. Each CAD program usually uses its own file format. However most CAD programs support exporting the geometry in different formats, e.g. STL, IGES, SAT. If CAD software is used to create the geometry the data has to be exported to be used by a meshing program. A common file format for this purpose is the STL format. *snappyHexMesh* can be used with STL³⁴ geometry definitions.

³⁴STL is infact a surface mesh enclosing the geometry. Therefore the term STL mesh or STL surface mesh is also valid.

14.2.1 OpenSCAD

OpenSCAD [<http://www.openscad.org/>] is an open source CAD tool for creating solid 3D CAD models. A CAD model is created by using primitive shapes (cubes, cylinders, etc.) or by extruding 2D paths. Models are not created interactively like in other CAD software. The user writes an input script which is interpreted by OpenSCAD. This makes it easy to create parametric models.

For further information on usage see the documentation http://en.wikibooks.org/wiki/OpenSCAD_User_Manual.

Pitfall: STL mesh quality

OpenSCAD is a tool to create CAD models. Therefore the requirements on the produced STL mesh are completely different than on a mesh for CFD simulations. OpenSCAD produces STL meshes that define the geometry correctly but the mesh is of a bad quality from a CFD point of view.

Figure 6 shows the STL mesh of a circular area. All triangles defining the circular area share one vertex. This vertex is probably the base point for the mesh creation of OpenSCAD. From a CFD point of view the triangular face elements are highly distorted and have a bad aspect ratio. However from a CAD point of view these triangles are perfectly sufficient to represent the circular area.

If a finite volume mesh is to be derived from the STL surface mesh (e.g. with GMSH) problems may arise. If the only purpose of the STL mesh is to represent some geometry – like it is the case with *snappyHexMesh* – then this quality issues can be ignored.

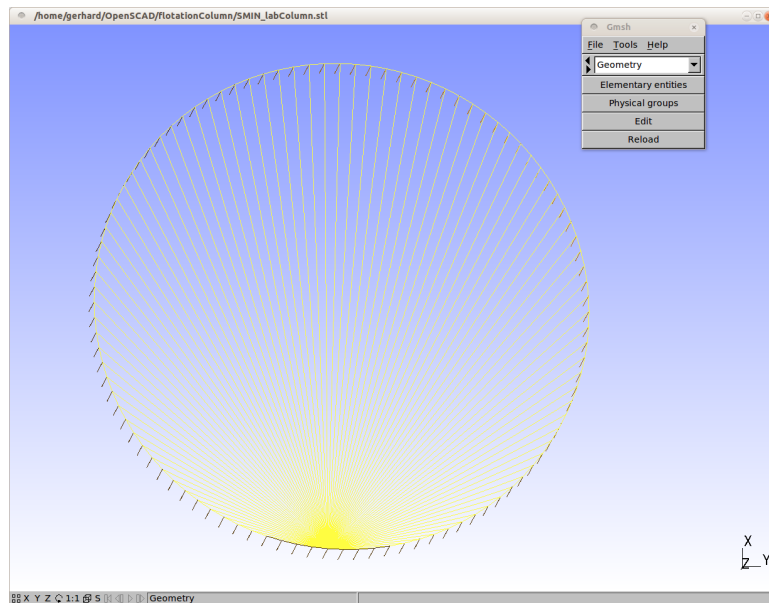


Figure 6: The STL mesh of a circular area generated by OpenSCAD

14.3 Salome

Salome [<http://www.salome-platform.org/>] is a powerful open source pre-processing software developed by EDF. Salome can be used to create a geometry interactively or by interpreting a python script³⁵. Salome comes with a number of internal and external meshing utilities. Salome has also a post-processing module.

Salome is a part of a collection of open source software developed by EDF. Salome serves as the pre- and post-processor for Code_Aster (structural analysis) and Code_Saturne (CFD).

14.3.1 Geometry

Salome can be used for geometry generation only. A common way of doing so, is to use Salome's meshing module to create a surface mesh of the CAD geometry, which can be exporting using the STL format. The resulting STL file can then be used by other meshing tools, e.g. *snappyHexMesh*.

³⁵Salome can be controlled completely by Python. Thus parametric geometry or mesh creation is possible.

14.3.2 Mesh

Salome can also be used to create the geometry and subsequently the mesh. This mesh needs to be exported by Salome in the UNV format, which can be converted by the *ideasUnvToFoam* utility of OpenFOAM.

See <http://caelinux.org/wiki/index.php/Doc:Salome> for documentation and usage examples of Salome, and Section 23 for some further points on creating the mesh with Salome.

14.4 GMSH

GMSH is a meshing tool with some pre- and post-processing capabilities [<http://www.geuz.org/gmsh/>]. The meshes generated by GMSH can be converted to OpenFOAM's format using the *gmshToFoam* utility.

15 *blockMesh*

blockMesh is used to create a mesh. The geometry is defined in *blockMeshDict*. This file also contains all necessary parameters needed to create the mesh, e.g. the number of cells. Therefore, *blockMesh* is a combined tool to define and mesh a geometry in contrast to other meshers that use CAD files to import a geometry created by some other software.

15.1 The block

The geometry created by *blockMesh* is based on the generic block. Figure 7 shows a generic block.

The blue numbers are the local vertex numbers of the block. The vertices are numbered counter-clockwise³⁶ in the local $x - y$ plane starting at the origin of the local coordinates³⁷. Then the vertices above the local $x - y$ plane are counter-clockwise numbered starting with the vertex on the local z axis.

The local vertex numbers are important when defining the block. The first part of the **blockMeshDict** is generally a list of vertices. From these vertices the blocks are constructed. A block is defined by a list of 8 vertices which have to be ordered in a way to match the local vertices. Therefore the first entry in the list of vertices is the local 0 vertex, then the local 1 vertex follows. The local vertex numbers define the order in which the vertices have to be passed when constructing a block.

The coordinate system originating from vertex 0 are the local coordinates. The local coordinates are important when specifying the number of cells or mesh grading (see *simpleGrading* in Section 15.4). The local coordinate axes do not need to be parallel or to coincide with the global coordinate axes.

The edges are also numbered and have a direction. Starting with the edge parallel to the local x axis the edges are numbered counter-clockwise starting with the edge emanating from the origin of the local coordinates. Next the edges parallel to the local y axis are numbered and finally the edges parallel to the local z axis. The edge number is important when specifying a grading for each edge individually (see *edgeGrading* in Section 15.4).

As it is indicated on Figure 7, the edges do not need to be parallel or straight. See Section 15.2.4 on how to define curved edges.

15.2 The blockMeshDict

The file **blockMeshDict** defines the geometry and controls the meshing process of *blockMesh*. Listing 84 shows a reduced example of the **blockMeshDict**. This file was taken from the *cavity* tutorial case.

```
/*-----*- C++ -*-----*/
| ===== |
| \ \ / F i e l d | OpenFOAM: The Open Source CFD Toolbox |
| \ \ / O p e r a t i o n | Version: 2.1.x |
| \ \ / A n d | Web: www.OpenFOAM.org |
| \ \ / M a n i p u l a t i o n |
|-----*/
```

³⁶In mathematics the positive direction of rotation is generally determined with the right-hand or cork-screw rule. Let the thumb of your right hand point in the positive direction of the rotation axis, then the fingers of the right hand point in the positive direction of revolution.

³⁷If we number all vertices in the $x - y$ plane then the local z axis is the axis of revolution. Thus the counter-clockwise direction is the mathematically positive direction of revolution.

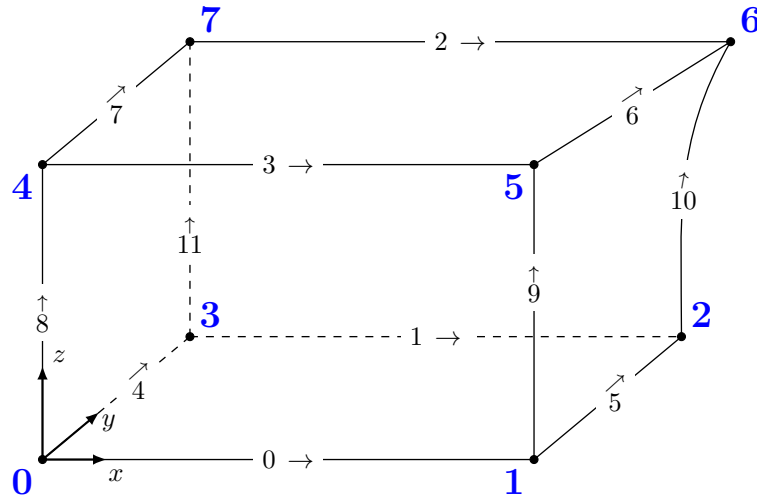


Figure 7: The generic block

```

FoamFile
{
    version      2.0;
    format       ascii;
    class        dictionary;
    object       blockMeshDict;
}
// * * * * *

convertToMeters 0.1;

vertices
(
    (0 0 0)      // 0
    (0 0 0.1)    // 1
    ...
);

blocks
(
    hex (0 1 2 3 4 5 6 7) (20 20 1) simpleGrading (1 1 1)
);

edges
(
);

boundary
(
    movingWall
    {
        type wall;
        faces
        (
            (3 7 6 2)
        );
    }
    ...
);

mergePatchPairs
(
);

// * * * * *

```

Listing 84: A minimal blockMeshDict

15.2.1 convertToMeters

`convertToMeters` is a scaling factor to convert the vertex coordinates of `blockMeshDict` into meters. If the vertex coordinates are entered in an other unit than meters, this value has to be chosen accordingly. Listing 85 shows how to set this factor if the vertex coordinates are entered in millimeters.

```
convertToMeters 0.001;
```

Listing 85: *convertToMeters*

If the keyword `convertToMeters` is missing in the `blockMeshDict`, then no scaling is used, i.e. the default value of 1 is assumed.

To make sure if a scaling factor has been used, the output of *blockMesh* can be checked. Listing 86 shows the message issued by *blockMesh* regarding the scaling factor defined with `convertToMeters`.

```
Creating points with scale 0.1
```

Listing 86: Output of *blockMesh* when `convertToMeters` is set to 0.1

`convertToMeters` is a uniform scaling factor. Non-uniform scaling or other operations can be performed with another tool. See Section 27.1 and 29.3.4.

15.2.2 vertices

The `vertices` sub-dictionary contains a list of vertices. Each vertex is defined by its coordinates in the global coordinate system. By default OpenFOAM treats these coordinates as in metres. However, with the help of the keyword `convertToMeters`, the vertices can be specified in other units.

The index of a vertex in this list is also the global number of this vertex, which is needed when constructing blocks from the vertices. Remember, counting starts from zero. Thus the first vertex in the list of vertices can be addressed by its index 0. A way to keep oneself aware of this fact is to add comments³⁸ to the vertex list as in Listing 84.

15.2.3 blocks

The only valid entry in the `blocks` sub-dictionary is the `hex` keyword. The `blocks` section of the `blockMeshDict` contains a list of `hex` commands. Listing 87 shows an example of a block definition with the `hex` keyword.

After the word `hex` a list of eight numbers defining the eight vertices of the block follows. The order of the entries in this list is the same order as the local vertex numbers of the block in Figure 7.

Then a list of three positive integer numbers follows. These numbers tell *blockMesh* how many cells need to be created in the direction of the local coordinate axes. Thus, the first number is the number of cells in the local *x* direction.

The next entry is a word stating the grading of the edges. This entry is in fact redundant. In OpenFOAM-2.1.x only the last entry, the list of expansion ratio, controls the grading. The third entry could even be omitted. However, maybe future versions of OpenFOAM make use of this entry. So the author does not advocate to omit this parameter.

The last entry of the block definition is a list of either three or twelve positive numbers. These numbers define the expansion ratio of the grading. In the case of three numbers, *simpleGrading* is applied. If twelve numbers are stated, then *edgeGrading* is performed.

If the list contains only one entry, then all edges share the same expansion ratio. Any other number of entries in this list leads to an error.

```
hex (0 1 2 3 4 5 6 7) (20 20 1) simpleGrading (2 4 1)
```

Listing 87: The `hex` command in `blockMeshDict`.

³⁸As OpenFOAM treats its dictionaries much in the same way as C/C++ source files are treated by the C/C++ compiler. Therefore comments work the same way as they do in C or C++.

Setting up cell zones

The cells belonging to a block can be assigned to a cell set at mesh creation by inserting the name of the to-be-created cell set between the vertex list and the list with the number of cells. This feature is not really documented in the official OpenFOAM User Guide, however, it seems to be present in OpenFOAM ever since.

```
hex (0 1 2 3 4 5 6 7) CELL_SET_NAME (20 20 1) simpleGrading (2 4 1)
```

Listing 88: The `hex` command in `blockMeshDict` with a cell set definition.

Creating a block with 6 faces

The `hex` instruction can also be used to create a prism with a triangular cross-section. Such blocks are needed for simulations that make use of axis-symmetry. In such a case, an “axis-patch” is created using only two vertices, as shown in Figure 8. This patch is constructed from the vertex list (4 5 5 4), instead of (4 5 6 7) as it would be the case for an ordinary block.

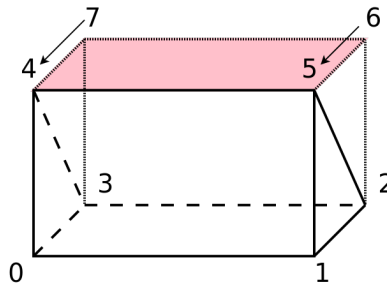


Figure 8: Creating a wedge geometry for 2D, axisymmetric domains. Reproduced after [50].

This collapsed patch results in an empty patch, i.e. one that contains no faces. In Listing 89, we see the summary on the patches, which is printed to the Terminal by `blockMesh`. Here, we can see the empty patch `axis`.

```
Patches
-----
patch 0 (start: 10159 size: 5) name: inletCH4
patch 1 (start: 10164 size: 70) name: wallOutside
patch 2 (start: 10234 size: 61) name: wallTube
patch 3 (start: 10295 size: 5) name: inletPilot
patch 4 (start: 10300 size: 60) name: inletAir
patch 5 (start: 10360 size: 71) name: outlet
patch 6 (start: 10431 size: 0) name: axis
patch 7 (start: 10431 size: 5170) name: frontAndBack_pos
patch 8 (start: 15601 size: 5170) name: frontAndBack_neg
```

Listing 89: The patch list of an axisymmetric case from the OpenFOAM tutorials.

When setting up a two-dimensional case, pay attention to velocity boundary conditions, especially those with a user-provided flow-rate. OpenFOAM is not aware, that the inlet patch is only a fraction of the real inlet. Thus, a flow rate at the inlet needs to be corrected by the area ratio, i.e. the ratio of the inlet patch of the two-dimensional simulation domain and the real inlet area.

Pitfall: writePrecision and wedge geometry

When we create and manipulate an axisymmetric geometry, we have two patches of the type `wedge`. If we use an insufficient number of digits to define the geometry, then OpenFOAM may issue warning messages similar to the one below, informing the us, the user, of non-planar faces.

```
--> FOAM Warning :
      From function virtual void Foam::wedgePolyPatch::calcGeometry(Foam::PstreamBuffers&)
```

```

in file meshes/polyMesh/polyPatches/constraint/wedge/wedgePolyPatch.C at line 70
Wedge patch 'front' is not planar. At local face at (4.07936 -0.000130603 0.0149657) the
normal (0 -0.999962 -0.00872414) differs from the average normal (8.95761e-21 -0.999962
-0.00872653) by 5.70802e-12
Either correct the patch or split it into planar parts

```

Listing 90: Insufficient *writePrecision* in *controlDict* when creating/using a 2D axisymmetric geometry.

If we create a single, wedge block, we specify only six points in space. All the intermediate points are computed and written by *blockMesh*. If in such a case, the *writePrecision* setting in the *controlDict* is insufficient, i.e. a too small number of digits is used to write the points file. *blockMesh*, by default³⁹, writes with at least 10 digits.

```

1 // Set the precision of the points data to 10
2 Iostream::defaultPrecision(max(10u, Iostream::defaultPrecision()));

```

Listing 91: Using at least 10 digits for writing the *points* file; excerpt from the file *blockMesh.C* of OpenFOAM-5.0

However, some mesh manipulation tools, simply use the setting from the *writePrecision* setting in the *controlDict*. If a too small number of digits is specified in combination with writing in ascii format, information (geometric precision) may be lost. This lost precision may be sufficient to trigger the warning messages, which were discussed above.

OpenFOAM is, at times, quite generous when it comes to the information provided in its error messages. Below, the example warning message is shown again. This time, the relevant bit of information is highlighted in red. This very small difference between the local face-normal vector and the patch-average normal-vector is indicative of an issue regarding writing precision.

```

--> FOAM Warning :
    From function virtual void Foam::wedgePolyPatch::calcGeometry(Foam::PstreamBuffers&)
in file meshes/polyMesh/polyPatches/constraint/wedge/wedgePolyPatch.C at line 70
Wedge patch 'front' is not planar. At local face at (4.07936 -0.000130603 0.0149657) the
normal (0 -0.999962 -0.00872414) differs from the average normal (8.95761e-21 -0.999962
-0.00872653) by 5.70802e-12
Either correct the patch or split it into planar parts

```

Listing 92: Insufficient *writePrecision* in *controlDict* when creating/using a 2D axisymmetric geometry.

This indication could also be gleaned from a comparison of the two listed vectors, the affected local face-normal and the patch-average normal-vector. However, the highlighted number, which is the magnitude of the difference, is much easier to read and comprehend.

Pitfall: *writePrecision* and wedge geometry II - inline code

A very similar problem was observed, when creating an axis-symmetric domain by using some inline code to compute vertex coordinates. Apparently, the *writePrecision* setting in the *controlDict* affects the precision of the calculated vertex coordinates. Thus, if the write precision is too low, *blockMesh* will warn about non-planar wedge patches. However, this can easily be remedied by using a write precision of 10 digits or above. Note, that writing in binary format does not alleviate this problem.

15.2.4 edges

The *edges* sub-dictionary contains pairs of vertices that define an edge. By default edges are straight, by explicitly specifying the shape of the edge, curved edges can be created. This sub-dictionary can be omitted. Listing 93 shows the message issued by *blockMesh* when *edges* is omitted.

```

No non-linear edges defined

```

Listing 93: Output of *blockMesh* when *edges* is omitted

³⁹At least in OpenFOAM-5.0. This might vary among the various version of OpenFOAM. Other versions have not been checked. This behaviour might have been introduced sometime in the past, or it have been there from the beginning.

Otherwise, *blockMesh* issues a message as in Listing 94 regardless whether curved edges are actually created or only an empty **edges** sub-dictionary is present.

Creating curved edges

Listing 94: Output of *blockMesh* when **edges** is present

Creating arcs

With the keyword **arc** a circular arc between two vertices can be created. Listing 95 shows the definition of a circular arc between the vertices 0 and 3. In order to define a circular arc three points are necessary. Therefore the third point follows the indices of the two vertices defining the edge.

```
edges
(
    arc 0 3 (0 0.5 0.05)
);
```

Listing 95: Definition of a circular edges in the **edges** sub-dictionary

The keyword **arc** can not be used to define a straight edge. If the two vertices and the additional interpolation point are co-linear, *blockMesh* will abort issuing an error message as in Listing 96.

```
--> FOAM FATAL ERROR:
Invalid arc definition - are the points co-linear?  Denom =0

    From function cylindricalCS arcEdge::calcAngle()
    in file curvedEdges/arcEdge.C at line 55.

FOAM aborting
```

Listing 96: Output of *blockMesh* when the three points defining an arc are co-linear

Creating splines

The keyword **spline** defines a spline. After the two vertices defining the edge a list of interpolation points has to follow.

```
edges
(
    spline 0 3 ((0 0.25 0.05) (0 0.75 0.05))
);
```

Listing 97: Definition of a spline in the **edges** sub-dictionary

Creating a poly-line

Other than a spline, a poly-line connects several points with straight lines.

```
edges
(
    polyLine 0 3 ((0 0.25 0.05) (0 0.75 0.05))
);
```

Listing 98: Definition of a poly-line in the **edges** sub-dictionary

Creating a straight line

For the sake of completeness there is the keyword `line`. This keyword takes the two vertices defining the edge as arguments. Straight lines are created by *blockMesh* by default. So there is no need for the user to specify straight lines.

```
edges
(
    line 0 3
);
```

Listing 99: Definition of a line in the `edges` sub-dictionary

Summary

Edges defined within the `blockMeshDict` are used to compute the locations of a block's internal nodes. The edge however, is approximated linearly as shown in Figure 9, i.e. the number of cells along the edge determine the resolution of the edges.

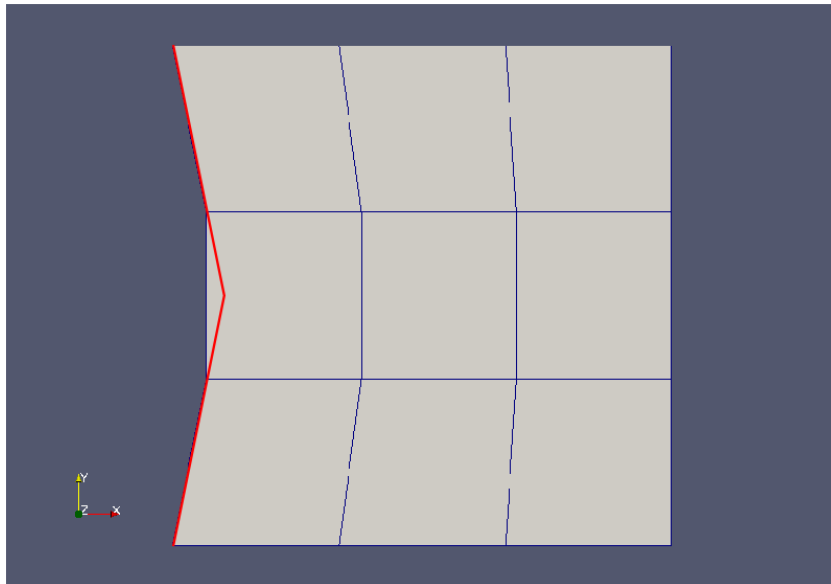


Figure 9: A block with a poly-line at the left side. The red line indicates the poly-line. This figure makes it obvious that edges defines in the `blockMeshDict` serve to compute the locations of the block's internal nodes. The block itself however, does not obey the poly-line.

Another feature of the edge definition is, that the two vertices defining the edge can be supplied in any order.

Pitfalls

Edge creation of *blockMesh* sometimes fails silently⁴⁰. If we define an arc between two vertices not directly connected by an edge, e.g. the vertices 0 and 2 in Figure 7, then *blockMesh* proceeds without any warning or error. The faulty edge definition seems to be simply ignored. This, silence in the face of error might make it hard for the user building his or her `blockMeshDict` to spot the reason why the definition of curved edges does not result in curved edges.

⁴⁰An example non *blockMesh* failing noisily is the definition of a co-linear interpolation point for an arc.

15.2.5 boundary

The `boundary` list contains a dictionary per patch. This dictionary contains the type of the patch and the list of faces composing the patch. Listing 100 shows an example of how a patch consisting of one face is defined.

```
boundary
(
    inlet
    {
        type patch;
        faces
        (
            (0 3 2 1)
        );
    }
    ...
);
```

Listing 100: The `boundary` list of `blockMeshDict`

Pitfall: `defaultFaces`

If faces are forgotten in the boundary definition, then *blockMesh* creates an additional patch named `defaultFaces`. This patch has an `empty` boundary condition automatically assigned. Listing 101 shows a warning message issued by *blockMesh*. In this case some faces were missing in the boundary definition. This, however, does not cause *blockMesh* to abort mesh generation. If a 2D mesh is to be created, the creation of the default patch with an `empty` boundary condition can be expected behaviour. However, it is not advisable to rely this kind of default behaviour when building a case.

```
Creating block mesh topology --> FOAM Warning :
  From function polyMesh::polyMesh(...) construct from shapes...
  in file meshes/polyMesh/polyMeshFromShapeMesh.C at line 903
  Found 6 undefined faces in mesh; adding to default patch.
```

Listing 101: A warning message of *blockMesh* caused by an incomplete boundary definition.

If faces are forgotten in the creation of a 3D mesh, this behaviour might hide the source of error. *blockMesh* quietly creates the mesh with the default patch – save the warning message as in Listing 101. Running the case with the erroneous mesh definition will not immediately crash the solver. Even the fact that none of the fields have a boundary condition specified for the default patch does not cause the solver to abort. A patch with an `empty` boundary condition does not require any further entries in the field-files (e.g. `U` or `p`). OpenFOAM knows already all it needs to know about this specific patch and there is no reason to throw an error message. When the case is run with a 3D mesh and one or more `empty` patches, the solver starts running without complaints. At some point the solution might run into numerical trouble.

Only running *checkMesh* is able to give an indication to detect such kind of error. Listing 102 shows the warning message issued by *checkMesh* when a 3D mesh contains one `empty` default patch. Although, the warning states that there is something wrong with the mesh, in the end *checkMesh* reports no failed mesh checks.

```
Checking topology...
  Boundary definition OK.
  ***Total number of faces on empty patches is not divisible by the number of cells in the mesh
  . Hence this mesh is not 1D or 2D.
```

Listing 102: A warning message of *checkMesh* caused by an incomplete boundary definition of a 3D mesh.

Patch groups

Patches can be grouped to save ourselves the hassle to prescript large numbers of identical boundary conditions. Patch groups were introduced with OpenFOAM-2.2.0 see <http://openfoam.org/release/2-2-0/pre-processing-macros-patch-groups/>. All boundaries of the constraint type, e.g. `empty` or `processor`,

are automatically added to patch groups of the same name. Furthermore, since OpenFOAM-2.3.0⁴¹, the patches of the type `wall` are added to a group named `wall`. Also, with OpenFOAM-2.3.0 the order of precedence for defining boundary conditions for fields was defined:

1. An exact match of the patch name, e.g. `inlet`
2. A match by a patchGroup
3. A match by regular expression, e.g. `"wallPatch.*"` for `wallPatch0815`

```
2
(
wall 4(0 1 2 3)
empty 1(4)
)
```

Listing 103: The automatically defined patch groups of the *cavity* tutorial of *icoFoam*. The list was created with the method `groupPatchIDs()` of the `Foam::polyBoundaryMesh` and printed to Terminal with the `Info` statement.

Pitfall: multiple patch group membership

If patches are members of more than two groups, and the boundary conditions are specified via group membership, then the actual boundary condition that get applied is kind of undetermined. Some tests done by the author suggest that the last patch group entry in the field file prevails.

To demonstrate the issue, the *cavity* tutorial was slightly modified. The three patches representing the fixed walls, are members of the patch group `wall` by default and are members of the patch group `banana`, see Listing 104 below.

```
3
(
wall 4(0 1 2 3)
banana 3(1 2 3)
empty 1(4)
)
```

Listing 104: The patch groups of the modified *cavity* tutorial of *icoFoam*.

The velocity field BC definition was changed to employ patch groups.

```
boundaryField
{
    movingWall
    {
        type            fixedValue;
        value            uniform (1 0 0);
    }
    wall
    {
        type            noSlip;
    }
    banana
    {
        type            fixedValue;
        value            uniform (-1 0 0);
    }
}
```

Listing 105: The velocity field boundary conditions of the modified *cavity* tutorial. The entry for `frontAndBack` is omitted for brevity. Only `movingWall` is specified by exact patch name. The fixed walls are specified via patch groups.

⁴¹<http://openfoam.org/release/2-3-0/pre-processing/>

The resulting initial velocity field depends on the order of the entries for **wall** and **banana**.

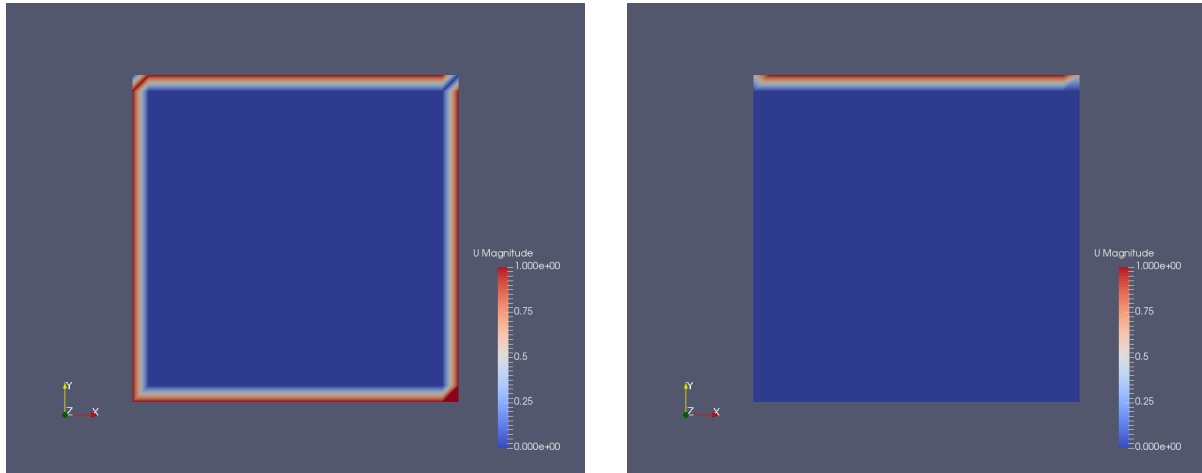


Figure 10: The initial velocity field depending on the order of the **wall** and **banana**. **Left:** Setting as in Listing 105. **Right:** **wall** and **banana** have changed places.

Thus, users are suggested to avoid situations involving multiple group membership when specifying boundary conditions via patch groups.

Pitfall: identical names of patches and patchGroups

When patches have the same name as a patchGroup, OpenFOAM may issue a warning or exit with an error. Up to, and including, OpenFOAM-4.0 a warning message was issued, as in Listing 106. In later versions, OpenFOAM aborts with an error, as in Listing 107.

```
--> FOAM Warning :
      From function const Foam::HashTable<Foam::List<int>, Foam::word>& Foam::polyBoundaryMesh::
      groupPatchIDs() const
      in file meshes/polyMesh/polyBoundaryMesh/polyBoundaryMesh.C at line 448
      Patch fixedWall01 specifies a group banana which is also a patch name. This might give
      problems later on.
```

Listing 106: OpenFOAM is warning about identical names of a patch and a patchGroup.

```
--> FOAM FATAL ERROR:
Patch 'fixedWall01' specifies the group 'banana' which clashes with a patch name.
Please choose patch names which are not patch type/group names.

      From function const Foam::HashTable<Foam::List<int>, Foam::word>& Foam::polyBoundaryMesh::
      groupPatchIDs() const
      in file meshes/polyMesh/polyBoundaryMesh/polyBoundaryMesh.C at line 448.

FOAM exiting
```

Listing 107: Identical names for patches and patchGroups are not allowed anymore.

Pitfall: patches

In older versions of OpenFOAM, there was a **patches** sub-dictionary instead of the **boundary** sub-dictionary, see <http://www.openfoam.org/version2.0.0/meshing.php>. In some tutorial cases the old **patches** sub-dictionary can be found. However, it is recommended to use the **boundary** sub-dictionary because in some cases the use of the **patches** sub-dictionary results in errors.

To find out if there are still tutorial cases present that use the **patches** sub-dictionary the command of Listing 108 searches all files with the name **blockMeshDict** in the tutorials for the word **patches**.

```
find $FOAM_TUTORIALS -name blockMeshDict | xargs grep patches
```

Listing 108: Find cases that still use the `patches` sub-dictionary in the `blockMeshDict` to define the boundaries

15.2.6 mergePatchPairs

The `mergePatchPairs` list contains pairs of patches that need to be connected by the mesher.

Nothing to merge

This entry can be omitted. Listing 109 shows the message issued by *blockMesh* when `mergePatchPairs` is omitted.

```
There are no merge patch pairs edges
```

Listing 109: Output of *blockMesh* when `mergePatchPairs` is omitted

Patches to merge

When two patches need to be merged, then the patch pair needs to be stated in the `mergePatchPairs` list. The first patch of the pair is considered the master patch the second is the slave patch. The reason and consequences of this are described in the official User Manual [50].

```
mergePatchPairs
(
    (master slave)
);
```

Listing 110: The `mergePatchPairs` list in the `blockMeshDict`

If the patches that are part of the merging operation contain faces which are unaffected by the merging, the merge operation will fail. When the blocks of Figure 14 are to be connected, then the patch pair consists only of the face (1 2 6 5) and (12 15 11 8). If one of the two patches contains an additional face, *blockMesh* will crash with an error. Thus the patches need to be defined as in Listing 111.

```
boundary
(
    master
    {
        type patch;
        faces
        (
            (1 2 6 5)
        );
    }
    slave
    {
        type patch;
        faces
        (
            (12 15 11 8)
        );
    }
    ...
);
```

Listing 111: The patch definitions needed to connect the blocks of Figure 14 with `mergePatchPairs` in the `boundary` sub-dictionary

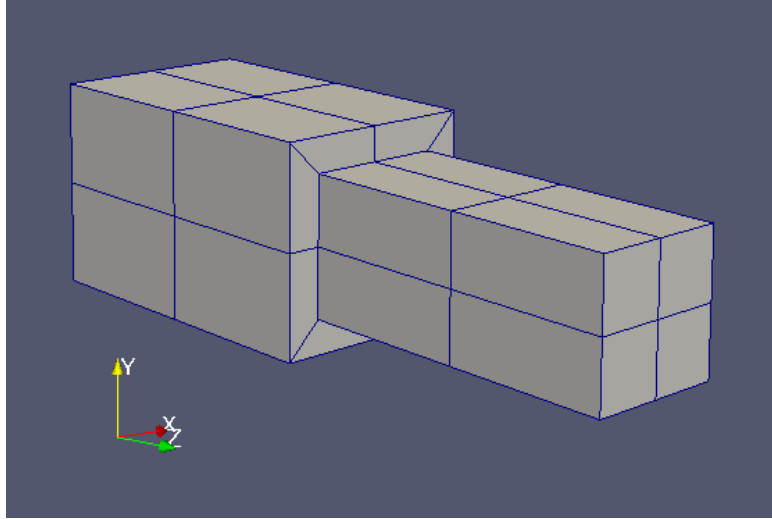


Figure 11: The mesh of two merged blocks

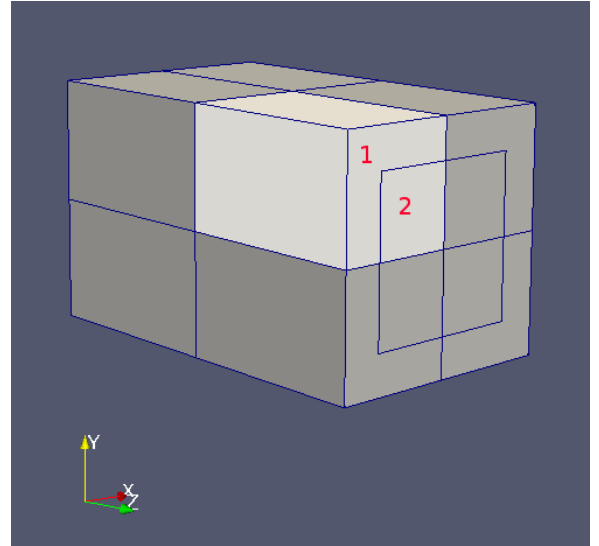
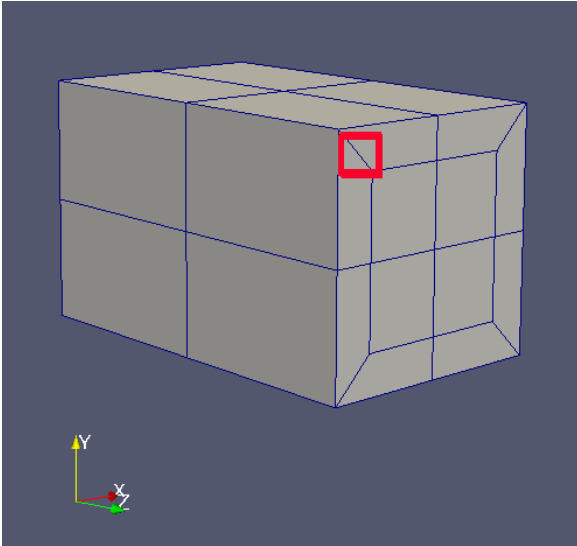


Figure 12: The mesh of two merged blocks. Left: screenshot of ParaView. Right: edited image to depict the actual faces.

blockMesh creates hanging nodes in order to connect the mesh of the blocks. Figure 11 shows the mesh of two merged blocks. Figure 12 shows the larger of the two blocks. The diagonal lines – one of them is marked with a red square in Figure 12 – are artefacts of the depiction of ParaView. The diagonal line that divides the L-shaped area is not present in the mesh. The right image in Figure 12 was edited with an image manipulation program to reflect the actual situation of the mesh. During the merging operation the face touching the second block is divided to match the second block. Thus, a quadrangular cell face is divided to two faces. The face denoted with the red 1 consists of 6 nodes and the face with the red 2 consists of four nodes.

15.3 Create multiple blocks

A single block is almost never sufficient to model the geometry of a CFD problem. *blockMesh* offers the possibility to create an arbitrary number of blocks which can be connected. If blocks are constructed in a fashion that they share vertices, then they are connected by *blockMesh* by default.

15.3.1 Connected blocks

Figure 13 shows two connected blocks. These blocks share vertices. Therefore, the blocks are connected automatically.

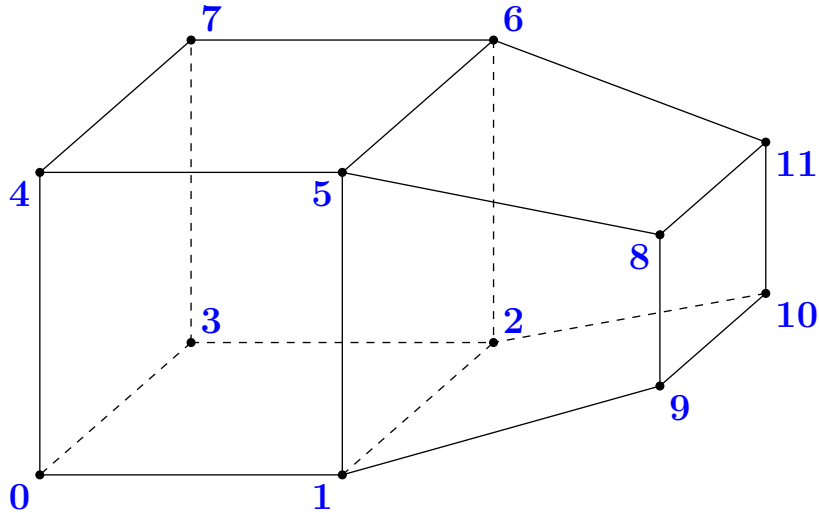


Figure 13: Two connected blocks

Listing 112 shows the **blocks** sub-dictionary to create two connected blocks as they are depicted in Figure 13. The global vertex numbering is arbitrary. However, the order in which the vertex numbers are listed after the **hex** keyword corresponds with the local vertex numbering of the generic block in Figure 7.

```
blocks
(
    hex (0 1 2 3 4 5 6 7) (10 10 10) simpleGrading (1 1 1)
    hex (1 9 10 2 5 8 11 6) (10 10 10) simpleGrading (1 1 1)
);
```

Listing 112: The **blocks** entries in **blockMeshDict** to create the connected blocks of Figure 13

15.3.2 Unconnected blocks

Figure 14 shows a situation in which two blocks were created that share no vertices. Creating multiple blocks is done simply by adding a further entry in the **blocks** list. The blocks are connected by the statements in the **mergePatchPairs** section of the **blockMeshDict**.

Listing 113 shows the **blocks** sub-dictionary to create two unconnected blocks as they are depicted in Figure 14.

```
blocks
(
    hex (0 1 2 3 4 5 6 7) (10 10 10) simpleGrading (1 1 1)
    hex (8 9 10 11 12 13 14 15) (10 10 10) simpleGrading (1 1 1)
);
```

Listing 113: The **blocks** entries in **blockMeshDict** to create the unconnected blocks of Figure 14

In order to generate a connected mesh of the two blocks, the **mergePatchPairs** section of the **blockMeshDict** has to be provided with the two touching patches.

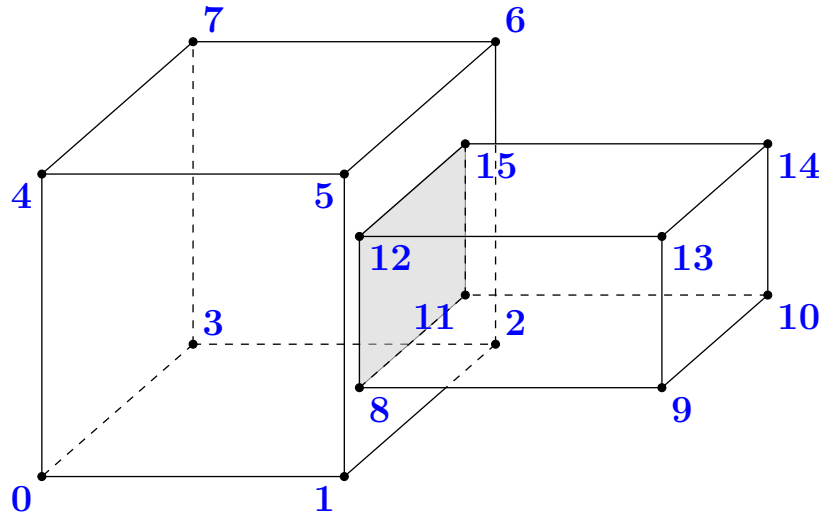


Figure 14: Two unconnected blocks

15.4 Grading

In the file `blockMeshDict` the grading can be defined globally for the edges of the block or for all edges individually. The grading is specified by the expansion ratio. This is the ratio of the widths of the first and the last cell along an edge. The direction of an edge is defined in the general definition of a block (see OpenFOAM Users Manual [50]).

simpleGrading

The global grading is defined for all edges parallel to the local x , y and z direction of the block. In Listing 114 the grading of all edges parallel to the local x axis of the block is one, the grading of all edges parallel to the local y axis is two and the grading of all edges parallel to the local z axis is three.

```
simpleGrading (1 2 3)
```

Listing 114: *simpleGrading*

edgeGrading

With the keyword `edgeGrading` the grading of each edge of the block is specified individually. Therefore, the value of this keyword is a list with 12 numbers. The numbering of the edges – the list index corresponds to the edge number – is defined in the general definition of a block (see OpenFOAM Users Manual [50]). Listing 115 has the same effect as Listing 114.

```
edgeGrading (1 1 1 1 2 2 2 2 3 3 3 3)
```

Listing 115: *edgeGrading*

Pitfall: inconsistent grading

When a mesh consists of more than one block, then the grading of coincident edges must be consistent, i.e. these edges must have the same grading. In Listing 116 the grading of the last block is erroneous – the grading is set to 2 instead of 3. The error message caused by this fault is shown in Listing 117. The message mentions the blocks 5 and 8. This is correct, because OpenFOAM counts – like C, C++ and many more programming languages – from 0. Therefore, block 8 is the ninth block.

```

blocks
(
  hex (0 16 20 4 1 17 21 5) (30 5 10) simpleGrading (1 0.5 0.33) // 1
  hex (1 17 21 5 2 18 22 6) (30 5 2) simpleGrading (1 0.5 1) // 2
  hex (2 18 22 6 3 19 23 7) (30 5 15) simpleGrading (1 0.5 3) // 3

  hex (4 20 24 8 5 21 25 9) (30 2 10) simpleGrading (1 1 0.33) // 4
  hex (5 21 25 9 6 22 26 10) (30 2 2) simpleGrading (1 1 1) // 5
  hex (6 22 26 10 7 23 27 11) (30 2 15) simpleGrading (1 1 3) // 6

  hex (8 24 28 12 9 25 29 13) (30 5 10) simpleGrading (1 2 0.33) // 7
  hex (9 25 29 13 10 26 30 14) (30 5 2) simpleGrading (1 2 1) // 8
  hex (10 26 30 14 11 27 31 15) (30 5 15) simpleGrading (1 2 2) // 9
);

```

Listing 116: Inconsistent grading

```

--> FOAM FATAL ERROR:
Inconsistent point locations between block pair 5 and 8
probably due to inconsistent grading.

From function blockMesh::calcMergeInfo()
in file blockMesh/blockMeshMerge.C at line 294.

FOAM exiting

```

Listing 117: Error message caused by inconsistent grading

Pitfall: inconsistent discretisation

When a mesh consists of more than one block, then the number of cells of neighbouring blocks must be consistent, i.e. the blocks must have the same number of cells along coincident axes. In Listing 118 the number of cells of the first block is erroneous – the number is set to 44 instead of 45 along the local z direction. The error message caused by this faulty definition is shown in Listing 119. The message mentions the blocks 0 and 1. This error message indicates more clearly – other than Listing 117 – that OpenFOAM counts from 0.

```

blocks
(
  hex (0 1 5 4 8 9 13 12 ) (9 1 44) simpleGrading (1 1 1) // 1
  hex (1 2 6 5 9 10 14 13 ) (2 1 45) simpleGrading (1 1 1) // 2
  hex (2 3 7 6 10 11 15 14 ) (9 1 45) simpleGrading (1 1 1) // 3
);

```

Listing 118: Inconsistent discretisation

```

---> FOAM FATAL ERROR:
Inconsistent number of faces between block pair 0 and 1

From function blockMesh::calcMergeInfo()
in file blockMesh/blockMeshMerge.C at line 221.

FOAM exiting

```

Listing 119: Error message caused by inconsistent discretisation

Interesting observation

The source code also allows to state a list with only one entry. This is not documented in the official User Manual [50].

Listing 120 proves this observation in the form of the responsible source code. The first command reads a scalar list from the input stream `is`. Then the three valid cases – one, three or twelve entries – are handled. If none of the three branches of the `if-else` branching is entered an error is reported.

This code listing is a beautiful example of deducting the behaviour of a program from its source code. Unfortunately not all parts of OpenFOAM's source code are that easy to read and understand.

```

1  scalarList expRatios(is)
2
3  if (expRatios.size() == 1)
4  {
5      // identical in x/y/z-directions
6      expand_ = expRatios[0];
7  }
8  else if (expRatios.size() == 3)
9  {
10     // x-direction
11     expand_[0] = expRatios[0];
12     expand_[1] = expRatios[0];
13     expand_[2] = expRatios[0];
14     expand_[3] = expRatios[0];
15
16     // y-direction
17     expand_[4] = expRatios[1];
18     expand_[5] = expRatios[1];
19     expand_[6] = expRatios[1];
20     expand_[7] = expRatios[1];
21
22     // z-direction
23     expand_[8] = expRatios[2];
24     expand_[9] = expRatios[2];
25     expand_[10] = expRatios[2];
26     expand_[11] = expRatios[2];
27 }
28 else if (expRatios.size() == 12)
29 {
30     expand_ = expRatios;
31 }
32 else
33 {
34     FatalErrorIn
35     (
36         "blockDescriptor::blockDescriptor"
37         "(const pointField&, const curvedEdgeList&, Istream&)"
38     ) << "Unknown definition of expansion ratios: " << expRatios
39     << exit(FatalError);
40 }

```

Listing 120: Some content of `blockDescriptor.C`

Pitfall: grading on a wedge geometry

Axisymmetric simulation domains are created in the form of a wedge geometry. In this case, the axis patch is constructed from two vertices, which causes the axis patch to collapse into a straight line. Internally, OpenFOAM knows what to do in such a case.

However, it was observed when applying some grading along the axis of the domain, that not all faces of the axis patch are collapsed, resulting in a non-empty axis patch.

In Listing 121 the relevant part of the output of `blockMesh` is shown. In this case, some grading, in the direction of the symmetry axis, was used. In that specific case, there were nearly 1500 cells in axis-direction and only 6 faces in the axis patch. This discrepancy is highly suspicious, and it was assumed that during grading the collapsing of faces on the axis patch somehow failed for some points, resulting in 6 triangular faces.

```

Patches
-----
patch 0 (start: 118329 size: 14) name: inlet
patch 1 (start: 118343 size: 7) name: outlet
patch 2 (start: 118350 size: 1577) name: walls
patch 3 (start: 119927 size: 6) name: axis
patch 4 (start: 119933 size: 59938) name: front
patch 5 (start: 179871 size: 59938) name: back

```

Listing 121: The patch list of an axisymmetric case with a non-empty axis patch.

Having a non-empty axis patch poses no problem for `blockMesh` during mesh-creation, however, all other tools and solvers fail to construct a proper mesh. Listing 122 shows an example of the error message issued by OpenFOAM. Unfortunately, this error message is relatively inexpressive.

```
Create polyMesh for time = constant

#0 Foam::error::printStack(Foam::Ostream&) at ???
#1 Foam::sigFpe::sigHandler(int) at ???
#2 ? in "/lib/x86_64-linux-gnu/libc.so.6"
#3 Foam::polyMesh::calcDirections() const at ???
#4 Foam::polyMesh::polyMesh(Foam::IObject const&) at ???
#5 ? at ???
#6 __libc_start_main in "/lib/x86_64-linux-gnu/libc.so.6"
#7 ? at ???
Floating point exception (core dumped)
```

Listing 122: The result of having an axisymmetric case with a non-empty axis patch.

This problem was overcome by manually changing the patch type in `constant/polyMesh/boundary` of the axis patch from `empty` to `patch`, running `collapseEdges`, and finally manually changing the patch type back to `empty`. The manual changing of the patch type is necessary for `collapseEdges` to be able to read and construct the mesh. In Listing 123 we see the output of `checkMesh` after the solution procedure was applied to that example case. Now, the axis patch is empty, i.e. it contains no faces. Whether the patch is of the type `empty` or of the type `patch`, is not printed by `checkMesh`.

```
Checking patch topology for multiply connected surfaces...
Patch      Faces    Points    Surface topology
inlet      14         29        ok (non-closed singly connected)
outlet      7         15        ok (non-closed singly connected)
walls     1577       3156      ok (non-closed singly connected)
axis        0          0        ok (empty)
front     59938     61486     ok (non-closed singly connected)
back     59938     61486     ok (non-closed singly connected)
```

Listing 123: The patch list of an axisymmetric case after fixing the problem with the non-empty axis patch.

Simply, increasing the `writePrecision` or switching to writing in binary format did not resolve the problem.

15.5 Parametric meshes by the help of *m4* and *blockMesh*

In `blockMeshDict` only plain text is allowed, i.e. no symbols can be used. Also, no calculations can be made by *blockMesh* with the exception of the keyword `convertToMeters`.

15.5.1 The `blockMeshDict` prototype

If the user wants to create parametrised meshes, i.e. properties of the mesh are calculated from certain parameters, an additional working step is necessary. In order to create a parametric mesh a prototype of the file `blockMeshDict` is needed. This prototype contains symbols. Listing 124 shows the block definition of such a prototype. This block definition is not fully parametric, only the number of cells is calculated. Note, that in local *y* direction only one cell is used for discretisation. This indicates a 2D problem.

```
blocks
(
    hex (0 1 5 4 8 9 13 12) (N1x 1 N1z) simpleGrading (1 1 1) // 1
    hex (1 2 6 5 9 10 14 13) (N2x 1 N1z) simpleGrading (1 1 1) // 2
    hex (2 3 7 6 10 11 15 14) (N1x 1 N1z) simpleGrading (1 1 1) // 3
);
```

Listing 124: Block definition of the prototype

15.5.2 The macro programming language *m4*

In order to replace the symbols of the prototype with meaningful numbers, the prototype has to be processed by a macro programming language interpreter. In this case the programming language *m4*⁴² is used. The interpreter of this language scans the prototype for valid expressions (macros) and replaces them with their result.

To replace a symbol of the prototype with a meaningful number, a macro has to be defined. Listing 125 shows the definition of the symbols used in Listing 124. In the first line a general variable *h* is defined. The second and the third instruction calculate the number of cells in the local *x* direction based on the variable *h*. The last instruction calculates the number of cells in the local *z* direction.

```
define(h,2)

define(N1x,'eval(9*h)')
define(N2x,'eval(2*h)')

define(N1z,'eval(45*h)')
```

Listing 125: Block definition of the prototype

This kind of parametrisation allows to specify a multiplier for the number of cells. The discretisation length can not be refined gradually this way. Specifying the discretisation length requires more complex math than integer operations.

Complex math - first shot

The builtin mathematic macros of *m4* are restricted to integer operations only. As *m4* supports system calls, floating point calculations can be done by an external program. Consequently, the symbol is replaced by the result of the system call.

In Listing 126 some variables are defined. In line 13 a macro is defined that passes its arguments to the operating system via a system call. The argument of the command *esyscmd* gets executed in the command line. This is the reason for the rather complicated argument of *esyscmd*. The output of the command *echo* is the input of the command *bc*⁴³. Note the use of the pipe.

The input of the command *echo* is composed of three successive operations that need to be performed by the calculator. The first instruction says that two digits after the decimal point should be used. The second instruction calculates the difference between the first two arguments and the last instruction divides this difference by the third argument. These operations compute first the length of the block that needs to be descretised. Then by dividing this length by the discretisation length the number of cells is calculated.

The output is then formatted by the macro *format*. Note the formatting string *%.0f*. This causes the result to loose its digits after the decimal point. This step is absolutely necessary, because only integers are allowed to define the number of cells.

```
1 // # enter discretization length
2 define(dx,0.005)
3 define(dz,0.005)
4
5 // # enter x coordinates
6 define(x1,0.0555)
7 define(x2,0.0945)
8
9 // # enter heights (z coordinates)
10 define(H1, 0.20)
11
12 // # relDiff: ($1 - $2) / $3 # decimal places truncated (done by format %.0f)
13 define(relDiff,'format('%.0f', esyscmd(echo "scale=2; a=$1-$2; a/$3" | bc))')
14
15 define(N1x,'relDiff(x1,0,dx)')
16 define(N2x,'relDiff(x2,x1,dx)')
17
18 define(N1z,'relDiff(H1,0,dz)')
```

⁴²*m4* is part of the GNU project. See <http://www.gnu.org/software/m4/manual/index.html>

⁴³*bc* is a calculator program. It is part of the GNU project.

Listing 126: Block definition of the prototype

Listing 126 allows to calculate the number of cells from a specified discretisation length. Due to rounding operations the specified discretisation length is not exactly met. Listing 127 shows the result after the macros from Listings 124 and 126 have been processed.

```
blocks
(
  hex (0 1 5 4 8 9 13 12 ) (11 1 40) simpleGrading (1 1 1) // 1
  hex (1 2 6 5 9 10 14 13 ) (7 1 40) simpleGrading (1 1 1) // 2
  hex (2 3 7 6 10 11 15 14 ) (11 1 40) simpleGrading (1 1 1) // 3
);
```

Listing 127: Resulting parametric block definition

Complex math - the better solution

The above described way to do mathematical operations is not very elegant. At this place a more elaborate solution is presented.

Listing 128 shows some examples taken from a *m4* script found in the tutorials. The first statement changes the delimiter for comments. By changing the delimiter to `//`, comments have the same delimiter as C or C++. Remember, OpenFOAM dictionaries follow the C++ syntax, therefore, anything following a `//` is treated as a comment. Now, commented lines are always treated as comments by *m4* as well as OpenFOAM. See the first line of Listing 126. There, the `//` starts a comment for OpenFOAM and the `#` starts a comment for *m4*. Setting the delimiter for comments to be the same as in C++ removes an ambiguity and a possible source for errors.

The second line of Listing 128 redefines the quote delimiter. Changing this delimiters from the standard to the brackets is probably done to improve readability.

In line 4 of Listing 128 a macro named `calc` is defined. This macro also uses a system call to outsource the actual math. In this case the interpreter of the script programming language Perl⁴⁴ is called. This interpreter receives a command line argument and an instruction. The command line argument `-e` tells the interpreter that only one line of code will follow. The interpreter will interpret this single line and exit. The instruction `print ($1)` is a function that prints its argument on the standard output. The argument of the `print` function is the argument of the `calc` macro. Therefore, the mathematical operation can be written directly in the code. See line 9 for an example. There, the symbols `rb` and `Rb` are replaced by *m4* by their definition. The argument of the `calc` macro is passed via the system call to the Perl interpreter. As Perl is able to do mathematical operations, the interpreter computes the result of the expression and executes the function `print`. The macro `esyscmd` returns the standard output of the command it executed.

Line 12 of Listing 128 shows that even more complex math – e.g. using trigonometric functions – is possible.

```
1  changecom(//)
2  changequote([,])
3
4  define(calc, [esyscmd(perl -e 'print ($1)'))
5
6  define(rb, 0.5)
7  define(Rb, 0.7)
8
9  define(ri, calc(0.5*(rb + Rb)))
10
11 define(pi, 3.14159265)
12 define(ca0, calc(cos((pi/180)*a0)))
```

Listing 128: Doing complex math with *m4*

15.5.3 Conclusion

Parametric meshes can be created by using the macro language *m4*, this is demonstrated in real live by the OpenFOAM tutorials. Also the author of this work has done so; up to a level which prompted his colleagues

⁴⁴See <http://www.perl.org/>

to make fun of him. This highlights the major shortcoming of using *m4* for parametric meshes. At some point, the parametric geometry creation poses the need for complex math or even high-level data structures. Thus, we soon are in need of a general purpose programming (or scripting) language.

The mesh in Figure 15 was created with a parametric geometry. It features a variable, user-selectable number of rotor-paddles n_b and stator-baffles n_p , with the constraint of that numbers being an integer divisor of 12. The two numbers n_b and n_p are independent of each other, as demonstrated in Figure 15. The infinitely thin baffles and paddles are created by preventing selected blocks from getting connected by the use of collocated points.

$$n_b, n_p \in \{1, 2, 3, 4, 6, 12\} \quad (7)$$

In total the mesh shown in Figure 15 consists of 459 blocks. This mesh (most probably⁴⁵) would have been impossible to create using *m4*. The scripting language of choice for this mesh was *python*⁴⁶, which is an interpreted high-level, general-purpose programming language.

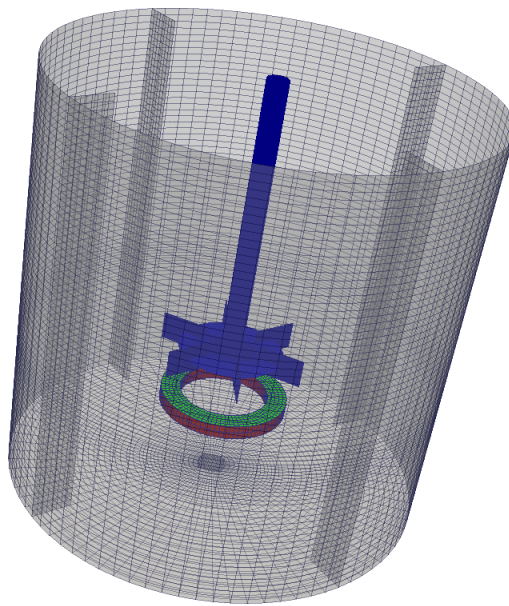


Figure 15: The mesh of a stirred tank with a Rushton impeller, stator baffles and an aeration device.

Thus, we conclude this section on using *m4* for geometry creation with **Eric S. Raymond**'s view on *m4*:

The *m4* macro language supports conditionals and recursion. The combination can be used to implement loops, and this was intended; *m4* is deliberately Turing-complete. But actually trying to use *m4* as a general-purpose language would be deeply perverse.

This quote from Eric S. Raymond [20] should not be seen as trying to discourage the use of *m4* for simple task. It is intended to point out the limitations of macro languages. The limitation met and experienced by the author are the following:

Math In the sections above, we discussed two ways to perform complex mathematical operations within an *m4* script, by utilizing *bc* or *perl* via a system call. In *python*, we can do complex math directly, without having to perform system calls to programs which, might or might not be installed on the user's system.

Data structures The mesh generation script for the stirred tank makes use of *python*'s high-level data structure reflecting the organisation of the points on the geometry. Thus, the resulting script is far better to understand than an even less complex *m4* script.

⁴⁵After some initial attempts, the author gave up.

⁴⁶<https://www.python.org/>

File I/O With *m4*, all we can do is macro substitution. Thus, everything comes from one file and goes to one file. With a high-level language such as *python*, we can write several files. Thus, all files containing geometric information can be written by the same script, e.g. the `blockMeshDict` and the `topoSetDict(s)`. This improves maintainability and reduces code duplication and manual labour.

15.5.4 Parametric meshes by the help of Python

As mentioned in the previous sub-section on the creation of parametric meshes with the scripting language *m4*, Python offers a great deal more versatility. This can partly be attributed to the passage of time, since *m4* first appeared in 1977 and Python was released in 1990. However, the main factor is language-design, since Python has been designed around high-level data structures among other things.

Furthermore, repeated tasks can be more easily encapsulated into functions, e.g. write a block-definition from a list of nodes and a list of cell numbers.

15.5.5 Parametric meshes with OpenFOAM's on-board tools

Using the `#calc` directive

We can do complex math by using OpenFOAM's `#calc` directive, as shown in Listing 129. Here we assign the result of an inline calculation to a variable, which we can use just as any other variable with OpenFOAM's macro expansion.

As Listing 129 shows, we can use all functions that are part of the C standard library.

```

1 // compute the x-discretisation of the first block
2 NX1 #calc "std::ceil( ($X1 - $X0) / $dx )";
3
4 // manually assign a value
5 NZ1 10;
6
7 // the block-definition of the first block; here NX1 and NZ1 are being used
8 hex (0 1 5 4 12 13 17 16) ($NX1 $NY1 $NZ1) simpleGrading (1 1 1) // 1

```

Listing 129: Using the `#calc` directive for inline calculations in `blockMeshDict`.

```

1 #include "dictionary.H"
2 #include "Ostream.H"
3 #include "Pstream.H"
4 #include "unitConversion.H"
5
6 namespace Foam
7 {
8
9 extern "C"
10 {
11     void codeStream_faaac6ab5d5819d340ee97b4ed5bc6ec9820e7fb
12     (
13         Ostream& os,          const dictionary& dict
14     )
15     {
16         {{{ begin code
17             #line 1 " " os << (std::ceil( (7.500000e-02 - 0.000000e+00) / 3.000000e-02 ));
18         }}} end code
19     }
20 }
21
22 } // End namespace Foam

```

Listing 130: OpenFOAM creates and runs this code from the `#calc` directive in Line 2 of Listing 129.

Use the following command to find examples on the use of the `#calc` directive in the OpenFOAM tutorials:

```
find $FOAM_TUTORIALS -type f | xargs grep '#calc'
```

Listing 131: Find usage examples of the `#calc` directive in the tutorials.

Drawback

Using OpenFOAM to do the math, or any other tasks, has the drawback that we don't get to see the final `blockMeshDict` file. If we have an error in our math or our logic, then *blockMesh* will interpret its input in the `blockMeshDict` file, and the error will lead to an error in mesh-creation. The result will be an error message on the part of OpenFOAM. However, since the process is as follows, and everything happens within *blockMesh*, debugging will be more difficult.

- Read and interpret the `blockMeshDict`
- Run all blocks of code
- Create the mesh

In the case of a scripting language, such as *m4* or Python, the interpreter runs the script and writes the `blockMeshDict` to disk. Any math or logic error can be found in the resulting `blockMeshDict` file, e.g. in terms of non-matching discretisation between adjacent blocks.

15.6 Trouble-shooting

15.6.1 Don't be misled by error messages

During manually building a small mesh with *blockMesh* by hand, i.e. writing the `blockMeshDict` using nothing but an ordinary text editor, I made a rather interesting observation. To save myself the effort of scrolling back and forth between the vertex list and the patch definition, I copied a part of the vertex list and pasted it right where I specified the boundary faces. Thus, vertex definitions ended up, where patch definitions are expected. After I ran *blockMesh* without removing the vertex definitions from the list of patches, *blockMesh* unsurprisingly failed. However, this example shows that OpenFOAM's error messages can be misleading.

Listing 132 shows the output of *blockMesh* resulting from the above outlined scenario. The warning message correctly reports an unexpected input, the error message however, reports a hanging pointer. The hanging pointer is certainly caused by the faulty entry, however, the error message does not indicate an error within the `blockMeshDict`. In this case, the warning message bears the relevant information, hence users are advised to carefully read OpenFOAM's output (warning and error messages) in case something goes wrong.

```
Creating topology blocks
Creating topology patches
--> FOAM Warning :
    From function entry::getKeyword(keyType&, Istream&)
    in file db/dictionary/entry/entryIO.C at line 80
    Reading /home/user/OpenFOAM/user-4.0/run/meshing/testCase/system/blockMeshDict.boundary
    found on line 135 the punctuation token '('
    expected either } or EOF

...

--> FOAM FATAL ERROR:
    hanging pointer at index 8 (size 12), cannot dereference

    From function const T& Foam::UPtrList<T>::operator[](Foam::label) const [with T = Foam::
    entry; Foam::label = int]
    in file /home/user/OpenFOAM/OpenFOAM-4.0/src/OpenFOAM/lnInclude/UPtrListI.H at line 107.

FOAM aborting

#0  Foam::error::printStack(Foam::Ostream&) at ????
```

Listing 132: The output of *blockMesh* with a faulty `blockMeshDict`. The red dots indicate removed warning messages were removed for brevity.

This observation the warning message carries more useful information than the error message might apply also to other parts of the OpenFOAM framework.

15.6.2 Viewing the blocks with *ParaView*

A mesh created by *blockMesh* consists of blocks. Listing 133 shows how *ParaView* can be used to visualise the blocks.

```
paraFoam -block
```

Listing 133: Visualising the blocks

This way, only the blocks are displayed. *ParaView* only reads the file `blockMeshDict`. Figure 16 shows the blocks of a parametric mesh. It consists of nine blocks. The image shows also the numbers of the vertices.

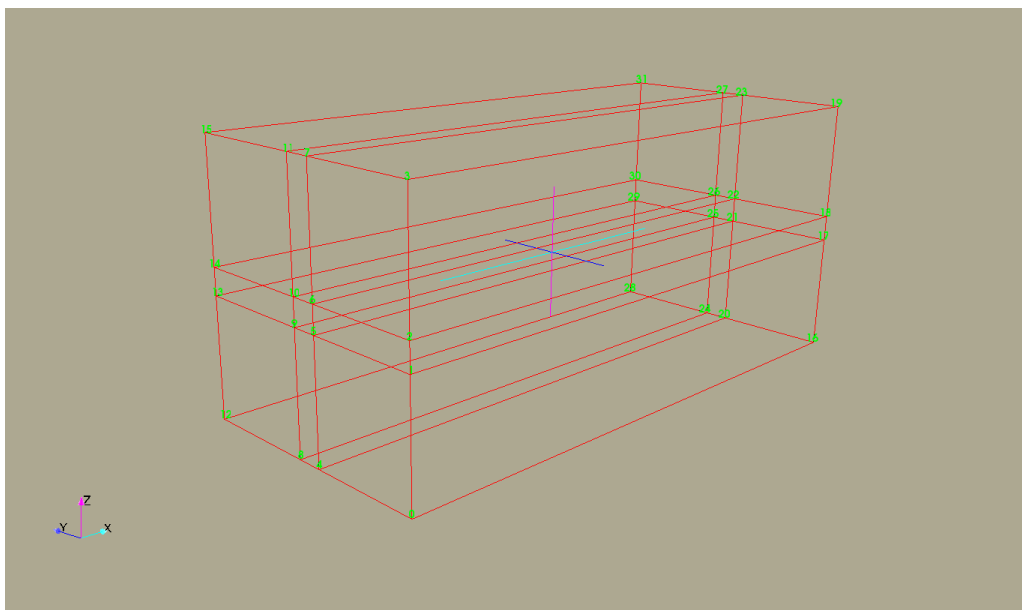


Figure 16: The blocks of a parametric mesh consisting of nine blocks.

15.6.3 Viewing the blocks with *pyFoam*

Troubleshooting can be difficult when *blockMesh* doesn't create a mesh and displays some error messages instead.

See Section 15.6.2 for the discussion of a tool which is able to display the blocks as they are defined in `blockMeshDict`. This tool even works, when *blockMesh* fails due to an erroneous definition in `blockMeshDict`.

16 *snappyHexMesh*

snappyHexMesh, also referred to as *snappy*, is a meshing tool that is able to mesh the space around an arbitrary triangulated surface, e.g. an STL surface-mesh. This is generally the case in external aerodynamics. *snappyHexMesh* can only be used in conjunction with *blockMesh*, since it requires a background mesh.

16.1 Documentation

Unfortunately, the complexity of *snappyHexMesh* outweighs the available on-board documentation. The on-board documentation (User Guide) can be found in `doc/Guides-a4` or `doc/Guides-usletter` of your local OpenFOAM installation or online at <http://www.openfoam.org/docs/user/>. You find a commented `snappyHexMeshDict` at `$FOAM_UTILITIES/mesh/generation/snappyHexMesh`. This is the case for all utilities which are controlled by an utility-specific dictionary file, such as *decomposePar*, *topoSet* and many more.

Individual features of *snappy* are in some cases discussed in the release notes of the release with which these features were rolled out. Another source of good documentation of *snappy* are presentations held at the OpenFOAM Workshops. An internet search with appropriate keywords will point the reader to them, since some of them are publicly available on the internets.

As with any other tool, the reader is encouraged to run the tutorials provided by OpenFOAM and play around with them. The tutorial cases also provide a good starting base for building your own cases.

16.2 Work flow

The creation of a mesh by *snappyHexMesh* is following a two step approach:

1. The background mesh is created by *blockMesh*. This is absolutely necessary to the later work of *snappy*. It is advised for the background mesh to consist of all-hex cells with an aspect ratio of 1, i.e. cube-shaped cells. It is furthermore beneficial to have many intersections of the background mesh's cell-edges with the tri-surface.
2. *snappyHexMesh* then performs three basic steps:
 - (a) *Castellating*
The tri-surface is approximated by splitting and removing cells outside the tri-surface.
Cell splitting The cells of the background mesh near the objects surface are refined.
Cell removal Cells of the background mesh inside the object are removed.
 - (b) *Snapping*
Cell snapping The remaining background mesh is modified in order to reconstruct the surface of the object.
 - (c) *Layer addition*
Layer addition Additional hexahedral cells are introduced on the boundary surface of the object to ensure a good mesh quality.

16.3 Example: Bath Tub

With the help of an actual example, we will now discuss some of *snappyHexMesh*'s features, as problems and insights most often come with practical use. Our bath tub has a non-trivial shape, thus we are not inclined to painfully create the `blockMeshDict` by hand or by script. For complicated geometries a sophisticated meshing tool such as *snappy* is the way to go.

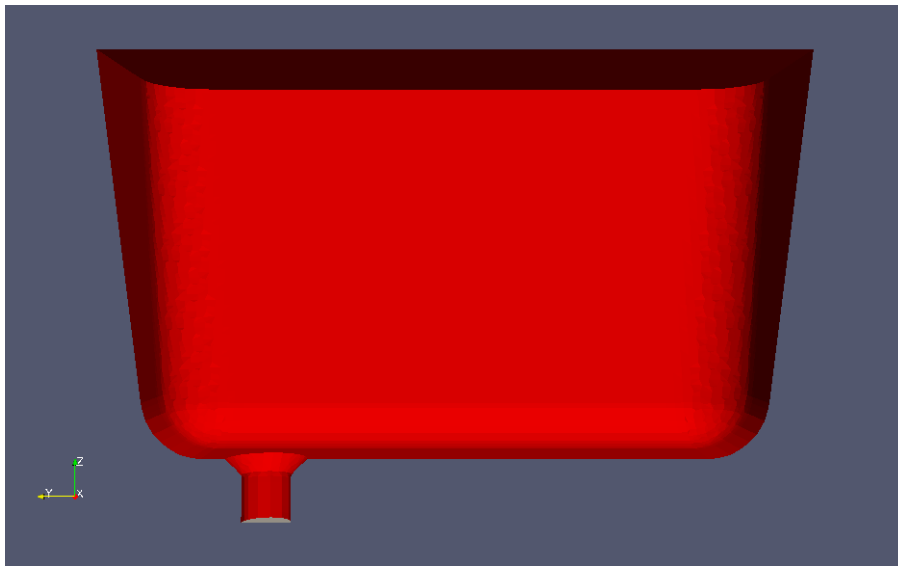


Figure 17: A bath tub. The outlet patch is marked grey at the very bottom of the drain tube.

16.3.1 Boundary layers

Boundary layers are added in the last stage of *snappy*'s operation. These are added on a per-patch basis. Thus, it is not possible to add layers only to parts of a patch. On the patch itself, we can control the regions in which to add a layer by the keyword `featureAngle`. The operation of the layer addition stage is controlled by the `addLayersControls` dictionary of `snappyHexMeshDict`.

Some of the entries of the `addLayersControls` dictionary are self-explanatory, such as the `layers` dictionary specifying the patches on which to add layers of cells. However, other parameters are not that obvious in their meaning.

`featureAngle`

The `featureAngle` is the angle between two consecutive faces. This parameter controls the behaviour of the layer addition stage at corners and bends.

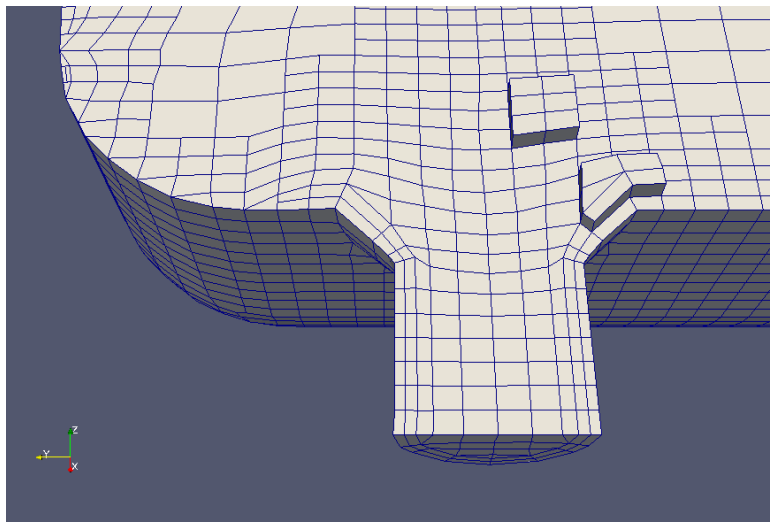


Figure 18: A badly chosen `featureAngle` causes *snappy* to add incomplete boundary layers.

`slipFeatureAngle`

At the outlet patch of our domain, the layer added to the wall patch meets the outlet patch, i.e. vertices need to be added to the outlet patch in order to properly grow a layer of cells onto the wall patch. See the left side of Figure 19. In order to achieve this, we must be able to alter the outlet patch during layer addition even though, we do not add a layer to the outlet patch itself.

This feature is discussed in the release notes⁴⁷ of OpenFOAM-2.2.0.

⁴⁷<http://www.openfoam.org/version2.2.0/snappyHexMesh.php>

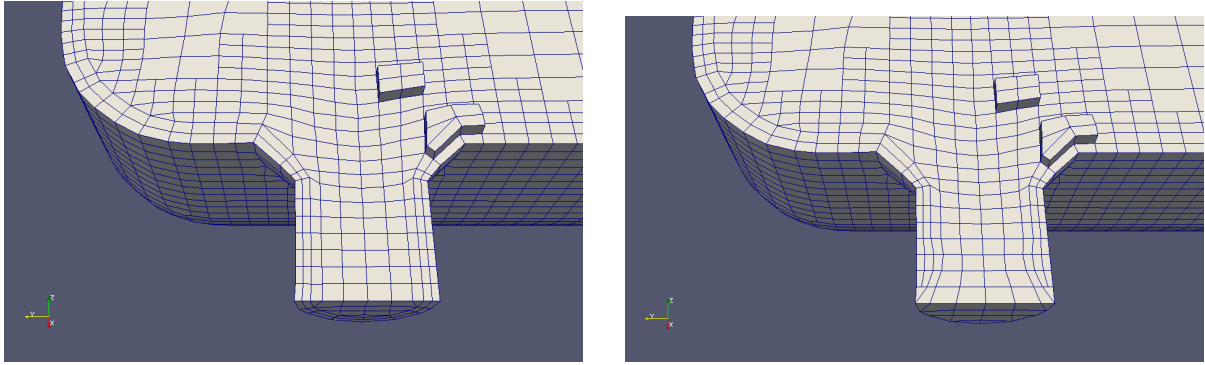


Figure 19: The boundary layers added by *snappy*. On the left, layer addition went as we intended it to do; on the right, we see the effect of the (missing) keyword `slipFeatureAngle` of the `addLayersControls` dictionary of `snappyHexMeshDict`.

Exclude patches

We have to freedom to tell *snappyHexMesh* to leave patches alone. Thus, during layer addition these patches remain untouched. This allows us to reverse the effect we achieved with the `slipFeatureAngle` parameter. By specifically excluding the outlet from any layer addition activity (see Listing 134), we end up with a collapsing cell layer at the boundary of the outlet patch, see Figure 20.

```

layers
{
    bathTub
    {
        nSurfaceLayers 2;
    }
    outlet
    {
        nSurfaceLayers 0;
    }
}

```

Listing 134: The `layers` sub-dictionary of the `addLayersControl` dictionary: specifically excluding a patch from layer addition.

This example of use may most probably not meet practical requirements, however, it demonstrates how *snappy* works. The take-away message might be that `nSurfaceLayers` beats `slipFeatureAngle`.

A non-academic (read less-useless) theoretical use-case for excluding patches from layer addition might be, when we later merge different meshes. In that case, we might want to preserve some patches for the merging operation.

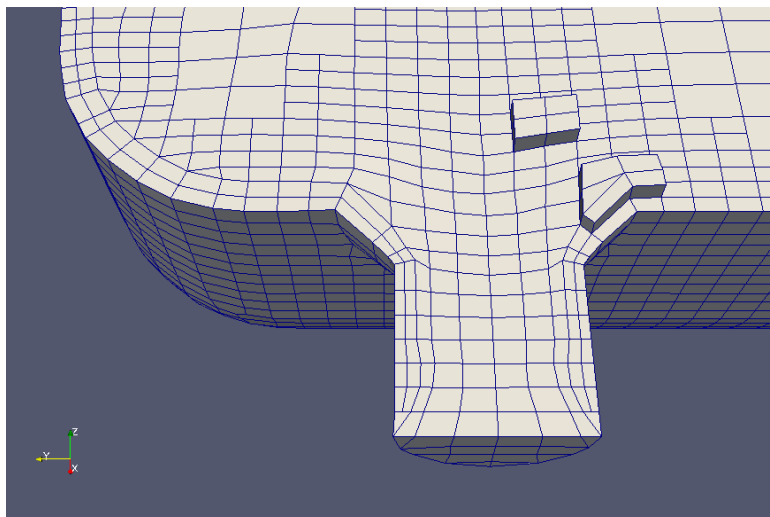


Figure 20: A collapsing boundary layer. Maybe we did not want the mesh that way, however, we told *snappy* to create it exactly that way.

16.3.2 Pitfalls, sources of error and hints on malfunction

Run time

If *snappyHexMesh* is finished in less than a second, then something is wrong. As *snappyHexMesh* performs up to three work intensive steps (castellation, snapping and layer addition), a run of *snappyHexMesh* takes a couple of seconds or even longer (tens of seconds).

Units

When creating a mesh with *snappyHexMesh* different scales (meter vs. millimeter) of the background mesh and the STL-mesh are a frequent source of error. Check the following things:

1. The unit of the vertex coordinates in `blockMeshDict`
2. The value of the `convertToMeters` keyword in `blockMeshDict`
3. The unit in which the STL was created

17 *foamyHexMesh*

With OpenFOAM-2.3.0⁴⁸ the new meshing tool *foamyHexMesh* was released. This tool is to some degree similar to *snappyHexMesh*. The main distinction between *foamyHexMesh* and *snappyHexMesh* is that meshes by *foamyHexMesh* are better aligned with the boundary surfaces. This is achieved by a different mode of operation. *foamyHexMesh* generates an internal tetrahedral mesh fitting the boundaries, and then generates and massages the dual mesh of this internal tetrahedral mesh.

17.1 Crude comparison between a snappy and a foamy bath tub

In this section we compare the way foamy- and snappyHexMesh work on the example of meshing a bath tub. For this demonstration an STL-surface of a bath tub was created using OpenSCAD.

Figure 21 shows the outline and a part of the background mesh as well as our bath tub.

⁴⁸<http://www.openfoam.org/version2.3.0/foamyHexMesh.php>

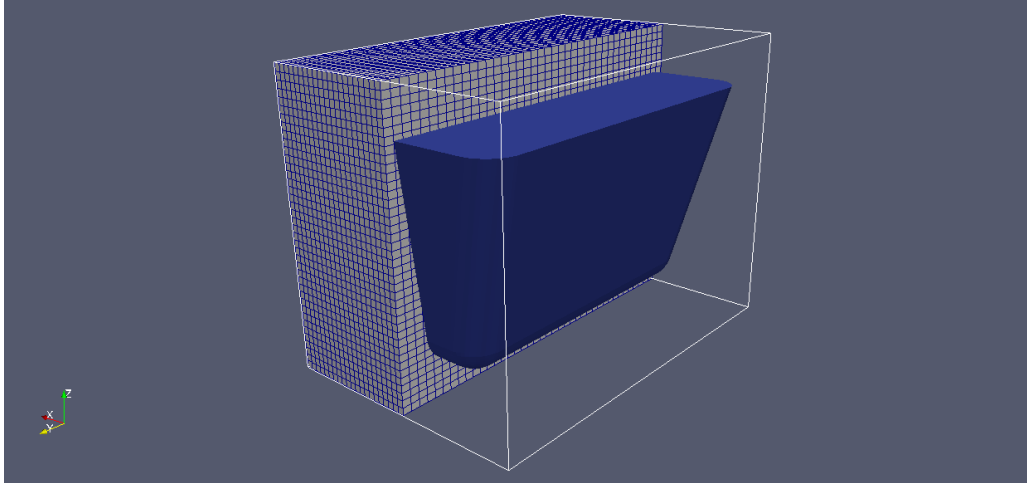


Figure 21: A bath tub with a background mesh enclosing the STL-surface of the bath tub.

17.1.1 SnappyBathTub

At first, the bath tub is meshed using *snappyHexMesh*. Figure 22 shows the resulting mesh. We clearly see, that the interior cells are aligned with the global coordinate axes. At the side walls, this leads to some minor flaws.

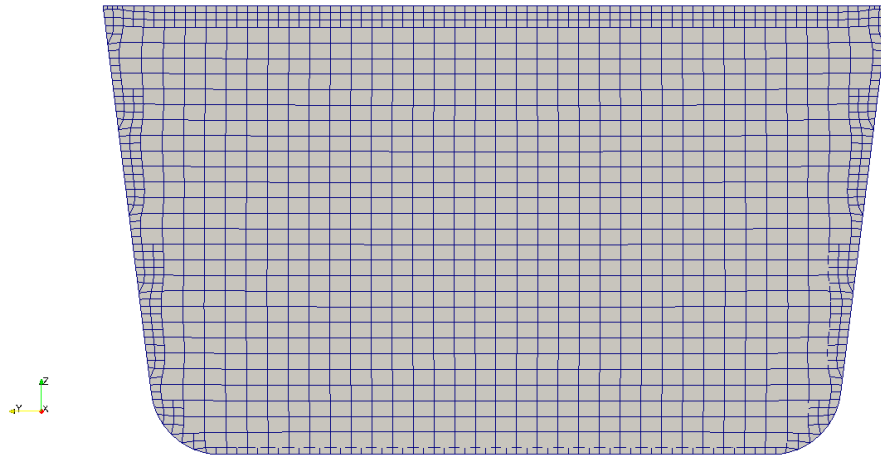


Figure 22: SnappyBathTub

17.1.2 FoamyBathTub

Next, the bath tub was meshed using *foamyHexMesh*. In Figure 23 we see a good alignment of the cells with the boundaries. The interior cells are not aligned with the global coordinate axes.

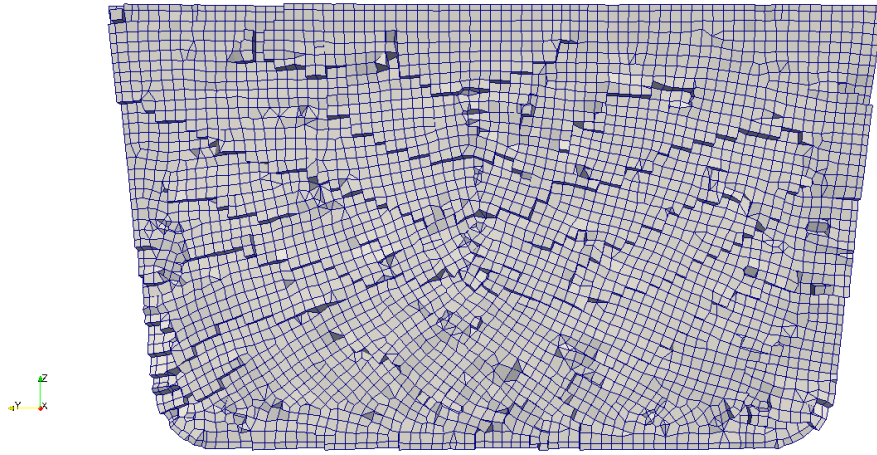


Figure 23: FoamyBathTub

18 *cfMesh*

cfMesh is a collection of meshing tools⁴⁹ provided by the company Creative Fields. This company offers the basic *cfMesh* suite under the GPL for free. At the time of writing *cfMesh* consists of four meshing tools which offer a workflow comparable to the workflow offered by *snappy-* and *foamyHexMesh*.

The meshing tools of *cfMesh* generate their mesh based on a user-provided surface-triangulation of the geometry. There is no need for a background mesh similar as it is the case with *foamyHexMesh*. All of the tools are capable of generating boundary layers on all or on selected surfaces. All the tools are controlled by a dictionary named `meshDict`, which resides in the `system` directory. In general the control of the user over the meshing tools is not as tight as with *snappy-* or *foamyHexMesh*. However, this less tight control manifests itself in a lightweight control dictionary compared to *snappy-* and *foamyHexMesh*.

The meshers of *cfMesh* are:

cartesian2DMesh is the tool to generate 2D meshes

tetMesh generates tetrahedral meshes

cartesianMesh generates meshes consisting mainly of hexahedrals, similar to *snappyHexMesh*

pMesh generates polyhedral meshes

cfMesh also provides a range of utilities (21 at the time of writing) for various tasks.

18.1 Usage

18.1.1 To treat feature edges, or not to ...

Feature edges must be specified explicitly by the user for *cfMesh* to obey these edges.

In the case of the bath tub, which has a single patch a boundary, we see the effect of not providing feature edges explicitly in Figure 24. In this case the provided STL surface was not obeyed perfectly in favour of nicer cells. If we wanted to resolve the feature edge, we need to split the boundary of the geometry into more than one patch. As the edges between neighbouring patches are resolved by default by the mesher, dividing the bath tub's boundary into several patches would solve the problem shown in Figure 24.

⁴⁹<http://cfmesh.com/>

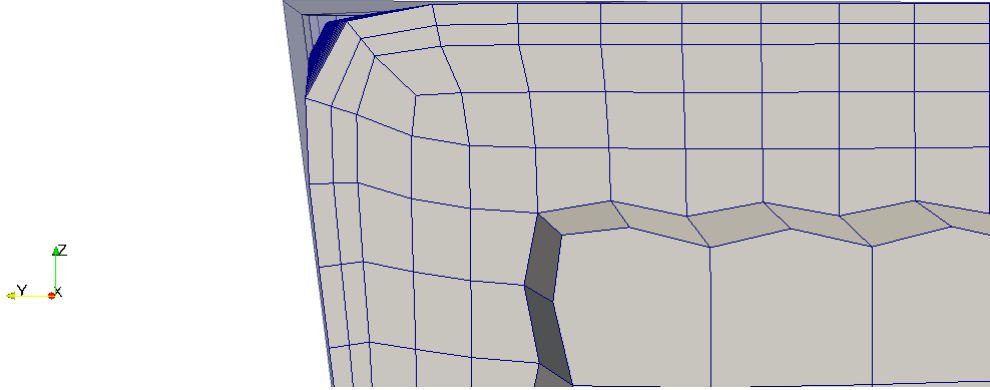


Figure 24: Poor feature edge resolution caused by not providing information on feature edges. Note, the whole geometry is bounded by a single patch.

If we want to resolve a feature edge which is not the boundary of two patches, we can use the utility tool *surfaceFeatureEdges* to extract the feature edges from the geometry. This tool checks the angles of neighbouring triangles of the surface triangulation and creates additional patches. E.g. the patch `wall` is divided into the patches `wall_0` to `wall_N`, if the specified feature angle results in `wall` being divided into `N` individual zones. *cfMesh* resolves the edges between neighbouring patches by default. Thus, the mesher is agnostic of our feature edge treatment. After finishing meshing, the mesher can rename patches. This feature of the mesher allows us to combine all the intermediate patches back into our initial `wall` patch.

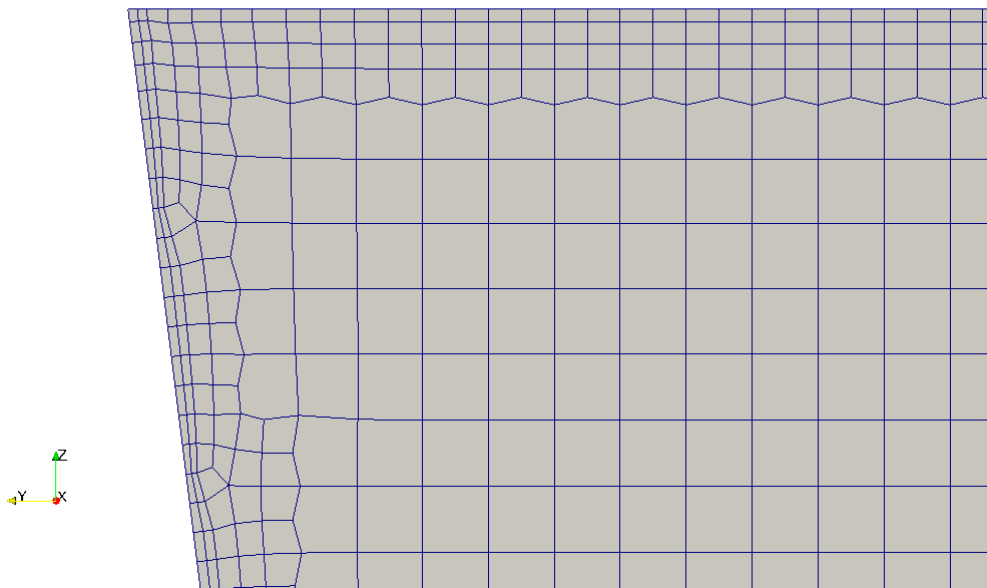


Figure 25: Resolved feature edge of the bath tub. In this case, the boundary consists of two patches: the top surface and the rest.

We note in Figure 25 the hanging nodes inserted by the mesher to join the different refinement levels. These hanging nodes protrude from the face they are inserted into. This prevents the faces connecting the cells of different refinement level from being coplanar, as it is the case with *snappyHexMesh*.

19 *checkMesh*

checkMesh is a tool to perform tests on an existing mesh. *checkMesh* is simply invoked by its name. Like other tools, *checkMesh* assumes to be called from the case directory. When *checkMesh* is to be called from another location than the case directory, the path to the case directory has to be specified with the option `-case`.

Listing 135 shows an error message produced by *checkMesh*, if *checkMesh* has been called with no mesh present. In this case the tool can't find the files specified in Section 13.1.

```
--> FOAM FATAL ERROR:
Cannot find file "points" in directory "polyMesh" in times 0 down to constant

    From function Time::findInstance(const fileName&, const word&, const IOobject::readOption,
    const word&)
    in file db/Time/findInstance.C at line 188.

FOAM exiting
```

Listing 135: No mesh present

A more thorough testing is performed when *checkMesh* is called with two additional options. Then *checkMesh* performs some further tests.

```
checkMesh -allGeometry -allTopology
```

Listing 136: Do more checks

checkMesh has also the `-latestTime` option like many other OpenFOAM tools. This option is particularly useful when examining meshes created by *snappyHexMesh*. *snappyHexMesh* stores intermediate meshes if it is

not told otherwise. By default, after a completed run of *snappyHexMesh* there are the background mesh and the results of the three basic stages of a *snappyHexMesh* run (castellation, snapping and layer addition). Depending on which of these steps are active up to four meshes may be present. Restricting *checkMesh* to the final mesh reduces runtime and avoids the unnecessary examination of an intermediate mesh.

19.1 Definitions

In order to understand the output of *checkMesh* it is necessary to define some quantities calculated by *checkMesh*.

19.1.1 Face non-orthogonality

Non-orthogonality is a property of the faces of the mesh. We need to discriminate between internal faces and boundary faces.

Internal faces

Each internal face connects two cells. The non-orthogonality is the angle between the vector connecting the cell centres and the face normal vector. In Figure 26 the vector connecting the cell centres is denoted $\mathbf{d}$ and the face normal vector⁵⁰ $\mathbf{S}$.

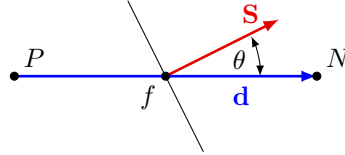


Figure 26: Definition of non-orthogonality for internal faces

In a perfectly orthogonal mesh the vectors $\mathbf{d}$ and $\mathbf{S}$ are parallel. If a mesh is non-orthogonal these vectors draw an angle as in Figure 26. This angle can be calculated from $\mathbf{d}$ and $\mathbf{S}$ by Eq. 10.

$$\mathbf{d} \cdot \mathbf{S} = \|\mathbf{d}\| \|\mathbf{S}\| \cos(\theta) \quad (8)$$

$$\frac{\mathbf{d} \cdot \mathbf{S}}{\|\mathbf{d}\| \|\mathbf{S}\|} = \frac{\|\mathbf{d}\| \|\mathbf{S}\| \cos(\theta)}{\|\mathbf{d}\| \|\mathbf{S}\|} = \cos(\theta) \quad (9)$$

$$\theta = \arccos\left(\frac{\mathbf{d} \cdot \mathbf{S}}{\|\mathbf{d}\| \|\mathbf{S}\|}\right) \quad (10)$$

Eq. 10 can also be found in the sources of OpenFOAM in the function `faceNonOrthogonality` in the file `cellQuality.C`⁵¹. Listing 137 shows a loop over all faces. For each face the non-orthogonality is computed. The vectors $\mathbf{d}$ and $\mathbf{s}$ are the connecting vector between the cell centres, and the face area vector, respectively. The scalar `cosDDotS` is the angle θ of Figure 26.

Note the two precautions that were taken to avoid numerical issues. First, the denominator is the sum of the product of the magnitudes and `VSMALL`. `VSMALL` is a number with a very small value to prevent division by zero. Second, the argument of the `acos` function is `min(1.0, (d & s)/(mag(d)*magS + VSMALL))`. Keeping the argument of the arc-cosine equal or below 1 makes perfectly sense, because the arc-cosine is defined only for values between -1 and 1. The limit of -1 is inherently ensured. The inner product of two vectors is always positive. `VSMALL` is also positive.

```

1  forAll(nei, faceI)
2  {
3      vector d = centres[nei[faceI]] - centres[own[faceI]];
4      vector s = areas[faceI];

```

⁵⁰The face normal vector or face area vector is a vector normal to a face. The length of this vector is equal to the area of the face.

⁵¹In the file `cellQuality.C` there are two methods defined: `nonOrthogonality()` and `faceNonOrthogonality()`. Comparing the code of this two methods reveals, that they compute the same thing. However, the method `nonOrthogonality()` returns the affected cells, whereas `faceNonOrthogonality()` returns the affected faces.

```

5     scalar magS = mag(s);
6
7     scalar cosDDotS =
8         radToDeg(Foam::acos(min(1.0, (d & s)/(mag(d)*magS + VSMALL))));
9     result[faceI] = cosDDotS;
10 }

```

Listing 137: A detail of the function `faceNonOrthogonality` in the file `cellQuality.C`

The non-orthogonality reported by `checkMesh` is the angle θ of Figure 26. Therefore the reported non-orthogonality lies in the range between 0 and 90. A non-orthogonality of 0 means the mesh is orthogonal and consists of hexahedra (cuboids) or regular tetrahedra. Listing 141 shows the output of `checkMesh`. In this case the mesh is orthogonal, the maximum and average non-orthogonality is 0.

Listing 143 shows the output of `checkMesh` in case of a non-orthogonal mesh. Listing 144 indicates that a non-orthogonality of above 70 triggers `checkMesh` to issue a warning message.

Boundary faces

Non-orthogonality is also defined for boundary faces. Figure 27 shows a schematic boundary face with its face center f . Non-orthogonality of boundary faces is defined as the angle in degrees between the face area vector $\mathbf{S}$ and the vector $\mathbf{d}$, which connects the cell center P and the face center f .

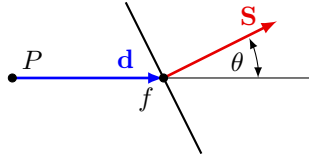


Figure 27: Definition of non-orthogonality for boundary faces

```

1  const labelUList& faceCells = mesh_.boundaryMesh()[patchI].faceCells();
2  const vectorField::subField faceCentres = mesh_.boundaryMesh()[patchI].faceCentres();
3  const vectorField::subField faceAreas = mesh_.boundaryMesh()[patchI].faceAreas();
4
5  forAll(nei, faceI)
6  {
7      vector d = faceCentres[faceI] - centres[faceCells[faceI]];
8      vector s = areas[faceI];
9      scalar magS = mag(s);
10
11      scalar cosDDotS =
12          radToDeg(Foam::acos(min(1.0, (d & s)/(mag(d)*magS + VSMALL))));
13      result[globalFaceI++] = cosDDotS;
14  }

```

Listing 138: A detail of the function `faceNonOrthogonality` in the file `cellQuality.C`

19.1.2 Face skewness

OpenFOAM defines skewness in a mesh different than other tools, e.g. Gambit. The reason for this OpenFOAM-specific definition is that this definition is associated with the definition of a skewness error in [36] as part of mesh induced discretisation errors.

Skewness is a property of the faces of the mesh. We need to discriminate between internal faces and boundary faces.

Internal faces

Each internal face connects two cells. Figure 28 shows the cell centres P and N of two adjacent cells. The face $face_{PN}$ is the face connecting these two cells. The point F is the face centre of the face $face_{PN}$. The line $c = \overline{PN}$ connects the cell centres. This connecting line intersects with the face $face_{PN}$. This intersection point I divides the line c into the two parts c_1 and c_2 .

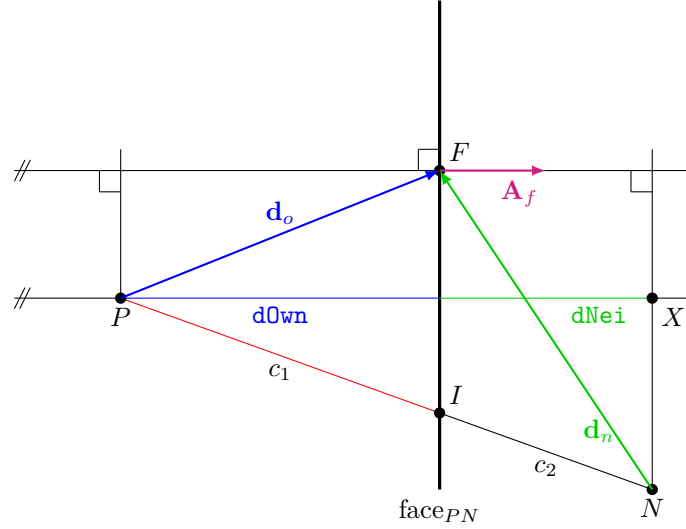


Figure 28: Definition of skewness of internal faces

To calculate the location of I the length of c_1 is of key interest because the skewness is defined in Eq. 11. The location (the vector to) the points P , N and F are easily obtained. From this three vectors $\mathbf{d}_o$, $\mathbf{d}_n$ and $\mathbf{c}$ is computed. With $\mathbf{d}_o$ and $\mathbf{d}_n$ the inner product with the face area vector $\mathbf{A}_f$ is computed to obtain $\mathbf{d0wn}$ and $\mathbf{dNei}$ ⁵².

$$\text{skewness} = \frac{|\overline{IF}|}{|\overline{PN}|} \quad (11)$$

$$\mathbf{d}_o = \vec{F} - \vec{P} \quad (12)$$

$$\mathbf{d}_n = \vec{F} - \vec{N} \quad (13)$$

$$\mathbf{c} = \vec{N} - \vec{P} \quad (14)$$

$$\mathbf{d0wn} = \frac{\mathbf{d}_o \cdot \mathbf{A}_f}{\|\mathbf{A}_f\|} \quad (15)$$

$$\mathbf{dNei} = \frac{\mathbf{d}_n \cdot \mathbf{A}_f}{\|\mathbf{A}_f\|} \quad (16)$$

$$\angle(XPN) = \alpha \quad (17)$$

$$\cos(\alpha) = \frac{\mathbf{d0wn}}{c_1} = \frac{\mathbf{d0wn} + \mathbf{dNei}}{c_1 + c_2} = \frac{\mathbf{d0wn} + \mathbf{dNei}}{\|\mathbf{c}\|} \quad (18)$$

$$c_1 = \frac{\mathbf{d0wn}}{\mathbf{d0wn} + \mathbf{dNei}} \|\mathbf{c}\| \quad (19)$$

$$\vec{I} = \vec{P} + c_1 \mathbf{c} \quad (20)$$

$$\text{skewness} = \frac{\|\vec{F} - \vec{I}\|}{\|\mathbf{c}\|} \quad (21)$$

Note that both $\vec{P}$ and $\mathbf{c}$ are vectors. The reader hopefully excuses this lack of consistency in mathematical notation. $\vec{P}$ denotes the position vector of the point P . In this case the symbol $\vec{P}$ is preferred to $\mathbf{P}$ in order to use symbols that can be found in Figure 28.

Listing 139 shows a detail of the function `faceSkewness` from the file `cellQuality.C`⁵³. There a loop over all internal faces is traversed. The loop body contains the calculation of the skewness. First `d0wn` and `dNei` are computed. Then the location of the point I is determined. The variable `faceIntersection` of the type

⁵²`d0wn` and `dNei` are actual variable names. Therefore these symbols are written in typewriter font.

⁵³In the file `cellQuality.C` there are two methods defined: `skewness()` and `faceSkewness()`. Comparing the code of this two methods reveals, that they compute the same thing. However, the method `skewness()` returns the affected cells, whereas `faceSkewness()` returns the affected faces.

`point` contains the position vector to the point I – the point at which the connection line between the cell centres intersects the face. Finally, the skewness is calculated (compare Eq. 21). Notice the precaution against a possible division by zero (adding `VSMALL` to the denominator).

```

1  forAll(nei, faceI)
2  {
3      scalar dOwn = mag
4      (
5          (faceCtrs[faceI] - cellCtrs[own[faceI]]) & areas[faceI]
6      )/mag(areas[faceI]);
7
8      scalar dNei = mag
9      (
10         (cellCtrs[nei[faceI]] - faceCtrs[faceI]) & areas[faceI]
11     )/mag(areas[faceI]);
12
13     point faceIntersection =
14         cellCtrs[own[faceI]]
15         + (dOwn/(dOwn+dNei))*(cellCtrs[nei[faceI]] - cellCtrs[own[faceI]]);
16
17     result[faceI] =
18         mag(faceCtrs[faceI] - faceIntersection)
19         /(mag(cellCtrs[nei[faceI]] - cellCtrs[own[faceI]]) + VSMALL);
20 }

```

Listing 139: A detail of the function `faceSkewness` in the file `cellQuality.C`

Boundary faces

Skewness is also defined and checked for boundary faces. Figure 29 shows the sketch of a boundary face with its face center F_C . The vector $\mathbf{d}$ from the cell center P to the face center F_C is depicted in red. At the point F_C we see the face normal vector $\mathbf{n}$. If we project the vector $\mathbf{d}$ on the vector $\mathbf{n}$ we gain the face-intersection point F_I . This is the point, where the face normal departing from the cell center intersects with the face. The face-intersection does not necessarily need to be part of the face, as it is the case in Figure 29.

We then compute the vector $\mathbf{f}$, which is the connection between the points F_I and F_C . The ratio of the magnitudes of the vectors $\mathbf{f}$ and $\mathbf{d}$ defines the skewness of a boundary face.

Listing 140 shows the code that computes the skewness of the boundary faces. The points P and F_C are returned by the methods `faceCells()` and `faceCentres()`. The normal vector $\mathbf{n}$ is easily computed from the face-area vector given by the method `faceAreas()`.

$$\mathbf{n} = \text{faceAreas}[\text{faceI}] / \text{mag}(\text{faceAreas}[\text{faceI}]) \quad (22)$$

$$\mathbf{d} = \text{faceCentres}[\text{faceI}] - \text{cellCtrs}[\text{faceCells}[\text{faceI}]] \quad (23)$$

$$\vec{F}_I = \text{cellCtrs}[\text{faceCells}[\text{faceI}]] + ((\text{faceCentres}[\text{faceI}] - \text{cellCtrs}[\text{faceCells}[\text{faceI}]]) \cdot \mathbf{n}) * \mathbf{n} \quad (24)$$

$$\vec{F}_I = \vec{P} + (\mathbf{d} \cdot \mathbf{n})\mathbf{n} \quad (25)$$

$$\mathbf{f} = \text{faceCentres}[\text{faceI}] - \text{faceIntersection} \quad (26)$$

$$\mathbf{f} = \vec{F}_C - \vec{F}_I \quad (27)$$

```

1  label globalFaceI = mesh_.nInternalFaces();
2
3  forAll(mesh_.boundaryMesh(), patchI)
4  {
5      const labelUList& faceCells =
6          mesh_.boundaryMesh()[patchI].faceCells();
7
8      const vectorField::subField faceCentres =
9          mesh_.boundaryMesh()[patchI].faceCentres();
10     const vectorField::subField faceAreas =
11         mesh_.boundaryMesh()[patchI].faceAreas();
12
13     forAll(faceCentres, faceI)

```

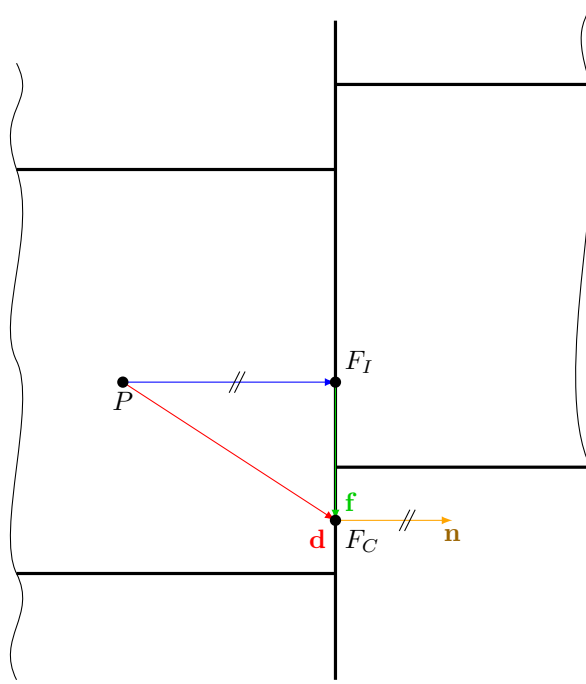


Figure 29: Definition of skewness of boundary faces

```

14 {
15     vector n = faceAreas[faceI]/mag(faceAreas[faceI]);
16
17     point faceIntersection = cellCtrs[faceCells[faceI]]
18         + ((faceCentres[faceI] - cellCtrs[faceCells[faceI]])&n)*n;
19
20     result[globalFaceI++] = mag(faceCentres[faceI] - faceIntersection)
21         /(
22             mag(faceCentres[faceI] - cellCtrs[faceCells[faceI]])
23             + VSMALL
24         );
25 }
26 }

```

Listing 140: A detail of the function `faceSkewness` in the file `cellQuality.C`

19.1.3 Face concavity

pending

19.1.4 Face warpage

A face is warped, when its vertices do not lie within a plane. Figure 30 shows a simplified situation of a warped face. Any three points, which do not fall onto a single line, span a plane. In Figure 30 the area vector $\mathbf{S}_1$ of the triangle $\Delta 457$ is parallel to the face area vector $\mathbf{S}_f$. Thus, we identify point 6 as being out-of-plane.

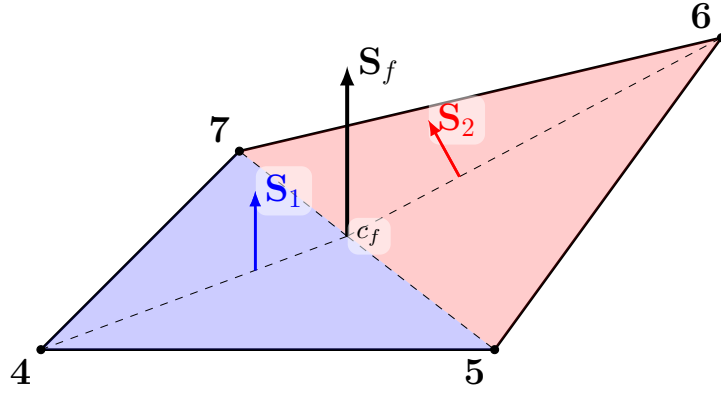


Figure 30: Face warpage

If we decompose the face into individual triangles, we can compare the individual triangle area vectors to the face normal vector. In Figure 30 a crude decomposition is chosen for simplicity. In OpenFOAM's internals, the individual triangles are defined by the face center and two consecutive vertices of the face. As, face vertices need to be stored consecutive, a simple loop over the vertices of a face is sufficient to generate all individual triangles. Thus, in OpenFOAM's implementation of the test for warpage, the face of Figure 30 would be decomposed into four triangles, as indicated by the thin dashed lines.

We bear in mind, that in OpenFOAM a face area vector has two important properties. It is normal to the face's plane and its magnitude is proportional to the face's area⁵⁴. By dividing the face area vector by its magnitude we gain the face normal vector, see (29).

OpenFOAM checks for warpage by computing the inner product of the triangle area vectors with the face normal vector, and summing up the results, see (30). This sum is equal to the magnitude of the face area vector, when all vertices are in-plane. If the two vectors of an inner product are not parallel, then the magnitude of the inner product is smaller by the cosine of the enclosed angle.

$$\|\mathbf{a} \cdot \mathbf{b}\| = \|\mathbf{a}\| \|\mathbf{b}\| \cos(\alpha) \quad (28)$$

$$\mathbf{n}_f = \frac{\mathbf{S}_f}{\|\mathbf{S}_f\|} \quad (29)$$

$$S_f \stackrel{?}{=} \sum_i \mathbf{n}_f \cdot \mathbf{S}_i \quad (30)$$

19.1.5 Cell concavity

When a cell is concave

19.2 Pitfalls

The results of *checkMesh* need to be taken with a grain of salt. Therefore, it is helpful to know how *checkMesh* defines the quality measures it tests for (Section 19.1) and also to know about the shortcomings of the tests performed by *checkMesh* (Section 19.2).

The tests performed by *checkMesh* do not necessarily guarantee the mesh to be suitable for simulation. Furthermore, if a mesh fails a test, that does not necessarily mean that it is unsuitable for calculation.

19.2.1 Mesh quality - aspect ratio

checkMesh performs a number of quality checks. However, the user has to be careful. *checkMesh* does only check if a mesh makes a simulation impossible. There are some situations in which *checkMesh* does not issue an error or a warning, however, a mesh can nevertheless be unsuitable for a successful calculation.

The aspect ratio is the ratio of the largest and the smallest dimension of the cells. For the aspect ratio there are no limits. Listing 141 shows the output of *checkMesh* when a mesh with high aspect ratio cells is tested.

⁵⁴Since a length can not be an area in terms of physical units, we avoid the statement, that the face normal vectors length is the face's area. However, the factor of proportionality is 1.

Although *checkMesh* does not complain, the mesh is not suitable for simulation. Even with extremely small time steps numerical problems appear.

```

Checking geometry...
Overall domain bounding box (0 0 0) (0.1 0.1 0.01)
Mesh (non-empty, non-wedge) directions (1 1 1)
Mesh (non-empty) directions (1 1 1)
Boundary openness (-9.51633e-17 1.17791e-18 -4.51751e-17) OK.
Max cell openness = 1.35525e-16 OK.
Max aspect ratio = 100 OK.
Minimum face area = 2.5e-07. Maximum face area = 2.5e-05. Face area magnitudes OK.
Min volume = 1.25e-09. Max volume = 1.25e-09. Total volume = 0.0001. Cell volumes OK.
Mesh non-orthogonality Max: 0 average: 0
Non-orthogonality check OK.
Face pyramids OK.
Max skewness = 2e-06 OK.
Coupled point location match (average 0) OK.

Mesh OK.

End

```

Listing 141: *checkMesh* output for a mesh with high aspect ratio

19.2.2 Mesh quality - *skewness*

There are different ways to calculate the skewness of a finite volume cell. To test whether *checkMesh* complains about high skewness, a mesh is distorted by the use of edge grading. Figure 31 shows this mesh. Parallel edges are graded alternately – alternating between the expand ratio and its reciprocal value. Listing 142 shows the grading settings. The test case for this examination is the *cavity* case of *icoFoam*. This case can be found in the tutorials.

```

hex (0 1 2 3 4 5 6 7) (20 20 2) edgeGrading (3 0.33 3 0.33 1 1 1 1 1 1 1 1)

```

Listing 142: Block definition in *blockMeshDict* to achieve high skewness

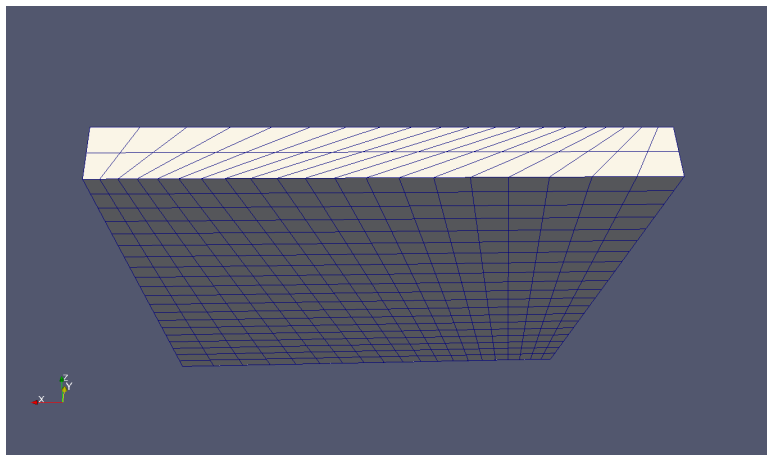


Figure 31: A distorted mesh

checkMesh issues no warnings for the value pair 3 and 0.33. The values 4 and 0.25 cause a warning about *severely non-orthogonal faces*.

However, a simulation is impossible for much lower values. The simulation runs for the value pair 1.33 and 0.75. The values 1.4 and 0.714 cause the simulation to crash. The limits of stability of a simulation are therefore reached earlier than the limits of *checkMesh*.

To conclude this section, the user should bear the following statement in mind. Numerical problems of a simulation may be caused by bad mesh quality. In some cases – like the one presented above – bad mesh quality is

the root of the problem, but *checkMesh* issues no warnings. However, the values of the quality characteristics may give a hint. Some manuals of CFD software propose numerical ranges for characteristics like aspect ratio to ensure good quality.

```

Checking geometry...
  Overall domain bounding box (0 0 0) (0.1 0.1 0.01)
  Mesh (non-empty, non-wedge) directions (1 1 1)
  Mesh (non-empty) directions (1 1 1)
  Boundary openness (4.23516e-18 9.03502e-18 1.60936e-16) OK.
  Max cell openness = 1.67251e-16 OK.
  Max aspect ratio = 3.63059 OK.
  Minimum face area = 1.42648e-05. Maximum face area = 7.1694e-05. Face area magnitudes OK.
  Min volume = 1.03854e-07. Max volume = 1.69673e-07. Total volume = 0.0001. Cell volumes OK
  .
  Mesh non-orthogonality Max: 69.4798 average: 32.8092      Non-orthogonality check OK.
  Face pyramids OK.
  Max skewness = 2.35485 OK.
  Coupled point location match (average 0) OK.

Mesh OK.

End

```

Listing 143: *checkMesh* output for the distorted mesh; grading ratios 3 and 0.33

```

Checking geometry...
  Overall domain bounding box (0 0 0) (0.1 0.1 0.01)
  Mesh (non-empty, non-wedge) directions (1 1 1)
  Mesh (non-empty) directions (1 1 1)
  Boundary openness (4.23516e-18 -6.21157e-18 1.18585e-16) OK.
  Max cell openness = 2.37664e-16 OK.
  Max aspect ratio = 4.23706 OK.
  Minimum face area = 1.23181e-05. Maximum face area = 8.67874e-05. Face area magnitudes OK.
  Min volume = 1.00882e-07. Max volume = 1.84055e-07. Total volume = 0.0001. Cell volumes OK
  .
  Mesh non-orthogonality Max: 73.1635 average: 36.2131
  *Number of severely non-orthogonal faces: 80.
  Non-orthogonality check OK.
<<Writing 80 non-orthogonal faces to set nonOrthoFaces
  Face pyramids OK.
  Max skewness = 2.93978 OK.
  Coupled point location match (average 0) OK.

Mesh OK.

End

```

Listing 144: *checkMesh* output for the distorted mesh; grading ratios 4 and 0.25

19.2.3 Possible non-pitfall: twoInternalFacesCells

If a mesh for a two-dimensional simulation is created and checked using *checkMesh* with the *-allTopology* option enabled⁵⁵, then *checkMesh* will issue a message like in Listing 145. This message indicates, that there are cells present with only two internal faces. This message can be ignored when 2D meshes are concerned. The corner cells of a rectangular mesh have – by definition – only two internal faces.

```

Checking topology...
  Boundary definition OK.
  Cell to face addressing OK.
  Point usage OK.
  Upper triangular ordering OK.
  Face vertices OK.
  Topological cell zip-up check OK.
  Face-face connectivity OK.

```

⁵⁵When the *-allTopology* option is enabled, *checkMesh* performs two additional topological checks. Checking the face connectivity is one of these checks.


```
<<Writing 4 cells with two non-boundary faces to set twoInternalFacesCells
Number of regions: 1 (OK).
```

Listing 145: *checkMesh* output for a 2D mesh with *-allTopology* option set.

If this message appears when a 3D mesh is examined, then there is probably some error in the definition of the mesh. A cell in a 3D mesh should have at least three internal faces. A message stating the presence of cells with two internal faces in a 3D mesh indicates non-connected regions.

If there are, in a 3D mesh, cells present with only two internal faces – this is sometimes the case with tetrahedral meshes and the tet-cells in corners – then these cells will be written into a *twoInternalFacesCells* cell set, furthermore, these cells will also be written into a cell set named *underdeterminedCells*.

Listing 146 and Figure 32 show such an example. An all-tet, 3D-mesh has been checked by *checkMesh*, and it complained about cells with two internal faces, and underdetermined cells. In both cases the number of affected cells was the same. Comparing the actual cellSets, as shown in Listing 146, revealed that in-fact, all cells with two internal faces are considered under-determined, as the contents of the two files exactly match with the name of the cell set being the only exception.

```
gerhard@gerhardWork:/home/user/OpenFOAM/user-6/case-directory/constant/polyMesh/sets$ diff
twoInternalFacesCells underdeterminedCells
14c14
< object twoInternalFacesCells;
---
> object underdeterminedCells;
```

Listing 146: Comparing the cell sets *twoInternalFacesCells* and *underdeterminedCells* in a case with an all-tet, 3D-mesh.

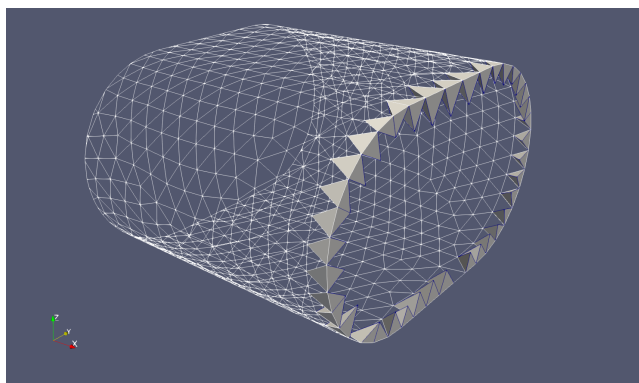


Figure 32: The cells with two internal faces in an all-tet, 3D-mesh.

19.2.4 Non-pitfall: underdetermined cells & aspect ratio

In certain cases, *checkMesh* reports under-determined cells, which are apparently the result of them having a relatively large aspect ratio, yet these cells pass the test of *checkMesh* for aspect ratio itself. Here, we discuss an example the author encountered when creating an axis-symmetric mesh. Listing 147 shows the output of *checkMesh*, which reports one failed test, the test for under-determined cells.

```
Checking geometry...
Overall domain bounding box (-1.87738e-14 -0.3105 -0.00628287) (0.359945 0.28176
0.00628287)
Mesh has 2 geometric (non-empty/wedge) directions (1 1 0)
Mesh has 3 solution (non-empty) directions (1 1 1)
Wedge back with angle 1 degrees
Wedge front with angle 1 degrees
All edges aligned with or perpendicular to non-empty directions.
Boundary openness (2.65459e-17 -2.56868e-15 -3.5724e-14) OK.
Max cell openness = 6.00189e-16 OK.
Max aspect ratio = 273.429 OK.
```

```

Minimum face area = 1.70066e-08. Maximum face area = 0.000559933. Face area magnitudes OK
.
Min volume = 9.31147e-12. Max volume = 6.7098e-06. Total volume = 0.000535521. Cell
volumes OK.
Mesh non-orthogonality Max: 35.0627 average: 10.6127
Non-orthogonality check OK.
Face pyramids OK.
Max skewness = 0.997295 OK.
Coupled point location match (average 0) OK.
Face tets OK.
Min/max edge length = 3.44218e-05 0.0349612 OK.
All angles in faces OK.
Face flatness (1 = flat, 0 = butterfly) : min = 1 average = 1
All face flatness OK.
Cell determinant (wellposedness) : minimum: 0.00012646 average: 2.46777
***Cells with small determinant (< 0.001) found, number of cells: 16
<<Writing 16 under-determined cells to set underdeterminedCells
Concave cell check OK.
Face interpolation weight : minimum: 0.289549 average: 0.486008
Face interpolation weight check OK.
Face volume ratio : minimum: 0.328662 average: 0.931884
Face volume ratio check OK.

Failed 1 mesh checks.

End

```

Listing 147: *checkMesh* output for a 2D, axi-symmetric mesh with `-allGeometry` and `-allTopology` option set.

In Figure 33, we see the under-determined cells, which were found by *checkMesh*. From Listing 147, we see that the cells have a high aspect ratio, with the maximum aspect ratio well above 250. However, *checkMesh* determined the test for the aspect ratio to be OK, yet *checkMesh* found under-determined cells. However, these cells, depicted in Figure 33 are regular hex cells, although quite thin ones.

As the simulation, this mesh was created for, runs perfectly fine, the reported under-determined cells pose no problem for the numerics of OpenFOAM. Most likely some threshold value was exceeded by these high-aspect ratio cells, which triggered *checkMesh* to report under-determined cells.

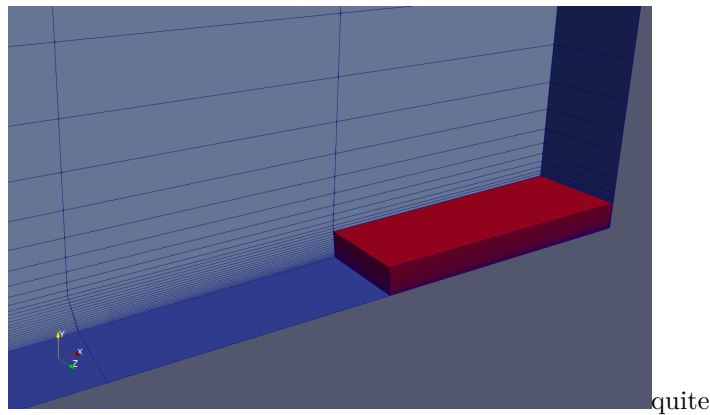


Figure 33: The under-determined cells, which were found by *checkMesh* in the mesh of an axi-symmetric simulation. These cells are on a far corner of the 2-D domain, and grading towards the more interesting regions led them to have a high aspect ratio. In fact, these cells have the highest aspect ratio of the whole mesh. The proximity to the lower wall results in these cells to be quite fine in the *y*-direction. Since, these cells are furthest from the axis of symmetry, they are relatively large in radial and tangential direction.

19.3 Useful output

The output of *checkMesh* in Listing 145 also shows another interesting thing to know about *checkMesh*. The line `<<Writing 4 cells with two non-boundary faces to set twoInternalFacesCells` tells the user that *checkMesh* created a set of cells that are found to have some problems.

Figure 34 shows the content of the case which resulted in Figure 31. There we see a directory named **sets** inside the **polyMesh** folder. The **sets** folder was created by *checkMesh* and inside this folder *checkMesh* stores any sets it creates. The file names are rather self-explanatory, e.g. the file **skewFaces** contains all faces which failed the test for skewness. All these cell or face sets can be viewed with *paraView*.

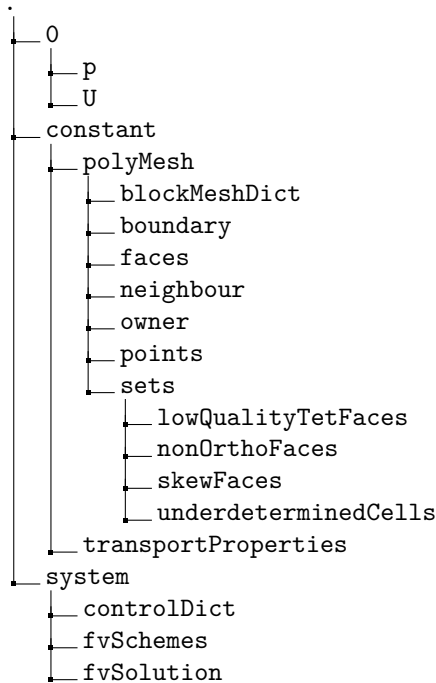


Figure 34: Sets created by *checkMesh* in the **sets** directory.

20 *extrudeMesh*

extrudeMesh is a rather special tool. OpenFOAM lists *extrudeMesh* under the mesh generation tools, however, *extrudeMesh* has a role between mesh generation and mesh manipulation. We can do mesh generation, e.g. extruding one cell layer from a 2D STL surface in order to prepare the mesh for a 2D study in OpenFOAM. However, we can also do mesh manipulation, which is essentially mesh extension, as we “grow” cell layers on surfaces.

20.1 Control

extrudeMesh is controlled by the file **extrudeMeshDict**. This file contains all necessary settings for using this tool, which can roughly be divided into the categories: “where to grow”, “what to grow”, and “how to grow”.

20.1.1 constructFrom

The **constructFrom** setting is used to determine the source of the extrusion. This basis for cell extrusion can be either a patch of an existing mesh or an STL surface. In the case of a patch of a mesh, the source may also be a patch from another case, e.g. extrude patch X from case Y to create the mesh of case Z.

For the case of an STL surface, the corresponding STL file needs to be provided using the **surface** keyword. If we use the patch of a mesh, we can choose between retaining the source mesh or discarding it. Figure 35 shows the difference between these two options, when extruding a patch from the current case.

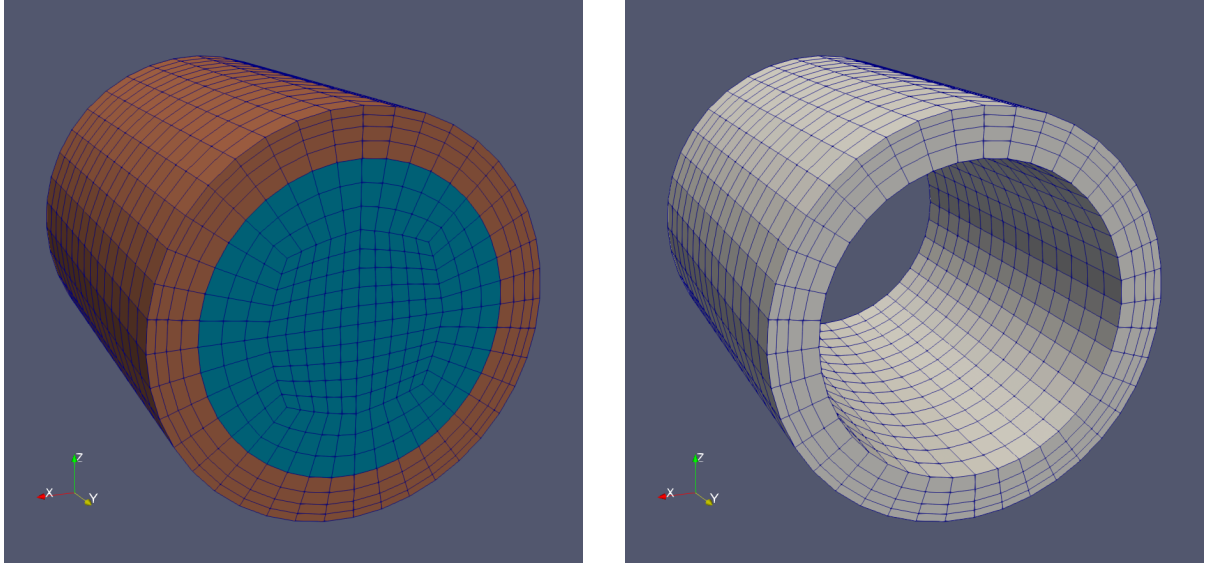


Figure 35: Extrude the wall patch from a cylinder mesh. left: `constructFrom mesh`. right: `constructFrom patch`.

20.1.2 Layer control

The “what to grow” part consists of the number of cells in thickness direction of the new cell layer, the thickness and an optional expansion ratio.

expansionRatio

Note, that the expansion ratio describes the expansion of thickness from one layer to the other. This is in contrast to the expansion ratio we use with the grading feature of blockMesh, there the expansion ratio describes the thickness ratio between the smallest and the largest cells at the boundaries of a block.

20.1.3 Extrusion models

The extrusion models control the “how to grow”. There is a number of models available, some of which will be discussed below.

Plane extrusion

The plane extrusion model is specifically for the creation of (quasi) 2D meshes. A single layer of cells is extruded in normal direction to the provided surface. By default the front and back patches are created to be of type `empty`.

In Figure 36 we see the mesh created from an STL, which was created by GMSH. In this case we could also have used GMSH to create a mesh with a single cell in thickness direction.

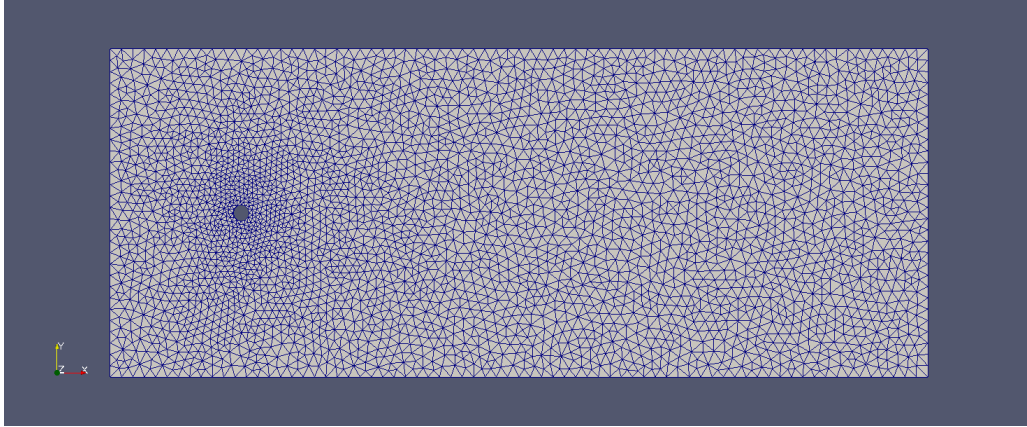


Figure 36: The mesh for a 2D study generated from an STL surface.

Sector extrusion

Figure 37 shows the result of the *sector* extrusion model. For this model, the user needs to specify a point in space (**axisPnt**), an axis of rotation (**axis**) and an **angle**. In this case the outlet patch of the original mesh (shown in grey) was extruded. The original mesh was created by *blockMesh* and consists of 5 blocks (easily scripted with e.g. Python). The **axisPnt** lies in the plane of the outlet patch, however, the point is well outside the patch. The distance between the **axisPnt** and the centerline of the original pipe mesh determines the radius of the pipe bend. The positive x-axis was selected as **axis**. The newly generated cells are by default added to a *cellSet* named *addedCells*.

This use of *extrudeMesh* opens a rather cheap way to create good meshes of pipe bends. The **blockMeshDict** for a straight pipe is easily scripted, and by extruding along the section of a circle, the mesh is continued along a bend. Directly scripting the **blockMeshDict** for a pipe bend would definitely be a little bit harder.

The **sector** extrusion model also has a 2D “cousin”, which is called **wedge**. The class underlying the **wedge** extrusion model is derived from the **sector** model. However, the **wedge** model is the axisymmetric analogue of the **plane** model. Thus, only one cell layer is created, which is centered about the source surface, i.e. the cell layer is extruded half the **angle** in both directions from the source surface.

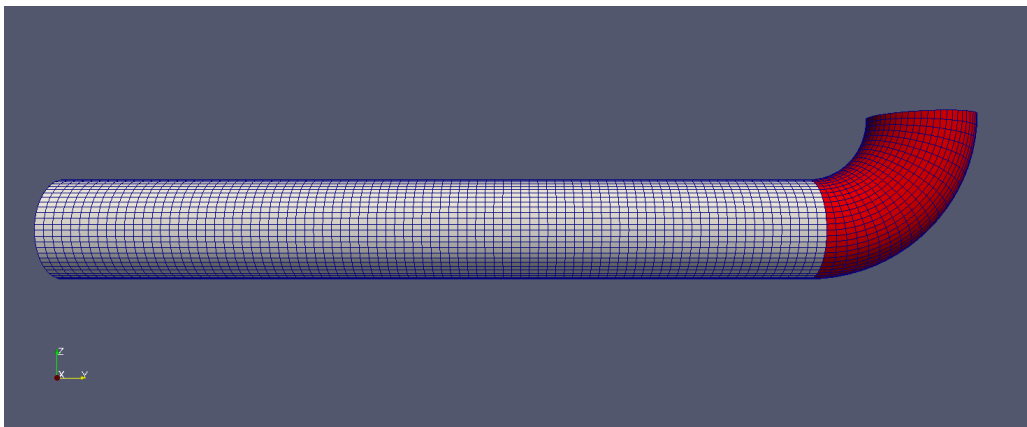


Figure 37: A cheap 90° pipe bend. The outlet patch of the original mesh was extruded along the sector of a circle.

Linear extrusion

There are two models for linear extrusion in *extrudeMesh*. There is **linearNormal**, which extrudes in normal direction of the underlying surface. This can be used to grow a cell layer on the pipe’s wall, see Figure 39. Furthermore, there is **linearDirection**, which extrudes cells along a specified direction.

In Figure 38 we see the result of subsequent use of *extrudeMesh*. Unfortunately, at the time of writing (using OpenFOAM-4.0), *extrudeMesh* does not offer the `-dict` option. Thus, we need to repeatedly edit the `extrudeMeshDict` for subsequent applications of *extrudeMesh*. First, a bend was created by using the `sector` model. Afterwards a straight pipe section was created using `linearNormal`, which is followed by a slanted pipe section, which was created using `linearDirection`.

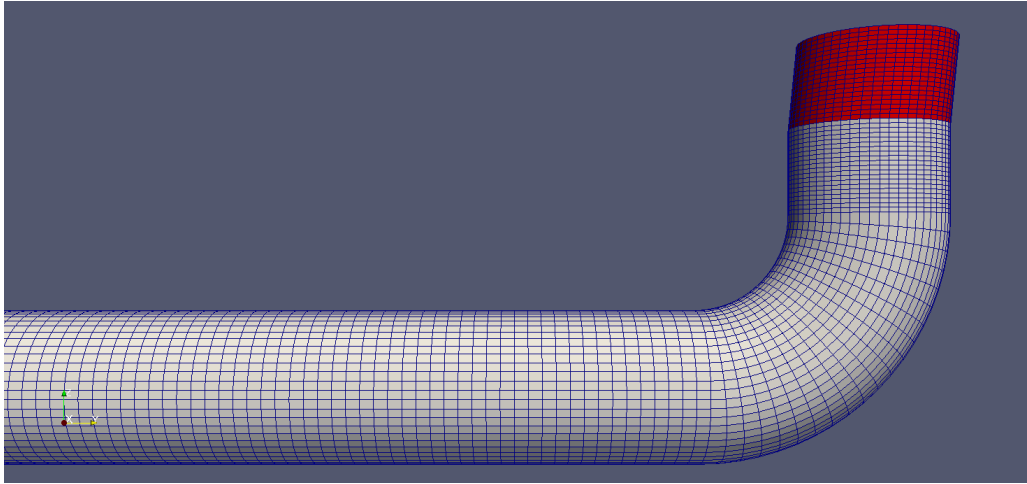


Figure 38: Subsequent mesh extrusions: `sector`, `linearNormal` and `linearDirection`.

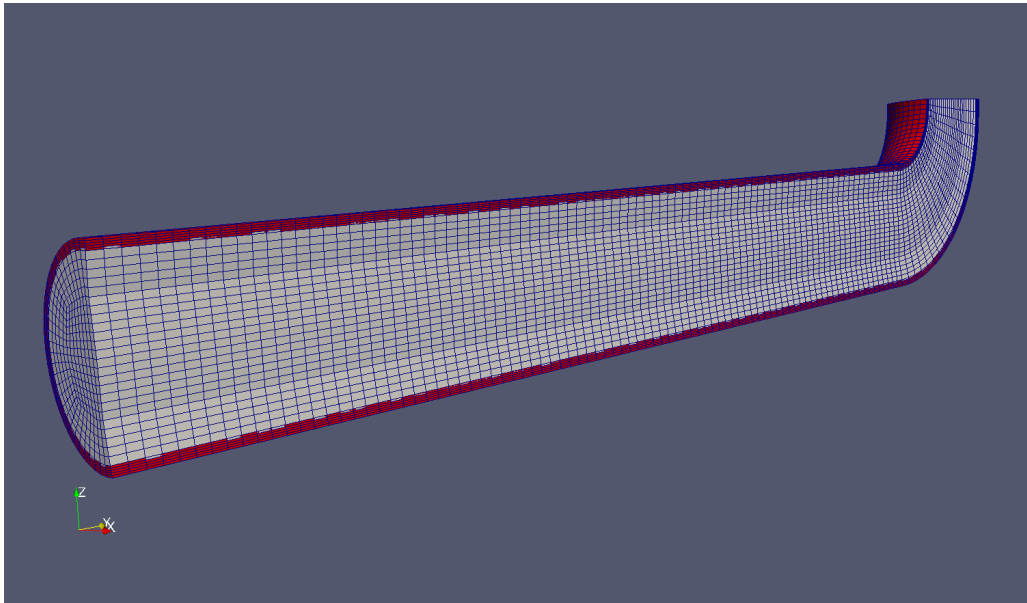


Figure 39: Grow a wall! The *walls* patch of the pipe mesh was extruded using the `linearNormal` model.

Pitfall: extruding a too narrow 2D slice

The mesh for an axis-symmetric simulation is a slice of the full 3D domain, which is 1 cell in circumferential direction. Figure 40 shows two examples, which differ only in the angle of the slice. Both seem like valid meshes, when viewed in ParaView.

Yet, with the narrow angle, OpenFOAM complains of the wedge-type patches being non-planar. Listing 148 shows an example of such a warning message. Increasing the angle from 1 to 5 degrees solved this problem. This problem could not be solved by increasing the write precision to large values.

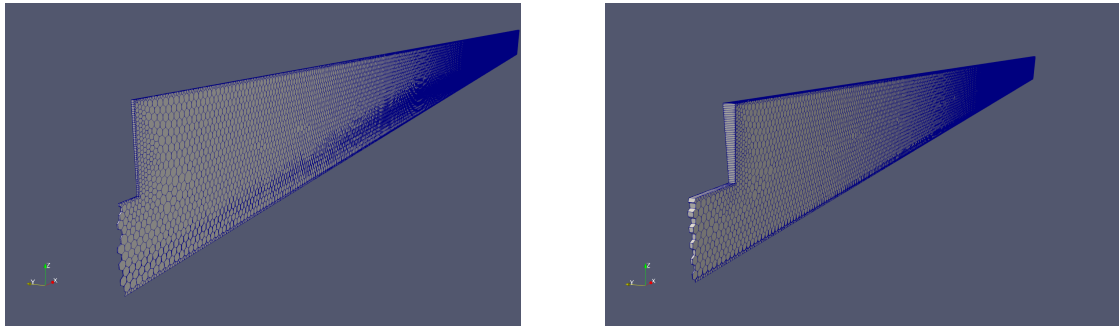


Figure 40: Extruding a slice. **Left:** a 1 degree sector extrusion, **Right:** a 5 degrees sector extrusion.

```
--> FOAM Warning :
    From function virtual void Foam::wedgePolyPatch::calcGeometry(Foam::PstreamBuffers&)
    in file meshes/polyMesh/polyPatches/constraint/wedge/wedgePolyPatch.C at line 70
    Wedge patch 'front' is not planar.
At local face at (5.433932e-05 -4.157483e-07 4.764003e-05) the normal (5.662783e-19 -0.9999619
-0.008726535)
differs from the average normal (-1.335286e-10 -0.9999618 -0.008682201) by 1.965596e-09
Either correct the patch or split it into planar parts
```

Listing 148: Warning message about a non-planar wedge-type patch.

21 polyDualMesh

This tool falls into a similar category as `extrudeMesh`. While it operates on an existing mesh, the result is very different from the initial mesh, so that this tool can be considered a mesh generation tool.

`polyDualMesh` takes any valid mesh and computes the dual of the provided mesh. As the dual of a mesh is a rather abstract concept, this is best explained by a simple example. In Figure 41, the Voronoi diagram is shown. In this example the centers of the circumcircles of a triangular grid are used to create the Voronoi diagram. Thus, a polygonal grid can be created from a triangular grid.

`polyDualMesh` applies this approach to the provided mesh, which can be of any type.

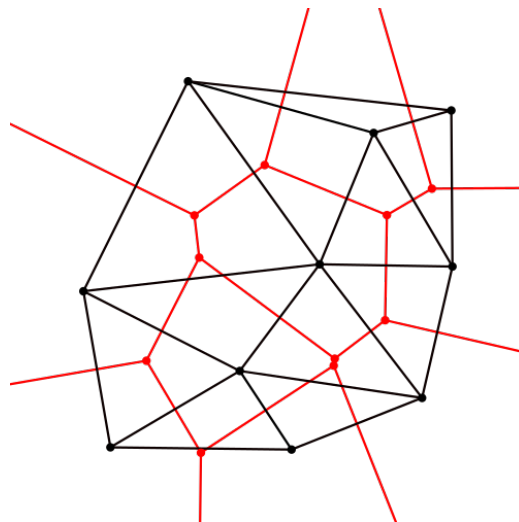


Figure 41: Connecting the centers of the circumcircles produces the Voronoi diagram (in red). Source https://commons.wikimedia.org/wiki/File:Delaunay_Voronoi.svg.

`polyDualMesh` is a vital tool to create polyhedral meshes. Since it takes tetrahedral (tet) meshes as its input, and tools to create tet meshes are plenty, `polyDualMesh` is our gateway to running simulations on polyhedral

meshes. Generally we can state, that the more elaborate meshes are to be, the rarer (in case of open source solutions) or the costlier (in case of proprietary tools) the mesh generation software is.

Example: the elbow tutorial

Figure 42 shows the result of running `polyDualMesh` on the *elbow* tutorial case. The initial mesh is a triangular mesh, and the resulting mesh is a mesh of polygons. Note, that OpenFOAM implements a 2D mesh as a 3D with a thickness of a single cell. Since `polyDualMesh` acts in all spatial dimensions, the resulting mesh is not a valid 2D mesh anymore. However, if we extrude the top patch as a single cell layer, and delete all old cells, we end up with a valid 2D mesh again.

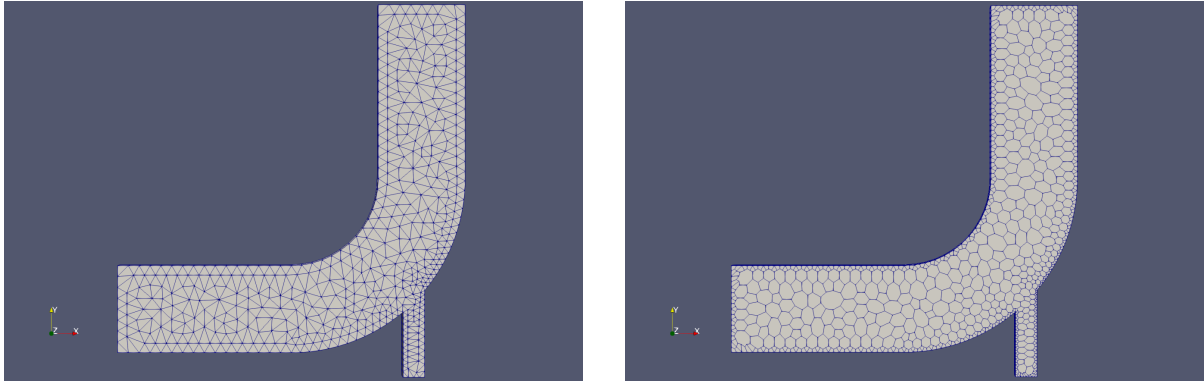


Figure 42: The mesh of the *elbow* tutorial case; before and after the application of `polyDualMesh`.

Example: 2D bubble column tutorial

`polyDualMesh` can take any valid mesh as its input. In Figure 43, on the right, we see the result of applying `polyDualMesh` on a 2D all-hex mesh. This example also shows how the resulting mesh is not a valid 2D mesh anymore, since the resulting mesh is 2 cells in thickness. The resulting mesh is hexahedral-dominant, since at the boundary polyhedrals are created, which resemble hexahedrals with intermediate hanging nodes.

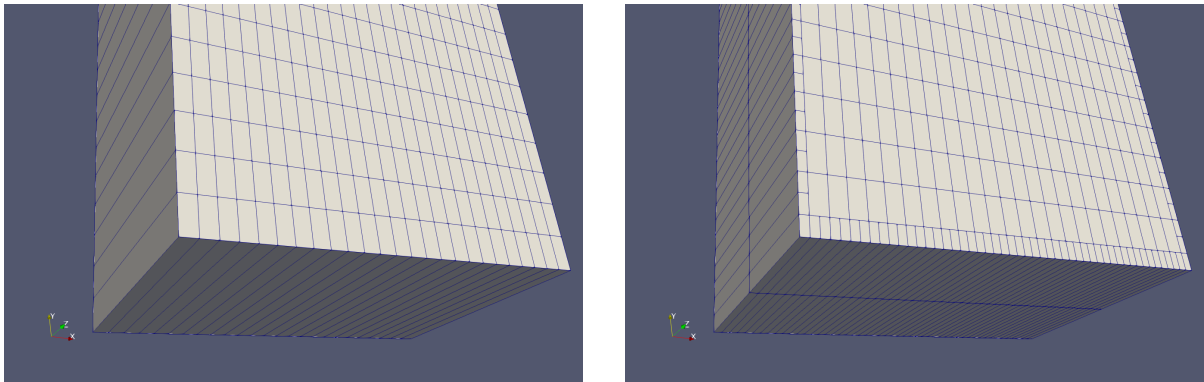


Figure 43: The mesh of the *bubble column* tutorial case; before and after the application of `polyDualMesh`.

Example: single cells

For curiosity's sake, we take the dual mesh of individual cells.

In Figure 44 we see the result of creating the dual mesh of an initial mesh consisting of a single tetrahedral cell. We see from the face decomposition, that the face-centroids are used as vertices for the new cells of the dual mesh. In addition with the volume-centroid, new cells can be constructed. As there are no cells to recombine the newly created cells, we end up with 4 hex-cells.

In Figure 45 we see the dual mesh of a mesh consisting of a single hexahedral cell. Here, we end up with 8 hex-cells.

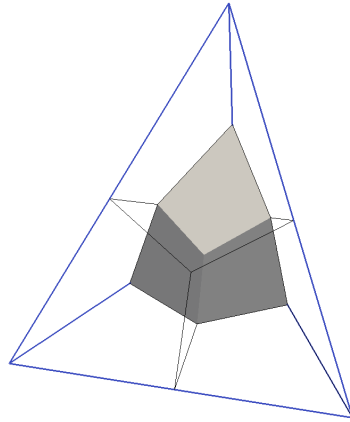


Figure 44: The dual mesh of a single tetrahedron: the original tet-cell is outlined in blue, the face-decomposition is outlined in black, and one of the resulting cells is shown in grey.

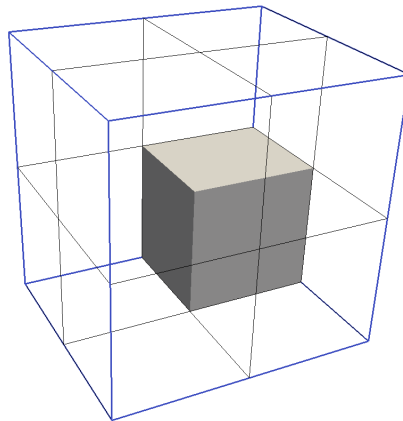


Figure 45: The dual mesh of a single hexahedron: the original hex-cell is outlined in blue, the face-decomposition is outlined in black, and one of the resulting cells is shown in grey.

22 combinePatchFaces

This mesh manipulation tool checks for multiple patch faces on same cell and combines them. Having multiple patch faces on the same cell might be result of some mesh manipulation operations, e.g. polyDualMesh in the example of the elbow tutorial created such cells at the domain boundary.

Example: the elbow tutorial

In Figure 42, on the right, we see the dual mesh of the elbow tutorial case. It appears as if at the boundary the cells were a lot smaller than within the domain. However, at close inspection, we see that the cells at the boundary have multiple faces belonging to the same patch. Thus, if we run combinePatchFaces, all of these multiple patch faces are combined, as we see in Figure 46 on the right. Now the cells at the boundary do not appear to be much smaller than the cells in the interior.

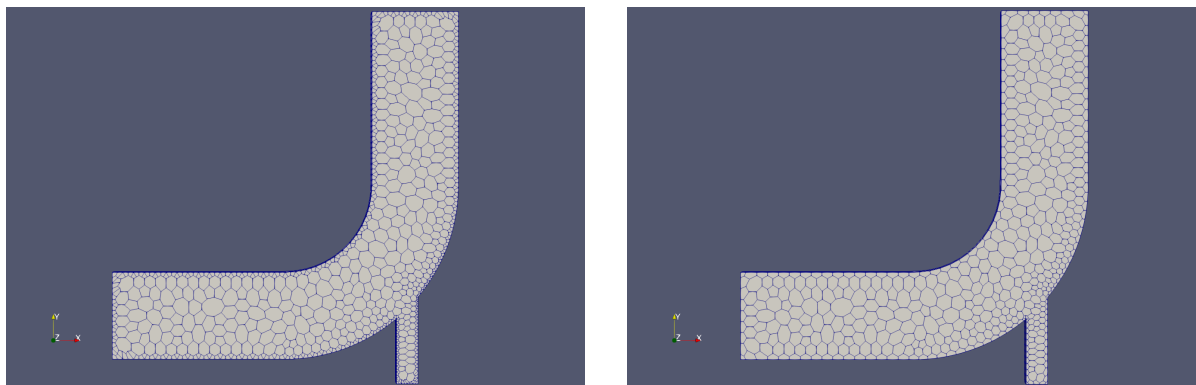


Figure 46: The dual mesh of the *elbow* tutorial case; before and after the application of `combinePatchFaces`.

23 Salome

The Salome platform is a powerful multitool, which features a meshing module. This meshing module offers a number of meshing tools.

23.1 Export & Conversion

Meshes can be exported by Salome into several formats. The go-to procedure for OpenFOAM-use is to export the mesh in the UNV format and use the `ideasUnvToFoam` mesh converter.

23.1.1 Salome's native UNV export

However, there is an issue when using the UNV format. Apparently, see Figure 47, Salome's mesh export tool for the UNV format does not export pyramid cells.

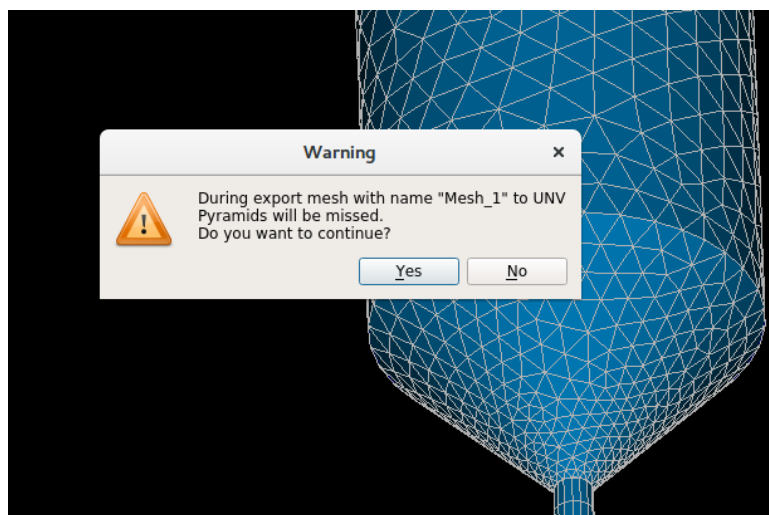


Figure 47: Mesh export issue in Salome with the UNV format.

23.1.2 salomeToOpenFOAM

A third-party Python script⁵⁶ can be used to export a mesh containing pyramid cells to OpenFOAM. This script directly writes the essential files⁵⁷. In order to export a mesh, simply select it in the Object Browser of Salome and then execute the `salomeToOpenFOAM.py` Python script. This can be done by using `File` `Load Script` menu.

⁵⁶<https://github.com/nicolasedh/salomeToOpenFOAM>

⁵⁷The files `boundary`, `faces`, `neighbour`, `owner` and `points` in `constant/polyMesh`.

23.1.3 Pitfall: Check your patches/patch types

In Salome we are able to define patches by creating face groups. These face groups are translated to patches, when exporting the mesh. However, Salome has no way to distinguish between general patches and wall patches. Thus, you may want to check `constant/polyMesh/boundary` for the patch types.

With Salome's native export to UNV function, all patches are equal. After importing with OpenFOAM's native `ideasUnvToFoam` converter all patches are of the type `patch`. You need to take care yourself, to assign the type `wall` to wall patches⁵⁸. This can be done manually or with the tool `changeDictionary`. See Section 30.1 for more information on `changeDictionary` and an example of how to change the patch type.

The third party conversion script `salomeToOpenFOAM.py` makes all patches wall patches, when their names contain the word `wall`.

```
1 if "wall" in gname.lower():
2     fileBoundary.write("wall;\n")
3 else:
4     fileBoundary.write("patch;\n")
```

Listing 149: Determining the patch type in `salomeToOpenFOAM.py`.

23.2 Combined operations: blockMesh & salome

23.2.1 Motivation

Figure 48 shows a simulation domain with a comparatively small geometric feature. If we were to create a `blockMesh`-mesh for this geometry we end up with plenty of rather useless fine cells next to that fine feature, as can be seen in Figure 49. Note that in Figure 49 we even can not make out individual cells, or even the cut-out slot in the center.

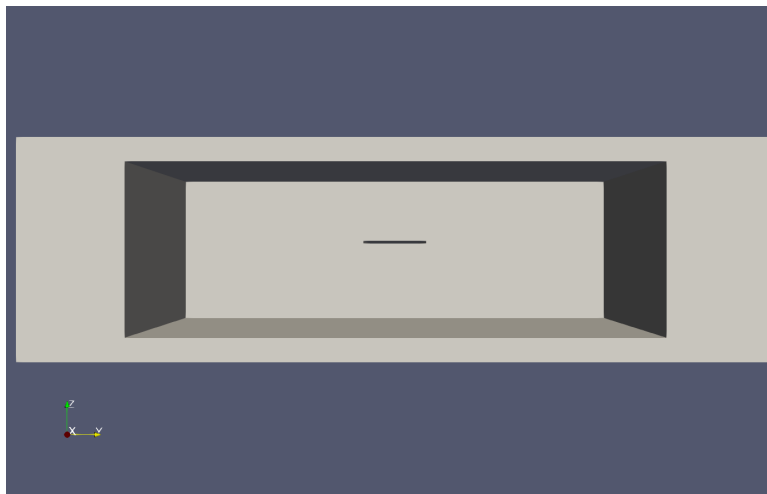


Figure 48: A big simulation domain with a quite small geometric feature: the geometry.

⁵⁸If this has not been done, the wall functions of turbulent simulations will complain about patch/data types.

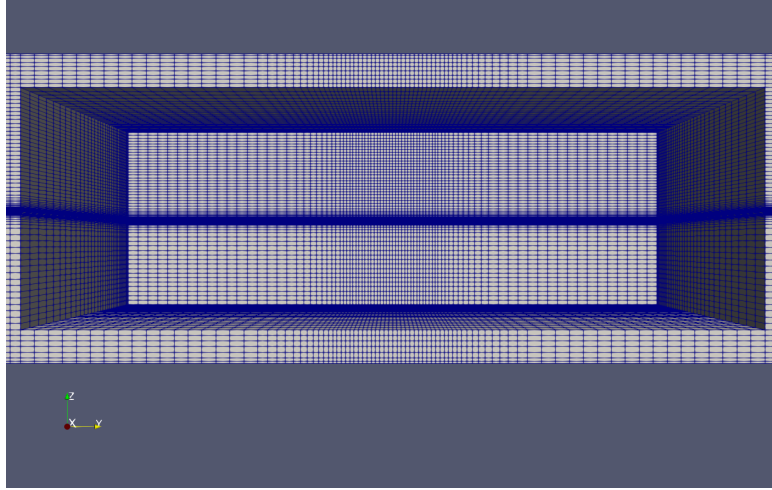


Figure 49: A big simulation domain with a quite small geometric feature: the *blockMesh*-only mesh.

23.2.2 Workflow

An alternative to an outcome as shown in Figure 49, we could choose to create a bigger central block which encompasses the cut-out slot, instead of representing the cut-out. This cut-out block will remain a part of the mesh, and then we will split the mesh into the cut-out block and the remaining mesh.

Then, we create a geometry representation of the cut-out block with the cut-out slot in Salome. When, we mesh the cut-out block, we follow this workflow.

1. Create groups on the geometry: the sides, the front face and the inner edges
2. Import the surface mesh from the meshed cut-out block, as shown in Figure 50
3. Mesh the geometry with the following sub-meshes, preferably in this order
 - (a) The sides with the “Import 1D-2D Elements from Another Mesh” algorithm
 - (b) The inner edges with a suitable 1D algorithm
 - (c) The front face with a suitable 2D algorithm
 - (d) The geometry with the “Extrusion 3D” algorithm

Figure 51 shows the mesh, which we created in Salome.

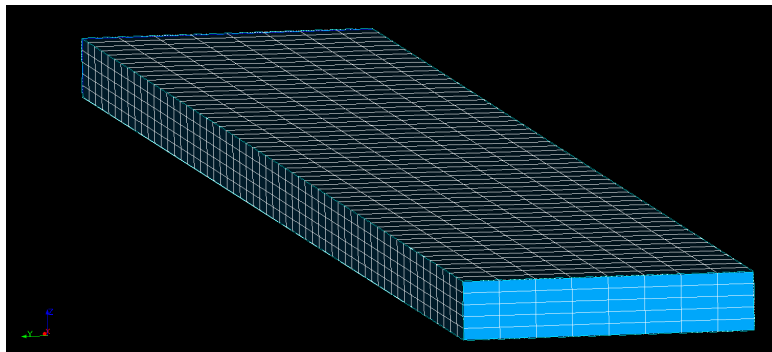


Figure 50: The mesh of the cut-out block in salome.

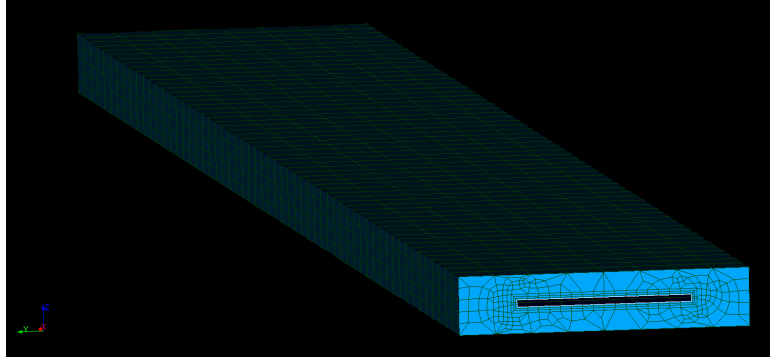


Figure 51: Meshing of a block in salome.

Finally, we need to export our mesh from Salome into OpenFOAM, and combine it with the mesh of the remaining domain. Figure 52 shows the result of our procedure. Thus, we are able to create a relatively fine mesh around the fine geometric feature, without the mesh fine-ness spreading out along the principal directions as in the example shown in Figure 49.

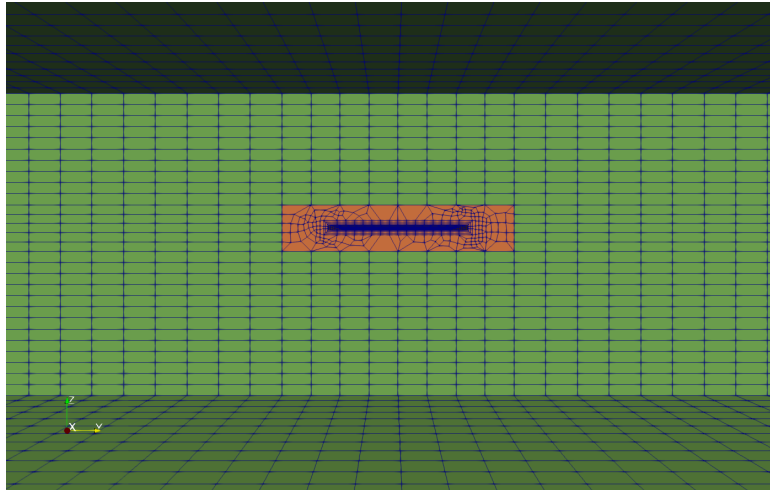


Figure 52: Combining blockMesh and salome: the meshed block was re-combined with the initial mesh.

24 Gmsh

Gmsh⁵⁹ is a 3D finite element meshing software. Gmsh is operated via its GUI or via ASCII input files in Gmsh's own scripting language. Gmsh is able to create all cell shapes from tets to hexes. The meshes generated by GMSH can be converted to OpenFOAM's format using the *gmshToFoam* utility.

⁵⁹<http://gmsh.info/>

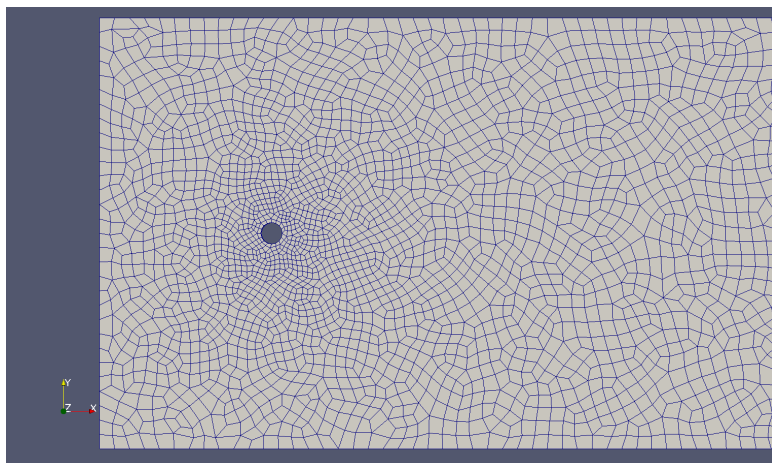


Figure 53: An extruded 2D mesh of quad elements created with Gmsh.

25 enGrid

enGrid⁶⁰ is an open source mesh generation software with CFD applications in mind. It uses the **netgen**⁶¹ meshing library. enGrid primarily creates tet meshes, however, it also allows for the creation of prismatic boundary layers and the conversion of tets to polyhedras. enGrid natively exports its meshes to OpenFOAM. enGrid is operated via its GUI.

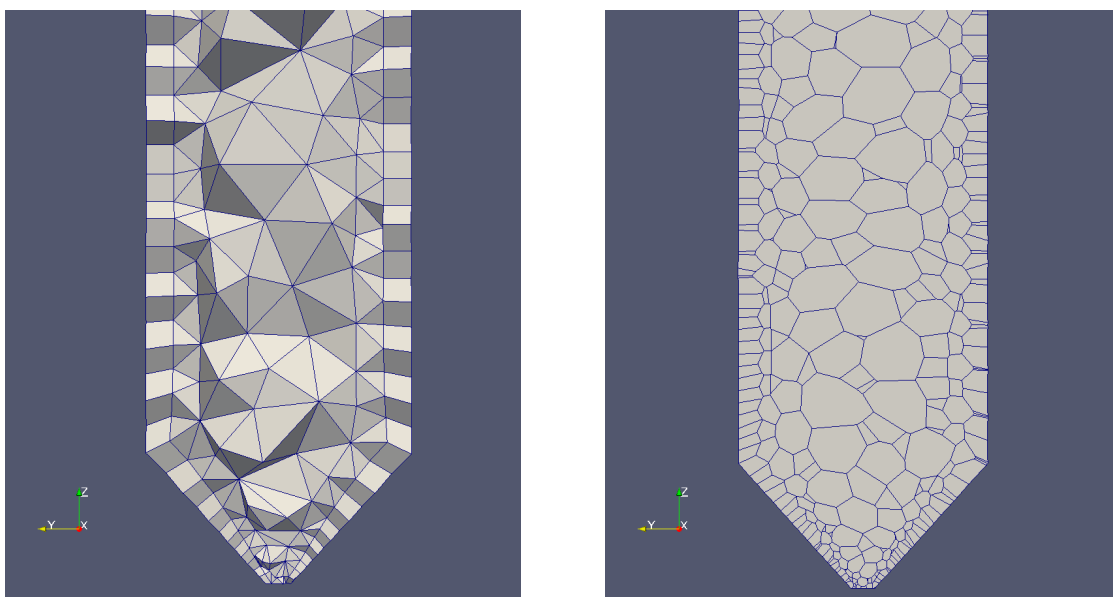


Figure 54: Meshes by enGrid: **left**: tet-mesh with prismatic boundary layer, **right**: polyhedral mesh with boundary layer.

26 Mesh converters

To use meshes created by programs other than *blockMesh* there is a number of converters. The User Guide [50] lists the following converters:

- *fluentMeshToFoam*

⁶⁰<https://github.com/enGits/engrid/wiki>

⁶¹<https://sourceforge.net/projects/netgen-mesher/>

- *starToFoam*
- *gambitToFoam*
- *ideasToFoam*
- *cfx4ToFoam*

The names of the converters are pretty self explanatory.

General recommendations

Always write in ASCII format

Any data exchange between your current installation of OpenFOAM and any other software is best handled in ASCII format.

26.1 *fluentMeshToFoam* and *fluent3DMeshToFoam*

fluentMeshToFoam converts meshes stored in the *.msh file format into the format of OpenFOAM. To be more specific, *fluentMeshToFoam* converts only 2D meshes, whereas 3D meshes can be converted using *fluent3DMeshToFoam*.

The converter expects the path to the *.msh file as an argument. The converter saves the mesh in the format of OpenFOAM in the `constant/polymesh` directory.

If converter is invoked from a directory other than the case directory, then the path to the case directory has to be specified via an additional argument. See Section 12.6.

If the mesh was created using an other dimension than in metres, the command line parameter `-scale` can be used to correct the scaling. OpenFOAM expects the mesh data to be expressed in metres.

All other possible option can be displayed with this command line parameter `fluentMeshToFoam -help`.

26.2 *ideasUnvToFoam*

ideasUnvToFoam is a converter which is commonly used to convert meshes in the UNV format to OpenFOAM's format, Salome is a well-known meshing software, which exports meshes in the UNV format, see Section 23.

26.3 Pitfall: length units

Third party meshes may be based on millimetres instead of metres as expected by OpenFOAM. Point coordinates will be interpreted by OpenFOAM as being expressed in metres, i.e. $P_{\text{in file}} = (120, 240, \text{left}(120\ 0))$ mm will be read by OpenFOAM as $P_{\text{in mesh}} = (120, 240, 0)$ m. Thus, the imported mesh will be scaled by a factor of 1000.

Some mesh converters (e.g. *fluent3DMeshToFoam*) offer a `-scale` option to fix the length scales along with mesh conversion. Other mesh converters (e.g. *ideasUnvToFoam*) do not offer such a scaling function. However, there is a utility (*transformPoints*) which, among other tasks, can be used to correct the length scales of the mesh.



Always run `checkMesh`, ideally with its options `-allGeometry` and `-allTopology`, to, first, check the mesh quality, and secondly, to check the mesh bounding box. The bounding box will be expressed in metres, as any other length in OpenFOAM. This will give you a chance to spot a millimetre vs. metre situation.

Listing 150 shows the relevant lines of *checkMesh*'s output. Unless, we calculate meteorological flows, a simulation domain in kilometre scale seems a bit off.

```
Checking geometry...
Overall domain bounding box (-752.264 -325 -684.294) (752.264 1400 3754.35)
```

Listing 150: The bounding box of our mesh

27 Other mesh manipulation tools

27.1 *transformPoints*

The tool *transformPoints* can be used to scale, translate or rotate the points a mesh. Section 29.3.4 contains a case in which this tool can be useful.

27.1.1 Rotation

Rotating the geometry can be specified in two ways.

Using two vectors

Here, the rotation is defined by providing a vector before and after the rotation. From these two vectors, the transformation matrix can be computed

Yaw, Pitch and Roll

There are actually two options: `rollPitchYaw` and `yawPitchRoll`. Here, the rotation is defined by specifying three angles in degrees, which are subsequently applied to the x-axis, the y-axis and the z-axis.

27.2 *topoSet*

The tool *topoSet* creates point, face or cell sets from a geometric definition. There are a number of ways to define the geometric region containing the intended points, faces or cells.

27.2.1 Usage

The dictionary `topoSetDict` is used to define the geometric region. Find some examples in the tutorials using the following command.

```
find $FOAM_TUTORIALS -name topoSetDict
```

Listing 151: Find examples for the use of *topoSet*

A face or cell set will contain only faces or cells whose centres lie within the specified geometric region.

27.2.2 Life hack: apply *topoSet* on decomposed cases

The tool *topoSet* can be applied to decomposed cases, by running the tool in parallel in the same fashion as we would run a solver in parallel.

```
mpirun -np 4 topoSet -parallel
```

Listing 152: Run *topoSet* in parallel, i.e. apply *topoSet* on a decomposed case.

This will apply all definitions for the creation of sets and zones to the sub-domains of the decomposed case.

This can also be done with cases that are currently running, e.g. as a long-duration simulation is running, we are preparing sets and zones for the subsequent post-processing. However, if we apply *topoSet* on a case, which is at that time being simulated, we need to take extra care not to alter, remove or mess in any other way with already existing sets or zones.

The sets and zones we created for the sub-domains of the decomposed case need to be reconstructed, so that they are present when the case is finished, and the subsequent post-processing is done on the reconstructed case. We can reconstruct sets and zones, which were created with the decomposed case, by calling *reconstructPar* with the `-constant` command line argument.

27.2.3 Pitfall: The definition of the geometric region

To demonstrate the function of `topoSet` a cell set was defined for the cavity tutorial-case. The mesh of the cavity case is $1 \times 1 \times 0.1$ m and the box defining the cell set was chosen to be $0.5 \times 0.5 \times 0.05$ m. The dimensions of this box are simply half the dimensions of the mesh. However, only cells whose cell centre is located in the box are contained in the cell set. As the mesh is one cell in depth and 0.1 m in depth, all the cell centres are exactly at $z = 0.05$ m. Due to inevitable numerical errors in calculating the cell centre⁶², the numerical errors decided whether a cell was included into the cell set or not.

To avoid this error, always make sure the geometric region contains all the intended cells.

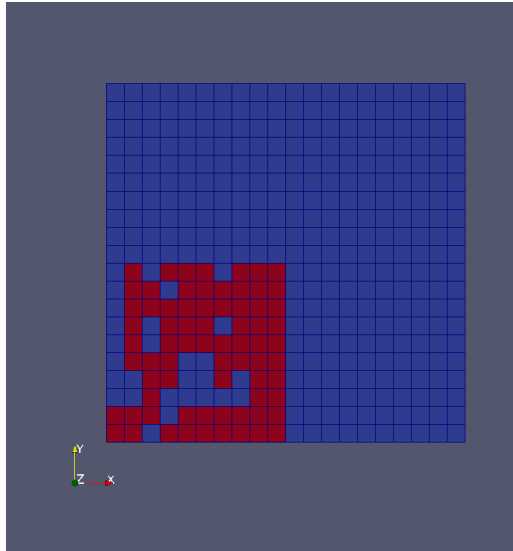


Figure 55: A faulty cell set definition. The red cells are part of the cell set. All other cells are blue.

27.2.4 Pitfall: the face-normal of internal faces

For simulations with run-time post-processing, we might need sets of internal faces, e.g. for extracting the pressure at certain cross-sectional planes. With *topoSet* we can create sets of internal faces by using the `boxToFace` source to select all faces in a slice of geometry, and the `normalToFace` source to select all faces with a face-normal parallel to the normal of the cross-sectional plane in question, and finally perform a boolean operation to eliminate all faces not oriented parallel to the cross-sectional plane.

While this procedure requires a bit of trial-and-error to determine the proper size of the box to include only a single layer of faces parallel to the cross-sectional plane, the procedure is generally appropriate for the task in question. However, there is one flaw in the discussed procedure: the orientation of the face-normal vectors of internal faces.

If we imagine a pipe with its principal axis parallel to the x axis, then the internal faces we need for our cross-sectional plane have their face-normals oriented parallel to the x axis. However, the face-normals of the internal faces may point into the positive x direction or into the negative x direction. Thus, our procedure from above needs to be extended by a second use of the `normalToFace` source, this time with the opposite normal vector.

27.2.5 Legacy pitfall: renumbered mesh

At the point of writing, using OpenFOAM-2.something, the utility *renumberMesh* does not consider cell sets⁶³. If *renumberMesh* is called after cell sets were created by *topoSet*, the cell set is invalid. The reason for this is, that the cell labels of the cell set remain unchanged as *renumberMesh* completely relabels the mesh. Thus, the cell set still exists and the number of cells is unchanged, however, as other cells bear the labels of the original members of the cell set, the cell set is invalid.

⁶²The location of the cell centre is not stored in any file, thus this quantity has to be computed.

⁶³This behaviour was reported in bug report 1377 (<http://openfoam.org/mantisbt/view.php?id=1377>), and has been fixed for OpenFOAM-3.0 and later versions.

To resolve this problem, *topoSet* needs to be run after *renumberMesh*. This even works in parallel, when the case has been decomposed.

27.3 *setsToZones*

The utility *setsToZones* serves the purpose to:

Add *pointZones*/*faceZones*/*cellZones* to the mesh from similar named *pointSets*/*faceSets*/*cellSets* [50].

This utility is needed when we create some *cellSets* which we later want to use e.g. with a *functionObject* (the *cellSource* *functionObject* acts on all cells or on a *cellZone*). *cellSets* can be created with *topoSet*. After we ran *topoSet* we simply run *setsToZones* without any further parameters or providing a dictionary. *setsToZones* creates *cellZones* which contain the same cells as the corresponding *cellSets*.

27.4 *refineMesh*

The tool *refineMesh* is used – just as the name suggests – to refine a mesh.

27.4.1 Usage

First a cell set has to be defined, this can be done using the tool *topoSet*.

With the dictionary **refineMeshDict** the rules for refining a particular cell set can be stated. When rules have been defined in **refineMeshDict**, then the command line option **-dict** has to be used.

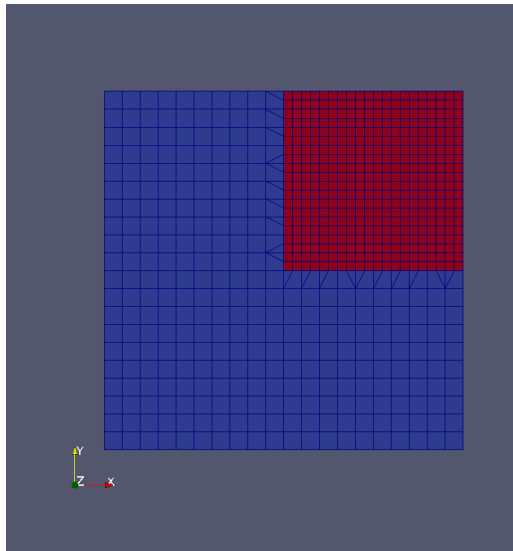


Figure 56: An example of a refined mesh. The refined region is marked in red.

27.4.2 Pitfall: no command line parameters

If the tool *refineMesh* is called without any command line parameters then the whole mesh is refined. For *refineMesh* to obey the rules set in the **refineMeshDict** the command line option **-dict** has to be used when calling *refineMesh*. See this useful post in the CFD-Online Forum <http://www.cfd-online.com/Forums/openfoam-meshing-utilities/61518-blockmesh-cellset-refinemesh.html#post195725>

Notice the different meaning of the **-dict** command line option of the tools *topoSet* and *refineMesh*. If you are in doubt about this difference, check the summary of the command line usage printed by the **-help** option.

27.5 *refineWallLayer*

refineWallLayer is a tool to refine cells that are adjacent to a set of user-specified patches.

27.5.1 Control

The list of patches is provided by the user via a command line argument. The next argument is the edge fraction, which is to be applied in the refinement. Listing 153 shows an example of how this tool is called. The list of patches and the edge fraction are mandatory arguments.

```
refineWallLayer -overwrite (top bottom sideLeft) 0.4
```

Listing 153: Invoking `refineWallLayer` for the cells adjacent to three patches.

Figures 57, 58 and 59 show the results of a successive application of this tool along with the base mesh. In this case, the patch of the inner void is refined. Figures 58 and 59 show the treatment of sharp, concave edges.

Figure 60 shows the result of `refineWallLayer` when the cells belonging only one patch are refined.

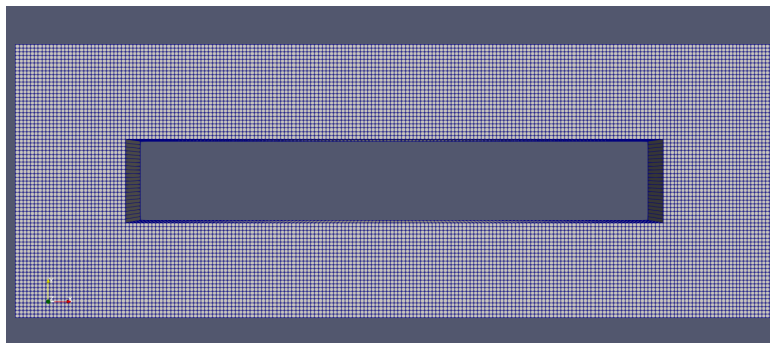


Figure 57: The base mesh for the wall layer refinement.

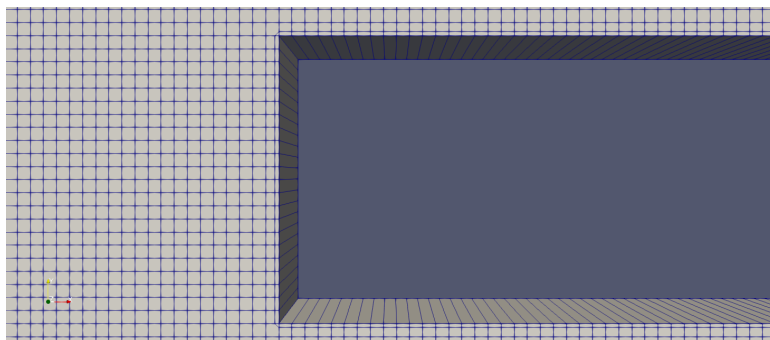


Figure 58: Applying the wall layer refinement once.

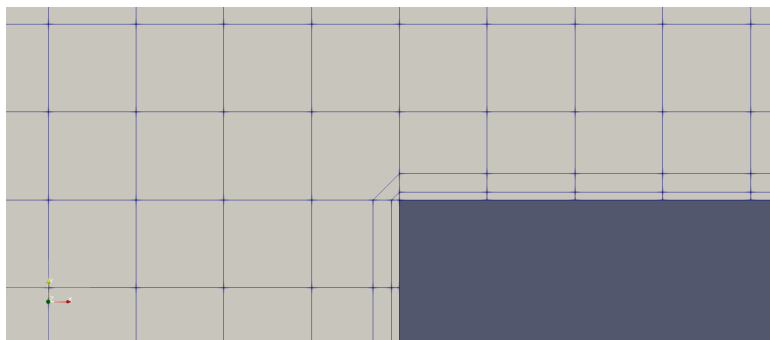


Figure 59: Applying the wall layer refinement a second time.

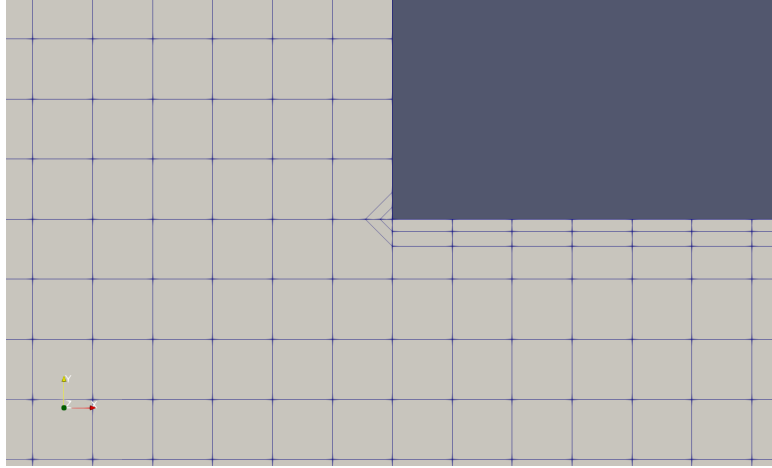


Figure 60: Applying the wall layer refinement twice on the horizontal patch at a concave edge.

Figures 61 and 62 show how `refineWallLayer` treats convex edges. If the edge is formed by two distinct patches, `refineWallLayer` can be applied to each patch individually, which leads to a different outcome, compare Figures 61 and 63.

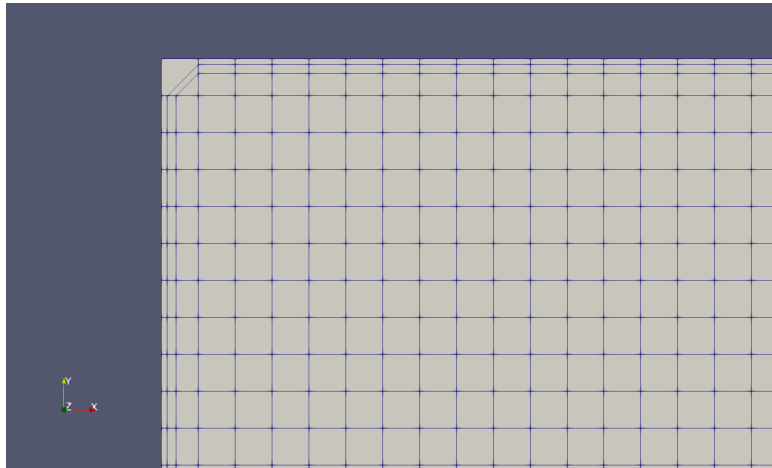


Figure 61: Applying the wall layer refinement twice on both patches of a convex edge.

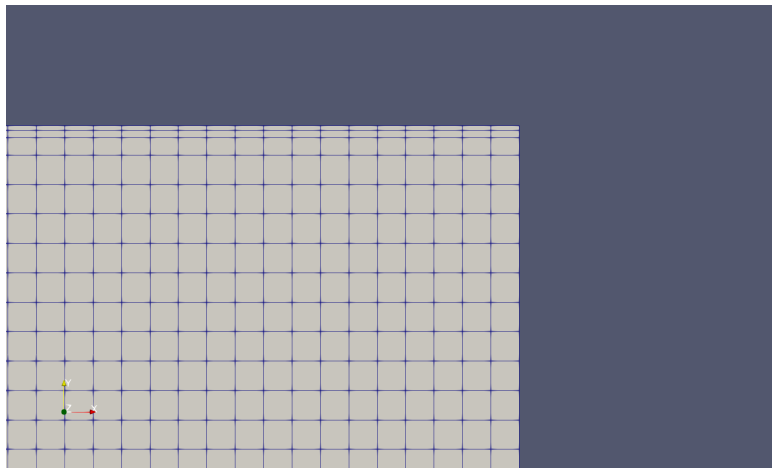


Figure 62: Applying the wall layer refinement twice on one patch of a convex edge.

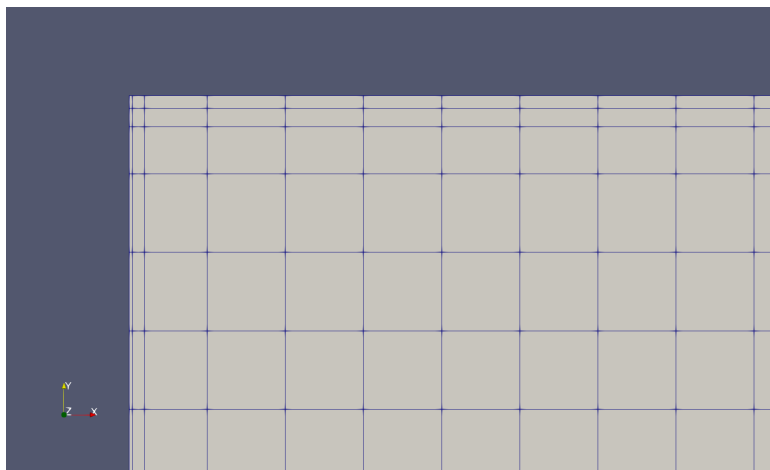


Figure 63: Applying the wall layer refinement successively on two patches of a convex edge. This approach leads to a different outcome than the one shown in Figure 61.

27.6 *renumberMesh*

27.6.1 General information

The tool *renumberMesh* modifies the arrangement of the cells of the mesh in order to create lower bandwidth for the numerical solution. For further information about the role and the influence of the bandwidth in numerical simulation see books on the numerical solution of large equation systems, e.g. [35].

Renumbering the mesh can reduce computation times as it re-arranges the data to benefit the numerical solution of the resulting equation system. The benefit of renumbering the mesh strongly depends on several factors. However, testing is recommended.

Renumbering the mesh even has an effect at the simplest possible simulation case – the cavity case of the tutorials. This mesh consists of a single block and it is quasi 2D (i.e. it is only 1 block in depth). The mesh resolution was chosen to $40 \times 40 \times 1$, resulting in 1600 cells. *icoFoam* was run for 10 s. Execution time was reduced by *renumberMesh* from 6.18 s to 6.08 s.

A simulation with a mesh consisting of 120000 cells defined by 9 blocks was run for 5 s of simulated time with *twoPhaseEulerFoam*. Execution time was reduced by *renumberMesh* from 9383.81 s to 9273.13 s.

Even though the reduction of execution time is small in this examples, this reduction comes at no cost. Running *renumberMesh* takes little time and at run-time of the simulation no additional work has to be done.



Run *renumberMesh* before any other tools which generate sets or zones. Why the order of execution of certain tools is significant is explained in Section ?? on a case which went slightly wrong.

27.6.2 Background

The discretized finite volume problem results in a linear equation system, which is usually expressed in matrix-form.

$$\mathbf{Ax} = \mathbf{b} \quad (31)$$

The vector $\mathbf{x}$ contains the field values at the cell centers. The matrix $\mathbf{A}$ contains non-zero elements for each pair of neighbouring cells. This is a consequence of our assumption that only adjacent cells interact. If we used some sort of higher order discretisation or interpolation, we might get into a situation where also second neighbours interact. However, for sake of ease, we limit ourselves in this discussion to direct neighbours.

Regardless of our computational mesh being one-, two- or three dimensional, we label all cells with positive ascending integers. Thus, we can store the values of a scalar field into a vector. The number of elements of this vector (N) is equal to the number of cells in our domain. Consequently, the matrix $\mathbf{A}$ is of the size $N \times N$. However, as only adjacent cells interact, most of the elements of $\mathbf{A}$ will be zero-entries.

If the cells with the labels i and j are adjacent, then the elements a_{ij} and a_{ji} of $\mathbf{A}$ will be non-zero. Since we focus on the general structure of $\mathbf{A}$ we do not care whether a_{ij} equals a_{ji} , or if both of them are actually non-zero⁶⁴.

The arrangement of the cells – or, to be more precise, the labelling – has a strong impact on the structure of the matrix $\mathbf{A}$, i.e. the distribution of the non-zero elements.

A simple example

Here we examine the effect of cell labelling with a very simple example. Figure 64 shows a simple mesh with 8 cells. Two different cell labelling schemes are indicated by the numbers inside the cells.

In Figure 65 we see the connections between the cells depicted as a graph. A $N \times N$ matrix can be from the interaction perspective seen as a graph with N nodes. An edge between the nodes i and j represents the non-zero elements a_{ij} and a_{ji} .

0	1	2	3
4	5	6	7

0	2	4	6
1	3	5	7

Figure 64: A simple mesh with 8 cells and different cell labelling schemes.

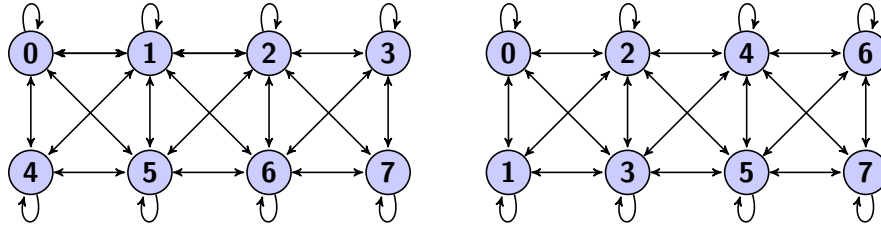


Figure 65: The connectivity graph of our mesh.

Figure 66 shows the corresponding matrix structure. The labelling scheme on the right hand side of Figures 64 and 65 results in a matrix with a lower bandwidth.

	0	1	2	3	4	5	6	7
0	*	*	0	0	*	*	0	0
1	*	*	*	0	*	*	*	0
2	0	*	*	*	0	*	*	*
3	0	0	*	*	0	0	*	*
4	*	*	0	0	*	*	0	0
5	*	*	*	0	*	*	*	0
6	0	*	*	*	0	*	*	*
7	0	0	*	*	0	0	*	*

	0	1	2	3	4	5	6	7
0	*	*	*	*	0	0	0	0
1	*	*	*	*	0	0	0	0
2	*	*	*	*	*	*	0	0
3	*	*	*	*	*	*	0	0
4	0	0	*	*	*	*	*	*
5	0	0	*	*	*	*	*	*
6	0	0	0	0	*	*	*	*
7	0	0	0	0	*	*	*	*

Figure 66: The matrix structure. A * denotes a non-zero element. Notice the lower bandwidth of the matrix on the right hand side. The number of zero-entries is equal, however, the different distribution leads to a different numerical behaviour.

⁶⁴The upwind differencing scheme causes the downstream cell to depend on the upstream cell. However, the upstream cell is not directly influenced by the downstream cell.

27.6.3 Pitfall: sets and zones will break my bones

The use of *renumberMesh* carries a certain risk. In simulation cases which make use of tools like *topoSet* and *renumberMesh*, the order in which those tools are invoked is of importance. **Update:** This has been resolved at some point. In OpenFOAM-4.0 this is no issue any more.

The reason behind this, is the way OpenFOAM stores its mesh information. The only actual geometric information is stored in the list of points in the file `constant/polyMesh/points`. The faces are defined via the point labels of the points defining the mesh. Thus, if the points P_k , P_m , P_u and P_w define a face, then the entry in `constant/polyMesh/faces` for this very face reads `(k m u w)`. The same principle applies for the definition of cells. There, the labels of the faces defining the cell are stored. This way, no redundant information is stored. If we define a *cellSet* with *topoSet* e.g. all cells within a certain geometrical region we simply store the cell labels of all cells for which the condition is fulfilled. Thus, if we now run *renumberMesh*, we shuffle the cells within the mesh. No actual change is applied in the mesh, however, the cell with the label A which was at the location (x_A, y_A, z_A) before renumbering, may or most certainly will be at location (x_B, y_B, z_B) with $B \neq A$ after renumbering.

Figure 67 shows the simulation domain of an aerated stirred tank. The red cells are part of a *cellZone* on which source terms using the *fvOptions* mechanism act⁶⁵. A run of *renumberMesh* after the *cellZone* was created caused the *cellZone* to get scrambled. However, the simulation worked nonetheless and yielded some unexpected results.

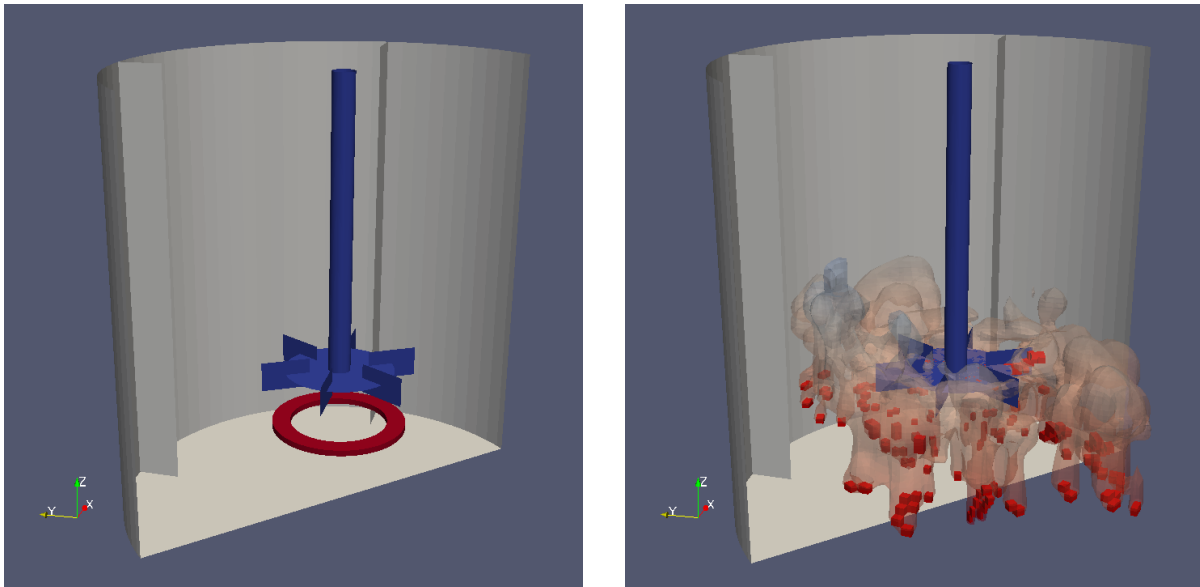


Figure 67: **Left:** The cut-away of the walls of a stirred tank with the rotor (blue) and the aeration device (red). The aeration device is a *cellZone* on which source terms are applied via the *fvOptions* mechanism in OpenFOAM-2.3.x.

Right: The stirred tank was simulated using parallel processes. After decomposing the domain, a parallel renumbering of the mesh was conducted. Renumbering the subdomains scrambled the *cellZone* within their respective subdomains. The transparent iso-volume shows the gas-phase volume fraction 0.25 s into the simulation. The cells of the *cellZone* act as source for the gas-phase, although not on their original location.

27.6.4 Life-hack: converting mesh (and fields) from binary to ASCII

You may run into a situation, my dear reader, when you might need to convert the mesh data of your OpenFOAM case from the binary format into the ASCII format, or vice-versa. What we need in this situation is a tool, which reads and writes the mesh, and *renumberMesh* is just the tool we need⁶⁶. In this case, we simply use it for its I/O, and not for its actual use, i.e. re-ordering the mesh to improve numerical solutions.

⁶⁵Have a look on the injection tutorial of *twoPhaseEulerFoam-2.3.x*.

⁶⁶We could think of using the tool `transformPoints` with a scaling factor of 1.0, yet this only affects the file `points`, but not `faces`, `neighbour` and `owner`.

One such occasion of the former need, i.e. convert a mesh from binary to ascii, is when you want to use tools of several OpenFOAM-variants, e.g. the `checkMesh` utility tool of `foam-extend-4.0` can not read a mesh in binary format, which was created by a mesh conversion tool (such as *fluentMeshToFoam*) of OpenFOAM-6. For some reason, the binary formats of `foam-extend` and OpenFOAM (foundation release) are not compatible⁶⁷. However, `foam-extend` is perfectly happy with meshes written by OpenFOAM in ASCII format. This, rather lengthy prelude, leads us a use-case of the tool *renumberMesh*, which might be considered harmless abuse: use the tool to change the format the mesh is stored on disk.

The procedure is rather simple:

1. Create a new case directory, in which the conversion should take place
2. Copy the relevant folders, i.e. `constant` and `system`, into this new case directory. Also copy time step folders, if necessary.
3. Change the `writeFormat` setting in `system/controlDict` from `binary` to `ascii`.
4. Run `renumberMesh -overwrite`

Now, all the mesh-files, such as `points` or `faces`, are stored on disk in ASCII format.

A note on converting fields

As *renumberMesh* is not a classical pre- or post-processing tool for the case's solution data, there is no option to renumber all fields present in a case. Hence, only one time step is subject to renumbering. This is generally the lowest time step.

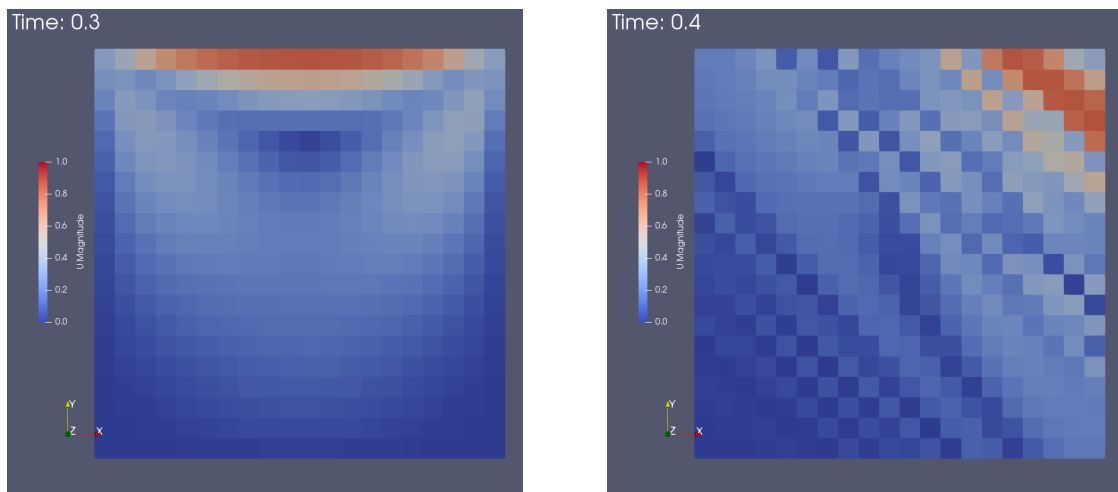


Figure 68: Renumbering the solution of the cavity case: the case was run, and all time steps prior to 0.3 were deleted. Then `renumberMesh -overwrite` was run. As 0.3 was the first time step, the fields in the time step 0.3 were renumbered along with the mesh. The later time steps, however, were left untouched.

Thus, if you want to convert a case with mesh and fields from binary to ASCII format, simply create a copy of the case with one time step only, and perform the renumbering. The tool *renumberMesh* only allows the user to specify one specific time, yet not a range of times. Thus, we consider it not possible to convert a case with multiple time steps from binary to ASCII, or the other around.

27.7 *subsetMesh*

subsetMesh is a tool to remove certain cells from a mesh. The tool expects the name of a *cellSet* as a command line argument. The cells of this *cellSet* will remain in the resulting mesh, all other cells are removed.

⁶⁷This incompatibility, however, is a uni-directional one. A binary mesh written by `foam-extend-4`, can be read and used by OpenFOAM-6.

Pitfall: sets and zones will break my bones

At the time of writing (OpenFOAM-4.0), *subsetMesh* does not treat *cellSets* or *cellZones*. Thus, when we use *subsetMesh* to remove large parts of the mesh, then the *cellSet* may contain cells that are no longer part of the mesh. This error will be felt when the cell indices associated with the *cellSet* or *cellZone* are larger than the total number of cells in the mesh. Otherwise, if the cell indices are smaller than the total number of cells, the *cellSet* might still be valid from OpenFOAM's point of view, but it may contain different cells.

27.8 *createPatch*

27.9 *stitchMesh*

27.10 *tetDecomposition*

The mesh manipulation tool *tetDecomposition* is part of the OpenFOAM variant foam-extend⁶⁸, and there is no comparable tool within the foundation release of OpenFOAM at the time of writing.

This tool takes an OpenFOAM mesh, computes the tet-decomposition and writes the resulting mesh to disk. Figures 69 and 70 illustrate how this tool works. The faces of the initial cell is decomposed into triangles. With such a triangle and the centroid of the cell, a sub-tetrahedron can be created.

Depending on the original mesh, the number of cells of the resulting mesh can rise dramatically. The single tetrahedron of Figure 69 is decomposed into 12 sub-tetrahedra. The initial hexahedron of Figure 70 is decomposed into 24 tetrahedra.

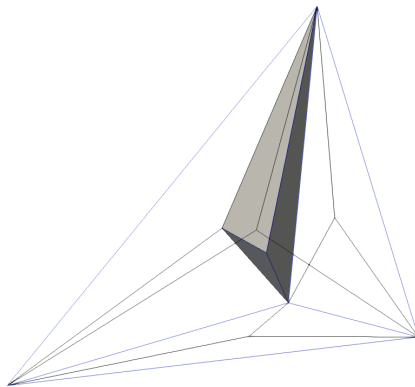


Figure 69: A tet-decomposed tetrahedron: the original tet-cell is outlined in blue, the face-decomposition is outlined in black, and one of the resulting sub-tets is shown in grey.

⁶⁸<http://www.foam-extend.org/>

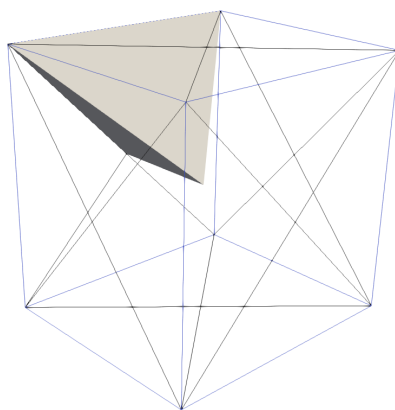


Figure 70: A tet-decomposed hexahedron: the original hex-cell is outlined in blue, the face-decomposition is outlined in black, and one of the resulting sub-tets is shown in grey.

27.11 decomposePar

The tool `decomposePar` is used to divide the domain for a parallel simulation run into smaller sub-domains.

27.11.1 Visualize the decomposition

The tool `decomposePar` has the command-line option `-cellDist`, which causes `decomposePar` to write the cell distribution as a field to disk. This cell distribution can then later be re-used with the `manual` decomposition method, or be visualized using ParaView. For this purpose, `decomposePar` writes a `volScalarField` and a `labelList` with the indices of the sub-domains.

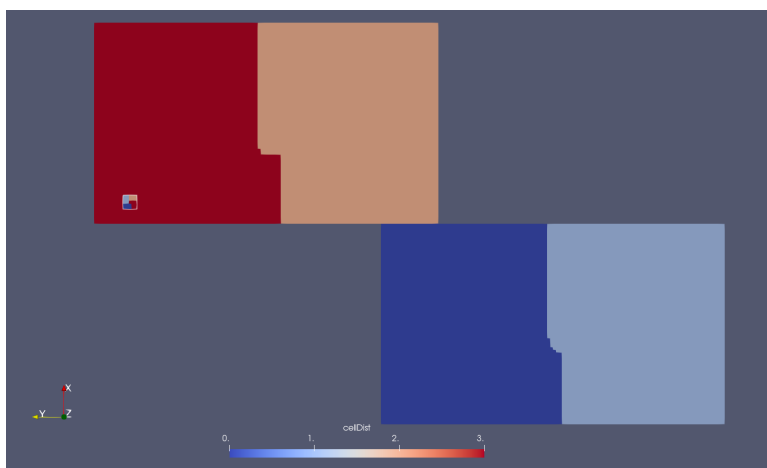


Figure 71: The cell distribution of a multi-region case, with 2 regions and 4 sub-domains for parallel processing. The small region outlined in white is the solid region, the surrounding larger region is the fluid region of this case. Each sub-domain (colour-coded from 0 to 3) is assigned a chunk of each region.

27.12 mirrorMesh

`mirrorMesh` is, similar to `extrudeMesh`, a mesh manipulation tool that is somewhere between mesh manipulation and mesh creation. This tool is controlled by entries in the file `system/mirrorMeshDict`. The set of parameters is quite limited, since we only need to provide a definition for the plane which is to act as the mirror, and a tolerance.

Listing 154 shows an example of a `mirrorMeshDict`, which consists of a plane definition (here we have three possible methods to choose from), and the tolerance.

```
planeType          pointAndNormal;

// Overall domain boundingBox: (0 0 0) (0.1 0.1 0.01)

pointAndNormalDict
{
    basePoint       (0.0 0.0 0.01);
    normalVector    (0 0 1);
}

// plane equation: ax + by + cz + d = 0
planeEquationDict
{
    a    0;
    b    0;
    c    1;
    d   -0.01;
}

embeddedPointsDict
{
    point1  (0 0 0.01);
    point2  (1 0 0.01);
    point3  (0 1 0.01);
}

planeTolerance     1e-5;
```

Listing 154: All the possible entries in the file `mirrorMeshDict` to mirror the initial mesh around an $x - y$ plane with a z -component of $z = 0.01$.

Potential pitfall: set and zones may break my bones

`mirrorMesh` does not apply the mirroring operation on the existing sets and zones of the mesh. This can be an issue depending on whether an existing `cellSet` or `cellZone` needs to be mirrored as well. If this is not necessary, users should remove all sets and/or zones prior to mirroring to avoid confusion.

In Figure 72, we see the result of the following sequence of operations: create the mesh using `blockMesh`, extrude a patch to grow a number of cell-layers, and finally mirror the mesh. The extrusion operation created the `cellSet` *addedCells*, which was not mirrored by `mirrorMesh`.

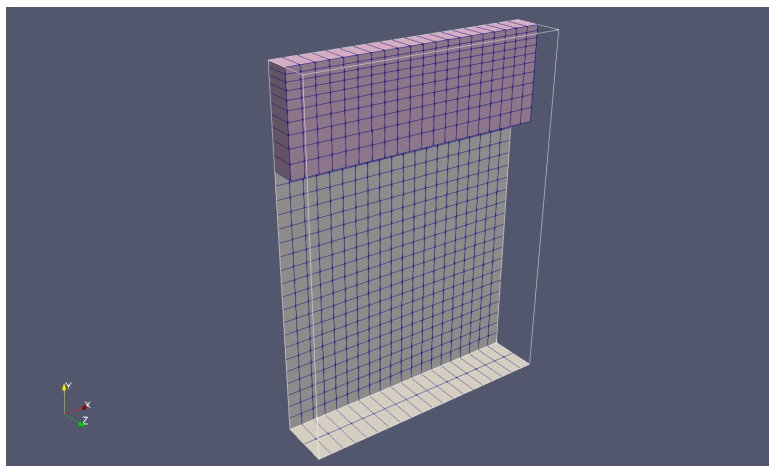


Figure 72: A `cellSet` after running `mirrorMesh` to mirror the mesh using the $x - y$ plane.

28 Surface mesh manipulation tools

OpenFOAM ships with a number of surface mesh manipulation tools. A probable use-case for this kind of tools is doing some preprocessing on STLs prior to creating a mesh with *snappyHexMesh* or *cfMesh*.

28.1 surfaceAdd

This tool can be used to merge two STLs into one file. With the command line switch `-mergeRegions` regions with an equal name get joined into one region. Otherwise the two regions would remain separate, regardless of having the same name.

```
surfaceAdd -mergeRegions input1.stl input2.stl out.stl
```

Listing 155: Usage of `surfaceAdd` when joining two STL files.

28.2 surfaceSubset

With this tool a subset of an STL can be extracted. Via the `surfaceSubsetDict` various conditions can be specified to define the subset. The user provides the STL to operate on (`input.stl`) and a file name for the subset to be stored in (`outSubset.stl`). The faces of the subset get removed from the original STL.

```
surfaceSubset surfaceSubsetDict input.stl outSubset.stl
```

Listing 156: Usage of `surfaceSubset` when extracting a certain subset from an STL.

28.3 surfaceFeatureExtract

This is a tool to extract features from an STL. E.g. *snappyHexMesh* pays extra attention to geometric features which are explicitly provided, *surfaceFeatureExtract* is a tool to generate the necessary data. The tool is controlled by the `surfaceFeatureExtractDict`. Surface feature extraction is controlled by a user-provided feature angle, which is used to determine whether an edge between two surface elements constitutes a feature edge, or not.

This tool writes the feature edges into an `*.eMesh` file located in the `constant/triSurface` folder.

In the file `surfaceFeatureExtractDict` we can enable the switch `writeObj`, which causes the tool to write all sorts of extracted data into `*.obj` files located in the `constant/extendedFeatureEdgeMesh` folder. These files can be viewed in ParaView to assess whether all surface features have been correctly and completely been extracted.

28.4 surfaceFeatureConvert

This tool can be used to convert `*.eMesh` files to `*.vtk` files to view them in ParaView. This is important when trying to find the proper settings, e.g. the feature angle for the file `surfaceFeatureExtractDict`.

28.5 surfaceTransformPoints

This tool is the surface mesh equivalent to the tool `transformPoints`, see 27.1, which can be used to scale, translate and rotate surface meshes. Apart from the obvious transformation options, this tool expects two file names: first, the input file; and second, the output file.

28.6 Third party surface manipulation tools

28.6.1 surfaceFeatureEdges

With this tool, provided by *cfMesh* (see Section 18), feature edges can be extracted from a surface mesh file, e.g. an STL.

```
surfaceFeatureEdges -angle 30 input.stl out.fts
```

Listing 157: Usage of `surfaceFeatureEdges` when extracting feature edges from an STL.

28.6.2 FMSToVTK

The tool FMSToVTK can be used to convert the `*.fms` files created by the tool `surfaceFeatureEdges` to the VTK format. Thus, the user can review how well the feature edges have been identified by `surfaceFeatureEdges`. This is important when trying to find the proper setting for the feature angle, which is passed with the `-angle` parameter.

This tool, provided by `cfMesh` (see Section 18).

28.7 The Linux command line

When doing some pre-processing with ASCII STLs the linux command line offers some nice features. Listing 158 shows the basic syntax of an STL file in ASCII format. STLs can be stored on harddisk either in ASCII, i.e. in plain text, or in binary format (non-human-readable). An STL file consists of solids, which are defined by their bounding surface.

```
solid SOLIDNAME
  facet normal X Y Z
    outer loop
      vertex X Y Z
      vertex X Y Z
      vertex X Y Z
    endloop
  endfacet
  ...
endsolid
```

Listing 158: The basic syntax of an STL in ASCII format. See [https://en.wikipedia.org/wiki/STL_\(file_format\)](https://en.wikipedia.org/wiki/STL_(file_format)) for more on this.

28.7.1 Renaming solids

Certain CAD tools do not offer the feature to name a part. E.g. OpenSCAD names the exported STL solid `OpenSCAD_Model`. If our STL pre-processing based on an STL generated by OpenSCAD yields an STL file per patch, a likely result when using `surfaceSubset`, we end up with a number of STLs, containing each one solid named `OpenSCAD_Model`. Now, we need to assign proper names to the solids, i.e. the STL solid of the file `inlet.stl` should be named `inlet`.

When the STL is in ASCII format, we can use `sed`⁶⁹ to perform a simple text replacement. Since an STL is very unlikely to contain the string `OpenSCAD_Model`, we simply can tell `sed` to replace every occurrence of `OpenSCAD_Model` with `inlet`.

```
sed -i s/OpenSCAD_Model/inlet/g inlet.stl
```

Listing 159: Renaming STL an solid with `sed`.

28.7.2 Joining STL files

If our pre-processing left us with a large number of STL files, e.g. an STL file for each patch, which is a likely result when using `surfaceSubset`, we might need to join these STLs, e.g. because the meshing tool expects only one STL to contain all the information. Joining text files is a task easily done from the command line with the tool `cat`⁷⁰, however, this could also be done using `surfaceAdd`. Other than with `surfaceAdd`, joining STLs with this approach is limited to STLs containing different solids.

⁶⁹<https://www.gnu.org/software/sed/manual/sed.html>

⁷⁰[https://en.wikipedia.org/wiki/Cat_\(Unix\)](https://en.wikipedia.org/wiki/Cat_(Unix))

Listing 160 shows how to join three STLs, each containing one solid, i.e. the information of one patch. The first line is simply a copy operation. Alternatively, we might use `cp` or `mv` for the first operation. Note, that the resulting STL gets written to a different folder, `constant/triSurface` is a folder in which some meshing tools expect STLs. The second and third lines show how to append the output of `cat` to the specified file.

The difference between the first and the following lines is the redirection operator (`>` vs. `>>`), see e.g. [https://en.wikipedia.org/wiki/Redirection_\(computing\)](https://en.wikipedia.org/wiki/Redirection_(computing)). The `>` operator simply redirects the output to the specified file, if this was used in the second line, then the contents from the first line would get overwritten in `myDomainMesh.stl`. Using the `>>` operator redirects and appends the output to the specified file. Using the `>` operator in the first line ensures to overwrite an eventual existing file.

```
cat walls.stl > constant/triSurface/myDomainMesh.stl
cat inlet.stl >> constant/triSurface/myDomainMesh.stl
cat outlet.stl >> constant/triSurface/myDomainMesh.stl
```

Listing 160: Joining STL files with `cat`.

29 Initialize Fields

29.1 Basics

There are two ways to define the initial value of a field quantity. The first is to set the field to a uniform value. Listing 161 shows the O/U file of the *cavity* tutorial. There the internal field is set to a uniform value.

If a non-uniform initialisation is desired, then a list of values for all cells is needed instead. Listing 168 shows some lines of such a definition. Entering such a nonuniform list by hand would be very tiresome. To spare the user of such a painful and exhausting task, there are some tools to provide help.

```
/*-----*- C++ -*-----*/
| ===== |
| \ \ / F i e l d | OpenFOAM: The Open Source CFD Toolbox |
| \ \ / O p e r a t i o n | Version: 2.1.x |
| \ \ / A n d | Web: www.OpenFOAM.org |
| \ \ / M a n i p u l a t i o n | |
|=====|
/*-----*/
FoamFile
{
    version      2.0;
    format       ascii;
    class        volVectorField;
    object       U;
}
// * * * * *
dimensions      [0 1 -1 0 0 0 0];

internalField    uniform (0 0 0);

boundaryField
{
    movingWall
    {
        type      fixedValue;
        value      uniform (1 0 0);
    }

    fixedWalls
    {
        type      fixedValue;
        value      uniform (0 0 0);
    }

    frontAndBack
    {
        type      empty;
    }
}
```

```
// ***** //
```

Listing 161: The file O/U of the *cavity* tutorial

29.2 *setFields*

setFields is a utility that allows to define geometrical regions within the domain and to assign field values to those regions. *setFields* reads this definitions from a file in the *system*-directory – the *setFieldsDict*. To initialize the field quantities **setFields** has to be executed after creating the mesh. *setFields* needs to read all files defining the mesh⁷¹.

In Listing 162 a box is defined in which the field *alpha1* is set to a different value.

```
/*-----*- C++ -*-----*\
| ===== |
| \ \      / | F i e l d | OpenFOAM: The Open Source CFD Toolbox |
| \ \      / | O p e r a t i o n | Version: 2.1.x |
| \ \      / | A n d | Web: www.OpenFOAM.org |
| \ \      / | M a n i p u l a t i o n |
\*-----*/
FoamFile
{
    version      2.0;
    format       ascii;
    class        dictionary;
    object       setFieldsDict;
}
// ***** //
defaultFieldValues
(
    volScalarFieldValue alpha1 1
);

regions
(
    boxToCell
    {
        box (-0.3 -0.3 0) (0.3 0.3 0.26);

        fieldValues
        (
            volScalarFieldValue alpha1 0
        );
    }
);
// ***** //
```

Listing 162: *setFieldsDict*

29.2.1 Defining regions

In Listing 162 we see the list named **regions** containing dictionaries defining regions in which to set field values. The **boxToCell** dictionary very much resembles something we also can see in the **topoSetDict** dictionary. In fact, *setFields* internally uses the machinery to create cell sets as *topoSet* does. A quick look into the main source files of *setFields* and *topoSet* reveals, that both make use of the class **topoSetSource**, which describes itself in the header file as: *Base class of a source for a topoSet*. All actual cell sources, such as **boxToCell** or **cylinderToCell**, directly inherit from **topoSetSource**. Thus, all cell sources available to *topoSet*, are also available to *setFields*. Such is the beauty of object-oriented programming.

29.2.2 Pitfalls

A nice, little collection of what may go wrong.

⁷¹Only the file *neighbour* can be missing for *setFields* not to crash.

Geometric region is not part of the domain

If the geometric region, in which to initialise a field with a specified value, lies outside the domain, *setFields* does not issue any warning or error message.

Geometric region covers the whole domain

This may happen if the geometric region is defined with respect to the vertex coordinates found in *blockMeshDict*. When the vertex coordinates are entered in millimeters – and *convertToMeters* is set appropriately – then it may happen, that the geometric region, based on the vertex coordinates in millimeters, is too large by the factor of 1000.

Listing 163 and 164 show the root of such a situation. The plan is to create a box and initialise it in a way, that the domain is half filled with one phase. The definition of the box in the *setFieldsDict* relies solely on the vertex coordinates ignoring the scaling factor *convertToMeters* resulting in a way too large box. After executing *setFields* the domain is completely filled with one phase instead of half filled.

```
convertToMeters 1e-3;

vertices
(
    (0      0      0)
    (50     0      0)
    (50     0      250)
    (0      0      250)
    (0      50     0)
    (50     50     0)
    (50     50     250)
    (0      50     250)
);
```

Listing 163: *blockMeshDict* entry for a box of $50 \times 50 \times 250$ mm

```
regions
(
    boxToCell
    {
        box (0.0 0.0 0.0) (50.0 50.0 125.0);

        fieldValues
        (
            volScalarFieldValue alpha1 0
        );
    }
);
```

Listing 164: *setFieldsDict* entry for a box of $50 \times 50 \times 125$ m

Field not found

If the *setFieldsDict* specifies a field which is not present, then OpenFOAM issues an error message similar to Listing 165. In this case the file *setFieldsDict* was copied from a case which uses the old naming scheme of *twoPhaseEulerFoam*, i.e. *alpha* instead of *alpha1*. See Section 45.2 for further information about the naming scheme. Therefore, the dictionary contained a definition for the field *alpha* which was not present in the *0*-directory.

```
Setting field default values
--> FOAM Warning :
    From function void setCellFieldType(const fvMesh& mesh, const labelList& selectedCells,
    Istream& fieldValueStream)
    in file setFields.C at line 103
    Field alpha not found

--> FOAM FATAL IO ERROR:
```

```
wrong token type - expected word, found on line 19 the label 1

file: /home/user/OpenFOAM/user-2.1.x/run/twoPhaseEulerFoam/bubbleColumn/system/setFieldsDict::
defaultFieldValues at line 19.

From function operator>>(Istream&, word&)
in file primitives/strings/word/wordIO.C at line 74.

FOAM exiting
```

Listing 165: Missing field

29.3 *mapFields*

mapFields is a utility to transfer field data from a source mesh to target mesh. This may be useful after the mesh of case has been refined and existing solution data is to be used for initialising the case with the refined mesh. *mapFields* preserves the format of the data, if the source data was stored in binary format, the target data will also be binary.

To use *mapFields* the file *mapFieldsDict* has to be existent in the *system* folder of the case⁷². *mapFields* expects as the only mandatory argument the path to the source case. The current directory is assumed to be the case directory of the target case. If there is no specification regarding time, the latest time steps of both cases are processes. That means the latest time step of the source case is mapped to the latest time step of the target case.

Listing 166 shows the last lines of output of *mapFields*. With lines like `interpolating alpha` *mapFields* indicates that it is processing some field data. Even when source and target meshes are equal and no interpolation is needed, *mapFields* displays lines like `interpolating alpha` anyway.

```
Source time: 0.325
Target time: 0
Create meshes

Source mesh size: 81000 Target mesh size: 273375

Mapping fields for time 0.325

interpolating alpha
interpolating p
interpolating k
interpolating epsilon
interpolating Theta
interpolating Ub
interpolating Ua

End
```

Listing 166: Output of *mapFields*

29.3.1 Pitfall: Missing files

mapFields issues no warning or error message when the source case contains no data. Listing 167 shows the output of *mapFields* as the target case contained no *0*-directory. Only the missing lines containing statements like `interpolating alpha` indicate that something is amiss and no field data is processed.

```
Source time: 0.325
Target time: 0
Create meshes

Source mesh size: 81000 Target mesh size: 273375

Mapping fields for time 0.325
```

⁷²In the most basic case *mapFieldsDict* contains no other information than the header and empty definitions. Although this file may seem of no use, it has to exist in the *system* folder, and it has to contain the header and the empty definitions.

End

Listing 167: Output of *mapFields*; Missing target *0*-directory

29.3.2 Pitfall: Unsuitable files

In the files containing the field data the values of the boundary fields as well as the values of the internal fields can be entered homogeneously (by the keyword **uniform**) or inhomogeneously (with the keyword **nonuniform**). Inhomogeneous field values have to be entered as a list of values. This list is preceded by the number of entries as well as the nature of the value. Listing 168 shows the beginning lines of the definition of a nonuniform vector field. The general syntax for such a list is the following:

```
nonuniform List<TYPE> COUNT ( VALUES )
```

the list. A wrong value of **COUNT** leads to reading errors.

If data is to be mapped from a source case, the source case's data will always be stored as a nonuniform list. Otherwise, mapping the data would make no sense, as uniform fields are most easily defined. If the data of the target case is uniform, then mapping makes no problems.

If the data of the target case is nonuniform – for whatever reason – then it is necessary that the nonuniform lists have the same length. Otherwise, *mapFields* will exit with an error message like in Listing 169. The target case should always be set up with uniform fields to avoid such errors. This is most easily done by removing the definition of the internal field. In the tutorials sometimes files with an **.org** file extension can be found. This is a way to preserve the uniform field data in the *0*-directory without causing any trouble.

```
dimensions      [0 1 -1 0 0 0 0];

internalField    nonuniform List<vector>
1600
(
(0.000174291 -0.000171512 0)
(0.000171022 -0.000143648 0)
(-0.000259297 0.000305772 0)
(-0.000380671 0.000374937 0)
(-0.00182755 0.000930701 0)
```

Listing 168: An inhomogeneous internal field definition in the file *0/U*

```
Mapping fields for time 0.325
```

```
    interpolating alpha
```

```
--> FOAM FATAL IO ERROR:
size 81000 is not equal to the given value of 10125
```

```
file: /home/user/OpenFOAM/user-2.1.x/run/twoPhaseEulerFoam/Case/0/alpha from line 18 to line
39.
```

```
From function Field<Type>::Field(const word& keyword, const dictionary&, const label)
in file /home/user/OpenFOAM/OpenFOAM-2.1.x/src/OpenFOAM/lnInclude/Field.C at line 236.
```

```
FOAM exiting
```

Listing 169: Error message of *mapFields*; unequal number of values

29.3.3 Pitfall: Mapping data from a 2D to a 3D mesh

In this section we deal with some difficulties of the *mapFields* utility. We have finished a simulation on a 2D mesh. The geometry of the 2D case is $20\text{ cm} \times 2\text{ cm} \times 45\text{ cm}$.

Now we want to transfer the 2D data to a 3D mesh to initialise the 3D simulation. The geometry of the 3D simulation is $20\text{ cm} \times 5\text{ cm} \times 45\text{ cm}$. Note the different dimension in *y*-direction.

Listing 170 shows the **mapFieldsDict** that was used. Because of the great similarity of the geometry, no entries are necessary.

```

/*-----*- C++ -*-----*/
| =====|
| \ \      / F i e l d      | OpenFOAM: The Open Source CFD Toolbox |
| \ \      / O p e r a t i o n | Version: 2.1.x |
| \ \      / A n d      | Web: www.OpenFOAM.org |
| \ \      / M a n i p u l a t i o n |
|-----*/
FoamFile
{
    version      2.0;
    format        ascii;
    class         dictionary;
    location      "system";
    object        mapFieldsDict;
}
// * * * * *

patchMap      ( );

cuttingPatches ( );

// * * * * *

```

Listing 170: The file `mapFieldsDict`

The problem

Figure 73 shows the result of the *mapFields* run. Only the field values inside the 2D domain were altered. The part of the 3D domain that lies outside the 2D domain remains unchanged. This behaviour is not satisfactory.

The work-around

One way to solve this problem would be to choose the 2D domain of a similar size as the 3D domain. However, if the 2D is already finished, then it would take some time to re-simulate the case with a redefined geometry.

Another solution is:

1. define the 3D domain to be of the same size as the 2D domain
2. map the fields
3. redefine the 3D domain to its intended size, without changing the total number of cells

29.3.4 The work-around: Mapping data from a 2D to a 3D mesh

The work-around to the problem of the previous section is rather unelegant. A 2D mesh that has the same depth as the 3D mesh but is discretised with only 1 cell in depth will have a very bad aspect ratio.

A more elegant solution is to transform the mesh after the 2D simulation has finished. In our example, the 2D mesh has the dimensions 20 cm × 2 cm × 45 cm and the 3D mesh is 20 cm × 5 cm × 45 cm big.

With the tool *transformPoints* the mesh can be scaled selectively in the three dimensions of space. Listing 171 shows how *transformPoints* can be used to scale the 2D mesh in *y*-direction by the factor of 2.5. After this scaling operation the 2D mesh has the desired dimensions of 20 cm × 5 cm × 45 cm.

```
transformPoints -scale '(1.0 2.5 1.0)'
```

Listing 171: Scaling the 2D mesh in *y*-direction with *transformPoints*

After the mesh transformation the utility *mapFields* can be used to map the field from the scaled 2D mesh to the 3D mesh.

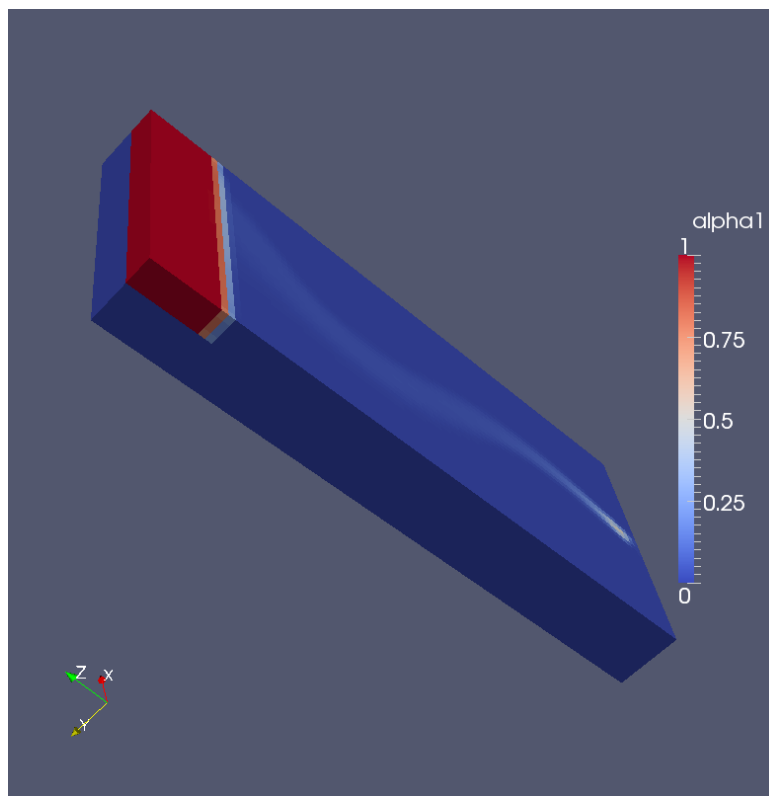


Figure 73: The mapped field

29.3.5 The importance of mapping

The purpose of this example is to highlight the need for the *mapFields* utility. A simulation of the bubble column has been made. Now, the user decides to change the size of the inlet patch. Thanks to the parametric mesh, this can be done easily only by changing some numbers in the file `blockMeshDict.m4`. See Section 15.5 for a discussion on creating a parametric mesh.

After the user changed the coordinates of some points, meshing yields a new mesh with the same number of cells as the old mesh had. Because the number of cells did not change, the data files from the finished simulation fit the new one. The user simply copies the necessary files from the latest time step of the finished simulation to the initial time step of the new simulation.

Starting the simulation resulted in a floating point exception. However, after reducing the time step, the simulation proceeded without any further errors. Figure 74 shows the initial `alpha` and `U1` fields of the new simulation. Due to a change in the numbering of the cells, the formerly smooth fields are now completely distorted. The single blocks of the mesh can be distinguished from the figures. This indicates, that OpenFOAM numbers the cells block-wise.

29.3.6 Pitfall: binary files

If the source case has binary data files, then the boundary conditions need to be defined before mapping the fields. Therefore, the boundary conditions need to be defined in a suitable ascii file. Then, the fields can be mapped. Editing a binary file with a text editor may render this file defective.

30 Case manipulation

This section contains a discussion on tools for the manipulation of the simulation case which to not create or modify the mesh or are used for initialisation. Utilities for the before mentioned tasks are already discussed in previous sections.

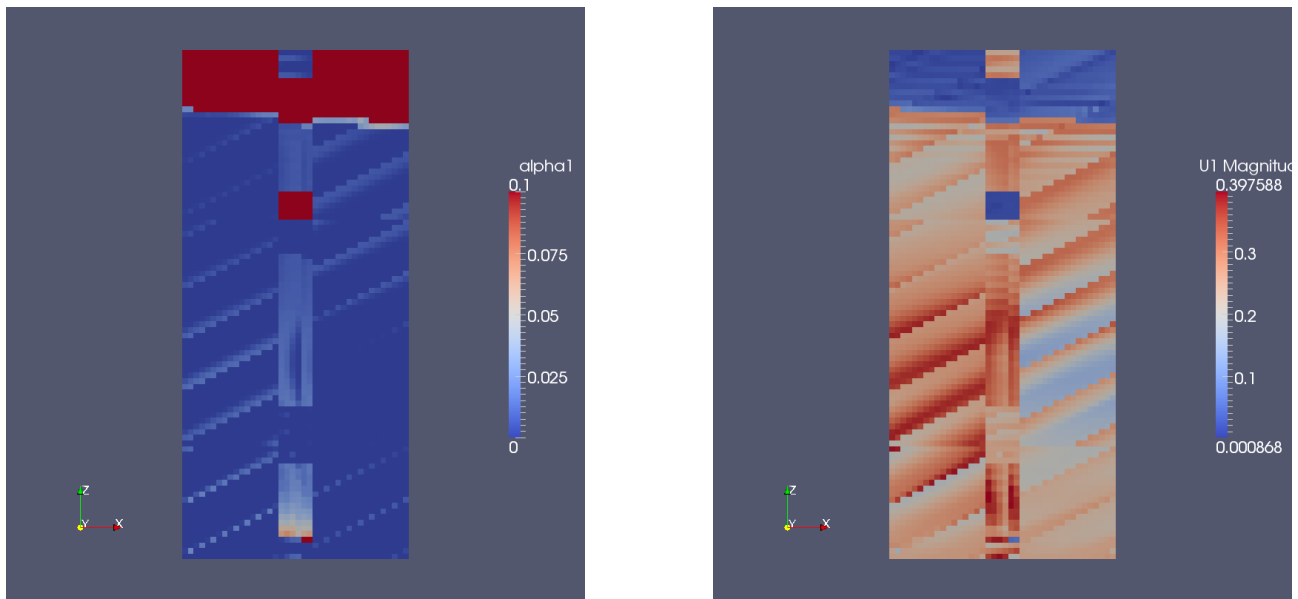


Figure 74: The unmapped fields

30.1 *changeDictionary*

The utility *changeDictionary* can be used to modify a dictionary, except those residing in **system**. We can of course manipulate any of our dictionaries using a simple text editor, even from the command line (*emacs*, *vim*, *nano*, etc.).

A possible scenario in which *changeDictionary* comes in handy is when we do spin-up simulations, i.e. run the simulation for a certain time with e.g. reduced inflow and continue afterwards with full inflow⁷³. This approach might improve the stability of the simulation.

Another case in which *changeDictionary* proves to be quite important is when we want to change boundary value of fields we have gained from a previous simulation. Editing ascii files which measure in the megabytes can be very tiresome with some text editors. If the files are stored in binary, using a text editor might not be an option anymore. In this *changeDictionary* provides a neat way to change boundary values.

Listing 172 shows a simple example of the *changeDictionaryDict*.

```
dictionaryReplacement
{
    U
    {
        boundaryField
        {
            inlet
            {
                type            fixedValue;
                value            uniform (20 0 0);
            }
        }
    }
}
```

Listing 172: A simple *changeDictionaryDict* used to change an inlet velocity

By default *changeDictionary* operates only on dictionaries living in the time step directories. By adding the command line option **-constant** the dictionaries of the **constant** folder can be edited.

30.1.1 A spin-up simulation

In this section we discuss what is termed a spin-up simulation in this manual. This simulation is intended to run without user intervention once the simulation is started. In this case we assume we have set up a simulation

⁷³In such a scenario we also would need to manipulate **controlDict** to increase the **endTime**. Well, we can't have everything.

with a reduced inflow. Thus, the flow establishes within the domain in a much gentler regime. After the flow is established we increase the inflow to the desired value. Again, the build-up of the flow within the domain happens in a gentler manner, as there is already a slower flow present through out the domain. Thus, we avoid punching the quiescent fluid in the domain with full force at the inlet⁷⁴.

Listing 173 shows the `Allrun` script for such a kind of simulation. In Line 11 the solver is run for the first time. Since none of the lines in the script is terminated by the ampersand (&), execution waits for the command of the current line to finish until the next command is invoked. Thus, we have to assume all commands are run in the stated order.

In Line 14 the log-file generated by `runApplication` is renamed (moving a file within a directory is essentially renaming). The reason for this operation is, that `runApplication` checks if there is already a log present. If there is, `runApplication` does not run the specified application.

In Line 15 `changeDictionary` is called. This is the step in which, in our example, we increase the inlet velocity. In Line 16 we use the GNU tool `sed` to edit `controlDict`⁷⁵.

In Line 19 we call the solver for the second time. Here it is crucial that the keyword `startAt` is set to `latestTime` in `controlDict`.

In Line 20 we apply the same renaming to the solver-log of the second run. This is not necessary in principle, however, if we are to perform automated processing of the logs, then a consistent naming scheme might be very helpful.

```

1  #!/bin/sh
2  cd ${0%/*} || exit 1      # run from this directory
3
4  # Source tutorial run functions
5  . $WM_PROJECT_DIR/bin/tools/RunFunctions
6
7  # Create the mesh using blockMesh
8  runApplication blockMesh
9
10 # Run the solver
11 runApplication pimpleFoam
12
13 # prepare second run
14 mv log.pimpleFoam log.pimpleFoamRun01
15 runApplication changeDictionary
16 sed -i 's/endTime      20/endTime      40/g' system/controlDict
17
18 # Run the solver again
19 runApplication pimpleFoam
20 mv log.pimpleFoam log.pimpleFoam02
21
22 # ----- end-of-file

```

Listing 173: The `Allrun` script of a spin-up simulation

The `Allrun` script was applied to a slightly modified *pitzDaily* tutorial case. A appropriate `changeDictionaryDict` file Listing 172 was added to the `system` directory, otherwise the tutorial is untouched. Figure 75 shows the flow field after `changeDictionary` was called. The increased inlet velocity is displayed as well as the established flow from the initial run with an inlet velocity of (10 0 0).

30.1.2 Change boundary types

Another, very handy use of `changeDictionary` is changing the type of certain boundaries. This may be necessary when using imported meshes. Some mesh formats or meshing tools do not support to assign patch-types to certain boundaries. Thus, when importing these meshes into OpenFOAM, all patches are of the general `patch` type. Hence, there is the necessity to change the patch type of the wall patches to the type `wall`. This could be done by manually editing the file `boundary`, yet this approach does not scale – what if we have hundreds of wall patches – and it is easy to forget this step.

⁷⁴Such a simple kind of thing could also be achieved with time-dependent boundary conditions. However, there are solvers which do not support time-variant boundary conditions, or we want to do something nastier, which can't be achieved with time-variant boundary conditions.

⁷⁵We could also do the edit manually with any text editor, as `controlDict` will never reach megabytes or be stored in binary format. However, the whole idea of the spin-up simulation idea is to avoid manual intervention.



Figure 75: The established flow field and the increased inlet boundary condition of the *pitzDaily* tutorial case at $t = 1$ s

Using *changeDictionary* to change the patch type is a scalable solution, i.e. once the changes are specified in the file *changeDictionaryDict*, there is no additional work if we do the whole process of importing the mesh and changing the patch type all over again.

The second main advantage of using *changeDictionary* instead of manual editing, is that using a tool is scriptable, whereas manual file manipulation is not. Scripting ensures that no intermediate operation is omitted in any process.

```
boundary
{
    walls
    {
        type    wall;
    }
}
```

Listing 174: A simple *changeDictionaryDict* used to change the type of a patch *walls* to *wall*.

30.2 The allmighty Linux Terminal

This section covers case manipulation we can do with the tools available in the Linux Terminal. The reason for this, is partly that for certain tasks there is to the authors knowledge no tool provided by OpenFOAM, or that the task can be done more conveniently with the tools of the Terminal rather than OpenFOAM's tools.

The primary reason we are able to manipulate cases with the might of the tools we find in a Linux Terminal is that everything is a text file in OpenFOAM's case definition. If there is one thing Linux, or UNIX in general, has an over-abundance, it is test editors and text editing tools. This is due to a design decision of the UNIX creators which is best clarified by quoting Eric S. Raymond [20]:

Unix tradition strongly encourages writing programs that read and write simple, textual, stream-oriented, device-independent formats. Under classic Unix, as many programs as possible are written as simple filters, which take a simple text stream on input and process it into another simple text stream on output.

Despite popular mythology, this practice is favored not because Unix programmers hate graphical user interfaces. It's because if you don't write programs that accept and emit simple text streams, it's much more difficult to hook the programs together.

Text streams are to Unix tools as messages are to objects in an object-oriented setting. The simplicity of the text-stream interface enforces the encapsulation of the tools. More elaborate forms of inter-process communication, such as remote procedure calls, show a tendency to involve programs with each others' internals too much.

With a little help from a friend

If we combine the powers of the Linux Terminal and OpenFOAM's tools, anything is possible. In remainder of this section, some case manipulation scenarios of the combined use of the tools provided by Linux and OpenFOAM are presented

30.2.1 Rename a patch and edit the corresponding boundary condition

If we are in the lucky situation to have a symmetric mesh, we can remove half the domain and apply a symmetry constraint on the newly formed boundary. This is achieved by the following sequence of steps:

1. Select all cells belonging to one half of the domain with *topoSet*
2. Remove the cells of the other half of the domain with *subsetMesh*
3. Change the definition of the newly formed boundary to a symmetry plane with *createPatch*.

After executing this steps, the mesh has a new symmetry boundary, however, all the fields retain their `oldInternalFaces` boundary introduced by *subsetMesh*. Thus, we need to rename the boundary condition in all fields and change the type of the boundary condition to symmetry.

Change BC type

We can change the type of a boundary condition with the tools *changeDictionary* or *foamDictionary*. The task of bulk-renaming is not possible with either of the tools. The tool *changeDictionary* is controlled by the `changeDictionaryDict` file. Within this file, we can use wildcards for the patch names, however, we can not use a wildcard for field names. Listings 175 and 176 show an allowed use-case of wildcards and one impossible use-case in `changeDictionaryDict`.

Our task, change the BC type for all fields, falls under the latter category. We could, however, work around this issue if we included all fields in the `changeDictionaryDict`.

```
T
{
    boundaryField
    {
        ".*"
        {
            type            zeroGradient;
        }
    }
}
```

Listing 175: Possible use of wildcards with *changeDictionary*: Change all boundary conditions of the field `T` to `zeroGradient`.

```
".*"
{
    boundaryField
    {
        oldInternalFaces
        {
            type            symmetry;
        }
    }
}
```

Listing 176: Impossible use of wildcards with *changeDictionary*: Change the type of the boundary condition for the patch `oldInternalFaces` of all fields to `symmetry`.

The tool *foamDictionary* can also be used to change the type of a boundary condition. This tool is controlled by command line arguments, Listing 177 shows how the tool is called.

```
foamDictionary FILE -entry boundaryField.oldInternalFaces -set "{type symmetry;}"
```

Listing 177: Calling up *foamDictionary* to change the type of the BC of the patch *oldInternalFaces* for the field defined in *FILE*.

However, we can only pass one file at a time. This is where the Linux Terminal comes into play. With a simple *for* loop, we can loop over all files contained in the *0* directory and call *foamDictionary* for each individual file. Done!

```
for file in 0/*; do
    foamDictionary $file -entry boundaryField.oldInternalFaces -set "{type symmetry;}"
done
```

Listing 178: Changing the type of the BC of the patch *oldInternalFaces* for all fields.

Rename all BCs for all fields

Next, we would like to change the patch name in all field files. This operation is necessary, as *createPatch* operates only on the mesh. The fields are untouched by this tool. As we used *createPatch* to change the name and the type of the *oldInternalFaces* boundary for the mesh, we need to change the boundary name also for the fields.

This operation is a simple find&replace, which can be done with any text editor. For the sake of automation, we use the stream editor *sed*. Again, we loop over all files present in the *0* directory and we apply the find&replace operation on each file.

```
for file in 0/*; do
    sed -i 's/oldInternalFaces/symmetry/g' $file
done
```

Listing 179: Changing the name of the of the patch *oldInternalFaces* for all fields to *symmetry*.

30.2.2 Rename phases

The multi-phase solvers of OpenFOAM⁷⁶ use the phase name as file extension in order to distinguish fields and files, e.g. *U.air* and *U.water*. In Listing 180 we see the content of the *0* directory of the bubble column tutorial. This clearly demonstrates the use of file extensions to distinguish phases.

<i>alpha.air</i>	<i>alphat.water</i>	<i>epsilon.water</i>	<i>k.water</i>	<i>p</i>	<i>Theta</i>	<i>U.water</i>
<i>alpha.air.orig</i>	<i>epsilon.air</i>	<i>k.air</i>	<i>nut.air</i>	<i>p_rgh</i>	<i>T.water</i>	

Listing 180: The content of the *0* directory of the bubble column tutorial.

If we wanted to simulate nitrogen gas in water, we would need to rename all files in order to follow this convention. Manually renaming files is tedious and can be automated by the use of the tools available in the Linux Terminal.

Rename files

First, we want to rename all files in order for them to have the proper file extension. This helps avoiding confusion. Thus, we use *find* to search for all files with the file extension *air* and pass them to *rename*, which renames the files according to the specified pattern.

```
find . -name '*.air' | xargs rename .air .nitrogen
```

Listing 181: Replacing the file extension *air* with the file extension *nitrogen* in all files in and below the current folder.

⁷⁶From the release of OpenFOAM-2.3.0 onwards, see <http://openfoam.org/release/2-3-0/multiphase/>.

Replace text

Next, we need to replace the old phase name within the files itself. Again we use `find` to search for all files and pass them to `sed`, which replaces all text fitting the specified pattern. Note: applying `sed` to all files can lead to some trouble, as the text “pair” also gets treated and thus becomes “pnitrogen”. However, if the application of `sed` causes such a side effect, OpenFOAM will crash if vital entries were damaged in this way.

```
find . -type f | xargs sed -i s/air/nitrogen/g
```

Listing 182: Replacing the word `air` with the word `nitrogen` in all files in and below the current folder.

Part V

Modelling

31 Solution dimensions

In OpenFOAM all meshes are three-dimensional, yet with a single-cell discretisation and a proper assignment of empty- or wedge-type boundary conditions, a simulation may have 0, 1, 2 or 3 solution dimensions, or solution directions as they are referred to by OpenFOAM's `checkMesh` tool.

```
Checking geometry...
Overall domain bounding box (0 -0.1 0) (0.07 0.11 0.001)
Mesh has 2 geometric (non-empty/wedge) directions (1 1 0)
Mesh has 2 solution (non-empty) directions (1 1 0)
All edges aligned with or perpendicular to non-empty directions.
```

Listing 183: A snippet of the output of `checkMesh`.

0-D simulations

In a 0-D simulation, the mesh consists of only a single cell, and all PDEs simplify into ODEs with time as the independent variable. This may seem too strong of a simplification, yet 0-D simulations may be useful to test the temporal development of certain models.

The OpenQBMM project⁷⁷ uses a 0-D solver to test their population balance model⁷⁸. In a 0-D domain, only sources and sinks can change the value of a field, as convection is ruled out.

1-D simulations

A 1-D simulation is resolved, respectively discretised, in only one spatial direction. This is a valid simplification for simulations when the axial flow dominates any lateral flow, e.g. very long pipelines. In the OpenFOAM tutorials, e.g. the *shockTube* tutorial case is a 1-D simulation.

2-D simulations

Many tutorials of OpenFOAM are 2-D cases, as they are detailed enough to represent realistic flow, yet they are simple enough to finish in a finite amount of time. 2-D simulations can be planar 2-D cases, e.g. a narrow channel, which is much greater in depth than it is in length and height; or axi-symmetric, e.g. a slice of a domain, which features rotational symmetry. The *cavity* tutorial case is an example of a planar 2-D simulation; and the *movingCone* case is an example of an axi-symmetric simulation case in the OpenFOAM tutorials.

3-D simulations

Full 3-D simulation feature no empty- or wedge-type patches.

31.1 Basic rules for simulations with reduced solution dimensions

An empty- or wedge-type boundary needs to be planar

One consequence of this rule is, that the two wedge-type patches need to be separate patches

There can't be a single wedge-type patch

Such a patch, consisting of both sides of the slice, would be non-planar, as the patch-mean normal-direction will be different from each face's normal-direction.

⁷⁷<http://www.openqbmm.org/>

⁷⁸<https://github.com/OpenQBMM/OpenQBMM/tree/master/applications/solvers/pbeFoam>

An empty-type boundary needs to be parallel to a coordinate plane

Listing 184 shows the output of `checkMesh`, after we rotated the mesh of the cavity tutorial case. Initially, the empty boundary is parallel to the x - y plane. However, after we rotated the mesh using `transformPoints`, the empty boundary normal direction is $(1, 1, 1)$. This, results in `checkMesh` reporting 0 solution directions. When we try to run the case, `icoFoam` fails with a rather obscure error message.

```
Checking geometry...
  Overall domain bounding box (-0.0211325 -0.0211325 -0.11547) (0.084641 0.084641 0.0057735)
  Mesh has 0 geometric (non-empty/wedge) directions (0 0 0)
  Mesh has 0 solution (non-empty) directions (0 0 0)
***Number of edges not aligned with or perpendicular to non-empty directions: 2121
<<Writing 882 points on non-aligned edges to set nonAlignedEdges
```

Listing 184: The output of `checkMesh` after we applied a rotation from $(0, 0, 1)$ to $(1, 1, 1)$ to the simulation domain of the cavity tutorial.

```
--> FOAM FATAL ERROR:
Continuity error cannot be removed by adjusting the outflow.
Please check the velocity boundary conditions and/or run potentialFoam to initialise the
  outflow.
Total flux          : 2.22507e-308
Specified mass inflow : 0.000211325
Specified mass outflow : 0
Adjustable mass outflow : 0

  From function bool Foam::adjustPhi(Foam::surfaceScalarField&, const volVectorField&, Foam
  ::volScalarField&)
  in file cfdTools/general/adjustPhi/adjustPhi.C at line 107.

FOAM exiting
```

Listing 185: The output of `icoFoam` after we applied a rotation from $(0, 0, 1)$ to $(1, 1, 1)$ to the simulation domain of the cavity tutorial.

A wedge-type boundary mustn't be parallel to a coordinate plane

The mesh of an axis-symmetric simulation is a discretised slice of the geometry, with a single cell in circumferential direction. This slice is supposed to straddle a coordinate plane. If one of the `wedge`-type patches is parallel to a coordinate plane, then OpenFOAM issues an error message, similar to the one shown in Listing 186.

```
--> FOAM FATAL ERROR:
wedge back plane aligns with a coordinate plane.
  The wedge plane should make a small angle (~2.5deg) with the coordinate plane
  and the pair of wedge planes should be symmetric about the coordinate plane.
  Normal of wedge plane is (6.451349e-20 1 5.042482e-20) , implied coordinate plane
  direction is (0 1 0)

  From function virtual void Foam::wedgePolyPatch::calcGeometry(Foam::PstreamBuffers&)
  in file meshes/polyMesh/polyPatches/constraint/wedge/wedgePolyPatch.C at line 110.

FOAM exiting
```

Listing 186: The output of `checkMesh` when a `wedge`-type patch is parallel to a coordinate plane.

32 Turbulence-Models

32.1 Organisation

The way the source for the turbulence models is organized changed over the time⁷⁹ the author is dealing with OpenFOAM. With the release of OpenFOAM-2.3.0⁸⁰ a new, (even) more general, way of code organisation was rolled out.

⁷⁹Since beginning of 2012 or OpenFOAM-2.0.x.

⁸⁰<http://www.openfoam.org/version2.3.0/multiphase.php>

The old way relied essentially on namespaces and inheritance to achieve generality and abstraction. The new way to do stuff is based on templates, inheritance and inheritance from templates. This section discusses both ways of code organisation. Especially the new way – with all its template madness – may lead to difficulties to understand the code at first glances. Thus, the author hopes to be able to shed some light into the mysteries of the new way to do things.

With the release of OpenFOAM-3.0, the transition to the new turbulence modelling framework has been completed⁸¹. There is no `$FOAM_SRC/turbulenceModels` directory anymore in the sources. Thus, the discussion of the old ways is on its way to be of purely historical interest. However, the author hopes, that even the outdated sections of this ever-growing collection of stuff may provide some insights.

32.1.1 The old ways

Although, this section is not intended as a rant against everything new, the organisation was easier to understand. You can inspect it at `$FOAM_SRC/turbulenceModels`. The old turbulence modelling framework is based on namespaces to draw the distinction between compressible and incompressible models.

The multiphase solvers within this old framework either use a turbulence model on mixture quantities (*multiphaseEulerFoam* or *interFoam*), or the turbulence model was applied to the continuous phase only (*twoPhaseEulerFoam*). Within the old framework, only one turbulence model was applied in multiphase simulations

Figure 76 shows the organisation of the old turbulence modelling framework. The class hierarchy is duplicated to some degree with largely identical or equivalent classes in each namespace, i.e. `Foam::compressible` and `Foam::incompressible`. A comparison of the files `RASModel.H` and `RASModel.C` in the namespaces `Foam::compressible` and `Foam::incompressible` reveals that these files share more common lines than they have differing lines.

This issue is also addressed in the release notes of the new turbulence framework in even more pressing terms:

The issue of compressibility has been managed for many years using two distinct turbulence modelling frameworks, one for constant density flows and another for variable density flows. However, neither framework is appropriate for multiphase systems, in conservative form, for which the phase-fraction must be included into all transport and source terms of the turbulence property equations. Code is largely duplicated between the two frameworks, which is inconsistent with the OpenFOAM code development policy to minimise code duplication to promote code maintainability and sustainability. Extension of the current code architecture to multiphase flows would increase the number of hierarchies from two to four, one for each combination of phase-fraction and density representation.

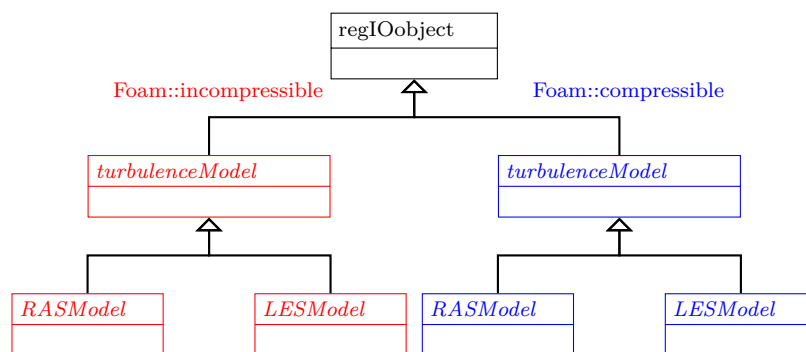


Figure 76: The class hierarchy of the basis of the old turbulence model framework. The namespaces `Foam::incompressible` and `Foam::compressible` are indicated by the colours red and blue.

32.1.2 The new order

The new framework for all turbulence models is located at `$FOAM_SRC/TurbulenceModels`, notice the capital T⁸². The use of templates is necessary, since this framework is meant to be used by all solvers of OpenFOAM

⁸¹<http://openfoam.org/version3.0.0/>

⁸²This is one of the reasons why OpenFOAM is not readily available on Windows, since it assumes that the file system is case-sensitive. In fact, OpenFOAM makes heavy use of case-sensitivity of the file system. Microsoft, however, reminds us not to

at some point of time. All solvers means compressible and incompressible, as well as single- and multiphase. This makes sense, since the concept of turbulence is general, and not related to the specific situation in question. The advantages of this approach is best said by the release note itself:

This new framework is very powerful and supports all of the turbulence modelling requirements needed so far. It will be enhanced and extended in future OpenFOAM releases to include a wide range of models and sub-models, with the expectation to replace the current dual hierarchies of turbulence models, to aid code maintainability and sustainability.

Initially the new turbulence modelling framework was introduced with an update of the multiphase solvers. In the OpenFOAM-2.3.0 release only *twoPhaseEulerFoam* and *DPMFoam*. As time progresses more and more solvers are updated to use the new framework instead of the old. By the time of writing this paragraph (October 2015) dozens of solvers in the OpenFOAM-dev repository were already ported.

One to rule them all

Whenever, a certain concept manifests itself in a variety of incarnations⁸³, the developers of OpenFOAM take this rough quote from Lord of the Rings by heart. A single turbulence model class was created to be applied to whatever physics OpenFOAM implements. For this to work, this most basic turbulence model contains only the things which can be abstracted enough to apply everything. The most trivial example of this (a feature independent of compressibility or the number of phases involved), is the sheer existence of a turbulence model⁸⁴.

Figure 77 shows the basic class hierarchy of the new turbulence framework. Besides this basic, non-templated class hierarchy, there is the templated hierarchy of the implementing classes. The basic classes represent the very abstraction. On top of the family tree is the class `I0dictionary`, which provides the IO facilities. From using OpenFOAM, we know, that there is a dictionary controlling the turbulence modelling. By deriving the turbulence model class from `I0dictionary`, the turbulence model is its dictionary.

From `I0dictionary` the class `turbulenceModel` is derived. Note the lower case letter at the beginning. This is not the only base class for turbulence models, we will also encounter a capital letter class. As already mentioned, OpenFOAM makes heavy use of the file system's case sensitivity. Thus, we need to pay attention to the letter (`turbulenceModel` $\neq$ `TurbulenceModel`).

The class `turbulenceModel` declares a large number of pure virtual functions which the derived classes down the family tree inevitably need to implement. This class is the source-code-wise incarnation of the fact that there is a turbulence model. No further information is as of this point known to the turbulence model, except that it is a turbulence model. The data of this class is consequently sparse. The most important data members of this class are references to the run-time object and the mesh. More information can be found in the file `$FOAM_SRC/TurbulenceModels/turbulenceModels/turbulenceModel.H`.

From the class `turbulenceModel` two classes are derived: `incompressibleTurbulenceModel` and `compressibleTurbulenceModel`. These two classes represent the fact, that flow can be considered incompressible or compressible. The consequence of this difference can be seen in the treatment of the density by these two classes. In Figure 77 we see, that the incompressible turbulence model has a `geometricOneField` as density data member, in contrast to the compressible model, which has a reference to the actual density field.

expect, e.g. NTFS, to be case-sensitive. See: https://msdn.microsoft.com/en-us/library/aa365247%28VS.85%29.aspx#naming_conventions

⁸³Such as turbulence is present in single-phase, multi-phase, compressible, and incompressible flow.

⁸⁴This is not a non-statement, however trivial this might sound. We can relate the existence of turbulence modelling to a certain class, namely `turbulenceModel`, which is derived from `I0dictionary`, and serves as the absolute basis for everything further down the family tree.

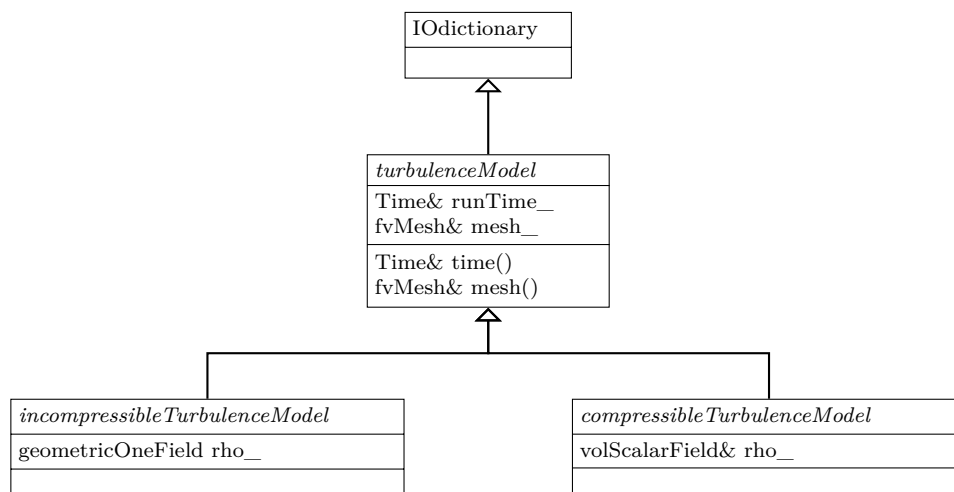


Figure 77: The class hierarchy of the basis of the new turbulence model framework.

A little note on ancestry: in the class hierarchy of the *ye olden ways*, see Figure 76, we saw that the base classes for the turbulence models, were derived from `regIOobject`. Thus, allowing access to the turbulence model via OpenFOAM's registry.

In the class hierarchy of the *fancy new order*, see Figure 77, we see that the base class for all turbulence models is derived from the class `IOdictionary`. In Figure 133, all the way down in Section 57.7, we see that the class `IOdictionary` is derived from `regIOobject`⁸⁵. Thus, the turbulence model base class is derived indirectly derived from `regIOobject`. Thus, allowing access to the turbulence model via OpenFOAM's registry.

A mere comparison of Figures 76 and 77 might have suggested otherwise, however, as the list of ancestors got longer for the new modelling framework, this fact (the derivation from `regIOobject`) has travelled up the family tree.

Many to rule the many

The distinction between incompressible and compressible, as well as, single-phase and multi-phase, turbulence modelling is made by passing appropriate template parameters to the base class `TurbulenceModel`. Note that `TurbulenceModel` is derived from the template parameter `BasicTurbulenceModel`. In Figure 78 we see the (templated) class hierarchy of the new turbulence modelling framework. This class hierarchy is related to the classes depicted in Figure 77 by the use of the template parameter `BasicTurbulenceModel`, which is either `incompressibleTurbulenceModel` or `compressibleTurbulenceModel`, note the lower case first letter.

The distinction between incompressible and compressible modelling is made by the template parameters `Rho` and `BasicTurbulenceModel`. In the case of incompressible models a `geometricOneField`⁸⁶ is passed for the parameter `Rho`. The distinction between single-phase and multi-phase modelling is made by the template parameter `Alpha`. In the case of single-phase modelling a `geometricOneField` is passed.

The approach, that `TurbulenceModel` is derived from its template parameter `BasicTurbulenceModel`, which is either an `incompressibleTurbulenceModel` or `compressibleTurbulenceModel`, which in turn are derived from a common base class, demonstrates the great flexibility a high-level programming language, such as C++. However, the presence of templates and their heavy, sophisticated use – as demonstrated in OpenFOAM – raises the bar when it comes to reading the source code and finding out what is happening.

⁸⁵In OpenFOAM-5.0, the class `IOdictionary` is derived from a base class `baseIOdictionary`, which in turn is derived from `regIOobject` and `dictionary`. In earlier versions of OpenFOAM, `IOdictionary` is directly derived from `regIOobject` and `dictionary`.

⁸⁶The header file of the class `geometricOneField` describes its intention as follows:

A class representing the concept of a GeometricField of 1 used to avoid unnecessary manipulations for objects which are known to be one at compile-time.

Used for example as the density argument to a function written for compressible to be used for incompressible flow.

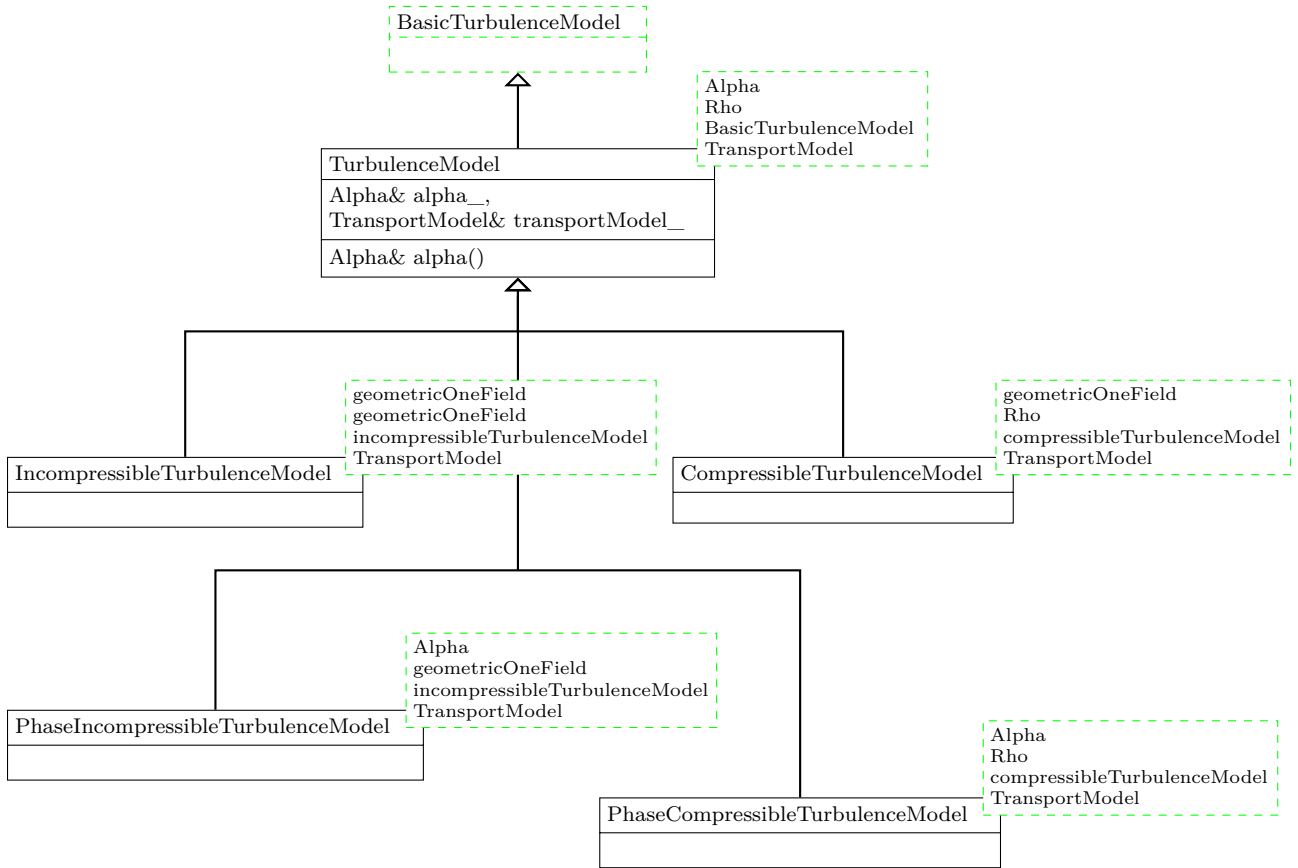


Figure 78: The base class `TurbulenceModel` has four template parameters and it is derived from one of its template parameters. Note, that the four derived classes – the four incarnations of the turbulence model – differ in the template parameters.

Branching the family tree

In turbulence modelling, we can identify three elementary choices: we can treat a fluid flow as laminar, or apply a RAS or LES turbulence model. This basic choice is reflected in the three classes derived from the template parameter in Figure 79. Since RAS and LES turbulence models are turbulence models⁸⁷, those two base classes are derived from the common template parameter `BasicTurbulenceModel`. The nature of `BasicTurbulenceModel` has been discussed above.

By treating the laminar case as a turbulence model, the OpenFOAM developers got rid of the special case laminar flow. In Figure 79, the behaviour of the `laminar` turbulence model is indicated by the methods `R()` and `nut()`. The `laminar` turbulence returns zero (with the appropriate dimension) for all turbulent quantities. Thus, the method `R()`, which computes the Reynolds stress tensor, returns a volumetric⁸⁸ field of symmetric tensors will all-zero components⁸⁹. This behaviour is indicated in Figure 79 with the `(= 0)` appended to the method's names.

The class `eddyViscosity` is a class which implements the ideas behind the *Boussinesq hypothesis*, which is discussed below.

⁸⁷Again, we encounter an *is a* relationship, which is a strong hint for relating two classes by inheritance.

⁸⁸I.e. all values are defined at the cell centers.

⁸⁹In the file `laminar.C`, we find this expression in the constructor of the returned tensor field: `dimensionedSymmTensor("R", sqr(this->U_.dimensions()), symmTensor::zero).`

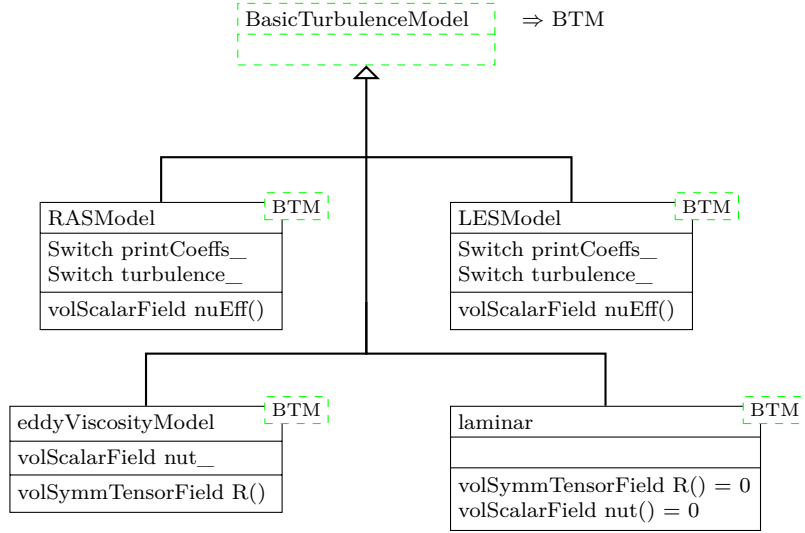


Figure 79: The class hierarchy of the elementary turbulence models of the new turbulence model framework. Note the shorthand notation BTM for the class `BasicTurbulenceModel`.

Further down the family tree

A great number of turbulence models are based on the so-called *Boussinesq hypothesis* which computes the Reynolds stresses from an *eddy viscosity* μ_t and the mean strain-rate tensor, and was proposed by Boussinesq [11] [64].

$$\mathbf{R} = \mu_t (\nabla \mathbf{u} + \nabla \mathbf{u}^T) - \frac{2}{3} \rho \mathbf{I} k \quad (32)$$

$$k = \frac{1}{2} \sum_i \overline{u'_i u'_i} = \frac{1}{2} \overline{\mathbf{u}' \cdot \mathbf{u}'} \quad (33)$$

The quantity k is the specific kinetic energy of the turbulent fluctuations. A great part of literature refers to k as *turbulent kinetic energy* [53, 35, 7, 8], most probably for reasons of keeping the vocabulary short. The unit tensor $\mathbf{I}$ is often denoted with the Kronecker delta δ_{ij} in literature.

The Boussinesq hypothesis is common to both RAS and LES turbulence models. This can be translated into a class relationship. In Figure 80 we see how the `kEpsilon` and the `Smagorinsky` turbulence models are derived. Those two models are discussed since these are widely used. The class `eddyViscosityModel` implements the general idea of the Boussinesq hypothesis, thus, it is the common base for both turbulence models. In the case of LES models, an intermediate class (`lesEddyViscosityModel`) is in between the class `eddyViscosityModel` and the actual turbulence model. This class serves to hold data and define methods specific to LES models using the Boussinesq hypothesis.

The distinction between RAS models and LES models is made by the template parameter inserted in `eddyViscosityModel`. In the case of RAS models, the template parameter of `eddyViscosityModel` from which e.g. the `kEpsilon` model is derived is `RASModel<BasicTurbulenceModel>`. Since `RASModel` is derived from `BasicTurbulenceModel`, the class `RASModel` is a `BasicTurbulenceModel`. Thus, this operation is perfectly valid. In the case of LES models, `LESModel<BasicTurbulenceModel>` is inserted as the template parameter of `eddyViscosityModel`.

Sounds complicated, which it probably also is. Nevertheless, we admire the versatility of generality of the new turbulence modelling framework and stomach the mental pain caused by all the template and inheritance wizardry.

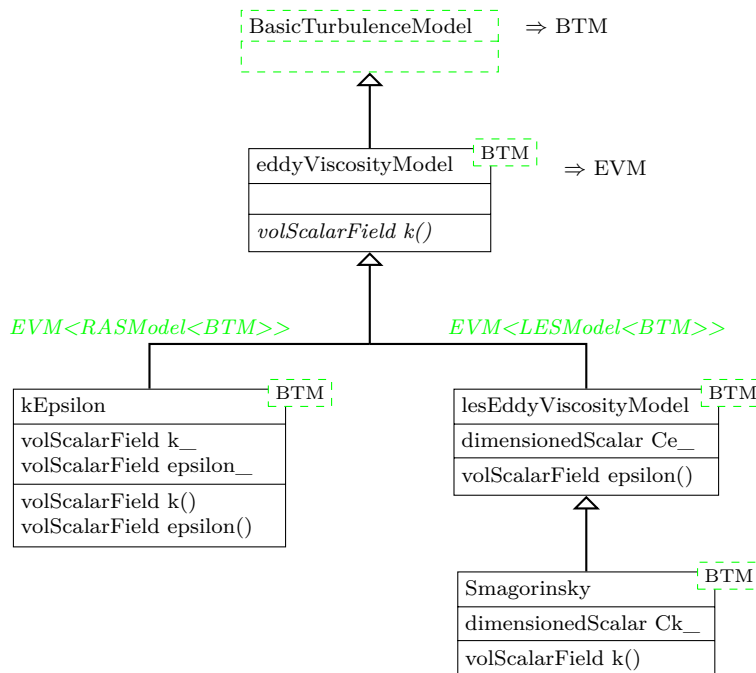


Figure 80: The class hierarchy of a selection of turbulence models of the new turbulence model framework. Note the shorthand notation BTM for the class `BasicTurbulenceModel`, and EVM for `eddyViscosityModel`.

The method signature in italics of the class `eddyViscosityModel` indicates a pure virtual function. This method has to be implemented by the classes derived from `eddyViscosityModel`. In the case of the `kEpsilon` class it is the class derived directly from `eddyViscosityModel` which implements `k()`. In the case of the `Smagorinsky` class, the pure virtual function was inherited via `lesEddyViscosityModel`. A class containing a pure virtual function can not be instantiated, thus, there can be no usable turbulence model `lesEddyViscosityModel`. This class can only serve as an intermediary.

32.2 Reynolds averaged models (RAS)

32.2.1 kEpsilon and low-Reynolds flows

The standard wall functions require the `yPlus` value to fall into a certain range. If the `yPlus` value is outside of that range, the simulation's results can become quite wrong. Thus, the use of the `kEpsilon` model creates an effective upper limit on the mesh resolution at the walls [19]. This becomes a problem, when we want to finely resolve the flow at and near the wall, e.g. when solving heat-transfer problems.

For cases, when the `yPlus` value is too small, i.e. it is smaller than the lower bound of the standard wall-function's validity range, there are the so-called *low Reynolds* formulations.

Wall functions

From OpenFOAM-5.0 onwards, there are no special low-Reynolds wall-functions for `epsilon`, as the standard and low-Reynolds formulation have been merged into a single wall-function⁹⁰.

However, other fields, such as `k` and `nut`, still require special wall-functions when a low-Reynolds model is used.

Disclaimer

Everything of Section 32 after this point has been created a while ago. The some of the content of the sub-sections below might be outdated by the time you read this.

⁹⁰See <https://github.com/OpenFOAM/OpenFOAM-dev/commit/f260780b7310528505137bbfc8d5f85084379120>

32.3 Categories

The desired category of turbulence models can be specified in the file `turbulenceProperties`. There are three possible entries.

laminar The flow is modelled laminar

RASModel A Reynolds averaged turbulence model (RAS-model) is used.

LESModel Turbulence is modelled by a *large-eddy* model.

The file `turbulenceProperties` contains only one entry. In case of a large eddy simulation, this entry reads:

```
simulationType  LESModel;
```

Listing 187: *turbulenceProperties*

32.4 RAS-Models

The entry in the file `turbulenceProperties` specifies only the class of turbulence models. The exact turbulence model is specified in the file `RASProperties`. This file must contain all necessary parameters.

Listing 272 shows the content of `RASProperties`. In this case a $k-\epsilon$ model is used and no further parameters are necessary.

```
RASModel      kEpsilon;  
turbulence    on;  
printCoeffs   on;
```

Listing 188: *RASProperties*

Depending on the exact model more parameters can be necessary.

32.4.1 Keywords

RASModel The name of the turbulence model. At this place laminar can also be chosen. The banana test (see Section 11.2.1) delivers a list of available models.

```
--> FOAM FATAL ERROR:  
Unknown RASModel type banana  
  
Valid RASModel types:  
  
17  
(  
  LRR  
  LamBremhorstKE  
  LaunderGibsonRSTM  
  LaunderSharmaKE  
  LienCubicKE  
  LienCubicKELowRe  
  LienLeschzinerLowRe  
  NonlinearKEShiih  
  RNGkEpsilon  
  SpalartAllmaras  
  kEpsilon  
  kOmega  
  kOmegaSST  
  kkLOmega  
  laminar  
  qZeta  
  realizableKE  
)
```

Listing 189: Possible RAS-model entries in *RASProperties*

turbulence This is a switch to activate or deactivate the turbulence modelling. Allowed values are: *on/off*, *true/false* or *yes/no*.

If this switch is deactivated, then a laminar simulation is conducted. This way of choosing a laminar model is not recommended, see Section 32.6.1.

printCoeffs If this switch is enabled, then the solver will display the coefficients of the selected turbulence model.

Even if the switch **turbulence** is disabled, the solver will display the coefficients at the beginning of the simulation, see Listing 196. The coefficients are not displayed only when **RASModel laminar** is chosen.

optional parameters Some models accept optional parameters to override the default values of the model. Listing 190 shows how the coefficients of the k- ϵ model can be overridden.

```
kEpsilonCoeffs
{
    Cmu          0.09;
    C1           1.44;
    C2           1.92;
    C3           -0.33;
    sigma_k      1.0;
    sigmaEps     1.11; //Original value:1.44
}
```

Listing 190: Definition of model parameters in *RASProperties*

32.4.2 Pitfall: meaningless Parameters

In the above section it was shown how to override default values of the model constants. In this procedure, there is one source of error hidden. This is not an actual error, but it can lead to a fruitless search for an error.

If nonsensical parameters are added to the **kEpsilonCoeffs** dictionary, these will be read and also printed. Listing 191 shows the **kEpsilonCoeffs** dictionary of the file **RASProperties**. This dictionary is used to override default values of the model constants. A fake model constant has been added to this dictionary.

Listing 192 shows parts of the solver output, when this dictionary is used in a simulation. All constants of the dictionary are read and printed again. It seems as if the constant **banana** is part of the turbulence model. Varying this parameter yields no results, which is no error.

The reason for this behaviour is, there is no check whether the defined constants in the dictionary make sense or not.

```
kEpsilonCoeffs
{
    Cmu          0.09;
    C1           1.44;
    C2           1.92;
    C3           -0.33;
    sigma_k      1.0;
    sigmaEps     1.11; //Original value:1.44
    banana       0.815; // nonsense parameter
}
```

Listing 191: Definition of model parameters in *RASProperties*

```
Selecting RAS turbulence model kEpsilon
kEpsilonCoeffs
{
    Cmu          0.09;
    C1           1.44;
    C2           1.92;
    C3           -0.33;
    sigma_k      1.0;
    sigmaEps     1.11;
    banana       0.815;
}
```

Starting time loop

32.5 LES-Models

32.5.1 Keywords

The keywords `turbulence` and `printCoeffs` have the same meaning with LES models. There is also the possibility – depending on the selected model – of defining optional parameters.

LESModel The name of the turbulence model. At this place laminar can also be chosen. The banana test (see Section 11.2.1) delivers a list of available models. Listing 193 shows the result of such a banana test. The model `dynamicSmagorinsky` was loaded from an external library. See Section 11.3.3 for how to include external libraries.

```
--> FOAM FATAL ERROR:   Unknown LESModel type banana

Valid LESModel types:

16
(
    DeardorffDiffStress
    LRDiffStress
    Smagorinsky
    SpalartAllmaras
    SpalartAllmarasDDES
    SpalartAllmarasIDDES
    dynLagrangian
    dynOneEqEddy
    dynamicSmagorinsky
    homogeneousDynOneEqEddy
    homogeneousDynSmagorinsky
    kOmegaSSTAS
    laminar
    mixedSmagorinsky
    oneEqEddy
    spectEddyVisc
)
```

Listing 193: Possible LES-model entries in *LESProperties*

32.5.2 Algebraic sub-grid models

Algebraic sub-grid models introduce no further transport equation to the simulation. The turbulent viscosity is calculated from existing quantities.

32.5.3 Dynamic sub-grid models

The dynamic sub-grid models calculate the model constant C_S from known quantities instead of prescribing a fixed value. The way how C_S is calculated is determined by the sub-grid model.

32.5.4 One equation models

A further class of LES turbulence models are one equation models. These models add one further equation to the problem. Usually, an additional equation for the sub-grid scale turbulent kinetic energy is solved.

32.6 Pitfalls

32.6.1 Laminar Simulation

As already mentioned – see Section 32.4 – turbulence modelling can be deactivated in a some ways.

In the following, different ways to conduct a laminar simulation are listed. This list applies only to solvers that utilize the generic turbulence modelling of OpenFOAM:

1. **turbulenceProperties:** `simulationType laminar`

This is the most general way to turn turbulence modelling off. `turbulenceProperties` controls the generic turbulence class. The generic turbulence class can take the form of the `laminar`, `RASModel` or `LESModel` class, see Figure 134. This is controlled by the parameter `simulationType`.

```
Selecting turbulence model type laminar
```

Listing 194: Solver output for `simulationType laminar`

2. **RASProperties:** `RASModel laminar`

LESProperties: `LESModel laminar`

In this case, a certain turbulence modelling strategy is chosen (`RASModel` or `LESModel`). However, there is a dummy turbulence model for laminar simulation. This dummy turbulence model is derived from the base class `RASModel` but it implements a laminar model. See Figure 135. Therefore, `RASModel laminar` selects the laminar RAS turbulence model. In this point `RASModel` and `LESModel` behave similar.

```
Selecting turbulence model type RASModel
Selecting RAS turbulence model laminar
```

Listing 195: Solver output for `RASModel laminar`

3. **RASProperties:** `turbulence off`

The switch `turbulence` can be used to enable or disable turbulence modelling. When the calculation is started, the turbulence model specified is used. However, in the source code of the solver, there is the test whether turbulence modelling is active or not. See Listing 264.

```
Selecting turbulence model type RASModel
Selecting RAS turbulence model kEpsilon
kEpsilonCoeffs
{
    Cmu          0.09;
    C1           1.44;
    C2           1.92;
    sigmaEps     1.3;
}
```

Listing 196: Solver output for `turbulence off`

Solver output

The last two possibilities to conduct a laminar simulation can lead to confusion because the solver output contains word like `RASmodel` or `RAS turbulence model`. See Listings 195 and 196. In both cases the simulation is laminar. In order to avoid this source of confusion, the user should use the parameter `simulationType` to perform a laminar calculation.

Independent from all other settings, `printCoeffs` prints the model constants of the selected turbulence model. This may also lead to confusion, when e.g. `turbulence off` is chosen to conduct a laminar simulation.

Exceptions

The above explanation only applies to solvers that utilize the generic turbulence models of OpenFOAM. However, there is no rule without its exceptions.

simpleFoam This solver uses only RAS turbulence models. Therefore, the entries of the file `turbulenceProperties` are redundant and the only ways to control turbulence modelling are items 2 and 3 of the list above.

twoPhaseEulerFoam This solver has the $k-\epsilon$ turbulence model hardcoded. Only item 3 of the list above applies to this solver. See Section 32.6.2 for a detailed discussion.

bubbleFoam The same as *twoPhaseEulerFoam*.

multiphaseEulerFoam This solver only uses LES turbulence models. Items 2 and 3 of the list above apply.

32.6.2 Turbulence models in *twoPhaseEulerFoam*

In the solver *twoPhaseEulerFoam*, the use of the k - ϵ turbulence model is hardcoded. This means that the solver does not use the generic turbulence modelling usually used by OpenFOAMs solvers. The only choice the user of *twoPhaseEulerFoam* has is whether to enable or disable the k - ϵ turbulence model.

For this reason, the file `constant/turbulenceProperties` is not needed any more. This file can safely be deleted.

Another consequence of the k - ϵ turbulence model being hardcoded into *twoPhaseEulerFoam* is that the keyword `turbulenceProperties` in the file `RASproperties` is also not needed any more. This entry is only read if the generic turbulence modelling is used and if there is any choice of which RAS-model to use. The only mandatory keyword in `RASproperties` is the switch `turbulence`. This switch is the only way to decide whether to use turbulence modelling or not with *twoPhaseEulerFoam*. Solvers which use the generic turbulence modelling offer three possible ways to disable turbulence modelling, see Section 32.6.1.

32.6.3 Laminar simulation with *twoPhaseEulerFoam*

If *twoPhaseEulerFoam* is used and a laminar simulation is conducted, then the presence of the files like `0/k` or `0/epsilon` is mandatory. The solver reads these files regardless of the fact that a laminar simulation is conducted. This is due to the fact that the use of the k - ϵ model is hardcoded into *twoPhaseEulerFoam*.

Other solvers read these files based on the condition if and which turbulence model is used. Otherwise there would be the need for all possible files (`0/k`, `0/epsilon`, `0/omega`, etc.) to be present in any case, which would be utter madness.

32.6.4 Initial and boundary conditions

All turbulence models can be divided into classes depending on their mathematical properties.

Algebraic models These models add an algebraic equation to the problem. The turbulent viscosity is computed from known quantities using an algebraic equation (e.g. the Baldwin-Lomax model)

One equation models These models introduce an additional transport equation to the problem. The eddy viscosity is computed from this additional quantity (e.g. the Spalart-Allmaras model)

Two equation models These models introduce two additional transport equations to the problem. The eddy viscosity is computed from these additional quantities (z.B. k - ϵ , k - ω)

Every field quantity of a turbulence model needs its initial and boundary conditions. Consequently, there may be the need for additional files in the `0`-directory. One way to find out which files are needed is to look at the tutorials. There, a case may be found which utilises the needed turbulence model.

If a simulation is started and the solver is missing files – i.e. the solver tries to read files which are not present – then OpenFOAM will issue a corresponding error message. Listing 197 shows an error message of a case with a missing `0/k` file.

```
Selecting turbulence model type RASModel
Selecting RAS turbulence model kEpsilon
--> FOAM FATAL IO ERROR:  cannot find file
file: /home/user/OpenFOAM/user-2.1.x/run/pisoFoam/cavity/0/k at line 0.

    From function regIOobject::readStream()
    in file db/regIOobject/regIOobjectRead.C at line 73.

FOAM exiting
```

Listing 197: Solver error message: missing file

32.6.5 Additional files

RAS turbulence models produce additional files. Most RAS models calculate the turbulent viscosity from certain quantities. These quantities are usually field quantities and depend on the used turbulence model. However, the aim of all RAS turbulence models is to calculate the turbulent viscosity. The turbulent viscosity itself is a field quantity.

Listing 198 shows the folder contents before and after a simulation with *pisofFoam*. The *0*-directory contains only the mandatory files, in this case pressure and velocity as well as the turbulent quantities *k* and ϵ .

After the simulation has finished, the *0*-directory contains more files. The reason for creating the *.old files is not known. However, the turbulence model created the file *nut* for storing the turbulent viscosity.

The file *phi* as well as the folder *uniform* is created by the solver.

```

user@host:~/OpenFOAM/user-2.1.x/run/pisoFoam/ras/cavity$ ls
0 constant system
user@host:~/OpenFOAM/user-2.1.x/run/pisoFoam/ras/cavity$ ls 0/
epsilon k p U
user@host:~/OpenFOAM/user-2.1.x/run/pisoFoam/ras/cavity$ pisoFoam > /dev/null
user@host:~/OpenFOAM/user-2.1.x/run/pisoFoam/ras/cavity$ ls
0 0.5 1 constant system
user@host:~/OpenFOAM/user-2.1.x/run/pisoFoam/ras/cavity$ ls 0/
epsilon epsilon.old k k.old nut p U
user@host:~/OpenFOAM/user-2.1.x/run/pisoFoam/ras/cavity$ ls 0.5/
epsilon k nut p phi U uniform
user@host:~/OpenFOAM/user-2.1.x/run/pisoFoam/ras/cavity$

```

Listing 198: Folder contents at the begin and the end of a simulation

The *0*-directories of some tutorial cases may already contain such additional files, e.g. *nut*. In some cases the *0*-directory may also contain several of such files due to a change in the naming scheme. Listing 199 shows the contents of the *0*-directory of the *pitzDaily* tutorial case of *simpleFoam*. The case has not been run, so the files *nut* and *nuTilda* have not been generated by the solver. None of these two files is necessary to run the case with the *k- ϵ* turbulence model.

```

epsilon k nut nuTilda p U

```

Listing 199: The content of the *0*-directory of the *pitzDaily* tutorial case of *simpleFoam*

32.6.6 Spalart-Allmaras

The Spalart-Allmaras is a one-equation turbulence model. Although it introduces only one additional equation to the problem it needs two additional files in the *0*-directory. Listing 200 shows the content of the *0*-folder of the *airFoil2D* tutorial case of *simpleFoam*. The files *nut* and *nuTilda* are both necessary to run the case. The former contains the turbulent viscosity and the latter contains the transported quantity of the turbulence model. Therefore, the rule *one additional transport equation entails one additional data file* is not violated.

Because the viscosity is not constant it has to be defined in a file in the *0*-directory. And, because the viscosity is not the transported quantity of the Spalart-Allmaras model another file is added to the *0*-directory.

```

nut nuTilda p U

```

Listing 200: The content of the *0*-directory of the *airFoil2D* tutorial case of *simpleFoam*

33 Thermophysical modelling

OpenFOAM features a rich selection of models to account for a fluid's thermophysical properties, reaching from constant properties to elaborate models. While the official guide, see cfd.direct/openfoam/user-guide/thermophysical/, does a good job of summarizing the models, I feel the need to share some (non-)wisdom of mine on this topic.

33.1 The modelling framework

Much like the fully-templated turbulence modelling framework, the thermophysical modelling framework is also heavily templated. From inspecting the source code of OpenFOAM-1.1, which is the oldest version⁹¹ that

⁹¹For the interested historians among the readers, visiting the following site is recommended: the repository at sourceforge <https://sourceforge.net/projects/foam/files/foam/>, and the archived versions section on the OpenFOAM homepage <https://openfoam.org/download/archive/>.

your trusted author has the source code for, this seems to have always been designed this way. However, the framework got extended over time to accommodate more models; and to be more granular, e.g. to also accommodate thermophysical modelling of solids.

33.1.1 The base classes

Similar to the turbulence modelling framework, there is a hierarchy of base classes, and a hierarchy of class templates. Figure 81 shows the class hierarchy of the base classes for the thermophysical modelling framework.

At the top is the class `basicThermo`, which is derived only from `IOdictionary`. `basicThermo` represents the smallest common denominator of all of OpenFOAM's thermophysical models. This class holds a reference to the pressure field and it contains the temperature field⁹² as well as the laminar thermal diffusivity.

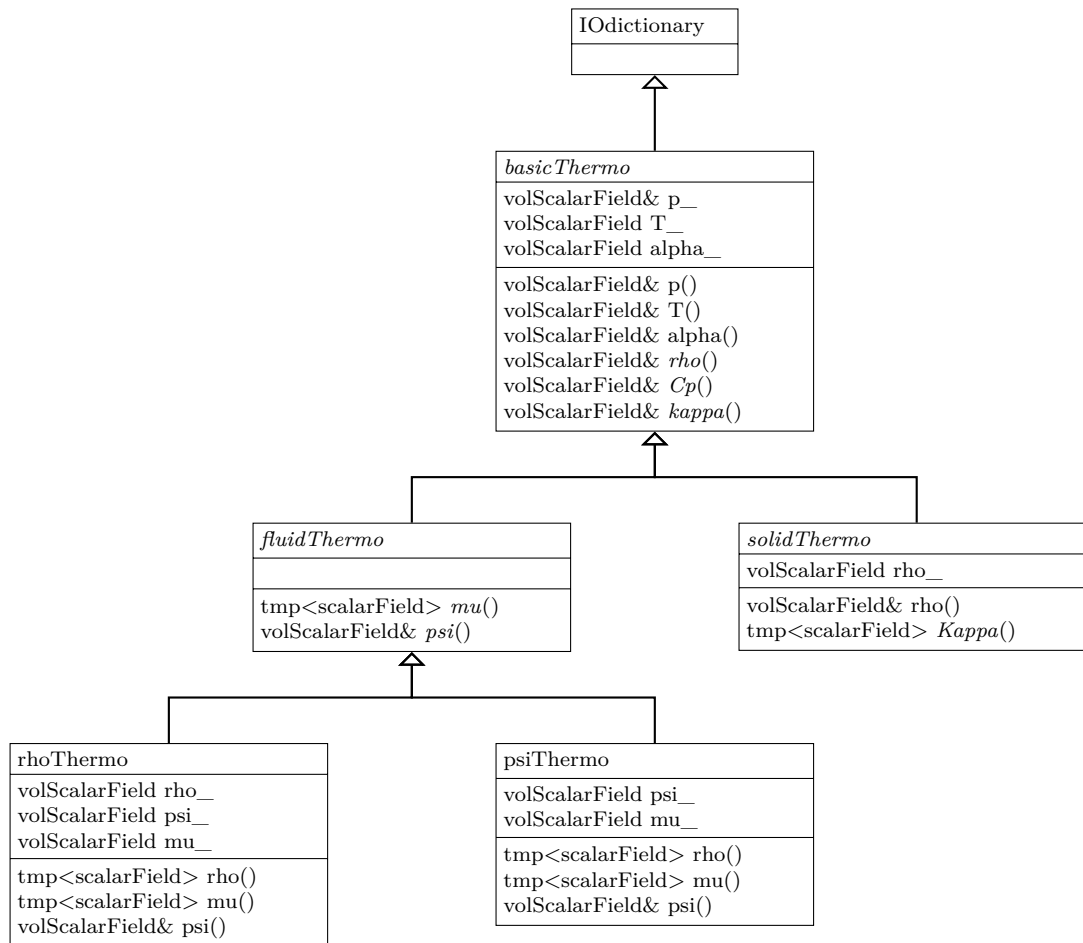


Figure 81: The class hierarchy of the basis of the thermophysical modelling framework in OpenFOAM-7.

The next classes down the family tree are `fluidThermo` and `solidThermo`. The class `solidThermo` is derived only from `basicThermo` and it holds a density field. The `solidThermo` class extends the thermal conductivity `kappa()`, which is provided by `basicThermo`, by an anisotropic thermal conductivity `Kappa()`. Anisotropic heat conduction is a property only solids can exhibit.

The class `fluidThermo` on the other hand is also derived from the class `compressibleTransportModel`, which is not shown in Figure 81. `fluidThermo` does not hold own data members, however it provides methods which are relevant for fluids, e.g. access to viscosity.

From `fluidThermo` the two classes `rhoThermo` and `psiThermo` are derived, which model thermodynamic properties based on density respectively compressibility.

⁹²This is the reason that we can't find any code which reads the temperature field when we study the source code of solvers that involve temperature in some way, e.g. heat-transfer or compressible solvers. Another hint for the thermophysical model being the owner respectively the handler of the temperature field is that in `chtMultiRegionFoam` the temperature is accessed via `thermo.T()`.

The classes discussed in this section are base classes since these classes are used by the solvers of OpenFOAM. See the Listings 201 and 202, in which the thermophysical model is created, and used to access the temperature field.

```

1  Info<< "Reading thermophysical properties\n" << endl;
2
3  autoPtr<rhoThermo> pThermo(rhoThermo::New(mesh));
4  rhoThermo& thermo = pThermo();
5  thermo.validate(args.executable(), "h", "e");
6
7  volScalarField rho
8  (
9      IOobject
10     (
11         "rho",
12         runtime.timeName(),
13         mesh,
14         IOobject::NO_READ,
15         IOobject::NO_WRITE
16     ),
17     thermo.rho()
18 );
19
20 volScalarField& p = thermo.p();

```

Listing 201: An extract of *buoyantPimpleFoam*'s `createFields.H` file. Note the creation and the use of the thermophysical model.

```

1  // Initialise solid field pointer lists
2  PtrList<solidThermo> thermos(solidRegions.size());
3
4  // Populate solid field pointer lists
5  forAll(solidRegions, i)
6  {
7      Info<< "*** Reading solid mesh thermophysical properties for region "
8          << solidRegions[i].name() << nl << endl;
9
10     Info<< "    Adding to thermos\n" << endl;
11     thermos.set(i, solidThermo::New(solidRegions[i]));
12
13     Info<< "    Adding to radiations\n" << endl;
14     radiations.set(i, radiationModel::New(thermos[i].T()));

```

Listing 202: An extract of *chtMultiRegionFoam*'s `createSolidFields.H` file. Note the creation and the use of the thermophysical model.

An interesting observation on the pressure field

In Listing 201, we see that the solver's reference to the pressure field is provided by the thermophysical model, see Line 20 of that listing. As the `basicThermo` model holds only a reference to the pressure field, in contrast, the `basicThermo` model holds the temperature field as a field; i.e. the `basicThermo` "owns" the temperature field but only refers to the pressure field. Thus, we are left with the puzzle who created the pressure field, since the thermophysical model is able to provide the pressure field immediately after its construction, we turn our attention to the base classes of the thermophysical models.

Taking a look at the constructor of the class `basicThermo`, it appears as if the pressure field needs to be already existing, see Line 19 in Listing 203. The method `lookupOrConstruct()` strongly reminds us of the various `lookup*`() methods provided by OpenFOAM's object registry, see Section 57.7. However, this is not the case with this class: the method `lookupOrConstruct()` is provided by the class `basicThermo` itself, see Listing 204. This is actually a pretty elegant way to handle the pressure field.

This approach allows the designers of OpenFOAM not to care about the pressure field in regards to the thermophysical models. E.g. in multiphase solvers, there are two or more thermophysical models constructed, one for each phase, and the constructor of the first thermophysical model creates the pressure field, whereas all subsequent constructor-calls result in a look-up from the object registry.

```

1 Foam::basicThermo::basicThermo
2 (
3     const fvMesh& mesh,
4     const word& phaseName
5 )
6 :
7     IOdictionary
8     (
9         IOobject
10        (
11            phasePropertyName(dictName, phaseName),
12            mesh.time().constant(),
13            mesh,
14            IOobject::MUST_READ_IF_MODIFIED,
15            IOobject::NO_WRITE
16        )
17    ),
18    phaseName_(phaseName),
19    p_(lookupOrConstruct(mesh, "p")),
20    T_
21    (
22        IOobject
23        (
24            phasePropertyName("T"),
25            mesh.time().timeName(),
26            mesh,
27            IOobject::MUST_READ,
28            IOobject::AUTO_WRITE
29        ),
30        mesh
31    ),

```

Listing 203: An extract of a constructor of the class `basicThermo`.

```

1 Foam::volScalarField& Foam::basicThermo::lookupOrConstruct
2 (
3     const fvMesh& mesh,
4     const char* name
5 ) const
6 {
7     if (!mesh.objectRegistry::foundObject<volScalarField>(name))
8     {
9         volScalarField* fPtr
10        (
11            new volScalarField
12            (
13                IOobject
14                (
15                    name,
16                    mesh.time().timeName(),
17                    mesh,
18                    IOobject::MUST_READ,
19                    IOobject::AUTO_WRITE
20                ),
21                mesh
22            )
23        );
24        // Transfer ownership of this object to the objectRegistry
25        fPtr->store(fPtr);
26    }
27    return mesh.objectRegistry::lookupObjectRef<volScalarField>(name);
28 }

```

Listing 204: The method `lookupOrConstruct()` of the class `basicThermo`.

33.1.2 The class templates

heThermo

Thermophysical modelling is all about the modelling of the temperature-dependent behaviour of a fluid or solid. Fully accommodating thermophysical modelling into a solver necessitates the inclusion of the energy conservation equation, which means we need to provide the solver with an energy field.

This energy field can be either the specific enthalpy h or the specific internal energy e . Using the specific enthalpy to solve the energy conservation equation was apparently done right from the start, code that uses h is around in OpenFOAM-1.1. The internal energy field e makes its debut⁹³ in OpenFOAM-1.6 with some of the compressible solvers.

From OpenFOAM-2.2.0 onwards⁹⁴, the class `heThermo` was used to provide the specific enthalpy respectively the specific internal energy. This is possible since both enthalpy and internal energy have the same physical unit, and are closely related.

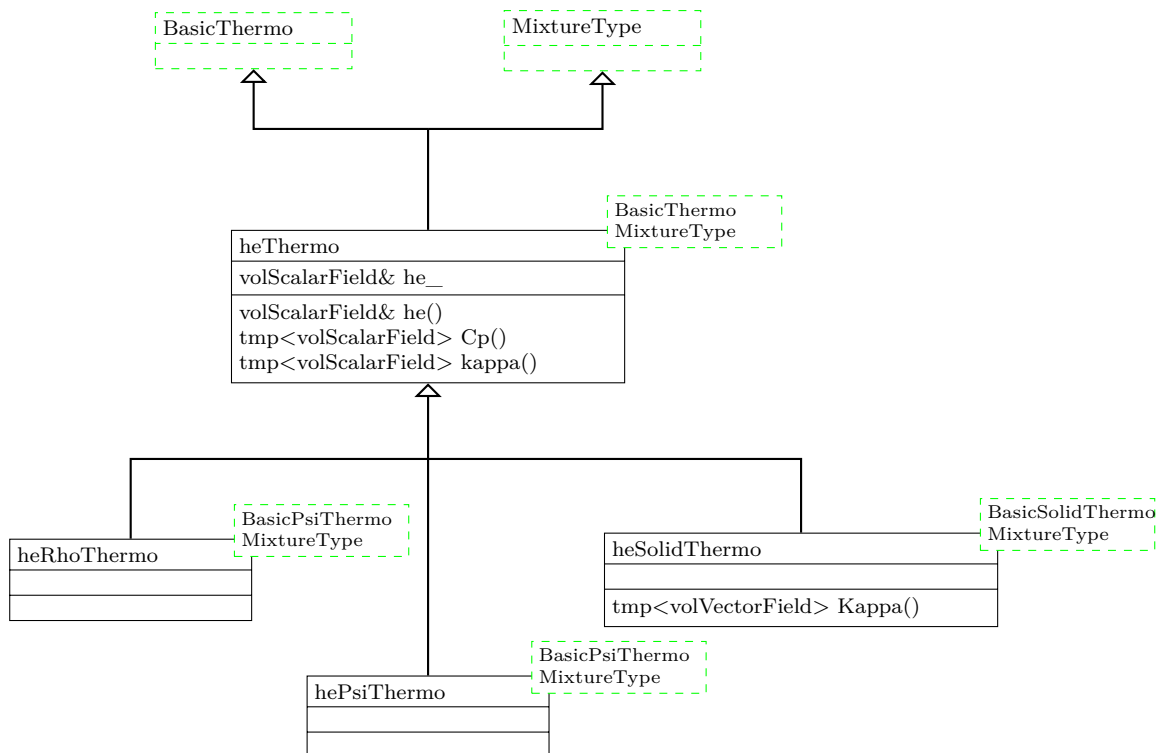


Figure 82: The class hierarchy of the templated classes of the thermophysical modelling framework in OpenFOAM-7.

Figure 82 shows the class hierarchy around the class `heThermo`, which serves the thermophysical modelling needs of both fluids and solids. As the need for a representation of thermal energy is independent of the simulated medium being a fluid or solid, the class providing this representation serves them both.

If we compare Figures 81 and 82, we see that the class `heThermo` provides the methods `Cp()` and `kappa()`, which were abstract methods in the class `basicThermo`. Furthermore, the class `heSolidThermo` now provides the method `Kappa()`, which was an abstract method in the class `solidThermo`.

33.2 Model selection

The framework for thermophysical modelling requires us to select a model for each of seven categories, see Listing 205 for an example. If you are unsure of the available options, just apply the *banana test*, i.e. enter a

⁹³Interestingly, the `ReleaseNotes` file of OpenFOAM-1.6 talks of a **New* (reinstated) =eThermo= thermodynamics package*. Prior to OpenFOAM-1.6, the specific internal energy e was only used by the *sonicFoam* solver in the form of a direct implementation without an underlying thermophysical model based on the internal energy.

⁹⁴<https://openfoam.org/release/2-2-0/thermophysical-multiphase-energy/>

ridiculous value, and OpenFOAM will tell you the valid options.

```
thermoType
{
    type            hePsiThermo;
    mixture         pureMixture;
    transport       sutherland;
    thermo          janaf;
    equationOfState perfectGas;
    specie          specie;
    energy          sensibleInternalEnergy;
}
```

Listing 205: The selection of thermophysical models in the file `thermophysicalProperties` in the `constant` directory.

The only entry in the `thermoType` dictionary, which eludes the *banana test* is the entry for the keyword `specie`, since this is the only available option.

33.3 transport models

The transport model deals among others with the computation of the fluid viscosity as well as thermal conductivity and diffusivity.

33.3.1 const

The `constantTransport` model provides constant properties. The model reads the dynamic viscosity μ and the Prandtl number Pr from its coefficient-dictionary.

$$\kappa = \frac{C_p \mu}{Pr} \quad (34)$$

33.3.2 Sutherland

The Sutherland transport model computes the viscosity from Sutherland's formula [58].

Viscosity

The Sutherland formula computes the dynamic viscosity μ from two model parameters A_s and T_s .

$$\mu = A_s \frac{\sqrt{T}}{1 + T_s/T} \quad (35)$$

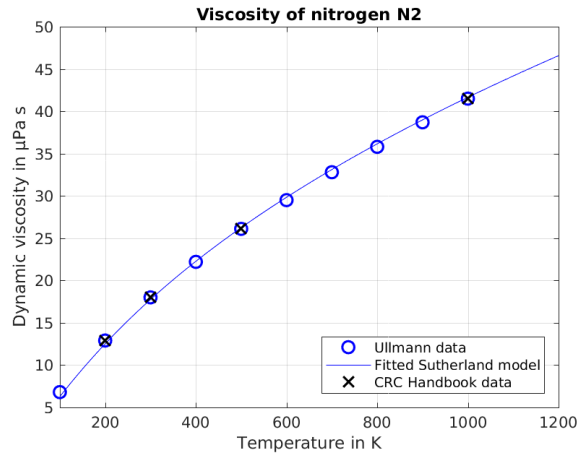


Figure 83: The dynamic viscosity of nitrogen: the model coefficients were fitted to data reported in [14], the model is compared to values reported in [2].

Thermal conductivity

For the thermal conductivity κ a modified Eucken correlation is used [52], which computes the thermal conductivity from the viscosity and other properties. This can be done, since the temperature dependence of the thermal conductivity is approximately the same as the temperature dependence of the viscosity [16, 21, 22].

$$\kappa = \mu c_v \left(1.32 + \frac{1.77 R_s}{c_v} \right) \quad (36)$$

Note that R_s in (36) is the specific gas constant. If we take a look at the actual source code in Listing 206, then we might be under the impression that we need the universal gas constant R . However, this is not the case. See Section 33.6.1 for further information.

In Line 8 of Listing 206 we see the modified Eucken relation from Eq. (36) translated into C++ and OpenFOAM.

```

1  template<class Thermo>
2  inline Foam::scalar Foam::sutherlandTransport<Thermo>::kappa
3  (
4      const scalar p, const scalar T
5  ) const
6  {
7      scalar Cv_ = this->Cv(p, T);
8      return mu(p, T)*Cv_*(1.32 + 1.77*this->R()/Cv_);
9  }
```

Listing 206: The method `kappa` of the class `sutherlandTransport`.

In Figure 84, we compare the thermal conductivity returned by the Sutherland model with data reported in literature [14, 2]. For the specific heat capacity, we used the JANAF model with OpenFOAM's model coefficients for nitrogen, which can be found in the file `$FOAM_ETC/thermoData/thermoData`. The good fit between the values of the Sutherland model and the reported data from literature shows the validity of the modified Eucken correlation.

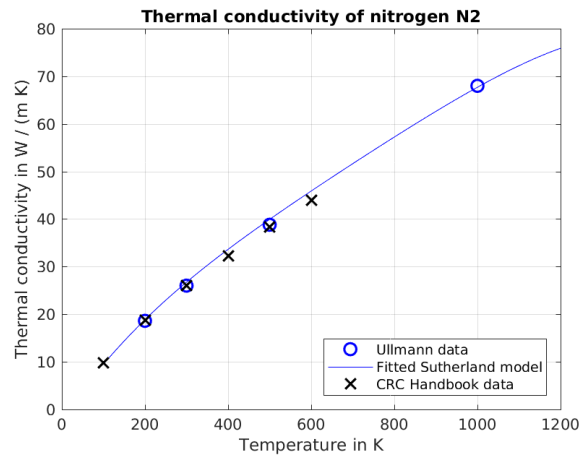


Figure 84: The thermal conductivity of nitrogen: the model is using a modified Eucken correlation with the viscosity model, which was fitted to viscosity data reported in [14], the model is compared to values of thermal conductivity reported in [2, 14].

Pitfall: standard units

When you are fitting model parameters for the Sutherland model to measured data from literature, make sure to use the correct values. OpenFOAM generally uses SI units⁹⁵ for all its computations. Thus, for dynamic

⁹⁵Technically, OpenFOAM can also use USCS units, yet why would one choose to use non-SI units?

viscosity the appropriate unit is the Pascal seconds Pa.s. However, in literature, we often find the values for dynamic viscosity given in micro Pascal seconds $\mu\text{Pa.s}$.

Fitting the model parameters to viscosity data given in $\mu\text{Pa.s}$ will cause the parameter A_s to be too high by a factor of 10^6 . Make sure to use the correct values, when fitting model parameters.

33.4 thermo

The thermo model deals with the computation of the specific heat and other derived properties.

33.4.1 JANAF

The name JANAF stems from the Joint-Army-Navy-Air Force Thermochemical Working Group, which published thermodynamic properties of many compounds from the 1960s onwards. We also find references to JANNAF, which brings the NASA into the abbreviation. Furthermore, the polynomials underlying the JANAF tables are also referred to as the *NASA polynomials* or the *7 term NASA polynomials*⁹⁶.

In the time of limited computer resources, especially memory, polynomials were a very effective way of making the vast body of experimentally determined thermodynamic data usable for computing. Thus, huge amounts of data were reduced by virtue of polynomial-fitting to a small, finite number of polynomial coefficients [13]. Furthermore, the recursive formulation of (38) is considered most efficient in terms of the involved computational operations and memory [13].

The JANAF model is one of the thermo models provided by OpenFOAM. The JANAF model depends on two sets of model coefficients. The `lowCpCoeffs` are used when the temperature is below the threshold `Tcommon`, when temperatures are higher the coefficients in `highCpCoeffs` are used. Mathematically spoken, the JANAF model implements the following equation:

$$a = \begin{cases} \text{lowCpCoeffs} & T_{\text{low}} \leq T < T_{\text{common}} \\ \text{highCpCoeffs} & T_{\text{common}} \leq T \leq T_{\text{high}} \end{cases} \quad (37)$$

$$C_p = (((a_4 T + a_3)T + a_2)T + a_1)T + a_0 \quad (38)$$

The JANAF model coefficients are two sets of 7 coefficients. Using two polynomials allows to cover a wide temperature range. Thus, the first polynomial usually covers the low temperature region, 200-1000 K, whereas the second covers the high temperature region, 1000-6000 K. The two polynomials are constrained to yield the 1000 K value. Furthermore, the polynomial for the low temperature region is *pinned* to 298.15 K, which serves, in combination with a pressure of 1 bar, as the standard reference state. Pinning the polynomial to the standard state ensures, that the thermodynamic properties are exactly reproduced at the standard state [13].

The first 5 coefficients of a polynomial are used for computing the specific heat capacity C_p , as shown above. The remaining two coefficients are used to compute the enthalpy and the entropy. In Listing 207 an example, for the way JANAF model coefficients are defined, is shown.

```
mixture
{
    thermodynamics
    {
        Tlow          200;
        Thigh         6000;
        Tcommon       1000;
        highCpCoeffs  ( 3.08793 0.00124597 -4.23719e-07 6.74775e-11 -3.97077e-15 -995.263 5.95961 );
        lowCpCoeffs   ( 3.5684 -0.000678729 1.55371e-06 -3.29937e-12 -4.66395e-13 -1062.35 3.71583 );
    }
}
```

Listing 207:
The model coefficients of the JANAF thermophysical model in the file `thermophysicalProperties` in the `constant` directory.

In OpenFOAM's `etc/thermoData/thermoData` the JANAF coefficients for a large number of species can be found. However, if we want to use the JANAF model outside of OpenFOAM, e.g. in a MATLAB/Octave script, it turns out that these coefficients need to be multiplied by the specific gas constant of the respective species. As shown in Listing 208, this is done by the constructor automatically, see Lines 14 and 15 of Listing 208.

⁹⁶http://combustion.berkeley.edu/gri_mech/data/nasa_plnm.html

```

1  template<class EquationOfState>
2  Foam::janafThermo<EquationOfState>::janafThermo(const dictionary& dict)
3  :
4      EquationOfState(dict),
5      Tlow_(readScalar(dict.subDict("thermodynamics").lookup("Tlow"))),
6      Thigh_(readScalar(dict.subDict("thermodynamics").lookup("Thigh"))),
7      Tcommon_(readScalar(dict.subDict("thermodynamics").lookup("Tcommon"))),
8      highCpCoeffs_(dict.subDict("thermodynamics").lookup("highCpCoeffs")),
9      lowCpCoeffs_(dict.subDict("thermodynamics").lookup("lowCpCoeffs"))
10 {
11     // Convert coefficients to mass-basis
12     for (label coefLabel=0; coefLabel<nCoeffs_; coefLabel++)
13     {
14         highCpCoeffs_[coefLabel] *= this->R();
15         lowCpCoeffs_[coefLabel] *= this->R();
16     }
17
18     checkInputData();
19 }

```

Listing 208: The constructor of the class `janafThermo`.

In Figure 85 the specific heat capacity of air computed with the JANAF model is compared to data from [62].

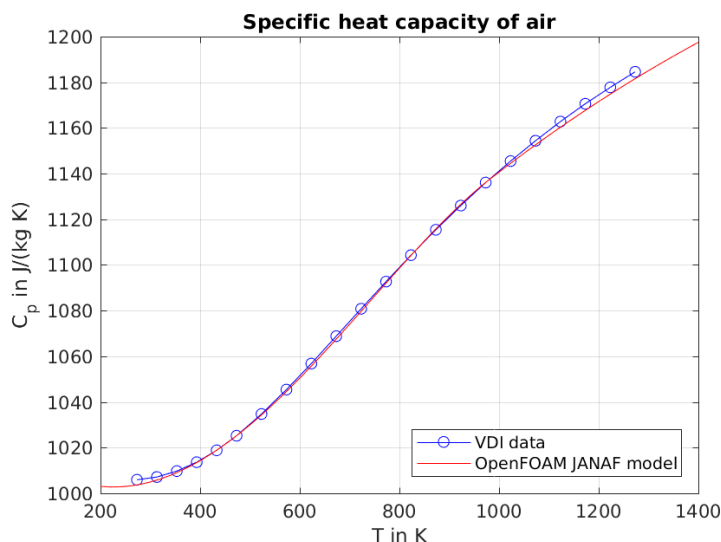


Figure 85: Specific heat capacity C_p of air computed with the JANAF model compared to data from [62].

33.5 equationOfState

The equation of state covers the computation of the fluid's density.

33.5.1 rhoConst

The `rhoConst` model provides a constant density. The model reads a density value from its coefficient dictionary.

33.5.2 perfectGas

This model uses the eponymous perfect gas model to compute the density from the other two state variables, i.e. pressure and temperature. This model does not require a coefficient dictionary, as it uses three states of the fluid (pressure, density and temperature), and the specific gas constant, which can readily be determined.

$$\rho = \frac{p}{R_s T} \quad (39)$$

Pitfall: zero pressure

Some rather odd behaviour was observed, when tinkering with cases. In this instance, files from an incompressible case ended up in an compressible case, which had the `perfectGas` selected as the `equationOfState`. Apart from a looming error due to the different dimensions of pressure in incompressible and compressible cases, another error was triggered.

As it is common with incompressible cases, the internal field value for the pressure was set to zero. When the compressible solver was launched, the following, quite cryptic error message was issued by OpenFOAM.

```
Selecting turbulence model type laminar
Selecting laminar stress model Stokes
Creating field dpdt

Creating field kinetic energy K

No MRF models present

No finite volume options present

#0 Foam::error::printStack(Foam::Ostream&) at ???
#1 Foam::sigFpe::sigHandler(int) at ???
#2 ? in "/lib/x86_64-linux-gnu/libc.so.6"
#3 Foam::divide(Foam::Field<double>&, Foam::UList<double> const&, Foam::UList<double> const&)
   at ???
#4 Foam::operator/(Foam::UList<double> const&, Foam::UList<double> const&) at ???
#5 ? at ???
#6 __libc_start_main in "/lib/x86_64-linux-gnu/libc.so.6"
#7 ? at ??? Floating point exception (Core dumped)
```

Listing 209: A quite un-informative error message of *rhoPimpleFoam*.

The error message looks different, when a turbulence model other than laminar is selected. In this case, the error message is slightly more informative, as the error occurs within the turbulence model framework. If such an error occurs there, then the error message contains at least the method name, in which the error happened.

At some point, at the construction of all the necessary models, OpenFOAM inevitably divides by the density, which evaluates to zero in the case of zero pressure. In the case shown above, the laminar model was selected, which does not need to divide by the density.

The remarkable thing with the error message in Listing 209 is, that it contains no information other than that a floating point exception occurred during dividing numbers.

33.6 Thermophysical properties

33.6.1 Pitfall: the gas constant(s)

The class `specie` provides the method `R()` to return the gas constant.

```
1  //- Gas constant [J/kg/K]
2  inline scalar R() const;
```

Listing 210: The declaration of the method `R()` in the header-file of the class `specie`.

```
1  inline scalar specie::R() const
2  {
3      return RR/molWeight_;
4  }
```

Listing 211: The implementation of the method `R()` in the header-file of the class `specie`.

The implementation in Listing 211 strongly suggests that the method `R()` returns a specific gas constant. This is confirmed by looking at Listing 212.

```
1  //- Universal gas constant (default in [J/kmol/K])
2  extern const scalar RR;
```

Listing 212: The declaration of the universal gas constant in the file `thermodynamicConstants.H`.

In this case, regarding the universal and specific gas constants, OpenFOAM's notation is not following the widely used convention to denote the universal gas constant as R , and the specific gas constant as R_s [48].

33.7 Mixture

OpenFOAM can not only deal with pure fluids, it can also deal with mixtures: e.g. a combustion solver solves for momentum and energy transport for a single gas phase, whereas the gas phase consists of the gaseous fuel (methane), the gaseous oxidizer (oxygen), the gaseous oxidation products (carbon-dioxide) and inert gases (nitrogen). The properties of the gas phase are evaluated on a cell-by-cell basis for the mixture of all gaseous species.

33.7.1 pureMixture

For a pure fluid, a dictionary named `mixture` contains the model definitions for all necessary sub-models.

```
mixture
{
    specie
    {
        //
    }

    thermodynamics
    {
        //
    }

    transport
    {
        //
    }
}
```

Listing 213: The definition of thermophysical modelling in the file `thermophysicalProperties` for a pure fluid.

33.7.2 multiComponentMixture

Thermophysical modelling

With the `multiComponentMixture` model, we need to provide a list of species, and a definition the thermophysical modelling for each individual species. The thermophysical modelling definition of the individual species follows the same pattern as the definition of the thermophysical modelling for pure fluids, see Section 33.7.1.

```
species
(
    Ar
    H2
);

Ar
{
    //
}

H2
{
    //
}
```

Listing 214: The definition of thermophysical modelling in the file `thermophysicalProperties` for a mixture of several fluids.

Composition

The composition of a `multiComponentMixture` is specified with the mass fractions of the individual components. The user can provide either a `Ydefault` field, if the mass fractions are to be initialized equally; or a mass fraction field for each component. In the example of Listing 214, a user can provide the components' mass fractions as fields with the name of the components, i.e. `Ar` and `H2`. We can also provide specific mass fraction fields for a subset of components, in this case the value of `Ydefault` is applied to the remaining components.

Upon construction of the mixture, the mass fractions are normalized, such that the sum of all mass fractions is equal to one. Thus, the values in the mass fraction fields do not necessarily sum-up to one.

The mixture's properties are computed using the mass fractions, e.g. below we see the relation for the mixture's density depending of the components' mass fractions and densities.

$$m_{\text{mix}} = m_1 + m_2 \quad (40)$$

$$m_{\text{mix}} = \rho_{\text{mix}} V_{CV} \quad (41)$$

$$m_{\text{mix}} = \rho_1 V_1 + \rho_2 V_2 \quad (42)$$

$$m_{\text{mix}} = \rho_{\text{mix}} (V_1 + V_2) \quad (43)$$

with the following relation for the individual species' mass m_i

$$m_i = Y_i m_{\text{mix}} = \rho_i V_i \quad (44)$$

we can derive a relation for the mixture density

$$m_{\text{mix}} = \rho_{\text{mix}} \left(\frac{Y_1 m_{\text{mix}}}{\rho_1} + \frac{Y_2 m_{\text{mix}}}{\rho_2} \right) \quad (45)$$

$$1 = \rho_{\text{mix}} \left(\frac{Y_1}{\rho_1} + \frac{Y_2}{\rho_2} \right) \quad (46)$$

$$\rho_{\text{mix}} = \frac{1}{\frac{Y_1}{\rho_1} + \frac{Y_2}{\rho_2}} \quad (47)$$

33.8 Model combination

Note, that models can not be freely combined.

33.8.1 Polynomial models

If you would like to use polynomial models, i.e. properties such as viscosity are computed from a polynomial depending on temperature, then you are basically restricted to use polynomial models for the categories `transport`, `thermo` and `equationOfState`. In Listing 215 all possible model combinations involving a polynomial model are shown. This list was generated by applying the *banana test* and filtering OpenFOAM's output for the occurrence of the expression `olynomial`, which is common to all polynomial models.

There are some combinations involving only one or two polynomial models.

hePsiThermo	pureMixture	polynomial	hPolynomial	PengRobinsonGas	specie	sensibleEnthalpy
hePsiThermo	pureMixture	polynomial	janaf	PengRobinsonGas	specie	sensibleEnthalpy
heRhoThermo	multiComponentMixture	polynomial	hPolynomial	icoPolynomial	specie	sensibleEnthalpy
heRhoThermo	multiComponentMixture	polynomial	hPolynomial	icoPolynomial	specie	sensibleInternalEnergy
heRhoThermo	pureMixture	polynomial	hPolynomial	PengRobinsonGas	specie	sensibleEnthalpy
heRhoThermo	pureMixture	polynomial	hPolynomial	icoPolynomial	specie	sensibleEnthalpy
heRhoThermo	pureMixture	polynomial	hPolynomial	icoPolynomial	specie	sensibleInternalEnergy
heRhoThermo	pureMixture	polynomial	janaf	PengRobinsonGas	specie	sensibleEnthalpy
heRhoThermo	reactingMixture	polynomial	hPolynomial	icoPolynomial	specie	sensibleEnthalpy
heRhoThermo	reactingMixture	polynomial	hPolynomial	icoPolynomial	specie	sensibleInternalEnergy

Listing 215: The selection of thermophysical models in the file `thermophysicalProperties` in the `constant` directory.

33.9 Solids : Thermophysical modelling

With the introduction of conjugate heat transfer (CHT) modelling, the need for thermophysical models for solids arises. In OpenFOAM, CHT was introduced with OpenFOAM-1.5 and OpenFOAM-1.6. With OpenFOAM-2.0.0, thermophysical modelling for solids was introduced. Prior to this version, all relevant fields, e.g. specific heat capacity or thermal conductivity, were simply read from disk.

The available models for solids are a subset of those for fluids. Some of the models can be used by both fluids and solids, whereas in other cases there are model specifically for solids.

33.9.1 transport

Similar to the thermophysical models for fluids, the **transport** model for solids is responsible to provide the thermal conductivity. However, other than with the fluids, no viscosity is provided.

constant

The **constant** model is the simplest available model. It takes only one model parameter, i.e. the thermal conductivity.

```
transport
{
    kappa          57.4781;
}
```

Listing 216: The model coefficients of the **constant** transport model in the file **thermophysicalProperties** in the **constant** directory.

polynomial

The **polynomial** model takes the polynomial coefficients as model parameters.

```
transport
{
    kappaCoeffs<8> (127.821 -0.267 2.844e-04 -8.84e-08 -1.3e-08 9.3e-12 -3.6e-15 5.6e-19);
}
```

Listing 217: The model coefficients of the **polynomial** transport model in the file **thermophysicalProperties** in the **constant** directory.

exponential

The **exponential** model requires three model parameters, the constant coefficient κ_0 , the exponent n_0 and the reference temperature T_{ref} .

$$\kappa = \kappa_0 \left(\frac{T}{T_{\text{ref}}} \right)^{n_0} \quad (48)$$

```
transport
{
    kappa0          0.166;
    n0              1.6187e-12;
    Tref            0.17385;
}
```

Listing 218: The model coefficients of the **exponential** transport model in the file **thermophysicalProperties** in the **constant** directory.

constant-anisotropic

A special case of solid materials are anisotropic properties, e.g. fiber composites have a distinct two-dimensional internal structure as they are made from layers of fabric infused with resin. This may lead to a variation of properties depending on whether we consider a material property in the plane of the fibers or perpendicular to this plane. Wood is another prominent example of a solid material exhibiting anisotropic properties.

The **constAnIso** model allows the user to specify the heat conductivity κ as a vector.

```

transport
{
    kappa          (42.0 48.0 42.0);
}

```

Listing 219: The model coefficients of the `constant` transport model in the file `thermophysicalProperties` in the `constant` directory.

In addition to the thermal conductivity in the form of a vector, the user must also provide a coordinate system. The necessity of reading the coordinate system is due to the solver *chtMultiRegionFoam*. This is useful, when the relevant local coordinates of the solid are not aligned with the global coordinate system of the simulation domain.

Pitfall: phase change

Some solid metals undergo a phase change, in which their crystalline structure changes, e.g. when iron changes from ferrite to austenite as it is heated up. Figure 86 shows the thermal conductivity of nickel, which decreases with increasing temperature, and at some point (its phase transition) increases with increasing temperature [27].

From the available models, only the `polynomial` model is capable to incorporate a transition of the kind shown in Figure 86. The `exponential` model only depict a strictly monotonic increase or decrease, however, not a transition from one to the other.

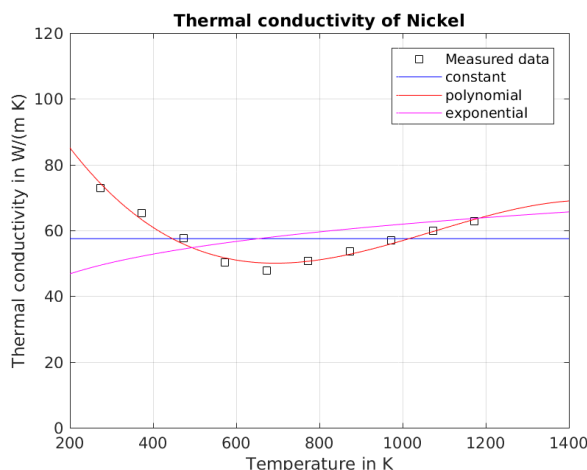


Figure 86: The thermal conductivity of nickel: data from literature [27], and the available, fitted models of OpenFOAM.

33.9.2 thermo

The `thermo` model is, similar to the case of fluids, responsible to provide the specific heat capacity, the entropy and various enthalpies.

In contrast to fluids, which offer models formulated in terms of enthalpy or internal energy, the thermophysical modelling of energy in solids is based solely on enthalpy.

hConst

The `hConst` model provides constant values. It reads in the specific heat capacity c_P and the standard enthalpy of formation H_f .

hPower

The `hPower` model computes the specific heat capacity according to Eq. (49). The model takes the model parameters of Eq. (49) and the standard enthalpy of formation H_f as inputs.

$$C_p(T) = c_0 \left(\frac{T}{T_{\text{ref}}} \right)^{n_0} \quad (49)$$

hPolynomial

The **hPolynomial** model computes its properties using user-provided polynomial coefficients. Additionally, it takes the standard enthalpy of formation H_f and the standard entropy S_f as model parameters.

33.9.3 equationOfState

rhoConst

The only valid equation of state for thermophysical modelling of solids is **rhoConst**.

33.9.4 energy

sensibleEnthalpy

The only valid model for energy transport for thermophysical modelling of solids is **sensibleEnthalpy**.

33.9.5 Model combination

Listing 220 shows the available models for solids in OpenFOAM-7. The model choice for solids is a small subset of the models available for fluids.

heSolidThermo	multiComponentMixture	constIso	hConst	rhoConst	specie	sensibleEnthalpy
heSolidThermo	pureMixture	constAnIso	hConst	rhoConst	specie	sensibleEnthalpy
heSolidThermo	pureMixture	constIso	hConst	rhoConst	specie	sensibleEnthalpy
heSolidThermo	pureMixture	exponential	hPower	rhoConst	specie	sensibleEnthalpy
heSolidThermo	pureMixture	polynomial	hPolynomial	rhoConst	specie	sensibleEnthalpy

Listing 220: The available thermophysical models for solids; determined with a *banana test* with the solver **chtMultiRegionFoam**.

34 Radiation modelling

Radiative heat transfer needs to be considered when temperatures are quite high. OpenFOAM offers several models to account for radiative heat transfer: the discrete ordinates model (fvDOM), the P1 model and the view factor model.

34.0.1 Solver control

The base class for all radiation models (**radiationModel**) provides controls on how often the radiation model needs to be solved. Since the radiation models can be computationally quite expensive, and temperature changes might be slower than the dynamics of the fluid flow, we may want to solve for radiation only every N-th time step.

This is achieved by the entry for the **solverFreq** keyword in the file **radiationProperties** dictionary. This keyword expects an integer value, which defaults to 1, and is also bounded to be at least 1 to prevent mischief caused by a user entering a negative value.

```

1 void Foam::radiationModel::initialise()
2 {
3     solverFreq_ = max(1, lookupOrDefault<label>("solverFreq", 1));
4
5     absorptionEmission_.reset
6     (
7         radiationModels::absorptionEmissionModel::New(*this, mesh_).ptr()
8     );
9
10    scatter_.reset(radiationModels::scatterModel::New(*this, mesh_).ptr());

```

```

11
12     soot_.reset(radiationModels::sootModel::New(*this, mesh_).ptr());
13 }

```

Listing 221: The `initialise()` method of the `radiationModel` class.

34.0.2 Radiative heat flux

Each of the radiation models writes, among other fields, the total radiative heat flux `qr` to disk. The sign-convention for the radiative heat flux is such⁹⁷, that it is positive, when the surface absorbs the heat flux. This is in line with the sign-convention for face normal vectors on a patch.

However, the convention for the wall heat flux, which is determined by a function object of that name is opposite. With the `wallHeatFlux` function object, a positive heat flux means that the heat flows from the wall into the domain.

34.1 Discrete ordinates - fvDOM

In OpenFOAM, the discrete ordinates (DO) model is named `fvDOM`. The eponymous discrete ordinates are a finite number of spatial directions, along which radiative heat transport is solved for. Listing 222 shows the diagnostic output of the model's constructor listing these directions.

```

fvDOM: Created 60 rays with average directions (dAve) and solid angles (omega)
Ray I0: dAve = (0.010235 0.0381976 0.0904495), omega = 0.0999985
Ray I1: dAve = (0.0279626 0.0279626 0.0904495), omega = 0.0999985
Ray I2: dAve = (0.0381976 0.010235 0.0904495), omega = 0.0999985
Ray I3: dAve = (0.0381976 -0.010235 0.0904495), omega = 0.0999985
Ray I4: dAve = (0.0279626 -0.0279626 0.0904495), omega = 0.0999985

```

Listing 222: The constructor of the `fvDOM` radiation model is reporting the discretized directions for radiative heat transport.

Each direction has its own radiative intensity field named `ILambda_XX_YY`, which has two suffixes: the first (XX) is a counter for the direction, and the second suffix (YY) is a counter for the wavelength band. Listing 223 shows an example of this naming scheme, which is taken from OpenFOAM's tutorials. In this specific case, we solve for 60 discrete ordinates, which is indicated in Listing 222, and for a single wavelength band. Thus, the first suffix runs from 0 to 59; and the second counter has only the value 0.

Since specifying identical BCs for each of the dozens of radiative intensity fields, the `fvDOM` model offers the possibility to specify BCs using a field named `IDefault`, which applies to all the fields.

```

Time = 910
DILUPBiCGStab: Solving for Ux, Initial residual = 0.000484957, Final residual = 8.59355e-07, No Iterations 1
DILUPBiCGStab: Solving for Uy, Initial residual = 0.000636378, Final residual = 1.3728e-06, No Iterations 1
DILUPBiCGStab: Solving for Uz, Initial residual = 0.00100732, Final residual = 1.64924e-06, No Iterations 1
DILUPBiCGStab: Solving for h, Initial residual = 0.000646664, Final residual = 9.12454e-07, No Iterations 1
Radiation solver iter: 0
GAMG: Solving for ILambda_0_0, Initial residual = 0.000289555, Final residual = 7.99208e-16, No Iterations 1
GAMG: Solving for ILambda_1_0, Initial residual = 0.000281662, Final residual = 7.86555e-16, No Iterations 1
GAMG: Solving for ILambda_2_0, Initial residual = 0.00028829, Final residual = 7.91439e-16, No Iterations 1
...
GAMG: Solving for ILambda_59_0, Initial residual = 0.00312848, Final residual = 2.98335e-06, No Iterations 1
DICPCG: Solving for p_rgh, Initial residual = 0.00253219, Final residual = 2.296e-05, No Iterations 35
time step continuity errors : sum local = 1.18082e-06, global = 1.41217e-19, cumulative = -7.81505e-19
rho max/min : 1.16452 0.69867
DILUPBiCGStab: Solving for epsilon, Initial residual = 0.000330527, Final residual = 6.62334e-06, No Iterations 1
DILUPBiCGStab: Solving for k, Initial residual = 0.000939773, Final residual = 1.06366e-05, No Iterations 1
ExecutionTime = 654.13 s ClockTime = 655 s

```

Listing 223: An extract of the solver output when using the `fvDOM` radiation model. The solver output for the radiation model follows the output of the solution for the momentum and energy equations. For each of the discrete ordiantes a transport equation for the radiative intensity is solved.

When the solver writes a time step to disk, the radiative intensity field for each discrete ordiante is written into the time step folder. The `fvDOM` radiation model also writes some additional fields to disk, which are according to the description in the header file of the `fvDOM` model:

a Total absorption coefficient [1/m]

⁹⁷See this note on OpenFOAM's bug tracker explaining the convention: <https://bugs.openfoam.org/view.php?id=2722#c9101>.

G Incident radiation [W/m^2]

qin Incident radiative heat flux [W/m^2]

qem Emitted radiative heat flux [W/m^2]

qr Total radiative heat flux [W/m^2]

34.2 P1

The P1 model is suited for optically thick problems, i.e. the medium through which the radiation is passing through does absorb or scatter some of the incident radiation.

34.3 View factor

The **viewFactor** model has been released with OpenFOAM-2.0.0⁹⁸. This model only considers radiative heat transfer between surfaces. Thus, this model is suitable for simulation cases with vacuum between hot surfaces, or for cases with non-participating gases. Non-participating gases do neither absorb nor scatter thermal radiation. Such gases are mono-atomic gases, such as argon or helium, or symmetric diatomic gases, such as oxygen, nitrogen or hydrogen [15, 29].

The announcement in the release notes of OpenFOAM-2.0.0 has the following to say about this model:

... view factor model for radiative heat transfer, specifically between surfaces. The method begins with the generation of rays between discrete faces of the surfaces, using the **viewFactorsGen** utility in OpenFOAM. Radiative heat transfer is then calculated by summing energy exchanges between ray end-points. A major benefit of this approach is that energy is only exchanged between parts of surface that are directly visible to one another, giving a representative solution to complex problems where some surfaces are shielded from radiative sources by others.

The view factor model is also referred to as *surface-to-surface (S2S)* model, e.g. in Fluent [7].

34.3.1 Pre-processing

The **viewFactor** model requires some pre-processing, which is a 2 step process that involves the following pre-processing tools:

faceAgglomerate

The announcement of the **viewFactor** model in the release notes of OpenFOAM-2.0.0 has the following to say about the computational requirements of the **viewFactor** model:

The computational time and memory requirement of the modelling is largely determined by the number of faces from which the rays emanate. In OpenFOAM, the cost can be reduced by grouping faces together using the **faceAgglomerate** pre-processing utility.

Hence, the **faceAgglomerate** tool is used in preparation to create a coarser representation of the surface to reduce computational effort. In Fluent, using face agglomeration to reduce the computational effort is referred to as *clustering* [7].

viewFactorsGen

This tool does the actual creation of rays between the involved surfaces, respectively it computes the view factors between the agglomerated faces.

⁹⁸<https://openfoam.org/release/2-0-0/thermophysical-modelling/>

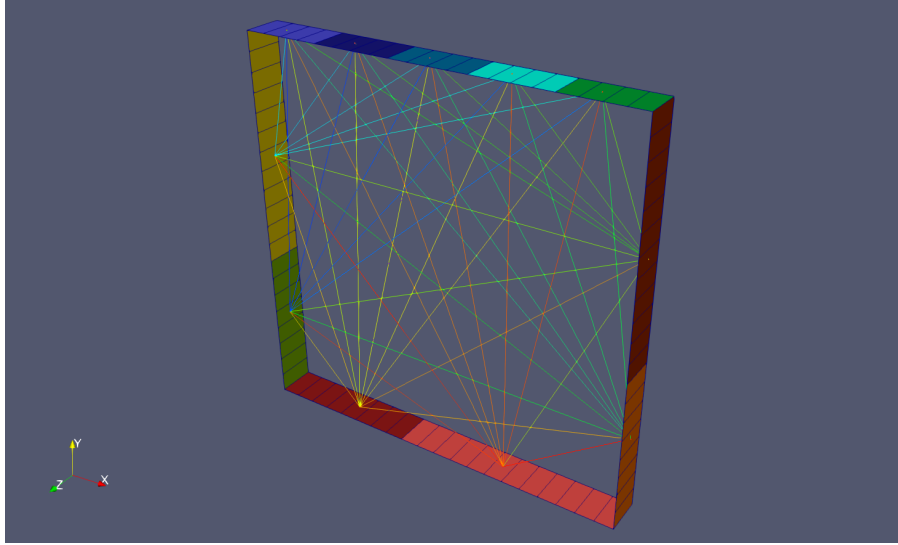


Figure 87: The result of `faceAgglomerate` and `viewFactorGen` applied to a slightly refined cavity case. The patches *movingWall* and *fixedWalls* have both been agglomerated, i.e. for the purposes of computing the view factor, several faces of a patch form a coarsened face. This is evident from the visualisation of the rays created by `viewFactorGen`, each bundle of rays starts from the center of such a coarsened face. The colour of the rays is a mere ID, not the actual view factor.

34.3.2 Visualising the pre-processing results

Inspecting the results of the pre-processing steps can be quite important, see Section 34.3.5. Also, you might wonder how Figure 87 was created.

Write the face-agglomeration

Enabling the `writeFacesAgglomeration` switch in the `viewFactorDict` dictionary causes `faceAgglomerate` to write a field named `facesAgglomeration`, which contains a trivial internal field, yet the boundary values correspond to a running index of the agglomerates. I.e. all faces with the value `X` belong to the agglomerate `X`.

Dump the rays

Enabling the `dumpRays` switch in the `viewFactorDict` dictionary causes `viewFactorsGen` to write a file named `allVisibleFaces.vtk`, which contains the rays from each agglomerate to all other agglomerates. These rays are shown in Figure 87 as lines connecting all agglomerates with each other.

34.3.3 Model use

The `viewFactor` model requires an additional field in the case setup, the new radiative heat flux `qr`. The unit of `qr` is given in the header description of the `viewFactor` model as W/m^2 . In the field definition of `qr`, the dimension set corresponds to the unit kg/s^3 , which works out to be the same unit once the unit Watt is replaced by its SI representation.

The model furthermore reads the `finalAgglom` field, which is created by the `faceAgglomerate` tool.

34.3.4 Solution procedure

Once the view factors are determined by the pre-processing tools, the model can be used in a simulation. The `viewFactor` model essentially boils down to an algebraic equation linking the net radiative heat flux with the surface temperature via the view factors and other properties.

The model equation underlying the `viewFactor` model is the following:

$$C_{ij}q_i = b_i \quad (50)$$

$$\left[\delta_{ij} \frac{1}{E_j} - \left(\frac{1}{E_j} - 1 \right) F_{ij} \right] q_i = (\delta_{ij} - F_{ij}) \sigma T_i^4 \quad (51)$$

With a static mesh, the view factors are constant, and the only major task for the **viewFactor** model is to compute the inverse of an $N \times N$ matrix **C**, with N being the number of coarsened faces. As **C** contains only constant entries, the inversion only needs to be done once, at the first iteration of the solver.

Thus, for big simulation cases, we can expect the **viewFactor** model to take long during pre-processing, especially when creating the rays using **viewFactorGen**; and it will take especially long, when it computes the inverse of **C** during the first iteration. This is compounded by the fact, that the computation of the radiative flux happens on the master-process, i.e. parallelisation does not accelerate the **viewFactor** model, since the major computational work is done by a single process.

Afterwards, however, the **viewFactor** model doesn't incur significant computational effort anymore, since the inverse of **C** is cached, and during each iteration, computing the radiative heat flux reduces to a simple matrix-vector multiplication. With a cached inverse matrix, we can solve Eq. (50) very efficiently over and over, since only the RHS changes from time step to time step.

The discussion above, about the computational effort, can be better understood if we take a look at the relevant code, shown in Listing 224. There, we see that the **viewFactor** model is only then efficient, when the emissivities are constant, otherwise we are not able to benefit from caching the inverse of the matrix **C**, since it would change from time step to time step.

```

1  if (Pstream::master())
2  {
3      // Variable emissivity
4      if (!constEmissivity_)
5      {
6          scalarSquareMatrix C(totalNCoarseFaces_, 0.0);
7
8          // build equation system - code removed for brevity
9
10         Info<< "\nSolving view factor equations..." << endl;
11         // Negative coming into the fluid
12         LUsolve(C, q);
13     }
14     else // Constant emissivity
15     {
16         // Initial iter calculates CLU and caches it
17         if (iterCounter_ == 0)
18         {
19             // build equation system - code removed for brevity
20
21             if (debug)
22             {
23                 InfoInFunction
24                     << "\nDecomposing C matrix..." << endl;
25             }
26             LUDecompose(CLU_(), pivotIndices_);
27         }
28
29         // build equation system - code removed for brevity
30
31         if (debug)
32         {
33             InfoInFunction
34                 << "\nLU Back substitute C matrix..." << endl;
35         }
36         LUBacksubstitute(CLU_(), pivotIndices_, q);
37         iterCounter_ ++;
38     }
39 }

```

Listing 224: A snippet of the **calculate()** method of the **viewFactor** model.

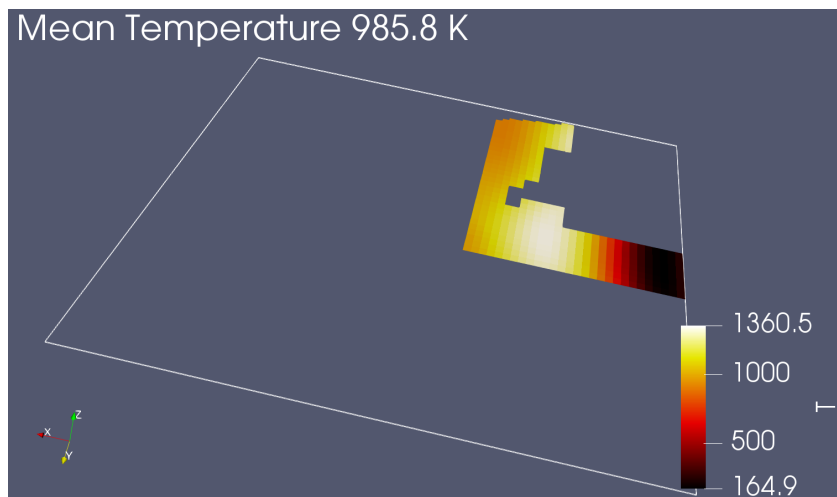


Figure 88: One of the aggressively agglomerated faces: On the right boundary the temperature is 300 K, yet the radiative heat-flux computed by the mean temperature cools the solid down to 165 K, which is very unphysical for a case that computes a heated solid interacting with a body of gas. No cooling should occur at all.

34.3.5 Pitfall: too aggressive faceAgglomeration

The pre-processing operation of face-agglomeration is an important one, since it reduces the computational effort for computing the view-factors by orders of magnitude. However, don't be too aggressive with face-agglomeration, at least in regions in which you expect large temperature gradients.

In Figure 88 we see the result of an overly aggressive agglomeration, agglomeration if you like. The case in question is a solid which is volumetrically heated by Joule heating, i.e. an electrical current passing through the solid. The solid is embedded in a fluid region with a gas (nitrogen) as working fluid. The large agglomerate causes the radiative heat transfer locally to be under- or overestimated. In regions colder than the mean temperature of the agglomerate, the radiative heat transfer acts to cool the local cells. In regions hotter than the mean temperature of the agglomerate, the heat transfer is underestimated, thus, these regions get hotter than they ought to be.

Always check the face-agglomeration, i.e. view it in ParaView and judge whether it is fine enough in regions where you expect temperature gradients, as the radiative heat-transfer depends on the fourth power of the temperature, even a moderate within an agglomerate can lead to great error.

35 Eulerian multiphase modelling

In Eulerian two-phase modelling both phases are considered continua even though one phase might consist of dispersed phase elements (DPEs) such as bubbles, drops or particles. In these simulations the two phases can be distinguished into a continuous phase and a dispersed phase. This naming scheme refers to the physical situation. Within the (Eulerian) mathematical description, however, both phases are continua.

As two momentum equations are solved (one per phase), each phase has its own velocity field. However, there is only one pressure field. Thus, the pressure is the same for both phases; this also applies to the VOF method. Due to the fact that two continuity⁹⁹ and two momentum equations are solved, this approach is often referred to as *two fluid model*.

The Eulerian description of multi-phase flow is not limited to two phases, however, for reasons of simplicity, we limit ourselves to the case of two phases.

⁹⁹The constraint that the sum of all volume fraction fields must yield unity, i.e. $\sum_i \alpha_i \stackrel{!}{=} 1$, allows for one continuity equation to be eliminated. In the case of two phases, only one continuity equation needs to be solved. However, both continuity equation can be combined.

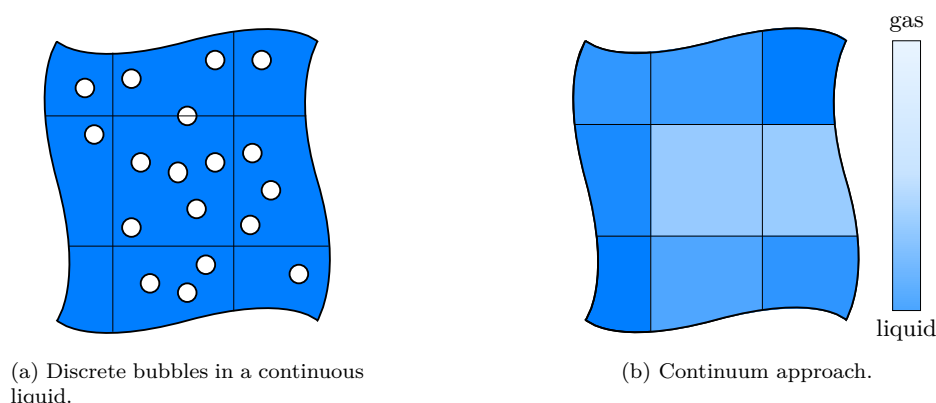


Figure 89: Modelling approach on the example of a gas-liquid two-phase system.

As the DPEs are considered to be a continuous phase, their properties are averaged over each cell of the computational domain. Thus, the properties of the dispersed phase are the mean properties of the dispersed matter. If all DPEs have equal properties (e.g. diameter, density, etc.), then the dispersed phase is referred to as being *mono-disperse*. Only in the case of mono-dispersity, the averaging over the cells introduces no additional errors. If the DPEs have variable properties (e.g. a diameter range), then the dispersed phase is referred to as being *poly-disperse*. The correct handling of poly-dispersity requires additional considerations on the models.

35.1 Phase model class

One of the strenghts of object oriented programming is that the class structure in the source code can reflect the properties and relations of real-world things.

The phase model class in the various two- and multi-phase solvers of OpenFOAM is one example of how techniques of object oriented programming can be applied. In terms of a multi-phase problem in fluid dynamics we distinguish different phases.

We now violate the unwritten law of *to not cite Wikipedia*^[Citation needed].

Phase (matter), a physically distinctive form of a substance, such as the solid, liquid, and gaseous states of ordinary matter—also referred to as a "macroscopic state"

<http://en.wikipedia.org/wiki/Phase>

In fluid dynamics phase is a commonly used term. When we intend our code to represent the reality we want to describe as closely as possible we need to introduce the concept of the phase into our source code. From a programming point of view properties of a phase – such as viscosity, velocity, etc. – are easy to implement. The viscosity of a phase is simply a field of values, velocity is another field of values.

Object orientation allows us to translate the idea of the phase into programming language. The basic idea is that a phase has a viscosity, it also has a velocity. We now create a class named `phaseModel` and this class needs to have a viscosity, a velocity and everthing else a phase needs to fit our needs.

The phase model classes follow the code of best practice in object oriented programming to hide internal data from the outer world and to provide access via the classes methods (data encapsulation, see http://www.tutorialspoint.com/cplusplus/cpp_data_encapsulation.htm).

No phases, please

In the single-phase solvers of OpenFOAM – such as *simpleFoam* – the concept of a phase is not used. As there is only one temperature and velocity to deal with, the concept of phases is not needed. In the single-phase solvers the phase-properties (viscosity, velocity, density, etc.) are linked according to the physical relations that are taken into account, but the concept of a phase is missing.

35.1.1 A comparison of the phase models in OpenFOAM-2.2

In this section we want to compare the implementation of the phase model class of the two solvers *twoPhaseEulerFoam* and *multiPhaseEulerFoam*.

twoPhaseEulerFoam

The phase model class in *twoPhaseEulerFoam-2.2.x* collects the properties of a phase and offers an interface for accessing these properties. Listing 225 shows the essence of the header file of the phase model class. The listing is syntactically correct, however all pre-processor instruction (e.g. the `#include` statements) have been removed. Furthermore, most of the comments have been removed and the formatting has been adapted to reduce the line number. The purpose of Listing 225 is to present the data members and methods of the class by actual source code.

```
1 namespace Foam
2 {
3
4 class phaseModel
5 {
6     // Private data
7     dictionary dict_;
8     word name_;
9     dimensionedScalar d_;
10    dimensionedScalar nu_;
11    dimensionedScalar rho_;
12    volVectorField U_;
13    autoPtr<surfaceScalarField> phiPtr_;
14
15 public:
16     // Member Functions
17     const word& name() const { return name_; }
18
19     const dimensionedScalar& d() const { return d_; }
20
21     const dimensionedScalar& nu() const { return nu_; }
22
23     const dimensionedScalar& rho() const { return rho_; }
24
25     const volVectorField& U() const { return U_; }
26
27     volVectorField& U() { return U_; }
28
29     const surfaceScalarField& phi() const { return phiPtr_(); }
30
31     surfaceScalarField& phi() { return phiPtr_(); }
32 };
33
34 } // End namespace Foam
```

Listing 225: A boiled-down version of the file `phaseModel.H`

The phase model class of *twoPhaseEulerFoam-2.2.x* contains all phase properties needed for an incompressible two-phase solver that makes use of an important consequence of being limited to two phase problems. By taking a look on the members of the class we see that there is no volume fraction field. In two phase problems one volume fraction field (`alpha1`) suffices as the volume fraction field of the other phase is instantly known (`alpha2 = 1 - alpha1`). Thus, the volume fraction can be treated separately from other phase information.

Another missing item is the pressure. Most two- or multi-phase Eulerian solvers assume/use a common pressure for all phases. Thus, the pressure is independent of the phases and can be treated separately.

multiphaseEulerFoam

One difference between the phase model class used in *twoPhaseEulerFoam* and the one used in *multiphaseEulerFoam* follows directly from the simplification made in the two-phase case. When dealing with an arbitrary number of phases, each phase must keep track of its own volume fraction. Thus, the volume fraction must be included into the phase model.

The straight-forward way would be to add another reference to the data members. As the volume fraction field is a scalar field, this reference would be a reference to a `volScalarField`. In *multiphaseEulerFoam* a more subtle approach was chosen. This also presents the application of another object-oriented programming technique.

The phase model class of *multiphaseEulerFoam* is derived from the class `volScalarField`. Thus, the phase model class is among other things its own the volume fraction field.

Listing 226 shows a stripped version of the header file of *multiphaseEulerFoam*'s phase model class. Again, large parts of the file have been removed leaving only the data members and the methods of the class.

```

1 namespace Foam
2 {
3
4 class phaseModel
5 :
6     public volScalarField
7 {
8     // Private data
9     word name_;
10    dictionary phaseDict_;
11    dimensionedScalar nu_;
12    dimensionedScalar kappa_;
13    dimensionedScalar Cp_;
14    dimensionedScalar rho_;
15    volVectorField U_;
16    volVectorField DDtU_;
17    surfaceScalarField phiAlpha_;
18    autoPtr<surfaceScalarField> phiPtr_;
19    autoPtr<diameterModel> dPtr_;
20
21 public:
22
23     // Member Functions
24     const word& name() const { return name_; }
25
26     const word& keyword() const { return name(); }
27
28     tmp<volScalarField> d() const;
29
30     const dimensionedScalar& nu() const { return nu_; }
31
32     const dimensionedScalar& kappa() const { return kappa_; }
33
34     const dimensionedScalar& Cp() const { return Cp_; }
35
36     const dimensionedScalar& rho() const { return rho_; }
37
38     const volVectorField& U() const { return U_; }
39
40     volVectorField& U() { return U_; }
41
42     const volVectorField& DDtU() const { return DDtU_; }
43
44     volVectorField& DDtU() { return DDtU_; }
45
46     const surfaceScalarField& phi() const { return phiPtr_(); }
47
48     surfaceScalarField& phi() { return phiPtr_(); }
49
50     const surfaceScalarField& phiAlpha() const { return phiAlpha_; }
51
52     surfaceScalarField& phiAlpha() { return phiAlpha_; }
53
54     void correct();
55
56     bool read(const dictionary& phaseDict);
57 };
58
59 } // End namespace Foam

```

Listing 226: A boiled-down version of the file `phaseModel.H`

The statements following the class keyword and the class name indicates the derivation of a class. The class name (`phaseModel`) and the name of the class we are deriving from (`volScalarField`) are separated by a colon (`:`). The name of the base class (`volScalarField`) is preceded by a visibility specifier (`public`). Here, we see a prototype of a class definition. The class we define (`phaseModel`) is derived from a base class (`volScalarField`).

```

class phaseModel : public volScalarField
{
    /* some c++ code */
}

```

This example highlights, that the class `phaseModel` is derived from the class `volScalarField`. This information alone does no proof that the phase model is its own volume fraction field. However, a glance on the constructor in the implementation file brings clarity.

In Listing 227 we see, that the first instruction in the initialisation list of the constructor reads the volume fraction field of the respective phase. This proves that the phase model is in fact its own volume fraction field. For an explanation why we come to this conclusion we refer to any C++ textbook or on-line resource that covers the concept of inheritance, see e.g. <http://www.learncpp.com/cpp-tutorial/114-constructors-and-initialization-of-derived-classes/> or [57].

```

// * * * * * Constructors * * * * *
Foam::phaseModel::phaseModel
(
    const word& name,
    const dictionary& phaseDict,
    const fvMesh& mesh
)
:
    volScalarField
    (
        IOobject
        (
            "alpha" + name,
            mesh.time().timeName(),
            mesh,
            IOobject::MUST_READ,
            IOobject::AUTO_WRITE
        ),
        mesh,
        name_(name),
        // code continues
    )

```

Listing 227: The first few lines of the constructor of the phase model.

Besides being its own volume fraction field the phase model class of *multiphaseEulerFoam* was extended by several fields bearing information for the simulation of thermodynamics.

We can also observe the rudiment of giving the phase model a more active role. The phase model class of *twoPhaseEulerFoam* is simply an information carrier. The phase model of *multiphaseEulerFoam* features a method named `correct()`. The `correct()` method is used in many models for actions performed at every time step. However, in *multiphaseEulerFoam-2.2.x* this method is empty.

With OpenFOAM-2.1.0 the class `diameterModel` was introduced into *multiphaseEulerFoam* and *compressibleTwoPhaseEulerFoam*. The phase model class of *multiphaseEulerFoam* uses a diameter model class for keeping track of the dispersed phase's diameter. The diameter model offers the choice of computing the diameter of the dispersed phase elements from thermodynamic quantities besides using a constant diameter. Thus, the data member `dimensionedScalar d_` is replaced by a reference to a diameter model (`autoPtr<diameterModel> dPtr_`).

35.1.2 A comparison of the phase models in OpenFOAM-2.3

In this section we want to compare the implementation of the phase model class of the two solvers *twoPhaseEulerFoam* and *multiphaseEulerFoam*.

A comment on *multiphaseEulerFoam*

The phase model class used for *multiphaseEulerFoam* in OpenFOAM-2.2.x and OpenFOAM-2.3.x differs very little with respect to the class's methods and members. Listing 228 shows that the header files of the `phaseModel` class of *multiphaseEulerFoam* differs only in the copyright notice. The implementation file shows slightly greater

differences¹⁰⁰. However, the behaviour of this class can be considered nearly identical in OpenFOAM-2.2.x and OpenFOAM-2.3.x.

```

user@host:~/OpenFOAM$ diff
  OpenFOAM-2.2.x/applications/solvers/multiphase/multiphaseEulerFoam/phaseModel/phaseModel/
    phaseModel.H
  OpenFOAM-2.3.x/applications/solvers/multiphase/multiphaseEulerFoam/multiphaseSystem/
    phaseModel/phaseModel.H
5c5
<  \ \ /      A nd          | Copyright (C) 2011 OpenFOAM Foundation
---
>  \ \ /      A nd          | Copyright (C) 2011-2013 OpenFOAM Foundation

```

Listing 228: The output of *diff* for the file `phaseModel.H` of the solver *multiphaseEulerFoam* of the versions OpenFOAM-2.2.x and OpenFOAM-2.3.x as of May 2014¹⁰¹.

twoPhaseEulerFoam

The two-phase model of *twoPhaseEulerFoam*-2.3.x makes heavy use of abstractions. The phase model class is used in conjunction with a class for the two-phase system.

```

1 namespace Foam
2 {
3
4 class phaseModel
5 :
6     public volScalarField,
7     public transportModel
8 {
9     // Private data
10     const twoPhaseSystem& fluid_;
11     word name_;
12     dictionary phaseDict_;
13     scalar alphaMax_;
14     autoPtr<rhoThermo> thermo_;
15     volVectorField U_;
16     surfaceScalarField alphaPhi_;
17     surfaceScalarField alphaRhoPhi_;
18     autoPtr<surfaceScalarField> phiPtr_;
19     autoPtr<diameterModel> dPtr_;
20     autoPtr<PhaseCompressibleTurbulenceModel<phaseModel>> turbulence_;
21
22 public:
23
24     // Member Functions
25     const word& name() const { return name_; }
26
27     const twoPhaseSystem& fluid() const { return fluid_; }
28
29     const phaseModel& otherPhase() const;
30
31     scalar alphaMax() const { return alphaMax_; }
32
33     tmp<volScalarField> d() const;
34
35     const PhaseCompressibleTurbulenceModel<phaseModel>&
36         turbulence() const;
37
38     PhaseCompressibleTurbulenceModel<phaseModel>&
39         turbulence();
40
41     const rhoThermo& thermo() const { return thermo_(); }
42
43     rhoThermo& thermo() { return thermo_(); }

```

¹⁰⁰The *diff* of the implementation file would be too long to be shown at this place. For general information on *diff* see Section ??.

¹⁰¹OpenFOAM Builds compared: 2.2.x-61b850bc107b and 2.3.x-0eb39ebe0f07.


```

44     tmp<volScalarField> nu() const { return thermo_>nu(); }
45
46     tmp<scalarField> nu(const label patchi) const { return thermo_>nu(patchi); }
47
48     tmp<volScalarField> mu() const { return thermo_>mu(); }
49
50     tmp<scalarField> mu(const label patchi) const { return thermo_>mu(patchi); }
51
52     tmp<volScalarField> kappa() const { return thermo_>kappa(); }
53
54     tmp<volScalarField> Cp() const { return thermo_>Cp(); }
55
56     const volScalarField& rho() const { return thermo_>rho(); }
57
58     const volVectorField& U() const { return U_; }
59
60     volVectorField& U() { return U_; }
61
62     const surfaceScalarField& phi() const { return phiPtr_(); }
63
64     surfaceScalarField& phi() { return phiPtr_(); }
65
66     const surfaceScalarField& alphaPhi() const { return alphaPhi_; }
67
68     surfaceScalarField& alphaPhi() { return alphaPhi_; }
69
70     const surfaceScalarField& alphaRhoPhi() const { return alphaRhoPhi_; }
71
72     surfaceScalarField& alphaRhoPhi() { return alphaRhoPhi_; }
73
74     void correct();
75
76     virtual bool read(const dictionary& phaseProperties);
77
78     virtual bool read() { return true; }
79 };
80
81 } // End namespace Foam
82

```

Listing 229: A boiled-down version of the file `phaseModel.H`

The data members of the phase model class in *twoPhaseEulerFoam-2.3.x* contain a reference to the two-phase model class. This makes the phase model class aware of the other phase. The data members also contain a reference to a turbulence model and a thermophysical model. This is up to now the greatest generalisation we could observe in the multi-phase solvers of OpenFOAM.

35.2 Phase system classes

In a multiphase solver we can not only create an abstraction for the physical phase, e.g. water. We can also create an abstraction for the multi-phase system, i.e. the entirety of the involved phases. Again, *multiphaseEulerFoam* was the forerunner for this idea. Since the introduction of *multiphaseEulerFoam* there is a class named `multiphaseSystem`. In *twoPhaseEulerFoam-2.3* the class `twoPhaseSystem` was introduced. The most obvious purpose of this class is the implementation of the phase continuity equation. In both solvers the solution of the continuity equation(s) hides behind the function call `fluid.solve()`.

35.2.1 The class `twoPhaseSystem`

We now take a detailed look on the class `twoPhaseSystem`. This class was introduced with *twoPhaseEulerFoam-2.3* and this class seems to be a consequent continuation of ideas introduced in the class `multiphaseSystem`. We focus on the class `twoPhaseSystem`, since the class `multiphaseSystem` has not really evolved from the release of OpenFOAM-2.1 til the release of OpenFOAM-2.3. The header and the implementation file are largely identical.

Phase models

Two data members of the class are the two involved phase models `phase1_` and `phase2_`. The class provides methods to access this phase models. There is also a method to access the other phase. As there are only two

phases involved, this operation is possible.

Phase pair models

In order to cover all possible flow situations the momentum exchange models are defined in the case pair-wise in a separated fashion, i.e. drag for air dispersed in water (bubbly flow) and drag for water dispersed in air (droplet flow).

The classes `phasePair` and `orderedPhasePair` provide an elegant way to deal with this situation. The phase pair models are used for blending the interfacial momentum exchange models.

Momentum exchange models

The class has member variables for the interfacial momentum exchange models. Listing 230 shows the members of the class related to momentum exchange models. The templated class `BlendedInterfacialModel<>` provides functionality that is needed for all momentum exchange models. As the class name suggests, the blending is covered by this class. The template parameter of this class stands for any one of the interfacial momentum exchange models.

```

1      //- Drag model
2      autoPtr<BlendedInterfacialModel<dragModel> > drag_;
3      //- Virtual mass model
4      autoPtr<BlendedInterfacialModel<virtualMassModel> > virtualMass_;
5      //- Heat transfer model
6      autoPtr<BlendedInterfacialModel<heatTransferModel> > heatTransfer_;
7      //- Lift model
8      autoPtr<BlendedInterfacialModel<liftModel> > lift_;
9      //- Wall lubrication model
10     autoPtr<BlendedInterfacialModel<wallLubricationModel> > wallLubrication_;
11     //- Wall lubrication model
12     autoPtr<BlendedInterfacialModel<turbulentDispersionModel> > turbulentDispersion_;

```

Listing 230: The declaration of the momentum exchange members of the class `twoPhaseSystem` in `twoPhaseSystem.H`

A momentum exchange model alone is nice, but what we really need are the contribution to the momentum equation. Thus, the class `twoPhaseSystem` provides methods to access the respective force terms or the respective coefficients. We have seen this force terms and coefficients in action in Section 45.6.

```

1      //- Return the drag coefficient
2      tmp<volScalarField> dragCoeff() const;
3      //- Return the virtual mass coefficient
4      tmp<volScalarField> virtualMassCoeff() const;
5      //- Return the heat transfer coefficient
6      tmp<volScalarField> heatTransferCoeff() const;
7      //- Return the lift force
8      tmp<volVectorField> liftForce() const;
9      //- Return the wall lubrication force
10     tmp<volVectorField> wallLubricationForce() const;
11     //- Return the wall lubrication force
12     tmp<volVectorField> turbulentDispersionForce() const;

```

Listing 231: The declaration of the accessing methods for the momentum exchange coefficients of the class `twoPhaseSystem` in `twoPhaseSystem.H`

35.2.2 The class `multiphaseSystem`

The solver *multiphaseEulerFoam* uses the class `multiphaseSystem`. This class seems to be the ancestor of the class `twoPhaseSystem`.

Phase pair

The class `multiphaseSystem` declares a nested class `interfacePair`. A nested class is a class definition within another class. Thus, the nested class is hidden from the outside world¹⁰².

The phase pair class is used to deal with surface tension, which by definition is a property of a pair of phases, and drag.

35.3 Turbulence modelling

35.3.1 Modelling strategies

The problem of turbulence modelling in multi-phase problems can be tackled in one of the following fashions. The methods are sorted by their perceived computational cost. Whereas the first two methods may be equivalent, the last is definitely more expensive in terms of memory and computational time. However, each of these methods has its strengths and weaknesses, and its use cases.

Continuous phase only This model solves computes the turbulent properties of the continuous phase and assumes an algebraic relationship between the turbulent properties of the continuous and the dispersed phase. The influence of turbulence on the dispersed phase can also be neglected altogether. In the Fluent Theory Guide [7] it is noted: [...] *is the appropriate model when the concentrations of the secondary phases are dilute. In this case, interparticle collisions are negligible and the dominant process in the random motion of the secondary phases is the influence of the primary-phase turbulence.* In Fluent this approach is referred to as *dispersed turbulence model*.

Mixture In this approach the turbulence model is evaluated for the mixture of all phases, i.e. the mixture velocity and mixture density are inserted into the turbulence model. The turbulent quantities of each individual phase are computed with the density ratio between the mixture and the corresponding phase. The applicability of this model is described in the Fluent Theory Guide [7] as follows: [...] *is applicable when phases separate, for stratified (or nearly stratified) multiphase flows, and when the density ratio between phases is close to 1.*

Per-phase In this case each phase has its own turbulent properties. Because there are additional transport equations to be solved per phase, this model is the most computational intensive. The Fluent Theory Guide [7] states: [...] *is the appropriate choice when the turbulence transfer among the phases plays a dominant role.*

35.3.2 Implementation in OpenFOAM

In Section 32.1 the frameworks for implementing turbulence modelling within OpenFOAM are discussed. Now we take a look on multi-phase turbulence and OpenFOAM's frameworks for modelling turbulence.

The old framework, see Section 32.1.1, allow only for the first two of the described strategies, since only one turbulence model is employed by the multiphase solvers. The turbulence model is generally a global object within the solver, as is also the mesh or the run-time object.

The new framework allows for greater flexibility. In the Eulerian multiphase solvers, the turbulence model has been moved to the phase model. Thus, each phase has its own turbulence model. This allows for all three modelling strategies discussed in Section 35.3.1. The turbulence modelling employed by *twoPhaseEulerFoam* within the new framework is discussed in Section 45.4.

35.4 Interfacial momentum exchange

On the RHS of the momentum equation there are two types of source terms. The first term $\mathbf{F}_{q,i}$ is a force density acting on the phase q . The second term is a force (density) coefficient $K_{qp,i}$ which is multiplied by the relative velocity $\mathbf{u}_R = \mathbf{u}_p - \mathbf{u}_q$ between the phases q and p .

The models for interfacial momentum transfer in OpenFOAM are implemented in a way, such that these models return either a force or a force coefficient¹⁰³. The distinction between forces and force coefficients is a

¹⁰²See http://pic.dhe.ibm.com/infocenter/compgb/v121v141/topic/com.ibm.xlcpp121.bg.doc/language_ref/cplr061.html for details.

¹⁰³The correct denomination would be force density and force density coefficient. In the source files of OpenFOAM related to these models, $\mathbf{F}_{q,i}$ and $K_{qp,i}$ are referred to as force and force coefficient, most probably for the sake of reducing typing effort. As OpenFOAM keeps track of the physical units of its variables, we can see from the actual source codes, that the force $\mathbf{F}_{q,i}$ is in fact a force density.

matter of convenience. Contributions directly proportional to the velocity, e.g. drag, can be treated differently than contributions indirectly proportional to the velocity, e.g. the virtual mass force which is proportional to the time derivative of the relative velocity. Terms directly proportional to the velocity are numerically treated differently than other terms.

The interfacial momentum transfer due to drag, lift and virtual mass are based on the force acting on a single bubble. The turbulent dispersion force is observed when the turbulent eddies of the liquid phase interact with a swarm of bubbles. This interaction tends to disperse bubble swarms [44]. Figure 90 gives a schematic representation of the different momentum exchange mechanisms between the liquid and the gas phase.

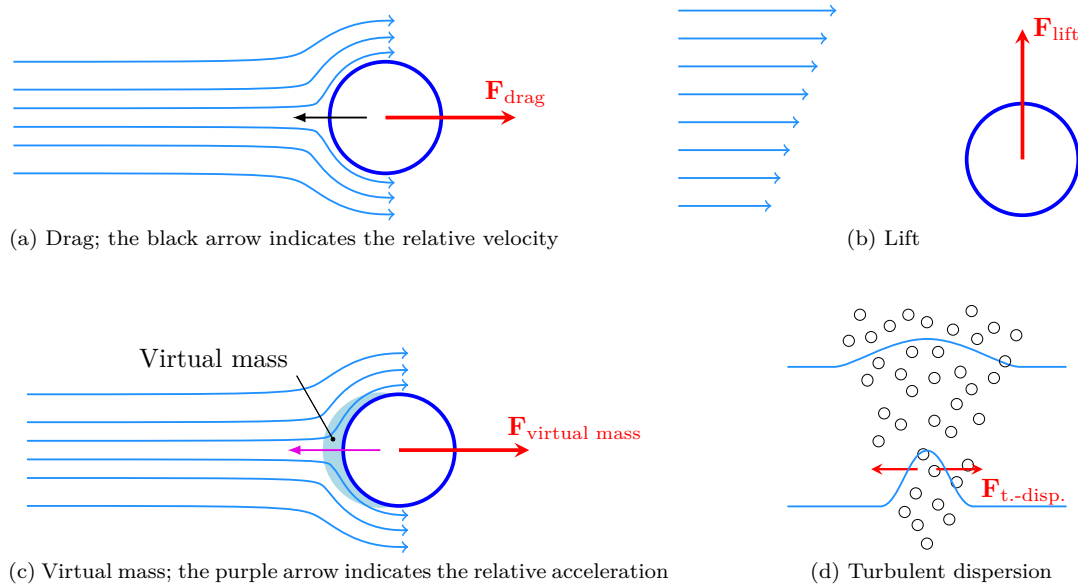


Figure 90: Modelling approach on the example of a gas-liquid two-phase system.

35.5 Diameter models

As mentioned in the previous Section, diameter models were introduced at some point in the multiphase models. The *multiphaseEulerFoam* offered since its introduction in version 2.1.0 two diameter models (constant and isothermal). With *twoPhaseEulerFoam-2.3* a further diameter model was introduced, which is available only in *twoPhaseEulerFoam*.

OpenFOAM	Constant, no model	Constant	Isothermal	IATE
<i>twoPhaseEulerFoam</i>				
2.0.x	x			
2.1.x	x			
2.2.x	x			
2.3.x		x	x	x
<i>multiphaseEulerFoam</i>				
2.1.x		x	x	
2.2.x		x	x	
2.3.x		x	x	

Table 4: Overview of diameter modelling in Eulerian multiphase solvers

35.5.1 No model

The older versions of *twoPhaseEulerFoam* ($\leq 2.2.x$) use no model for the diameter of the dispersed phase elements (DPE). In all of these versions the phase diameter is a scalar of type **dimensionedScalar** that is read

from the `transportProperties` dictionary.

35.5.2 Constant

The `constantDiameter` diameter model is the implementation of a constant diameter in a framework that allows for a variable diameter.

Internally, the diameter is still a scalar which is read from `transportProperties` respectively from `phaseProperties`. However, the phase model returns the diameter as a field quantity. Listing 232 shows how a `volScalarField` is returned. The private variable `d_` is of the type `dimensionedScalar`.

```

1 Foam::tmp<Foam::volScalarField>
2 Foam::diameterModels::constant::d()
3 const
4 {
5     return tmp<Foam::volScalarField>
6     (
7         new volScalarField
8         (
9             IOobject
10            (
11                "d",
12                phase_.U().time().timeName(),
13                phase_.U().mesh()
14            ),
15            phase_.U().mesh(),
16            d_
17        )
18    );
19 }
```

Listing 232: Accessing the diameter in `constantDiameter`.

35.5.3 Isothermal

Gas bubbles change their diameter as the ambient pressure changes. The `isothermalDiameter` model implements this behaviour by assuming the change of state to be isothermal.

Generally, the ideal gas law (52) governs the state of a gas.

$$pV = nRT \quad (52)$$

under the assumption of an isothermal state

$$pV = \text{const} \quad (53)$$

Next we introduce the bubble volume

$$V = \frac{d^3 \pi}{6} \quad (54)$$

Thus, we gain the relation

$$p_1 d_1^3 \frac{\pi}{6} = p_2 d_2^3 \frac{\pi}{6} \quad (55)$$

This leads to the isothermal diameter model

$$d_2 = \sqrt[3]{d_1 \frac{p_1}{p_2}} \quad (56)$$

For the `isothermalDiameter` model the user needs to specify a reference pressure and diameter. Listing 233 shows the `d()` method of the class `isothermalDiameter`. The reference pressure `p0_` and diameter `d0_` are private data members of the class¹⁰⁴. With Eqn. (56) the local diameter is computed (Line 10).

¹⁰⁴An underscore (`_`) as suffix to the variable name apparently indicates private variables. Although the coding style guidelines of OpenFOAM (<http://openfoam.org/contrib/code-style.php>) do not explicitly say so. However, this is recommended style by other communities, e.g. <http://geosoft.no/development/cppstyle.html>.

```

1 Foam::tmp<Foam::volScalarField>
2 Foam::diameterModels::isothermal::d()
3 const
4 {
5     const volScalarField& p = phase_.U().db().lookupObject<volScalarField>
6     (
7         "p"
8     );
9
10    return d0_*pow(p0_/p, 1.0/3.0);
11 }

```

Listing 233: The method `d()` of the class `isothermalDiameter`.

35.5.4 IATE

IATE stands for *interfacial area transport equation*. This model is based on [33]. The IATE diameter model solves a transport equation for the interfacial curvature `kappai_`.

Solves for the interfacial curvature per unit volume of the phase rather than interfacial area per unit volume to avoid stability issues relating to the consistency requirements between the phase fraction and interfacial area per unit volume.

Class description in `IATE.H`

In Section 64 we cover the derivation of the governing equations implemented in OpenFOAM from the equations in [33].

35.6 Thermophysical models

Each phase in a multi-phase simulation is assigned its own set of models with regards to thermo-physical properties, such as density, dynamic viscosity and specific heat capacity. The set of available models is discussed in Section 33. In this section, we focus on the use of thermophysical modelling with respect to multiphase simulations.

35.6.1 A note on using thermophysical properties in BCs

If we want to specify a massflow boundary condition in a multiphase simulation, we need to specify which density is to be used for computing the velocity BC from the mass flow. Listing 234 shows an example, of how we can specify massflow BCs. Note, the entry `rho`, which tells OpenFOAM which density to use to compute the actual velocity BC.

```

boundaryField
{
    inlet
    {
        type                flowRateInletVelocity;
        massFlowRate        0.2;
        rho                  thermo:rho.air;
        extrapolateProfile  no;
        value                uniform (0 0 0);
    }
    // ...
}

```

Listing 234: Specifying a massflow BC in an `U.air` file, here we need to specify the phase-density.

In multiphase solvers, access to the density field is generally handled by the phase-model class. Thus, the multiphase solvers do not create registered density fields, in contrast to compressible single-phase solvers, such as `rhoPimpleFoam`. As access to the density is handled via the phase model's thermophysical model, this reflects in the way we refer to the phase's density when we e.g. specify a massflow BC.

Note, that in Line 7 of Listing 234 a single colon (`:`) is used as a separator. This can potentially be confused with the double colons (`::`), which are used in C++ to separate namespace qualifiers. However, in a certain way, we find that this is also a sort of namespace, since the `thermo` prefix indicates that this field is a field which is provided by the thermophysical model, as opposed to solver-managed fields, such as `p` or `U`.

35.6.2 A note on post-processing thermophysical quantities

Section 35.6.1 also applies, when we want to do some post-processing on thermophysical quantities, e.g. write out the gas-density field as shown in Listing 235. Here, we also use a single colon (:) to separate the **thermo** qualifier from the name of the field, i.e. **rho.air**. Using a double colon, or just the field's name alone would result in an error.

```
writeObjects1
{
    type            writeObjects;
    libs            ("libutilityFunctionObjects.so");

    objects        (
        thermo:rho.air
    );
    writeOption    anyWrite;
}
```

Listing 235: Writing the gas-density field to disk, using the **writeObjects** function object.

36 Boundary conditions

When the geometry of a problem is meshed, then the boundary patches – i.e. the faces delimiting the geometry – need to be specified. Every boundary patch is of a certain type. In Section 36.1 the possible types are discussed.

36.1 Base types

36.1.1 Geometric boundaries

Some kinds of boundary patches can be described purely geometrically. The numerical treatment of this kind of patches is inherently clear to the solver and needs no more modelling.

symmetry plane If a problem is symmetric, then only half of the domain needs to be modelled. The boundary that lies in the symmetry plane is of type *symmetry plane*.

empty OpenFOAM creates always three-dimensional meshes. If a two-dimensional simulation needs to be conducted, then the mesh must be one cell in thickness. The boundaries that are parallel to the considered plane must be of the type *empty* to cause the simulation to be two-dimensional.

wedge If a geometry is axisymmetric, then the problem can be simplified. In this case, only a part of the geometry – a wedge – is modelled. The additional boundaries are of type *wedge*.

cyclic Cyclic boundary.

processor A boundary between sub-domains created during the domain decomposition is of type *processor*.

36.1.2 Complex boundaries

Some kinds of boundary patches are more than just a geometric boundary of the domain. E.g. on a wall, the no-slip condition usually applies, therefore there is need for further modelling.

patch This is the generic type for all boundaries. A boundary is of this type, if none of the following types applies.

wall This is a special type for walls. This type is mandatory for using wall models when modelling turbulence.

The boundaries of the types *patch* and *wall* need to be specified further. These boundaries can have boundary conditions of the *primitive* or *derived* types.

36.2 Primitive types

The most important *primitive type* boundary conditions are:

fixedValue The value of a quantity is prescribed directly.

fixedGradient The gradient of a quantity is prescribed directly.

zeroGradient The gradient of a quantity is prescribed to zero.

type	fixedValue;
value	uniform (0 0 0);

Listing 236: **fixedValue** boundary condition

36.3 Derived types

The boundary condition of the *derived types* are derived from the boundary conditions of the *primitive types*. The boundary conditions of this type can be used to model more complex situations.

36.3.1 inletOutlet

The behaviour of the *inletOutlet* boundary condition depends of the flow direction. If the flow is directed outwards, then a *zeroGradient* boundary condition is applied. If the flow is inwards, then a fixed value is prescribed. The value of the inflowing quantity is provided by the **inletvalue** keyword. The **value** keyword has to be present, but it is not relevant.

type	inletOutlet;
inletValue	uniform (0 0 0);
value	uniform (0 0 0);

Listing 237: **inletOutlet** boundary condition

36.3.2 surfaceNormalFixedValue

The *surfaceNormalFixedValue* boundary condition prescribes the norm of a vector field. The direction is taken from the surface normal vector of the patch. A positive value for **refValue** means, that this quantity is directed in the same direction as the surface normal vector. A negative value means the opposite direction.

type	surfaceNormalFixedValue;
refValue	uniform -0.1;

Listing 238: **surfaceNormalFixedValue** boundary condition

36.3.3 pressureInletOutletVelocity

This boundary condition is a combination of *pressureInletVelocity* and *inletOutlet*.

36.4 Pitfalls

36.4.1 Syntax

When assigning a **fixedValue** boundary condition, OpenFOAM expects the keyword **uniform** or **nonuniform** after the **value** keyword.

Listing 239 shows the file 0/k. There the inlet boundary definition differs from Listing 236. Note the missing **uniform** keyword. The reaction of OpenFOAM differs from the value after the keyword **version**.

Listing 240 shows the warning message OpenFOAM issues, when the value after the keyword **version** is 2.0 like in Listing 239. In this case, OpenFOAM assumes **uniform**.

If the value after the keyword **version** is 2.1, then OpenFOAM will issue an error message like in Listing 241.

In both cases OpenFOAM-2.1.x was used. The author assumes the reason for this distinction between version 2.0 and 2.1 lies in an extension of the possible boundary conditions See the release notes of OpenFOAM-2.1.0 (<http://www.openfoam.org/version2.1.0/boundary-conditions.php>).

```

FoamFile
{
    version      2.0;
    format       ascii;
    class        volScalarField;
    object       k;
}

// * * * * *

dimensions      [0 2 -2 0 0 0 0];

internalField    uniform 1e-8;

boundaryField
{
    inlet
    {
        type      fixedValue;
        value      1e-8;
    }
}

```

Listing 239: The file 0/k

```

--> FOAM Warning :
    From function Field<Type>::Field(const word& keyword, const dictionary&, const label)
    in file /home/user/OpenFOAM/OpenFOAM-2.1.x/src/OpenFOAM/lnInclude/Field.C at line 262
    Reading "/home/user/OpenFOAM/user-2.1.x/run/twoPhaseEulerFoam/bubblePlume/case/0/k::
    boundaryField::inlet" from line 25 to line 26
    expected keyword 'uniform' or 'nonuniform', assuming deprecated Field format from Foam
    version 2.0.

```

Listing 240: Warning message: missing keywords

```

--> FOAM FATAL IO ERROR:
    expected keyword 'uniform' or 'nonuniform', found on line 26 the doubleScalar 1e-08

file: /home/user/OpenFOAM/user-2.1.x/run/twoPhaseEulerFoam/bubblePlume/case/0/k::boundaryField
::inlet from line 25 to line 26.

    From function Field<Type>::Field(const word& keyword, const dictionary&, const label)
    in file /home/user/OpenFOAM/OpenFOAM-2.1.x/src/OpenFOAM/lnInclude/Field.C at line 278.

FOAM exiting

```

Listing 241: Warning message: missing keywords

36.5 Time-variant boundary conditions

Time-variant boundary conditions can help to avoid problems from an inept initialisation of the solution data. The most easy initialisation is to prescribe all values to be zero throughout the domain, see Listing 161 in Section 29.

At the start of a simulation when the non-zero values of some boundary meet the zero values of the neighbouring cells stability problems may arise due to the large relative velocities. One solution would be to choose a very small time step at the beginning. Another solution would be to prescribe a time-variant boundary condition. Thus, the field-values at the boundary are initially small and grow during a certain time span to their final value.

36.5.1 uniformFixedValue

This boundary condition is an generalisation of the `fixedValue` BC. See <http://www.openfoam.org/version2.1.0/boundary-conditions.php>.

Listing 242 shows the definition of a time-variant boundary condition with a fixed value. Between the time $t = 0.0\text{s}$ and $t = 5.0\text{s}$ the value of the boundary condition is linearly interpolated between the values for both ends of the interval. After this interval has ended, the value of the boundary condition remains constant.

```
inlet
{
    type                uniformFixedValue;
    uniformValue        table
    (
        ( 0.0      (0.0 0.0 0.0) )
        ( 5.0      (0.0 0.0 0.1) )
    );
}
```

Listing 242: Definition of a time-variant boundary condition

Pitfall: old two-phase solvers

This boundary condition does not work with two-phase solvers of old OpenFOAM versions. With OpenFOAM-4 using time-variant boundary conditions poses no problem anymore.

37 Mesh interfaces: AMI and ACMI

In a perfect world, the gods of CFD look favourably upon us mere earthly creatures. However, as the world is far from perfect, the gods of CFD, in their infinite wisdom, blessed us with some of their beloved nasties: non-conformal meshes, moving meshes and various other things.

However, not all hope is lost. OpenFOAM offers AMI and ACMI to address the issues of unconnected meshes and moving meshes.

37.1 AMI and ACMI in brevity

AMI

The arbitrary mesh interface (AMI) can be used for connecting two unconnected meshes, when the connecting patches fully overlap, e.g. a rotating, cylindrical mesh within a surrounding stationary mesh. AMI was introduced in OpenFOAM-2.1¹⁰⁵.

AMI can be used not only to solve rotating mixer within a stationary domain, as e.g. in the various *mixerVessel* tutorials of solvers capable of employing a dynamic mesh. We can also use AMI to couple two unconnected, stationary meshes. Thus, we may have an easier time at mesh creation since, we can mesh mesh sub-domains individually and couple them in the simulation using AMI.

ACMI

With OpenFOAM-2.3¹⁰⁶ the AMI method was extended and the arbitrary coupled mesh interface (ACMI) was introduced. The ACMI allows for having partially overlapping patches.

37.2 Arbitrary Mesh Interface - AMI

37.2.1 Use case - varying body orientation

If we wanted to study the drag on a triangular body at various orientations, we could either create a mesh for each individual orientation of the body in question, or we could harness the power of AMI.

In Figure 91 we see a simulation domain consisting of two unconnected regions. The outer domain, in grey, has a cylindrical void. The inner domain contains the body under investigation, and its outer boundary is such that the inner domain fills the void of the outer domain. The inner boundary of the outer domain as well as the outer boundary of the inner domain are both of the type *cyclicAMI*.

¹⁰⁵ See <https://openfoam.org/release/2-1-0/ami/>

¹⁰⁶ See <https://openfoam.org/release/2-3-0/non-conforming-ami/>

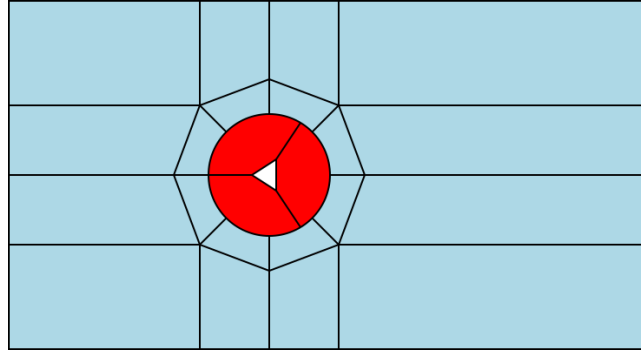


Figure 91: Our simulation domain. The block structure of outer domain, in blue, does not match the block structure of the inner domain, in red. However, by keeping the two regions separate, we can mesh each region individually, which is fairly easy, and use AMI to essentially connect the two regions. After meshing, we can rotate the inner domain with respect to the outer domain to change the orientation of the triangle in the center of the inner domain.

Thus, AMI will deal with the information exchange between the two unconnected mesh regions. When simulating the flow, the two regions will act as if they were a single one. Furthermore, AMI frees us from the worry to create a matching mesh on the connecting boundary, i.e. the mesh of the inner patch of the outer domain, and the mesh of the outer patch of the inner domain do not need to match. The only thing we need to specify is which patch needs to talk with which other patch, i.e. we need to specify the neighbouring patch of each AMI patch.

When we study the flow for various orientations of the body in question, we simply rotate the inner domain with respect to the outer domain to get a proper mesh for current orientation. Thus, while we created a single mesh, we can study an arbitrary number of orientations without the need to create the same number of meshes. We simply use the utility `moveMesh` to rotate the inner domain to the new orientation, and we are good to go.

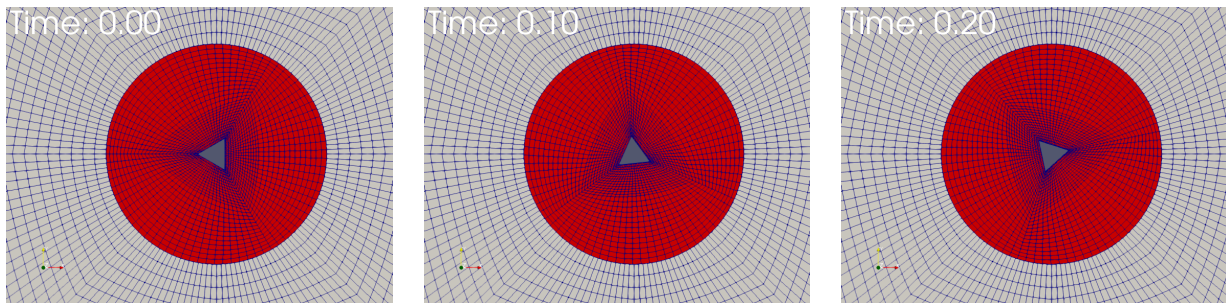


Figure 92: Varying the orientation of the investigated body. Since the body-fitting mesh of the inner domain is in an over-all cylindrical shape, we simply can rotate the inner domain with respect to the outer domain, to change the orientation of the investigated body with respect to the incident flow. After creating the mesh, we can use `moveMesh` to rotate the inner domain. With appropriate settings for the angular velocity and the write interval, we can get a mesh for any orientation of the triangular prism.

This case demonstrates the use of AMI for studies with a purely stationary mesh. If we apply all the steps described above, to each time step of a simulation, instead of an individual simulation of a parametric study, we end up with a sliding mesh simulation case. Thus, AMI enables simulations with a dynamic, sliding mesh.

However, AMI can also be used for simulations with purely stationary meshes. Sometimes it is easier to mesh individual regions of the simulation domain individually, and let AMI handle the connection of this regions. In the case of two stationary unconnected mesh regions¹⁰⁷, we could alternatively use `stitchMesh` to connect the two regions. Using `stitchMesh` will result in a mesh with one single region.

¹⁰⁷Or any number of unconnected mesh regions.

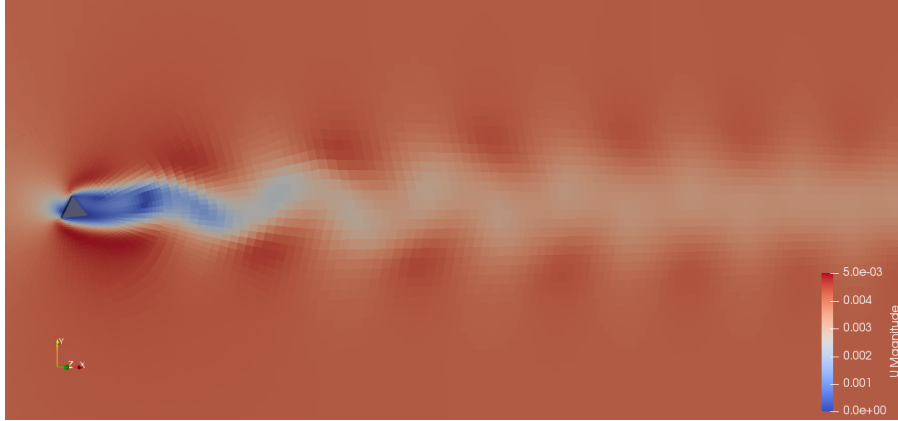


Figure 93: A 2D simulation of the triangular, prismatic body in laminar cross-flow using AMI to connect the body fitted mesh of the inner region with the mesh of the outer domain.

37.3 Arbitrary Coupled Mesh Interface - ACMI

The arbitrary coupled mesh interface works in a similar way as the AMI with the exception that the involved patches do not need to overlap completely.

37.3.1 Use case - varying body orientation

To demonstrate the ACMI, we use again the example from above. Only this time, our inner domain does not fill the outer domain completely, see Figure 94. In this example, the remaining void represents the body under investigation, i.e. a prismatic body with a semi-circular base. Again, by rotating the inner domain with respect to the outer domain, we change the orientation of the body under investigation.

The inner boundary of the outer domain is a patch with the form of a cylinder shell. The outer boundary of the inner domain, however, has the shape of the shell of a semi-cylinder. Thus, the two patches forming the interface of the outer and inner domain overlap only partially.

The ACMI needs to distinguish whether a face has a neighbour, i.e. the face is part of the overlapping section of the patch; or the face has no neighbour, i.e. the face acts as a wall. Thus, we need a pair of patch definitions for each domain: *couple* and *blockage*. In the case of *couple*, we need to specify the neighbouring patch, as we needed in the AMI case. In the case of *blockage*, we need to specify the behaviour, in most cases: being a wall.

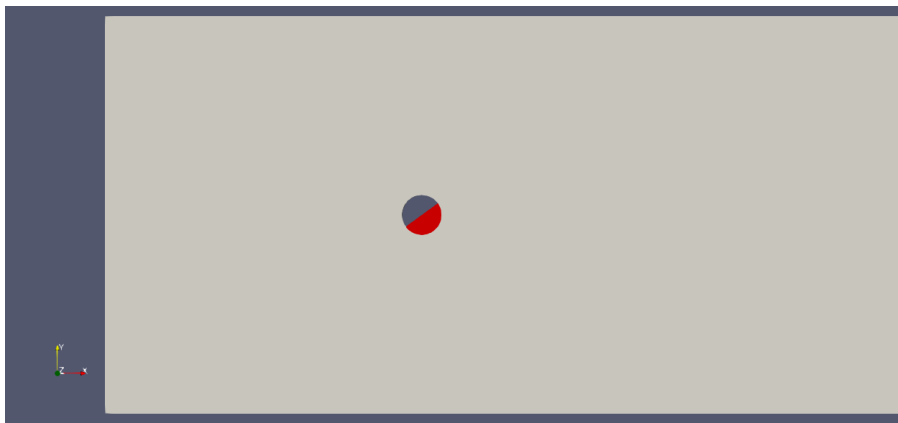


Figure 94: A simulation domain with two unconnected regions. Note that the inner region only partially fills the void of the outer region.

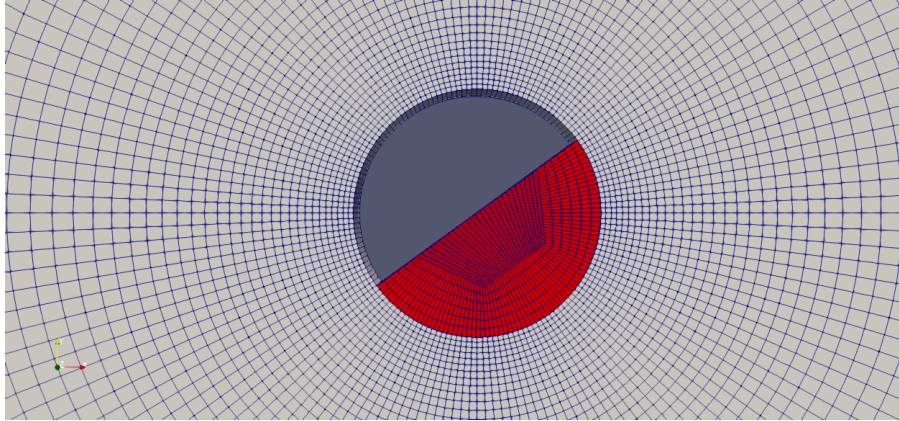


Figure 95: The inner domain in more detail. The block-structure of the inner domain is clearly recognizable. The remaining void is the body under investigation, in this case a semi-cylinder.

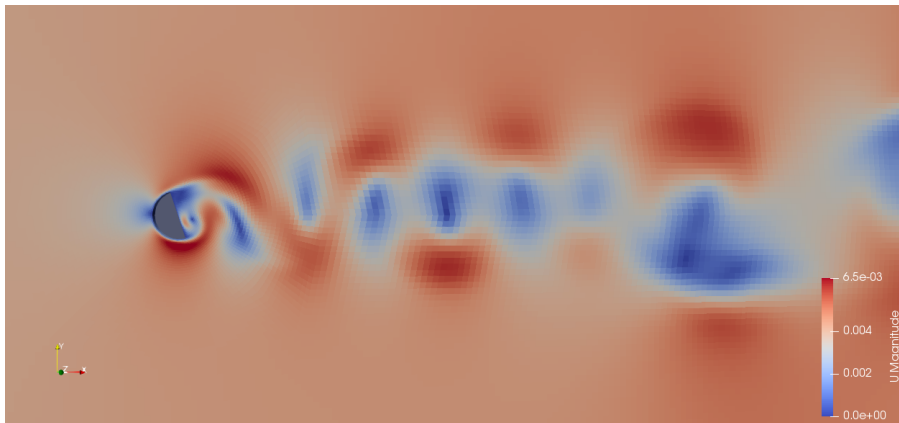


Figure 96: A 2D simulation of the semi-cylinder in laminar cross-flow using ACMI to connect the mesh of the inner region with the mesh of the outer domain.

37.4 Avoiding errors

37.4.1 AMI and MRF

When using AMI in conjunction with MRF, add the AMI patches to the list of non-rotating patches in the file `MRFProperties`.

38 The MRF method

The MRF method allows for the simulation of rotating machinery without an actually rotating mesh. By using multiple reference frames, we can simulate the problem with a static mesh. Although, this simplification introduces certain modelling errors, the reduced complexity and faster computation times compared to a simulation with a moving mesh, as well as the sufficient precision on a global scale, justifies this method under the right circumstances. See Section 65 for the theoretical background of the MRF method.

38.1 Usage

The MRF method is applied to cell zones. This is clearly indicated by the `cellZone` keyword in the `MRFProperties` dictionary, see Listing 243. Apart from where to apply the MRF method, we can enable/disable the application of the MRF method using the `active` keyword.

Another important input is the list of non-rotating patches, aptly named `nonRotatingPatches`, as the solid body rotation according to the rotation of the reference frame is applied to all wall patches. However, there

might be the case of patches within the MRF cell zone, that are actually stationary, hence the option to exclude individual patches from the application of the MRF method.

Finally, in Listing 243, the nature of the rotation of the reference frame is specified. This is done by providing the axis of rotation (**axis**) and a point in space (**origin**). The point origin must be located somewhere on the axis of rotation. The keyword **omega** is used to specify the rotational velocity.

```

zone1
{
    cellZone    rotor;
    active      yes;

    // Fixed patches (by default they 'move' with the MRF zone)
    nonRotatingPatches ();

    origin      (0 0 0);
    axis        (0 0 1);

    omega       table
                2(
                    (0      0.0)
                    (0.75  20.0)
                );
}

```

Listing 243: Specifying the necessary inputs for the MRF method in the **MRFProperties** dictionary.

Figure 97 shows the cell zone for the mesh of a stirred tank. The cell zone, to which the MRF method is applied, is shown as white wireframe. Note, that this zone is of cylindrical shape and is aligned with the axis of rotation.

If we extended the cylinder from the very bottom to the very top, then the bottom and top patches of the stator would need to be entered into the list of non-rotating patches.

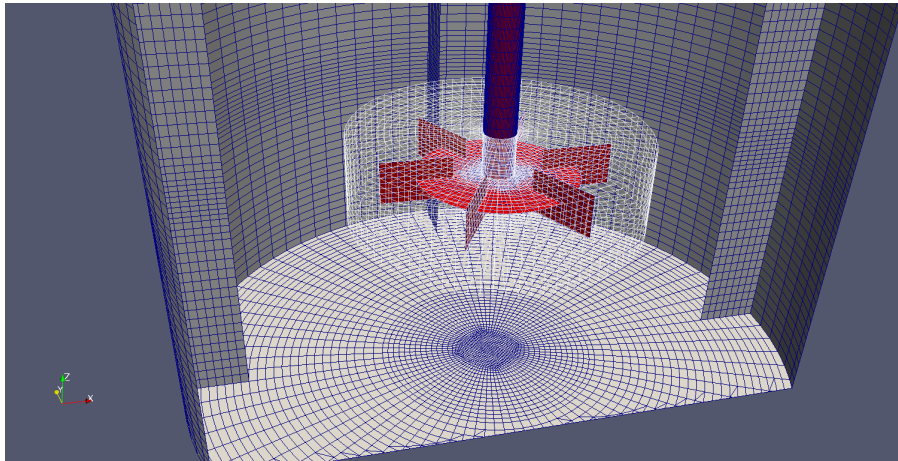


Figure 97: A baffled stirred tank with a Rushton impeller. The stator patch is shown in grey and the rotor patch is shown in red. The white wireframe shows the boundary of the rotor zone. For all cells of the rotor zone the MRF method is applied.

38.2 Avoiding errors

38.2.1 Non-rotating patches

AMI and MRF

It has been found, that AMI-type patches need to be added to the list of non-rotating patches, see Section 37.4.1.

Cyclic patches and MRF

It has been observed, that when using `cyclic`-type patches, e.g. when simulating half a stirred tank as in Figure 98, we need to add the `cyclic`-type patches to the list of non-rotating patches in the file `MRFPProperties`.

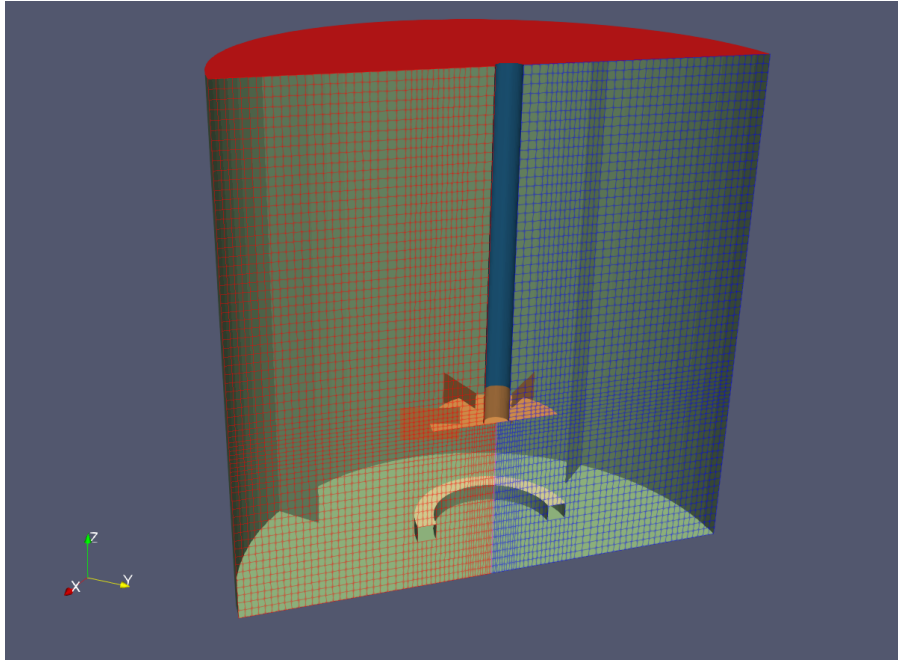


Figure 98: Half a baffled stirred tank with a Rushton impeller. The `cyclic`-type patches are shown as coloured wireframes, and all other patches are shown as coloured surfaces.

39 The fvOption framework

The `fvOption` framework handles sources and constraints of our numerical flow model. The `fvOption` framework allows us to plug various constraints or sources into an existing solver without any solver modification. Besides general sources and constraints, there is a number of specialised sources representing a specific physical models, e.g. the effect of a porous zone on the momentum equation.

The `fvOptions` framework was introduced with the release of OpenFOAM-2.2.0¹⁰⁸ in 2013 and has expanded since.

Motivation

The use of the `fvOption` framework is best explained on an example. Equation (57) shows a convective transport equation for a general scalar C , e.g. a passive tracer concentration. On the RHS, we see the general, linearized source term. If, in our example, the tracer enters the simulation domain through the inlet, then all is well and an inlet BC for the tracer concentration C suffices. If, however, the tracer is introduced via a probe, which we do not want to resolve with our mesh¹⁰⁹, then we need a mechanism to introduce the tracer C within our simulation domain. This is where `fvOptions` come to the rescue. These offer us to specify, at the location of the tip of the probe, an injection rate or a fixed value for the field C .

$$\frac{\partial C}{\partial t} + \nabla \cdot (C\mathbf{u}) = S_u + S_p C \quad (57)$$

The `fvOption` framework offers a fixed-value constraint and a semi-implicit source, with which we can model our tracer generating probe.

¹⁰⁸<https://openfoam.org/release/2-2-0/fv-options/>

¹⁰⁹E.g. the smoke probe used in wind tunnels for the study of external aerodynamics.

Application

Below, in Listing 244, we see the energy equation of a compressible flow solver of OpenFOAM. In this Listing, all calls to the fvOptions framework are marked in red, green and blue. There, we see that the fvOptions framework can act on the governing equation of a certain field itself, as well as act on the computed field after the governing equation has been solved.

```
1 {
2     volScalarField& he = thermo.he();
3     fvScalarMatrix EEqn
4     (
5         fvm::ddt(rho, he) + fvm::div(phi, he)
6         + fvc::ddt(rho, K) + fvc::div(phi, K)
7         + (
8             he.name() == "e"
9             ? fvc::div
10             (
11                 fvc::absolute(phi/fvc::interpolate(rho), U),
12                 p,
13                 "div(phi,p)"
14             )
15             : -dpdt
16         )
17         - fvm::laplacian(turbulence->alphaEff(), he)
18         ==
19         fvOptions(rho, he)
20     );
21
22     EEqn.relax();
23     fvOptions.constrain(EEqn);
24     EEqn.solve();
25     fvOptions.correct(he);
26     thermo.correct();
27 }
```

Listing 244: The energy equation in the file `EEqn.H` of the compressible, single-phase solver *rhoPimpleFoam*

A solver, that uses the fvOptions framework creates an object of the type `fvOptionList`, which is a list of fvOptions. This is also the reason why, for all calls to the fvOptions framework, parameters have to be passed. Otherwise, the framework would not be able to assign the options to their respective equations. In the case of a porous zone in the domain of a single-phase simulation, we might use the fvOptions framework to account for the additional flow resistance in the momentum equation and also to apply certain source terms for the turbulence model equations. Thus, the list of options has to be traversed to find and apply the proper fvOption to the respective model equation.

In the Listing 245 below, we see the method `fvOptionList::constrain()` as an example of how OpenFOAM applies the fvOptions. We traverse the list of all options, and for each options, we check whether this option applies to the provided model equation. If the option applies, it is determined whether the option is enabled, i.e. whether it is active, and if this is the case, the option is applied by calling the `constrain()` method of the `fvOption` class, i.e. calling the `constrain()` method of the source itself.

```
1 template<class Type> void Foam::fv::optionList::constrain(fvMatrix<Type>& eqn)
2 {
3     checkApplied();
4     forAll(*this, i)
5     {
6         option& source = this->operator[](i);
7         label fieldi = source.applyToField(eqn.psi().name());
8         if (fieldi != -1)
9         {
10             source.setApplied(fieldi);
11             if (source.isActive())
12             {
13                 if (debug)
14                 {
15                     Info<< "Applying constraint " << source.name()
16                     << " to field " << eqn.psi().name() << endl;
17                 }
18             }
19         }
20     }
21 }
```



```

18         source.constrain(eqn, fieldi);
19     }
20 }
21 }
22 }

```

Listing 245: The method `constrain()` of the class `fvOptionList`

The mode of operation shown above is the same for all calls to the `fvOptions` framework, we see in Listing 244. Each call causes the list of options to be traversed, and all applicable options to be applied.

39.1 Controlling space & time

A number of `fvOptions` are derived from the base class `cellSetOption`, which implements control over when and where the `fvOption` is to be active. This is used for porous zones, which make up only part of the simulation domain, or for tracer injection, which might be of limited duration.

The active time of the `fvOptions` derived from `cellSetOption` is controlled by the keywords `timeStart` and `duration`. The region in which the option is to be acting can be selected by providing the name of a `cellSet` or a `cellZone`, by specifying points in space, or by providing cell labels.

This, however, does not apply to all `fvOptions`. For some of them, a restriction to a limited time span or a certain region would make no sense at all, e.g. for considering buoyant forces on the momentum equation.

39.2 Types of options

The `fvOptions` framework implements a number of different options, which fall into one of the following categories:

39.2.1 Constraint

A constraint acts on an already computed field. We can infer this from the point in the code when the method `fvOptions.constrain()` is called, the relevant line is high-lighted in green in Listing 244.

With a constraint, fixed values at certain locations can be enforced in an otherwise freely computed field, e.g. if we are solving the passive transport of a scalar acting as a tracer or marker, we can define certain cells, in which the value of the scalar has a fixed value. These locations would be points in space, where the tracer is injected into our simulation domain.

Available constraints, at the time of writing, are:

- `fixedValueConstraint`
- `fixedTemperatureConstraint`

39.2.2 Correction

A correction is a type of option, that acts on a field after it has been computed. We can infer this from the point in the code when the method `fvOptions.correct()` is called, the relevant line is high-lighted in blue in Listing 244.

Available corrections, at the time of writing, are:

- `limitTemperature`
- `limitVelocity`

39.2.3 Source

A source adds source and sink terms to the governing equation of a certain field. Thus, a source influences a field via the solution process of its governing equation. We can infer this from the point in the code when `fvOptions()` is called, the relevant line is high-lighted in red in Listing 244. In contrast to the constraints and the corrections, the sources call the `()`-operator class `fvOptionList`.

There is a large number of sources available in OpenFOAM, ranging from very general ones, such as `codedSource` or `semiImplicitSource` to specific sources for certain specific purposes, e.g. `actuationDiscForce`, `buoyancyForce` and many others.

39.3 Sources

This sub-section contains an incomplete list of sources from the `fvOptions` framework.

39.3.1 Porosity models

The `fvOptions` framework can be used to model the influence of porous zones on the flow, e.g. the catalytic converter in an exhaust system. The presence of a porous medium acts as a sink in the fluid's momentum equation. The `explicitPorositySource` `fvOption` can be used to model the effect of porous zones in a flow simulation.

Listing 246 shows the momentum equation of *pimpleFoam*. Here, the contributions from the `fvOptions` framework are on the RHS. A porosity model will introduce a negative term on the RHS, since porous zones require additional force to drive the fluid through them.

```
1      fvm::ddt(U) + fvm::div(phi, U)
2      + MRF.DDt(U)
3      + turbulence->divDevReff(U)
4      ==
5      fvOptions(U)
```

Listing 246: The momentum equation in the file *UEqn.H* of *pimpleFoam*

`fixedCoeff`

The `fixedCoeff` model calculates a momentum contribution $\mathbf{S}$ which is proportional to the velocity and the squared velocity. The model features two model constants: α and β .

$$\mathbf{S} = -\rho_{\text{Ref}}(\alpha + \beta|\mathbf{u}|)\mathbf{u} \quad (58)$$

or a little rearranged

$$\mathbf{S} = -\rho_{\text{Ref}}(\alpha|\mathbf{u}| + \beta|\mathbf{u}|^2)\mathbf{u}$$

In OpenFOAM's implementation α and β are vectorial quantities. In addition with a reference coordinate system for the porous zone, anisotropic porosity can be considered. Isotropic porosity is then a special case covered by choosing all components of α and β to be equal.

`powerLaw`

The `powerLaw` model computes a momentum contribution $\mathbf{S}$, which is proportional to a model constant C_0 and the C_1 -th power of the velocity. The `powerLaw` model does not support anisotropy.

$$\mathbf{S} = -\rho C_0|\mathbf{u}|^{(C_1-1)}\mathbf{u} \quad (59)$$

or a little rearranged

$$\mathbf{S} = -\rho C_0|\mathbf{u}|^{C_1}\mathbf{e}_u$$

`DarcyForchheimer`

The `DarcyForchheimer` model is very similar to the `fixedCoeff` model. It also has two contributions proportional to the linear and the squared velocity. This model is a combination of the Darcy model $U = -\frac{\kappa}{\mu}\frac{dp}{dx}$, which is valid for laminar flow, and its extension to higher Reynolds numbers, known as Forchheimer model $-\frac{dp}{dx} = \frac{\mu}{\kappa}U + \frac{\rho}{k_2}U^2$. The Darcy model is proportional to κ , the permeability of the porous medium, $[\kappa] = \text{m}^2$. The Forchheimer model introduces k_2 , the inertial permeability, $[k_2] = \text{m}$.

There are two model constants of OpenFOAM's implementation of the `DarcyForchheimer` model. The Darcy coefficient `d` is the inversed permeability $\mathbf{d} = \frac{1}{\kappa}$, and the Forchheimer coefficient `f` is the inverse inertial

permeability $\mathbf{f} = \frac{1}{k_2}$.

$$\mathbf{S} = -(\mu d + 1/2 \rho |\mathbf{u}| f) \mathbf{u} \quad (60)$$

or a little rearranged

$$\mathbf{S} = -\rho (\nu d |\mathbf{u}| + 1/2 f |\mathbf{u}|^2) \mathbf{e}_u$$

In OpenFOAM's implementation $\mathbf{d}$ and $\mathbf{f}$ are vectorial quantities, as are the coefficients of the `fixedCoeff` model. In addition with a reference coordinate system for the porous zone, anisotropic porosity can be considered. Isotropic porosity is then a special case covered by choosing all components of $\mathbf{d}$ and $\mathbf{f}$ to be equal.

39.3.2 phaseLimitStabilization

With OpenFOAM-6¹¹⁰ an `fvOption` source was introduced to specifically, numerically stabilize transport equations when the phase fraction falls below a certain threshold. This source adds an implicit source term to a transport equation for all cells in which the phase fraction falls below a certain limit.

Solving transport equations, which are formulated using a phase fraction field, becomes numerically more and more difficult when the phase fraction field tends to zero in some cells. Adding an implicit source term in those cells ensures the discretized equation system does not become ill-posed. When a transport equation becomes ill-posed due to a phase fraction tending towards zero, the solver of the linear equation system fails with a floating-point error.

Listing 247 shows the code of the relevant method of this source class. The `max()` statement ensures that the source is only applied to cells in which the phase fraction `alpha` is smaller than the limit `residualAlpha`.

```

1  template<class Type> void Foam::fv::PhaseLimitStabilization<Type>::addSup
2  (
3      const volScalarField& alpha,
4      const volScalarField& rho,
5      fvMatrix<Type>& eqn,
6      const label fieldI
7  )
8  {
9      const GeometricField<Type, fvPatchField, volMesh>& psi = eqn.psi();
10     uniformDimensionedScalarField& rate =
11         mesh_.lookupObjectRef<uniformDimensionedScalarField>(rateName_);
12
13     eqn -= fvm::Sp(max(residualAlpha_ - alpha, scalar(0))*rho*rate, psi);
14 }

```

Listing 247: The method `addSup()` of the class `PhaseLimitStabilization`

We see from the code in Listing 247, that a `uniformDimensionedScalarField rate` is used to compute the implicit source. This `uniformDimensionedScalarField rate` can be created by a `codedFunctionObject` and then be registered with the mesh. Only when the `uniformDimensionedScalarField rate` is registered with the mesh, can the `PhaseLimitStabilization` source access it via the `lookup()` method of the mesh. Listing 248 shows an example of how this can be done. The relevant line of code to register the computed rate field is high-lighted in red. If this line is omitted, then the `fvOption` source will abort with an error caused by trying to lookup a non-existent field. See Section 57.7 for in-depth information on the object registry.

```

functions
{
    rate
    {
        libs          ("libutilityFunctionObjects.so");
        type coded;
        name rate;
        codeRead
        #{
            static autoPtr<uniformDimensionedScalarField> pField;
            if (!pField.valid())

```

¹¹⁰<https://openfoam.org/release/6/>

```

{
    pField.set
    (
        new uniformDimensionedScalarField
        (
            IOobject
            (
                "rate.water",
                mesh().time().timeName(),
                mesh(),
                IOobject::NO_READ,
                IOobject::NO_WRITE
            ),
            dimensionedScalar("rateInit",dimensionSet(0, 0, -1, 0, 0, 0, 0), 1.0)
        )
    );
}
uniformDimensionedScalarField& rate = pField();
rate.checkIn();
#};
}
}

```

Listing 248: Computation of a rate field using a `codedFunctionObject` in the file `controlDict`.

40 The Lagrangian world

In OpenFOAM not only the *finite volume method* (FVM), which is part of the Eulerian world, is implemented. There are also Lagrangian methods available. The Lagrangian methods available in OpenFOAM cover fields such as:

- molecular dynamics
- discrete particle method
- sprays
- general Lagrangian particle tracking
- reacting and combusting particles

This section covers general Lagrangian particle tracking. The basics behind the Lagrangian methods apply to all models listed above, e.g. the molecule and the spray parcel are based on the `particle` class.

40.1 Background

40.1.1 Interaction between Lagrangian particles and Eulerian flow

The coupling between Lagrangian particles and the surrounding (Eulerian) flow can be characterised by their degree of interaction.

<i>one-way</i>	<i>two-way</i>	<i>four-way</i>
flow acts on particles	flow acts on particles particles act on the flow	flow acts on particles particles act on the flow particle-particle collisions
e.g. snow drift	e.g. dense particulate flows	e.g. fluidized beds

Table 5: Levels of coupling between Lagrangian particles and (Eulerian) flow

Controlling the level of interaction

OpenFOAM's Lagrangian model library is able to accommodate for one-way, two-way and four-way interaction.

40.1.2 Particle tracking

For particle tracking there are two general approaches, the *lose-find* method and the *face-to-face* method [47, 40]. Knowing the cell in which a particle is located is important when interaction with the flow fields is to be considered.

The *lose-find* method tracks the particle along its path according to its velocity. The information on the cell in which the particle is located, however, is lost in this process. Hence, this method is referred to as *lose-find*. Whenever, the current cell in which the particle is located is needed, the neighbouring cells need to be searched until the particle is found. This approach can pose some problems [47].

The *face-to-face* method, which is implemented by OpenFOAM, tracks the particles to the cell faces, updates the cell information and tracks the particle further on [40]. Thus, only once at the start of the simulation the cells at the particles' locations need to be searched. During the simulation the cell index to which a particle belongs is continuously updated whenever the particle crosses a cell face.

Barycentric tracking

With the release of OpenFOAM-5.0, the LPT algorithm was changed to barycentric tracking¹¹¹, see also the relevant commit message¹¹². Barycentric tracking has been implemented to increase the robustness of the tracking algorithm.

40.2 Libraries

OpenFOAM offers two choices for implementing or using Lagrangian particle tracking (LPT). A discussion on these can be found in [42].

particle

The class `particle` is the root of all LPT in OpenFOAM, since it implements the tracking (i.e. the motion) of the particles itself.

40.2.1 basic solidParticle

The basic choice for LPT is the class `solidParticle`, which is derived from `particle`. The class `solidParticle` adds little to its ancestor class. The two additional data members are the particle's diameter and velocity. The two most important methods of `solidParticle` are `move()` and `hitWallPatch()`. With these two methods the particle's drag (via modifying the particle's velocity in `move()`) and the wall interaction (i.e. wall collision, via modifying the particle's velocity in `hitWallPatch()`) can be implemented. This is sufficient for one-way and two-way coupled simulations.

40.2.2 intermediate parcels

The advanced implementation of LPT in OpenFOAM is the `intermediate` library¹¹³ in `$FOAM_SRC/lagrangian`. This library contains some heavily templated classes which provide a general framework to implement a range of additional models for LPT, e.g. collision modelling, heat transfer or reactions. The `intermediate` library was first published with OpenFOAM-1.5beta¹¹⁴.

The basis for LPT itself is again the class `particle`, although hidden under layers of templates, Listings 249 and 250 show a prime example of OpenFOAM's template insanity.

```
1 namespace Foam
2 {
3     typedef ReactingMultiphaseParcel
4     <
5         ReactingParcel
6         <
7             ThermoParcel
```

¹¹¹<https://cfd.direct/openfoam/free-software/barycentric-tracking/>

¹¹²<https://github.com/OpenFOAM/OpenFOAM-dev/commit/371762757d3e3cd38a3841a547a9bc8c1aff0b85>

¹¹³`$FOAM_SRC/lagrangian/intermediate` is actually a library, since it is a separate compilation unit and is compiled into `$(FOAM_LIBBIN)/liblagrangianIntermediate`.

¹¹⁴<http://www.openfoam.org/download/version1.5beta.php>

```

8         <
9             KinematicParcel
10        <
11            particle
12        >
13    >
14    >
15    > basicReactingMultiphaseParcel;
16
17    /* the rest of the code ... */

```

Listing 249: The class definition of the `ReactingMultiphaseParcel` class, in `basicReactingMultiphaseParcel.H`

The class `KinematicParcel` is an example for the hardships one faces when trying to understand C++. `KinematicParcel` is a templated class, with `ParcelType` as template parameter. In addition `KinematicParcel` also is derived from its template parameter `ParcelType`.

Thus, `KinematicParcel` is a templated class built around `ParcelType`, however, it *is a* `ParcelType` too (by inheritance).

```

1  template<class ParcelType>
2  class KinematicParcel
3  :
4      public ParcelType
5  {
6  public:
7
8      /* the rest of the code ... */

```

Listing 250: The class definition of the `KinematicParcel` class, in `KinematicParcel.H`

To underpin the claim made, that `particle` is the very root of LPT, we have a look at the most basic parcel-based class of the `intermediate` library of OpenFOAM. Listing 251 shows the definition of the class `basicKinematicParcel`, which is the class `particle` passed to the templated class `KinematicParcel` as a template parameter. From one of the above paragraphs, we know that this means also that `basicKinematicParcel` is derived from `particle`, hence it *is a* `particle`.

```

1  namespace Foam
2  {
3      typedef KinematicParcel<particle> basicKinematicParcel;
4
5      template<>
6      inline bool contiguous<basicKinematicParcel>()
7      {
8          return true;
9      }
10 }

```

Listing 251: The class definition of the `basicKinematicParcel` class, in `basicKinematicParcel.H`

40.3 Cloudy, with a chance of particles

In OpenFOAM and its class layout there is the distinction between the single particle and the entirety of all particles. The particle class defines the features and the behaviour of the single particle. The Lagrangian solver, however, needs to deal with all particles. Not all particles are equal, but the solver should not have to deal with this. In order to provide a common interface for the solver, OpenFOAM's creators thought of the `cloud` class.

The `cloud`¹¹⁵ class acts as a connection between the solver and the individual particles. It makes sure that commands are passed on to all particles within the cloud.

¹¹⁵Not to be mixed up with the “cloud” in terms of information technology (IT) as in cloud storage, cloud computing, etc..

40.3.1 The code to rule them all

This section is one of the many examples of OpenFOAM's sources being case-sensitive. The class `Cloud` and the class `cloud` are completely different things. Admittedly, `Cloud` is derived from `cloud`, thus every `Cloud` *is* a `cloud`, however, not vice-versa. Always keep in mind: case matters.

The Cloud

A class is best described by taking a look on the code that actually defines it. Listing 252 shows from which classes `Cloud` is derived from. Looking at the inheritance actually tells us what the class `Cloud` is, since an inheritance relation is an “*is a*” relation. If A is derived from B, then A *is a* B.

The listing shows us, that `Cloud` is a `cloud` and a `IDLList`. This poses two new questions, what is a `cloud` and a `IDLList`?

```
1  template<class ParticleType>
2  class Cloud
3  :
4      public cloud,
5      public IDLList<ParticleType>
6  {
7      // code
8  }
```

Listing 252: The class definition of `Cloud` in the file `Cloud.H`; the ancestry.

In anticipation of the following paragraphs we can state, that the inheritance from two base classes is an example of applied division of labour. As we will see, the `cloud` heritage is in charge of input and output (I/O) whereas the `IDLList` legacy deals with the management of the single particles which form the cloud.

The cloud

The class `cloud` is an object registry similar to the mesh class¹¹⁶. `cloud` is derived from the class `objectRegistry`, and so are `fvMesh` and `Time`. This enables us to register fields with the particle cloud. The class `objectRegistry` is in turn derived from `regIOobject` which is in turn derived from `IOobject`. Thus, the ancestry of `cloud` allows us to read and write the particle cloud to disk¹¹⁷. See Sections 57.7 and 57.8 for a more detailed discussion on I/O and the concepts around the class `regIOobject`.

Disk I/O is most often seen in code that creates or reads fields. Via the `cloud` branch of a Lagrangian cloud's ancestry, disk I/O is controlled in a similar fashion. If we were able to sneak the line of code in Listing 253 e.g. into `DPMFoam`, right after the construction of the `kinematicCloud` object, then writing the Lagrangian solution data to disk would be permanently disabled.

```
1  kinematicCloud.writeOpt() = IOobject::NO_WRITE;
```

Listing 253: Disabling writing to disk for `kinematicCloud`

The IDLList

The `IDLList` is an *intrusive doubly-linked list*. The concept of a linked list is taught at programming classes when it comes to objects and data-structures. The traditional linked-list consists of a list class and a node class. The node class contains a pointer to, or the list-element itself. If the node class is implemented in a generic fashion, using templates, then one list implementation is sufficient for all datatypes. Otherwise, the node class would need to be implemented specifically for every datatype that is to be used by the list.

An intrusive linked-list is a very efficient implementation of a linked-list. However, the actual layout differs from the standard layout of a linked list¹¹⁸. In an intrusive list, the list element serves also as the node. Figure 99 compares the schematic layouts of traditional and intrusive linked lists.

¹¹⁶In fact the class `polyMesh` is derived from `objectRegistry`. `fvMesh` is in turn derived from `polyMesh`. The mesh in a solver or an utility application is of the type `fvMesh`. Almost all solvers and utilities include the file `createMesh.H`, which resides in `OpenFOAM/include` of your installation.

¹¹⁷Fields, such as `volScalarField` and others, are also derived from `regIOobject` via `GeometricField` and `DimensionedField`.

¹¹⁸http://www.boost.org/doc/libs/1_43_0/doc/html/intrusive/intrusive_vs_nonintrusive.html

Intrusive linked-lists are generally considered as being much more efficient than traditional linked-lists¹¹⁹. One of the downsides of using intrusive lists is that the implementation of the datatype which is to be used within the list is mangled with the implementation of the list itself. Generally, this (mangling the implementation of unrelated concepts) is considered a bad practice in *object-oriented design* (OOD). However, due to the performance gain, intrusive lists are widely used in fields where performance beats conformity with standards, such as computer games or number crunching.

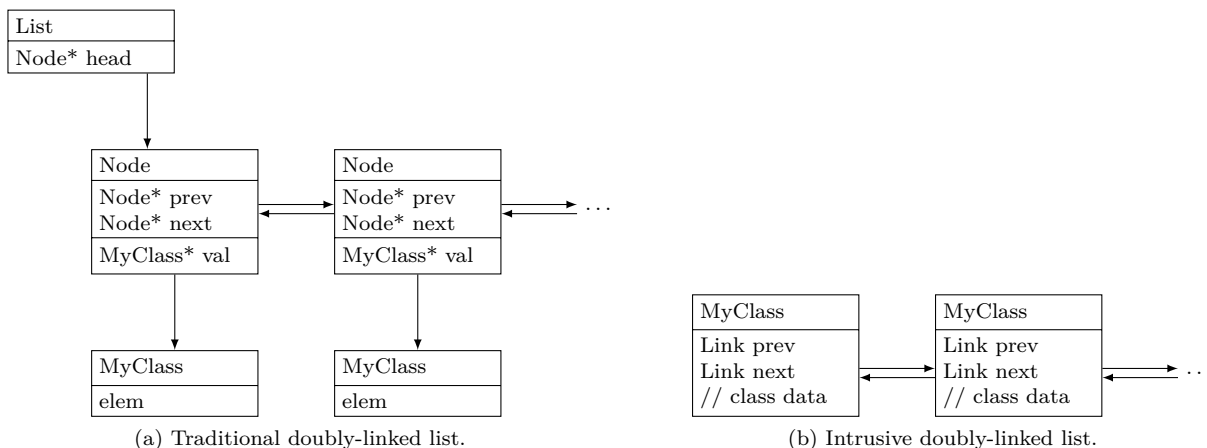


Figure 99: Schematic diagrams of doubly-linked lists.

Again, we can take a look at the actual source code to find out what is really going on. Figure 100 shows the class diagram behind the singly- and doubly-linked intrusive lists. This diagram is in fact a great example of how far C++ developers can go with abstraction and encapsulation. The classes `SLListBase` and `DLListBase` define the behaviour as being single-linked or doubly-linked. The classes `UILList` and `ILList` are more or less helper or base classes. The class `UILList` provides STL-conforming iterators, whereas `ILList` adds some member functions. The reason for `UILList` and `ILList` being separate classes is unknown to the author.

In the case of classic linked lists (non-intrusive lists, either singly- or doubly-linked), the class `LList` derived from its template parameter `LListBase` provides the base class for concrete non-intrusive linked lists.

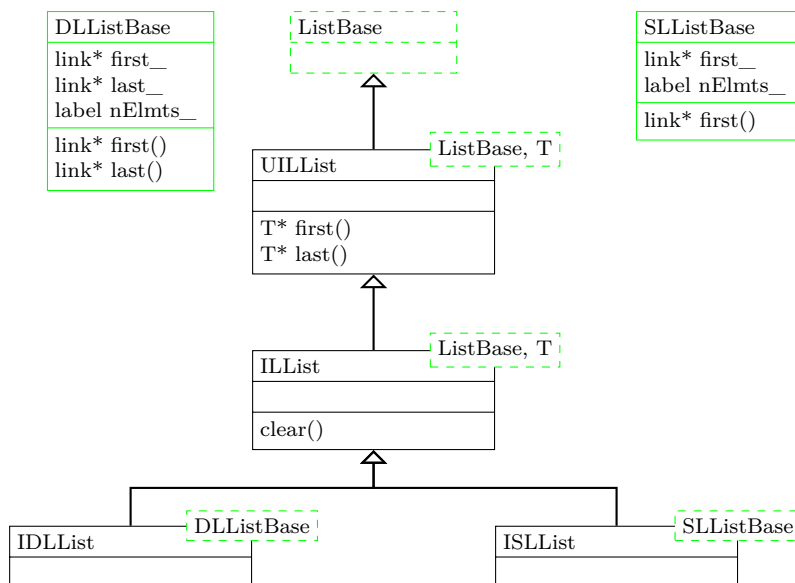


Figure 100: The class hierarchy needed for intrusive lists of objects of type `T`; this diagram can be regarded as a subset of the class diagram for singly- and doubly-linked lists, both classic and intrusive.

¹¹⁹http://www.boost.org/doc/libs/1_58_0/doc/html/intrusive/performance.html

40.4 Cloudy Templates

The Lagrangian models are modularized by making heavy use of inheritance and templating. This compartmentalisation of functionality and data is done for the clouds as well as the particles themselves. This approach makes perfect sense as particles of the type X need a cloud of the type X to make X's properties available to the outside world (i.e. the solver using model X). To be more specific, if we want to solve heat transport with Lagrangian particles, the particles themselves need to provide appropriate data (temperature) and methods (heat transfer), as well as the cloud needs to offer appropriate methods (heat transfer from the particles to the carrier phase and vice versa). Thus, most of the points discussed below hold also for the particles.

The templates for both the clouds and the particles create a framework which allows to create specific clouds and particles simply by combining the templates carrying the intended functionality.

40.4.1 Base classes

kinematicCloud

This virtual abstract class (note the lower case k in kinematic) specifies the behaviour of the templated class **KinematicCloud** (note the capital K in the class' name).

thermoCloud

Also the virtual abstract class **thermoCloud** declares some abstract methods, i.e. methods that a derived class must implement.

40.4.2 Templates

There are two kinds of cloud templates: the ones that are derived from a base class and their template parameter **CloudType**, and the ones that are solely derived from their template parameter. In the following we try to shed some light into the *tempest of templates*.

Base class + template parameter

One example for such a cloud template is the class **KinematicCloud** (note the capital K). This class is derived from the class **kinematicCloud** (minor k) and its template parameter **CloudType**. The base class defines the behaviour of the kinematic part, and the template parameter allows to add additional functionality.

A class passed as a template parameter can provide data and methods as well as a separate base class. However, pure abstract methods can realistically only be provided via the separate base class. The **kinematicCloud** (lower case k) base class provides, among others, the pure virtual method **nParcels()**, which returns the total number of parcels. If this method was provided by the **KinematicCloud** (capital K) template class, then we would not be able to instantiate the **basicKinematicCloud**, as in Listing 256, since we can not create objects of abstract classes.

Another reason for deriving a cloud template from a base class and their template parameter is to introduce the debugging mechanism of the run-time selection framework. Listing 254 shows the relevant lines for the example of the base class **thermoCloud**. This allows the use of the **debug** flag in the class **ThermoCloud**.

```
1 namespace Foam
2 {
3     defineTypeNameAndDebug(thermoCloud, 0);
4 }
```

Listing 254: Introduction of **thermoCloud** into the debugging mechanism in the file **thermoCloud.C**

Template parameter

The classes **CollidingCloud** and **MPPICCloud** are templated clouds, which are derived only from their template parameter. Both classes provide modelling for particle-particle interactions¹²⁰.

¹²⁰See <https://openfoam.org/release/2-3-0/dpm/>

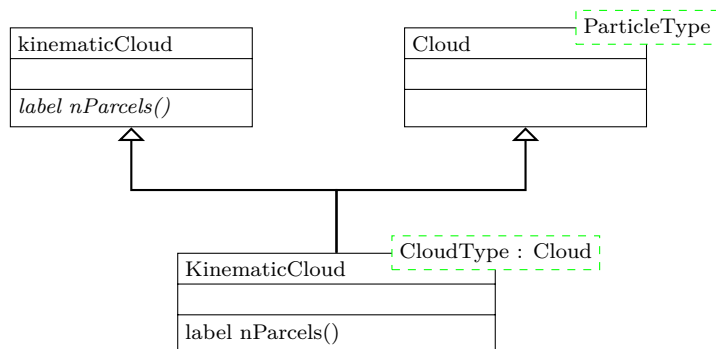


Figure 101: The class hierarchy of the class `basicKinematicCloud`.

40.4.3 Derived clouds

The derived cloud classes are the classes which we can actually use. The most basic derived cloud class is `basicKinematicCloud`, which offers the minimal set of functionality. As we see in Listing 255, the `basicKinematicCloud` is build from the `KinematicCloud` template with a `Cloud` of `basicKinematicParcel`¹²¹ as template parameter. Here the dual templated hell of Lagrangian modelling reveals itself nicely. The template parameter of `Cloud` is a particle type `basicKinematicParcel`. The definition of this particle type is shown in Listing 256. The type `basicKinematicParcel` is also the most basic particle type provided by the Lagrangian `intermediate` library of OpenFOAM.

We note, that the kinematic cloud is a cloud of kinematic particles. The kinematic particle class provides the means to move the particle around and the kinematic cloud type provides the means to move the particles collectively. This is useful since a solver using Lagrangian particles does not operate on the individual particles.

```

1 namespace Foam {
2     typedef KinematicCloud<Cloud<basicKinematicParcel>> basicKinematicCloud;
3 }

```

Listing 255: The class definition of `basicKinematicCloud` in the file `basicKinematicCloud.H`

```

1 namespace Foam {
2     typedef KinematicParcel<particle> basicKinematicParcel;
3 }

```

Listing 256: The class definition of `basicKinematicParcel` in the file `basicKinematicParcel.H`

To substantiate the claim from above, that the templates allow us to build custom Lagrangian models, we take a look at some more derived clouds. In Listing 257 we see the definition of a cloud class, which offers heat transfer modelling. This class throws the `ThermoCloud` template into the mix of the already known `KinematicCloud` class. In combination with Listing 258, we see that the `basicThermoCloud` is a `Cloud` of `basicThermoParcels`. Again, the type of the cloud is reflected in the type of the parcel.

```

1 namespace Foam {
2     typedef ThermoCloud
3     <
4         KinematicCloud
5         <
6             Cloud
7             <
8                 basicThermoParcel
9             >
10        >
11    > basicThermoCloud;
12 }

```

Listing 257: The class definition of `basicThermoCloud` in the file `basicThermoCloud.H`

¹²¹As the reader might remember, the class `Cloud` takes a template parameter `ParticleType`.

```

1 namespace Foam {
2     typedef ThermoParcel<KinematicParcel<particle>> basicThermoParcel;
3 }

```

Listing 258: The class definition of `basicThermoParcel` in the file `basicThermoParcel.H`

40.5 Run-time post-processing

The cloud class offers function object to perform some run-time post-processing. These can be found at `$FOAM_SRC/lagrangian/intermediate/submodels/CloudFunctionObjects`. Among the cloud function objects are ones for counting particles crossing a certain patch or computing the volume fraction field of the lagrangian particles. See Section 40.7.2 for a more detailed discussion on cloud function objects.

40.6 Times of Use

40.6.1 Not so telling error messages

Out of domain

As OpenFOAM's Lagrangian particle framework keeps track of the cells in which a particle is located, a Lagrangian solver needs to determine the cell label of each particle's initial position. OpenFOAM's particle tracking algorithm is described among other resources in [43, 40, 47].

When a particle is placed outside the domain, i.e. the position in the `positions` file is outside the domain, OpenFOAM is unable to find a cell label for this very particle. Note that failing to find a cell which contains the particle's location may happen also for other reasons than placing it outside the domain. As the error message in Listing 259 suggests, this might also happen through a combination of insufficient write precision and domain decomposition or reconstruction. However, plainly putting them outside the domain is also a possibility, especially, when a script is used to create the initial particle distribution.

```

--> FOAM FATAL ERROR:

cell, tetFace and tetPt search failure at position (0.0026 0.0026 0.4502)
for requested cell 0
If this is a restart or reconstruction/decomposition etc. it is likely that the write
precision is not sufficient.
Either increase 'writePrecision' or set 'writeFormat' to 'binary'

From function void Foam::particle::initCellFacePt()
in file /home/user/OpenFOAM/OpenFOAM-2.3.x/src/lagrangian/basic/lnInclude/particleI.H at
line 758.

FOAM aborting

```

Listing 259: Error message issued by OpenFOAM when a Lagrangian simulation is started with particle positions defined outside of the domain; `checkMesh` reports for this case an *Overall domain bounding box (0 0 0) (0.15 0.15 0.45)*; Note the position (Line 3) at which the search failure occurs

40.7 Sub models

40.7.1 Injection models

There are a number of models to insert Lagrangian particles or parcels into the simulation domain.

Common controls inherited from base classes

The injection models have a number of base classes which determine common behaviour and provide common data. Common control parameters are the *start of injection* (SOI), and the mass to be injected (`massTotal`).

ManualInjection

The `manualInjection` model is probably the simplest model. The user needs to provide the to-be-injected particle mass and the injection positions. All parcels are introduced at SOI.

CellZoneInjection

The `cellZoneInjection` model works similar to the manual injection model. The locations at which parcels are injected, however, are determined using a user-provided `cellSet`. The actual injection locations are randomly distributed across the `cellSet`. The number of inserted parcels is determined by the volume of the `cellSet` and the user-specified target parcel number density. All parcels are introduced at once at SOI.

FieldActivatedInjection

The `fieldActivatedInjection` model is also related to the manual injection model. In addition to the user-provided injection locations, a scalar factor and the names of a threshold field and a referenceField have to be specified. Parcels are injected only at locations at which the following relation holds:

$$\text{factor} * \text{referenceField}[\text{celli}] \geq \text{thresholdField}[\text{celli}] \quad (61)$$

Parcel injection is controlled by the injector positions read from the positions file. All positions, which fulfil the condition above are valid injector positions. Furthermore, the number of parcels per injector (`parcelsPerInjector`) has to be specified. Each injector injects `parcelsPerInjector` parcels which account for a `parcelsPerInjector`-th of `massTotal`. Parcels are injected from SOI onwards until `massTotal` is reached.

PatchInjection

The `patchInjection` model introduces parcels at a patch rather than within a volume like the models discussed above. This injection model implements a classical inflow condition for Lagrangian particles. The injection position on the patch is chosen randomly. Parcels are injected from SOI until `massTotal` is reached.

40.7.2 CloudFunctionObjects

`CloudFunctionObject` are, similar to the standard `functionObjects` for fields, function objects for Lagrangian clouds. The main purpose of OpenFOAM's cloud function objects – judging by the implemented ones at the time of writing – is post-processing. However, cloud function objects can also be used to actively influence the Lagrangian particles.

PatchPostProcessing

This cloud function object can be used to accumulate data from all particles leaving the domain through a specified patch. Thus, e.g. a residence time distribution (RTD) can be generated using this data.

FacePostProcessing

This cloud function object serves a similar purpose as the `PatchPostProcessing` cloud function object, only this one acts on face sets.

VoidFraction

This cloud function object can be used to create the volume fraction field associated with the particles. Thus, the name `VoidFraction` can be misleading.

If we take a look at the code, we clearly see that the volume fraction of the particles is computed. In Listing 260, we see the two methods of this cloud function object which are responsible for computing the result. The field `theta` is initialized to zero, this is not shown in the listing. In the method `postMove()`, which is called for each parcel after it has moved within the current timestep, the particle's volume is added to the field `theta`. In the method `postEvolve()`, which is called after evolving the Lagrangian cloud for the current time step has been completed, the field `theta` is divided by the mesh's volume, i.e. a field in which the value of a cell is that cell's volume.

Thus, as in each cell the volume of the particles is accumulated and subsequently divided by the volume of the cell, we end up with the volume fraction of the particles.

```

1  template<class CloudType> void Foam::VoidFraction<CloudType>::postMove
2  (
3      // ...
4  )
5  {
6      volScalarField& theta = thetaPtr_();
7      theta[celli] += dt*p.nParticle()*p.volume();
8  }
9
10 template<class CloudType> void Foam::VoidFraction<CloudType>::postEvolve()
11 {
12     volScalarField& theta = thetaPtr_();
13     const fvMesh& mesh = this->owner().mesh();
14     theta.primitiveFieldRef() /= mesh.time().deltaTValue()*mesh.V();
15     CloudFunctionObject<CloudType>::postEvolve();
16 }

```

Listing 260: Two methods of `VoidFraction.C`

ParticleCollector

The ParticleCollector cloud function object can be used to count particles traversing an arbitrary surface defined by a set of polygons or the area enclosed by concentric circles. The data accumulated by this cloud function object involves the accumulated particle mass and the mass flow rate passing through the surface.

Optionally, counted particles can be removed from the simulation.

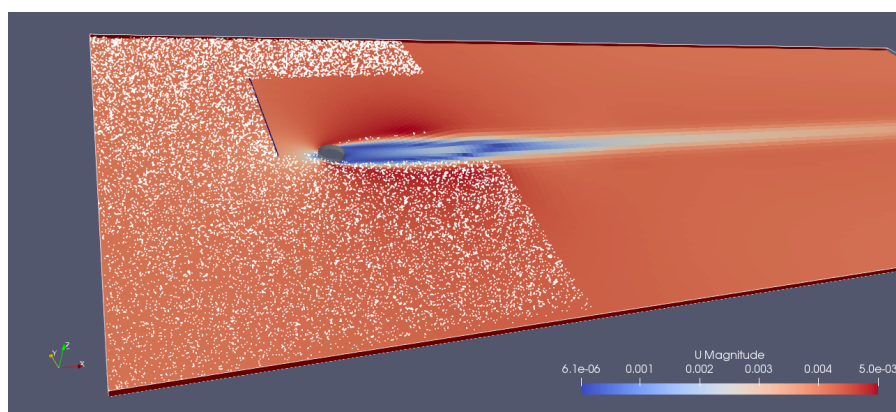


Figure 102: A set of polygons has been defined to count and remove traversing particles. In this case of a cylinder in laminar cross-flow, particles are inserted through the inlet patch. The ParticleCollector cloud function object was set to remove all counted particles, which is clearly visible in this snapshot.

Part VI

Solver

41 Solution Algorithms

The solution of the Navier-Stokes equations require solving the coupled equations of velocity and pressure fields. There are several solution algorithms that try to decouple the equations and compute the velocity and pressure separately. In order to decouple the computation of velocity and pressure, a predictor-corrector strategy is followed. This approach is referred to in literature on numerical methods as *segregated solution*.

The alternative to solving for velocity and pressure in a segregated fashion is to solve for the fully coupled – also referred to as block coupled – equation system. In general, the momentum equation yields three equations for the three velocity components and the pressure equation, which is derived from the continuity equation, yields an equation for pressure. Instead of solving each of the four discretized equations individually, the fully coupled set of equations could also be solved for. This translates a possible improvement in converge rate, however, it also encompasses a much larger memory requirement and an altered convergence behaviour.

This section will deal with the segregated solution methods, as they are the most commonly used. In the last sub-section we will briefly discuss coupled methods.

41.1 SIMPLE

Figure 103 shows the flow chart of the SIMPLE algorithm. The SIMPLE algorithm predicts the velocity and then corrects both the pressure and the velocity. This is repeated until a convergence criteria is reached. The labels in Figure 103 are related to the terminology used in the source code of the `simpleFoam` solver. The solution procedure can be described as follows

1. Check if convergence is reached – `simple.loop()`
2. Predict the velocities using the momentum predictor– `UEqn.H`
3. Correct the pressure and the velocities– `pEqn.H`
4. Solve the transport equations for the turbulence model¹²²– `turbulence->correct()`
5. Go back to step 1

In OpenFOAM the SIMPLE algorithm is used for steady-state solvers.

41.1.1 Predictor

The predictor of *simpleFoam* is a momentum predictor.

```
1 // Momentum predictor
2 tmp<fvVectorMatrix> UEqn
3 (
4     fvm::div(phi, U)
5     + turbulence->divDevReff(U)
6     ==
7     sources(U)
8 );
9
10 UEqn().relax();
11
12 sources.constrain(UEqn());
13
14 solve(UEqn() == -fvc::grad(p));
```

Listing 261: Predictor in *UEqn.H* of *simpleFoam*

¹²²In case of a laminar simulation an empty function is called. Turbulence is modelled in OpenFOAM in a very generic way. Therefore, a laminar simulation uses the `laminar` turbulence model.

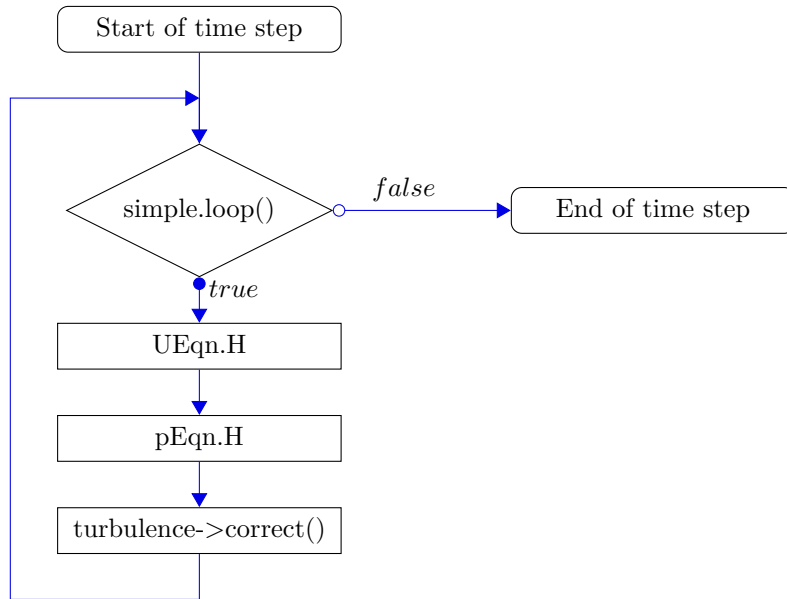


Figure 103: Flow chart of the SIMPLE algorithm

41.1.2 Corrector

The corrector is used to correct the pressure field by using the predicted velocity. This corrected pressure is used to correct the velocities by solving the continuity equation.

The non-orthogonal pressure corrector loop is necessary only for non-orthogonal meshes [50].

```

p.boundaryField().updateCoeffs();

volScalarField rAU(1.0/UEqn().A());
U = rAU*UEqn().H();
UEqn().clear();

phi = fvc::interpolate(U, "interpolate(HbyA)") & mesh.Sf();
adjustPhi(phi, U, p);

// Non-orthogonal pressure corrector loop
while (simple.correctNonOrthogonal())
{
    fvScalarMatrix pEqn
    (
        fvm::laplacian(rAU, p) == fvc::div(phi)
    );
    pEqn.setReference(pRefCell, pRefValue);

    pEqn.solve();

    if (simple.finalNonOrthogonalIter())
    {
        phi -= pEqn.flux();
    }
}

#include "continuityErrs.H"

// Explicitly relax pressure for momentum corrector
p.relax();

// Momentum corrector
U -= rAU*fvc::grad(p);
U.correctBoundaryConditions();
sources.correct(U);

```

Listing 262: Corrector in *pEqn.H* of *simpleFoam*

41.2 PISO

The PISO algorithm also follows the predictor-corrector strategy. Figure 104 shows the flow chart of the PISO algorithm. The velocity is predicted using the momentum predictor. Then, the pressure and the velocity is corrected until a predefined number of iterations is reached. Afterwards, the transport equations of the turbulence model are solved.

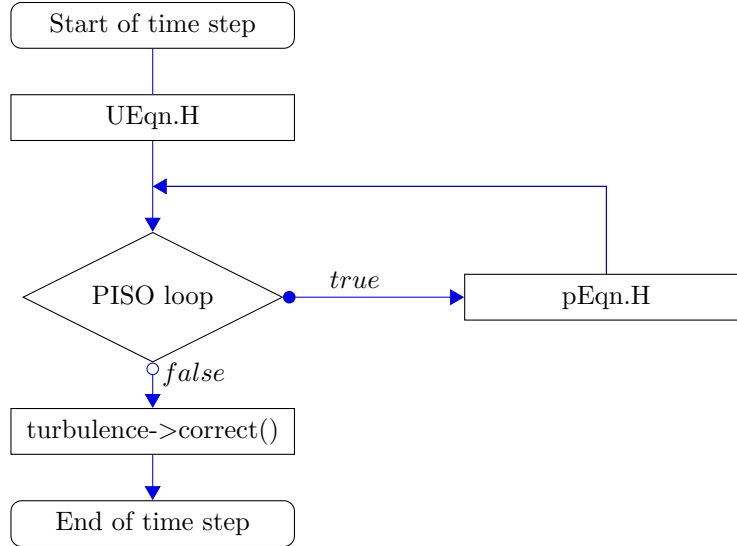


Figure 104: Flow chart of the PISO algorithm

41.3 PIMPLE

The PIMPLE algorithm, which is a combination of the SIMPLE and PISO algorithms, is discussed further in Sections 42 and 42.2.

41.4 Block-coupled solution

The block-coupled approach is something completely different than the aforementioned segregated algorithms. In block-coupled solutions all equations (three equations for velocity and one for pressure) are solved at the same time. This requires the construction of a fully coupled, discretised equation system for all four variables. Thus, block-coupled solvers have a much larger memory footprint than sequential solvers.

41.4.1 Block-coupled solvers

The foam-extend project <https://foam-extend.sourceforge.io/>, at the time of writing, distributes two solvers using the block-coupled solution approach.

blockCoupledScalarTransportFoam

This solver is derived from the standard *scalarTransportFoam* solver and solves the transport of two coupled passive scalars.

$$\nabla \cdot (\mathbf{u}T) + \nabla \cdot (D_T \nabla T) = \alpha (T_s - T) \quad (62)$$

$$\nabla \cdot (D_{T_s} \nabla T_s) = \alpha (T - T_s) \quad (63)$$

pUCoupledFoam

This solver is a steady state solver for incompressible, turbulent single phase flow. This solver solves for velocity and pressure simultaneously.

42 *pimpleFoam*

pimpleFoam is a transient incompressible solver using the PIMPLE algorithm, which is a combination of PISO and SIMPLE. The solver is described in the file `pimpleFoam.C` as follows:

```
Large time-step transient solver for incompressible, flow using the PIMPLE
(merged PISO-SIMPLE) algorithm.
```

```
Turbulence modelling is generic, i.e. laminar, RAS or LES may be selected.
```

42.1 Governing equations

42.1.1 Continuity equation

The general continuity equation reads as follows:

$$\frac{\partial \rho}{\partial t} + \nabla \cdot (\rho \mathbf{u}) = 0 \quad (64)$$

we now assume incompressible fluids: $\rho = \text{const}$

$$\nabla \cdot \mathbf{u} = 0 \quad (65)$$

or in alternative notation

$$\text{div}(\mathbf{u}) = 0 \quad (66)$$

$$\frac{\partial u_i}{\partial x_i} = 0 \quad (67)$$

42.1.2 Momentum equation

Departing from the Navier-Stokes equations, the momentum equation of *pimpleFoam* are derived.

$$\frac{\partial \rho \mathbf{u}}{\partial t} + \nabla(\rho \mathbf{u} \mathbf{u}) + \nabla \cdot \boldsymbol{\tau} = -\nabla p + \mathbf{g} \quad (68)$$

because we assume a constant density we can divide by ρ

$$\frac{\partial \mathbf{u}}{\partial t} + \nabla(\mathbf{u} \mathbf{u}) + \frac{1}{\rho} \nabla \cdot \boldsymbol{\tau} = -\frac{\nabla p}{\rho} + \frac{\mathbf{g}}{\rho} \quad (69)$$

The last term is defined a general source term

$$\frac{\partial \mathbf{u}}{\partial t} + \nabla(\mathbf{u} \mathbf{u}) + \frac{1}{\rho} \nabla \cdot \boldsymbol{\tau} = -\frac{\nabla p}{\rho} + \mathbf{Q} \quad (70)$$

the shear stresses and the pressure are denoted by new symbols: $\frac{\boldsymbol{\tau}}{\rho} = \mathbf{R}^{eff}$ und $\frac{p}{\rho} = p$

$$\frac{\partial \mathbf{u}}{\partial t} + \nabla(\mathbf{u} \mathbf{u}) + \nabla \cdot \mathbf{R}^{eff} = -\nabla p + \mathbf{Q} \quad (71)$$

The Boussinesq hypothesis allows us to add the Reynolds stresses to the shear stresses. This stress tensor – containing shear as well as Reynolds stresses – is denoted $\mathbf{R}^{eff}$, the effective stress tensor. Both RAS as well as LES turbulence models are based on the Boussinesq hypothesis.

$$\mathbf{R}^{eff} = -\nu^{eff} (\nabla \mathbf{u} + (\nabla \mathbf{u})^T) \quad (72)$$

$$R_{ij}^{eff} = -\nu^{eff} \left(\frac{\partial u_i}{\partial x_j} + \frac{\partial u_j}{\partial x_i} \right) \quad (73)$$

The trace of τ fulfills the continuity equation for incompressible fluids

$$\text{tr}(\mathbf{R}^{eff}) = R_{ii}^{eff} = -2\nu^{eff} \left(\frac{\partial u_i}{\partial x_i} \right) = 0 \quad (74)$$

$$\frac{\partial u_i}{\partial x_i} = \nabla \cdot \mathbf{u} = 0 \quad (75)$$

Therefore, we can replace $\mathbf{R}^{eff}$ with the deviatoric part of $\mathbf{R}^{eff}$

$$\mathbf{R}^{eff} = \underbrace{\text{dev}(\mathbf{R}^{eff})}_{\text{deviatoric part}} + \underbrace{\frac{1}{3}\text{tr}(\mathbf{R}^{eff})\mathbf{I}}_{\text{hydrostatic part}} \quad (76)$$

$$\text{dev}(\mathbf{R}^{eff}) = \mathbf{R}^{eff} - \underbrace{\frac{1}{3}\text{tr}(\mathbf{R}^{eff})\mathbf{I}}_{=0} \quad (77)$$

Therefore, the momentum equation can be rewritten

$$\frac{\partial \mathbf{u}}{\partial t} + \nabla(\mathbf{u}\mathbf{u}) + \underbrace{\nabla \cdot (\text{dev}(\mathbf{R}^{eff}))}_{=\text{div}(\text{dev}(\mathbf{R}^{eff}))} = -\nabla p + \mathbf{Q} \quad (78)$$

Finally, we use Eq. (72)

$$\mathbf{R}^{eff} = -\nu^{eff} (\nabla \mathbf{u} + (\nabla \mathbf{u})^T) \quad (72)$$

to gain

$$\frac{\partial \mathbf{u}}{\partial t} + \nabla(\mathbf{u}\mathbf{u}) + \nabla \cdot (\text{dev}(-\nu^{eff} (\nabla \mathbf{u} + (\nabla \mathbf{u})^T))) = -\nabla p + \mathbf{Q} \quad (79)$$

42.1.3 Implementation

The momentum equation is implemented in the file `UEqn.H`. The first two terms of Eq. (79) can easily be identified in the source code in Listing 263.

The first term is the local derivative of the momentum – due to the incompressibility of the fluid, the density was eliminated – can be found in line 5 of Listing 263. Here, the instruction in the source code reads very much the same as the mathematical notation.

$$\frac{\partial \mathbf{u}}{\partial t} \quad \Leftrightarrow \quad \text{fvm}::\text{ddt}(\mathbf{U})$$

The second term of Eq. (79) is the convective transport of momentum. The use of the identifier `phi` should not lead to confusion. In order to read the equations from the source code, `phi` can be replaced with `U` without changing the meaning of the equations. The reason why `phi` is used in the source code lies in the solution procedure. See Section 63 for a detailed discussion about `phi`.

$$\underbrace{\nabla(\mathbf{u}\mathbf{u})}_{\text{div}(\mathbf{u}\mathbf{u})} \quad \Leftrightarrow \quad \text{fvm}::\text{div}(\text{phi}, \mathbf{U})$$

The third term of Eq. (79) is the diffusive momentum transport term. Diffusive momentum transport is caused by the laminar viscosity as well as turbulence. Therefore, the turbulence model handles this term. See line 7 of Listing 263.

$$\underbrace{\nabla \cdot (\text{dev}(\mathbf{R}^{eff}))}_{=\text{div}(\text{dev}(\mathbf{R}^{eff}))} \quad \Leftrightarrow \quad \text{turbulence} \rightarrow \text{divDevReff}(\mathbf{U})$$

The terms on the *rhs* of Eq. (79) are the pressure gradient and the source term.

$$\underbrace{-\nabla p}_{=-\text{grad } p} \Leftrightarrow -\text{fvc}::\text{grad}(p)$$

$$\mathbf{Q} \Leftrightarrow \text{sources}(U)$$

```

1 // Solve the Momentum equation
2
3 tmp<fvVectorMatrix> UEqn
4 (
5     fvm::ddt(U)
6     + fvm::div(phi, U)
7     + turbulence->divDevReff(U)
8 );
9
10 UEqn().relax();
11
12 sources.constrain(UEqn());
13
14 volScalarField rAU(1.0/UEqn().A());
15
16 if (pimple.momentumPredictor())
17 {
18     solve(UEqn() == -fvc::grad(p) + sources(U));
19 }

```

Listing 263: The file *UEqn.H* of *pimpleFoam*

42.2 The PIMPLE Algorithm – or, what’s under the hood?

This Section deals with the way *pimpleFoam* and *twoPhaseEulerFoam*, which also uses the PIMPLE algorithm, work. Therefore, we examine the implementation of *pimpleFoam*. Listing 264 shows the main loop of *pimpleFoam*.

The first instruction is the loop over all time steps. Then there are some operations – the three `#include` instructions – concerning time step control. After incrementing the time step (Line 7), the PIMPLE loop comes (from Line 10 onwards).

Inside this loop, first the momentum equation is solved (Line 12), then the pressure correction loop is entered (Line 17).

At the end of the PIMPLE loop the turbulent equations¹²³ – if there are any present¹²⁴ – are solved (Line 22). At the end of each time step the data is written.

```

1 while (runTime.run())
2 {
3     #include "readTimeControls.H"
4     #include "CourantNo.H"
5     #include "setDeltaT.H"
6
7     runTime++;
8
9     // --- Pressure-velocity PIMPLE corrector loop
10    while (pimple.loop())
11    {
12        #include "UEqn.H"
13
14        // --- Pressure corrector loop
15        while (pimple.correct())
16        {
17            #include "pEqn.H"
18        }
19    }

```

¹²³In case of a $k-\epsilon$ model, there are two transport equations to be solved. Other turbulence models require the solution of less or none transport equation.

¹²⁴In case of a laminar simulation, no operation is carried out.

```

20     if (pimple.turbCorr())
21     {
22         turbulence->correct();
23     }
24 }
25
26     runTime.write();
27 }

```

Listing 264: The main loop of *pimpleFoam*

Figure 105 shows the flow chart of the PIMPLE algorithm. This algorithm is executed every time step. If the PIMPLE loop is entered only once, then the algorithm is essentially the same as the PISO algorithm. Listing 271 draws this conclusion from the code itself.

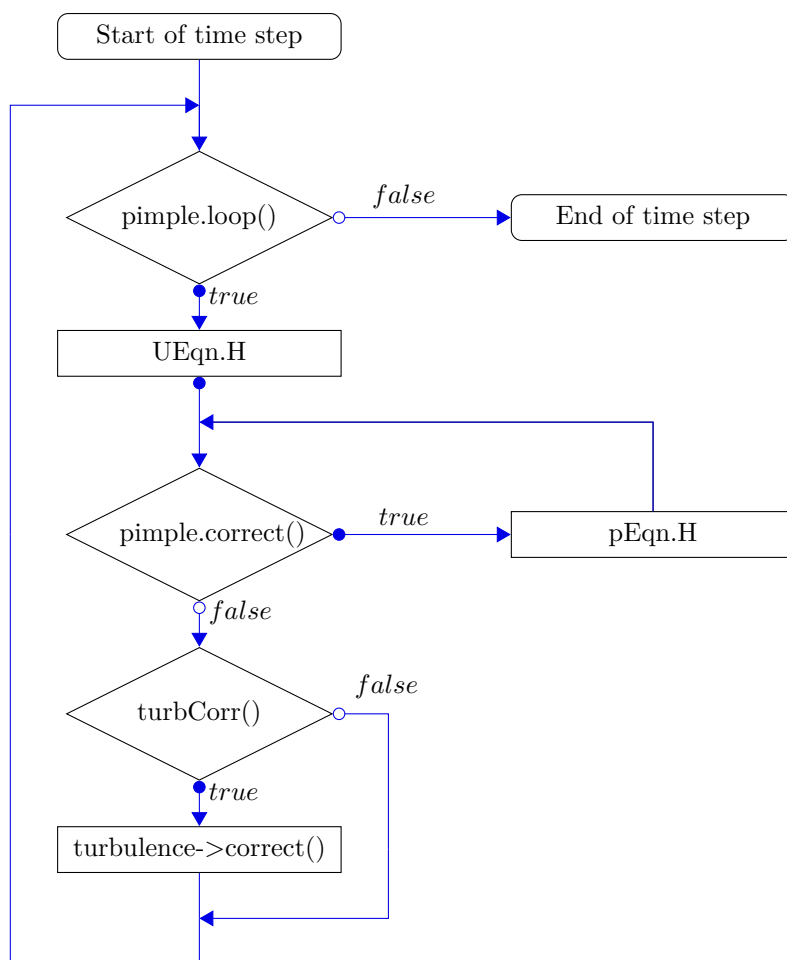


Figure 105: Flow chart of the PIMPLE algorithm

42.2.1 readTimeControls.H

In line 3 of Listing 264 the file `readTimeControls.H` is included to the source code using the `#include` preprocessor macro. This is a very common way to give the code of OpenFOAM structure and order. Code which is used repeatedly is outsourced into a separate file. This file is then included with the `#include` macro. Thus, code duplication is prevented. The file `readTimeControls.H` might be included into every solver that is able to use variable time steps. If this code was not outsourced into a separate file, this code would be found in every variable time step solver. Maintaining this code, would be tiresome and prone to errors.

Listing 421 shows the contents of `readTimeControls.H`. The first instruction reads from `controlDict` the `adjustTimeStep` parameter. If there is no entry matching the name of the parameter ("`adjustTimeStep`"), then

```

1  const bool adjustTimeStep =
2      runTime.controlDict().lookupOrDefault("adjustTimeStep", false);
3  scalar maxCo =
4      runTime.controlDict().lookupOrDefault<scalar>("maxCo", 1.0);
5  scalar maxDeltaT =
6      runTime.controlDict().lookupOrDefault<scalar>("maxDeltaT", GREAT);

```

Listing 265: The content of `readTimeControls.H`

a default value is used. So, omitting the parameter *adjustTimeStep* in *controlDict* will result in a simulation with a fixed time step.

This is a very straight forward example of determining the behaviour of a solver using only the source code. In this case the names of the source file as well as variable and function names are rather self explaining. In other cases one has to dig deeply into the code to learn about what a certain command does.

42.2.2 pimpleControl

Examining the files `pimpleControl.H` and `pimpleControl.C` will generate some knowledge of the inner life of *pimpleFoam*.

Solution controls

Listings 266 and 267 show parts of `pimpleControl.H` and `pimpleControl.C`. Listing 266 shows the declaration of protected¹²⁵ data in `pimpleControl.H`.

```

1  // Protected data
2  // Solution controls
3  //- Maximum number of PIMPLE correctors
4  label nCorrPIMPLE_;
5
6  //- Maximum number of PISO correctors
7  label nCorrPISO_;
8
9  //- Current PISO corrector
10 label corrPISO_;
11
12 //- Flag to indicate whether to only solve turbulence on final iter
13 bool turbOnFinalIterOnly_;
14
15 //- Converged flag
16 bool converged_;

```

Listing 266: Protected data in `pimpleControl.H`

```

1  void Foam::pimpleControl::read()
2  {
3      solutionControl::read(false);
4
5      // Read solution controls
6      const dictionary& pimpleDict = dict();
7
8      nCorrPIMPLE_ = pimpleDict.lookupOrDefault<label>("nOuterCorrectors", 1);
9
10     nCorrPISO_ = pimpleDict.lookupOrDefault<label>("nCorrectors", 1);
11
12     turbOnFinalIterOnly_ = pimpleDict.lookupOrDefault<Switch>("turbOnFinalIterOnly", true);
13 }

```

Listing 267: Read solution controls in `pimpleControl.C`

¹²⁵Most programming languages provide *access specifiers* to specify the visibility of variables. The keyword `protected` means, that the variables can be accessed only inside the class `pimpleControl` and all classes inherited from `pimpleControl`.

Reading the code we can see which keyword in the `PIMPLE` dictionary – it is a part of the `fvSolution` dictionary (see Section 11.5) – is connected to which variable in the code. Three of the protected variables of Listing 266 are assigned in Listing 267. One of them has the same name in both the code and the dictionary. The other two have different names.

Pitfall: no sanity checks

The two variables `nCorrPimple` and `nCorrPiso` control the solution algorithm. If the corresponding entry in the `PIMPLE` dictionary in `fvSolution` is missing, then default values are used, see Section 57.3 for details behind the method `lookupOrDefault()`. However, the user can provide any number in `fvSolution` as long as it is legal¹²⁶. Thus, a zero or negative number is a legal entry from the source codes point of view. With respect to the solution algorithm a zero or negative entry makes no sense at all.

The connection between keywords and the algorithm

The keyword `nOuterCorrectors` translates – with the help of Listing 267 to the variable `nCorrPIMPLE_`. This variable controls how often the `PIMPLE` loop is traversed. Listing 268 shows parts of the definition of the function `loop()` of the class `pimpleControl`. The return value of this function decides whether the `PIMPLE` loop is entered or not. In line 5 of Listing 268 an internal counter is incremented – the `++` operator of C++ adds 1 to the variable the operator is applied to. Afterwards, the internal counter is compared to the value of `nCorrPIMPLE_`. If this internal counter is then equal to the sum of `nCorrPIMPLE_ + 1`, then the function `loop()` returns `false`.

The internal counter is initialised to the value of 0. Listing 269 shows the constructor of the class `solutionControl`. The class `pimpleControl` is derived from `solutionControl`. So, every instance of `pimpleControl` has an internal counter `corr_` inherited from `solutionControl`. Line 9 of Listing 269 how the counter `corr_` is initialised to zero.

```

1  bool Foam::pimpleControl::loop()
2  {
3      read();
4
5      corr_++;
6
7      /* code removed for the sake of brevity */
8
9      if (corr_ == nCorrPIMPLE_ + 1)
10     {
11         if ((!residualControl_.empty()) && (nCorrPIMPLE_ != 1))
12         {
13             Info<< algorithmName_ << ": not converged within "
14                 << nCorrPIMPLE_ << " iterations" << endl;
15         }
16
17         corr_ = 0;
18         mesh_.data::remove("finalIteration");
19         return false;
20     }
21
22     /* code continues */

```

Listing 268: Some content of `pimpleControl.C`

```

1  Foam::solutionControl::solutionControl(fvMesh& mesh, const word& algorithmName)
2  :
3      mesh_(mesh),
4      residualControl_(),
5      algorithmName_(algorithmName),
6      nNonOrthCorr_(0),
7      momentumPredictor_(true),
8      transonic_(false),
9      corr_(0),
10     corrNonOrtho_(0)
11 {}

```

¹²⁶See Section 57.4.2 for details on the `label` datatype.

Listing 269: The constructor of the class `solutionControl` in `solutionControl.C`

The keyword `nCorrectors` translates – with the help of Listing 267 to the variable `nCorrPISO_`. This variable controls how often the PISO loop – or the corrector loop – is traversed. Listing 266 shows, that there are two variables related to the PISO loop, `nCorrPISO_` and `corrPISO_`. The first variable is the limit and the second is the counter.

`nCorrPISO_` is read from the `fvSolution` dictionary by the use of the `nCorrectors` keyword. This number tells the solver, how many times the corrector loop should be traversed. The corrector loop is a feature of the PISO algorithm. Hence, the maximum number of corrector loop iterations is called `nCorrPISO_`.

The variable `corrPISO_` is declared in the constructor of the class `pimpleControl`, see Listing 271. There the variable is initialised to zero.

Listing 270 shows the definition of the function `correct()` of the class `pimpleControl`. The return value of this function controls if the corrector loop is entered. In line 3 the counter `corrPISO_` is incremented every time this function is called. In line 10 the value of the counter is compared to the maximum number of corrector loop iterations.

```
1 inline bool Foam::pimpleControl::correct()
2 {
3     corrPISO_++;
4
5     if (debug)
6     {
7         Info<< algorithmName_ << " correct: corrPISO = " << corrPISO_ << endl;
8     }
9
10    if (corrPISO_ <= nCorrPISO_)
11    {
12        return true;
13    }
14    else
15    {
16        corrPISO_ = 0;
17        return false;
18    }
19 }
```

Listing 270: The inline function `correct()` in `pimpleControlI.H`

PIMPLE or PISO algorithm

Listing 271 shows parts of the code of the constructor of the class `pimpleControl`. At first some data fields are set to initial values. Then the `read()` function is called, this function is shown in Listing 267. After reading the solution controls the variable `nCorrPIMPLE_` is tested. If this value is equal to one, then the solution algorithm equates the PISO algorithm. In this case an according message is printed to the Terminal.

```
1 Foam::pimpleControl::pimpleControl(fvMesh& mesh) :
2     solutionControl(mesh, "PIMPLE"),
3     nCorrPIMPLE_(0),
4     nCorrPISO_(0),
5     corrPISO_(0),
6     turbOnFinalIterOnly_(true),
7     converged_(false)
8 {
9     read();
10
11     if (nCorrPIMPLE_ > 1)
12     {
13         /* code removed for shortness of listing */
14     }
15     else
16     {
17         Info<< nl << algorithmName_ << ": Operating solver in PISO mode" << nl << endl;
18     }
19 }
```

Listing 271: Constructor of `pimpleControl` in `pimpleControl.C`

42.2.3 Residual control of the outer PIMPLE-loop

The keyword `nOuterCorrectors` controls the number of outer-loop iterations used for each time step by the PIMPLE algorithm. In addition to the fixed number `nOuterCorrectors`, the number of outer-loop iterations can also be controlled by the residuals of the actual solution. If this residual-control is used, then `nOuterCorrectors` sets the upper limit for outer-loop iterations, if the specified tolerance for the residuals is met with less iterations, then the outer-loop is considered converged.

A note on OpenFOAM-6

With OpenFOAM-6¹²⁷ the residual-control framework of the PIMPLE solution control class was reorganized, making it consistent with the SIMPLE solution control class. With this changes, `residualControl` acts on the simulation as a whole, whereas the new `outerCorrectorResidualControl` manages the outer-loop of the PIMPLE algorithm.

```
PIMPLE
{
    // ...

    outerCorrectorResidualControl
    {
        p_rgh
        {
            relTol          0;
            tolerance       0.0001;
        }
    }
}
```

Listing 272: *outerCorrectorResidualControl*

43 rhoPimpleFoam

43.1 General remarks

`rhoPimpleFoam` is a compressible, single-phase solver.

43.2 Solution algorithm

`rhoPimpleFoam` uses, as the name indicates, the PIMPLE algorithm.

43.3 Governing equations

43.3.1 Density

The solver *rhoPimpleFoam* is a pressure-based solver, which solves for the velocity and the pressure field. The alternative would be a density-based solver, which solves for velocity and density. However, density-based solvers are quite rare in OpenFOAM, as the only solver known to the author is `rhoCentralFoam`.

For the density there are two options: either compute the density from the continuity equation; or compute it from the pressure via the thermodynamic model, i.e. the equation of state. These two options are controlled by the keyword `SIMPLErho` in the PIMPLE dictionary of the file `fvSolution`.

¹²⁷<https://github.com/OpenFOAM/OpenFOAM-dev/commit/4c8122783aedaa7dadf0486163a98350e625db32#diff-13b1708e0fc8512ad44fa3edd4175459>

Solving the continuity equation

In Listing 273 the continuity equation is shown. This equation is solved, when the keyword `SIMPLErho` is set to false.

```
1 {
2     fvScalarMatrix rhoEqn
3     (
4         fvm::ddt(rho)
5         + fvc::div(phi)
6         ==
7         fvOptions(rho)
8     );
9
10    fvOptions.constrain(rhoEqn);
11
12    rhoEqn.solve();
13
14    fvOptions.correct(rho);
15 }
```

Listing 273: The continuity equation in the file `rhoEqn.H`

Computing the density via the equation of state

The other option to compute the density is to compute it via the equation of state. This option is used when the keyword `SIMPLErho` is set to true.

Below, the equation of state for a perfect gas is shown.

$$\rho = \frac{p}{RT} \quad (80)$$

In the example of a perfect gas, using the equation of state has the advantage that the density is computed from the pressure and the temperature, which have already been computed.

This option was described in the relevant commit message¹²⁸ as follows:

With this option the density is updated from thermodynamics rather than continuity after the pressure equation which is better behaved if pressure is relaxed and/or solved to a loose relative tolerance.

44 *twoPhaseEulerFoam*

This section is valid for OpenFOAM-2.0 til OpenFOAM-2.2.

44.1 General remarks

twoPhaseEulerFoam is a solver for two-phase problems. According to the CFD-Online Forum (<http://www.cfd-online.com/Forums/openfoam/>) this solver as well as *bubbleFoam* is based on the PhD thesis of Henrik Rusche [54]. In the course of an update of OpenFOAM-2.1.x in July 2012 the solution algorithm of the continuity equation was changed.

44.1.1 Turbulence

twoPhaseEulerFoam can only use the k- ϵ turbulence model. This model is so to say hardcoded and can only be turned on or off.

¹²⁸<https://github.com/OpenFOAM/OpenFOAM-dev/commit/79ff91350ef6092b2dd864029a6023bec2b66b50>

44.1.2 Kinetic theory

twoPhaseEulerFoam can make use of the kinetic theory for granular simulations, e.g. air flowing through a bed of small particles. This model can also be turned on or off.

In the following sections kinetic theory is ignored for the reason of keeping listings and explanations short.

44.2 Solver algorithm

twoPhaseEulerFoam is based on the PIMPLE algorithm. However, there are some modifications necessary for solving two-phase problems. Listing 274 shows the main part of this solver. The first two lines inside the main loop (`pimple.loop()`) differ from *pimpleFoam*. These lines deal with the two-phase continuity equation and the inter-phase momentum exchange coefficients.

Next, in line 6, comes the momentum predictor. It contains the momentum equations for both phases and solves them subsequently, thus the filename `UEqns.H`.

After the predictor comes the corrector. The corrector is in fact a corrector loop. Inside this loop (`pimple.correct()`) the correction of pressure and velocity is computed. Inside the corrector loop (line 15) there is also a conditional second call of the continuity equation. The condition consists of two boolean statements. The first is a boolean variable, which is set in a dictionary by the user. The second is generated by the solution control.

After the corrector loop the total time derivatives of the velocities are calculated. Finally, the turbulent transport equations are solved. In this case it is the k - ϵ model that is called explicitly (line 23).

```
1 // --- Pressure-velocity PIMPLE corrector loop
2 while (pimple.loop())
3 {
4     #include "alphaEqn.H"
5     #include "liftDragCoeffs.H"
6     #include "UEqns.H"
7
8     // --- Pressure corrector loop
9     while (pimple.correct())
10    {
11        #include "pEqn.H"
12
13        if (correctAlpha && !pimple.finalIter())
14        {
15            #include "alphaEqn.H"
16        }
17    }
18
19    #include "DDtU.H"
20
21    if (pimple.turbCorr())
22    {
23        #include "kEpsilon.H"
24    }
25 }
```

Listing 274: The main loop of *twoPhaseEulerFoam*

Figure 106 shows the flow chart of all operations that are performed during one time step.

44.2.1 Continuity

The continuity equation is implemented in the file `alphaEqn.H`.

Second call

In line 15 of Listing 274 the continuity equation is called again inside an `if`-statement. The condition depends on two boolean expressions.

The first, `correctAlpha`, is controlled by the `fvSolution` dictionary. Assigning a value to this keyword – the keyword has the same name as the boolean variable in the source code – is mandatory. The reading operation

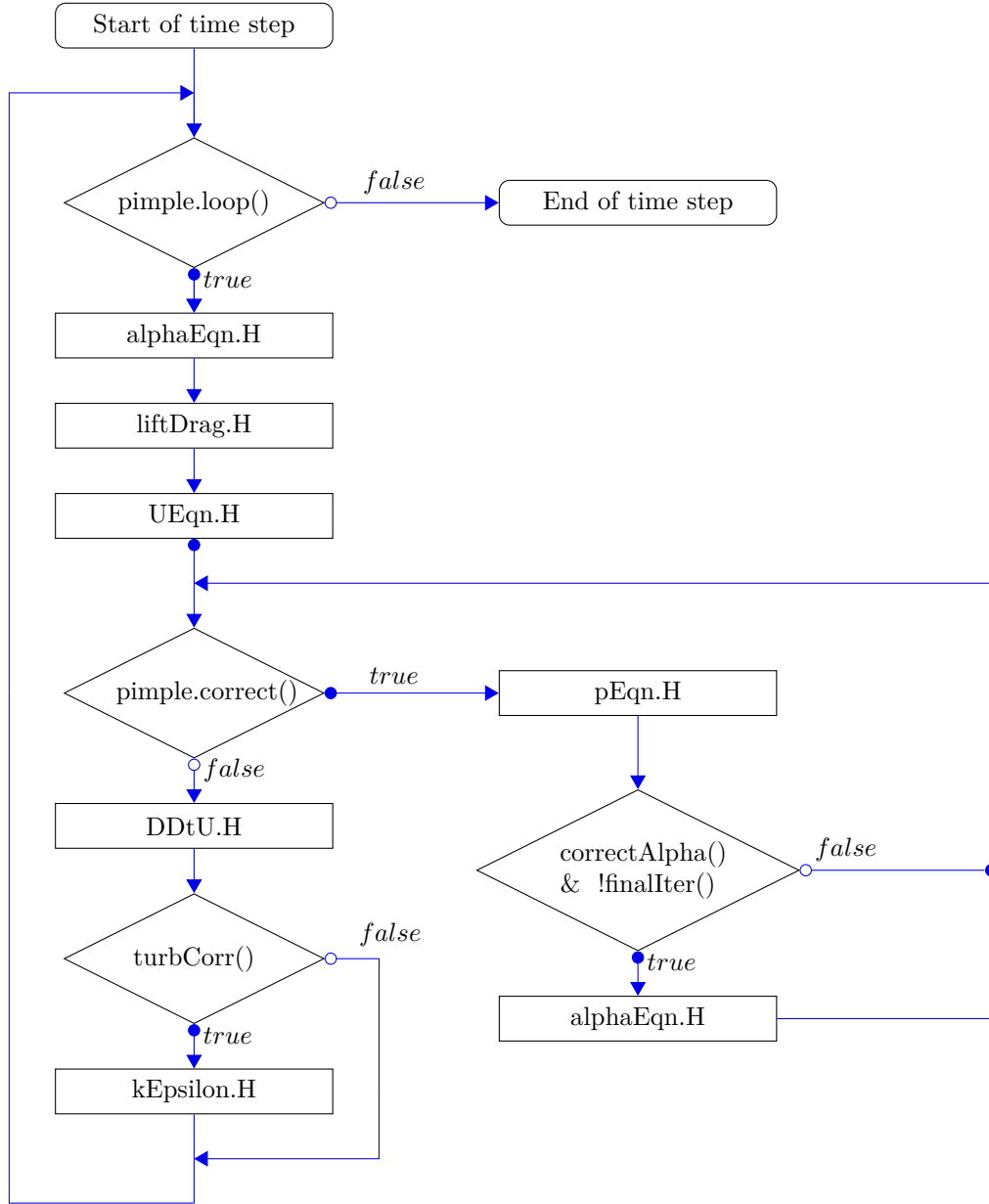


Figure 106: Flow chart of the main loop of *twoPhaseEulerFoam*

of this keyword from the dictionary can be found in the source file `readTwoPhaseEulerFoamControls.H` and is shown in Listing 275.

Three keywords are looked up from the `fvSolution` dictionary. All of them are related to the solving algorithm for the continuity equation. Those entries are read from the dictionary by invoking the function `lookup()`. See Section 57.3 for a detailed discussion about looking up keywords from dictionaries.

```

1 #include "readTimeControls.H"
2
3 int nAlphaCorr(readInt(pimple.dict().lookup("nAlphaCorr")));
4 int nAlphaSubCycles(readInt(pimple.dict().lookup("nAlphaSubCycles")));
5 Switch correctAlpha(pimple.dict().lookup("correctAlpha"));

```

Listing 275: The content of `readTwoPhaseEulerFoamControls.H`

The second boolean expression controlling the second call in line 15 of Listing 274 is controlled by the number of iterations of the PIMPLE loop. See Section 42.2 for a discussion about the PIMPLE algorithm.

The expression `pimple.finalIter()` is `true` when the last iteration of the PIMPLE algorithm is entered. Therefore, the expression `!pimple.finalIter()` is `true` if, and only if, the value of `nOuterCorrectors` or `nCorrPIMPLE_` is greater than one. Because only then, there is more than one PIMPLE iteration and only then, there is an iteration other than the final one.

If the PIMPLE loop is traversed only once, then `alphaEqn.H` is not entered a second time.

The file `alphaEqn.H`

The examination of the file `alphaEqn.H` results in the flow chart in Figure 107. The corrector loop is traversed a specified number of times. This number is set by the keyword `nAlphaCorr` of the `fvSolution` dictionary. The corrector loop is a simple `for` loop.

Inside the corrector loop is a sub-cycle loop. Inside this loop the continuity equation is solved. After the sub-cycle the volume fraction of the continuous phase is updated. The sub-cycle loop is also traversed a specified number of times. This number is set by the keyword `nAlphaSubCycles` of the `fvSolution` dictionary.

When the corrector loop is not entered anymore, the mixture density is updated.

44.3 Momentum exchange between the phases

44.3.1 Drag

The solver *twoPhaseEulerFoam* offers a number of drag models. In the sources of *twoPhaseEulerFoam* there are these models

- Ergun
- Gibilaro
- GidaspowErgunWenYu
- GidaspowSchillerNaumann
- SchillerNaumann
- SyamlalOBrien
- WenYu

The equations behind these models can be found in [25] or [63].

Drag is considered in the governing equations by the use of the so-called drag-function K . This drag-function is either computed directly, or it is computed by the use of the drag coefficient C_d . The drag force is the product of the drag-function and the relative velocity between the phases $\mathbf{U}_r$ [25].

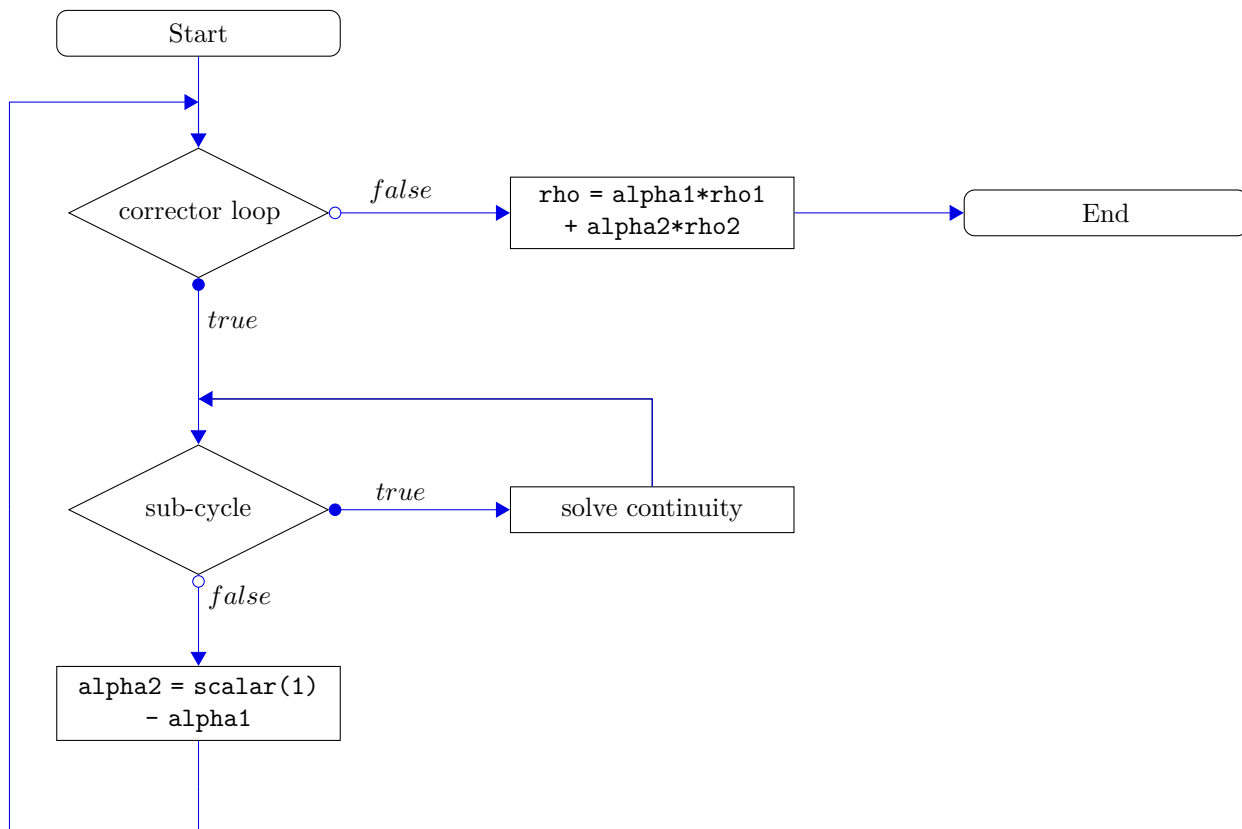


Figure 107: Flow chart of the operations in `alphaEqn.H`

Schiller-Naumann drag

We use the Schiller-Naumann drag model as an example to demonstrate how OpenFOAM calculates the drag force. This drag model utilizes a drag coefficient that is a function of the Reynolds number.

$$C_d = \begin{cases} \frac{24}{Re} (1 + 0.15Re^{0.687}) & \text{if } Re \leq 1000 \\ 0.44 & \text{if } Re > 1000 \end{cases} \quad (81)$$

$$K = \frac{3}{4} C_d \rho_B \frac{U_r}{d_A} \quad (82)$$

The drag coefficient is dimensionless, whereas the product of the drag-function K and the relative velocity has the dimension of a force density.

$$[K] = [C_d] \cdot [\rho_B] \cdot \left[\frac{U_r}{d_A} \right] = 1 \cdot \frac{\text{kg}}{\text{m}^3} \cdot \frac{\text{m}}{\text{s}} \frac{1}{\text{m}} = \frac{\text{kg}}{\text{m}^3 \text{s}}$$

$$[K \cdot U_r] = \frac{\text{kg}}{\text{m}^3 \text{s}} \cdot \frac{\text{m}}{\text{s}} = \frac{\text{kgm}}{\text{s}^2} \cdot \frac{1}{\text{m}^3} = \frac{\text{N}}{\text{m}^3}$$

Listing 276 shows, how the drag-function is computed by the Schiller-Naumann drag model.

```

Foam::tmp<Foam::volScalarField> Foam::SchillerNaumann::K
(
    const volScalarField& Ur
) const
{
    volScalarField Re(max(Ur*phasea_.d()/phaseb_.nu(), scalar(1.0e-3)));

    volScalarField Cds
    (

```

```

    neg(Re - 1000)*(24.0*(1.0 + 0.15*pow(Re, 0.687))/Re)
    + pos(Re - 1000)*0.44
);

return 0.75*Cds*phaseb_.rho()*Ur/phasea_.d();
}

```

Listing 276: Calculation of the drag-function in the file `SchillerNaumann.H`

The drag force contributes to the momentum balance. Probably for numerical reasons, one part of the drag is considered in the momentum equation and the other part is considered in the pressure equation.

44.3.2 Lift

The lift model of *twoPhaseEulerFoam* is described in [54]. The lift model computes the lift force on a rigid sphere in shear flow. The force density is calculated from the relative velocity between the phases and the vorticity of the mixture.

$$\frac{F_L}{V_B} = C_L \rho_c |\mathbf{U}_r \times (\nabla \times \mathbf{U}_c)| \quad (83)$$

mit

$$\begin{aligned} \mathbf{U}_r &= \mathbf{U}_A - \mathbf{U}_B \\ \mathbf{U}_c &= \alpha \mathbf{U}_A + \underbrace{(1 - \alpha)}_{=\beta} \mathbf{U}_B \\ \rho_c &= \alpha \rho_A + \beta \rho_B \end{aligned}$$

The lift force is computed in the file `liftDragCoeffs.H`. The vector field `liftCoeff` contains the lift force density.

```

volVectorField liftCoeff(Cl*(beta*rhob + alpha*rhoa)*(Ur ^ fvc::curl(U)));

```

Listing 277: Berechnung Auftriebskraft; *liftDragCoeffs.H*

The dimensions of the field `liftCoeff` is the dimension of a force density.

$$[liftCoeff] = [C_L] \cdot [\rho_c] \cdot [\mathbf{U}_r \times (\nabla \times \mathbf{U}_c)] = 1 \cdot \frac{\text{kg}}{\text{m}^3} \cdot \frac{\text{m}}{\text{s}} \frac{1}{\text{m}} \frac{\text{m}}{\text{s}} = \frac{\text{kgm}}{\text{s}^2} \cdot \frac{1}{\text{m}^3} = \frac{\text{N}}{\text{m}^3}$$

44.3.3 Virtual mass

The virtual mass – an accelerating bubble needs not only to accelerate its own mass, it also needs to accelerate some of the displaced fluid – is considered in the momentum equation.

$$M_{A,VM} = \beta \frac{\rho_B}{\rho_A} C_{VM} \left(\frac{D_B \mathbf{U}_B}{Dt} - \frac{D_A \mathbf{U}_A}{Dt} \right) \quad (84)$$

In the source code, the momentum exchange term due to virtual mass is split into two parts. One part is included in the *rhs* of the momentum equation, the other is considered in the *lhs*. This separation is probably for numerical reasons.

```

UaEqn =
(
    (scalar(1) + Cvm*rhob*beta/rhoa)*
    (
        fvm::ddt(Ua)
        + fvm::div(phiA, Ua, "div(phiA,Ua)")
        - fvm::Sp(fvc::div(phiA), Ua)
    )
)

```

```

+ /* other terms */
==
/* other terms */
- beta/rhoa*(liftCoeff - Cvm*rhob*DDtUb)
);

```

Listing 278: Terms including virtual mass in the file `UEqns.H`

44.4 Kinetic Theory

For the simulation of dense gas-solid particulate flows the particulate phase can be modelled using the kinetic theory model.

45 *twoPhaseEulerFoam-2.3*

This section is valid for OpenFOAM-2.3.

With the release of OpenFOAM-2.3 the two-phase Eulerian solver *twoPhaseEulerFoam* has seen some major changes. See the release notes for further details: <http://www.openfoam.org/version2.3.0/multiphase.php>.

45.1 Physics

The most important change in *twoPhaseEulerFoam* from version $\leq 2.2.x$ to 2.3 is that the solver is based on a completely different set of physical models. In version 2.3 phases are modelled using OpenFOAMs thermo-physical models. The phases are considered compressible, therefore all simplifications when considering a phase incompressible do not hold anymore.

45.1.1 Pressure

In *twoPhaseEulerFoam-2.3* the pressure is now a real physical pressure. In an incompressible simulation the absolute value of the pressure has no meaning, only pressure differences count. In a compressible model, the absolute value of the pressure has an effect, e.g. when using the `isothermalDiameter` diameter model to determine the diameter of the dispersed phase elements.

Thus, when migrating a simulation case from OpenFOAM-2.2 or lower to 2.3, check the pressure initial condition and the boundary conditions.

45.1.2 Temperature

As the new version of the solver uses thermo-physical models for the phases, the user is required to specify not only the thermo-physical properties of the phases, the user also has to provide initial and boundary conditions for the temperature of both phases. Thus, two additional fields are present – or need to be present – in the time directories, e.g. `T.air` and `T.water`.

45.2 Naming scheme

The overhaul of *twoPhaseEulerFoam* in version 2.3 aims for reuseability and generality of the solver code itself as well as of the case data. A general distinction of data concerning a single phase and data concerning the whole simulation case can be made.

Case data is named as usual (e.g. `fvSchemes`, `controlDict`, `g`, etc.). Data related to a specific phase is now stored in files with a filename that consists of two parts. The naming scheme follows the well known `FILENAME.EXTENSION` naming scheme. In this case `FILENAME` denotes the type of information and `EXTENSION` denotes the phase itself. This naming scheme is much more general than other naming schemes that are/were used in OpenFOAM (cf. `U1`, `U2` vs. `Uwater`, `Uair` vs. `U.air`, `U.water`).

Listing 279 shows the contents of the `0` and `constant` folders of the bubble column tutorial case. There we see the `FILENAME.EXTENSION` naming scheme applied. As each phase has a velocity and a temperature, we see two files for velocity and temperature. The volume fraction is an exception, as there are only two phase considered, the volume fraction of water is easily calculated, i.e. `alpha.water = 1.0 - alpha.air`. As the pressure is share by all phases, the pressure file has no file-extension. In the `constant` folder there is also data

that applies to one phase and data that applies to the simulation case. The files `g` and `phaseProperties` have no extensions because they contain no information specific to one phase. The thermophysical properties of the phases air and water are stored in the appropriate files.

The naming scheme that was introduced with *twoPhaseEulerFoam-2.3* is fit to create a material data library. The way the phases or the phase data is organized within the solver is now independent of the way the phase data is organized within the case.

```

user@host:~/OpenFOAM/OpenFOAM-2.3.x/tutorials/multiphase/twoPhaseEulerFoam/RAS/bubbleColumn$
ls 0 -1
alpha.air
alpha.air.org
epsilon.air
epsilon.water
k.air
k.water
nut.air
nut.water
p
T.air
Theta
T.water
U.air
U.water
user@host:~/OpenFOAM/OpenFOAM-2.3.x/tutorials/multiphase/twoPhaseEulerFoam/RAS/bubbleColumn$
ls constant -1
g
phaseProperties
polyMesh
thermophysicalProperties.air
thermophysicalProperties.water
turbulenceProperties.air
turbulenceProperties.water

```

Listing 279: Content of the `0` and `constant` folders of the bubble column tutorial case of *twoPhaseEulerFoam* in OpenFOAM-2.3.x

45.3 Solver capabilities

Not only the naming scheme is more general in version 2.3, also the solver itself is more generalized.

Compressibility all phases are treated as compressible. In the file `thermophysicalProperties` the behaviour of a phase can be specified.

Energy equation *twoPhaseEulerFoam* solves an energy equation for all phases. This can not be turned off.

Phase interaction has been extended. A great number of models specific for gas-liquid systems have been included.

Turbulence Turbulence is treated in a more general way. A number of turbulence models can be used in contrast to earlier versions of *twoPhaseEulerFoam* that had *kEpsilon* hard-coded.

45.4 Turbulence models

twoPhaseEulerFoam-2.3 uses a whole new class of turbulence models. As the governing equations of *twoPhaseEulerFoam* – namely the momentum equation – aren't phase intensive anymore, also the governing equations of the turbulence model are formulated in their general multi-phase form¹²⁹.

This limits the choice of turbulence models to a small number of multi-phase turbulence models. Listings 280 and 281 show the list of available turbulence models at the time of writing (May 2014).

Valid RASModel types:

6

¹²⁹<http://www.openfoam.org/version2.3.0/multiphase.php>


```
(
LaheyKEpsilon
continuousGasKEpsilon
kEpsilon
kineticTheory
mixtureKEpsilon
phasePressure
)
```

Listing 280: Valid RAS turbulence models of *twoPhaseEulerFoam*.

Valid LESModel types:

```
5
(
NicenoKEqn
Smagorinsky
SmagorinskyZhang
continuousGasKEqn
kEqn
)
```

Listing 281: Valid LES turbulence models of *twoPhaseEulerFoam*.

45.4.1 Naming scheme

One feature of the multi-phase turbulence model framework is that the additional turbulent viscosity is now named **nut**, regardless of whether a RAS or an LES model is used. This is possible, since both additional viscosities stem from the application of the Boussinesq-hypothesis.

In single-phase simulations an LES turbulence model works with the field **nuSgs**, whereas a RAS model uses **nut**. See textbooks on CFD for the theory behind RAS and LES turbulence models and the origin and meaning of ν_t and ν_{sgs} [35]. Sections 61 and 62 cover the incompressible $k - \epsilon$ model respectively some basics on LES turbulence models.

45.4.2 kEpsilon

Listing 282 shows the governing equations of the compressible multi-phase formulation of the $k - \epsilon$ model. The governing equations are largely equivalent to the compressible formulation of the single-phase $k - \epsilon$ model. The formulation deviates from the compressible single-phase formulation in two aspects. First, the convective term is corrected with the continuity error, see Lines 5 and 18. Furthermore, there is an additional source term on the RHS, see Lines 11 and 24.

```
1 tmp<fvScalarMatrix> epsEqn
2 (
3     fvm::ddt(alpha, rho, epsilon_)
4     + fvm::div(alphaRhoPhi, epsilon_)
5     - fvm::Sp(fvc::ddt(alpha, rho) + fvc::div(alphaRhoPhi), epsilon_)
6     - fvm::laplacian(alpha*rho*DepsilonEff(), epsilon_)
7     ==
8     C1_*alpha*rho*G*epsilon_/k_
9     - fvm::SuSp(((2.0/3.0)*C1_ + C3_)*alpha*rho*divU, epsilon_)
10    - fvm::Sp(C2_*alpha*rho*epsilon_/k_, epsilon_)
11    + epsilonSource()
12 );
13
14 tmp<fvScalarMatrix> kEqn
15 (
16     fvm::ddt(alpha, rho, k_)
17     + fvm::div(alphaRhoPhi, k_)
18     - fvm::Sp(fvc::ddt(alpha, rho) + fvc::div(alphaRhoPhi), k_)
19     - fvm::laplacian(alpha*rho*DkEff(), k_)
20     ==
21     alpha*rho*G
22     - fvm::SuSp((2.0/3.0)*alpha*rho*divU, k_)
23     - fvm::Sp(alpha*rho*epsilon_/k_, k_)
24     + kSource()
```

Listing 282: Governing equations of the `kEpsilon` turbulence model.

45.4.3 LaheyKEpsilon

The `LaheyKEpsilon` turbulence model is a derivation of the standard `kEpsilon` turbulence model, see Listing 283. The `LaheyKEpsilon` turbulence model is an extension of the standard $k - \epsilon$ model to account for the effect of the dispersed phase on the turbulence of the continuous phase. This effect is referred to as *bubble induced turbulence* (BIT).

There are essentially two ways to account for BIT. One follows the idea of Sato and Sekoguchi [55], there are additional viscosity models the effect of the increased turbulence caused by the wakes of the bubbles. The other approach is based on the work of Pflieger and Becker [51]. They included additional source terms in the transport equations for k and ϵ .

The Lahey model uses with its standard coefficients both approaches.

```

1  template<class BasicTurbulenceModel>
2  class LaheyKEpsilon
3  :
4      public kEpsilon<BasicTurbulenceModel>
5  {
6      /* class definition */
7  }
```

Listing 283: The first lines of the `LaheyKEpsilon` turbulence model definition.

Pitfall: the other phase

When using the `LaheyKEpsilon` model for one phase phase, the other phase is not allowed to be modelled as laminar. Listing 284 shows the method `phaseTransferCoefficient()` of the `LaheyKEpsilon` turbulence model. In Line 13 of Listing 284 we find the function call `gasTurbulence.k()` in the denominator. If `laminar` is chosen as turbulence model for the other phase, then the method `k()` of the `laminar` turbulence model is called. Listing 285 shows the definition of this method. We easily see, that the zero return value will cause problems in the `phaseTransferCoeff()` method of the `LaheyKEpsilon` turbulence model.

```

1  template<class BasicTurbulenceModel>
2  tmp<volScalarField>
3  LaheyKEpsilon<BasicTurbulenceModel>::phaseTransferCoeff() const
4  {
5      const volVectorField& U = this->U_;
6      const alphaField& alpha = this->alpha_;
7      const rhoField& rho = this->rho_;
8      const turbulenceModel& gasTurbulence = this->gasTurbulence();
9      return
10         (
11             max(alphaInversion_ - alpha, scalar(0))
12             *rho
13             *min(gasTurbulence.epsilon()/gasTurbulence.k(), 1.0/U.time().deltaT())
14         );
15 }
```

Listing 284: The method `phaseTransferCoeff()` of the `LaheyKEpsilon` turbulence model.

```

1  template<class BasicTurbulenceModel>
2  Foam::tmp<Foam::volScalarField>
3  Foam::laminar<BasicTurbulenceModel>::k() const
4  {
5      return tmp<volScalarField>
6      (
7          new volScalarField
8          (
9              IOobject
10             (
```

```

11         IOobject::groupName("k", this->U_.group()),
12         this->runTime_.timeName(),
13         this->mesh_,
14         IOobject::NO_READ,
15         IOobject::NO_WRITE
16     ),
17     this->mesh_,
18     dimensionedScalar("k", sqr(this->U_.dimensions()), 0.0)
19 );
20 };
21 }

```

Listing 285: The method `k()` of the `laminar` turbulence model.

Pitfall: the dispersed phase

It is not possible to assign the `LaheyKEpsilon` turbulence model to the dispersed phase, either to the dispersed phase alone or to both phases. In any case the attempt to do so results in a segmentation fault when first using the turbulence model at the initialisation of the simulation case. The reason for this is not entirely known to the author.

45.4.4 mixtureKEpsilon

Usage

The $k - \epsilon$ model is computed for the mixture, i.e. the transport equations are solved for using the mixture properties. Thus, the solution variables are named `km` and `epsilonM`, see Listing 286.

```

DILUPBiCG: Solving for epsilonM, Initial residual = 0.0114325, Final residual = 2.79117e-09,
           No Iterations 2
DILUPBiCG: Solving for km, Initial residual = 0.0078252, Final residual = 6.13173e-09, No
           Iterations 2

```

Listing 286: Solver output of *twoPhaseEulerFoam* using the `mixtureKEpsilon` turbulence model.

In order to use the mixture $k - \epsilon$ model, it needs to be specified in both `turbulenceProperties` files. Listing 287 shows the resulting error message when `mixtureKEpsilon` is specified for only one of the phases. As the turbulence model for the mixture applies to both phases, it needs to be specified for both phases.

```

--> FOAM FATAL ERROR:

lookup of turbulenceProperties.water from objectRegistry region0 successful
but it is not a mixtureKEpsilon, it is a LaheyKEpsilon

From function objectRegistry::lookupObject<Type>(const word&) const
in file /home/user/OpenFOAM/OpenFOAM-2.3.x/src/OpenFOAM/lnInclude/objectRegistryTemplates.
C at line 181.

FOAM aborting

```

Listing 287: Solver output of *twoPhaseEulerFoam* when the `mixtureKEpsilon` turbulence model is specified for only one of the two phases.

Theory

The governing equations of the mixture $k - \epsilon$ model can be found in the sources at `\$FOAM_SRC/TurbulenceModels/phaseCompressible/RAS/mixtureKEpsilon` and in [10]. The biggest difference between the equations stated in [10] and the code of `mixtureKEpsilon` can be found in the Lines 5 and 18 of Listing 288. There, the continuity equation of the mixture appears on the of the governing equations. This minor difference between the formulation of the equation can be resolved in two steps. First, we take a look on the first two terms of the

governing equations in [10] (local derivative and convective term), see Eqns. (85) to (88).

$$\frac{\partial \rho_m \epsilon_m}{\partial t} + \nabla \cdot (\rho_m \mathbf{u}_m \epsilon_m) + \dots \quad (85)$$

$$\rho_m \frac{\partial \epsilon_m}{\partial t} + \epsilon_m \frac{\partial \rho_m}{\partial t} + \epsilon_m \nabla \cdot (\rho_m \mathbf{u}_m) + \rho_m \mathbf{u}_m \cdot \nabla \epsilon_m + \dots \quad (86)$$

$$\rho_m \frac{\partial \epsilon_m}{\partial t} + \epsilon_m \underbrace{\left(\frac{\partial \rho_m}{\partial t} + \nabla \cdot (\rho_m \mathbf{u}_m) \right)}_{=0} + \rho_m \mathbf{u}_m \cdot \nabla \epsilon_m + \dots \quad (87)$$

$$\rho_m \frac{\partial \epsilon_m}{\partial t} + \rho_m \mathbf{u}_m \cdot \nabla \epsilon_m + \dots \quad (88)$$

In order to derive equations equivalent to the code implemented in OpenFOAM, we begin with Eq. (88) and use the product rule of differentiation, cf. Eqns. (85) and (86).

$$\rho_m \frac{\partial \epsilon_m}{\partial t} + \rho_m \mathbf{u}_m \cdot \nabla \epsilon_m + \dots \quad (88)$$

$$\frac{\partial \rho_m \epsilon_m}{\partial t} - \epsilon_m \frac{\partial \rho_m}{\partial t} + \nabla \cdot (\rho_m \mathbf{u}_m \epsilon_m) - \epsilon_m \nabla \cdot (\rho_m \mathbf{u}_m) + \dots \quad (89)$$

$$\frac{\partial \rho_m \epsilon_m}{\partial t} + \nabla \cdot (\rho_m \mathbf{u}_m \epsilon_m) - \epsilon_m \left(\frac{\partial \rho_m}{\partial t} + \nabla \cdot (\rho_m \mathbf{u}_m) \right) + \dots \quad (90)$$

Eq (90) is now equivalent to the first terms of the ϵ equation of Listing 288. The exact reason why this formulation was chosen is unknown to the author, a probable reason might be a better numerical behaviour.

```

1  tmp<fvScalarMatrix> epsEqn
2  (
3      fvm::ddt(rhom, epsilon)
4      + fvm::div(phim, epsilon)
5      - fvm::Sp(fvc::ddt(rhom) + fvc::div(phim), epsilon)
6      - fvm::laplacian(DepsilonEff(rhom*nutm), epsilon)
7      ==
8      C1_*rhom*Gm*epsilon/km
9      - fvm::SuSp(((2.0/3.0)*C1_)*rhom*divUm, epsilon)
10     - fvm::Sp(C2_*rhom*epsilon/km, epsilon)
11     + epsilonSource()
12 );
13
14 tmp<fvScalarMatrix> kmEqn
15 (
16     fvm::ddt(rhom, km)
17     + fvm::div(phim, km)
18     - fvm::Sp(fvc::ddt(rhom) + fvc::div(phim), km)
19     - fvm::laplacian(DkEff(rhom*nutm), km)
20     ==
21     rhom*Gm
22     - fvm::SuSp((2.0/3.0)*rhom*divUm, km)
23     - fvm::Sp(rhom*epsilon/km, km)
24     + kSource()
25 );

```

Listing 288: Governing equations of the `mixtureKEpsilon` turbulence model.

The basic relations between the turbulent quantities of the mixture and the turbulence quantities of the individual phases are based on the turbulence response coefficient C_t , which is the ratio between the r.m.s. values of the velocity fluctuations of the dispersed and the continuous phase [10].

$$C_t = \frac{U'_d}{U'_c} \quad (91)$$

with this coefficient, we can now express the following relations, which we can find in the file `mixtureKEpsilon.C`

$$\rho_m = \alpha_c \rho_c + \alpha_d \rho_d \quad (92)$$

$$C_c^2 = \frac{\rho_m}{\alpha_c \rho_c + C_t^2 \alpha_d \rho_d} \quad (93)$$

$$k_c = C_c^2 k_m \quad (94)$$

$$k_d = C - t^2 k_c \quad (95)$$

$$\epsilon_c = C_c^2 \epsilon_m \quad (96)$$

$$\epsilon_d = C_t^2 \epsilon_c \quad (97)$$

$$\nu_t = C_\mu \frac{k_m^2}{\epsilon_m} \quad (98)$$

$$\nu_{c,eff} = \nu_c + \nu_t \quad (99)$$

$$\nu_{d,eff} = \nu_d + C_t^2 \frac{\nu_c}{\nu_d} \nu_t \quad (100)$$

What remains to clarify is how the turbulence response coefficient C_t is determined. OpenFOAM implements the model proposed by Issa [34] and validated by Hill [26] [54, 10]. Furthermore, the turbulence response coefficient is modified to account for the influence of the dispersed phase's volume fraction α_d , see e.g. [54, 10].

$$C_{t,0} = \frac{3 + \beta}{1 + \beta + 2\rho_d/\rho_c} \quad (101)$$

$$\beta = \frac{2A_d L_e^2}{\rho_c \nu_c Re_t} \quad (102)$$

$$Re_t = \frac{U'_c L_e}{\nu_c} \quad (103)$$

$$L_e = C_\mu \frac{k_c^{3/2}}{\epsilon_c} \quad (104)$$

$$U'_c = \sqrt{\frac{2k_c}{3}} \quad (105)$$

$$C_t(\alpha_d) = 1 + (C_{t,0} - 1)e^{-f(\alpha_d)} \quad (106)$$

$$f(\alpha_d) = 180\alpha_d - 4.71 \cdot 10^3 \alpha_d^2 + 4.26 \cdot 10^4 \alpha_d^3 \quad (107)$$

45.4.5 SmagorinskyZhang LES

The SmagorinskyZhang turbulence model is a zero equation LES turbulence model. This turbulence model corrects the turbulent viscosity by a contribution due to bubble induced turbulence (BIT) [68]. As there is no use of turbulent quantities of the other phase, there is no limitation in turbulence model choice for the other phase.

45.4.6 NicenoKEqn LES

The NicenoKEqn turbulence model is an LES model which solves a transport equation for the unresolved turbulent kinetic energy k_{SGS} . Similar to the model of Lahey, the model of Niceno is able to account for effects of bubble induced turbulence. This is done through an additional viscosity and/or an additional source term in the transport equation for the turbulent kinetic energy.

Similar to the Lahey model, the Niceno model accesses turbulent quantities of the other phase, for this reason it is not possible to model the other phase as a laminar phase. As we can see in Line 13 of Listing 289, the Niceno model takes the square root of the gas phase's turbulent kinetic energy, when computing the phase transfer coefficient. The method `k()` of the laminar turbulence models returns zero for the turbulent kinetic energy. This triggers a floating point exception (FPE).

```

1  template<class BasicTurbulenceModel>
2  tmp<volScalarField>
3  NicenoKEqn<BasicTurbulenceModel>::phaseTransferCoeff() const

```

```

4 {
5     const volVectorField& U = this->U_;
6     const alphaField& alpha = this->alpha_;
7     const rhoField& rho = this->rho_;
8     const turbulenceModel& gasTurbulence = this->gasTurbulence();
9     return
10    (
11        max(alphaInversion_ - alpha, scalar(0))
12        *rho
13        *min(this->Ce_*sqrt(gasTurbulence.k())/this->delta(), 1.0/U.time().deltaT())
14    );
15 }

```

Listing 289: The method `phaseTransferCoeff()` of the Niceno turbulence model.

45.4.7 Pitfall: phase inversion

Phase inversion is the situation when the volume fraction of the continuous phase vanishes in some regions. As almost all terms of the governing equations are weighted with the volume fraction `alpha`, a vanishing volume fraction can lead to serious numerical problems.

Solution via model choice

The following example demonstrates the problems which may be faced when dealing with phase inversion. An air-water bubble column is modelled including some of the air above the water surface. Figure 108 shows the air volume fraction within the bubble column.

When `mixtureKEpsilon` is selected as turbulence model, the volume fraction is not included in the governing equation, so phase inversion poses no big problem, see Listing 288 or Eq. (90).

When `kEpsilon` is selected for the liquid phase, the volume fraction in the governing equations is the volume fraction of the liquid phase. This volume fraction vanishes above the water surface. Thus, in parts of the domain the solution of the governing equations faces numerical problems. The governing equations can still be solved in this case, but preconditioning the resulting matrix equation fails. Preconditioning is a step that is intended to improve the iterative solution of the resulting matrix equation. In the case of the `kEpsilon` turbulence model for the liquid phase, the only way to avoid crashing the simulation is to use a CG-solver with no preconditioning. The GAMG-solver and the smooth solver fail completely.

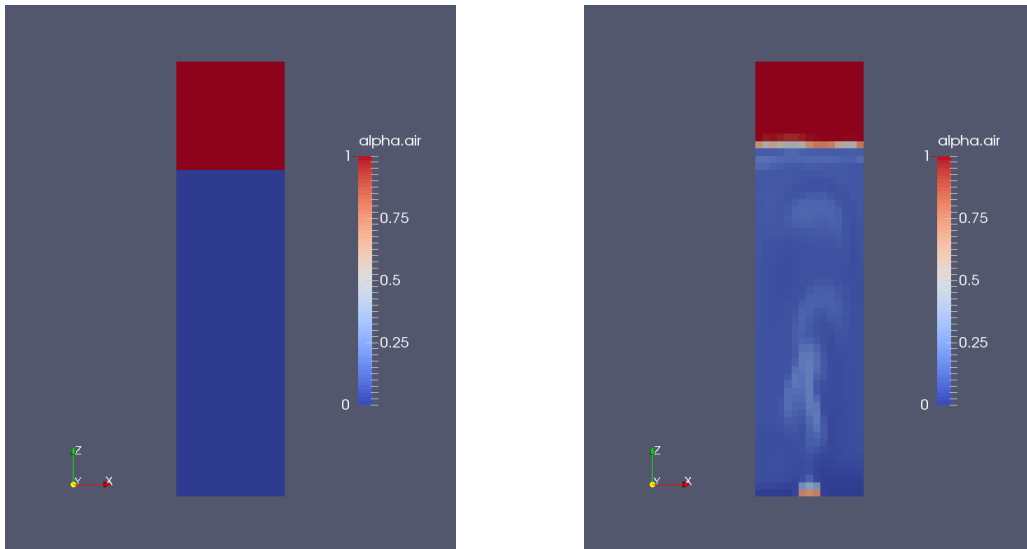


Figure 108: Air volume fraction of the bubble column. Initial field (left) and solution at $t = 10$ s (right).

Table 6 lists possible model choices for two-phase simulations including phase-inversion.

Liquid	Gas	constraints / remarks
kEpsilon	laminar	solver: PBiCG, preconditioner: none
mixtureKEpsilon	mixtureKEpsilon	
Lahey	continuousKEqn	
SmagorinskyZhang	laminar	no issue, since SmagorinskyZhang is a zero-equation model
Niceno	laminar	solver: PBiCG, preconditioner: none
Niceno	continuousKEqn	

Table 6: Turbulence model combinations for phase-inversion cases.

Solution via phaseLimitStabilization

With OpenFOAM-6¹³⁰ an fvOption source was introduced to specifically, numerically stabilize transport equations when the phase fraction falls below a certain threshold. This source adds an implicit source term to a transport equation for all cells in which the phase fraction falls below a certain limit. This fvOption is further discussed in Section 39.3.2.

In our example, we can use the phaseLimitStabilization fvOption to stabilize the transport equations for the turbulence fields. This way, we can also eliminate, or alleviate, the problems caused by phase inversion, as in all regions with a vanishing phase fraction, the phaseLimitStabilization source prevents the transport equations from becoming ill-posed.

45.5 Energy equation

In OpenFOAM-2.3 the *twoPhaseEulerFoam* solver incorporates the functionality of *compressibleTwoPhaseEulerFoam*¹³¹. Accounting for compressibility necessitates the solution of the energy equation. The solving of the energy equation requires the specification of additional discretisation schemes and a solver in **fvSchemes** and **fvSolution**. Depending on the simulation parameters the energy equation is solved in terms of the enthalpy **h** or internal energy **e**.

The energy equation is formulated in a generic form in terms of **he**. The actual decision to solve for **h** or **e** is made at run-time after the thermophysical properties of the two phases have been read.

Besides the internal energy or enthalpy the energy equation involves also the kinetic energy **K**, which is in fact a specific kinetic energy. Listing 290 shows how this kinetic energy is computed. This source code translates into the following mathematical relation.

$$K_i = \frac{1}{2} |U_i|^2 \quad (108)$$

```
Info<< "Creating field kinetic energy K\n" << endl;
volScalarField K1(IObject::groupName("K", phase1.name()), 0.5*magSqr(U1));
volScalarField K2(IObject::groupName("K", phase2.name()), 0.5*magSqr(U2));
```

Listing 290: Definition of the kinetic energy field in the file **createFields.H** of *twoPhaseEulerFoam*.

The solution of the energy equation can not be deactivated. Even if thermophysical parameters are chosen to represent incompressible phases, the energy equation will be solved each time step.

45.5.1 Governing equations

Listing 291 shows the energy equation for one phase. In Line 3 we see the local derivative and the convection term of the generic internal energy/ enthalpy **he**. In Line 5 is the local derivative and the convection term of the specific kinetic energy **K**.

In the Lines 4 and 6 we see a correction for the continuity error. See Section 59.3 for a detailed discussion.

From Lines 8 to 10 we see the term regarding the mechanical work done. Here we see a conditional expression depending whether the equation is solved for internal energy or enthalpy. All other terms in the equation are formulated generically. Besides the use of the abstract **he**, which is internal energy or enthalpy, the use of the

¹³⁰<https://openfoam.org/release/6/>

¹³¹<http://www.openfoam.org/version2.3.0/multiphase.php>

variable `Cpv` is also a characteristic of this generic formulation. This variable stands for either the heat capacity at constant pressure or the heat capacity at constant volume.

Lines 12 to 17 contain the diffusive heat flux. Line 19 represents the heat flux between the two phases. Line 22 contains possible heat sources.

Lines 20 and 21 can be considered a numerical trick. If we ignore the `fvm::Sp()` for a while and add the terms of the two lines, we see that they add up to zero. Adding zero is mathematically allowed. If we do not ignore the `fvm::Sp()`, we need to find out, what is happening. `fvm::Sp()` is an implicit source term, i.e. the contribution of this term goes into the system matrix of the resulting linear equation system. An implicit source term not only contributes to the system matrix, these terms go into the diagonal entries of the system matrix. When solving linear equation systems iteratively, it is preferable to work on a *diagonally dominant* system matrix [35]. Exactly, this is achieved by the Lines 20 and 21. The term in Line 21 adds to the diagonal of the system matrix, whereas the term of Line 20 adds to the right hand side of the ensuing linear equation system. As both sides of the equation have been equally treated, nothing was done wrong mathematically. However, as *diagonal dominance* is numerically a good thing, the convergence behaviour was probably improved.

```

1  fvScalarMatrix he1Eqn
2  (
3      fvm::ddt(alpha1, rho1, he1) + fvm::div(alphaRhoPhi1, he1)
4      - fvm::Sp(contErr1, he1)
5      + fvc::ddt(alpha1, rho1, K1) + fvc::div(alphaRhoPhi1, K1)
6      - contErr1*K1
7      + (
8          he1.name() == thermo1.phasePropertyName("e")
9          ? fvc::ddt(alpha1)*p + fvc::div(alphaPhi1, p)
10         : -alpha1*dPdt
11     )
12      - fvm::laplacian
13      (
14          fvc::interpolate(alpha1)
15          *fvc::interpolate(thermo1.alphaEff(phase1.turbulence().mut())),
16          he1
17      )
18      ==
19      heatTransferCoeff*(thermo2.T() - thermo1.T())
20      + heatTransferCoeff*he1/Cpv1
21      - fvm::Sp(heatTransferCoeff/Cpv1, he1)
22      + fvOptions(alpha1, rho1, he1)
23  );

```

Listing 291: Energy equation in the file `EEqns.H` of *twoPhaseEulerFoam*.

45.6 Momentum equation

Due to the changes on the modelling side and some restructuring, the momentum equation has a different form compared to previous versions of this solver.

The most general form of the momentum conservation equation for two-phase flow is as follows¹³²

$$\frac{\partial \alpha_q \rho_q \mathbf{u}_q}{\partial t} + \nabla \cdot (\alpha_q \rho_q \mathbf{u}_q \mathbf{u}_q) - \nabla \cdot \tau_q = \sum_i \mathbf{F}_{q,i} + \sum_i K_{pq,i} (\mathbf{u}_p - \mathbf{u}_q) \quad (109)$$

with

$$\begin{aligned} K_{pq,i} &= -K_{qp,i} \\ K_{qq,i} &= 0 \end{aligned}$$

45.6.1 Units

Now we shall take a short look on the units of this equation. Each term of the equation has to have the same unit. We take the local derivative to determine the unit of all terms in this equation.

¹³²The phase q is the considered phase and phase p denotes the other phase.

$$\left[\frac{\partial \alpha_q \rho_q \mathbf{u}_q}{\partial t} \right] = \frac{1}{s} \frac{\text{kg}}{\text{m}^3} \frac{\text{m}}{s} = \frac{1}{\text{m}^3} \underbrace{\frac{\text{kg m}}{\text{s}^2}}_{\text{N}} = \frac{\text{N}}{\text{m}^3} \quad (110)$$

We see that all terms of the momentum equation have the unit of a force density. On the RHS of the momentum equation we have two kinds of source terms.

The first kind of source terms – $\mathbf{F}_i$ – can be referred to as body forces, e.g. the gravitational force. This is consistent with our observation, that this terms have the unit of a force density.

$$[\mathbf{F}_i] \stackrel{!}{=} \frac{\text{N}}{\text{m}^3} = \frac{\text{kg}}{\text{m}^2 \text{s}^2} \quad (111)$$

The second kind of source terms – $K_{pq,i} (\mathbf{u}_p - \mathbf{u}_q)$ – are phase interaction terms. This terms are the product of a coefficient $K_{pq,i}$ with the relative velocity $\mathbf{u}_R = \mathbf{u}_p - \mathbf{u}_q$. Such a phase interaction term might be due to drag. Now we determine the unit of the interphase momentum exchange coefficient $K_{pq,i}$.

$$[K_{pq,i} (\mathbf{u}_p - \mathbf{u}_q)] \stackrel{!}{=} \frac{\text{N}}{\text{m}^3} = \frac{1}{s} \frac{\text{kg}}{\text{m}^3} \frac{\text{m}}{s} \quad (112)$$

$$[K_{pq,i}] = \frac{\text{kg}}{\text{m}^3 \text{s}} \quad (113)$$

45.6.2 Implemented equations

Listing 292 shows one of the momentum conservation equations. On Line 3 we see the local derivative and the convective term. The origin of the term in Line 4 is explained in 59.3. On Line 5 we see a term stemming from the MRF approach. On Line 6 is the momentum diffusion.

On the RHS there are a number of force terms. Although, they are named ***Force**, they are in fact force density terms. On Line we see a part of the drag force. The force due to gravity and the other part of the drag are considered in the pressure equation [54].

```

1      U1Eqn =
2      (
3          fvm::ddt(alpha1, rho1, U1) + fvm::div(alphaRhoPhi1, U1)
4          - fvm::Sp(contErr1, U1)
5          + mrfZones(alpha1*rho1 + virtualMassCoeff, U1)
6          + phase1.turbulence().divDevRhoReff(U1)
7      ==
8          - liftForce
9          - wallLubricationForce
10         - turbulentDispersionForce
11         - virtualMassCoeff
12         *(
13             fvm::ddt(U1)
14             + fvm::div(phi1, U1)
15             - fvm::Sp(fvc::div(phi1), U1)
16             - DDtU2
17         )
18         + fvOptions(alpha1, rho1, U1)
19     );
20     U1Eqn.relax();
21     U1Eqn += fvm::Sp(dragCoeff, U1);
22     fvOptions.constrain(U1Eqn);

```

Listing 292: The code of the momentum conservation equation of phase 1 of *twoPhaseEulerFoam* in **UEqns.H**

The interfacial momentum exchange terms are computed prior to the construction of the momentum equation. Listing 293 shows the relevant lines of the file **UEqns.H**. We see that the momentum exchange terms are provided by some methods. We know that the variable **fluid** is of the type **twoPhaseSystem**. Thus, the methods called to compute the momentum exchange terms are methods of the class **twoPhaseSystem**, see Section 35.2.1.

```

1  volScalarField  dragCoeff(fluid.dragCoeff());
2
3      volScalarField  virtualMassCoeff(fluid.virtualMassCoeff());
4      volVectorField  liftForce(fluid.liftForce());
5      volVectorField  wallLubricationForce(fluid.wallLubricationForce());
6      volVectorField  turbulentDispersionForce(fluid.turbulentDispersionForce());

```

Listing 293: The definition of the interfacial momentum exchange force terms of the momentum conservation equations of *twoPhaseEulerFoam* in *UEqns.H*

45.7 Interfacial interaction

45.7.1 Blending

The interfacial momentum exchange models need to work over the whole range of flow situations. These range from $\alpha_1 = 0$ to $\alpha_1 = 1$. In order to well-posedness of the governing equations special care needs to be taken for the case of phase inversion.

There are three options for blending available: none, linear and hyperbolic.

```

// create x

if (model_.valid())
{
    x() += model_->K()*(f1() - f2());
}

if (model1In2_.valid())
{
    x() += model1In2_->K()*(1 - f1);
}

if (model2In1_.valid())
{
    x() += model2In1_->K()*f2;
}

// other code

return x;

```

Listing 294: The application of blending; part of the method *K()* in *BlendedInterfacialModel.C*

No Blending

The blending model **none**, which is defined in the files *noBlending.H* and *noBlending.C*, is quite instructive. This blending model, which is essentially a non-model, returns the blending factors *f1* and *f2* as it is demanded by the base class of all blending models.

As there is no blending with the **none** blending model, the user needs to specify which phase is the continuous phase. In *twoPhaseEulerFoam-2.3* there is no implicit assumption on which phase is the dispersed and which is continuous. Listing 295 shows how the **none** blending model is selected. There we also see the explicit specification of the continuous phase.

```

blending
{
    default
    {
        type          none;
        continuousPhase water;
    }
}

```

Listing 295: Choosing not to use blending as the blending method

Now, we have a look on the blending factors returned by the `none` model. Listing 296 shows the definition of the methods `f1()` and `f2()`. These methods return a newly created temporary scalar field (`volScalarField`) that is in turn created from a constant expression.

In the case of `f1()`, the constant expression is `phase2.name() != continuousPhase_` which returns a boolean value. In the case of `f2()` the corresponding expression is `phase1.name() == continuousPhase_`, which also returns a boolean value. Here, we enter the realm of implicit type conversions¹³³. Implicit type conversions are part of the language's standard. Thus, if we look up the working draft of the C++11 standard, we find the following sentence in the section on *Integral promotions*:

A prvalue of type `bool` can be converted to a prvalue of type `int`, with `false` becoming zero and `true` becoming one.

Thus, we find that the blending factors returned by `none` are of the values zero or one, which is the set of values we would expect in this case. If the boolean expressions yield the correct factors can be tried out with a simple *pen-and-paper test*. Choose a continuous phase (i.e. `phase2` is the continuous phase) and evaluate all expressions (i.e. determine the values of `f1` and `f2`, and apply these values on the expressions found in Listing 294.).

```

Foam::tmp<Foam::volScalarField> Foam::blendingMethods::noBlending::f1
(
    const phaseModel& phase1, const phaseModel& phase2
) const
{
    const fvMesh& mesh(phase1.mesh());

    return
        tmp<volScalarField>
        (
            new volScalarField
            (
                IOobject( /* arguments removed */,
                    mesh,
                    dimensionedScalar
                    (
                        "f",
                        dimless,
                        phase2.name() != continuousPhase_
                    )
                )
            );
        }
Foam::tmp<Foam::volScalarField> Foam::blendingMethods::noBlending::f2
(
    const phaseModel& phase1, const phaseModel& phase2
) const
{
    const fvMesh& mesh(phase1.mesh());

    return
        tmp<volScalarField>
        (
            new volScalarField
            (
                IOobject( /* arguments removed */,
                    mesh,
                    dimensionedScalar
                    (
                        "f",
                        dimless,
                        phase1.name() == continuousPhase_
                    )
                )
            );
        }

```

Listing 296: Computing the blending factors. The arguments of the constructor of the `IOobject` class have been removed to save space.

¹³³See e.g. http://en.cppreference.com/w/cpp/language/implicit_cast

Linear

As we saw from the `none` model, the blending factors `f1` and `f2` have two extreme values, i.e. zero and one. The model name `linear` suggests that this models yields a linear variation between these two limiting values.

The `linear` blending model was two model parameters, shown in Listing 297. These represent the limits up to which a phase can be considered to be fully dispersed, i.e. a clear distinction between dispersed phase and continuous phase is possible. The second parameter is the limit up to which the phases can be considered partly dispersed. These two limits are necessary, as the solver is intended to handle phase inversion, i.e. situations in which one phase is the dispersed phase in only parts of the domain.

The definition of the blending factor `f1` is shown in Listing 298. We limit the discussion on `f1`, as the other blending factor is defined analogously. The interested reader is encouraged to analyse `f2`. The code of Listing 298 can be translated into equation (114).

$$f_1(\alpha) = \begin{cases} 1 & \text{if } \alpha \leq \text{maxFullyDispersedAlpha} \\ \frac{\alpha - \text{maxFullyDispersedAlpha}}{\text{maxPartlyDispersedAlpha} - \text{maxFullyDispersedAlpha}} & \text{if } \alpha \leq \text{maxPartlyDispersedAlpha} \\ 0 & \text{if } \alpha > \text{maxPartlyDispersedAlpha} \end{cases} \quad (114)$$

```

//- Maximum fraction of phases which can be considered fully dispersed
HashTable<dimensionedScalar, word, word::hash>
    maxFullyDispersedAlpha_;

//- Maximum fraction of phases which can be considered partly dispersed
HashTable<dimensionedScalar, word, word::hash>
    maxPartlyDispersedAlpha_;

```

Listing 297: Model parameters of the `linear` blending model; declaration in the file `linear.H`

```

Foam::tmp<Foam::volScalarField> Foam::blendingMethods::linear::f1
(
    const phaseModel& phase1, const phaseModel& phase2
) const
{
    const dimensionedScalar
        maxFullAlpha(maxFullyDispersedAlpha_[phase1.name()]);
    const dimensionedScalar
        maxPartAlpha(maxPartlyDispersedAlpha_[phase1.name()]);

    return
        min
        (
            max
            (
                (phase1 - maxFullAlpha)
                / (maxPartAlpha - maxFullAlpha + SMALL),
                scalar(0.0)
            ),
            scalar(1.0)
        );
}

```

Listing 298: Computing the linear blending factor `f1` in the file `linear.C`

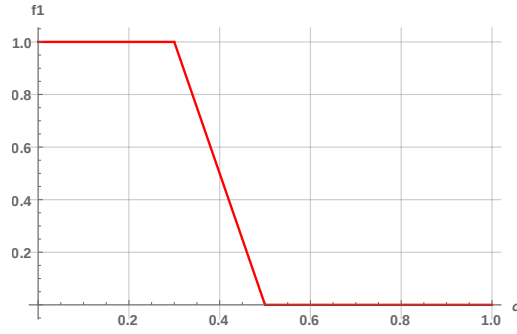


Figure 109: The value of `f1` over α ; model parameters are set to `maxFullAlpha` = 0.3 and `maxPartAlpha` = 0.5; these settings are taken from the *bubble column tutorial* case of *twoPhaseEulerFoam*.

Hyperbolic

The **hyperbolic** blending model offers a continuous function for the blending factor for the whole range of the dispersed phase's volume fraction, see Figure 110. Again, we analyse only the definition of `f1` and leave the reader the opportunity to follow the argument made, with the definition of `f2`.

The **hyperbolic** blending model needs in total three model parameters. The parameter `transitionAlphaScale` controls how steep the transition between 0 and 1 is. The other two parameters are `maxDispersedAlpha` for each phase. At this parameter the blending function (115) has the value $1/2$.

$$f_1(\alpha) = \frac{1}{2} \left(1 + \tanh \left(\frac{4(\alpha - \text{maxDispersedAlpha})}{\text{transitionAlphaScale}} \right) \right) \quad (115)$$

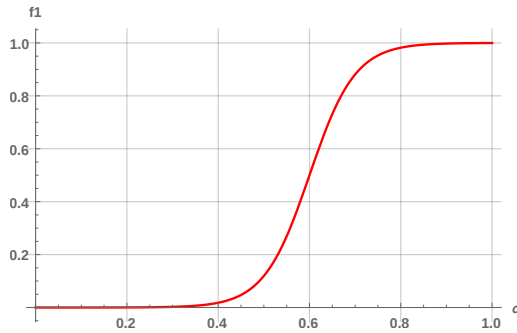


Figure 110: The value of `f1` over α ; model parameters are set to `maxDispersedAlpha` = 0.6 and `transitionAlphaScale` = 0.4;

45.8 Interfacial momentum exchange

45.8.1 Drag

Units

From viewing the governing equations we saw, that the drag term consists of a coefficient and the relative velocity between the phases.

$$\mathbf{F}_{drag} = K_{pq,drag} (\mathbf{u}_p - \mathbf{u}_q) \quad (116)$$

We find the same structure in the terms of the implemented equations. The Listing below shows one part of the drag term – as the drag term consists of the coefficient and a velocity difference, we can split the term up into two contributing parts.

```
U1Eqn += fvm::Sp(dragCoeff, U1);
```

As we know from our considerations about the units of the terms of the momentum equation, the drag force contribution in general needs to have the unit of a force density. Thus, we determined the unit of the coefficient, see Eqn. (113).

$$[\text{dragCoeff}] \stackrel{!}{=} \frac{\text{kg}}{\text{m}^3 \text{s}} \quad (117)$$

By having a close look on the base class for the drag models, we can check the unit of the coefficient. The base class of the drag model has a static data member that carries the information about the unit of the provided coefficient. In fact, all interfacial momentum exchange models have such a member. In the header file of the base class for the drag models, a constant static member¹³⁴ `dimK` is declared.

```
//- Coefficient dimensions
static const dimensionSet dimK;
```

In the implementation file, the static data member is initialised to the appropriate value. In Section 9 we reviewed OpenFOAMs feature to provide physical units. There we can see, that the order of units in a `dimensionSet` is [kg m s K mol].

```
const Foam::dimensionSet Foam::dragModel::dimK(1, -3, -1, 0, 0);
```

Thus, we see, that the drag force coefficient has indeed the unit we derived from our earlier considerations.

Returning the output

Other than the drag models of prior versions of *twoPhaseEulerFoam* (version 2.2 and below), the drag models in *twoPhaseEulerFoam-2.3* return the product of drag coefficient C_D and the Reynolds number Re . Consequently, the method returning the output of the individual drag models is named `CdRe()`.

The drag model itself, i.e. the base class returns the drag force coefficient K . This drag force coefficient is provided by the method `K()` which is a method of the base class `dragModel`. The base class also has a pure virtual method named `CdRe()`. Pure virtual means that derived classes need to implement this method and that we are unable to create an instance of the base class itself. We only can create instances of one of the derived classes. As a derived class must implement all pure virtual methods, we are guaranteed that these methods actually exist. The Listings 299 and 300 show the relevant parts of code of the class `dragModel`. The method `K()` calls the method `CdRe()`, see Line 5 of Listing 300.

```
1  //- Drag coefficient
2  virtual tmp<volScalarField> CdRe() const = 0;
3
4  //- The drag function K used in the momentum equation
5  // ddt(alpha1*rho1*U1) + ... = ... K*(U1-U2)
6  // ddt(alpha2*rho2*U2) + ... = ... K*(U2-U1)
7  virtual tmp<volScalarField> K() const;
```

Listing 299: The declaration of the methods `K()` and `CdRe()` in `dragModel.H`

```
1  Foam::tmp<Foam::volScalarField> Foam::dragModel::K() const
2  {
3      return
4          0.75
5          *CdRe()
6          *max(pair_.dispersed(), residualAlpha_)
7          *swarmCorrection_->Cs()
8          *pair_.continuous().rho()
9          *pair_.continuous().nu()
```

¹³⁴A static data member of a class exists only once for all instances of this class, i.e. regardless of how many actual objects of this class exist, the data member exists only once. This makes perfect sense for common properties such as the unit of the coefficient, which is the same for all drag models.

```

10     /sqr(pair_.dispersed().d());
11 }

```

Listing 300: The definition of the method `K()` in `dragModel.C`

If we translate Listing 300 into math we yield

$$K = \frac{3}{4} C_D Re \alpha C_S \frac{\rho_C \nu_C}{d_B^2} \quad (118)$$

Now, we insert the definition of the bubble Reynolds number

$$K = \frac{3}{4} C_D \frac{d_B U_R}{\nu_C} \alpha C_S \frac{\rho_C \nu_C}{d_B^2} \quad (119)$$

$$K = \frac{3}{4} \alpha C_S C_D \frac{\rho_C}{d_B} U_R \quad (120)$$

If we now take a look on the units

$$[K] = \left[\frac{\rho_C}{d_B} U_R \right] = \frac{\text{kg}}{\text{m}^3} \frac{1}{\text{m}} \frac{\text{m}}{\text{s}} = \frac{\text{kg}}{\text{m}^3 \text{s}} \quad (121)$$

Again, we find the proper physical unit for the drag force coefficient.

Here we show the definition of the method `CdRe()` from the class `SchillerNaumann` as an example since the Schiller Naumann drag model is well known.

```

1 Foam::tmp<Foam::volScalarField> Foam::dragModels::SchillerNaumann::CdRe() const
2 {
3     volScalarField Re(pair_.Re());
4
5     return
6         neg(Re - 1000)*24.0*(1.0 + 0.15*pow(Re, 0.687))
7         + pos(Re - 1000)*0.44*max(Re, residualRe_);
8 }

```

Listing 301: The relevant lines of code in `SchillerNaumann.C`

Swarm correction

The drag models offer swarm correction of the drag force, since it is observed that swarms of bubbles behave different from single bubbles. At the time of writing (September 2014) there are two choices.

noSwarm This model simply returns unity when `swarmCorrection_->Cs()` is called.

TomiyamaSwarm This model computes the swarm correction factor according to [61].

The Tomiyama swarm correction factor depends on the bubble volume fraction α and a model parameter l .

$$C_{S,Tomiyama} = (1 - \alpha)^{3-2l} \quad (122)$$

Both swarm correction models are derived from an abstract base class `swarmCorrection`. Thus the framework is ready for future extension of model choice.

45.8.2 Lift

The lift force on a dispersed phase element (DPE) is defined as

$$F_L = C_L \alpha \rho_C (\mathbf{U}_R \times (\nabla \times \mathbf{U})) \quad (123)$$

with

C_L	lift force coefficient
α	volume fraction of the dispersed phase
ρ_C	density of the continuous phase
$\mathbf{U}_R$	relative velocity between the phases
$\mathbf{U}$	mixture velocity

Units

In contrast to the drag model, the lift model provides the actual force term for the governing equations. The base class of the lift models declares a static constant data member `dimF` for storing the unit of the force term computed by the lift model.

```
// - Force dimensions
static const dimensionSet dimF;
```

In the implementation file `liftModel.C` the static data member is initialized and it has indeed the unit of a force density. Note: the order of units in a `dimensionSet` is `[kg m s K mol]`.

$$[\mathbf{F}_i] \stackrel{!}{=} \frac{\text{N}}{\text{m}^3} = \frac{\text{kg}}{\text{m}^2 \text{s}^2} \quad (111)$$

```
const Foam::dimensionSet Foam::liftModel::dimF(1, -2, -2, 0, 0);
```

Returning the output

The general computation of the lift force is done – similar to the drag models – within the method `F()` of the base class. The base class calls the method `C1()` of the concrete lift model for the lift force coefficient. This is similar to the method `K()` of the drag model base class calling the method `CdRe()` of the concrete drag model classes.

The method `F()` of the base class returns the force density field due to the lift force.

```
1 // - Lift coefficient
2 virtual tmp<volScalarField> C1() const = 0;
3
4 // - Lift force
5 virtual tmp<volVectorField> F() const;
```

Listing 302: The declaration of the methods `F()` and `C1()` in `liftModel.H`

```
1 Foam::tmp<Foam::volVectorField> Foam::liftModel::F() const
2 {
3     return
4         C1()
5         *pair_.dispersed()
6         *pair_.continuous().rho()
7         *(
8             pair_.Ur() ^ fvc::curl(pair_.continuous().U())
9         );
10 }
```

Listing 303: The definition of the method `F()` in `liftModel.H`

The actual lift force coefficient is provided by the concrete lift force model. Again, analogue to the drag model classes, the base class for the lift models declares the pure virtual method `C1()`. This means, every lift model derived from the base class has to implement `C1()` and we are not able to create an instance of the base class itself. Thus, the existence of the method `C1()` is guaranteed. The implementation of `C1()` is the remaining degree of freedom for the individual lift force models.

There are several choices available to the user:

noLift this model returns a zero field when either `F()` or `C1()` is called. This class overwrites the method `F()` which is inherited from the base class with its own implementation. Thus, when `F()` is called, the implementation of the class `noLift` is called, i.e. `noLift::F()`. All other lift force models do not implement `F()`, thus, `liftModel::F()` is called.

constantCoefficient this model is the easiest implementation of a lift force model. The constant lift force coefficient C_L is provided by the user. `C1()` simply returns this value in the form of the appropriate data type, i.e. the coefficient provided by the user is a dimensionless number (declared as `const dimensionedScalar C1_;`), however, the method `C1()` returns a `volScalarField`.

lift force model X there are several models available that compute the lift force coefficient from flow properties.

45.8.3 Virtual mass

The class structure for the virtual mass models follow the example of the drag and lift models. There is an abstract base class providing a method `F()` for the force term F_{VM} due to virtual mass. The force term due to virtual mass is defined as

$$F_{VM} = C_{VM} \alpha \rho_C \quad (124)$$

with

C_{VM}	virtual mass coefficient
α	volume fraction of the dispersed phase
ρ_C	density of the continuous phase

The derived classes provide the virtual mass coefficient C_{VM} via the method `Cvm()`. The user has the choice between:

noVirtualMass this class returns zero when `F()` is called. This model overwrites the method `F()` with its own implementation returning a zero field. All other classes make use of the base classes implementation of `F()` which all derived classes inherited. The method `Cvm()` also returns a zero field.

```

Foam::tmp<Foam::volScalarField>
Foam::virtualMassModels::noVirtualMass::K() const
{
    return Cvm()*dimensionedScalar("zero", dimDensity, 0);
}

```

constantVirtualMassCoefficient this class computes the contribution due to virtual mass based on a constant virtual mass coefficient C_{VM} which is provided by the user.

Lamb this model computes the virtual mass coefficient C_{VM} depending on the aspect ratio of the dispersed phase elements. With the help of aspect ratio models a particle shape different from spheres and even shape variation can be modelled within some limits.

45.8.4 Aspect ratio models

When dealing with non-spherical bubbles or particles, the shape has to be considered in the interfacial momentum exchange models. One way of dealing with this situation is to formulate those models to incorporate the aspect ratio of the dispersed phase elements.

Here, the aspect ratio models come into play. These compute the aspect ratio of the dispersed phase elements depending on material and possibly flow properties. However, the influence of shape can also be considered using other approaches.

The aspect ratio is used in the `TomiyamaAnalytic` drag model and the `Lamb` virtual mass model. The interested reader can find this out by invoking the following commands.

```
cd $FOAM_APP/solvers/multiphase/twoPhaseEulerFoam/interfacialModels
find -name *.C | xargs grep 'pair_.E()'
```

The second command is a combination of a *find* command and a *grep* command. *find* finds all files with the file extension `.C` and *grep* searches this files for the pattern `pair_.E()`. This pattern is the function call which returns the aspect ratio E of a phase pair.

45.8.5 Wall lubrication

The wall lubrication force pushes bubbles away from the walls. The class structure is similar to the aforementioned models. There is an abstract base class and derived classes implementing a specific model. The base class declares the pure virtual method `F()` which returns the force term due to wall lubrication. The derived class have to implement this method.

There is a derived class named `noWallLubrication` which simply implements the method `F()` in way to return a zero field. There are also three models computing the wall lubrication force.

45.8.6 Turbulent dispersion

Turbulent dispersion describes the effect of turbulent motion of the liquid phase on the gas phase. The models are also derived from an abstract base class. There is a class named `noTurbulentDispersion` which returns a zero field for the force term and there are a number of classes implementing individual models. The base class declares the method `F()` as a pure virtual method. This means there is no generic formulation as in the case of the drag or lift models.

constantTurbulentDispersionCoefficient

The constant coefficient model implements the following model for the force due to turbulent dispersion.

$$\mathbf{F}_{TD} = C_{TD} \alpha \rho_C k_C \nabla \alpha \quad (125)$$

with

C_{TD}	turbulent dispersion coefficient
α	volume fraction of the dispersed phase
ρ_C	density of the continuous phase
k_C	kinetic turbulent energy of the continuous phase

Burns

The Burns model implements the following model for the force due to turbulent dispersion.

$$\mathbf{F}_{TD} = K_{Drag} \frac{\nu_{C,t}}{\sigma} \nabla \alpha \left(1 + \frac{\alpha}{1 - \alpha} \right) \quad (126)$$

with

K_{Drag}	drag force coefficient due to drag
α	volume fraction of the dispersed phase
$\nu_{C,t}$	turbulent viscosity of the continuous phase
σ	surface tension

Note that K_{drag} is not evaluated by calling method `K()` of the class `dragModel`. Listing 304 shows the actual code that computes the force term of the Burns model.

The reason for computing the drag force coefficient K “by hand” rather than calling `dragModel::K()` might be the run-time. By not calling `K()` we can save one virtual function call¹³⁵. The operations to compute K have to be done anyway, so there is a net saving of one virtual function call.

¹³⁵Virtual function calls are considered to be more expensive in terms of run-time than direct function calls, since the correct function to call has to be determined at run-time [23].

```

1 Foam::tmp<Foam::volVectorField>
2 Foam::turbulentDispersionModels::Burns::F() const
3 {
4     const fvMesh& mesh(pair_.phase1().mesh());
5     const dragModel&
6         drag
7         (
8             mesh.lookupObject<dragModel>
9             (
10                IObject::groupName(dragModel::typeName, pair_.name())
11            )
12        );
13
14     return
15         - 0.75
16         *drag.CdRe()
17         *pair_.dispersed()
18         *pair_.continuous().nu()
19         *pair_.continuous().turbulence().nut()
20         /(
21             sigma_
22             *sqr(pair_.dispersed().d())
23         )
24         *pair_.continuous().rho()
25         *fvc::grad(pair_.continuous())
26         *(1.0 + pair_.dispersed()/max(pair_.continuous(), residualAlpha_));
27 }

```

Listing 304: The definition of the method F() in the file **Burns.C**

Gosman

The Gosman model implements the following model for the force due to turbulent dispersion.

$$\mathbf{F}_{TD} = K_{Drag} \frac{\nu_{C,t}}{\sigma} \nabla \alpha \quad (127)$$

45.9 MRF method - avoiding errors

The MRF method can be used to simulate stirred vessels. By the time of writing, this is the only way to do so with the Eulerian multiphase solvers, since none of the Eulerian solvers has dynamic mesh capability. The basics behind the MRF method are discussed in Section 65.

45.9.1 Inlet boundaries and MRF zones

The MRF method corrects the velocities at the boundaries within the MRF zone. Thus, if a gas inlet BC is placed within the MRF zone, the simulation takes an unintended route. In Figure 111 we see the outcome of a gas inlet boundary placed within an MRF zone. Note, the tangential alignment of the velocity vectors on the right image. The initial inlet definition (visible on the left image) is overridden by the MRF's constraint.

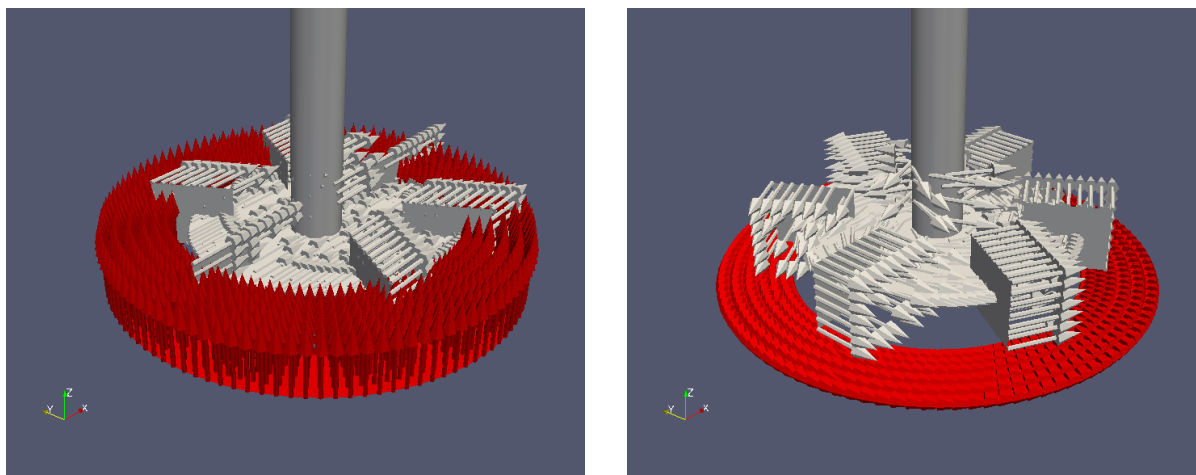


Figure 111: Velocity vectors of the gaseous phase at the inlet boundary (red vectors) in an aerated stirred tank. That the gas inlet boundary lies within the MRF zone. On the left, we see the initial condition and on the right we see the boundary condition after the constraints by the MRF method have been applied.

46 reactingTwoPhaseEulerFoam

With the release of OpenFOAM-3.0.0¹³⁶ in 2015 the solvers *reactingTwoPhaseEulerFoam* and *reactingMultiphaseEulerFoam* were introduced. These solvers feature expanded modelling capabilities compared to the ancestors *twoPhaseEulerFoam* and *multiphaseEulerFoam*. These two solvers are one of the results of a collaboration of companies from the chemical and process industries, and the makers of OpenFOAM¹³⁷.

reactingTwoPhaseEulerFoam and *reactingMultiphaseEulerFoam* form the reactingEulerFoam family of Euler-Euler solvers. These two solvers use a largely common code base for underlying models, such as phase-models, interfacial momentum-transfer models and interfacial composition models. Many of the developments that went into the evolution of the twoPhaseEulerFoam solver from OpenFOAM-2.2 and earlier to OpenFOAM-2.3 have been continued and led to a further generalization of the code base.

Since both solvers derive from a largely common code base, this section mostly applies to both *reactingTwoPhaseEulerFoam* and *reactingMultiphaseEulerFoam*.

46.1 Solver basics

This solver uses the PIMPLE algorithm for solving the governing equations for the two-phase flow.

46.2 Phase modelling

The code base of the phase modelling library is a good example of the principles of encapsulation, abstraction and generic programming at work.

46.2.1 Phase model classes

There is a number of phase-model template classes, each implementing a different aspect of multi-phase modelling, e.g. momentum-transfer, heat-transfer, etc.. Thus, for a specific model a number of these templates has to be nested in a similar fashion as Lagrangian particle models are constructed by plugging template classes into each other.

The class templates, which provide modelling of a certain aspect, can also have an opposite twin-implementation. This allows to construct phase models which either include a certain aspect, or not, by selecting a certain set of templates at compile-time. One example for such a twin-implementation is the aspect of reactions. A phase can either be a reacting one, or an inert one. Thus, there are two template classes dealing with reactions: the templates `InertPhaseModel` and `ReactingPhaseModel`. The name of the template class is quite suggestive to as how each class implements this aspect.

¹³⁶<https://openfoam.org/release/3-0-0/>

¹³⁷<https://openfoam.org/chemical-process-engineering/>

phaseModel

This is the base class for all phase models, and it defines the behaviour of a phase. This class is itself derived from the class `volScalarField`. Thus, the phase model is its own volume fraction field. This class holds very little data, apart from the phase's index, its name and a pointer to the phase's diameter model.

However, this class defines a large number of abstract methods, which the various template classes need to implement.

MovingPhaseModel

This class template provides the functionality and the data for a moving phase, i.e. it holds the velocity field and the flux fields. This class also provides a momentum equation for the phase. This class also holds a pointer to the turbulence model.

```
1  template<class BasePhaseModel>
2  Foam::tmp<Foam::fvVectorMatrix>
3  Foam::MovingPhaseModel<BasePhaseModel>::UEqn()
4  {
5      const volScalarField& alpha = *this;
6      const volScalarField& rho = this->thermo().rho();
7
8      return
9      (
10         fvm::ddt(alpha, rho, U_)
11         + fvm::div(alphaRhoPhi_, U_)
12         + fvm::SuSp(- this->continuityError(), U_)
13         + this->fluid().MRF().DDt(alpha*rho, U_)
14         + turbulence_->divDevRhoReff(U_)
15     );
16 }
```

Listing 305: Constructing the momentum equation of a moving phase in the class `MovingPhaseModel`

The momentum equation provided by this class template bears a striking resemblance to the momentum equation of a compressible solver. If we compare the Listings 305 and 306, we recognize the four of the five terms from Listing 305 as the RHS of the momentum equation of Listing 306. The only difference is, that the terms of the multi-phase momentum equation contain the volume fraction field `alpha`, which takes a uniform value of 1 in the single-phase solver.

```
1  tmp<fvVectorMatrix> tUEqn
2  (
3      fvm::ddt(rho, U)
4      + fvm::div(phi, U)
5      + MRF.DDt(rho, U)
6      + turbulence->divDevRhoReff(U)
7      ==
8      fvOptions(rho, U)
9  );
```

Listing 306: Constructing the momentum equation of a compressible, single-phase solver, on the example of `rhoPimpleFoam`

As the list of options is read and created by the solver, the phase model is oblivious to the presence of any options. Thus, there are no calls to the `fvOptions` framework within the phase models. This is a task for the solver to deal with options from the `fvOptions` framework.

AnisothermalPhaseModel

This class template provides the data and the methods necessary to implement energy transfer, i.e. an energy equation.

IsothermalPhaseModel

This class is the counterpart of the `AnisothermalPhaseModel`, it implements all abstract methods associated with the transport of thermal energy in a trivial manner.

MultiComponentPhaseModel

This template class provides the data and methods associated with species transport within a phase, e.g. the gas phase consisting of several gases.

PurePhaseModel

This template class is used for phases that consist of a single species. Thus there is no need to solve species transport equations. Hence, this class template implements all abstract methods defined in the base class `phaseModel` in a trivial manner, e.g. it returns an empty list of mass fractions when the method `phaseModel::Y()` is called, which is intended to return the species mass fractions.

InertPhaseModel

This template class is used for non-reactive phases, and it implements all methods dealing with reactions in a trivial way. Thus, it returns zero for the heat release rate and the mass transfer rate.

ReactingPhaseModel

This template class is used for reactive phases. Thus, the heat release rate and the mass transfer rate are computed by the underlying reaction model and returned.

46.2.2 Phase system classes

46.2.3 Solution

Phase transport

Solving the transport equation for a phase is handled by the concrete phase system class, e.g. by `twoPhaseSystem` in the case of *reactingTwoPhaseEulerFoam*. This is implemented by the method `solve()` of the phase system class.

Momentum, energy and species transport

Solving the momentum, energy and species transport equations is handled by the solvers itself. The phase models and the phase system models provide the transport equations and transfer terms.

In Listing 307 we see a snippet of *reactingTwoPhaseEulerFoam*'s source code, which deals with the construction of the momentum equation for one phase. This is also an example of the interaction of phase models and phase system models. The LHS of the momentum equation is provided by the phase model, note the call to `phase1.UEqn()`. This call is actually implemented in the template `MovingPhaseModel`, which we see in Listing 305. This template class provides the local derivative, the convective term, a phase-conservation correction, a correction from the MRF framework and the momentum diffusion term.

The RHS of the momentum equation in Listing 307 is a momentum transfer term and a contribution from the `fvOptions` framework. The momentum transfer term is provided by the phase system model.

```
1 U1Eqn =
2 (
3     phase1.UEqn()
4     ==
5     *momentumTransfer[phase1.name()]
6     + fvOptions(alpha1, rho1, U1)
7 );
8 U1Eqn.relax();
9 fvOptions.constrain(U1Eqn);
10 fvOptions.correct(U1);
```

Listing 307: Constructing the momentum equation of the first phase in *reactingTwoPhaseEulerFoam*

46.3 Turbulence modelling

reactingTwoPhaseEulerFoam supports per-phase turbulence modelling, which also allows to treat a phase as laminar, as laminar modelling can be seen as a special case of turbulence modelling. By setting both phases to *mixtureKEpsilon*, a turbulence model for the mixture can be solved, with this specific model being the only option. Thus, all modelling strategies discussed in Section 35.3.1 are available with this solver.

46.4 Interfacial momentum exchange

47 *multiphaseEulerFoam*

multiphaseEulerFoam is an Eulerian solver for n phases. This solver differs in some points from the solver *twoPhaseEulerFoam*.

47.1 Fields

The naming scheme of the fields differs from other multiphase solvers. *multiphaseEulerFoam* directly uses names (e.g. *Uair*, *Uwater*, *Uoil*, etc.).

47.1.1 *alphas*

A specialty of *multiphaseEulerFoam* is the field *alphas*. This field does not represent the volume fraction of a certain phase and is therefore not bounded by 0 and 1. This field is used to represent all phases in a single scalar field. *alphas* is computed by summing up the products of phase index and phase fraction.

$$\text{alphas} = \sum_{i=0}^{n-1} i * \alpha_i \quad (128)$$

Because *alphas* is computed quantity, the file *alphas* can be missing in the θ -directory.

47.2 Momentum exchange

The parameters for the momentum exchange, e.g. the drag model, need to be specified pair-wise.

47.2.1 *drag*

```
drag
(
    (air water)
    {
        type blended;

        air
        {
            type SchillerNaumann;
            residualPhaseFraction 0;
            residualSlip 0;
        }

        water
        {
            type SchillerNaumann;
            residualPhaseFraction 0;
            residualSlip 0;
        }

        residualPhaseFraction 1e-2;
        residualSlip 1e-2;
    }

    /* further definitions */
```

Listing 308: Pair-wise definition of the drag model in the file *transportProperties*

47.2.2 virtual mass

The coefficients for considering virtual mass must also be specified pair-wise. Listing 309 shows how the coefficients for virtual mass are specified in the *damBreak* tutorial.

```
virtualMass
(
  (air water)      0.5
  (air oil)        0.5
  (air mercury)    0.5
  (water oil)      0.5
  (water mercury)  0.5
  (oil mercury)    0.5
);
```

Listing 309: Pair-wise definition of Coefficients for virtual mass in the file `transportProperties`

47.2.3 lift force

Currently (OpenFOAM 2.1.1) there is no lift model in *multiphaseEulerFoam*.

48 driftFluxFoam

`driftFluxFoam` is a solver of OpenFOAM to simulate e.g. settling of disperse particles in a liquid. `driftFluxFoam` is the successor of `settlingFoam`, which has been discontinued with the release of OpenFOAM-2.3.1¹³⁸. `settlingFoam` was used by Brennan [12] in his thesis, which contains a lot of information on deriving the drift flux model from the Eulerian two-fluid model equations. The header of `driftFluxFoam` describes this solver as follows:

Solver for 2 incompressible fluids using the mixture approach with the drift-flux approximation for relative motion of the phases.

Used for simulating the settling of the dispersed phase and other similar separation problems.

`driftFluxFoam` complies with the generic solver design of OpenFOAM, thus this solver can use all available turbulence models. It also can use the MRF method and the *fvOptions* framework.

48.1 Governing equations

The governing equations for the mixture are derived from the two-fluid model [12, 31].

48.1.1 Mixture continuity equation

The mixture continuity equation can be easily derived by adding the continuity equations of the two phases:

$$\frac{\partial \alpha_k \rho_k}{\partial t} + \nabla \cdot (\alpha_k \rho_k \mathbf{u}_k) = 0 \quad (129)$$

with the constitutive relations

$$\rho_m = \alpha_1 \rho_1 + \alpha_2 \rho_2 \quad (130)$$

$$\rho_m \mathbf{u}_m = \alpha_1 \rho_1 \mathbf{u}_1 + \alpha_2 \rho_2 \mathbf{u}_2 \quad (131)$$

we gain

$$\frac{\partial \rho_m}{\partial t} + \nabla \cdot (\rho_m \mathbf{u}_m) = 0 \quad (132)$$

¹³⁸<http://www.openfoam.org/version2.3.1/>

48.1.2 Mixture momentum equation

Derivation from literature

The derivation of the mixture momentum equation is analogous to the derivation of the mixture continuity equation. Therefore, we skip the general derivation and refer the interested reader to the appropriate literature [12, 31]. In this section, we want to focus on the derivation of the specific equations implemented in `driftFluxFoam`.

We start from the derivation given in the appendix of Brennan [12]:

$$\frac{\partial \rho_m \mathbf{u}_m}{\partial t} + \nabla \cdot (\rho_m \mathbf{u}_m \mathbf{u}_m) = -\nabla p_m + \nabla \cdot \left(\boldsymbol{\tau} + \boldsymbol{\tau}_t - \sum \alpha_k \rho_k \mathbf{u}_{km} \mathbf{u}_{km} \right) + \rho_m \mathbf{g} + \mathbf{M}_k \quad (133)$$

we pay special attention to the diffusion stress $\sum \alpha_k \rho_k \mathbf{u}_{km} \mathbf{u}_{km}$, which represents momentum diffusion due to the relative motion between the phases.

$$\sum \alpha_k \rho_k \mathbf{u}_{km} \mathbf{u}_{km} = \alpha_1 \rho_1 \mathbf{u}_{1m} \mathbf{u}_{1m} + \alpha_2 \rho_2 \mathbf{u}_{2m} \mathbf{u}_{2m} \quad (134)$$

For convenience we introduce the symbol $\boldsymbol{\tau}_{dm}$ for the diffusion stress

$$\boldsymbol{\tau}_{dm} = \sum \alpha_k \rho_k \mathbf{u}_{km} \mathbf{u}_{km} \quad (135)$$

with $\mathbf{u}_{km}$, the velocity of the phase k relative to the mixture's centre of mass; $\mathbf{u}_{km}$ is also referred to as *diffusion velocity* of the phase k

$$\mathbf{u}_{km} = \mathbf{u}_k - \mathbf{u}_m \quad (136)$$

Ishii and Hibiki [31] states a relation between the diffusion velocities of the two phases:

$$\alpha_1 \rho_1 \mathbf{u}_{1m} + \alpha_2 \rho_2 \mathbf{u}_{2m} = \mathbf{0} \quad (137)$$

Thus, we can eliminate $\mathbf{u}_{1m}$ from the diffusion stress $\boldsymbol{\tau}_{dm}$

$$\boldsymbol{\tau}_{dm} = \alpha_1 \rho_1 \left(\frac{\alpha_2 \rho_2}{\alpha_1 \rho_1} \right)^2 \mathbf{u}_{2m}^2 + \alpha_2 \rho_2 \mathbf{u}_{2m}^2 \quad (138)$$

$$\boldsymbol{\tau}_{dm} = \alpha_2 \rho_2 \mathbf{u}_{2m}^2 \left(\frac{\alpha_2 \rho_2}{\alpha_1 \rho_1} + 1 \right) \quad (139)$$

$$\boldsymbol{\tau}_{dm} = \alpha_2 \rho_2 \mathbf{u}_{2m}^2 \left(\frac{\alpha_2 \rho_2 + \alpha_1 \rho_1}{\alpha_1 \rho_1} \right) \quad (140)$$

$$\boldsymbol{\tau}_{dm} = \alpha_2 \rho_2 \mathbf{u}_{2m}^2 \left(\frac{\rho_m}{\alpha_1 \rho_1} \right) \quad (141)$$

$$\boldsymbol{\tau}_{dm} = \rho_m \frac{\alpha_2 \rho_2}{\alpha_1 \rho_1} \mathbf{u}_{2m}^2 \quad (142)$$

Implementation

From the source code in Listing 310 we see the diffusion stress as the fourth term on the LHS of the momentum equation.

```

1  fvVectorMatrix UEqn
2  (
3      fvm::ddt(rho, U)
4      + fvm::div(rhoPhi, U)
5      + MRF.DDt(rho, U)
6      + fvc::div(UdmModel.tauDm())
7      + turbulence->divDevRhoReff(U)
8      ==
9      fvOptions(rho, U)
10 );
```

Listing 310: The momentum equation of `driftFluxFoam`

Next, we take a look at the implementation of the diffusion stress.

```

1 tmp<volSymmTensorField> Foam::relativeVelocityModel::tauDm() const
2 {
3     volScalarField betac(alphac_*rhoc_);
4     volScalarField betad(alphad_*rhod_);
5
6     // Calculate the relative velocity of the continuous phase w.r.t the mean
7     volVectorField Ucm(betad*Udm_/betac);
8
9     return tmp<volSymmTensorField>
10    (
11        new volSymmTensorField
12        (
13            "tauDm",
14            betad*sqr(Udm_) + betac*sqr(Ucm)
15        )
16    );
17 }

```

Listing 311: The diffusion stress of `driftFluxFoam` computed by the `relativeVelocityModel`

And now, we translate the source code into some math:

$$\boldsymbol{\tau}_{bm} = \beta_d \mathbf{u}_{dm}^2 + \beta_c \mathbf{u}_{cm}^2 \quad (143)$$

with

$$\beta_d = \alpha_d \rho_d \quad (144)$$

$$\beta_c = \alpha_c \rho_c \quad (145)$$

$$\mathbf{u}_{cm} = \frac{\beta_d}{\beta_c} \mathbf{u}_{dm} \quad (146)$$

we gain

$$\boldsymbol{\tau}_{bm} = \beta_d \mathbf{u}_{dm}^2 + \beta_c \left(\frac{\beta_d}{\beta_c} \right)^2 \mathbf{u}_{dm}^2 \quad (147)$$

$$\boldsymbol{\tau}_{bm} = \beta_d \mathbf{u}_{dm}^2 \left(1 + \frac{\beta_d}{\beta_c} \right) \quad (148)$$

$$\boldsymbol{\tau}_{bm} = \alpha_d \rho_d \mathbf{u}_{dm}^2 \left(1 + \frac{\alpha_d \rho_d}{\alpha_c \rho_c} \right) \quad (149)$$

$$\boldsymbol{\tau}_{bm} = \alpha_d \rho_d \mathbf{u}_{dm}^2 \left(\frac{\alpha_c \rho_c + \alpha_d \rho_d}{\alpha_c \rho_c} \right) \quad (150)$$

$$\boldsymbol{\tau}_{bm} = \alpha_d \rho_d \mathbf{u}_{dm}^2 \frac{\rho_m}{\alpha_c \rho_c} \quad (151)$$

$$\boldsymbol{\tau}_{bm} = \rho_m \frac{\alpha_d}{\alpha_c} \frac{\rho_d}{\rho_c} \mathbf{u}_{dm}^2 \quad (152)$$

We notice, that (152) derived from the source code, equals (142), derived from literature with phase 2 being the disperse phase d .

Relative velocity

The diffusion velocity $\mathbf{u}_{dm}$ and the drift velocity $\mathbf{u}_{dj}$ are linked by a constitutive relation:

$$\mathbf{u}_{dm} = \frac{\rho_1}{\rho_m} \mathbf{u}_{dj} \quad (153)$$

We find this relation also in the source code in Listings 316 and 317. Ishii and Hibiki [31] state, that in the case of dispersed two-phase flow the drag correlation should be expressed in terms of the drift velocity $\mathbf{u}_{dj}$.

The relative velocity models provide a method that returns $\mathbf{u}_{dm}$, however, in the source code of the Listings 316 and 317 we find relation (153) translated into C++. There, the expression for $\mathbf{u}_{dm}$ consists of the density ratio and a relation for the drift velocity, which links the terminal velocity of a single particle and the volume fraction of the disperse phase.

48.2 incompressibleTwoPhaseInteractingMixture

The class `incompressibleTwoPhaseInteractingMixture` serves as the transport model for `driftFluxFoam`. This class holds all the information of the two phases and provides the mixture quantities. `driftFluxFoam` solves the momentum and pressure equations for the mixture. Thus, this solver is in between a single-phase solver and a full two-fluid solver such as *twoPhaseEulerFoam*.

Via this transport model, the mixture quantities propagate to the turbulence model, since the turbulence model receives a transport model class as template parameter at construction. This is one example for the versatility of the new, templated turbulence modelling framework. The precursor `settlingFoam` had a hard-coded $k - \epsilon$ turbulence model. Also the viscosity model was kind of hard-coded.

48.3 Mixture viscosity models

Settling equipment is often operated with solids concentrations at which the presence of the solid particles affect fluid properties. Besides using the mixture density, a mixture viscosity also has to be used.

48.3.1 mixtureViscosityModel

The class `mixtureViscosityModel` is the abstract base class for the actual viscosity models. This class serves a similar purpose as the base class for the single-phase viscosity models `viscosityModel` located in `$FOAM_SRC/transportModels/incompressible/viscosityModels/viscosityModel`. These two base classes are rather similar and there are only slight differences in their implementations.

48.3.2 slurry

The `slurry` mixture viscosity model is a correction for the Newtonian viscosity with reference to Thomas [60].

$$\mu = \mu_c \left(1 + 2.5\alpha_d + 10.05\alpha_d^2 + 0.00273 e^{16.6\alpha} \right) \quad (154)$$

The source code computing the mixture viscosity is a direct translation of the math above into C++.

```

1 Foam::tmp<Foam::volScalarField>
2 Foam::mixtureViscosityModels::slurry::mu(const volScalarField& muc) const
3 {
4     return
5     (
6         muc*(1.0 + 2.5*alpha_ + 10.05*sqr(alpha_) + 0.00273*exp(16.6*alpha_))
7     );
8 }
```

Listing 312: The calculation of the mixture viscosity by the `slurry` mixture viscosity model.

48.3.3 plastic

The plastic viscosity model is based on a generic viscosity model (155) for liquids exhibiting plastic behaviour.

$$\tau = aC^{b\alpha} \quad (155)$$

The `plastic` model implemented in `driftFluxFoam` translates to:

$$\mu = \min [\mu_c + k * (10^n - 1), \mu_{max}] \quad (156)$$

Listing 313 shows the source code computing the mixture viscosity. The -1 in the second term ensures, that we retain the laminar viscosity of the continuous phase in the case the dispersed volume fraction vanishes, since anything to the power of zero equals one.

```

1 Foam::tmp<Foam::volScalarField>
2 Foam::mixtureViscosityModels::plastic::mu(const volScalarField& muc) const
3 {
4     return min
5     (
6         muc
7         + plasticViscosityCoeff_
8         *(
9             pow
10            (
11                scalar(10),
12                plasticViscosityExponent_*alpha_
13            ) - scalar(1)
14        ),
15         muMax_
16     );
17 }

```

Listing 313: The calculation of the mixture viscosity by the `plastic` mixture viscosity model.

48.3.4 BinghamPlastic

`BinghamPlastic` is a Bingham plastic model.

48.4 Relative velocity models - hindered settling

In this section we use the symbol v for the velocity to follow the notation of Brennan [12] as well as the source code of OpenFOAM.

48.4.1 The base class

The base class holds the data common to the derived models. The base class holds the private field `Udm_` for the diffusion velocity $\mathbf{u}_{dm}$ and declares an abstract method `correct()`. The method `correct()` is used by the derived classes to compute the diffusion velocity `Udm_`.

The method `Udm()` of the base class simply returns `Udm_`, and the method `tauDm()` returns the diffusion stress computed from the diffusion velocity.

The diffusion velocity

The class for the relative velocity model holds a vector field for the diffusion velocity. The internal field values are determined from the actual model in use, however, the boundary conditions are taken over from the mixture velocity field.

This, we can read from the source code of the base class. In Listing 314 we see the initializer responsible for the diffusion velocity.

```

1 Udm_
2 (
3     IOobject
4     (
5         "Udm",
6         alphac_.time().timeName(),
7         alphac_.mesh(),
8         IOobject::NO_READ,
9         IOobject::AUTO_WRITE
10    ),
11    alphac_.mesh(),
12    dimensionedVector("Udm", dimVelocity, vector::zero),
13    mixture.U().boundaryField().types()
14 )

```

Listing 314: The initializer entry for `Udm_` in the constructor of the `relativeVelocityModel` class.

For the interpretation of Listing 314 we need to dig out the appropriate constructor of the class `GeometricField`¹³⁹. In Listing 315 we see that the constructor receives five arguments, of which the last has a default value. If we pass only four arguments, the fifth will be determined from the default value.

```

1  //- Constructor given IObject, mesh, dimensioned<Type> and patch types.
2  GeometricField
3  (
4      const IObject&,
5      const Mesh&,
6      const dimensioned<Type>&,
7      const wordList& wantedPatchTypes,
8      const wordList& actualPatchTypes = wordList()
9  );

```

Listing 315: The signature of the constructor called by the code in Listing 314.

If we compare the arguments of the constructor call of Listing 314 and the signature in Listing 315, we see that the first argument passed is clearly an `IObject`. The second argument is a reference to the mesh itself, which is obvious from the call to `alphac_.mesh()` in Listing 314.

The third argument determines the type of the field as well as the initial value. The template parameter `Type` determines whether the field is a scalar, a vector or a tensor field. As a `dimensionedVector` is passed in Listing 314, `Type` evaluates to `vector`¹⁴⁰.

The fourth argument is a list of patch types, since we passed only one dimensioned value as the third argument, there has been no information passed on the boundary conditions of the field up to now. By passing the list of boundary types of the mixture velocity field (`mixture.U()`), the boundary conditions of the field `Udm_` are specified.

As there is no fifth argument passed in Listing 314, the return value of the call `wordList()` is used.

48.4.2 simple

The model named `simple` is similar to the model used by Brennan [12] with attribution to Dahl [17]. This model is very similar to the Vesilind [65] model (158), Brennan [12] explains the change of the base from the Euler number e to the base 10 with a closer fit to experimental data gathered by Dahl [17].

$$v_s = v_0 10^{-k\alpha} \quad (157)$$

The implementation of the `simple` model is more or less a direct translation from math (157) to C++. In the exponent the maximum of the dispersed volume fraction and zero is taken to avoid numerical trouble from negative values of the volume fraction. Reversing the sign in an exponent is never a good idea in numerical simulation.

```

1  void Foam::relativeVelocityModels::simple::correct()
2  {
3      Udm_ = (rhoc_/rho())*V0_*pow(scalar(10), -a_*max(alphad_, scalar(0)));
4  }

```

Listing 316: The calculation of the dispersed diffusion velocity `Udm_` by the `simple` relative velocity model.

48.4.3 general

The model referred to as `general` is most probably based on the model of Takács [59], there is no reference to any literature in the header file. The Takács [59] model (159) is a so-called double-exponential model based on the model of Vesilind [65], see (158) [28, 12].

$$v_s = v_0 e^{-nX} \quad (158)$$

$$v_s = v_0 (e^{-r_h X} - e^{-r_p X}) \quad (159)$$

¹³⁹Bear in mind, that `volVectorField` and others are specialisations of the templated class `GeometricField`.

¹⁴⁰Bear in mind, that `dimensionedVector` is a specialisation of the templated class `dimensioned<Type>` and `dimensionedVector` is a shorthand for `dimensioned<vector>`.

with

$$\begin{aligned}
 v_s & \text{ settling velocity} \\
 v_0 & \text{ maximum settling velocity} \\
 n & \text{ model parameter} \\
 r_h & \text{ settling parameter for hindered settling} \\
 r_p & \text{ settling parameter for low solids concentration} \\
 X & \text{ suspended solids concentration}
 \end{aligned}
 \tag{160}$$

The implementation .

```

1 void Foam::relativeVelocityModels::general::correct()
2 {
3     Udm_ =
4         (rhoc_/rho())
5         *V0_
6         *(
7             exp(-a_*max(alphad_ - residualAlpha_, scalar(0)))
8             - exp(-a1_*max(alphad_ - residualAlpha_, scalar(0)))
9         );
10 }

```

Listing 317: The calculation of the dispersed diffusion velocity U_{dm} by the **general** relative velocity model.

48.5 settlingFoam

Here we take a closer look on **settlingFoam** (of OpenFOAM-2.2.x), which is the predecessor of **driftFluxFoam**. By comparing the implementations of these two solvers we can observe the transition of the OpenFOAM source code base to a more encapsulated approach.

48.5.1 Mixture viscosity

settlingFoam was/is restricted to the plastic or Bingham viscosity models. Listing 318 shows the code of **settlingFoam**, which computes the mixture viscosity. This code is located in a source file, which is included into the body of the PIMPLE loop of the solver.

Thus, for this solver, the treatment of mixture viscosity is not encapsulated. The viscosity models are not located in separate files and the code of the solver itself contains all the knowledge of the viscosity models. Extending the solver with one or more mixture viscosity models would entail building an extended **if**-cascade within the file **correctViscosity.H**.

```

1 {
2     /* compute plastic viscosity */
3     mul = muc +
4         plasticViscosity
5         (
6             /* code removed for brevity */
7         );
8
9     if (BinghamPlastic)
10    {
11        volScalarField tauy = yieldStress
12        (
13            /* see yieldStress.H */
14        );
15        mul =
16            /* compute contribution of yield stress */
17            + mul;
18    }
19
20    mul = min(mul, muMax);
21 }

```

Listing 318: The calculation of the mixture viscosity in the file `correctViscosity.H` of `settlingFoam` of OpenFOAM-2.2.x. Comments added by the author.

48.5.2 Relative velocity models

`settlingFoam` of OpenFOAM-2.2.x offers the same choice of relative velocity models as `driftFluxFoam` at the time of writing. However, implementation-wise we note, that model selection is, again, done in an `if`-statement cascade.

```
1  if (VdjModel == "general")
2  {
3      Vdj = V0*
4      (
5          exp(-a*max(alpha - alphaMin, scalar(0)))
6          - exp(-a1*max(alpha - alphaMin, scalar(0)))
7      );
8  }
9  else if (VdjModel == "simple")
10 {
11     Vdj = V0*pow(10.0, -a*alpha);
12 }
13 else
14 {
15     FatalErrorIn(args.executable())
16         << "Unknown VdjModel : " << VdjModel
17         << abort(FatalError);
18 }
19
20 Vdj.correctBoundaryConditions();
```

Listing 319: The calculation of the relative velocity in the file `calcVdj.H` of `settlingFoam` of OpenFOAM-2.2.x.

48.5.3 Turbulence

Turbulence in `settlingFoam` was/is implemented in a similar fashion as in `twoPhaseEulerFoam` of that time. Both solvers feature a hard-coded $k - \epsilon$ turbulence model, which is adapted to the solvers needs.

Part VII

Postprocessing

There are two principal possibilities for post processing in OpenFOAM. First, there are tools that are executed after a simulation has finished. These tools work on the written data of the solution. *sample* and *paraView* are two examples for such tools.

Besides that, there is *run-time post processing*. Run-time post processing performs certain operations on the solution data as it is generated. Consequently, run-time post processing allows for a much finer time resolution. The function objects – e.g. for calculating forces or force coefficients – are an example for run-time post processing. The big disadvantage of this method is, that the user has to know the intended post processing steps before starting a simulation. See <http://www.openfoam.com/features/runtime-postprocessing.php> for more information about run-time post processing.

49 *functions*

A function object - in general - are objects, which are used in a similar manner as a function. This main benefit of function objects is that they can have a permanent state. This state can be used to store data between “calls” of the function object. A very illustrative example is OpenFOAM’s function object *fieldAverage*, which computes the temporal average of fields. This function object needs to update itself with every time step computed and the current time-averaged fields need to be preserved between “calls” of *fieldAverage*.

Function objects usually serve one specific purpose, e.g. compute the time average of a field quantity. Thus, there is a large, ever growing number of function objects available in OpenFOAM. Some of which are listed below to give an impression of the wide range of tasks currently covered by OpenFOAM’s function objects:

fieldAverage compute the temporal average of field quantities

the fieldValue family compute the spatial average (or other operations) of field quantities

forces compute the forces on a body (surface)

forceCoeffs compute force coefficients, e.g. for drag, lift and torque

sampledSet save the field values of a certain region, e.g. along a line

probes save field values at certain points

streamLine compute streamlines

scalarTransport solve a passive scalar transport equation

codedFunctionObject implement your own function object in a not-entirely-from-scratch framework

The list above is only a small selection of available functions. Check out OpenFOAM’s sources for a complete overview on available function objects¹⁴¹.

49.1 Stay up to date

Run-time post-processing with function objects is a feature of OpenFOAM which is very much in the realm of application and daily use, in contrast to the inner workings of the mesh class. Thus, function objects receive much attention from the developers in form of new function objects, extending the features of existing function objects or reorganizing and renaming existing function objects.

The first two points (addition and extension) are clearly for the benefit of the users, whereas the latter one (reorganisation) most probably benefits the developers in reducing code duplication or easing maintenance. Reorganisation might go hand in hand with renaming of function objects¹⁴². In such a case you might need to modify the function object definitions of your cases when migrating them to a newer¹⁴³ version.

¹⁴¹New users might hate this statement, but *the documentation is in the code*.

¹⁴²As an illustrative example, the function object for processing field values within a cell set has changed its name from *cellSource* (OpenFOAM-3.0) over *volRegion* (OpenFOAM-4.0) to *volFieldValue* (OpenFOAM-5.0). This has been done, most probably, to keep the naming scheme consistent with the underlying class names or make the name descriptive of its task. However, as in many aspects of life in general, there is no unique, best way to do things. In this case, the function object’s name can describe *what it is* or *what it does*. Both ways are equally valid and there might even be more aspects to choose from.

¹⁴³The same is the case for migrating cases to older versions, however, using the current version should be the norm.

Apart from all the other benefits of using the latest version of OpenFOAM, especially when it comes to function objects, you might be able to save yourself from developing a function object for your needs, if a newer version of OpenFOAM already contains a function object implementing the functionality you need.

49.2 Definition

Function objects are defined in the file `controlDict`. There, a function dictionary is created which contains all necessary informations. Listing 320 shows the basic structure of such a definition.

Every function has a name. This name is stated at the place of the `NAME` placeholder in Listing 320. This name is also the name of the folder OpenFOAM creates in the case directory. There, all data generated by the function object is stored.

Each function object also has a type. This type needs to be specified at the place of the `TYPE` placeholder. The type needs to be from the list of the available functions. To find out, which functions are available, the *banana-trick*¹⁴⁴ can be used. Listing 321 shows the error message that is caused by the banana-trick.

The placeholder `LIBRARY` marks the place where the name of the library needs to be entered. A function object is not a program that is executable on its own. It is merely a library that is used by other programs. In our case, the function objects are called by the solvers. Therefore, the function objects are not compiled into executables. The compiler creates libraries when the function objects are compiled. These libraries contain the functions in a machine readable form.

The keyword `enabled` is optional. With this keyword function objects can be excluded from execution.

```
functions
{
    NAME
    {
        type                TYPE;
        functionObjectLibs ("LIBRARY");
        enabled             true;
        /*
        Definition
        */
    }
}
```

Listing 320: Definition of function objects in the file `controlDict`

```
--> FOAM FATAL ERROR:
Unknown function type banana

Valid functions are :

13
(
cellSource
faceSource
fieldAverage
fieldCoordinateSystemTransform
fieldMinMax
nearWallFields
patchProbes
probes
readFields
sets
streamLine
surfaceInterpolateFields
surfaces
)
```

Listing 321: Output of the *banana-trick*; applied to the keyword `type`

¹⁴⁴If OpenFOAM expects a keyword from a limited set of allowed keywords, stating an invalid keyword usually causes OpenFOAM to print the list of allowed entries.

49.3 Control

All function objects are related more or less directly to the base class `functionObject`, which is defined in the file `$FOAM_SRC/OpenFOAM/db/functionObjects/functionObject/functionObject.H`. This class defines the smallest common behaviour of all function objects. Thus, it may pay off to study this class, as all we learn from it applies to all function objects.

49.3.1 Time control

The stages of function objects

Some OpenFOAM function objects might have some internal state which needs to be updated, whereas others might have no need for an internal state. Simple function object, which write selected data to disk might fall under the latter category, e.g. the function object `surfaceRegion` from the `fieldValue` family of function objects simply writes the data from specified patches to disk. This is done only at write time.

On the other hand, function objects might need to compute data from the solution data and thus need an internal state, e.g. the `fieldAverage` function object needs to continuously update its internal fields for the temporal averages, even if it writes them less frequently to disk.

Thus, the operational stages of a function object are divided in *execute* (for updating its internal state) and *write* (for writing data, and computing to-be-written-data, which is not an internal state).

Execution & write control

The attributes of function objects for when-to-write are `writeControl` and `writeInterval`, in older versions of OpenFOAM these were `outputControl` and `outputInterval`. These control when and how often the data from the function object is written to disk.

A similar pair of controls (`executeControl` and `executeInterval`) exists for controlling when the function object is executed, i.e. its internal state is updated.

Note, that while can use the setting `writeInterval adjustableRunTime`; for the write interval of the functionObject, we need to ensure that the same setting is made for the simulation in general. If the simulation uses the setting `writeInterval runTime`;, then the setting of the functionObject is over-ruled.

Enablement

The `enabled` flag controls whether the function object is enabled. This takes a boolean value and control whether to execute the function object or not. This might be useful for testing or debugging simulation cases. This flag allows you to define function objects and stop them from being used without deleting them from `controlDict`. A rather brute-force alternative to this flag, from the case file editing perspective, would be to comment the function object definition. However, changing the flag from `on` to `off` and vice-versa requires less characters changed than commenting and uncommenting¹⁴⁵.

The pair `timeStart` and `timeEnd` control when to begin using a function object and when to stop. These controls are optional and are in most cases omitted. The default behaviour, when these controls are omitted, is to execute function objects from the start of the simulation and to execute them until the simulation finishes. In fact, the default values are a negative, ridiculously large number¹⁴⁶ for `timeStart` and a ridiculously large number for `timeEnd`. Thus, no reasonable simulation will start before the default value of `timeStart` or run any longer than the default value of `timeEnd`. This way, there is no need for conditional statements, a simple comparison against the current time suffices.

49.3.2 Region control

The `region` keyword controls to which mesh region the function object applies. This can be omitted in all cases with only one mesh region, as stating the mesh region is superfluous. However, there are cases in OpenFOAM with more than one mesh region, such as conjugated heat transfer simulations. In this case we have one mesh (region) for the fluid region and one mesh (region) for the solid parts of our domain to solve for heat transfer. In this case we can use a function object to log the average temperature of both the fluid and the solid regions.

¹⁴⁵Admittedly, with the use of multi line comments (`/* no comment */`), the amount of changed characters is four, compared to one or two when using the enabled flag. Potential savings are minuscule.

¹⁴⁶The source code of the `scalar` class defines a static variable `VGREAT`, which is close to the maximum representable floating point number. `VGREAT` and similar static variables are frequently used for initialisation of variables.

Thus, we specify two function objects to evaluate the volumetric average temperature, however, we need to specify the region for which the function objects are to be executed.

The (abstract¹⁴⁷) base class responsible for selecting a mesh region to operate on is `regionFunctionObject`. A number of function objects are derived more or less directly from `regionFunctionObject`. Among these are function objects of the `fieldValues` family as well as `fieldMinMax` and `fieldAverage`.

49.4 probes

The function *probes* saves the values of certain field quantities at specific points in space. Listing 322 shows an example of the definition of a *probes* function object.

This function object is of the type `probes`. The name of the function object is `probes1`. The data generated by this function is stored in the directory `probes1`. This directory contains a sub-directory. The name of this sub-directory corresponds to the time at which the simulation is started. This prevents files from being overwritten in case a simulation is continued at some point in time.

Figure 112 shows the directory tree after a simulation ended. There, the folder `probes1` contains a sub-directory named 0. This is the time the simulation started. The 0 folder contains the files `p` and `U`.

The keywords `outputControl` and `outputInterval` are optional. They control – as their names suggest – the way the data is written to the hard drive.

`fields` contains the names of the fields that are of interest. `probeLocations` contains a set of points. The data of a specified field is computed for this locations and written to a file. The name of this file is the fields of interest. Listing 322 will result in two files. The file `p` contains the values of the pressure for all locations, the file `U` will contain the values of the velocity at all locations.

The function *probes* is contained in the file `libsampling.so`. This information can be gained from the tutorials. See Section 66.3 for more information about how to search the tutorials for specific information.

```
functions
{
    probes1
    {
        type                probes;
        functionObjectLibs ("libsampling.so");
        enabled              true;
        outputControl        timeStep;
        outputInterval       1;

        fields
        (
            P
            U
        );

        probeLocations
        (
            ( 0.0254 0.0253 0 )
            ( 0.0508 0.0253 0 )
        );
    }
}
```

Listing 322: The definition of *probes* in the file `controlDict`

¹⁴⁷The class `regionFunctionObject` inherits two pure virtual methods from its base class `functionObject` which it does not implement. Thus, `regionFunctionObject` is an abstract class.

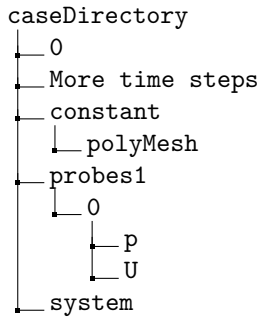


Figure 112: A part of the directory tree after the simulation ended

49.4.1 Pitfalls

Probe location outside the domain

If the probe location is outside of the domain OpenFOAM will issue a warning message and continue with the simulation.

```

--> FOAM Warning :
    From function findElements::findElements(const fvMesh&)
    in file probes/probes.C at line 102
    Did not find location (0.075 0 0.48) in any cell. Skipping location.

```

Listing 323: probe location outside of the domain

Unknown or non-existent field

If the probes dictionary contains fields that are not present to be probed, then no warning or error message will be issued. OpenFOAM simply continues computation. If the dictionary contains no valid fields to be probed, then the probe function will not be executed. Consequently no folder for storing the data will be created.

Using probes with moving mesh

When we use probes, OpenFOAM searches the mesh for the cells containing the specified probe locations. More specifically, it determines the cell indices of the cells containing the probe locations. Thus, OpenFOAM can easily obtain the cell value of the probed field. In essence a field in OpenFOAM is a list of values, and the cell index is used to navigate the list.

However, when we use moving meshes, the cell index related to a certain point in space may change over time, e.g. when probing a point within a rotating mesh region. Consequently, everytime the mesh is updated, we need to ensure that the cell indices of the probed are updated as well.

In OpenFOAM probes have a boolean flag (`fixedLocations`) controlling this updating of the cell indices. By default, this flag is set to true, meaning that no updating is required. Below, in Listing 324, we see the description of this flag from the header file `probes.C`.

```

//- Fixed locations, default = yes
// Note: set to false for moving mesh calations where locations
//       should move with the mesh
bool fixedLocations_;

```

Listing 324: Description of the `fixedLocations` boolean flag in `probes.H`

When using moving meshes, add the `fixedLocations` flag to your `probesDict`, as shown in Listing 325 below.

```

fields
(
    ...
)

```

```

probeLocations
(
    ...
)

// add this for cases using moving meshes
fixedLocations    false;

```

Listing 325: Add the `fixedLocations` flag to the `probesDict` when using probes with moving meshes

49.5 *fieldAverage*

fieldAverage computes time-averaged fields. Listing *lst:fieldAverageControlDict* shows an example of how this function is set up.

```

functions
{
    fieldAverage1
    {
        type                fieldAverage;
        functionObjectLibs ( "libfieldFunctionObjects.so" );
        enabled              true;
        outputControl        outputTime;
        fields
        (
            Ua
            {
                mean          on;
                prime2Mean     off;
                base           time;
            }
        );
    }
}

```

Listing 326: Definition of a *fieldAverage* function object in the file `controlDict`

The *fieldAverage* function object can be provided with a averaging window size and name to compute a sliding average. In this case, the resulting averaged field bears the window name as a file name suffix besides the field name and the suffix **Mean**. With this feature, multiple averages of a field can be computed. Listing 327 shows two averages of the field **U.water**.

If no window is specified, *fieldAverage* computes the average from the start time of the function object. The resulting field bears the name of the to-be-averaged field and the suffix **Mean**. If a window size and a window name is specified, the resulting field's name is extended with the window name.

```

U.water    U.waterMean    U.waterMean_w1

```

Listing 327: Multiple averages of the field **U.water**

49.6 *faceSource*

49.6.1 Average over a plane

faceSource extracts data from surfaces (faces). Listing 328 shows how the average of a field quantity over a cutting plane is set up.

```

functions
{
    faceObj1
    {
        type                faceSource;
        functionObjectLibs ( "libfieldFunctionObjects.so" );
        enabled              true;
    }
}

```

```

outputControl    outputTime;

// Output to log&file (true) or to file only
log              true;

// Output field values as well
valueOutput      false;

// Type of source: patch/faceZone/sampledSurface
source           sampledSurface;

sampledSurfaceDict
{
    // Sampling on triSurface
    type           cuttingPlane;
    planeType      pointAndNormal;
    pointAndNormalDict
    {
        basePoint ( 0 0 0.3 );
        normalVector ( 0 0 1 );
    }
    interpolate true;
}

// Operation: areaAverage/sum/weightedAverage ...
operation         areaAverage;
fields
(
    alpha
);
}
}

```

Listing 328: Definition of a *faceSource* function object in the file `controlDict`

49.6.2 Compute volumetric flow over a boundary

Listing 329 shows the definition of a function object that is used to compute the volumetric flow over a boundary face. The key points for this are the definition of a weight field and the use of the summation operation. The weight field is automatically applied to the processed field, there is no need to specifically an operation such as *weightedSum*. If no weight field is defined, no weight field is used.

```

functions
{
    faceIn
    {
        type           faceSource;
        functionObjectLibs ("libfieldFunctionObjects.so");
        enabled         true;
        outputControl    timeStep;
        log              true;
        valueOutput      false;
        source           patch;
        sourceName        spargerInlet;
        surfaceFormat     raw;
        operation         sum;
        weightField        alpha1;

        fields
        (
            phi1
        );
    }
}

```

Listing 329: Definition of a *faceSource* function object in the file `controlDict`

49.6.3 Pitfall: valueOutput

The option `valueOutput` writes the field values on the sampled surface to disk. This can lead to massive disk space usage when setting `outputControl` to `timeStep`. In this case the field values are written for every time step. The option `valueOutput` should be disabled unless it is really needed.

Figure 113 shows the contents of the `postProcessing` folder after two time steps have been written to disk. For each sampled field the field values on the sampled patch are written to disk in files in the `surface` folder.

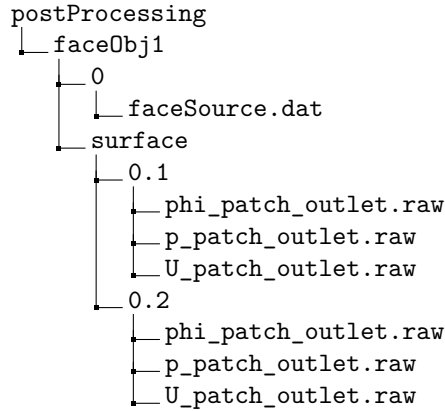


Figure 113: The content of the `postProcessing` folder

49.7 cellSource

The `cellSource` function object acts on all cells of the mesh or on the cells of a `cellZone`.

Listing 330 shows the definition of a `cellSource` function object. In this case, a part of the domain is contained in the `cellZone left`. The function object calculates the volume-average value of the volume fraction of air. The keyword `valueOutput` is set to the value `false` and marked as evil by the comment for reasons explained in Section 49.6.3.

```
1 functions
2 {
3     airContent_left
4     {
5         type                cellSource;
6         functionObjectLibs ("libfieldFunctionObjects.so");
7         enabled              true;
8         outputControl        timeStep;
9         log                  true;
10        valueOutput          false; // evil
11        source                cellZone;
12        sourceName            left;
13        operation              volAverage;
14
15        fields
16        (
17            alpha.air
18        );
19    }
20 }
```

Listing 330: A usage example of the `cellSource` function object

49.8 readFields

If we want to post-process a case, which we computed with a custom solver that created custom fields, and the post-processing involves these custom fields, it is very likely that OpenFOAM will complain of those fields not being present in the database when we run our post-processing function objects. That's when `readFields` comes into play.

The description in the header files reads:

Reads fields from the time directories and adds them to the mesh database for further post-processing.

This function object serves the purpose to read specified fields, and add those to the database. Thus, adding this function object to our post-processing function objects allows us to post-process any custom fields with OpenFOAM's standard function objects.

49.9 writeObjects

If we want to post-process fields, that are not written by default, then we can tell OpenFOAM to write specified fields to disk. The description of the header file for this function object states:

Allows specification of different writing frequency of objects registered to the database.

49.10 Execute C++ code as functionObject

OpenFOAM makes it possible to execute C++ code as a functionObject¹⁴⁸. This feature is disabled by default. To activate it a flag has to be changed. This is done for a single user in `~/OpenFOAM/$WM_PROJECT_VERSION/controlDict` or system wide in `$WM_PROJECT_DIR/etc/controlDict`. In one of these files the flag shown in Listing 331 has to be set to one. It can be, that the first of these files does not exist, i.e. there are no user specific settings. The question of precedence (User setting over system wide setting) has not been pursued by the author.

Listing 332 shows an example of this feature. The field quantities $U1$, $U2$ and p are read in and some calculated values are printed to the Terminal.

```
// Allow case-supplied C++ code (#codeStream, codedFixedValue)
allowSystemOperations 1;
```

Listing 331: Allow case-supplied C++ code

```
1  extraInfo
2  {
3      type                coded;
4      functionObjectLibs ( "libutilityFunctionObjects.so" );
5      redirectType        average;
6      code
7      #{
8          const volVectorField& U1 = mesh().lookupObject<volVectorField>("U1");
9          const volVectorField& U2 = mesh().lookupObject<volVectorField>("U2");
10         Info << "max U1 = " << max(mag(U1)).value() << ", U2 = " << max(mag(U2)).value() << endl;
11         const volScalarField& p = mesh().lookupObject<volScalarField>("p");
12         Info << "p min/max = " << min(p).value() << ", " << max(p).value() << endl;
13     #};
14 }
```

Listing 332: Define a *functionObject* using C++

When the solver is invoked, the so called coded functionObject is compiled on the fly. Listing 333 shows a portion of the solver output. Between the entry into the time loop and the first calculations, the code is read from `controlDict` and pasted into a template of a coded functionObject.

```
Starting time loop

Using dynamicCode for functionObject extraInfo at line 69 in "/home/user/OpenFOAM/user-2.1.x/
run/twoPhaseEulerFoam/bubbleColumn/system/controlDict::functions::extraInfo"
Creating new library in "dynamicCode/average/platforms/linux64GccDP0pt/lib/
libaverage_731fed868edc5a1d75988808649ac874cf00e044.so"
Invoking "wmake -s libso /home/user/OpenFOAM/user-2.1.x/run/twoPhaseEulerFoam/bubbleColumn/
dynamicCode/average"
wmakeLnInclude: linking include files to ./lnInclude
```

¹⁴⁸The release notes of OpenFOAM-2.0.0 suggest that this feature was introduced with version 2.0.0. See <http://www.openfoam.org/version2.0.0/>


```

Making dependency list for source file functionObjectTemplate.C
Making dependency list for source file FilterFunctionObjectTemplate.C
'/home/user/OpenFOAM/user-2.1.x/run/twoPhaseEulerFoam/bubbleColumn/dynamicCode/average/../../
platforms/linux64GccDP0pt/lib/libaverage_731fed868edc5a1d75988808649ac874cf00e044.so' is
up to date.
Courant Number mean: 1.68517e-05 max: 0.00363
Max Ur Courant Number = 0.00363
Time = 0.001

MULES: Solving for alpha1

```

Listing 333: On the fly compilation of C++ coded functionObjects

OpenFOAM creates a directory named `dynamicCode` in the case directory. There, all files related to the coded functionObject can be found, source files as well as binaries. Figure 114 shows the directory tree after OpenFOAM compiled the coded functionObject.

```

caseDirectory
├── 0
├── constant
│   └── polyMesh
├── dynamicMesh
│   ├── average
│   │   ├── lnInclude
│   │   ├── Make
│   │   └── linux64GccDP0pt
│   ├── platforms
│   │   ├── linux64GccDP0pt
│   │   └── libs
└── system

```

Figure 114: Directory tree after compilation of a coded functionObject

49.11 wallHeatFlux

The `wallHeatFlux` function object can be used to determine the wall heat flux. By default, `wallHeatFlux` computes the wall heat flux for all patches of the type `wall`. Alternatively a list of patches can be passed to the function object using the `patches` keyword.

Determining the physical units

In Line 7 of Listing 334, we see that the wall heat flux field is initialized with the physical unit kilogram per cubic second. In Line 17, we see that the wall heat flux is computed from the product of the thermal diffusivity for the enthalpy and the gradient of the `he`-field¹⁴⁹.

$$[\text{wallHeatFlux}] = [\alpha] \cdot [\text{he.snGrad}()] \quad (161)$$

$$[\text{wallHeatFlux}] = \frac{\text{kg}}{\text{m s}} \cdot \frac{\text{J}}{\text{kg m}} \frac{1}{\text{m}} = \frac{1}{\text{m s}} \cdot \frac{\text{W s}}{\text{m}} = \frac{\text{W}}{\text{m}^2} \quad (162)$$

The equations above, demonstrate the how to derive the physical unit of the resulting wall heat flux computed by this function object. Indeed, the result is Watt per square meter. Note, that the `alpha`-field is the thermal diffusivity of the enthalpy¹⁵⁰.

```

1 tmp<volScalarField> twallHeatFlux
2 (

```

¹⁴⁹With compressible solvers, the user can choose to either solve the energy for the specific enthalpy `h` or the specific internal energy `e`. The `he`-field is a convenient placeholder for either one. Note that the specific enthalpy and the specific internal energy both have the physical unit of Joule per kilogram.

¹⁵⁰You can find the definition of `alpha` in the files `basicThermo.H` and `basicThermo.C`

```

3     volScalarField::New
4     (
5         type(),
6         mesh_,
7         dimensionedScalar(dimMass/pow3(dimTime), 0)
8     )
9 );
10
11 /* some code removed for brevity */
12
13 forAll(wallHeatFluxBf, patchi)
14 {
15     if (!wallHeatFluxBf[patchi].coupled())
16     {
17         wallHeatFluxBf[patchi] = alphaBf[patchi]*heBf[patchi].snGrad();
18     }
19 }

```

Listing 334: The computation of the wall heat flux in `wallHeatFlux.C`

The `wallHeatFlux` function object, not only computes the wall heat flux as a field, it also prints the extrema and the integral-value per patch to the Terminal, and also writes those values into a data file. The integral value is of course the amount of energy transferred per-patch in terms of Watts.

49.12 Execute *functions* after a simulation has finished

49.12.1 *execFlowFunctionObjects*

`execFlowFunctionObjects` is a post-processing tool of OpenFOAM. This tool allows the user to execute function objects after a simulation is finished. Normally, function objects are executed during the simulation. However, in some cases it is useful to apply a function to the data set of a already completed simulation, e.g. for testing the function.

Defining function objects in a sepearte file

Listing 335 shows a file which contains only the definition of a function object. For the sake of clarity, this file is named `functionDict`. Defining functions in a sepearte file reflects the division of labor in some way. The file `controlDict` is controlling the solver, whereas the file `functionDict` defines the function objects. The file `functionDict` can be included into the file `controlDict` by an `#include` statement. See Section 11.3.5 for examples.

```

FoamFile
{
    version      2.0;
    format       ascii;
    class        dictionary;
    location     "system";
    object       functionDict;
}
// * * * * *

functions
{
    probes1
    {
        type probes;
        functionObjectLibs ("libsampling.so");
        dictionary probesDict;
    }
}

```

Listing 335: Define functions in a separate dictionary. The file `functionDict`

Run *execFlowFunctionObejcts*

execFlowFunctionObjects has to be told, that the functions are defined in a seperate file. By default, the tool reads the file `controlDict`. By using the parameter `-dict` the user can specify an alternative file containing the function dictionary.

```
execFlowFunctionObjects -noFlow -dict functionDict
```

Listing 336: Invokation of *execFlowFunctionObjects*

50 *sample*

sample is a simple post processor. This tool is controlled by the file `sampleDict`. *sample* extracts data from the solution of a specific region. *sample* can extract data from the following geometric regions:

- from one or several points in space
- along a line
- on a face

sample is usually executed after a simulation has finished.

50.1 Usage

The simplest way to use *sample* is to call the command `sample`. In this case *sample* looks for a file named `sampleDict` located in the *system* directory. With the `-dict` an alternative file with a different name can be specified. However, this file has to reside in the *system* directory.

By default *sample* operates on all time steps. The option `-latestTime` can be used to sample only the latest solution data. The option `-time` can be used to specify a certain time or a time range to operate on.

Specifying a limited number of time steps to perform sampling on significantly reduces the time needed for this operation. The disk space used by the data generated by *sample* is usually in the order of up to a few megabytes. Therefore saving hard disk space is not an issue when using *sample*.

50.2 *sampleDict*

The file `sampleDict` controls what and where data is to be sampled.

50.2.1 Output format

There are 6 possible output formats (*csv*, *gnuplot*, *jplot*, *raw*, *vtk*, *xmgr*). The difference between the listed formats is the way how the data is organised inside the file.

sample creates one file for scalar quantities and one for vector quantities. The names of the data files are built from the names of the sampled fields, the output format and the name of the geometric set. E.g. `lineXuniform_Ua_Ub.csv`, this file contains the velocity fields `Ua` and `Ub` along the line `lineXuniform`. The data format of the sampled data is comma seperated values (*csv*).

50.2.2 Fields

The fields that are to be sampled are listed in the list `fields`.

Invalid entries are ignored, without any warning message. In the example of Listing 337 the list of fields contains the name `banana`. However, there is no field named `banana`, so *sample* will simply ignore this entry – *sample* will not issue any warning or error message. Thus, a typo in the `sampleDict` is not that easy to find. *sample* reports no warning but the intended field is not sampled. Always double check the entries in the fields sub-dictionary for typos, especially when sampling fields with composite names, e.g. `U2Mean` or `U2Prime2Mean`.

```
// Fields to sample.
fields
(
    alpha
    banana
    Ua
    Ub
);
```

Listing 337: Fields to sample in the file `sampleDict`

50.2.3 Geometric regions

The geometric regions on which *sample* can operate are

sets A set can contain one or several points or a line. Along a line, points can be distributed in an equidistant fashion.

surfaces A surface can be defined in several ways. Possible are, among others, cutting planes or iso-surfaces.

50.2.4 Pitfalls

Missing keywords

If the keywords *sets* and *surfaces* are missing in *sampleDict*, *sample* will run without producing any error messages or any data. If in Listing 338 the word *banana* would be replaced by *sets* and *orange* by *surfaces*, *sample* would work as expected. If *sample* is called with a *sampleDict* like in Listing 338, *sample* produces no data and issues no warning.

```
setFormat raw;

surfaceFormat vtk;

formatOptions
{
    ensight
    {
        format ascii;
    }
}

interpolationScheme cellPoint;

fields
(
    p
    U
);

banana
(
    lineX1
    {
        type            uniform;
        axis            distance;
        start            (0.0015  0.5027  0.05);
        end              (0.0995   0.5027  0.05);
        nPoints          20;
    }
);

orange
(
);
```

Listing 338: Not working example of `sampleDict`

Faulty line definition

If the data along a line is to be sampled and the definition of the line is erroneous so that the line is outside the domain, *sample* will issue a warning message. Listing 339 shows an example of such a warning message. However, *sample* will not report an error and it will finish its run. So, when the output of *sample* is not checked, this might go unnoticed.

```
--> FOAM Warning :  
    From function sampledSets::combineSampledSets(..)  
    in file sampledSet/sampledSets/sampledSets.C at line 102  
    Sample set lineX0 has zero points.
```

Listing 339: Warning message of *sample* due to a faulty line definition

Insufficient write precision

In the OpenFOAM tutorials, the majority of cases use a write precision of 6 digits, which should be sufficient in most cases. However, in certain instances, a greater number of digits may be required.

When we are sampling the pressure field, it depends on whether we are using an incompressible or a compressible solver, for the 6 digits of write precision to be adequate or not. An incompressible solver uses a pressure field, which is divided by the fluid density. Furthermore, for incompressible solvers, the absolute value of the pressure is irrelevant. Thus, many cases use 0 as their reference or ambient pressure, e.g. at an outlet. With compressible solver, on the other hand, we use the pressure field and the absolute value of the pressure is relevant. Thus, the pressure field has, in many cases, the ambient pressure is 100000 Pa. This is where the write precision comes into the picture. In Listing 340 we see the result of sampling the pressure of a compressible case, when using a write precision of 6 digits. The reference pressure of 100000 Pa nearly drowns-out all information on the pressure variation in this case, as the 6 digits are nearly completely used-up to represent the ambient pressure. The little variations within the solution domain can not be fully resolved due to a lack of digits.

```
x,p  
0,100001  
0.00055,100001  
0.0011,100001  
0.00165,100001  
0.0022,100001  
0.00275,100001  
...
```

Listing 340: Sampling the pressure field of a compressible case with a write precision of 6 digits.

Even worse is the result when using only 5 digits. In this case, see Listing 341, OpenFOAM switches to scientific notation, and is only able to represent the ambient pressure. The small variation of pressure is now completely lost.

```
x,p  
0,1e+05  
0.00055,1e+05  
0.0011,1e+05  
0.00165,1e+05  
0.0022,1e+05  
0.00275,1e+05  
...
```

Listing 341: Sampling the pressure field of a compressible case with a write precision of 5 digits.

Using a sufficient number of digits ensures that the pressure variation within the solution domain can be sampled and processed further.

```
x,p  
0,100001.1161  
0.00055,100001.116  
0.0011,100001.1153
```

```
0.00165,100001.1145
0.0022,100001.1134
0.00275,100001.1123
...
```

Listing 342: Sampling the pressure field of a compressible case with a write precision of 10 digits.

Choosing a sufficient number of digits seems like really obvious solution. So, when do we run into the problem of sampling data with an insufficient number of digits? When we run a simulation using the binary write format for the field data. In this case, the pressure field is written to disk in binary format, with maximum precision. However, the sampled data will always be written to disk in ascii format. This is when we run into the troubles discussed in this section. When we simply do not care about the number of digits, because we feel save by using the binary format.

50.3 Update OpenFOAM-4

The post processing utility *sample* and others have been superseded by the tool `postProcess`, which bundles post processing tasks. Fortunately, not all is lost. All utilities that are superceded by `postProcess` issue a warning message about them being obsolete now. However, this message contains very helpful information on how to proceed. In the case of *sample*, existing `sampleDicts` can be used further with little modification.

```
sample has been superceded by the postProcess utility:
  postProcess -func sample

To re-use existing 'sampleDict' files simply add the following entries:
  type sets;
  libs ("libsampling.so");
and run
  postProcess -func sampleDict
```

Listing 343: Warning message in OpenFOAM-4 when trying to use *sample*

51 ParaView

ParaView is a graphical post-processor. This program is called by invoking the command `paraFoam`. `paraFoam` is a script that calls *ParaView* with additional OpenFOAM libraries.

51.1 Reader's choice

Post-processing OpenFOAM cases offers the choice between two variants, i.e. we can choose between the native OpenFOAM-reader of ParaView, and OpenFOAM's reader module. In general, both readers are equivalent, yet there are some differences between those two.

For this document, OpenFOAM's reader is generally used, unless when explicitly mentioning the native reader.

51.1.1 OpenFOAM's reader

OpenFOAM's reader is used when we launch ParaView using the `paraFoam` command. `paraFoam` is actually a script, which launches ParaView with the appropriate libraries.

From observation, we see that when we run the `paraFoam` command, a file gets created named `CASE_NAME.OpenFOAM`. When ParaView is closed again, this file is automatically removed. The important bit of this file, which we refer to as the case-file, is the file extension. The file itself is an empty file.

Thus, in order to use ParaView with OpenFOAM's reader module, we can simply create a file with the `*.OpenFOAM` file extension¹⁵¹, open ParaView and open this file using the `File > Open` menu.

The OpenFOAM reader module pre-dates the native reader and supports only serial or reconstructed cases. The OpenFOAM reader is considered to support all of OpenFOAM's features¹⁵².

¹⁵¹E.g. using the command: `touch foo.OpenFOAM`

¹⁵²<https://www.openfoam.com/documentation/user-guide/paraview.php>

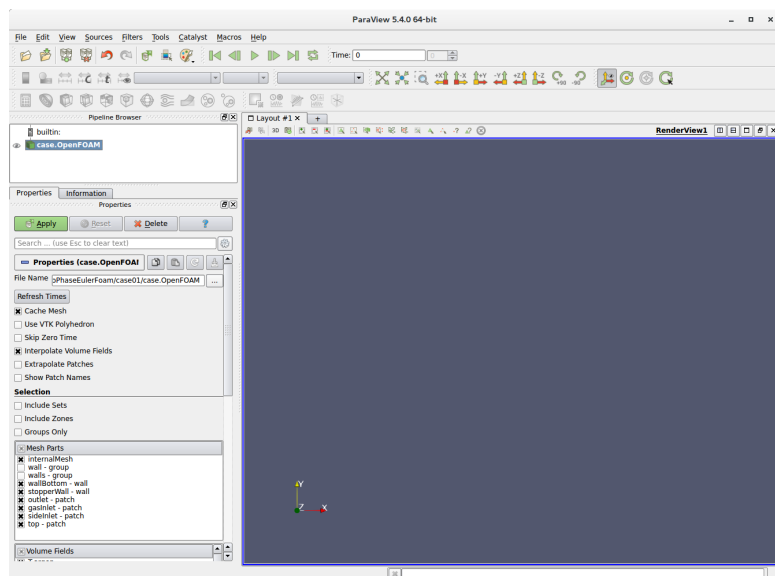


Figure 115: Launching ParaView with OpenFOAM's reader for OpenFOAM cases.

51.1.2 Native reader

If we launch ParaView, we can't simply use the **File** > **Open** menu to open an OpenFOAM case. Thus, in order to open an OpenFOAM case, we need to create a case-file with a *.foam file extension. This file can then be opened using the **File** > **Open** menu, which causes ParaView to use its native reader for OpenFOAM cases.

Decomposed cases

One major advantage of the native reader is, that it is fully parallelized. Thus, we can post-process decomposed cases without prior reconstruction of the case. Thus, we can avoid the duplication of data when post-processing an ongoing case. Figure 116 shows a screenshot, when opening a case-file for the native reader. In the properties panel on the left, there is a drop-down menu, which shows the setting *Reconstructed Case* in Figure 116. This setting can be changed to *Decomposed Case*, in order to read the parallelized case data.

The native ParaView reader is considered to potentially encounter problems with complex dictionary input.

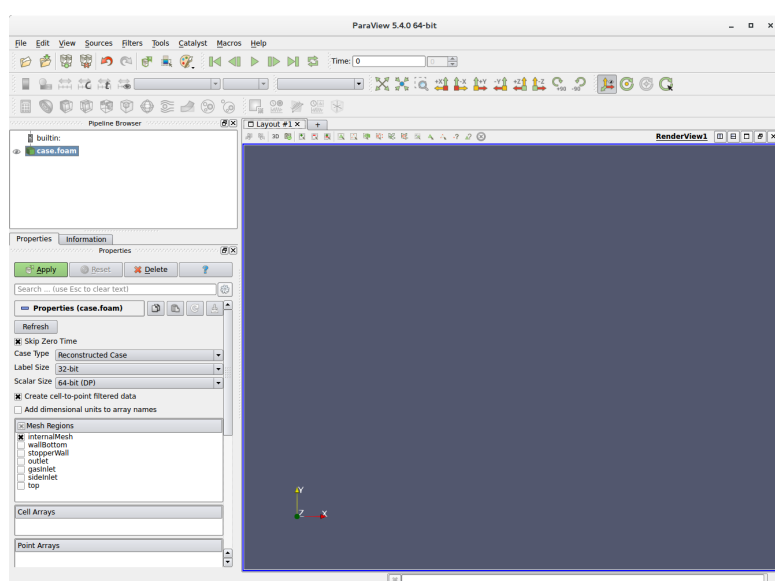


Figure 116: Launching ParaView with its native reader for OpenFOAM cases.

Multi-region cases

Another neat feature of the native reader is that it recognizes multi-region cases. Thus, by simply opening a *.foam file, the reader automatically reads-in the individual regions of the case. See Figure 117 for an example.

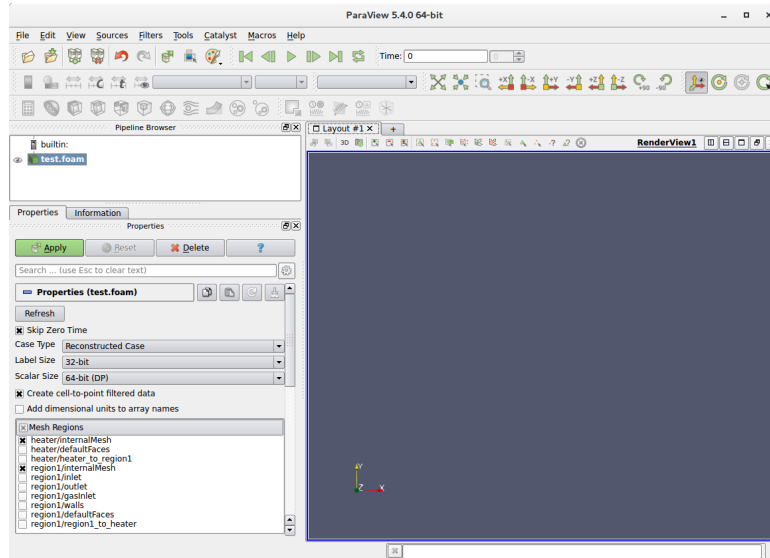


Figure 117: Using ParaView with its native reader to read a multi-region OpenFOAM case.

Pitfall: decomposed multi-region cases with constant/polyMesh still being present

One instance, the native reader struggles is when the mesh of a multi-region case has been created from a single-region mesh, and the initial mesh is still around. When such a case is decomposed, then the native reader will encounter the initial single-region mesh and the individual region-meshes in the `constant` directory, however, in the `processor*` directories, only the region-meshes are present.

In Listing 118 shows the directories of a case that triggers the problem described above. The mesh of this case has been created by subsequent execution of `blockMesh`, `topoSet` and `splitMeshRegions`; which created the initial mesh, created cellZones and split the mesh into regions according to the cellZones. Since there is no way to decompose both the initial mesh and the region-meshes, which would by the way make no sense at all, we need to remove the initial mesh in `constant/polyMesh` for the native reader to work properly.


```

.
├── 0.org
│   ├── heater
│   └── region1
├── constant
│   ├── heater
│   │   └── polyMesh
│   ├── polyMesh
│   ├── region1
│   │   └── polyMesh
├── processor0
│   ├── constant
│   │   ├── heater
│   │   └── region1
├── processor1
│   ├── constant
│   │   ├── heater
│   │   ├── region1
│   │   └── system
├── heater
└── region1

```

Figure 118: The case directories of a decomposed, multi-region case with the initial mesh still present. This directory tree was generated by calling the following command from the case directory: `tree -L 3 -d`

51.2 View the mesh

Besides viewing and post-processing simulation results, *ParaView* can be used to view only the mesh. E.g. when refining a mesh it is important to check neighbouring blocks for the transition of mesh fineness.

When no fields are selected, *paraView* only reads the mesh information. Therefore, it is possible to view the mesh without the rest of the case properly set up. After the **Apply** button has been pressed and *paraView* has read all the data, the user has to choose from the representation drop-down menu in the toolbar the option **Surface with edges**.

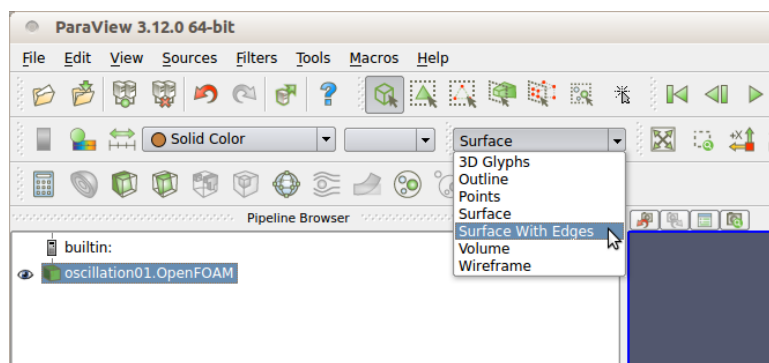


Figure 119: Select the proper representation to view the mesh

51.2.1 Sort-of-pitfall: cell shape representation

When examining meshes, we would like to get the cell shapes correctly represented. However, there seems to be an issue with ParaView's ability to display various cell shapes. Figure 120 shows an example in which the cell shape is not represented correctly. By default, ParaView seems to decompose polyhedrals into hexahedra, tetrahedra and other simple shapes.

Thus, the image shown in Figure 120 wrongly suggests a mesh consisting of mainly hexahedra, and many cells being distorted.

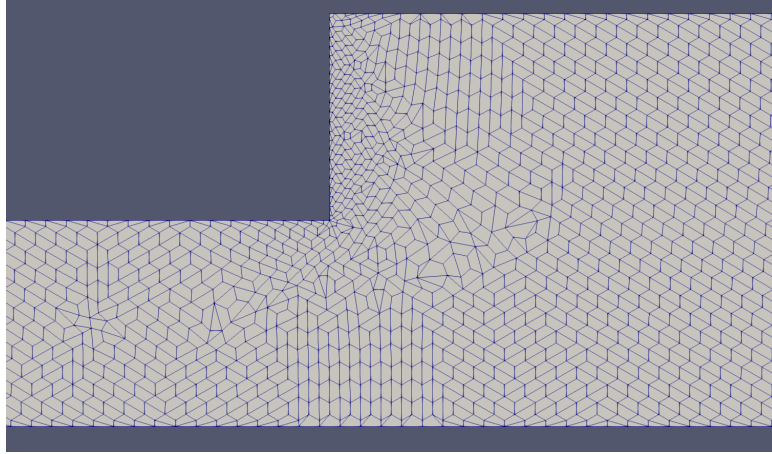


Figure 120: Viewing a polyhedral mesh with ParaView’s standard settings.

If we want to correctly display the mesh, we need to check the box labelled *Use VTK Polyhedron* in the properties panel. In Figure 115, this is the second box from the top. If we check this box, and read the case, or check this box and refresh ParaView’s state by clicking on the **Apply** button, the displayed mesh changes to the representation shown in Figure 121. Now we can clearly see the polyhedral cell shapes.

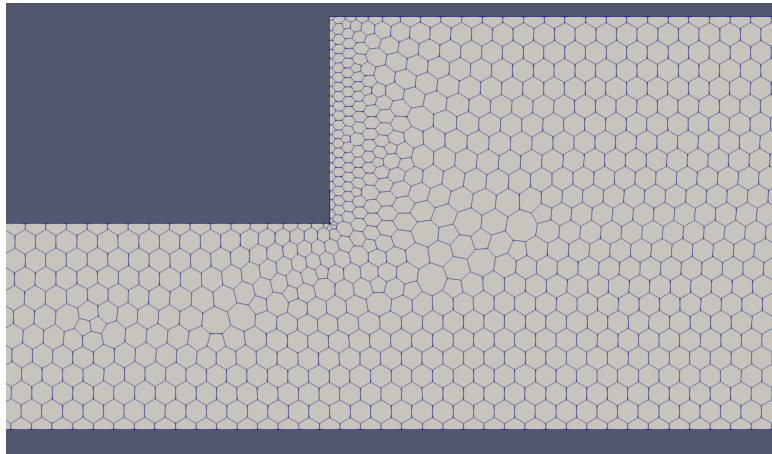


Figure 121: Viewing a polyhedral mesh with adjusted settings.

The native reader

It is the author’s assumption that the underlying cause of this representation issue has its origins in ParaView itself, because the native reader of ParaView lacks a comparable option to adjust cell shape representation. Viewing the mesh with the native reader yields the result shown in Figure 122, which is a comparable representation as in Figure 120. In both cases, the polyhedra are decomposed into hexahedra and other simple shapes, yet the decomposition differs a little.

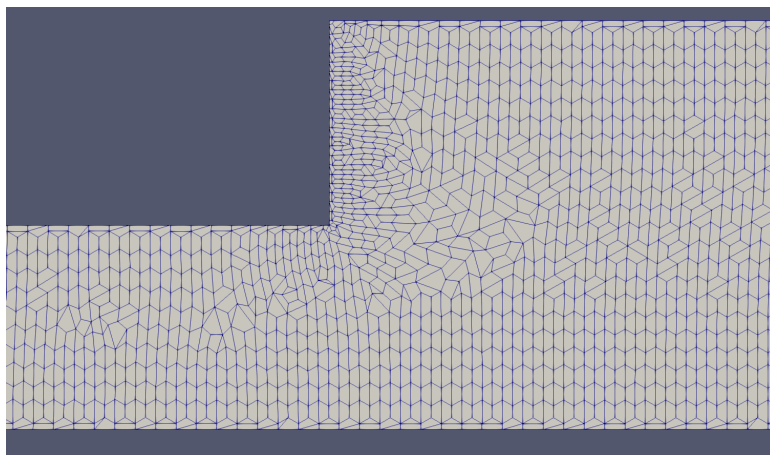


Figure 122: Viewing a polyhedral mesh with ParaView’s native OpenFOAM reader.

51.2.2 Pitfall: default field selection

If a user works on the refinement of the mesh and the definition of boundary conditions has not been made, then calling *ParaView* can crash because of its default selection of the pressure field. After pressing the **Apply** button *ParaView* tries to read in all selected fields. In case of a faulty definition of the boundary fields, this ends in the termination of the program. Listing 344 shows a corresponding error message.

```
--> FOAM FATAL IO ERROR:
keyword bottom is undefined in dictionary "/home/user/OpenFOAM/user-2.1.x/run/icoFoam/case01
/O/p::boundaryField"

file: /home/user/OpenFOAM/user-2.1.x/run/icoFoam/case01/O/p::boundaryField from line 25 to
line 35.

From function dictionary::subDict(const word& keyword) const
in file db/dictionary/dictionary.C at line 461.

FOAM exiting
```

Listing 344: Reading error due missing boundary field definition

51.3 Saving animations

51.3.1 Save as animation - video format

The straight-forward way to save animations in ParaView is to save a view as an animation in the form of a video file. Simply choose **File** > **Save Animation ...** from the menu, and select a video format¹⁵³ when ParaView asks for a file name.

¹⁵³ParaView appears to support only the OGV format, at least with the tested versions 5.4.0 and 5.6.0. For the OGV format, see e.g. <https://en.wikipedia.org/wiki/Theora> or <https://www.theora.org>.

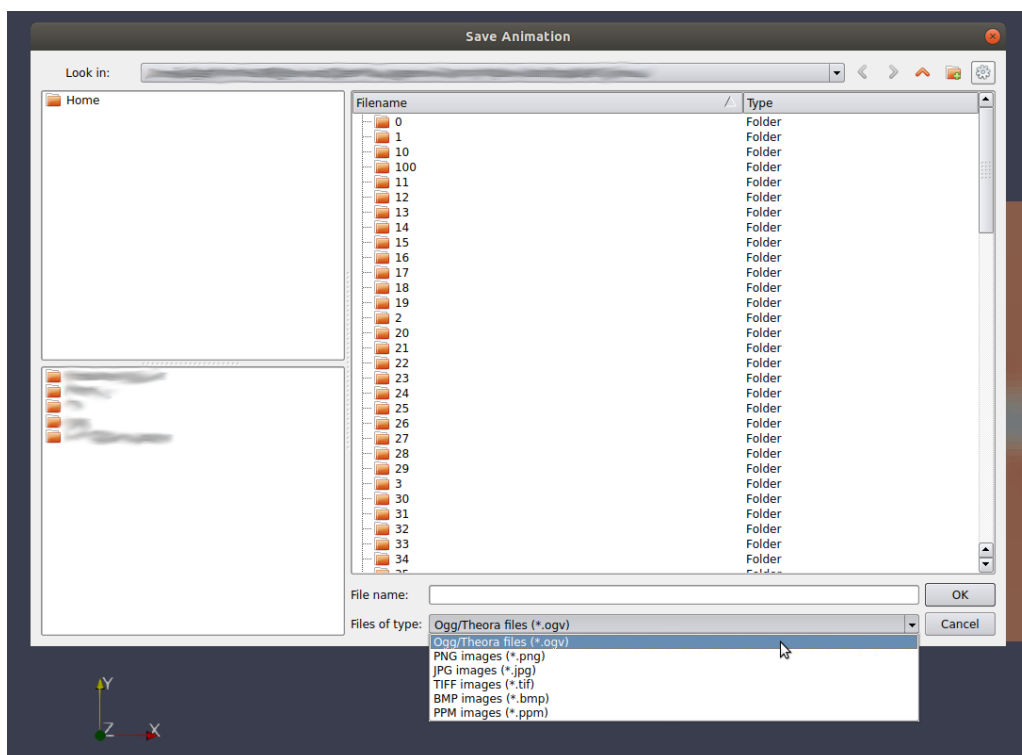


Figure 123: Selecting the file name for the animation.

51.3.2 Save as animation - individual images

Saving a view from ParaView in the form of a video file may sometimes yield unsatisfactory results, e.g. dropping individual frames. Hence, we might want an option to create an animation with more control over the animation-making process.

For this, we save the animation from ParaView as a sequence of individual images, which will be automatically numbered sequentially by ParaView. From these images, we can then create an animation using a video encoding software of our choice.

Using FFmpeg

In this example we use FFmpeg, which is a widely available, open source multimedia library. See <https://en.wikipedia.org/wiki/FFmpeg> and <https://ffmpeg.org> for more information. Listing 345 shows an example of how an animation can be created from the individual images saved by ParaView. Here, we assume that we saved our view under the file name `velField.png`, ParaView then automatically inserts sequential numbering. In the call to FFmpeg, we specify a frame rate in frames per second using the `-r` command line argument.

With the command line argument `-i` we specify the input for our animation. Here, we use the file name we used earlier to save the individual frames, however, we account for the sequential numbering by inserting a place-holder, i.e. `%04d`, which stands for a zero-padded integer with a fixed width of 4 characters. FFmpeg will insert sequential numbers starting from 0.

If we wanted to start our animation from a frame other than the one named `velField.0000.png`, we can provide an alternative starting number with the command line argument `-start_number`.

The last argument of the command in Listing 345 is the name of the resulting video file.

```
# change to the directory containing the individual images
user@host:~$ ls
velField.0000.png velField.0012.png velField.0024.png velField.0036.png velField.0048.png
... # output deleted for clarity
user@host:~$ ffmpeg -r 1 -start_number 19 -i velField.%04d.png velocityField.ogv
```

Listing 345: Creating an animation from individual images.

52 postProcess

With OpenFOAM-4.0 the function object framework was rewritten. In the course of this rewrite a `postProcess` utility was introduced and a `postProcess` option was added to most of the solvers¹⁵⁴. The `postProcess` utility also supersedes certain post-processing utilities, e.g. *sample*.

52.1 Usage

52.1.1 Pre-configured function objects

There are a number of pre-configured function objects, which are ready to use with `postProcess`. These can be found in `$FOAM_ETC/caseDicts/postProcessing`. They can also be listed using the `-list` option of `postProcess`.

```
user@host:~$ postProcess -list
```

Listing 346: Determine the extrema by magnitude of a velocity field

52.1.2 Passing parameters

Listing 347 shows how to determine the minimum and maximum magnitude of a velocity field.

```
user@host:~$ postProcess -fields '(U.water)' -func "minMaxMagnitude(U.water)"
```

Listing 347: Determine the extrema by magnitude of a velocity field

Listing 348 shows how to process two velocity fields at once, we simply pass the names of the fields in a comma-separated list.

```
user@host:~$ postProcess -fields '(U.air U.water)' -func "minMaxMagnitude(U.air,U.water)"
```

Listing 348: Determine the extrema by magnitude of two velocity fields

In Listing 349 we process a single velocity field and pass additional parameters to the function object. In this case, we do not want to know the location of the minimum and maximum velocity magnitude.

```
user@host:~$ postProcess -fields '(U.air)' -func "minMaxMagnitude(U.air,location=off)"
```

Listing 349: Determine the extrema by magnitude of a velocity field with no location

After running our function objects, we see that data was written into the `postProcessing` directory. We note that the folder names correspond to the argument passed via the `-func` option.

```
user@host:~$ ls postProcessing
minMaxMagnitude(U.air,location=off)  minMaxMagnitude(U.air,U.water)
```

Listing 350: The contents of the `postProcessing` directory after running two function objects from the above listings

52.1.3 Post-processing decomposed cases

We can run `postProcess` in parallel in the same way we run a solver in parallel. This allows us to post-process a case, which has not been reconstructed.

```
mpirun -np 4 postProcess -parallel
```

Listing 351: Run `postProcess` in parallel, i.e. post-process a decomposed case.

¹⁵⁴Counting the solvers with `find $FOAM_SOLVERS -name 'files' | xargs grep 'EXE' | wc` yielded a number of 82 solvers, of which 73 included the `postProcess.H` header file, which provided the `postProcess` option. The second number was determined with the following command: `find $FOAM_SOLVERS -name '*.C' | xargs grep '\#include[[:space:]]\"postProcess.H\"' | wc`.

Part VIII

External Tools

Besides *paraView*, there are a number of other useful tools, which do not come from the OpenFOAM Foundation. This section will cover such tools.

53 *pyFoam*

pyFoam is a collection of useful Python¹⁵⁵ scripts. These scripts are mostly written to serve one specific task. Further information can be found at http://openfoamwiki.net/index.php/Contrib_PyFoam.

53.1 Installation

The installation of *pyFoam* is described at http://openfoamwiki.net/index.php/Contrib_PyFoam#Installation. The major prerequisite for the use of *pyFoam* is, that a Python interpreter is installed. To check if a Python interpreter is installed on the system, simply type `python --version` in the Terminal. If a version number is displayed, like `Python 2.7.3`, then Python is installed. Otherwise, the operating system would display an error message, stating that the command `python` can not be found.

Further information about Python are found at <http://python.org/> and <http://docs.python.org/>.

53.2 *pyFoamPlotRunner*

The script *pyFoamPlotRunner* starts a simulation and plots the residuals like Fluent would do.

```
user@host:~/OpenFOAM/user-2.1.x/run/twoPhaseEulerFoam/columnCase$ pyFoamPlotRunner.py
twoPhaseEulerFoam
```

Listing 352: Calling *pyFoamPlotRunner*

53.2.1 Plotting options

Listing 353 shows the plotting options offered by *pyFoam*.

```
What to plot
-----
Predefined quantities that the program looks for and plots

--no-default          Switch off the default plots (linear, continuity and
                        bound)
--no-linear            Don't plot the linear solver convergence
--no-continuity        Don't plot the continuity info
--no-bound             Don't plot the bounding of variables
--with-iterations      Plot the number of iterations of the linear solver
--with-courant         Plot the courant-numbers of the flow
--with-execution      Plot the execution time of each time-step
--with-deltat          'Plot the timestep-size time-step
--with-all            Switch all possible plots on
```

Listing 353: Plotting flags of the *pyFoamPlot** utilities

53.3 *pyFoamPlotWatcher*

The script *pyFoamPlotWatcher* is intended to visualize solution data (e.g. residuals, time steps, Courant number, etc.) after the simulation has finished. This requires that the solver output is written into a file, see Section 12.1.1. *pyFoamPlotWatcher* does essentially the same job as *pyFoamPlotRunner* with the difference that the former tool is for finished simulations and the latter monitors a running simulation. So the description of the features of *pyFoamPlotWatcher* holds also true for *pyFoamPlotRunner*.

¹⁵⁵Python is an interpreted programming language.

```
user@host:~/OpenFOAM/user-2.1.x/run/twoPhaseEulerFoam/columnCase$ pyFoamPlotWatcher.py LOGFILE
```

Listing 354: Calling *pyFoamPlotWatcher*

By default *pyFoamPlotWatcher* plots the curves of the residuals, continuity information and bounded variables. With options several other curves can be plotted (e.g. time step, iterations, Courant number, etc.). With regular expressions user specified data can be extracted from the log file.

Listing 355 shows the invocation of *pyFoamPlotWatcher* to plot additionally to the default selection also the Courant number. The processing of the solver output stored in the file `LOGFILE` is limited with the option `--end` with a specific value – 0.1 s in this case. There is also a `--start` option. The plot created by the command in Listing 355 is shown in Figure 124.

```
pyFoamPlotWatcher.py LOGFILE --end=0.1 --with-courant
```

Listing 355: Calling *pyFoamPlotWatcher* with some options

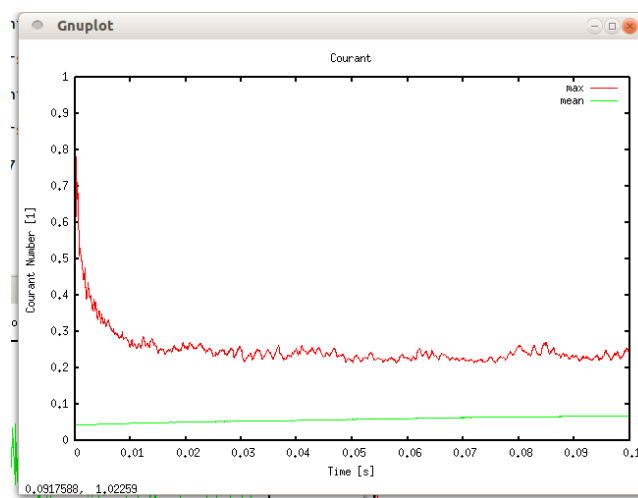


Figure 124: The Courant number plotted with *pyFoamPlotWatcher*.

53.3.1 Custom regular expressions

With regular expressions *pyFoamPlotWatcher* can extract arbitrary data from the solver output. This section elaborates this feature by the example of plotting the Courant number based on the relative velocity of a two-phase solver.

General information

pyFoamPlotWatcher has no option to display the history of the Courant number based on U_r , the relative velocity between the phases. Listing 356 shows some lines of the solver output of the two-phase solver *twoPhaseEulerFoam*. The line in red displays the Courant number based on the relative velocity U_r . The line above the red colored line displays the Courant number based on the mixture velocity, see Section 57.6.4 and 57.6.4 for information on the definition of the Courant number and the Courant number of the two-phase solver *twoPhaseEulerFoam*.

```
DILUPBiCG: Solving for k, Initial residual = 0.000824921, Final residual = 1.47595e-06, No
Iterations 2
ExecutionTime = 70870.7 s ClockTime = 71186 s

Calculating averages

Courant Number mean: 0.103485 max: 0.422517
Max Ur Courant Number = 0.448791
deltaT = 0.00380929
```

```
Time = 72.5848
MULES: Solving for alpha1
MULES: Solving for alpha1
```

Listing 356: Some lines of the solver output of *twoPhaseEulerFoam*

Extracting the information

To extract the information from the log file we need to create a file containing the regular expression.

```
{"expr":"Max Ur Courant Number = (%f%)","name":"UrCoNum"}
```

Listing 357: The file *customRegexp*

If *pyFoamPlotWatcher* finds a file named *customRegexp* in the case directory, this file will be processed automatically. If the file containing the regular expression has another name or is located in another place the option `--regexp-file=REG_EXP_FILE` can be used to specify the path to that file.

Listing 357 contains comma separated entries ("expr" and "name"). The values are separated by a colon from the name of the entries (e.g. "name":"UrCoNum"). The first entry contains the regular expression to extract the data. The second provides the name of the extracted data, but this entry can be omitted.

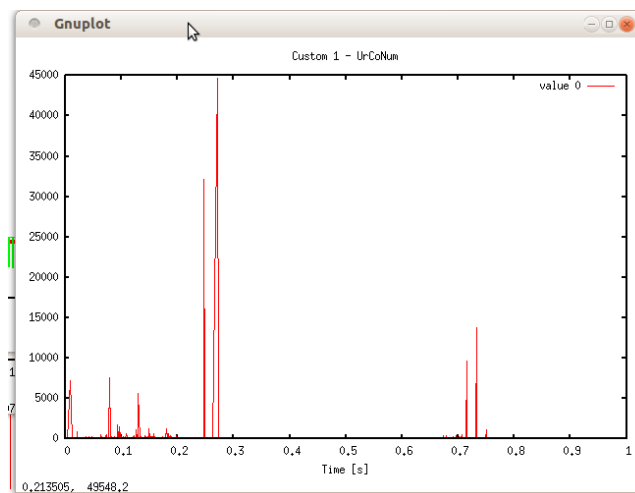


Figure 125: The Courant number based on the relative velocity plotted with *pyFoamPlotWatcher*

The absurdly high value of the Courant number indicates that the simulation did not go well. The need for plotting the Courant number based on *Ur* emanated from a trouble-shooting episode. Thus this section was written to preserve the gained knowledge.

53.3.2 Custom regular expression revisited

The plotting utilities of *pyFoam* (*pyFoamPlotRunner* and *pyFoamPlotWatcher*) accept custom regular expressions also in a different format than the format of Listing 357. This new format was introduced with version 0.5.3. See http://openfoamwiki.net/index.php/Contrib_PyFoam#Plotting_with_customRegexp-files for further information. The new format looks resembles an OpenFOAM dictionary.

Listing 358 shows an example of the solver output that will be post-processed. The goal is to draw curves of the quantities of the red line. Listing 359 shows the corresponding regular expression. The plotting utilities of *pyFoam* offer the `--dump-custom-regegeexp` option to generate the custom regular expression in the new format from the old format. Listing 360 is the result of this operation.

```
DILUPBiCG: Solving for beta, Initial residual = 0.000307666, Final residual = 7.36162e-08, No
Iterations 2
```



```
DILUPBiCG: Solving for T, Initial residual = 0.000514273, Final residual = 2.57279e-07, No
Iterations 1
Concentration = 0.0509085 Min T = 0.00498731 Max T = 0.218343
Bubble load = 0.00623198 Min beta = 0 Max beta = 0.0677904
Time = 19.96
```

Listing 358: Some lines of the solver output to post-process

```
{"expr":"Concentration = (%f%) Min T = (%f%) Max T = (%f%)","name":"Concentration","titles":
["avg","min","max"]}
```

Listing 359: The custom regular expression in the odl format

```
Custom01
{
    accumulation first;
    enabled yes;
    expr "Concentration = (%f%) Min T = (%f%) Max T = (%f%)";
    name Custom01_Concentration;
    persist no;
    raisit no;
    theTitle "Custom 1 - Concentration";
    titles
    (
        avg
        min
        max
    );
    type regular;
    with lines;
    xlabel "Time [s]";
}
```

Listing 360: The custom regular expression in the new format

53.3.3 Special treatment of certain characters

Note that the solver output we processed so far contained no parentheses. The parentheses are interpreted by the regular expression. In order to deal with parentheses in the solver output they need to be escaped properly. The same is true for brackets. So the following example is also valid, when brackets are contained in the solver output that is to be processed with regular expressions.

Listing 361 shows some lines of solver output of `twoPhaseEulerFoam`. The line marked in red contains parentheses. In order to post-process these lines with regular expressions these parentheses need to be escaped in the regular expression. Listing 362 shows the corresponding regular expression. Note the escaped parentheses marked in red.

```
Time = 19.9957

MULES: Solving for alpha1
MULES: Solving for alpha1
Dispersed phase volume fraction = 0.0168317 Min(alpha1) = 3.92503e-87 Max(alpha1) = 0.2
GAMG: Solving for p, Initial residual = 9.46269e-05, Final residual = 1.65711e-06, No
Iterations 1
time step continuity errors : sum local = 2.08826e-05, global = 4.51574e-08, cumulative =
-0.0334048
```

Listing 361: Some lines of the solver output of *twoPhaseEulerFoam*

```
{"expr":"Dispersed phase volume fraction = (%f%) Min\\(alpha1\\) = (%f%) Max\\(alpha1\\) = (%f%)
","name":"Volume fraction","titles":["avg","min","max"]}
```

Listing 362: The regular expression to extract the information about the volume fraction

Not only the parentheses have a special meaning in regular expressions. An internet search¹⁵⁶ or detailed knowledge on regular expressions will yield the knowledge which characters have to be escaped.

¹⁵⁶E.g. <http://stackoverflow.com/questions/399078/what-special-characters-must-be-escaped-in-regular-expressions>

53.3.4 Ignoring stuff

Listing 362 extracts three numbers from the line marked in Listing 361. Using this regular expression plots all three curves. If we are interested in only the first number – the average volume fraction – we replace the second and third (`%f%`) with a `.*` to ignore the second and third number. In this special case this seems an overkill – we could also delete parts of the expression since we are only interested in the first number – but if we are interested in the first and the third number, then we need to ignore the second number.

53.3.5 Producing images

The Figures 124 and 125 are screenshots of the images plotted by *pyFoamPlotWatcher*. However, there is the option `--hardcoded` that tells the *pyFoam* plot utilities to save the plots on the disk. By default a PNG image is produced but with the option `--format-of-hardcopy=HARDCOPYFORMAT` other formats can be chosen.

Figure 126 shows the plot produced by the regular expression of Listing 362.

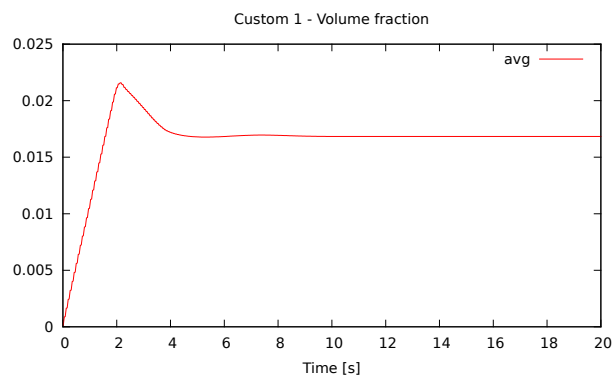


Figure 126: The average volume fraction plotted with *pyFoamPlotWatcher* and a custom regular expression

53.3.6 Writing data

Producing images is often not enough for post-processing. The option `--write-files` causes *pyFoam* to write the extracted data to the hard drive. Thus the extracted data can be processed by other programs.

53.3.7 Case analysis

The option `--with-all` generate a number of plots that can be helpful to examine the performance of simulation case. See Listing 353 for an explanation of the available plots.

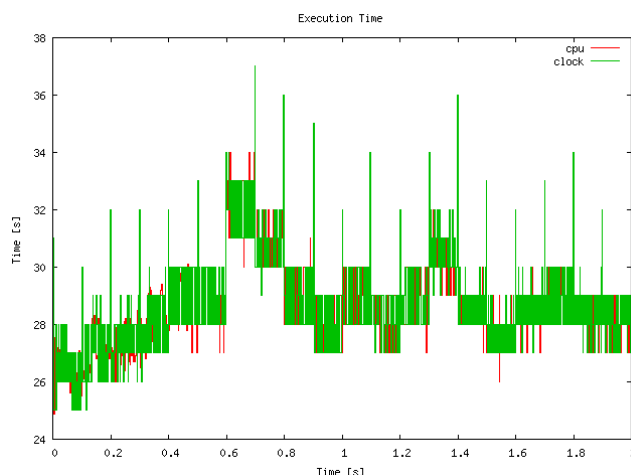


Figure 127: The execution time plotted over time with *pyFoamPlotWatcher*. The occasional writing of the data to harddisk are clearly visible as spikes in the execution time.

53.4 *pyFoamClearCase*

As the name implies, *pyFoamClearCase* cleans the case directory. This script deletes all time directories save the 0 directory. By the use of command line options, a finer control of the actions of *pyFoamClearCase* is possible. Some of these options are:

- keep-last** keep the last time step
- keep-regular** keep all time steps
- after=T** delete all time steps for $t > T$
- remove-processor** delete the *processor** directories

The script is invoked by typing its name in the Terminal. Listing 363 shows how this script is executed. The options cause *pyFoamClearCase* to keep the last time directory and to remove all *processor** folders.

```
pyFoamClearCase.py . --keep-last --remove-processor
```

Listing 363: Calling *pyFoamClearCase*

Note the file ending *.py* after the name of the script. This ending indicates, that the script is written in Python. It also indicates, that *pyFoamClearCase* is an executable script rather than a program on its own.

53.5 *pyFoamCloneCase*

This script is used to copy a case. By default the 0, the *constant* and the *system* directory are copied. Additionally, there are various command line arguments to control the operation of the script, e.g. copy also the latest time step or the *processor** directories.

53.6 *pyFoamDecompose*

This script is used to decompose the computational domain. Other than the tool *decomposePar*, this script does not need an existing *decomposeParDict*. This script receives command line arguments, generates the *decomposeParDict* and calls *decomposePar*.

In Listing 364 the script is called with two arguments. The first argument is the path to the case directory. In this case the dot refers to the current directory. The second argument is the number of sub-domains. From this arguments, *pyFoamDecompose* creates a *decomposeParDict*. The first argument is necessary to tell the script where to save the newly created file. The second argument is the most fundamental information for domain decomposition – the number of sub-domains.

There is a large number of additional arguments which allow to exert more control over the way the domain is decomposed.

```
pyFoamDecompose.py . 4
```

Listing 364: Invocation of *pyFoamDecompose*

Listing 365 contains the `decomposeParDict` created by the command of Listing 364.

```
// * * * * * //
FoamFile
{
    version 0.5;
    format ascii;
    root "ROOT";
    case "CASE";
    class dictionary;
    object nix;
}
method scotch;
numberOfSubdomains 4;
scotchCoeffs
{
}
```

Listing 365: The file `decomposeParDict` generated by *pyFoamDecompose decomposeParDict*

The output of *pyFoamDecompose* is stored in the file `Decomposer.logfile`.

53.7 *pyFoamDisplayBlockMesh*

If there is a problem with mesh topology and one isn't able to find the error in the *blockMeshDict*, this tool can be of great help. *pyFoamDisplayBlockMesh* does exactly what the name of the tool suggests. It reads *blockMeshDict* and displays the topology of the mesh. One might think, that that's exactly what is described in Section 15.6.2 (display the blocks with *paraView*). However, if the definition of the mesh is erroneous, *blockMesh* will not create a mesh and *paraView* is therefore not able to display the blocks.

pyFoamDisplayBlockMesh is a tool that allows the user to visualise a faulty mesh. This is of great help to find e.g. an error in the block definition, especially when there are more than one blocks. In Figure 128 a screenshot of the GUI of this tool is shown. In the main panel the vertices and the edges are displayed. With the two sliders below single blocks as well as patches can be marked and coloured. The local axes of a single block are displayed as tubes labelled with the corresponding names of the axes.

The blocks shown in Figure 128 have a faulty definition, so *blockMesh* produces an error message instead of creating a mesh. With the help of this tool, the cause for the error is easily found. The marked block should be in the right part of the geometry, so vertex number 5 should not be part of this block.

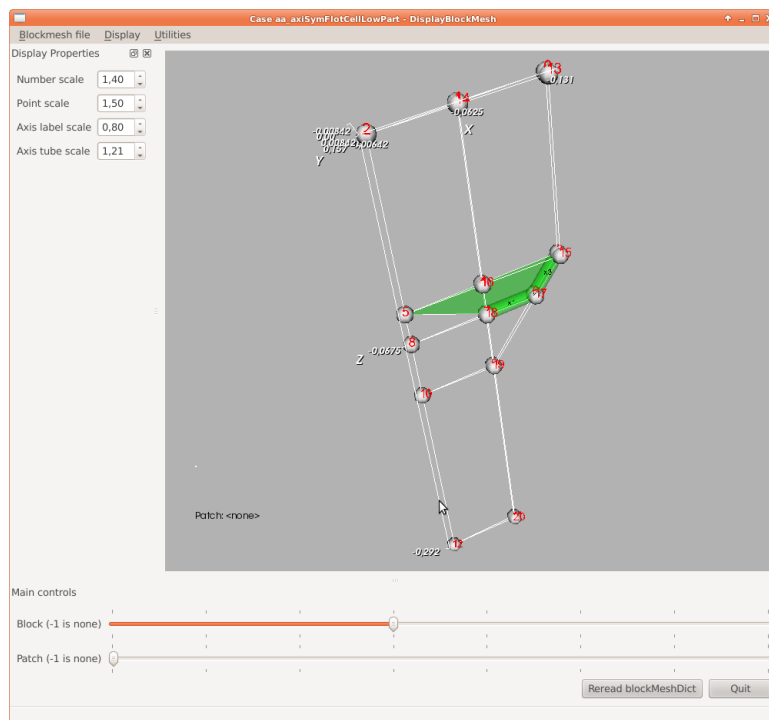


Figure 128: Screenshot of *pyFoamDisplayBlockMesh*

Right of the main panel the output of the standard meshing utilities *blockMesh* and *checkMesh* can be displayed (not shown in the picture). These utilities can be executed from the menu of this tool. Moreover, the *blockMeshDict* can be edited with this tool.

53.8 *pyFoamCaseReport*

The tool *pyFoamCaseReport* generates a summary of the simulation case. The amount of information displayed can be controlled by command line flags. Listing 366 shows how to create a full summary of a case. However, the full information lies within the dictionaries of the case. This tool provides only selected information.

```
pyFoamCaseReport.py --full-report .
```

Listing 366: Create a summary of the case with *pyFoamCaseReport*

54 *swak4foam*

The name *swak4foam* comes from *SWiss Army Knife for Foam*. *swak4foam* evolved from a collection of tools like *groovyBC*, *funkySetFields* and *simpleFunctionObjects*. The documentation of *swak4foam* is located at <http://openfoamwiki.net/index.php/Contrib/swak4Foam>.

54.1 Installation

To install *swak4foam* one needs to download the source code and compile them. The source code of *swak4foam* is managed by the use of a *subversion*¹⁵⁷ repository. Listing 367 shows how the source code is downloaded by subversion. The first command changes the working directory of the terminal to `~/OpenFOAM`. The second command creates a directory named `swak4foam`. The third command changes the working directory of the terminal to the newly created folder and the last commands actually downloads the source code to the current directory.

¹⁵⁷*subversion*, abbreviated SVN, is a version control software to manage software projects.

```
cd ~/OpenFOAM
mkdir swak4foam
cd swak4foam
svn checkout https://openfoam-extend.svn.sourceforge.net/svnroot/openfoam-extend/trunk/
Breeder_2.0/libraries/swak4Foam/
```

Listing 367: Installation of *swak4foam*

After downloading, the sources need to be compiled by calling `Allwmake`.

54.2 *simpleSwakFunctionObjects*

simpleSwakFunctionObjects is an extension of *simpleFunctionObjects*. The functions of this library are used to post process data and extend functionality of OpenFOAM.

54.2.1 Extrema of a field quantity

If only the extrema of a field quantity are of interest, the tools of OpenFOAM (*probes*, *sample*) are of little use. One way of solving this problem could be, to modify the solver to write the extrema to the standard output. In Listing 368 some line of the standard output of *twoPhaseEulerFoam* are shown. This solver prints the mean value as well as the extrema of the volume fraction of the dispersed phase. The corresponding lines of source code can serve as a blueprint for a solver modification.

However, if the user is not inclined to modify and compile OpenFOAM solvers, *simpleSwakFunctionObjects* provide the solution.

```
DILUPBiCG: Solving for alpha, Initial residual = 3.48391e-05, Final residual = 2.94111e-12,
No Iterations 2
Dispersed phase volume fraction = 0.00824276 Min(alpha) = -1.66816e-19 Max(alpha) = 0.6
DILUPBiCG: Solving for alpha, Initial residual = 3.71563e-07, Final residual = 8.16115e-14,
No Iterations 2
Dispersed phase volume fraction = 0.00824276 Min(alpha) = -3.31819e-19 Max(alpha) = 0.6
```

Listing 368: Solver-Ausgabe von *twoPhaseEulerFoam*

swakExpression

The function to do the job is called *swakExpression*. This function is part of the library *libsimpleSwakFunctionObjects*. Listing 369 shows how this function is set up as a function object in the file `controlDict`. In this example the minimal value of the field `alpha` is saved. Notice the statement in last line of the Listing. This statement tells the solver to use the specified library. This library contains the function `swakExpression`. See Section 11.3.3 for further information about using external libraries.

```
functions
{
    minAlpha
    {
        type swakExpression;
        verbose true;
        accumulations ( min );
        valueType internalField;
        expression "min(alpha)";
    }
}

libs ("libsimpleSwakFunctionObjects.so");
```

Listing 369: Definition of the function *swakExpression* in the file `controlDict`

Keywords

This section explains the most important keywords of Listing 369.

type specifies the type the function object

verbose a switch that controls whether the generated data is to be printed on the solver output or not. The data is written into a file anyway.

accumulations allowed entries: {min,max,average,sum}. Quote from the CFD-Online Forum¹⁵⁸: *accumulations is only needed if you need "a single number" to print to the screen. For instance if you use a swakExpression-FO to print the maximum and minimum of your field to the screen.*

valueType defines the type of the geometric region on which the function is applied. Allowed entries: {internalField cellSet faceZone patch faceSet set surface cellZone}

expression defines the quantity that is sought for. This can be a simple statement or a formula computing a quantity.

55 blockMeshDG

blockMeshDG is a modification of the meshing tool *blockMesh* to allow for double grading. Double grading means, that the ratio between the discretisation length of the middle and the ends of an edge is prescribed. This tool was developed by some users of OpenFOAM and is was published in the CFD-Online OpenFOAM Forum (<http://www.cfd-online.com/Forums/openfoam/70798-blockmesh-double-grading.html>). There is also a page in the OpenFOAM Wiki (http://openfoamwiki.net/index.php/Contrib_blockMeshDG).

Notice

This topic is outdated, as *blockMesh* of standard OpenFOAM already includes a feature for the discussed purpose. This was introduced with OpenFOAM-3.0¹⁵⁹.

55.1 Installation

The downloaded source code is ready for compilation after unpacking. All necessary entries have already been made to prevent the new utility to collide with the standard utilities of OpenFOAM. The make script creates an executable named *blockMeshDG*.

55.2 Usage

To discern between normal grading and double grading, the expansion ratio needs to be negative for double grading¹⁶⁰. A positive entry causes normal grading to be applied just like it is the case with the standard utility.

55.3 Pitfalls

55.3.1 Uneven number of cells

blockMeshDG obviously has a problem with an uneven number of cells. Figure 129 shows the resulting mesh, when 15 cells are used for the double graded edge. In this case, although the mesh is of bad quality, *checkMesh* reports no error. However, the output of *checkMesh* contains some indications that something is not alright.

Listing 370 shows some lines of the output of *checkMesh*. The very high aspect ratio is an indicator that something is wrong with the mesh. Also the fact that the minimum and maximum values of face area or cell volume differ by up to three orders of magnitude should lead to the same conclusion. Unfortunately, *checkMesh* issues not even a warning message.

¹⁵⁸<http://www.cfd-online.com/Forums/openfoam/103504-swak4foam-calculating-velocity-transformations.html>

¹⁵⁹<https://openfoam.org/release/3-0-0/>

¹⁶⁰A negative entry unequal to unity causes *blockMesh* to crash with a floating point exception. Therefore, using negative entries for double grading does not alter the standard behaviour.

Checking geometry...

```
Max aspect ratio = 81 OK.  
Minimum face area = 3.8395e-08. Maximum face area = 1.68746e-05. Face area magnitudes OK.  
Min volume = 9.59875e-11. Max volume = 4.21864e-08. Total volume = 4.92214e-05. Cell  
volumes OK.  
Mesh non-orthogonality Max: 42.2304 average: 11.7938  
Non-orthogonality check OK.  
Min/max edge length = 3.079e-05 0.00508035 OK.
```

Listing 370: Some output of *checkMesh*

So far, the only solution to this problem is to use an even number of cells.

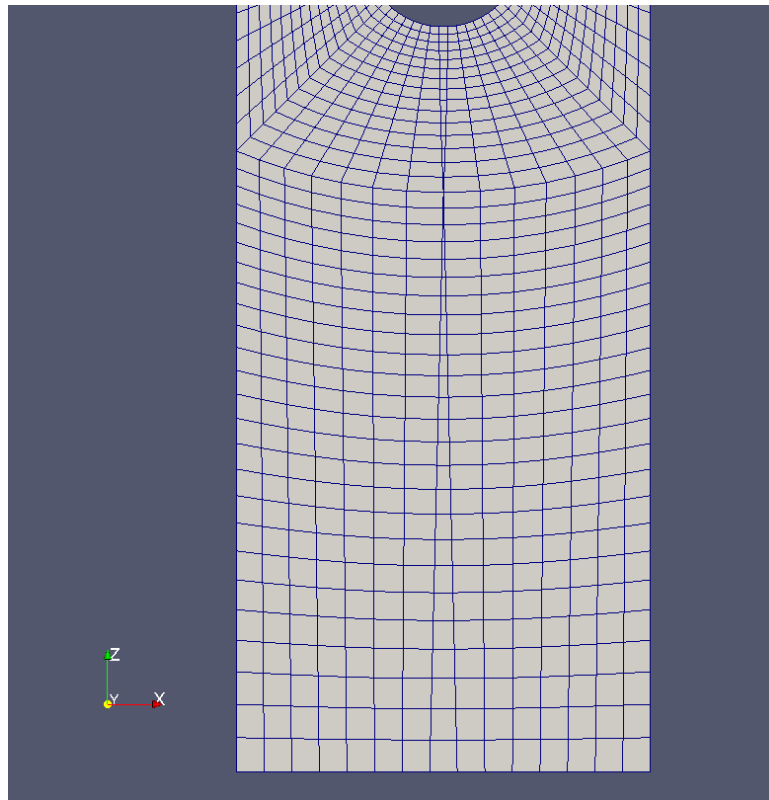


Figure 129: Double grading problem

Part IX

Source Code & Programming

56 Understanding some C and C++

In this Section some features of the C++ programming language are discussed.

56.1 Definition vs. Declaration

In C and C++ there is the distinction between the declaration and the definition of a variable. Briefly explained, declaring a variable only tells the compiler that the variable exists and has a certain type. The declaration does not specify what the variable actually is.

A definition also tells the compiler what exactly a variable is. This does not necessarily mean that the variable is assigned a value.

Further information on that matter can be found in [57, 37] or http://www.cprogramming.com/declare_vs_define.html.

56.1.1 A classy example

In Listing 371 we define the class `phaseInterface`, i.e. we tell the compiler what the class looks like (data members, methods, etc.). Within the class `phaseInterface` we want to use the class `phaseModel`. This class already exists and is defined elsewhere, so there is no need for us to repeatedly define the class `phaseModel`. Creating our own definition of `phaseModel` would be useless and stupid.

To be able to use the existing class `phaseModel` we need to introduce this class to the compiler. In Line 4 of Listing 371 we do exactly this. We tell the compiler, that there is a class named `phaseModel`, that is all the information needed by now. This is sometimes referred to as *forward declaration*.

When we compile our class we need to make sure that we include the definition of `phaseModel`, e.g. via linking to the library in which `phaseModel` is defined.

```
1 namespace Foam
2 {
3
4   class phaseModel;
5
6   class phaseInterface
7   {
8       // lots of C++ code
9   };
10
11 }
```

Listing 371: Declaration and definition of classes

56.2 Namespaces

Namespaces are a feature of C++ to support a logical structure within the program. The basic idea behind namespaces put in simple words is *to keep things (variables and functions) visible where they need to be visible*. Like any other method of keeping things neat and tidy you could also survive without namespaces. However, to loosely quote Prof. Jasak, one of the founders of OpenFOAM: *OpenFOAM is an example of how to make proper use of C++*. Therefore, we have a closer look on namespaces in OpenFOAM.

General information about the concept of namespaces can be found here:

- <http://www.cplusplus.com/doc/tutorial/namespaces/>
- <http://www.cprogramming.com/tutorial/namespaces.html>
- <http://www.learncpp.com/cpp-tutorial/711-namespaces/>

Some OpenFOAM specific aspects related to namespaces are discussed in Section 57.2.

56.3 const correctness

The `const` keyword has several uses and using `const` has some implications.

56.3.1 Constant variables

This is the most easy part. Any variable can be declared constant by using the `const` keyword. This can precede the datatype or the variable name. Both lines in Listing 372 are correct statements.

```
const int limit = 5;
int const answer = 42;
```

Listing 372: Constant variables

56.3.2 Constants and pointers

Pointing to a constant

A pointer can be used to point to a constant variable. The pointer itself is not constant and therefore changeable. However, the keyword `const` has to be used when declaring a pointer pointing to a constant variable. However, a pointer pointing to a constant can also point to a non-constant variable.

```
int const constVar1 = 42;
const int constVar2 = 13;
int variable = 11;

const int* pointer = &constVar1;

std::cout << "The pointer points to " << *pointer << std::endl;

// change the pointer
pointer = &constVar2;

std::cout << "The pointer points to " << *pointer << std::endl;

// point to a non-constant
pointer = &variable;

std::cout << "The pointer points to " << *pointer << std::endl;
```

Listing 373: Pointing to constant variables

```
The pointer points to 42
The pointer points to 13
The pointer points to 11
```

Listing 374: Output of Listing 373

A constant pointer

A pointer can be constant regardless of the variable it points to. So, the address stored in the pointer can not be changed, the pointer will always point to the same variable. However, the variable itself can be altered. Listing 375 shows an example.

```
int variable = 11;

int* const constPointer1 = &variable;

std::cout << "The constant pointer points to " << *constPointer1 << std::endl;

variable = 79;

std::cout << "The constant pointer points to " << *constPointer1 << std::endl;
```

Listing 375: Using constant pointers

```
The constant pointer points to 11
The constant pointer points to 79
```

Listing 376: Output of Listing 375

A constant pointer to a constant

It is also possible to create a constant pointer pointing to a constant variable.

However, the last line of Listing 377 seems a bit unlogical but it isn't. To get the meaning of this line correctly, we need to read the left hand side of the assignment from right to left. First of all `constpointer4` is the name of the new variable. Secondly, `int* const` tells the compiler that the new variable is a constant pointer to an integer. This means, that the pointer itself – the location it points to – can not be changed. The last statement `const` at the very beginning of the line, means, that the variable the pointer points to can not be changed. However, `variable` is not a constant, so it can be altered anyway. The last line of Listing 377 does not change the nature of the variable `variable`, but it restricts the pointer to read-only operations. So, `variable` can be changed, but not using `constPointer4`.

```
int const constVar1 = 42;
int variable = 11;

const int* const constPointer2 = &constVar1;
const int* const constPointer4 = &variable;
```

Listing 377: A constant pointer to a constant

56.4 Function inlining

Motivation

Functions that carry out only a small number of operations are not very efficient, because the function call might take more time than the execution of all the operations. Especially if such a function is often called, the performance of the program suffers. However, writing functions is a good way to keep the code tidy.

On the one hand, functions enable the programmer to separate code in a logical way. Code that is written for a specific task is outsourced into a function with a hopefully meaningful name. This improved readability and maintainability of the code.

On the other hand is writing functions a proper way to avoid code redundancy. Tasks that are carried out repeatedly are best put into a function. Therefore, the code has to be written only once and the function can be used wherever it is necessary.

The inline statement

The solution for this conflict is function inlining. The `inline` statement allows the compiler to replace the function call with the function body, i.e. the operations performed by the function. This enables the programmer to keep the code tidy without the disadvantage of wasting time for time consuming function calls.

Listing 378 shows the definition of an inline function. The function body contains only two logical operations. The `inline` statement precedes the data type of the return value. So, writing inline functions is not different than writing ordinary functions.

```
inline bool Foam::pimpleControl::finalIter() const
{
    return converged_ || (corr_ == nCorrPIMPLE_);
}
```

Listing 378: The definition of an inline function

The use of the `inline` statement does not guarantee that the compiler replaces the function call. This depends on the compiler and the compiler settings.

OpenFOAM specifics

The OpenFOAM Code Style Guide (<http://www.openfoam.org/contrib/code-style.php>) demands from programmers to separate the definition of inline and non-inline functions.

Use inline functions where appropriate in a separate *classNameI.H* file.

Listing 379 shows the contents of the folder `pimpleControl`. Dividing the code of a program or a module into *.C and the *.H file is the common way to separate declarations from the rest of the program. The *.dep file is generated by the compiler during compilation. The fourth file in the folder is a second header file as demanded by the Code Style Guide. Listing 378 is a part of `pimpleControlI.H`.

```
pimpleControl.C  pimpleControl.dep  pimpleControl.H  pimpleControlI.H
```

Listing 379: Content of the folder `pimpleControl`

56.5 Constructor (de)construction

In object oriented programming (OOP) everything is an object. All object are created by a constructor and if necessary destroyed by a destructor.

56.5.1 General syntax

The constructor is a method of a class like any other function or method¹⁶¹. However, the constructor is bound to comply some rules.

- The constructor always has the same name as its class
- The constructor has no return value

Listing 380 shows a simple class describing a point in a two-dimensional domain. This class has two constructors. The first constructor receives no arguments and initialises the member variables with zero. The second constructor receives two integer variables as arguments and uses this variables to initialize the member variables `xPos` and `yPos`.

Writing two or more constructors is possible because C++ supports function overloading. This means there can be several functions with the same name differing in the input arguments.

```
1  class Point
2  {
3      int xPos;
4      int yPos;
5
6      public:
7          Point()
8          {
9              /* constructor code */
10             xPos = 0;
11             yPos = 0;
12         }
13         Point(int x, int y)
14         {
15             xPos = x;
16             yPos = y;
17         }
18 };
```

Listing 380: A class for a 2D point

¹⁶¹The terms function and method are used interchangeably. However, the method indicates the use of object oriented programming. The term function is also used in procedural programming and does not automatically indicate the use of OOP.

Listing 381 demonstrates how to create new variables of the type `Point`. The first line creates a variable of the type `Point`. Because no arguments are passed in this line, the first constructor of Listing 380 is called by the compiler.

The second line creates also a point. The numbers inside the parenthesis are passed to the constructor. Therefore the second constructor of Listing 380 is called and the member variables are initialised based on the arguments.

```

1 Point p1;
2 Point p2(3, 8);

```

Listing 381: Using the class for a 2D point

56.5.2 Copy-Constructor

The copy constructor is used to create a copy of an object. The C++ compiler will create a default copy constructor if the programmer does not write one. However, the default copy constructor has restrictions regarding the handling of complex classes.

```

1 Point::Point(Point & p)
2 {
3     /* copy constructor code */
4     xPos = p.xPos;
5     yPos = p.yPos;
6 }

```

Listing 382: The copy constructor for the 2D point class

Hiding the copy constructor

A copy constructor can be hidden. Therefore, no copying is allowed. To do so, the copy constructor must be defined using a `private` modifier.

Listing 383 shows a simple example of a copy constructor that is declared as `private`. This means the copy constructor can only be called from within the class itself, i.e. only within the class `Point`.

Listing 384 shows an example from within the source code of OpenFOAM. There, the copy constructor of the class `turbulenceModel` is hidden by declaring it `private`.

```

1 class Point
2 {
3     private:
4         Point(Point & p);
5 };

```

Listing 383: Hiding the copy constructor

```

1 class turbulenceModel
2 :
3     public regIOobject
4 {
5     private:
6         // Private Member Functions
7
8         //- Disallow default bitwise copy construct
9         turbulenceModel(const turbulenceModel&);
10
11     /* code continues */

```

Listing 384: Hiding the copy constructor

56.5.3 Initialisation list

A class in C++ can have member variables of any type. Complex classes may need some kind of initialisation to ensure all variables have a defined state. When an instance of a class is created by the constructor, the initialisation list contains all statements to initialise member variables of the class.

Listing 385 shows a simple example of a constructor with an initialisation list. Listing 488 in Section 61.2.2 shows an usage example of an initialisation list in the OpenFOAM sources.

```
1  class Rectangle
2  {
3      Point topLeft;
4      Point bottomRight;
5
6      public:
7          Rectangle()
8          {
9              topLeft = Point();
10             bottomRight = Point();
11         }
12
13         Rectangle(Point a, Point b)
14         :
15             topLeft(a),
16             bottomRight(b)
17         {
18             /* constructor code */
19         }
20 }
```

Listing 385: A constructor with an initialisation list

56.6 Object orientation

56.6.1 Abstract classes

See Section 57.10 for a discussion about the implementation of the generic turbulence models in OpenFOAM. This generic turbulence modelling makes heavy use of abstract classes and inheritance.

56.7 Templates

OpenFOAM makes heavy¹⁶², clever use of templates. Templates are a language feature of C++ that allow for generic programming. An illustrative example for the use of templates in programming is the implementation of container classes, e.g. linked lists. Without the use templates, the multiplicity of possible container contents would force us to implement a vast number of specialized classes, e.g. `nodeList`, `faceList` and `cellList` for lists of nodes, faces and cells.

Such a problem could be solved by the use of multiple inheritance. This way, we would need to implement one base class for a list. The specialized classes would then inherit from the base list class and from the class of the intended content. This solution, however, has several disadvantages [5]. As complexity grows, the path via multiple inheritance is doomed to become a problem in its own, instead of alleviating or solving the original problem.

Templates offer us a way to tell a class: use the type `T`, which can be any type the compiler allows. Thus, we create one templated container class. Later, when we need to create lists of nodes, faces and cells, we tell the compiler to substitute `T` for the concrete types. The compiler then generates the appropriate code. Checks done by the compiler ensure, that specializing a valid templated class produces little to no surprises.

Listing 386 shows the use of templates. We first implement a generic list. Later, we specialize this list for the types of nodes, faces and cells. The `typedef` instruction allows us to define a convenient name. Once this names are defined, we may even stop being aware that we are using a templated class.

¹⁶²The command `find $FOAM_SRC -name '*.CH' | xargs grep 'template' | wc` yields – at the time of writing – 24646 occurrences of the word `template` in `$FOAM_SRC`. This makes 12323 occurrences within the source code itself – remember the presence and the use of the `lnInclude` directories.

```

template <class T>
class list
{
    // define a list of type T
}

typedef list<node> nodeList;
typedef list<face> faceList;
typedef list<cell> cellList;

```

Listing 386: Templated lists

OpenFOAM follows a similar strategy, who would guess from the top-level code, that `volScalarField` is in fact a templated class with three template parameters, see Listing [reflst:volScalarField](#). Besides being a more convenient name¹⁶³ we also save a lot of typing effort due to the shorter name¹⁶⁴. The use of type definitions – `typedef` statements – is not mere convenience. Using the full specialisation of `GeometricField` instead of `volScalarField` translates to *hardcoding*. If the developers of OpenFOAM, at some point, decide to base `volScalarField` on the class `smartScalar` instead of `scalar`, only one line of code needs to be changed instead of thousands. Thus, the use of `typedefs` strongly supports code readability and maintainability [5].

```

typedef GeometricField<scalar, fvPatchField, volMesh> volScalarField;

```

Listing 387: The `typedef` defining `volScalarField`

56.7.1 Use of templates by OpenFOAM

Since this document is not a book on any specific topic, certain topics are addressed in a manner ranging from structured to completely random. Templates have already been discussed in a number of sections, mostly describing the use of templates on specific code examples. Since, there is no fun and varying benefit in restructuring a large document, we will give pointers to other sections in which templates are discussed:

We discuss the use of templates in Section [32.1](#) where we compare the implementation of turbulence modelling in OpenFOAM. There is a non-templated implementation, which was superseded by a templated one starting from the release of OpenFOAM-2.3.0.

We discuss the use of templates in Section [40](#) where we take a look at the implementation of Lagrangian particle tracking with a little excursion to the topic of linked lists.

The use of templates is also discussed in Section [57.3.2](#) at the example of keyword lookup from dictionary files.

56.7.2 Do not fear the template

The syntax for templated code is different from the syntax encountered in non-templated code. Here we will discuss some features of templated code, which may seem mysterious to the novice.

Template template parameter

In the introduction of this section, we stated, that the template parameter `T` is a placeholder for a concrete type. However, the template parameter may itself be a templated class. A templated template parameter is referred to as *template template parameter*. We could avoid using template template parameters, however, they help us to avoid code duplication and lead to safer code [5].

The library implementing Lagrangian particle tracking is a very illustrative example of nested templates, i.e. templated classes being used as template parameters. Furthermore, we observe deriving a class from its template parameter. Check out Section [40](#).

¹⁶³`volScalarField` field carries roughly the same essential information as `GeometricField<scalar, fvPatchField, volMesh>`.

¹⁶⁴We count 15 versus 46 characters. At the time of writing, with the command `find $FOAM_SRC -name '*.CH' | xargs grep 'volScalarField' | wc` we count 8752 occurrences of `volScalarField` in the source code of OpenFOAM-dev at the time of writing. This leads to an estimated 4376 occurrences in the code itself.

57 Under the hood of OpenFOAM

This section contains short code examples that in some way explain the behaviour of OpenFOAM in certain situations. All examples in this section are motivated by other parts of this manual. In some cases the source code of some applications is examined somewhere else.

57.1 Solver algorithms

See Sections 41, 42 and 44 in Part VI.

57.2 Namespaces

57.2.1 Constants

Physics is full of constants. Therefore it would be nice to have a central location in which physical or mathematical constants are defined. OpenFOAM provides constants within the namespace `Foam::constant`. There the pre-defined constants are divided into the groups, such as

- electromagnetic
 - `mu0` - the magnetic permeability of vacuum
 - `epsilon0` - the electrical permittivity of vacuum
- physicoChemical
 - `R` - the universal gas constant
- mathematical
 - `pi` - π
 - `e` - the Euler number

In Listing 388 it is demonstrated how to access the constant `pi` within the source code. Listing 389 shows all the mathematical constants defined in OpenFOAM-2.2.x. From a computational performance point of view it makes perfect sense to pre-define often used constants such as two pi. Also note that instead of dividing pi by 2.0 it is multiplied with 0.5. Mathematically these operations are equivalent, however, in terms of computational cost the floating point multiplication is to be preferred over the floating point division as it is much faster [1].

Also note that OpenFOAM does not define e and π on its own, it rather uses the constants provided by the system library. See e.g. http://www.gnu.org/software/libc/manual/html_node/Mathematical-Constants.html for the mathematical constants provided by the GNU C library (glibc). Thus `e` and `pi` are defined by accessing `M_E` and `M_PI`.

Further note that the constants are declared with the `const` specifier, which is the only sane way to define constants in C and C++.

```
1 scalar foo = constant::mathematical::pi;
```

Listing 388: A useless code example demonstrating the access to π with OpenFOAM's source code

```
1 const scalar e(M_E);
2 const scalar pi(M_PI);
3 const scalar twoPi(2*pi);
4 const scalar piByTwo(0.5*pi);
```

Listing 389: The mathematical constants provided by `mathematicalConstants.H`

In the FOAM-extend the access to e.g. the mathematical constants works the same way. Only the namespace is named `mathematicalConstants` instead of `constant::mathematical`. This is due to the fact that FOAM-extend is largely based on OpenFOAM-1.6.

57.3 Keyword lookup from dictionary

There are generally two kinds of keywords in a dictionary. There are mandatory keywords and optional ones.

57.3.1 Mandatory keywords

When a mandatory keyword is not found in a dictionary, OpenFOAM issues an error message and terminates.

Listing 390 shows the reading operation for three mandatory keywords. The function `lookup()` can be examined further in Listing 391.

```
1 #include "readTimeControls.H"
2
3 int nAlphaCorr(readInt(pimple.dict().lookup("nAlphaCorr")));
4 int nAlphaSubCycles(readInt(pimple.dict().lookup("nAlphaSubCycles")));
5 Switch correctAlpha(pimple.dict().lookup("correctAlpha"));
```

Listing 390: The content of `readTwoPhaseEulerFoamControls.H`

The code

Line 32 in Listing 391 shows, that the function `lookup()` simply calls value of `lookupEntry()`. This method also calls another method (`lookupEntryPtr()`) and does the error handling. The error handling routine clearly shows, that OpenFOAM will terminate in case the keyword wasn't found (see line 19).

```
1 const Foam::entry& Foam::dictionary::lookupEntry
2 (
3     const word& keyword,
4     bool recursive,
5     bool patternMatch
6 ) const
7 {
8     const entry* entryPtr = lookupEntryPtr(keyword, recursive, patternMatch);
9
10    if (entryPtr == NULL)
11    {
12        FatalIOErrorIn
13        (
14            "dictionary::lookupEntry(const word&, bool, bool) const",
15            *this
16        )
17        << "keyword " << keyword << " is undefined in dictionary "
18        << name()
19        << exit(FatalIOError);
20    }
21
22    return *entryPtr;
23 }
24
25 Foam::ITstream& Foam::dictionary::lookup
26 (
27     const word& keyword,
28     bool recursive,
29     bool patternMatch
30 ) const
31 {
32     return lookupEntry(keyword, recursive, patternMatch).stream();
33 }
```

Listing 391: Some content of `dictionary.C`

57.3.2 Optional keywords

A method that is used to read an optional keyword from a dictionary is usually provided with a default value. This default value is used in the case that the keyword is non-existent in the dictionary.

Listing 392 shows the reading operation for three optional keywords. The read function is called with two arguments. The first is the keyword and the second is the default value. If the function `lookupOrDefault()` finds no entry, then the default value is returned.

```

1  const bool adjustTimeStep =
2      runTime.controlDict().lookupOrDefault("adjustTimeStep", false);
3  scalar maxCo =
4      runTime.controlDict().lookupOrDefault<scalar>("maxCo", 1.0);
5  scalar maxDeltaT =
6      runTime.controlDict().lookupOrDefault<scalar>("maxDeltaT", GREAT);

```

Listing 392: The content of `readTimeControls.H`

The code

Listing 393 shows the definition of the function `lookupOrDefault()`. This function also calls another function to lookup the keyword – actually it looks for the value assigned to the specified keyword in the dictionary – and enters a conditional branch. In case the keyword was found, the corresponding value is returned (line 14). If the keyword was not found, then the default value is returned (line 18).

In Listing 393 the function is defined with four input arguments. However, in Listing 392 this function is called with only two arguments.

The solution for this contradiction can be found in the file `dictionary.H`, where this function is declared. This declaration can also be found in Listing 394. There, in lines 6 and 7, default values for two arguments are specified. Therefore, the function can be called with only two arguments – with the two arguments that have no default value¹⁶⁵. If the function is called with all its arguments, the passed argument overrides the default value.

When declaring a function that uses default values for its arguments, the arguments without default value must precede the arguments that have a default value. Otherwise, there could be ambiguity.

```

1  template<class T>
2  T Foam::dictionary::lookupOrDefault
3  (
4      const word& keyword,
5      const T& deflt,
6      bool recursive,
7      bool patternMatch
8  ) const
9  {
10     const entry* entryPtr = lookupEntryPtr(keyword, recursive, patternMatch);
11
12     if (entryPtr)
13     {
14         return pTraits<T>(entryPtr->stream());
15     }
16     else
17     {
18         return deflt;
19     }
20 }

```

Listing 393: Some content of `dictionaryTemplates.C`

```

1  template<class T>
2  T lookupOrDefault
3  (
4      const word&,
5      const T&,
6      bool recursive=false,
7      bool patternMatch=true
8  ) const;

```

Listing 394: Some content of `dictionary.H`

¹⁶⁵The function could also be called with three arguments, then the default value of the third argument would be overridden and the fourth argument would have its default value.

57.3.3 Superseding mandatory keywords

In some cases, a base class might demand the presence of a keyword, which some derived classes make use of and others do not. This creates the situation that a keyword-value pair must be specified, which, depending on the user's choice, has an effect or not.

However, as the keyword is demanded by the base class, it remains mandatory, regardless of the derived class' behaviour. A derived class can not alter the behaviour of its base class. Thus, in some edge cases a keyword-value pair has to be provided even though it does nothing.

The decision of how to distribute the necessary data among the base class and its derived classes may not always be as straight forward. Apart from the two obvious limiting cases: data needed only by one specific derived class, and data needed by all derived classes; the decision of what data to put where is up to the software designers.

If a sufficient number of derived classes are very similar to each other, then an intermediate base class might be warranted. With an intermediate base class, data common to all classes derived from the intermediate base class can be handed over to the intermediate base class. However, the class hierarchy of a certain model can not always exactly reflect the inner logic of data usage of that model family.

In Figure 130 we see such an example of an intermediate base class. The data members shown in the class diagram are read by their respective classes as mandatory entries. The base class `InjectionModel` reads the scalar `massTotal`, which is used to determine how many particles to inject. However, some derived injection models allow for a direct specification of the number of particles, thus rendering the mandatory value for `massTotal` moot. In these cases, the injection model issues a warning message informing the user that `massTotal` has no effect.

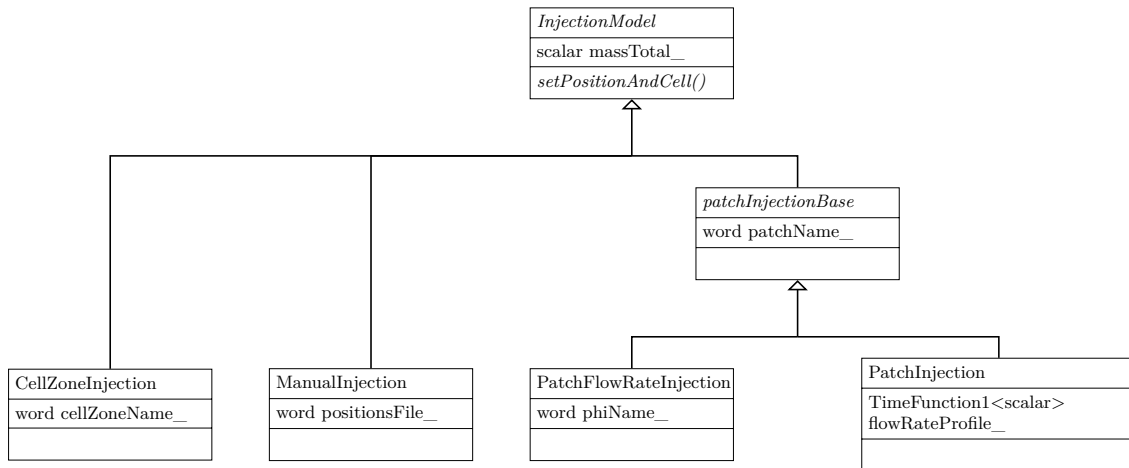


Figure 130: Class hierarchy of some injection models for Lagrangian particles. An intermediate base class is used to reduce code duplication from closely related, yet different injection models.

57.4 OpenFOAM specific datatypes

57.4.1 The Switch datatype

A lot of settings in dictionaries are switches to activate or deactivate a feature. Listing 395 shows the part of the source code defining all valid values. Inside the source code a switch can only be true or false, as the class `Switch` is used as a boolean data type. However, in the dictionaries a switch can have more values – provided they denote a decision. Human languages usually have more ways of answering a yes-no question, this may be the motivation for allowing this range of values for switches.

```

1 // NB: values chosen such that bitwise '&' 0x1 yields the bool value
2 // INVALID is also evaluates to false, but don't rely on that
3 const char* Foam::Switch::names[Foam::Switch::INVALID+1] =
4 {
5     "false", "true",
6     "off",   "on",
7     "no",    "yes",

```

```

8     "n",      "y",
9     "f",      "t",
10    "none",   "true", // is there a reasonable counterpart to "none"?
11    "invalid"
12 };

```

Listing 395: Some content of Switch.C

Listing 396 shows an example of how the `Switch` datatype can be used in the code. This example reads from the `transportProperties` dictionary. If no valid entry named `testSwitch` is present, then the value of the switch is set to `false`. Notice the second argument of the method `lookupOrDefault()`, it reads `Switch(false)`. This means, that a new object of the type `Switch` is created with the boolean value `false` being passed to the constructor of the class `Switch`. This new object of type `Switch` is then used – if necessary – as default value for the switch named `testSwitch`.

```

1 Switch testSwitch(transportProperties.lookupOrDefault<Switch>("testSwitch", Switch(false)));

```

Listing 396: Usage example of the `Switch` datatype

57.4.2 The label datatype

When examining the solution algorithms, like in Section 42.2, counters can be found. OpenFOAM uses a datatype called `label` for such counters, e.g. see Listing 266.

The following applies to versions prior to OpenFOAM-3.0

The most obvious datatype for a counter would be the integer datatype `int`. Listing 397 contains some lines of the file `label.H`, where this datatype is defined. Depending on system or compilation parameters, `label` is of the type `int`, `long` or `long long`¹⁶⁶.

Listing 397 shows the definition of `label` in case `int` is used as the underlying datatype.

```

1 namespace Foam
2 {
3     typedef int label;
4
5     static const label labelMin = INT_MIN;
6     static const label labelMax = INT_MAX;
7
8     inline label readLabel(Istream& is)
9     {
10         return readInt(is);
11     }
12
13 } // End namespace Foam

```

Listing 397: Some content of label.H

The following applies to versions from OpenFOAM-3.0 onwards

OpenFOAM offers, at the time of writing, essentially two choices for the size of the `label` datatype: 32 or 64 bit. This, essentially boils down to the decision of whether to use `int32_t` or `int64_t`. Since, the data types `int`, `long` and `long long` only guarantee a minimum size¹⁶⁷, the fixed-width integers¹⁶⁸ `int32_t` or `int64_t` are used as a base for the `label` data type.

The label size can be selected prior to compilation with the compiler option `WM_LABEL_SIZE`, which can take the value of 32 or 64. From the compiler option and some pre-processor macros the integer type is constructed.

¹⁶⁶In C as well as in C++ the domain of `long` is greater or equal than the domain of `int`. `long long` was defined in the C99 standard of C and was later introduced to the C++11 standard. The domain of `long long` is again larger or equal than the domain of `long`. The type `long long` uses at least 64 bit. So it is on 64 bit systems the largest possible datatype. The datatype `long` can use – depending on the compiler – 32 or 64 bit. The type `long long` guarantees the use of 64 bit.

¹⁶⁷See <http://en.cppreference.com/cpp/language/types>

¹⁶⁸See <http://en.cppreference.com/cpp/types/integer>

```

1 #define INT_ADD_SIZE(x,s,y) x ## s ## y
2 #define INT_ADD_DEF_SIZE(x,s,y) INT_ADD_SIZE(x,s,y)
3 #define INT_SIZE(x,y) INT_ADD_DEF_SIZE(x,WM_LABEL_SIZE,y)
4
5 #if WM_LABEL_SIZE != 32 && WM_LABEL_SIZE != 64
6     #error "label.H: WM_LABEL_SIZE must be set to either 32 or 64"
7 #endif
8
9 namespace Foam
10 {
11
12     typedef INT_SIZE(int, _t) label;
13
14     /* code removed for brevity */
15
16 }

```

Listing 398: Some content of `label.H`

In Listing 398, we see some helper macros, and a sanity check. The sanity check throws an error if the `label` size is not 32 or 64. The last line of Listing 398 is the actual `typedef` for the `label` data type¹⁶⁹. The macro `INT_SIZE` is used via several helper macros and the compiler option `WM_LABEL_SIZE` to construct a fixed-width integer data type `int32_t` or `int64_t`. The `##` operator in the `INT_ADD_SIZE` macro is used to concatenate the macro's arguments¹⁷⁰.

57.4.3 The scalar datatype

Similar, to the integer data type, there is an OpenFOAM-specific floating-point data type, the `scalar`. A `scalar` is depending on the compiler settings either a float or a double. In 2018¹⁷¹ an extended precision double was added as another option.

The basic floating-point data type `float` is 32 bits wide¹⁷², i.e. its binary representation takes up 32 bits. The `double` type uses twice the number of bits, hence the name double. Using either `float` or `double` for computing is often referred to as using single or double precision.

OpenFOAM follows this convention by naming the corresponding compiler option `WM_PRECISION_OPTION`. This can have the value `SP` or `DP`, for single precision or double precision. The latest addition¹⁷³, long double¹⁷⁴, can be chosen by the value `LP`.

Using fixed-width data types, integers as well as floating-point types, makes the application more portable, e.g. binary result files computed by one computer can be opened and processed on a different computer, provided that both OpenFOAM installations are based on the same data type size, e.g. 32 bit integers and double precision floating-points.

Listing 399 shows the definition of the type `floatScalar`, which refers to the standard data type `float`. Analogously, there is also a type `doubleFloat`, which refers to the standard data type `double`. By introducing the type `scalar`, OpenFOAM's source code is independent of the floating-point type which is actually used. The types `floatScalar` and `doubleScalar` are intermediaries to aid the selection by the user prior to compilation.

The types `floatScalar` and `doubleScalar` are mainly used by low-level code tasked with input and output (I/O). No applications (solvers and utilities) and no high-level libraries, e.g. turbulence, use these data types. All high-level code is abstracted from the actual floating-point data type.

```

1 namespace Foam
2 {
3
4     typedef float floatScalar;
5
6     /* code removed for brevity */

```

Listing 399: Some content of `floatScalar.H`

¹⁶⁹See <https://en.wikipedia.org/wiki/Typedef>

¹⁷⁰See https://en.wikibooks.org/wiki/C_Programming/Preprocessor_directives_and_macros

¹⁷¹See <https://github.com/OpenFOAM/OpenFOAM-dev/commit/d82cc36c5af97e799a82fadf455e06d192ae1e65>

¹⁷²See https://en.wikipedia.org/wiki/IEEE_754

¹⁷³See footnote 171

¹⁷⁴See https://en.wikipedia.org/wiki/Long_double

Listing 400 shows the relevant lines of the file `scalar.H` which defines the type `scalar` based on the evaluation of `WM_SP` or `WM_DP`. Depending on whether `WM_SP` or `WM_DP` has been defined, the type `scalar` refers to `floatScalar` or `doubleScalar`.

```

1  #if defined(WM_SP)
2
3  // Define scalar as a float
4  namespace Foam
5  {
6      typedef floatScalar scalar;
7      /* code removed for brevity */
8  }
9
10 #elif defined(WM_DP)
11
12 // Define scalar as a double
13 namespace Foam
14 {
15     typedef doubleScalar scalar;
16     /* code removed for brevity */
17 }
18
19 #endif

```

Listing 400: Some content of `scalar.H`

57.4.4 The `tmp<>` datatype

There is a special class for all temporary data. Because there is no memory management in C++ the programmer has to delete unused variables. The author assumes that the `tmp` class for all kinds of temporary data is meant to distinguish temporary variables from other variables.

The `tmp` class uses a technique called generic programming.

57.4.5 The `IObject` datatype

The class `IObject` handles the behaviour of all kinds of data structures. Although, there are no variables of the type `IObject`, understanding some parts of this class will help to understand certain aspects of OpenFOAM.

Listings 401 and 402 show some examples from the sources of the solver *twoPhaseEulerFoam*. There, the class `IObject` is used in the creation of fields as well as the creation of dictionary objects.

In Listing 401 two `volScalarField` variables are created. The constructor of the class `volScalarField` receives two arguments. In both cases the first argument is an `IObject`.

Let us read the arguments of the `IObject` constructor call. The first argument is the name of the `IObject`. The two last arguments are the read and write flags.

In the case of the fields `alpha1` and `alpha2` the read and write flags are different. The field `alpha1` is read at the start of the application. The write flag causes the field `alpha1` to be written to disk, whenever the data is written. The field `alpha2` on the contrary is not written to disk and the application also does not try to read it.

The name of the `IObject` is also the name which the application uses as file name. Therefore the field `alpha1` will be written to disk in a file named `alpha1`. Also when the application tries to read `alpha1`, it tries to read from the file `alpha1`.

```

1  volScalarField alpha1
2  (
3      IObject
4      (
5          "alpha1",
6          runTime.timeName(),
7          mesh,
8          IObject::MUST_READ,
9          IObject::AUTO_WRITE
10     ),
11     mesh
12 );

```

```

13
14 volScalarField alpha2
15 (
16     IOobject
17     (
18         "alpha2",
19         runtime.timeName(),
20         mesh,
21         IOobject::NO_READ,
22         IOobject::NO_WRITE
23     ),
24     scalar(1) - alpha1
25 );

```

Listing 401: Definition of volume fraction fields in `createFields.H`

Listing 402 shows the definition of an `IOdictionary`. The constructor of the class `IOdictionary` receives also an `IOobject` as argument. Again, the name of the `IOobject` is also the name of the file the application tries to read when reading in the dictionary. Notice also the read flag. This flag causes the application to check if the file has been modified during run-time. If this is the case, the file will be read again.

```

1 IOdictionary ppProperties
2 (
3     IOobject
4     (
5         "ppProperties",
6         runtime.constant(),
7         mesh,
8         IOobject::MUST_READ_IF_MODIFIED,
9         IOobject::NO_WRITE
10    )
11 );

```

Listing 402: Definition of a dictionary in `readPPProperties.H`

Read & write flags

In the constructor so called read and write flags are provided as arguments, see e.g. Lines 8 and 9 of Listing 402.

Listing 403 shows the available read/write flags. The flag `MUST_READ_IF_MODIFIED` was introduced with OpenFOAM-2.0.0¹⁷⁵. The available read flags offer quite some flexibility.

```

1     //- Enumeration defining the valid states of an IOobject
2     enum objectState
3     {
4         GOOD,
5         BAD
6     };
7
8     //- Enumeration defining the read options
9     enum readOption
10    {
11        MUST_READ,
12        MUST_READ_IF_MODIFIED,
13        READ_IF_PRESENT,
14        NO_READ
15    };
16
17    //- Enumeration defining the write options
18    enum writeOption
19    {
20        AUTO_WRITE = 0,
21        NO_WRITE = 1
22    };

```

Listing 403: Definition of the object states and read/write flags of `IOobject` in `IOobject.H`

¹⁷⁵<http://www.openfoam.org/version2.0.0/runtime-control.php>

Pitfall: Solving for a NO_READ field

The author stumbled across an interesting error during modifying a solver. This falls into the category copy & paste error. However, the author wishes to share the experience.

If we like to extend an existing solver with a scalar transport equation, we need to create the field we want to solve for, in our case a `volScalarField`. There are plenty of files from which we can copy the relevant code. Listing 404 shows an example. The name of the field was changed as was the write flag. Since we want to create colourful images, the write flags needs to be set to `AUTO_WRITE`. However, no care was taken of the read flag.

```
1     volScalarField T
2     (
3         IOobject
4         (
5             "T",
6             runtime.timeName(),
7             mesh,
8             IOobject::NO_READ,
9             IOobject::AUTO_WRITE
10        ),
11        mesh,
12        dimensionedScalar("zero", dimensionSet(0, 0, 0, 0, 0), 0.0)
13    );
```

Listing 404: Creating a field with an `IOobject::NO_READ` read flag.

After we created out field `T`, and composed the transport equation for this field (`TEqn`), we want to solve this transport equation. However, the call `TEqn.solve()` yields some unexpected outcome. Listing 405 shows the error message issued by OpenFOAM.

```
--> FOAM FATAL ERROR:

valueInternalCoeffs cannot be called for a calculatedFvPatchField

on patch inlet of field T in file "/home/user/OpenFOAM/user-2.3.x/run/foo/case/0/T"
You are probably trying to solve for a field with a default boundary condition.

From function calculatedFvPatchField<Type>::valueInternalCoeffs(const tmp<scalarField>&)
const
in file fields/fvPatchFields/basic/calculated/calculatedFvPatchField.C at line 154.

FOAM exiting
```

Listing 405: Error message of OpenFOAM caused by trying to solve for a no-read field.

At first, the message seems counter-intuitive, since we checked the boundary conditions in the file `T` over and over. Also changing the boundary conditions does not produce a different outcome.

The error message says, we wanted to solve for a field with default boundary conditions. This is perfectly true, however, we need to find out why. Since, we created the field with a `NO_READ` flag, no boundary conditions were provided. Thus, OpenFOAM assigns default boundary conditions. This is also the case if we leave patches in the `boundaryField` dictionary of the files that are read from disk.

Continued Problems

Changing the read flag in Listing 404 alone does not solve the problem. Changing the read flag from `NO_READ` to `MUST_READ` yields the same error message as in Listing 405.

The reason for this are the arguments of the constructor call in Listing 404. If a field is to be read from disk, we must not pass a value (Line 12 in Listing 404).

For our modified solver to work, we need to remove the argument passed in Line 12 in Listing 404. The developers of OpenFOAM have foreseen this case, thus OpenFOAM issues a warning message, when a value is passed to a constructor with a `MUST_READ` or `MUST_READ_IF_MODIFIED` read flag, see Listing 406.

```
--> FOAM Warning :
From function GeometricField<Type, PatchField, GeoMesh>::readIfPresent()
```

```

in file /home/user/OpenFOAM/OpenFOAM-2.3.x/src/OpenFOAM/lnInclude/GeometricField.C at line
108
read option IOobject::MUST_READ or MUST_READ_IF_MODIFIED suggests that
a read constructor for field T would be more appropriate.

```

Listing 406: Warning message of OpenFOAM caused by inappropriate constructor arguments concerning read flags and initial values.

57.4.6 Random stuff

OpenFOAM features a random number generator (RNG). The generated numbers within the sequence itself – depending on the quality of the algorithm – are close to being random. Random number generators on computers are also referred to as pseudo-random number generators as they are generally deterministic. Otherwise, nobody would be able to write code for such random number generators.

The randomness enters the scene in the form of the initial state of the random number generator, also known as seed. Choosing a non-constant seed value is key to obtain good random numbers. Using a constant seed value – using the same value each time the application is run – leads to an ever-recurring random number sequence, i.e. for the same initial conditions the RNG generates the same sequence of numbers.

The good, the bad and the ugly – in reverse order

The worst thing to do is to use a constant value for seeding the RNG. In Listing 407 we use zero als seed value. This value is equal every time we run the application. Thus, it comes as no suprise, when the random numbers we print to the Terminal are always the same, i.e. we print the same sequence of 20 numbers between one and a hundred every time we run the application containing the code of Listing 407.

```

1 // random stuff
2 #include "Random.H"
3 Random ranGen(0);
4
5 for (int j = 0; j < 20; j++)
6 {
7     Info << ranGen.integer(1, 100) << endl;
8 }

```

Listing 407: A simple test for random numbers; the ugly.

In order to obtain different sequences, we need to choose a better seed value. In fact, we need to choose a seed value that is different every time we run our application. The time would be a perfect example for such a seed value. However, we need to make errors in order to learn something. In the sources, we came across the method `osRandomInteger()`. This sounds great, use a random number to seed a random number generator. On a second thought, this sounds more of a chicken-egg problems, but let's continue.

So we implement the code of Listing 408, which is simply a different seed value. However, when we run the code, we find out, that we obtain the same sequences over and over, just as in the previous case.

Digging into the code, we find out, that `osRandomInteger()` uses the random number generator provided by POSIX. However, there seems to be no proper seeding of the POSIX random number generator.

```

1 // random stuff
2 #include "Random.H"
3 Random ranGen(osRandomInteger());
4
5 for (int j = 0; j < 20; j++)
6 {
7     Info << ranGen.integer(1, 100) << endl;
8 }

```

Listing 408: A simple test for random numbers, the bad.

As mentioned above, the time is the perfect seed value. However, since we are now at the good solution, we need something other than time. In Listing 409, we use the PID of the application as the seed value for the RNG. The PID is unlikely to be equal when the application is run several times. In fact, the kernel of the OS assigns the PIDs sequentially from a range of integer numbers, e.g. on the authors Linux machine the PID of

a process is in the range between 1 and 32768. If the end of the number range is reached, the kernel starts all over, skipping numbers which are still in use. Furthermore, the PID is guaranteed to be different, when running an application in parallel, i.e. all the sub-processes have a unique PID.

```
1 // random stuff
2 #include "Random.H"
3 Random ranGen(pid());
4
5 for (int j = 0; j < 20; j++)
6 {
7     Info << ranGen.integer(1, 100) << endl;
8 }
```

Listing 409: A simple test for random numbers; the good.

The even better

As already mentioned, using the time gives us a different seed value every time, the application is run. The method `getTime()` returns the number of seconds that have passed since January, 1st 1970. The code of Listing 410 now yields different number sequences every time we run the application. Also, PID-reuse is also not an issue anymore, since, whenever a PID gets reused, the time is certainly different. As we use the time to seed the RNG, the year 2038 problem¹⁷⁶ is a non-issue to us, since we are only interested in unique values rather than correct representation of time.

```
1 // random stuff
2 #include "Random.H"
3 #include "clock.H"
4 Random ranGen(clock::getTime());
5
6 for (int j = 0; j < 20; j++)
7 {
8     Info << ranGen.integer(1, 100) << endl;
9 }
```

Listing 410: A simple test for random numbers; the even better.

The perfect

The solution above is nearly perfect, the only issue left is running in parallel. This might seem a non-issue when we just want to implement random numbers for an application we only will use in serial. However, the trick is rather easy.

We use the current time as seed value and add the PID. This will ensure, that when multiple processes are spawned at the same time, when starting a parallel run, each process has its unique seed value thanks to the contribution of the PID.

```
1 // random stuff
2 #include "Random.H"
3 #include "clock.H"
4 Random ranGen(clock::getTime()+pid());
5
6 for (int j = 0; j < 20; j++)
7 {
8     Info << ranGen.integer(1, 100) << endl;
9 }
```

Listing 411: A simple test for random numbers; the perfect.

¹⁷⁶https://en.wikipedia.org/wiki/Year_2038_problem

57.5 OpenFOAM specific macros for convenient programming

57.5.1 For-loops

... for lists

The `UList` class defines some macros for looping over the contents of the list. We frequently encounter for loops using statements such as `forAll`, `forAllIter` or `forAllConstIter`. These statements, however, are not part of the C or C++ programming language, they are macros provided by the `UList` class.

In Listing 412 we see the definition of the `forAll` macro.

```
1 #define forAll(list, i) \
2     for (Foam::label i=0; i<(list).size(); i++)
```

Listing 412: The definition of the *forAll* macro in `UList.H`

When the `forAll` macro is in use, a for-loop might look as the example in Listing 413.

```
1 forAll(someList, i)
2 {
3     // list body
4 }
```

Listing 413: Using the `forAll` macro to traverse a list.

The code in Listing 413 is, at compile time, expanded into the code shown in Listing 414.

```
1 for (Foam::label i=0; i<(someList).size(); i++)
2 {
3     // list body
4 }
```

Listing 414: Expanding the `forAll` macro. This is what the C++ pre-processor does with Listing 413.

... for containers

For container data types, which have an associated iterator data type, there are the `forAllIter` and `forAllConstIter` macros. The concept of the container is an abstraction to encompass sequenced containers (lists) and associative containers (hash table). The concept of the iterator is used to decouple the access to elements of the container from the internal organisation of the container¹⁷⁷.

Below, in Listing 415 we see the definition of the `forAllIter` macro, which uses an iterator to traverse the container, note the usage of the methods `begin()` and `end()`, as well as to access the individual element, access is shown by example in Listing 416.

```
1 #define forAllIter(Container, container, iter) \
2     for \
3     ( \
4         Container::iterator iter = (container).begin(); \
5         iter != (container).end(); \
6         ++iter \
7     )
```

Listing 415: The definition of the *forAllIter* macro in `UList.H`

In the example below, in Listing 416, a cloud of Lagrangian particles is traversed, and each particle can be accessed via the iterator. This piece of code, respectively the developer working on this code, is completely oblivious of the underlying data structure which the Lagrangian cloud class uses.

By using the iterator for traversing the cloud, and accessing its elements, the developers of OpenFOAM could change the cloud's internal way of storing the data without breaking the higher-level code.

¹⁷⁷https://en.wikipedia.org/wiki/Iterator_pattern

```

1  forAllIter(Cloud<solidParticle>, c, iter)
2  {
3      solidParticle& p = iter();
4
5      // ...
6  }

```

Listing 416: Using the *forAllIter* macro to access all particles in a Lagrangian particle cloud.

... for everything else

At the time of writing, OpenFOAM-5 is the current release, such for-loop macros are only defined for the class `UList`. Thus, the macro can only be used on data types derived from `UList`.

If our for-loop is not about traversing a list, or any container data type derived from `UList`, then we need to write the for-loop in the traditional fashion.

57.6 Time management

57.6.1 Time stepping

Transient solvers solve the governing equations each time step at least once. Depending on the solution algorithm there are several inner iterations (iterations within a time step) during one outer iteration.

pimpleFoam

Listing 417 shows the beginning of the main loop of *pimpleFoam*. After the three `include` instructions, the `runTime` object is incremented. This means, the current time step is incremented to the next time step.

```

1  /* code removed for the sake of brevity */
2
3  Info<< "\nStarting time loop\n" << endl;
4
5  while (runTime.run())
6  {
7      #include "readTimeControls.H"
8      #include "CourantNo.H"
9      #include "setDeltaT.H"
10
11      runTime++;
12
13      Info<< "Time = " << runTime.timeName() << nl << endl;
14
15      /* code continues */

```

Listing 417: The beginning of the main loop of *pimpleFoam* in `pimpleFoam.C`

isoFoam

Listing 418 shows the beginning of the main loop of *isoFoam*.

```

1  /* code removed for the sake of brevity */
2
3  Info<< "\nStarting time loop\n" << endl;
4
5  while (runTime.loop())
6  {
7      Info<< "Time = " << runTime.timeName() << nl << endl;
8
9      #include "readPISOControls.H"
10     #include "CourantNo.H"
11
12     // Pressure-velocity PISO corrector
13     {
14         /* code continues */

```

Listing 418: The beginning of the main loop of *pisoFoam* in *pisoFoam.C*

There, there is no incrementation of any `runTime` object. The explanation for this, lies in the condition of the `while` statement. In *pisoFoam*, the `while` statement is controlled by the return value of the function call `runTime.loop()`. Whereas, in *pimpleFoam*, the `while` statement is controlled by the return value of the function call `runTime.run()`.

Let's have a closer look on `runTime.loop()`. Listing 419 shows, that the function `loop()` calls the function `run()` and then increments the `runTime` object by calling `operator++()`.

The ++ operator of the Time class

Listing 420 shows the first lines of the definition of the `++` operator of the `Time` class. The last instruction of Listing 420 set the time value to the current time value plus the time step.

```
1 bool Foam::Time::loop()
2 {
3     bool running = run();
4
5     if (running)
6     {
7         operator++();
8     }
9
10    return running;
11 }
```

Listing 419: The definition of the function `loop()` in *Time.C*

```
1 Foam::Time& Foam::Time::operator++()
2 {
3     deltaT0_ = deltaTSave_;
4     deltaTSave_ = deltaT_;
5
6     // Save old time name
7     const word oldTimeName = dimensionedScalar::name();
8
9     setTime(value() + deltaT_, timeIndex_ + 1);
10
11    /* code removed for the sake of brevity */
```

Listing 420: The definition of the operator `++` in *Time.C*

57.6.2 Setting the new time step

Transient simulations can be run with fixed and variable time steps. In a simulation with fixed time step the time step is constant. The value of the time step must be set before the simulation is started. The time step influences the accuracy and stability of the simulation. The value of the time step determines the time scales that can be resolved in the simulation. Via the Courant-Friedrichs-Lewy (CFL) criterion the time step is linked to the stability of the time integration method.

Most transient OpenFOAM solvers offer the possibility of transient simulations with variable time steps. The user then provides the limits for the determination of the time steps. The most obvious limit is the maximum time step `maxDeltaT`. This is the upper limit for the value of each new time step. This is the parameter for the user to determine the time scale to be resolved.

The second limit for determining the time steps is the maximum Courant number. This parameters purpose is to maintain stability of the numerical solution.

Listing 421 shows the code that reads the time controls. The first instruction reads the entry in `controlDict` specifying whether to use variable time steps or not. This code is rather self-explanatory. If there is not entry in `controlDict` then a fixed time step is used. The other two instructions read values for the maximum Courant number and the maximum time step. The default value for the maximum Courant number is 1.0, which is the limit for the explicit Euler time integration method.

```

1  const bool adjustTimeStep =
2      runTime.controlDict().lookupOrDefault("adjustTimeStep", false);
3  scalar maxCo =
4      runTime.controlDict().lookupOrDefault<scalar>("maxCo", 1.0);
5  scalar maxDeltaT =
6      runTime.controlDict().lookupOrDefault<scalar>("maxDeltaT", GREAT);

```

Listing 421: The content of the file `readTimeControls.H`

Determining the new time step

The value of the new time step has to obey both limit mentioned above, the maximum time step and the maximum Courant number. In order to prevent oscillations the increase of the time step is damped. Listing 422 shows how the time step is computed each time step.

```

1  if (adjustTimeStep)
2  {
3      scalar maxDeltaTFact = maxCo/(CoNum + SMALL);
4      scalar deltaTFact = min(min(maxDeltaTFact, 1.0 + 0.1*maxDeltaTFact), 1.2);
5
6      runTime.setDeltaT
7      (
8          min
9          (
10             deltaTFact*runTime.deltaTValue(),
11             maxDeltaT
12         )
13     );
14
15     Info<< "deltaT = " << runTime.deltaTValue() << endl;
16 }

```

Listing 422: The content of the file `setDeltaT.H`

Let us have a look on what the code is actually doing.

$$\text{maxDeltaTFact} = \frac{\text{maxCo}}{\text{Co} + \text{SMALL}} \quad (163)$$

$$\text{deltaTFact} = \min(\min(\text{maxDeltaTFact}, 1.0 + 0.1 * \text{maxDeltaTFact}), 1.2) \quad (164)$$

The scalar `maxDeltaTFact` (Line 3 in Listing 422 and Eq. (163)) is the relation between the maximum Courant number and the current Courant number (see Section 57.6.4 on how the Courant number is determined). The role of the constant `SMALL` is to prevent division by zero, which would cause the solver to crash.

The scalar `deltaTFact` is computed from `maxDeltaTFact`. This line of code (Line 4 and Eq. (164)) implements the damping, i.e. the rate of increase of the time step is limited. The nested use of two `min()` functions determines the minimum of three values. The most obvious of these three values is the last argument. If this value is the smallest, then the next time step is 20 % larger than the last one.

Eq. (164) shows the minimum of the first two arguments in a mathematical way. Figure 131 shows the three arguments of Eq. (164). We use the symbol x for the scalar `maxDeltaTFact`. In Figure 131 the values for x are greater than one. Eq. (166) elaborates why this is the case. x is the ratio of the maximum Courant number Co_{max} and the current Courant number Co . As the current Courant number is always smaller than the maximum Courant number we replace Co with fCo_{max} , with $f < 1$. After cancelling Co_{max} the inverse of f remains. Thus x is always greater than one.

$$\min(x, 1 + 0.1x) = \begin{cases} x & x < \frac{10}{9} \\ 1 + 0.1x & x > \frac{10}{9} \end{cases} \quad (165)$$

$$x = \frac{Co_{max}}{Co} = \frac{Co_{max}}{f Co_{max}} = \frac{1}{f} \quad (166)$$

$$\Rightarrow x > 1 \quad (167)$$

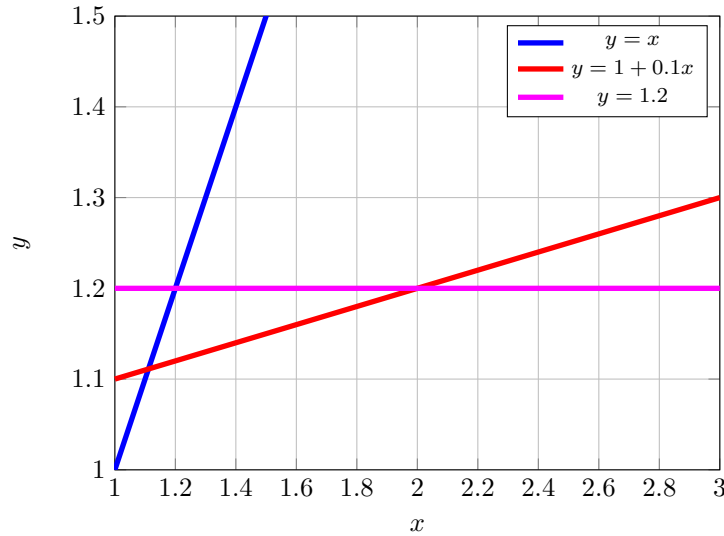


Figure 131: The three arguments of Eq. (164) plotted over x

The argument of the function `setDeltaT()` contains the abundance of the first limit, the maximum time step. There the minimum of the newly calculated and the maximum time step is passed on.

57.6.3 A note on the passing of time

In this section we will take a closer look at the implementation of the `Time` class.

Class design

A quick glance at the file `Time.H` reveals some very interesting information on the nature of time, or more precisely, the nature of the `Time` class. Listing 423 shows us, that the class `Time` class inherits from five base classes¹⁷⁸.

```
class Time
:
    public clock,
    public cpuTime,
    public TimePaths,
    public objectRegistry,
    public TimeState
{
    /* class definition */
}
```

Listing 423: The information on inheritance of the `Time` class; an extract of `Time.H`.

The `TimeState` class

From Listing 423 we see that `Time` is a `TimeState` due to inheritance. In Listing 424 we see the information on inheritance of the `timeState` class. There we see, that `TimeState` is a `dimensionedScalar`.

```
class TimeState
:
    public dimensionedScalar
{
    /* class definition */
}
```

Listing 424: The information on inheritance of the `TimeState` class; an extract of `TimeState.H`.

¹⁷⁸Literarily spoken, the `Time` class is not only Dr. Jekyll and Mr. Hyde, it is also Citizen Kane, Mrs. Robinson and the Tambourine Man.

Distinguishing between time steps

The fact that `Time` is a `TimeState` which in turn is a `dimensionedScalar` helps to understand the Lines 6, 13 and 21 of Listing 425. There, the `name()` method of the `dimensionedScalar` name space is called.

```
1 Foam::Time& Foam::Time::operator++()
2 {
3     // some code removed for brevity
4
5     // Save old time name
6     const word oldTimeName = dimensionedScalar::name();
7
8     setTime(value() + deltaT_, timeIndex_ + 1);
9
10    // some code removed for brevity
11
12    // Check that new time representation differs from old one
13    if (dimensionedScalar::name() == oldTimeName)
14    {
15        int oldPrecision = precision_;
16        do
17        {
18            precision_++;
19            setTime(value(), timeIndex());
20        }
21        while (precision_ < 100 && dimensionedScalar::name() == oldTimeName);
22
23        WarningIn("Time::operator++()")
24            << "Increased the timePrecision from " << oldPrecision
25            << " to " << precision_
26            << " to distinguish between timeNames at time " << value()
27            << endl;
28
29        if (precision_ == 100 && precision_ != oldPrecision)
30        {
31            // Reached limit.
32            WarningIn("Time::operator++()")
33                << "Current time name " << dimensionedScalar::name()
34                << " is the old as the previous one " << oldTimeName
35                << endl
36                << "    This might result in overwriting old results."
37                << endl;
38        }
39        // some code removed for brevity
40    }
```

Listing 425: The increment operator (`++`) of the `Time` class; an extract of `Time.C`.

From Line 23 to 27 of Listing 425 we see the code which generates the warning message we saw in Listing 41 in Section 11.3.2.

In Lines 21 and 29 we find the hard-coded limit for the time precision. If the time precision reaches a value of 100, then it is no more increased.

Naming the time with precision

In Section 11.3.2 we saw that the value of the `timePrecision` can be a source of error. We will now elaborate on the actual causes of this error.

Listing 426 shows the definition of the method `timeName(const scalar)`. This method is used to create a properly formatted time name¹⁷⁹ from a given scalar representing the time. In this method the time precision comes into play in the form of the data member `precision_`, which is a static data field of the `Time` class with a `protected` visibility. In this method the time value with high precision is converted to a string representation (the time name) with limited precision¹⁸⁰.

¹⁷⁹The data type of the return value of this method is `word`, which is a string data type of OpenFOAM. Thus, the time name is a string representation of the time. It is important to note, that the string representation of the time is different than the actual value of the time.

¹⁸⁰It is this method which creates the time name 0.102 from the time value 0.1023, when precision is set to three digits, as it is the case in the example described in Section 11.3.2.

```

1  //- Return time name of given scalar time
2  Foam::word Foam::Time::timeName(const scalar t)
3  {
4      std::ostringstream buf;
5      buf.setf(ios_base::fmtflags(format_), ios_base::floatfield);
6      buf.precision(precision_);
7      buf << t;
8      return buf.str();
9  }

```

Listing 426: The method `timeName(const scalar)` of the class `Time`; an extract of `Time.C`. Note, that the descriptive comment is taken from the header file `Time.H`.

When the time is advanced, e.g. using the increment operator of the `Time` class, the method `setTime()` is called. Listing 427 shows the definition of this method. The new time value is passed to this method. In the second instruction we see how the time name is updated to the new value¹⁸¹.

```

1  void Foam::Time::setTime(const scalar newTime, const label newIndex)
2  {
3      value() = newTime;
4      dimensionedScalar::name() = timeName(timeToUserTime(newTime));
5      timeIndex_ = newIndex;
6  }

```

Listing 427: The method `setTime()` of the class `Time`; an extract of `Time.C`.

The method `setTime()` gets called e.g. by the operator `*` of the `Time` class, see Line 8 of Listing 425. There, the time index is increased by one. From the header file of the `TimeState` class, we see, that the time index is of the data type `label`, which is essentially an integer data type. Thus, we see, that the time index is a consecutive number counting the time steps.

57.6.4 The Courant number

The Courant number Co is the ratio of the time step Δt and the characteristic convection time scale $u/\Delta x$. Eq. (168) shows the definition of the Courant number. However in a practical CFD code the Courant number will be computed in a slightly different way. Eq. (169) shows how Eq. (168) is expanded with A/A to gain a formulation featuring the flux and the volume of the control volume instead of the velocity and the discretisation length. Eq. (170) shows the extension of Eq. (169) for a one-dimensional finite volume formulation. The mean of the fluxes of the faces E and W defines the convective time scale. This definition seems obvious in some way in the one-dimensional case. For two or three-dimensional cases the choice of how to define the characteristic flux seems not straight forward.

$$Co = \frac{u\Delta t}{\Delta x} \quad (168)$$

$$Co = \frac{u\Delta t}{\Delta x} = \frac{u\Delta t}{\Delta x} \frac{A}{A} = \frac{\phi\Delta t}{\Delta V} \quad (169)$$

$$Co = \frac{\frac{|\phi_E| + |\phi_W|}{2} \Delta t}{\Delta V} = \frac{1}{2} \frac{(|\phi_E| + |\phi_W|) \Delta t}{\Delta V} \quad (170)$$

The Courant number in OpenFOAM

In OpenFOAM the Courant number is computed for all cells. In fact OpenFOAM computes a maximum Courant number, i.e. the largest Courant number of all cells, and a mean Courant number, i.e. the mean Courant number of all cells.

Listing 428 shows the code responsible for computing the Courant number. Line 8 of Listing 428 translates to Eq. (171). `sumPhi` is a scalar field containing the sum of the magnitudes of all face fluxes of every cell, i.e. for each cell the magnitude of the face fluxes are summed up. Eq. (171) holds for every cell.

¹⁸¹The call of `timeToUserTime()` can be ignored. This method simply returns the passed value. This method has a non-trivial implementation in the `engineTime` class, which keeps track of time in terms of engine RPM and crank-shaft angle. `engineTime` is derived from `Time`.

Eq. (172) is the mathematical representation of line 11. There the maximum value of the ratio between the values of `sumPhi` and the cell volume is determined. Both variables `sumPhi` and `mesh.V()` contain values for every cell. Therefore the `gMax()` function returns the maximum value.

Eq. (173) represents line 14.

```

1 scalar CoNum = 0.0;
2 scalar meanCoNum = 0.0;
3
4 if (mesh.nInternalFaces())
5 {
6     scalarField sumPhi
7     (
8         fvc::surfaceSum(mag(phi))().internalField()
9     );
10
11     CoNum = 0.5*gMax(sumPhi/mesh.V().field())*runTime.deltaTValue();
12
13     meanCoNum =
14         0.5*(gSum(sumPhi)/gSum(mesh.V().field())*runTime.deltaTValue());
15 }
16
17 Info<< "Courant Number mean: " << meanCoNum
18 << " max: " << CoNum << endl;

```

Listing 428: The content of the file `CourantNo.H`

$$\text{sumPhi} = \sum_{f_i} |\phi_{f_i}| \quad (171)$$

$$\text{CoNum} = \frac{1}{2} \max_{\text{all cells}} \left(\frac{\text{sumPhi}}{V_{\text{cell}}} \right) \Delta t \quad (172)$$

$$\text{meanCoNum} = \frac{1}{2} \frac{\sum \text{sumPhi}}{\sum V_{\text{cell}}} \Delta t \quad (173)$$

Discussion

The way to compute the Courant number in a three dimensional case is not straight forward as mentioned above. This section reflects the authors way of understanding. So there is no guarantee of validity. The factor of $1/2$ and the summation of ϕ_{f_i} is explained by the author as follows.

We base our reflections on a two dimensional control volume. Eq. (175) shows the summation written in the long form. This equation is then rearranged to yield Eq. (176). In Eq. (176) the summation is reduced to two terms. These terms are the arithmetic mean of the face flux in the principal directions $N - S$ and $W - E$. This summation is then identified as the L_1 norm of the mean face fluxes in the principal directions.

The reason for choosing the L_1 norm is not self-evident. In any case is the L_1 norm computationally cheaper than the Euklidian or L_2 norm. However, the use of the L_1 norm seems justified since it measures the distance covered by a movement, see http://en.wikipedia.org/wiki/Taxicab_geometry.

$$Co = \frac{1}{2} \frac{\sum_{f_i} |\phi_{f_i}|}{V_{\text{cell}}} \Delta t \quad (174)$$

$$Co = \frac{1}{2} \frac{|\phi_N| + |\phi_E| + |\phi_S| + |\phi_W|}{V_{\text{cell}}} \Delta t \quad (175)$$

$$Co = \frac{\frac{|\phi_N| + |\phi_S|}{2} + \frac{|\phi_E| + |\phi_W|}{2}}{V_{\text{cell}}} \Delta t \quad (176)$$

$$Co = \frac{\overline{|\phi|}^{NS} + \overline{|\phi|}^{WE}}{V_{\text{cell}}} \Delta t \quad (177)$$

$$Co = \frac{\| \overline{|\phi|}^{\mathbf{x}_i} \|_1}{V_{\text{cell}}} \Delta t \quad (178)$$

We introduce the following symbols

$$\frac{1}{2} \sum_{f_i} |\phi_{f_i}| = \|\overline{|\phi|}^{\mathbf{x}_i}\|_1 = \|\Phi\|_1 \quad (179)$$

$$Co = \frac{\|\Phi\|_1}{V_{cell}} \Delta t \quad (180)$$

The way the mean Courant number is computed seems incorrect at the first glance but it isn't.

$$Co = \frac{\|\Phi\|_1}{V_{cell}} \Delta t \quad (180)$$

The mean value of the quantity x is defined as follows

$$\bar{x} = \frac{1}{N} \sum_{i=1}^N x_i \quad (181)$$

Next we write the mean value of the Courant number. An unmarked summation is a summation over all cells.

$$\overline{Co} = \frac{1}{N} \sum \left(\frac{\|\Phi\|_1}{V_{cell}} \right) \Delta t \quad (182)$$

$$\overline{Co} = \frac{1}{N} \underbrace{\sum V_{cell}}_{=1} \underbrace{\sum \|\Phi\|_1}_{=1} \sum \left(\frac{\|\Phi\|_1}{V_{cell}} \right) \Delta t \quad (183)$$

$$\overline{Co} = \sum \frac{\|\Phi\|_1}{V_{cell}} \underbrace{\frac{1}{N} \sum V_{cell} \sum \left(\frac{\|\Phi\|_1}{V_{cell}} \right)}_X \Delta t \quad (184)$$

Eq. (184) now resembles Eq. (173). Now we concentrate on the term X which is the only difference between Eqns. (184) and (173).

$$X = \frac{1}{N} \sum \frac{V_{cell}}{\|\Phi\|_1} \sum \left(\frac{\|\Phi\|_1}{V_{cell}} \right) \quad (185)$$

$$X = \underbrace{\frac{\sum V_{cell}}{N}}_{=V_{cell}} \frac{1}{\sum \|\Phi\|_1} \sum \left(\frac{\|\Phi\|_1}{V_{cell}} \right) \quad (186)$$

$$X = \frac{\overline{V_{cell}}}{\sum \|\Phi\|_1} \sum \left(\frac{\|\Phi\|_1}{V_{cell}} \right) \quad (187)$$

$$X = \frac{1}{\sum \|\Phi\|_1} \sum \left(\frac{\|\Phi\|_1}{\frac{V_{cell}}{V_{cell}}} \right) \quad (188)$$

We assume $\frac{V_{cell}}{V_{cell}} \approx 1$

$$X = \frac{1}{\sum \|\Phi\|_1} \sum \left(\frac{\|\Phi\|_1}{1} \right) \quad (189)$$

$$X = \frac{\sum \|\Phi\|_1}{\sum \|\Phi\|_1} = 1 \quad (190)$$

Thus we have shown that the way the mean Courant number `meanCoNum` is computed is actually the mean Courant number $\overline{Co}$. However, this attempt of a proof is based on some assumptions.

First, the way the author explains the meaning of the summation of the face fluxes relies on hexahedral cells. The argument made seems not to be applicable on tetrahedral cells. Secondly, the assumption $\frac{V_{cell}}{V_{cell}} \approx 1$ is valid for homogeneous grids. For a uniform grid this assumption would be ideally fulfilled. If the volume of the largest and smallest cells differs a lot this assumption is not justified.

Some thoughts on the computational costs

Why the formula for the mean Courant number is rearranged from

$$\overline{Co} = \frac{1}{N} \sum \left(\frac{\|\Phi\|_1}{V_{cell}} \right) \Delta t \quad (191)$$

to

$$\overline{Co} = \frac{\sum \|\Phi\|_1}{\sum V_{cell}} \Delta t \quad (192)$$

is unknown to the author.

It is the opinion of the author that this is made for reasons of computational cost. Two times the summation over all values of a field plus one division is computationally cheaper than an elementwise division of two fields and one subsequent summation over all elements of the resulting field.

This would be the case if the division operation takes more time than the summation operation which is very likely the case. Depending on the system the floating point division operation can take several times longer than a floating point multiplication.

In the first case n times one division and one addition needs to be made, with n the number of field values. In the second case $2n$ times additions and one division is to be made.

$$T_1 = n(T_d + T_s) \quad T_2 = 2nT_s + T_d \quad (193)$$

We introduce the factor δ , that is the ratio between T_d and T_s .

$$T_1 = n(\delta T_s + T_s) \quad T_2 = 2nT_s + \delta T_s \quad (194)$$

$$T_1 = nT_s(1 + \delta) \quad T_2 = T_s(2n + \delta) \quad (195)$$

$$\frac{T_1}{T_s} = n(1 + \delta) \quad \frac{T_2}{T_s} = (2n + \delta) \quad (196)$$

Next we assume that n is very large

$$\frac{T_1}{T_s} = n(1 + \delta) \quad \frac{T_2}{T_s} \approx 2n \quad (197)$$

So the first formula takes $1 + \delta$ operations, whereas the second formula takes approximately $2n$ operations. If δ is larger than one, the second formula will take less time for computation. A δ smaller than one is highly unlikely or even impossible as the addition is a very simple operation. Remember, δ is the ratio between the time a division takes and the time an addition takes. The actual ratio vary according to the system architecture, the compiler and the implementation, e.g. [1] reports a factor of 5 to 6 for single and double precision floating point division. This argument does not consider the memory usage of the operations involved, it only focuses on the number of floating point operations.

Because the Courant number is computed after every time step the time needed to calculate the Courant number has an impact on the simulation time.

57.6.5 The two-phase Courant number

In a two-phase simulation there are several choices of how to compute the Courant number. In total, there are 4 velocity fields (U_1 , U_2 , U and U_r). These are the velocities of the phases 1 and 2 as well as the mixture and relative velocities. The solver *twoPhaseEulerFoam* computes the Courant number for the mixture and the relative velocities.

Listing 429 shows the content of the file `CourantNos.H` which is part of the source code of this solver. Line 1 computes the mixture Courant number by including the file `CourantNo.H`. This is the file described in Section 57.6.4. As this code operates on the field `phi`, which happens to be the flux of the mixture, the mixture Courant number is computed.

The next lines compute the Courant number based on the relative phase flux. At line 11 the maximum of this two Courant numbers is determined and stored into the variable `CoNum`.

`CoNum` is the Courant number used by the time stepping mechanism. So the variable time steps of the *twoPhaseEulerFoam* solver are based on the maximum of the mixture and relative velocity Courant number.

```

1  #include "CourantNo.H"
2
3  {
4      scalar UrCoNum = 0.5*gMax
5      (
6          fvc::surfaceSum(mag(phi1 - phi2))().internalField()/mesh.V().field()
7      )*runTime.deltaTValue();
8
9      Info<< "Max Ur Courant Number = " << UrCoNum << endl;
10
11     CoNum = max(CoNum, UrCoNum);
12 }

```

Listing 429: The content of the file `CourantNos.H`

57.7 The registry

At some point in our study of OpenFOAM's sources, its documentation or the internet we all came across words like *registered objects* or similar expressions. This section tries to cast some light on this topic, or at least present the thoughts and findings of the author. This section is closely related to Section 57.8.

57.7.1 The classes involved

Here is an extract of the descriptions found in the header files of the respective classes.

IObject `IObject` defines the attributes of an object for which implicit `objectRegistry` management is supported, and provides the infrastructure for performing stream I/O.

regIOobject `regIOobject` is an abstract class derived from `IObject` to handle automatic object registration with the `objectRegistry`.

objectRegistry registry of `regIOobjects`

In Figure 132 a detail of the class hierarchy surrounding the class `regIOobject` is shown.

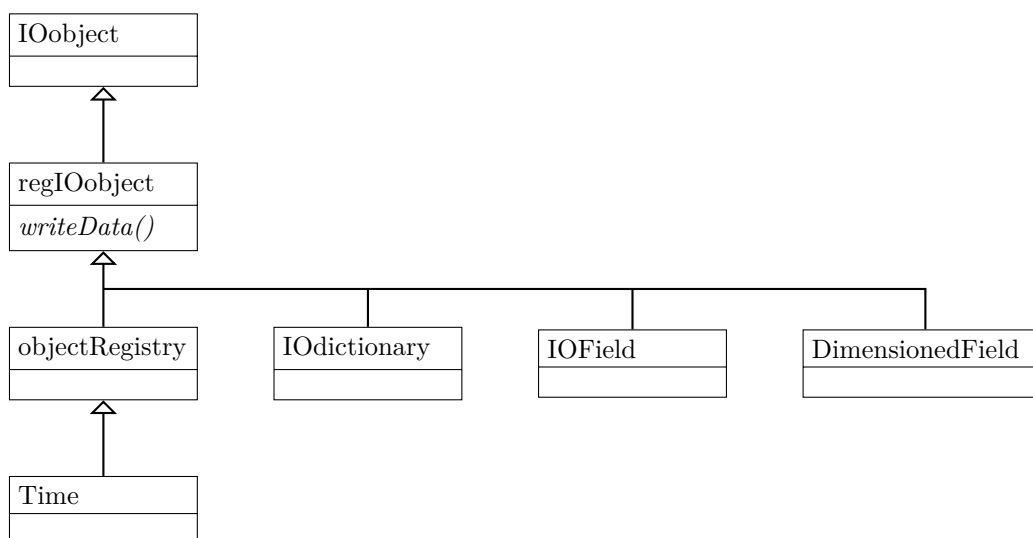


Figure 132: A partial view of the class hierarchy involving `regIOobject`; note that this diagram is complete only for the classes `IObject` and `regIOobject` – meaning `IObject` is not derived from any other class and `regIOobject` is derived from only `IObject`; the other classes have more base classes than shown in this diagram.

IObject

This class provides the basic facilities for I/O. In Section 57.4.5 the practical or typical use of this class is shown.

regIOobject

This class is an abstract class as the description in the header mentions. In Figure 132 the name of the pure virtual method which makes this class an abstract class is shown in an italic font. This means all classes derived from `regIOobject` must implement this pure virtual method. This also means, that we can not create an object of the type `regIOobject` directly. Thus, in all of OpenFOAM's sources we find a constructor call for the class `regIOobject` only in the initializer list of classes derived from `regIOobject`.

objectRegistry

The `objectRegistry` is eponymous to this section. In fact there is not the one registry in OpenFOAM, there are several. Among others, the classes `Time`, `cloud`, and `polyMesh` are derived from `objectRegistry`. Figure 133 shows the classes from which `objectRegistry` is derived.

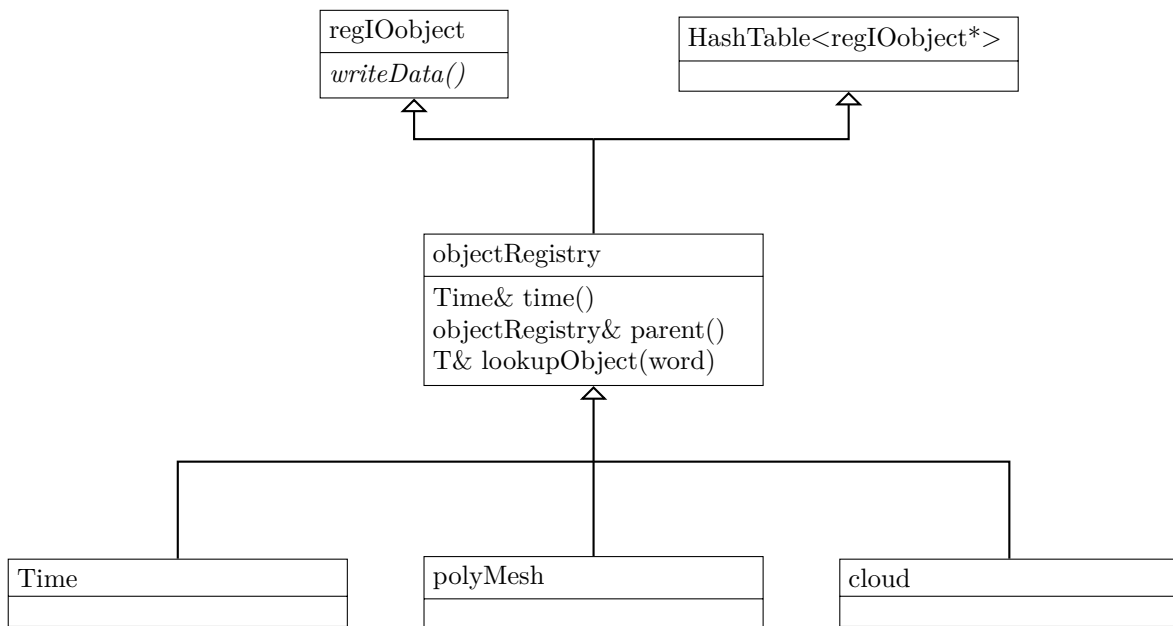


Figure 133: The base classes of the class `objectRegistry`; this class is derived from `regIOobject` and a `HashTable`; note that the template parameter of the `HashTable` is a pointer to `regIOobject`; thus `objectRegistry` is an `regIOobject` as well as a `HashTable` of `regIOobject` pointers – this is C++'s template madness and inheritance wizardry in action.

objectRegistries: space and time

In OpenFOAM there is usually only one object of the type `Time`, usually named `runTime`. There are no solvers to the knowledge of the author, which use more than one instance of the class `Time`. As most solvers also feature only one mesh, the separation between `Time` and `polyMesh` as being a registry seems to be overdone.

However, there are solvers which feature several meshes, e.g. the conjugate heat transfer solvers. In the simplest configuration there is one mesh for the solid part of the domain and one mesh for the fluid part of the domain. However, the solver `chtMultiRegionFoam` supports an arbitrary number of fluid and solid domains. In this case the temperature field `T` of the solid region *i* needs to be registered with the appropriate registry, namely the mesh of the solid region *i*. Since the fluid in the fluid region also has a temperature, there will also be a temperature field `T` registered to the appropriate fluid mesh. Thus, the separation of the object registries, i.e. mesh(es) and time, saves us from a potential name clash, as we have two temperature fields `T`. Thus, the mesh(es) are registered with the object registry `runTime`, whereas fields are registered with the appropriate mesh.

An OpenFOAM solver has a number of object registries in use, the most prominent are the `runTime` and the `mesh` objects. For fields it is important to know that they belong to a mesh, since the entity *field* is a mere list of values. Only the connection to the mesh gives the field an actual meaning, i.e. the entry at position *i* in

the list is the cell centre value of cell i . Furthermore, the field also needs a connection to the actual time state of the simulation, otherwise there would be no meaningful way to define or calculate a temporal derivative.

57.7.2 Using the registry

Of what use could a possible object registry be? Well, ask the code.

In Section 66.3 we showed a way to search files for a certain pattern. Now we search all files with the file extension `.C` for the pattern `lookupObject` and count the hits¹⁸². Listing 430 shows the command we can use. First we use `find` to look for all files with the specified pattern for the file name. The result is then piped to `grep` which searches the files for the specified pattern. Lastly, the result of `grep` is piped to `wc`, which counts lines, words and bytes. Thus the first number returned by this sequence of commands tells us the number of hits. The actual number of hits is approximately half the displayed number, since in the process of building OpenFOAM from sources, symbolic links are created within the `lnInclude` folders¹⁸³.

```
find $FOAM_SRC -name '*.C' | xargs grep 'lookupObject' | wc
```

Listing 430: Find and scan files with file extension `.C` for the pattern `lookupObject` and count the hits

The command of Listing 430 results in 1068 hits in the author's OpenFOAM-2.3.x installation at the time of writing. 537 of these hits come from symbolic links of `lnInclude` directories. This means that `lookupObject()` gets used a lot. So what is `lookupObject()` good for?

Need to know vs. want to know

One principle of *encapsulation* or *information hiding* is a fundamental principle of object-oriented programming¹⁸⁴. The general idea is to hide the actual implementation of something behind a publicly accessible interface. Thus, the inner workings of a class may change without affecting its use. The iterator concept is a good example of the benefits of information hiding. Typical container classes implement a feature called iterators that are used to iterate over all elements of the container. By using the public interface of the iterator, the actual container behind may be any kind of data structure (a linked list, a vector, a hash table, etc.).

Besides providing and using interfaces for accessing the data of a class it is also a common and good practice to restrict the scope of data, e.g. temporary data being local to the class or method where it is actually used. Thus, in the design of the classes we implement we limit the data contained within and/or passed to the class to the necessary minimum, i.e. the viscosity law used in a solver does not need to know about the solver we used to solve the discretized equation system. However, there might arise the need to access data, which the original designers of a certain family of classes did not anticipate.

Namespaces & scopes

Another aspect are namespaces and variable scopes within our source codes. A variable is visible in the namespace and scope it is declared. If we look at the top level code of a solver, e.g. `twoPhaseEulerFoam`, we see a lot of `#include` statements and the `main()` method of the program. Although, we find no direct statement involving the namespace, in the file `fvCFD.H` a statement is hidden which causes the compiler to use the namespace `Foam`. This is the reason why we can later e.g. in `createFields.H` use typenames such as `volScalarField` which are defined in the namespace `Foam`. Otherwise we would need to explicitly specify the namespace as well, e.g. `Foam::volScalarField`. Thus, all objects created by a solver such as `mesh`, `runTime`, etc. are visible in the namespace `Foam`.

Models however, have their own namespaces. Listing 431 shows an example of such a model with its own namespace. Within the namespace `Foam` a new namespace `diameterModels` is created. Within this namespace the class `isothermal` is defined. Thus the classes implementing diameter models do not pollute the `Foam` namespace.

Although, the `diameterModels` namespace is a subset of the `Foam` namespace and everything declared within `Foam` is also visible within `Foam::diameterModels`, the diameter models are compiled with other models into a

¹⁸²The method `lookupObject()` can be used to ask the registry for a registered object. The usefulness will be explained in the subsequent paragraphs.

¹⁸³The `lnInclude` folders collect links to all files of a certain library, thus when compiling a solver that uses this library we need to include only the `lnInclude` folder and not the whole directory tree of the library's sources. This minimizes the number of entries in the `Make/options` files.

¹⁸⁴Information hiding and encapsulation are often used synonymously, however, strictly spoken they are not exactly the same.

shared library. Thus, when these files are compiled, the compiler knows nothing of the objects in the namespace `Foam` created in e.g. `createFields.H`.

```
namespace Foam
{
    namespace diameterModels
    {

        class isothermal
        :
            public diameterModel
        {
            // code removed
        }
    }
}
```

Listing 431: The class definition of the `isothermal` class, derived from the class `diameterModel` in `isothermalDiameter.H`

Looking up stuff

Listing 432 shows the definition of the method `d()` of the class `isothermal`. For the reasons explained above `isothermal.C` and `createFields.H` being in different compilation units, we can not access the pressure field `p` directly from within the method body, even though `p` is part of the namespace `Foam`. However, other diameter models do not need to access the pressure field, e.g. `constant` which implements a constant diameter.

```
Foam::tmp<Foam::volScalarField> Foam::diameterModels::isothermal::d() const
{
    const volScalarField& p = phase_.U().db().lookupObject<volScalarField>
    (
        "p"
    );

    return d0_*pow(p0_/p, 1.0/3.0);
}
```

Listing 432: The definition of the method `d()` of the class `diameterModel` in `isothermalDiameter.C`

The example above shows the value of the lookup mechanism. Since some sub-models operate on some fields, it is easy to get a reference to the mesh from the field, as it is done in `phase_.U().db()`. `phase_` is a member of the base class of the diameter models¹⁸⁵. The call `phase_.U()` returns a reference to the velocity field of the phase in question. As the velocity field is registered with the mesh otherwise we wouldn't know which velocity value belongs to a certain cell we get a reference to the mesh by calling `db()`, which is a method of the class `IObject`. This handy mechanism saves us from polluting sub-models with references to the mesh, the time, to fields we might need at some point or some derived classes might need in special cases.

Thus the `lookupObject()` method provides a tool for us to get references to fields which at compile-time may not be declared and thus usable. Remember, the pressure field is declared in the solver's `createFields.H` file, which is in a different compilation unit as the library we are compiling our diameter model for. If the code of the diameter model and the solver would be in the same compilation unit (the solver's executable) we would not need the lookup mechanism. However, since the developers of OpenFOAM aim for modularity, placing everything into a single compilation unit is against the design principles of modularity and reusability.

The `lookupObject()` method is templated since we can register anything with the mesh, in fact anything that is derived from `regIOobject`, since an `objectRegistry` is a `HashTable` of `regIOobject` pointers. Thus, at compile-time the method and the compiler do not know exactly which data types it is going to handle. This is where templates come into play. The templated method is implemented once for the template parameter, and when we use the method, we simply replace the template parameter with the actual type, as in `lookupObject<volScalarField>("p")`. The compiler then does the rest of the work and generates the appropriate code. We could resolve this issue without templates by using function overloading at the price of massive code duplication and poor maintainability.

¹⁸⁵It is a convention of OpenFOAM's developers to append an underscore character (`_`) to the names of the data members of a class in order to make them easily distinguishable from method parameters.

57.7.3 Printing the registry

If you are curious you can add the following lines of code to a test utility of yours to check what is registered with the `mesh` and the `runTime` object registry. Note that `mesh` and `runTime` must be accessible from the place you put the code into. Also the names of the objects might differ in some cases.

```
Info << "mesh.names() " << mesh.names() << nl << endl;
Info << "runTime.names() " << runTime.names() << endl;
```

Listing 433: Printing the contents of the object registries `mesh` and `runTime` to Terminal

57.8 I/O - input & output

Some aspects of I/O were already covered in Sections 57.4.5 and 57.3. However as this collection of stuff is fragmented by design or by the lack of such we cover the topic of I/O in a more general manner.

57.8.1 Output to Terminal - OpenFOAM's very own `printf()`

In programming we have often the need to print stuff to the Terminal, e.g. for `printf()` debugging¹⁸⁶. With C++ general I/O was implemented on the basis of I/O streams. C++'s I/O streams provide a type-safe and uniform way to implement I/O for both built-in and user-defined types [57]. See Listings 434 and 435 for the use of C's `printf()` function and C++'s streams.

```
#include <stdio.h>

int main(int argc, char** argv)
{
    printf("Hello, World!\n");

    return 0;
}
```

Listing 434: The *Hello World!* example of C.

```
#include <iostream>

int main()
{
    std::cout << "Hello World!" << std::endl;

    return 0;
}
```

Listing 435: The *Hello World!* example of C++.

OpenFOAM implements its own stream library. The generic stream library of OpenFOAM is based on the class `Iostream`. The description of this class in its header file sheds some light on the reasons for doing so:

An `Iostream` is an abstract base class for all input/output systems; be they streams, files, token lists etc.

The basic operations are construct, close, read token, read primitive and read binary block. In addition version control and line number counting is incorporated. Usually one would use the read primitive member functions, but if one were reading a stream on unknown data sequence one can read token by token, and then analyse.

OpenFOAM handles all kinds of communication in terms of streams, among others: Terminal I/O with the user, file I/O and inter-process communication for parallel processing. The *Hello World!* example for the OpenFOAM world in Listing 436 looks very similar to the example of C++.

¹⁸⁶Named after C's ubiquitous `printf()` function, see <http://stackoverflow.com/a/189570/2055536>

```

#include "Istream.H"

using namespace Foam;

int main(int argc, char *argv[])
{
    Info << "Hello OpenFOAM!" << endl;

    return 0;
}

```

Listing 436: The *Hello World!* example written in OpenFOAM.

Conditional (debug) output

`printf()` debugging is a very handy, low-level technique to trouble-shoot pieces of code. In the case of actual debugging, we will remove all lines of code printing to the Terminal once we are done debugging. However, we might want to create software, which may be either talkative or silent¹⁸⁷. In this case we need conditional `Info` statements.

Listing 437 shows a *Hello World!* example with conditional output. This listing is quite lengthy, since we decided not to use simple boolean to control the conditional output. Instead we opted for a real case scenario, in which the verbosity is controlled by a command line option. This, however, entailed some more lines of code to deal with command line parameters.

```

#include "argList.H"

bool verbose(false);

using namespace Foam;

int main(int argc, char *argv[])
{
    argList::addNote
    (
        "This is a \"Hello World!\" program for the OpenFOAM world."
    );

    argList::noBanner();
    argList::noParallel();

    argList::removeOption("noFunctionObjects");
    argList::removeOption("case");

    argList::addBoolOption
    (
        "verbose",
        "control the chatty-ness of me"
    );
    Foam::argList args(argc, argv);

    if (args.optionFound("verbose"))
    {
        verbose = true;
    }

    Info << "Hello OpenFOAM!" << endl;

    if (verbose) Info << "... and hello to all other non-OpenFOAM worlds!" << endl;

    return 0;
}

```

Listing 437: The *Hello World!* example written in OpenFOAM with conditional chattiness.

¹⁸⁷Have you ever come across `-v` or `--verbose` command line switches when using UNIX or LINUX computers?

In addition to the boolean command line switch, we added a note informing the user about the executable. This note gets displayed, when the usage message is shown by invoking the executable with the command line option `-help`. OpenFOAM adds a number of command line parameters by default, thus we remove some of them (the ones that make no sense for a *Hello World!* program, such as the parallel option).

The second to last line of code is the one that actually controls the conditional output. This is done by a good old `if` statement.

In the source code of the function objects of OpenFOAM-2.3.x we observed another possibility to define conditional output. There, we can pass an argument to `Info`. With OpenFOAM-2.4.x and higher versions this does not compile anymore. Listing 438

```
// OpenFOAM-2.3.x
Info(log_)<< "    Including porosity effects" << endl;

// OpenFOAM-2.4.x and higher
if (log_) Info<< "    Including porosity effects" << endl;
```

Listing 438: Implementing conditional output, controlled by the `Switch log_`, in different OpenFOAM versions. This example is taken from the `force` function object. See the file `force.C`.

A variant of the conditional `Info(log_)` statement was reinstated in May 2016¹⁸⁸, with the `Log` macro, which is defined as follows:

```
//- Report write to Foam::Info if the local log switch is true
#define Log
    if (log) Info
```

Listing 439: The definition of the `Log` macro, originally for conditional output of function objects. See the file `messageStream.H`.

57.8.2 The registry and the I/O or the truth behind `runTime.write()`

Registering fields with the `runTime` object registry also allows makes our lives easier when we want to write the current state of the simulation to disk. In a great number of solvers, possibly in all of them, we find an instruction like `runTime.write()` within the main loop of the `main` method. This call to the method `write()` causes fields to be written to disk. As every solver write a different set of fields to disk, we may ask ourselves how the solver or OpenFOAM knows which fields to write when we call the `write()` method of the `runTime` object? Here, the registry nature of the `Time` class comes into play. Since we register all our fields, which we eventually want to read or write, with the `runTime` object, the `runTime` object has a list of objects (`regIOobjects` in fact) which are to (or might) be written¹⁸⁹. In fact, since `objectRegistry` is derived from the type `HashTable`, an object registry *is a* list of objects which are to (or might) be written¹⁹⁰. The call of the write method of the `Time` class causes `Time` to iterate over its self (`runTime is a` list of `regIOobjects` by inheritance¹⁹¹) and call the `write()` method of every single item within the list. The method `write()` is defined in the `regIOobject` class.

The closer look into the sources is revealing if we take some of C++'s rules into consideration. Listing 440 shows us the method that is called when we call `write()` on `runTime`, bear in mind that `Time` is derived in second generation from `regIOobject` via the class `objectRegistry`. The listing shows a call of the method `writeObject()`.

```
bool Foam::regIOobject::write() const
{
    return writeObject
    (
        time().writeFormat(),
        Iostream::currentVersion,
        time().writeCompression()
    )
}
```

¹⁸⁸<https://github.com/OpenFOAM/OpenFOAM-dev/commit/48e58170392e5ef60538cfef28dd70c70a62e17b>

¹⁸⁹depending on the write flags of the `IOobject` part of the type. See Section 57.4.5 for a discussion on the read and write flags of the `IOobject` class.

¹⁹⁰A hash table is not really a list, however, we can iterate over a hash table the same way we can iterate over a list. The description in the header file of the `HashTable` class describes the class as being *An STL-conforming hash table*.

¹⁹¹think around the family tree, e.g. in Figure 133

```

1  bool Foam::Time::writeObject
2  (
3      Iostream::streamFormat fmt,
4      Iostream::versionNumber ver,
5      Iostream::compressionType cmp
6  ) const
7  {
8      if (outputTime())
9      {
10         // some code removed
11
12         timeDict.regIOobject::writeObject(fmt, ver, cmp);
13         bool writeOK = objectRegistry::writeObject(fmt, ver, cmp);
14
15         // further code removed

```

Listing 441: Parts of the method `writeObject()` of the class `Time` in `TimeIO.C`

```

    );
}

```

Listing 440: The method `write()` of the class `regIOobject` in `regIOobjectWrite.C`

If we search the sources of `Time` and all its base classes we find out that `Time`, `regIOobject` and `objectRegistry` all define a method called `writeObject()`¹⁹². All of these three methods share the same signature¹⁹³, i.e. they receive the same function arguments. Since the call of `writeObject()` is not further specified for a certain namespace, it is the method `writeObject()` of the class `Time`, which is called when we call `runTime.write()` as `runTime` is of the type `Time`.

In Listing 441 we see a portion of the definition of the method `writeObject()` of the class `Time`. There we also see calls explicitly to the methods `writeObject()` of the classes `regIOobject` and `objectRegistry`.

Thus, the method `writeObject()` of all three classes (`Time`, `regIOobject` and `objectRegistry`) are called when `runTime.write()` is called. It is worth noticing that the call of `regIOobject::writeObject()` is invoked on the `timeDict` object. The definition of this object is part of the removed code prior to the call. A look into the source code reveals, that `timeDict` is an `IOdictionary` which is a class also derived from `regIOobject`, see Figure 132. The call of `timeDict.writeObject()` is the piece of code which creates the `uniform` folders within the time step directories¹⁹⁴.

The method `writeObject()` of the class `objectRegistry` does the actual iteration over all elements within the registry. Listing 442 shows the actual iteration over the hash table of `regIOobject` pointers. For each element `writeObject()` is called if the write flag is not set to `NO_WRITE`. Now the method `writeObject()` of the class `regIOobject` is called, since the iteration is over `regIOobject` pointers. This call on Line 16 of Listing 442 causes a registered field to be written to disk.

```

1  bool Foam::objectRegistry::writeObject
2  (
3      Iostream::streamFormat fmt,
4      Iostream::versionNumber ver,
5      Iostream::compressionType cmp
6  ) const
7  {
8      bool ok = true;
9
10     forAllConstIter(HashTable<regIOobject*>, *this, iter)
11     {
12         // code removed handling debug output
13
14         if (iter()->writeOpt() != NO_WRITE)
15         {

```

¹⁹²The arguments of the function are dropped in the text for the sake of brevity. In fact there is no method named `writeObject()` with an empty parameter list. This can be checked via these commands: `find $FOAM_SRC -name '*.CH' | xargs grep 'writeObject()'`

¹⁹³The function signature consists of the name of the function and its parameters.

¹⁹⁴In case you ever wondered where these come from.

```

16         ok = iter()->writeObject(fmt, ver, cmp) && ok;
17     }
18 }
19
20 return ok;
21 }

```

Listing 442: Parts of the method `writeObject()` of the class `objectRegistry` in `objectRegistry.C`

In conclusion we have learned by digging the source code of OpenFOAM the magical inner workings of the call `runTime.write()`. First the `Time` class writes its state to disk into the `uniform` folder and then the `objectRegistry` part of the `runTime` object writes all registered fields. It was already mentioned in Section 57.6 that the class `Time` has a multiply divided personality. And some of those even bring along an ancestry. This highlights the need to have a certain understanding of C++ in order to be able to deduce what's going on from the sources of OpenFOAM as OpenFOAM makes very heavy use of C++'s language features such as multiple inheritance, polymorphism and templates. In the context of programming paradigms involved, OpenFOAM makes use of (among others): *object-orientation* and *generic programming*.

57.9 Making an argument – passing arguments

In Listing 437 of Section 57.8 command line arguments were used to influence an application's behaviour. In Section 11.1 different means of exerting control over an application are discussed. There, the point was made that command line arguments are the lowest level of control over an application. Command line arguments need to be specified each time an application is run. Only default values for optional arguments are permanent.

This section discusses some points about command line arguments.

57.9.1 The order of things

If we study applications, which make use of their own command line arguments, we see that there is certain order of things. Listing 443 shows a minimal example of how to define command line arguments.

First, the static method `addBoolOption` method is called to add our own command line argument. Then, the file `setRootCase.H` is included. Listing 444 shows the contents of this file. We see that a variable of the type `Foam::argList` is created and all command line arguments¹⁹⁵ are passed to the constructor of `Foam::argList`. The constructor of `Foam::argList` only performs checks on the validity of the already defined options. In fact no further options can be added after the call to the constructor of `Foam::argList`. Once the constructor has been called, an object named `args` exists, which can be used to lookup options and extract information.

```

argList::addBoolOption
(
    "verbose",
    "be more talkative"
);

#include "setRootCase.H"

const bool verbose = args.optionFound("verbose");

```

Listing 443: The order of things in the source code for defining command line arguments

```

Foam::argList args(argc, argv);
if (!args.checkRootCase())
{
    Foam::FatalError.exit();
}

```

Listing 444: The content of the file `setRootCase.H`

¹⁹⁵In C++ `argc` is the number of command line arguments, and `argv` is the actual command line arguments. `argc` and `argv` are passed from the Terminal to the `main()` method of the application, see `main()`'s method signature: `int main(int argc, char *argv[])`

57.9.2 Dealing with SPAAAAACE!

Having a space in an argument's name is not really a good idea, since the Terminal generally interprets a space as the end of an argument. In common UNIX/Linux tools multi-word arguments are generally separated with hyphens, e.g. `--auto-compress`. In the OpenFOAM universe, we see the use of camel case¹⁹⁶, e.g. `-noFunctionObjects` to deal with multi-word argument names.

However, if we really want to have spaces within our argument's name, OpenFOAM allows us to do so. Listing 445 demonstrates how to define an argument named `search point`.

```
argList::addOption
(
    "search point",
    "vector",
    "find the cell containing the specified coords at <vector> - eg, '(1 0 0)'"
)

/* no comment */

vector v;
if (args.optionReadIfPresent("search point", v))
{
    Info<< "Searching cell at point: " << v << endl;
}
```

Listing 445: Reading a point's coordinates from a command line argument

Using an argument with a space in it, requires taking special care, as shown in Listing 446. Quotes are used to prevent the Terminal from interpreting the space within the argument's name as the end of the argument's name.

Listing 447 shows the danger of defining arguments with spaces. In this case the quotes were not used, just as we are used to.

```
findCellByPointCoords -'search point' '(-0.993389 -1.90411 12.4942)'
```

Listing 446: Passing a point's coordinate to an application; note the quotes around the argument's name

```
user@host:~$ findCellByPointCoords -search point '(-0.993389 -1.90411 12.4942)'
```

...

```
--> FOAM FATAL ERROR:
Wrong number of arguments, expected 0 found 2
Invalid option: -search

FOAM exiting
```

Listing 447: Passing a point's coordinate to an application; omitting the quotes around the argument's name leads to a misinterpretation

57.10 Turbulence models

In Section 32.3 it is stated that the user can choose between three options.

1. A laminar simulation
2. Using a RAS turbulence model
3. Using a LES turbulence model

This statement is reflected in the relationship between the classes implementing the turbulence models in OpenFOAM. Object oriented programming allows the programmer to translate relationships directly from human language to source code. Two statements can be made about turbulence models

¹⁹⁶https://en.wikipedia.org/wiki/Camel_case

1. All RAS turbulence models are turbulence models, but not all turbulence models are RAS turbulence models.
2. A RAS turbulence model is not the same as an LES turbulence model, however, both are turbulence models.

Both statements are reflected by the class diagram of the turbulence models. On the top is the abstract class `turbulenceModel`. This abstract class, provides the framework for all derived turbulence classes. Also, all functionality common to all possible turbulence classes can be defined in this class. All derived classes will then inherit this functionality.

Each turbulence model is derived from this abstract base class. Each turbulence class will implement specific functionality individually.

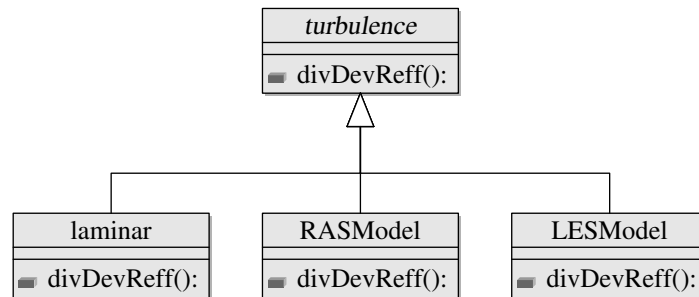


Figure 134: Graphic representation of inheritance of the turbulence model classes.

57.10.1 The abstract base class `turbulenceModel`

The base class `turbulenceModel` is an abstract class¹⁹⁷. It contains several pure-virtual functions. To be able to call this functions, these functions must be overridden by the classes that are derived from the base class. A pure-virtual class can not be called. Listing 448 shows the declaration of pure-virtual or abstract methods. The `= 0` indicates that a method is abstract.

```

// - Return the turbulence viscosity
virtual tmp<volScalarField> nut() const = 0;

// - Return the effective viscosity
virtual tmp<volScalarField> nuEff() const = 0;
  
```

Listing 448: Declaration of the virtual methods in `turbulenceModel.H`

The base class contains not only virtual functions. It also contains functions that are the same for all derived classes. Consequently, this functions are implemented by the base class. Listing 449 shows the implementation of the function `nu()`. This function is used to access the laminar or molecular viscosity. The laminar viscosity is a property of the fluid itself and has nothing to do with turbulence. However, the turbulence models need to access the laminar viscosity.

```

// - Return the laminar viscosity
inline tmp<volScalarField> nu() const
{
    return transportModel_.nu();
}
  
```

Listing 449: Implementation of `nu()` in `turbulenceModel.H`

Every class derived from an abstract class must at least override the abstract methods. The non-abstract methods of the base class – like `nu()` from Listing 449 – can be used by the derived classes. No matter if a RAS or a LES turbulence model is used, the laminar viscosity will always be the same.

¹⁹⁷A class that contains one or more abstract methods is called an abstract class. If a class contains only abstract methods, then it is sometimes called a pure-abstract class.

57.10.2 The class `RASModel`

The class `RASModel` is derived from the abstract class `turbulenceModel`. The class `RASModel` itself is the base class for all RAS turbulence models. It is also an abstract class because it does not override all abstract methods inherited from `turbulenceModel`.

However, the class `RASModel` implements all methods that are common to all RAS turbulence models. Listing 450 shows the implementation of the method `nuEff()` in the class `RASModel`.

```
//- Return the effective viscosity
virtual tmp<volScalarField> nuEff() const
{
    return tmp<volScalarField>
    (
        new volScalarField("nuEff", nut() + nu())
    );
}
```

Listing 450: Implementation of `nuEff()` in `RASModel.H`

The effective viscosity `nuEff` is calculated from the laminar viscosity, which is a property of the fluid, and the turbulent viscosity. The turbulent viscosity is a property of the turbulence model. The function `nu()` in Listing 450 is implemented in the class `turbulenceModel`, see Listing 449. The function `nut()` is not implemented by the class `RASModel`. Therefore, this method must be implemented by the classes derived from `RASModel`.

57.10.3 RAS turbulence models

All RAS turbulence models are derived from the class `RASModel`. Each derived class must implement all remaining abstract methods. Figure 135 shows a simplified class diagram – there is a number of RAS turbulence models available in OpenFOAM.

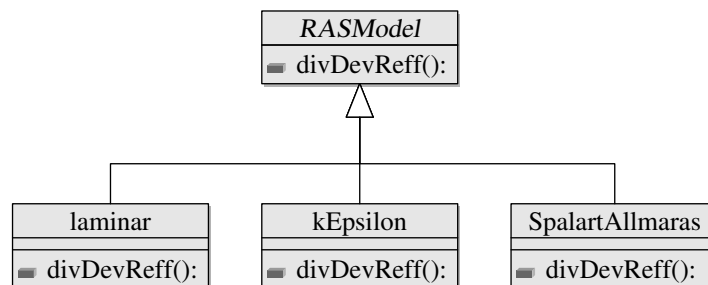


Figure 135: Inheritance of RAS turbulence models

57.10.4 The class `kEpsilon`

The class `kEpsilon` is derived from `RASModel`.

```
class kEpsilon
:
    public RASModel
{
    /* class definition */
}
```

Listing 451: Class definition of `kEpsilon` in `kEpsilon.H`

The function `nut()` has to be implemented by `kEpsilon`. Listing 452 shows how the function `nut()` is implemented. This function simply returns the class member `nut_`.

```

//- Return the turbulence viscosity
virtual tmp<volScalarField> nut() const
{
    return nut_;
}

```

Listing 452: Implementation of `nut()` in `kEpsilon.H`

The way how `nut_` is calculated differs between the RAS turbulence models. See Listing 486 in Section 61.2.2.

57.11 Debugging mechanism

OpenFOAM brings along a handy debugging mechanism. This mechanism can be used when creating additional model libraries. The OpenFOAM wiki features a section explaining the built-in debug mechanism¹⁹⁸.

The global debug flags – controlling the behaviour of the debugging system-wide – are specified in `\$FOAM_SRC/./etc/controlDict`. From OpenFOAM-2.2.0 onwards the global debug flags can be overridden by stating the debug flags of choice in the case's `controlDict`¹⁹⁹.

As this debugging mechanism relies on internal variables no re-compiling is involved when using this kind of debugging mechanism. This kind of debugging is sometimes referred to as *printf debugging*²⁰⁰.

By default all debug switches are initialised with a zero value, therefore the debug feature for the specific class is disabled. However, when the solver sets up the case, the global and local entries are checked. Listing 453 shows the entry in the `controlDict` to override debug switches. Listing 454 shows the solver output informing us of the local settings in `controlDict`.

```

DebugSwitches
{
    DefaultStability          0;
    YoonLuttrellAttachment    1;
}

```

Listing 453: Specifying debug switches in the case's `controlDict`

```

Overriding DebugSwitches according to controlDict
    DefaultStability 0;
    YoonLuttrellAttachment 1;

```

Listing 454: Solver output when specifying debug switches in the case's `controlDict`

57.11.1 Using the debugging mechanism

If the debugging mechanism is enabled for a class²⁰¹, Listing 455 shows how to actually use it. The code is amazingly simple. The magic behind the scenes provides a variable named `debug`. We simply use this variable in an `if` statement.

```

// print debug information
if (debug)
{
    // debug action
}

```

Listing 455: Using the debug mechanism in a class.

¹⁹⁸http://openfoamwiki.net/index.php/HowTo_debugging#Getting_built-in_feedback_from_OpenFOAM

¹⁹⁹<http://www.openfoam.org/version2.2.0/runtime-control.php>

²⁰⁰See <http://oopweb.com/CPP/Documents/DebugCPP/Volume/techniques.html> or <http://en.wikipedia.org/wiki/Debugging#Techniques>

²⁰¹See Section 57.12 on the background of the debugging mechanism.

57.11.2 Use case: Write intermediate fields

Listing 456 shows the definition of a method named `Ea`. For debugging purposes we want to write intermediate fields to disk. In Line 7 of Listing 456 we compute a Reynolds number and store it in `ReB`. This is used to generate the return value of the method. In normal operation only the return value is of interest. When debugging also intermediate results may be of interest. The field `ReB` is by default not written to disk and ceases to exist when the scope leaves the method, i.e. when the method reaches its end the variable `ReB` is automatically deleted²⁰².

Note the arguments passed in Line 7. The first is the name of the field. We could omit this argument, however, when we write the variable `ReB` to disk the first argument determines the file name. If this argument was omitted, then an automatically generated name – based on the way the field was generated – would be used. In this very case the file written would be named `max(((mag((U1-U2))*d)|nu),0.001)`. We easily recognize the formula of Line 7. A file name containing special characters (non-alphanumeric characters) is generally not advisable²⁰³.

In Line 14 we manually call the `write()` method. This method is available to all registered input/output objects²⁰⁴. As we construct the local variable `ReB` from the registered i/o object `Ur` we can safely assume that `ReB` will also be of this type.

```
1 Foam::tmp<Foam::volScalarField> Foam::YoonLuttrellAttachment::Ea
2 (
3     const volScalarField& Ur, const dimensionedScalar& dP
4 ) const
5 {
6     // do stuff
7     volScalarField ReB("ReB", max( Ur*dP/phase2_.nu(), scalar(1.0e-3) ));
8
9     // debug instructions
10    if (debug)
11    {
12        if (Ur.time().outputTime())
13        {
14            ReB.write();
15        }
16    }
17
18    // do more stuff
19 }
```

Listing 456: Manually writing intermediate fields for debugging.

57.12 A glance behind the run-time selection and debugging magic

OpenFOAM offers some amazing features. E.g. at compile-time of a fluid solver nobody knows which turbulence model will be used with the solver. In fact it can be none at all or any of the available. The same is true for drag models and the two-phase Eulerian solver with the exception that you can not use no drag law.

The entire wisdom behind the run-time selection mechanism, however, is more complex than what is presented in this section. Here, we focus on the macros we can find in the source files of the `SchillerNaumann` drag model class. We know, this drag model is derived from the base class `dragModel`. For the run-time selection mechanism to work, the base class also needs to do some preparations. See http://openfoamwiki.net/index.php/OpenFOAM_guide/runTimeSelection_mechanism for a discussion on the run-time selection mechanism. This section hopefully sheds some light into some of the inner workings of the run-time selection mechanism.

We shall now have a look behind the magic powers of OpenFOAM using the `SchillerNaumann` drag model as an example. The Listings 457 and 458 (Lines 10 and 3) show the two harmlessly looking lines of code enabling all the magic.

```
1 namespace Foam
```

²⁰²This behaviour is subsumed under the term *automatic variable*. See e.g. http://en.cppreference.com/w/cpp/language/storage_duration

²⁰³See e.g. http://www.teamdrive.com/Invalid_characters_in_file_and_folder_names.html

²⁰⁴See http://openfoamwiki.net/index.php/OpenFOAM_guide/Input_and_Output_operations_using_dictionaries_and_the_IObject_class and http://openfoamwiki.net/index.php/OpenFOAM_guide/objectRegistry

```

2 {
3     class SchillerNaumann
4     :
5     {
6         public dragModel
7     }
8     public:
9         //- Runtime type information
10        TypeName("SchillerNaumann");
11    }
12 }

```

Listing 457: The relevant lines of code in `SchillerNaumann.H`

```

1 namespace Foam
2 {
3     defineTypeNameAndDebug(SchillerNaumann, 0);
4 }

```

Listing 458: The relevant lines of code in `SchillerNaumann.C`

57.12.1 Part 1 - `TypeName`

First we will examine Line 10 of Listing 457.

```

    TypeName("SchillerNaumann");

```

What looks like a function call is actually a preprocessor macro²⁰⁵ with parameters²⁰⁶. The macro `TypeName` is defined in the file `typeInfo.H`. Listing 459 shows its definition.

A `\#define` macro consists of at least two parts. First comes the identifier, then comes the optional parameter list in parentheses and at least the replacement token list until the end of the line²⁰⁷. As the macro is expanded by the preprocessor, the identifier (in this case `TypeName`) is replaced with the replacement tokens (all instructions after the parameter list). A macro can not cover more than one line, however, by using the backslash (`\`) the current line is continued with the next line²⁰⁸.

```

1  //- Declare a ClassName() with extra virtual type info
2  #define TypeName(TypeNameString)                                \
3      ClassName(TypeNameString);                                  \
4      virtual const word& type() const { return typeName; }

```

Listing 459: The macro definition in `typeInfo.H`

Thus, the line `TypeName("SchillerNaumann");` expands to.

```

    ClassName("SchillerNaumann");
    virtual const word& type() const { return typeName; }

```

The second line is a function definition. As this function definition is made by the macro, this function is defined for every class where the `TypeName` macro is stated in the class definition. This demonstrates one of the major reasons for using preprocessor macros – the ability to write recurring pieces of code just once.

The first line of the above listing is itself a macro. Listing 460 shows the macro definitions that are necessary to expand the `ClassName` macro.

```

1  //- Add typeName information from argument \a TypeNameString to a class.
2  // Also declares debug information.
3  #define ClassName(TypeNameString)                                \
4      ClassNameNoDebug(TypeNameString);                            \
5      static int debug

```

²⁰⁵http://en.wikipedia.org/wiki/C_preprocessor

²⁰⁶See e.g. <http://www.cplusplus.com/doc/tutorial/preprocessor/>

²⁰⁷<https://gcc.gnu.org/onlinedocs/cpp/The-preprocessing-language.html>

²⁰⁸<https://gcc.gnu.org/onlinedocs/cpp/The-preprocessing-language.html#The-preprocessing-language>

```

6
7  //- Add typeName information from argument \a TypeNameString to a class.
8  // Without debug information
9  #define ClassNameNoDebug(TypeNameString) \
10     static const char* typeName_() { return TypeNameString; } \
11     static const ::Foam::word typeName

```

Listing 460: Two macro definitions in `className.H`

Thus, we further expand the `TypeName("SchillerNaumann")` macro.

```

static const char* typeName_() { return "SchillerNaumann"; } \
static const ::Foam::word typeName
static int debug
virtual const word& type() const { return typeName; }

```

As the `TypeName("SchillerNaumann")` macro was put into the class definition of the `verb+SchillerNaumann+` class, the macro added two function definitions (first and last line), one of which is a static method, and two static variables (the two center line).

Static elements of class (variables or methods) are elements that exist only once for all instances of a class²⁰⁹. In the case of a two-phase Eulerian solver two instances of the `SchillerNaumann` class might exist – in the case this model was specified for both phases. No matter which of the two instances of the class call the method `typeName()` it is always the same function called. In this case – returning the name of the class – the use of a static method makes perfect sense and is the only sensible way to implement this task.

The `TypeName("SchillerNaumann")` macro is used to create a method that returns the name of the class and a method that return the name of the type. Obviously, the class name and the type name were not considered equivalent when designing OpenFOAM²¹⁰. The variables created by the `TypeName("SchillerNaumann")` macro are a static variable containing the type name and a static variable named `debug`. This `debug` variable controls the debug mechanism covered in Section 57.11.

57.12.2 Part 2 - `defineTypeNameAndDebug`

Now we will examine Line 3 of Listing 458 which is repeated just below.

```
defineTypeNameAndDebug(SchillerNaumann, 0);
```

The `defineTypeNameAndDebug` macro is defined the file `className.H`.

```

1  //- Define the typeName and debug information
2  #define defineTypeNameAndDebug(Type, DebugSwitch) \
3      defineTypeName(Type); \
4      defineDebugSwitch(Type, DebugSwitch)

```

Listing 461: A macro definition in `className.H`

Thus our macro expands to two macros.

```

defineTypeName(SchillerNaumann);
defineDebugSwitch(SchillerNaumann, 0);

```

Listing 462 shows the macro definitions necessary to expand the above two macros.

```

1  //- Define the typeName, with alternative lookup as \a Name
2  #define defineTypeNameWithName(Type, Name) \
3      const ::Foam::word Type::typeName(Name)
4
5  //- Define the typeName
6  #define defineTypeName(Type) \
7      defineTypeNameWithName(Type, Type::typeName_())
8

```

²⁰⁹http://www.tutorialspoint.com/cplusplus/cpp_static_members.htm

²¹⁰See Section 57.12.3 for an example when class name and type name are different.

```

9  //- Define the debug information, lookup as \a Name
10 #define defineDebugSwitchWithName(Type, Name, DebugSwitch)          \
11     int Type::debug(::Foam::debug::debugSwitch(Name, DebugSwitch))
12
13 //- Define the debug information
14 #define defineDebugSwitch(Type, DebugSwitch)                        \
15     defineDebugSwitchWithName(Type, Type::typeName_(), DebugSwitch); \
16     registerDebugSwitchWithName(Type, Type, Type::typeName_())

```

Listing 462: Four macro definitions in debugName.H

Thus, our macros expand to:

```

const ::Foam::word SchillerNaumann::typeName(SchillerNaumann::typeName_());
int SchillerNaumann::debug(::Foam::debug::debugSwitch(SchillerNaumann, 0));
registerDebugSwitchWithName(SchillerNaumann, SchillerNaumann, SchillerNaumann::typeName_());

```

The first line of the expansion of the macro `defineTypeNameAndDebug(SchillerNaumann, 0)` assigns the return value of the function `typeName_()` to the static variable `typeName`. This has the effect that the class name and the type name have an equal value. However, the way this framework is set up allows for different names.

The second line assigns the return value of the function call `::Foam::debug::debugSwitch(SchillerNaumann, 0)` to the static variable `SchillerNaumann::debug`. The reason why the value is not directly used to assign the value to the static variable is that the called method adds the debug switch to a dictionary, see Listing 463.

The last line of the macro expansion invokes another macro. Listing 464 shows the macro definition of `registerDebugSwitchWithName`.

```

1  int Foam::debug::debugSwitch(const char* name, const int defaultValue)
2  {
3      return debugSwitches().lookupOrAddDefault
4      (
5          name, defaultValue, false, false
6      );
7  }

```

Listing 463: Adding the debug switch to the dictionary in debug.C

```

1  //- Define the debug information, lookup as \a Name
2  #define registerDebugSwitchWithName(Type, Tag, Name)                \
3      class add##Tag##ToDebug                                         \
4      :                                                               \
5      public ::Foam::simpleRegIOobject                               \
6      {                                                               \
7      public:                                                         \
8          add##Tag##ToDebug(const char* name)                       \
9          :                                                           \
10             ::Foam::simpleRegIOobject(Foam::debug::addDebugObject, name) \
11          {}                                                         \
12          virtual ~add##Tag##ToDebug()                               \
13          {}                                                         \
14          virtual void readData(Foam::Istream& is)                  \
15          {                                                           \
16              Type::debug = readLabel(is);                          \
17          }                                                         \
18          virtual void writeData(Foam::Ostream& os) const           \
19          {                                                           \
20              os << Type::debug;                                     \
21          }                                                         \
22      };                                                             \
23      add##Tag##ToDebug add##Tag##ToDebug_(Name)

```

Listing 464: Definition of the `registerDebugSwitchWithName` macro in debugName.H

57.12.3 A walk in the park: demonstrate some of this magic

In the above sections we took a look behind two very powerful pre-processor macros. So, what is this all for?

The turbulence models are very prominent examples for the usefulness of the run-time selection mechanism. At compile-time – the time we or the OpenFOAM developers compile a solver – nobody knows, what exact turbulence model we want to use for our simulation. Thus, we need to decide at run-time – at the time the solver reads all the case information – which turbulence model to use. In order to save us from writing a solver for each turbulence model, solvers can be written in a generic way. I.e. at the time we compile the solver nobody, not even the compiler, cares about the actual turbulence model. The base class `turbulenceModel` tells the compiler and the solver how a turbulence model works, that is all we need to know at compile time.

However, at run-time we need to decide which turbulence model to use. Fortunately, OpenFOAM takes care of that and we do not need to bother. In some cases, however, we would like to know which turbulence model is currently used. We could achieve this by either reading the case data²¹¹ or by making use of the run-time magic.

Listing 465 shows three lines of code. The intention behind this line is to print the return values of the methods `typeName_()` and `type()`. These two methods were provided by the two macros dissected in Sections 57.12.1 and 57.12.2.

```

1 Info << "Happy printf() debugging:" << endl;
2 Info << turbulence->typeName_() << endl;
3 Info << turbulence->type() << endl;
```

Listing 465: Applying some of the magic, the source code.

Listing 466 shows the results of the three lines of code of Listing 465. The code in Listing 465 presumes that turbulence modelling is used in its generic form, as it is the case in e.g. *pimpleFoam*. In this example the variable `turbulence` is of the type `autoPtr<incompressible::turbulenceModel> turbulence`.

From the output we see, that the variable `turbulence` is indeed of type `turbulenceModel`. However, as the class `turbulenceModel` is an abstract base class, no solver will ever actually use `turbulenceModel` itself²¹². In this case, the solver used the `kOmega` turbulence model. Thus, the method `type()` returns the name of the actual turbulence model. Here we also see the sense behind the distinction between the class name and the type name as discussed some paragraphs above. In the example of an concrete class those are the same. For a base class, however, this distinction makes perfect sense.

```

1 Happy printf() debugging:
2 turbulenceModel
3 kOmega
```

Listing 466: Applying some of the magic, the output.

57.13 Notes on running OpenFOAM in parallel

OpenFOAM is perfectly capable of running on multiple processors, however, even as OpenFOAM hides some or most of the technical issues arising from running in parallel from the programmer by its very intelligent design, there are still some things we need to consider and pitfalls we may stumble into.

First of all, let us take a moment and thank the universe, that OpenFOAM does so much on its own, in the background, when it comes to parallel running²¹³. If you restrict yourself to OpenFOAM's high-level data structures, as any sensible human being should, then you get *running in parallel* basically for free. You would not want to get into the details of how Lagrangian particles traverse sub-domains, or how to compute the gradient of a field at the very boundary of a sub-domain.

This section is devoted to topics which offer us mere mortals the chance to nevertheless mess up despite all the magic OpenFOAM brings to the table. Other topics, such as printing to Terminal²¹⁴, are presented out of curiosity, as in *how to do X*?

²¹¹This would mean re-programming existing functionality. The case data related to turbulence modelling was already read by the constructor of the turbulence model. Manually reading this information again would result in some kind of code duplication. The more elegant way to solve this problem is to access the information already gathered.

²¹²See Section 57.10 for information about how turbulence models are organized in OpenFOAM.

²¹³If you think otherwise, or are curious, see <http://mpitutorial.com/tutorials/> as a starting point for getting into the nitty-gritty of implementing parallel communication using MPI.

²¹⁴It would require a bit of (evil) creativity on the part of the user to mess up something as simple as printing to Terminal.

57.13.1 Printing to Terminal

When running an application in parallel, we might want for each one of the parallel processes being able to print to the Terminal, or we might want that only the master process can print to the Terminal. OpenFOAM allows you to do both.

Hush! The master is printing

If you want only the master to print to the Terminal, all is well. Just continue to use the `Info` statement we are all familiar with.

```
1 // this should be printed only once
2 Info << "Hello OpenFOAM World!" << endl;
```

Listing 467: Printing to the Terminal using the `Info` statement.

The output of Listing 467 is shown in Listing 469, which is the result of a parallel run with 4 processes²¹⁵.

Parallel print, the choir of processes

If we want all of the parallel processes to have their say, respectively print to Terminal, we can use the `Pout` statement. This statement allows each running process to print to the Terminal.

```
1 // this should be printed only once
2 Info << "Hello OpenFOAM World!" << endl;
3
4 // this should be printed once for each running process
5 Pout << " ... this is process " << pid() << " speaking!" << endl;
```

Listing 468: Printing to the Terminal using the `Pout` statement.

The `Pout` statement prepends the output message with the processor number, which starts counting from 0, and 0 being the master process. Note in Listing 469 that the order in which the individual processes print to the Terminal is not fixed.

```
1 Hello OpenFOAM World!
2 [1] ... this is process 2315 speaking!
3 [2] ... this is process 2316 speaking!
4 [0] ... this is process 2314 speaking!
5 [3] ... this is process 2317 speaking!
6 Finalising parallel run
```

Listing 469: Resulting output of Listing 468.

57.13.2 Condensing values (sums, extrema)

If we want to sum up all the values of a field, or if we want to determine the extrema (minimum and maximum), we most probably want to perform these operations globally, i.e. across all our sub-domains in a parallel run.

What needs to be done, is to perform the operation, e.g. summing up, within the sub-domain, communicating the result (e.g. all processes report to the master), and continuing the operation (the master determines the final result).

As OpenFOAM's developer designed the code base to be most friendly to the user, there are statements named `gSum`, `gMin`, `gMax` and others, which do exactly that.

```
1 // correct in serial, however, not so in parallel
2 Info<< "Sum of field U : " << sum(mag(X)*mesh.V()) << endl;
3
4 // correct summing-up
5 Info<< "Sum of field U : " << gSum(mag(X)*mesh.V()) << endl;
```

Listing 470: Computing and printing the total amount of field `X` to the Terminal.

²¹⁵... if the author is to be believed.

If we were to sum up `mesh.V()`, which is a scalar field made-up of the cell's volumes, we get the total volume of the domain, when we use `gSum(mesh.V())`. If we used `sum(mesh.V())`, then we would get roughly the total domain's volume divided by the number of processes. This is roughly an N -th of the domain's volume, as decomposition does not always yield sub-domains with a size exactly one N -th of the globale domain, e.g. when having an even number of cells and dividing into three parts.

57.14 Math-like syntax in OpenFOAM

OpenFOAM is designed in a way, for its top-level code to resemble mathematical notation, or to quote the original paper [66]:

The intention is to develop a C++ class library that makes it possible to implement complicated mathematical and physical models as high-level mathematical expressions. This is facilitated by making the high levels of the code resemble as closely as possible standard vector and tensor notation.

57.14.1 Pitfalls

Operator precedence: use caution with vector products

In mathematics certain operations are evaluated with priority over others²¹⁶. The same holds true for most programming languages. The general rules of operator precedence²¹⁷ in C++ closely follow the rules known from mathematics.

However, as OpenFOAM extends the rudimentary mathematical capabilities of standard C++ with concepts such as vectors and tensors, the question of operator precedence becomes interesting. Since, there are no vector products in C++ and completely new operators can not be defined, some existing operators have to be overloaded to implement the vector products. Furthermore, the operator precedence of existing operators can not be altered.

As it turns out, the bitwise AND (`&`) and bitwise XOR (`^`) have been overloaded to implement the inner and outer vector product. An unfortunate side-effect of using these operators is, that the inner and outer vector product have a lower precedence than the basic operators such as addition, subtraction and multiplication. This is because the bitwise AND and bitwise XOR operators have a quite low precedence.

To the unsuspecting user, it may come as a surprise when the following formula results in a compilation error:

$$\mathbf{a} \cdot \mathbf{b} - c$$

As mentioned above, the unexpected operator precedence leads to:

$$\mathbf{a} \cdot (\mathbf{b} - c)$$

Trying to compile the formula from above, translated into OpenFOAM source code:

```

1  const vector a(1.0, 2.0, 3.0);
2  const vector b(0.0, 42.0, 666.0);
3  const scalar c(0.815);
4
5  vector d( a & b - c);

```

Listing 471: This simple vector operation leads to a compiler error.

leads to a compiler error. Among the potentially plentiful lines of compiler warning and error messages, the line below is the relevant one. It informs us of our intent to subtract a scalar from a vector, which is undefined.

```

error: no match for 'operator-' (operand types are 'const vector {aka const Foam::Vector<
double>}' and 'const scalar {aka const double}')
```

Listing 472: The compiler error message

²¹⁶https://en.wikipedia.org/wiki/Order_of_operations

²¹⁷https://en.cppreference.com/w/cpp/language/operator_precedence

Thus, users are advised to use plenty of parentheses to avoid such problems. The issue with operator precedence is well explained by Hrv Jasak in a forum post²¹⁸:

That's because operator`&` and operator`^` are really binary C++ operators in C++ and the language does not allow you to define your own operators or change operator precedence.

We have chosen to re-use the operators for nice syntax - makes the code look nice. Therefore, all I can recommend is lots of brackets in vector/tensor products.

The same also applies to the double inner product, which takes two tensors of at least 2nd order. This operator is implemented by overloading the logical AND (`&&`) operator, which has an even lower precedence than the bitwise binary operators.

58 General remarks on OpenFOAM programming

This section covers some general advice for users who want to implement their own models and solvers or modify existing ones.

58.1 Preparatory tasks

58.1.1 Create user specific directories

In order to be able to distinguish between the standard solvers and models from the solvers and models created by the user, some new directories have to be created. Not only do we need to keep our models and applications apart from the models and applications of the standard OpenFOAM installation, we also need to keep our models and applications from those of other users. Thus, every user will get his or her own user directory. The name of this user directory follows OpenFOAM's naming convention which combines the user name and the version number of OpenFOAM, i.e. `user-4.0`. Thus, we can also keep our own stuff for OpenFOAM-X separate from the stuff for OpenFOAM-Y.

In this, we follow the organisational scheme of OpenFOAM's standard solvers and models, of which the source code resides in `$WM_PROJECT_DIR/applications/solvers` and `$WM_PROJECT_DIR/OpenFOAM-2.1.x/src`. Therefore, we need to create some folders to place our sources in: `$WM_PROJECT_USER_DIR/applications/solvers` and `$WM_PROJECT_USER_DIR/src`. Listing 473 lists the necessary commands. Open a Terminal and type the commands of the Listing to do the job.

```
cd $WM_PROJECT_USER_DIR
mkdir -p applications/solvers
mkdir src
```

Listing 473: Create the proper directories for a user's solvers and models

Note the use of the variable `$WM_PROJECT_USER_DIR`, which resolves to your OpenFOAM installation's user directory, which also contains the run-directory (`$FOAM_RUN`).

58.2 Start from existing code

58.2.1 Copy the sources

If you want to create a new model or solver, it is generally recommended to base it on the model or solver from OpenFOAM's standard installation, which comes closest to your intended set of features.

58.2.2 Change compilation settings

Before proceeding any further certain compilation settings have to be changed from the settings of OpenFOAM's standard code base.

²¹⁸<https://www.cfd-online.com/Forums/openfoam/61023-operator-precedence.html>

Change the executable's or library's name

The executable's or library's name is determined by a setting in the file `Make/files` of the source code of the respective solver or model. For a solver the name of the executable is determined by the setting `EXE`. Listing 475 shows how the name of the executable for the solver `pimpleFoam` is defined.

```
EXE = $(FOAM_APPBIN)/pimpleFoam
```

Listing 474: Setting the name of `pimpleFoam`'s executable in the file `Make/files` of `pimpleFoam`'s source code²¹⁹

Analogously, defining the name of the shared library²²⁰ is done by the setting `LIB` in the file `Make/files` of the library's source code.

```
LIB = $(FOAM_LIBBIN)/liblagrangian
```

Listing 475: Setting the name of the shared object of the Lagrangian particle library in the file `Make/files` of Lagrangian library source code

The settings for `EXE` and `LIB` are full file paths. Thus, next to the assignment (the `=` symbol) we find a directory, the path name separator (the `/` symbol) and the actual name of the executable or shared library.

Change the executable's or library's location

Avoiding mixing up user created solvers and models from the ones provided by OpenFOAM's standard installation involves apart from changing the file name of the executable or shared object also the path the executable or shared object resides in. This is specified, as already shown above, in the file `Make/files`.

For user created solvers and models, users are advised to change the path specifier to `FOAM_USER_APPBIN` or `FOAM_USER_LIBBIN` respectively. Thus, user generated solvers and libraries are also spatially separated²²¹ from standard solvers and libraries.

If you use a system-wide OpenFOAM installation, then you most probably have only read access to the `FOAM_APPBIN` and `FOAM_LIBBIN` directories. Thus, trying to compile your solver or model will fail, even when there are no errors. In this case, the resulting error message might contain some hint about missing permissions.

Check Make/files

After adjusting the compilation settings, check and re-check the file `Make/files`. Listing 476 shows the vital entries of the file highlighted. In the listing, the source file has the same name as the executable. Furthermore, the executable will be located in the user's application directory.

```
myApplication.C
```

```
EXE = $(FOAM_USER_APPBIN)/myApplication
```

Listing 476: The content of `Make/files`

The file `Make/files` controls what is compiled and where the resulting executable will be stored. Thus, getting `Make/files` right will save yourself from breaking something else, i.e. the model or application you base your new model or application on. On the other hand, the file `Make/options` controls what is needed to successfully compile the model or application. Getting `Make/options` wrong initially will do no harm.

²¹⁹The executeable does not necessarily have to have the same name as the source file. However, different names can lead to confusion and make code maintenance harder. Therefore, it is strongly recommended to use consistent names, i.e. to name the source file `SOLVER.C` and the executable `SOLVER`.

²²⁰The main purpose of dividing code into applications and libraries is to allow for multiple unconnected applications using certain implemented behaviour (the libraries). Thus, libraries are shared by an arbitrary number of applications. Hence, libraries are compiled into files which are referred as *shared objects* or *shared libraries*. The names of these files are appended by the filename extension `so`, i.e. `libraryName.so`.

²²¹by residing in different directories

Check Make/options

The file `Make/options` tells the compiler where to find additional source files, and it tells the linker²²² where to find additional libraries. The file `Make/options` only needs to be edited, if you use existing models, or source files from other directories. This, however, is often the case.

The content of the file `Make/options` is divided in two parts. First, there are the paths for compiler to look for source files to include. The second part is a list of libraries for the linker to link the current model or application with.

```
EXE_INC = \  
-I$(LIB_SRC)/meshTools/lnInclude \  
-I$(LIB_SRC)/finiteVolume/lnInclude  
  
LIB_LIBS = \  
-lmeshTools
```

Listing 477: Content of `Make/options`

Initial compilation

Once the existing sources have been copied and, most importantly, the compilation settings have been changed, we can run an initial compilation. Although, at this point, nothing in the source code has changed, running an initial compilation is recommended to check whether we have got the compilation settings right.

For applications simply execute `wmake`, for shared libraries the compiler gets an additional parameter: `wmake libso`. After the compilation the compiled binary should show up in `FOAM_USER_APPBIN` or `FOAM_USER_LIBBIN` respectively.

58.3 Create the source code from scratch

The steps discussed above may not be needed by certain users. OpenFOAM provides some macros to create the basic source-code-skeleton for, among others, new applications, boundary conditions or functions objects. In the case of creating a new application from scratch, the user simply calls the macro `foamNewApp` and provides the desired application name. The executable's path will automatically be set to `FOAM_USER_APPBIN`. New function objects or boundary conditions will automatically be compiled into `FOAM_USER_LIBBIN`.

58.4 Using a user-created libraries

Distinguishing between solvers and libraries is a good thing, since we can create and reuse certain models. If we want our application to use a model of ourself, we need to tell the compiler and the linker where to find our (already compiled) model.

In Listing 478 we see the necessary entries for the file `Make/options` for an application which is to use a user-created library with the very creative name *myLibrary*. The green line in Line 4 of the listing tells the compiler where to find the source code of the library *myLibrary*.

After the compilation stage finished successfully, the compiled application needs to be linked to the compiled library, i.e. shared object. This is shown in the Lines 8 and 9 of the Listing. The red line defines an additional directory in which to look for shared objects. As good style dictates us to compile our own libraries into `FOAM_USER_LIBBIN`, the additional directory for the linker is `FOAM_USER_LIBBIN`.

The blue line, Line 9 of the Listing, then tells the linker the name of the shared object of *myLibrary*.

```
EXE_INC = \  
-I$(LIB_SRC)/meshTools/lnInclude \  
-I$(LIB_SRC)/finiteVolume/lnInclude \  
-I$(MY_LIB_SRC_PATH)/myLibrary/lnInclude  
  
LIB_LIBS = \  
-lmeshTools \  
-L$(FOAM_USER_LIBBIN) \  
-lmyLibrary
```

²²²Compilation of C or C++ code is usually done in two steps. First all files are compiled and then the object files generated by the compiler are linked together to form the executable.

58.5 Pitfalls

Modifying existing models or creating new ones bears the potential for many bugs and errors. This section tries to discuss some of them, which either occur regularly or may be difficult to track down. Such a list can never be complete and it is clearly biased towards the errors made and encountered by the author. These errors are not necessarily restricted to OpenFOAM, moreover, modifying OpenFOAM involves programming and programming involves bugs and errors. Enjoy reading.

58.5.1 Segfault due to modified library and failing to update the solver

Libraries are reusable parts of code, which are independent of the solver(s) using them. However, this independence may create problems if we modify the library and fail to recompile the solver(s) that are using said library, even if said solver(s) have not been touched. The problem that might occur is a segmentation fault (*segfault* in short) at construction of the modified objects. Unfortunately, a segfault does not produce very telling error messages, see Listing 479. Note that we assume that compilation of the library finished successfully.

```
#0 Foam::error::printStack(Foam::Ostream&) at ???  
#1 Foam::sigSegv::sigHandler(int) at ???  
#2 ? in "/lib/x86_64-linux-gnu/libc.so.6"  
#3 ? in "/lib/x86_64-linux-gnu/libc.so.6" Segmentation fault (Core dumped)
```

Listing 479: A segmentation fault

The reason for this behaviour, the solver violently fails at start-up due to a segmentation fault, is that our modifications to the library changed its memory layout. As we did not recompile the solver, the solver had no means to learn about the changed memory layout. Thus, at solver start-up, the solver reserves an incorrect amount of memory for the objects of the library. At construction of these objects, this mismatch causes the segmentation fault.

Recompiling the solver, which has not been touched seems at first counter-intuitive. However, when the memory layout of the library changed, the solver will reserve memory according to the old memory layout. Recompiling the solver will simply update memory allocation.

The tricky bit of this error, is that it does not always occur. If we change a library and our changes do not alter the memory layout, all is well. No error will occur. This opens the possibility of *modifying the library and running the solver using that library* to sometimes work and sometimes cause this error.

Further reading

<https://stackoverflow.com/questions/2346806/what-is-a-segmentation-fault>

https://en.wikipedia.org/wiki/Segmentation_fault

58.5.2 (Failing to) Tell the solver to use a library

Modifying an existing library, or creating your own from scratch, results in a new shared object file in the `FOAM_USER_LIBBIN` directory. However, if you want your solver to make use of this library, you need to tell the solver to do so. *Models in FOAM_USER_LIBBIN are not loaded by an application by default!*

This is done via the `libs` list entry in `controlDict`, see Section 11.3.3. Failing to do so results in a very odd situation. The model compiles without error; the solver, if you are using your own, also compiles without error²²³; the case setup seems without error²²⁴; and yet the case does not run, when you try to make use of your own model. In this situation, the seemingly correct case setup yields the same result as the *banana test* (see Section 11.2.1), since OpenFOAM is unable to find the correct model associated with the name we provided in the case files.

²²³If the library compilation had failed, the linker would throw an error, if you tried to compile a solver using that library.

²²⁴i.e. the case runs, when you use only standard OpenFOAM features.

58.6 Tips

Check and double-check that you compile to the user-directories

Always make sure, that you compile your applications to `FOAM_USER_APPBIN` and your libraries to `FOAM_USER_LIBBIN`. The reason for this, and the way how to ensure this, are discussed in [58.2.2](#).

When modifying a library, also recompile the solver(s) using the library

This means a seemingly extraneous step²²⁵, however, it might prevent weird behaviour. One reason for recompiling the solver using a recently modified library was discussed above in [Section 58.5.1](#).

Furthermore, especially when templates are involved, e.g. you modify a templated class, it is vital to compile the library which provided the templated model and the solver that uses the templated model. Templates need special attention, care and love, do not neglect them.

Use `Allwclean` or `wclean` liberally

Clean-up your temporary build-files for good measure by running `wclean`. Especially when templated classes are involved, clearing build-files might be essential.

Also, if you update the underlying OpenFOAM version, recompiling your custom libraries and solvers will also require cleaning of the build-files. If the underlying OpenFOAM installation, then the dependency list, i.e. the `*.dep` file present in the `Make` directory of your code, may contain the file-names of files which might have ceased to exist.

²²⁵In some cases it is sufficient only to compile the library, e.g. when fixing a type in an `Info` message.

Part X

Theory

This section covers more detailed topics and tries to look *under the hood* of OpenFOAM from a non-programming view.

59 Discretization

59.1 Temporal discretization

59.2 Spatial discretization

The purpose of spatial discretization schemes is to compute the face values of fields whose values are stored at the cell centre. The face values are then used e.g. for computing the spatial derivatives.

59.2.1 upwind scheme

An upwind scheme determines the face value of a quantity simply by choosing the cell centered value of the cell that is located upwind of the face in question.

59.2.2 linearUpwind scheme

The `linearUpwind` scheme is equivalent to FLUENTs *Second-Order Upwind Scheme*.

59.2.3 QUICK scheme

The FLUENT Theory Guide [7] states:

For quadrilateral and hexahedral meshes, where unique upstream and downstream faces and cells can be identified, ANSYS FLUENT also provides the QUICK scheme for computing a higher-order value of the convected variable at a face.

59.2.4 MUSCL scheme

59.3 Continuity error correction

In the governing equations of some solvers in OpenFOAM – e.g. in *twoPhaseEulerFoam* of OpenFOAM-2.3.x – we find a special correction for the continuity error.

59.3.1 Conserving the form

Before we start our considerations, we take a closer look on the *conservation* and *nonconservation* form of a transport equation. First, we recall the definition of the substantial derivative:

$$\frac{D}{Dt} = \frac{\partial}{\partial t} + (\mathbf{u} \cdot \nabla) \quad (198)$$

For example applied to an arbitrary scalar K

$$\frac{DK}{Dt} = \frac{\partial K}{\partial t} + \mathbf{u} \cdot \nabla K \quad (199)$$

Continuity equation

As a first example we look up the differential form of the continuity equation.

$$\text{conservation form:} \quad \frac{\partial \rho}{\partial t} + \nabla \cdot (\rho \mathbf{u}) = 0 \quad (200)$$

$$\text{nonconservation form:} \quad \frac{D\rho}{Dt} + \rho \nabla \cdot \mathbf{u} = 0 \quad (201)$$

Both forms are equivalent to each other, since we can express one equation easily by the other one with the help of some simple mathematical operations.

$$\frac{\partial \rho}{\partial t} + \nabla \cdot (\rho \mathbf{u}) = 0 \quad (202)$$

$$\frac{\partial \rho}{\partial t} + \nabla \rho \cdot \mathbf{u} + \rho \nabla \cdot \mathbf{u} = 0 \quad (203)$$

$$\underbrace{\frac{\partial \rho}{\partial t} + \mathbf{u} \cdot \nabla \rho}_{\frac{D\rho}{Dt}} + \rho \nabla \cdot \mathbf{u} = 0 \quad (204)$$

Transport equation

For the next example we use the right hand side of the transport equation of enthalpy in a multiphase problem. This example is motivated by the energy equation of *twoPhaseEulerFoam* in OpenFOAM-2.3.x. We could also have used the momentum equation, however, we want to avoid confusion by the repeated occurrence of the velocity.

We look up the energy equation for multiphase flows from a textbook or other resources [7, 6]. For the sake of brevity, we state only the left hand side of the equation. The equation we looked up (Eqn. (205)) happens to be formulated in the *conservation* form. We now rearrange the equation in order to gain the *nonconservation* form.

$$\frac{\partial \alpha_k \rho_k h_k}{\partial t} + \nabla \cdot (\alpha_k \rho_k \mathbf{u}_k h_k) = RHS \quad (205)$$

by partial derivation of the LHS, we gain

$$\frac{\partial \alpha_k \rho_k}{\partial t} h_k + \alpha_k \rho_k \frac{\partial h_k}{\partial t} + h_k \nabla \cdot (\alpha_k \rho_k \mathbf{u}_k) + \alpha_k \rho_k \mathbf{u}_k \cdot \nabla h_k = RHS \quad (206)$$

$$\underbrace{h_k \left(\frac{\partial \alpha_k \rho_k}{\partial t} + \nabla \cdot (\alpha_k \rho_k \mathbf{u}_k) \right)}_I + \underbrace{\alpha_k \rho_k \left(\frac{\partial h_k}{\partial t} + \mathbf{u}_k \cdot \nabla h_k \right)}_{II} = RHS \quad (207)$$

We now pay attention to the term marked by I, we recognize the phase-continuity equation which equals zero. The term marked with II is the substantial derivative of h_k . Thus we gain with Eqn. (208), the *nonconservation* form of the energy equation.

$$\alpha_k \rho_k \left(\frac{\partial h_k}{\partial t} + \mathbf{u}_k \cdot \nabla h_k \right) = RHS \quad (208)$$

$$\alpha_k \rho_k \frac{Dh_k}{Dt} = RHS \quad (209)$$

All the operations we applied to get from Eqn. (205) to (208) applied only to the left hand side. Thus, the distinction in *conservation* and *nonconservation* form applies only to the left hand side of the equation.

59.3.2 Continuity error

In theory and in the mathematical sense the *conservation* and *nonconservation* forms are equivalent. However, in we do not solve the s we gain from physics, but the linear equation system stemming from discretizing those

PDEs. The resulting linear equation system we solve is not necessarily a direct representation of our initial PDEs. The difference between the (exact) solution of the system of algebraic equations and the (unknown) solution of the mathematical model (the PDEs) is generally referred to as *discretisation error* [36].

We now use Eqns. (205) and (208) to to some rearrangement.

$$\frac{\partial \alpha_k \rho_k h_k}{\partial t} + \nabla \cdot (\alpha_k \rho_k \mathbf{u}_k h_k) = \alpha_k \rho_k \left(\frac{\partial h_k}{\partial t} + \mathbf{u}_k \cdot \nabla h_k \right) + h_k \left(\frac{\partial \alpha_k \rho_k}{\partial t} + \nabla \cdot (\alpha_k \rho_k \mathbf{u}_k) \right) \quad (210)$$

$$\frac{\partial \alpha_k \rho_k h_k}{\partial t} + \nabla \cdot (\alpha_k \rho_k \mathbf{u}_k h_k) - h_k \left(\frac{\partial \alpha_k \rho_k}{\partial t} + \nabla \cdot (\alpha_k \rho_k \mathbf{u}_k) \right) = \alpha_k \rho_k \left(\frac{\partial h_k}{\partial t} + \mathbf{u}_k \cdot \nabla h_k \right) \quad (211)$$

We now want to solve the energy equation. For this we choose the *nonconservative* form (208).

$$\alpha_k \rho_k \left(\frac{\partial h_k}{\partial t} + \mathbf{u}_k \cdot \nabla h_k \right) = RHS \quad (208)$$

Using Eq. (211), we could also write

$$\frac{\partial \alpha_k \rho_k h_k}{\partial t} + \nabla \cdot (\alpha_k \rho_k \mathbf{u}_k h_k) - h_k \left(\frac{\partial \alpha_k \rho_k}{\partial t} + \nabla \cdot (\alpha_k \rho_k \mathbf{u}_k) \right) = RHS \quad (212)$$

Mathematically, Eqns. (208) and (212) are equivalent. However, when we now discretize both equations in order to solve them numerically, the left hand sides of Eqns. (208) and (212) might actually be different, as the discretised phase continuity equation might not equal zero.

We now take a break from math and take a look into the source code of *twoPhaseEulerFoam-2.3.x*. In Listing 481 we see the first terms of the energy equation of one phase. For a discussion on the full energy equations see Section 45.5.

In Lines 3 and 4 of Listing 481 we see the left hand side of Eqn. (212).

```

1  fvScalarMatrix he1Eqn
2  (
3      fvm::ddt(alpha1, rho1, he1) + fvm::div(alphaRhoPhi1, he1)
4      - fvm::Sp(contErr1, he1)
5      /* other stuff */
6  );

```

Listing 480: The first terms of the energy equation in the file `EEqns.H` of *twoPhaseEulerFoam*.

```

1  volScalarField contErr1
2  (
3      fvc::ddt(alpha1, rho1) + fvc::div(alphaRhoPhi1)
4      - (fvOptions(alpha1, rho1)&rho1)
5  );

```

Listing 481: The definition of the continuity error in the file `twoPhaseEulerFoam.C`.

We can create more resemblance if we repeat Eqn. (212) and name some of the terms. In Listing 481 the definition of the continuity error differs slightly from Eqn. (212). This is due to the fact, that the solver considers phase sources, see Line 4 of Listing 481.

$$\underbrace{\frac{\partial \alpha_k \rho_k h_k}{\partial t}}_{\text{fvm::ddt(alpha1, rho1, he1)}} + \underbrace{\nabla \cdot (\alpha_k \rho_k \mathbf{u}_k h_k)}_{\text{fvm::div(alphaRhoPhi1, he1)}} - h_k \underbrace{\left(\frac{\partial \alpha_k \rho_k}{\partial t} + \nabla \cdot (\alpha_k \rho_k \mathbf{u}_k) \right)}_{\text{contErr1}} = RHS \quad (212)$$

60 Momentum diffusion in an incompressible fluid

60.1 Governing equations

In Section 42.1 we discussed the governing equations of a solver for incompressible fluids.

$$\frac{\partial \mathbf{u}}{\partial t} + \nabla(\mathbf{u}\mathbf{u}) + \underbrace{\nabla \cdot (\text{dev}(\mathbf{R}^{eff}))}_{=\text{div}(\text{dev}(\mathbf{R}^{eff}))} = -\nabla p + \mathbf{Q} \quad (78)$$

$$\mathbf{R}^{eff} = -\nu^{eff} (\nabla \mathbf{u} + (\nabla \mathbf{u})^T) \quad (72)$$

$$\frac{\partial \mathbf{u}}{\partial t} + \nabla(\mathbf{u}\mathbf{u}) + \nabla \cdot (\text{dev}(-\nu^{eff} (\nabla \mathbf{u} + (\nabla \mathbf{u})^T))) = -\nabla p + \mathbf{Q} \quad (79)$$

The momentum diffusion term is handled by the turbulence model.

$$\underbrace{\nabla \cdot (\text{dev}(\mathbf{R}^{eff}))}_{=\text{div}(\text{dev}(\mathbf{R}^{eff}))} \Leftrightarrow \text{turbulence} \rightarrow \text{divDevReff}(\mathbf{U})$$

60.2 Implementation

All turbulence model of OpenFOAM are based on a generic turbulence model class. Figure 134 in Section 57.10 shows a class diagram. There, it is shown, that all RAS turbulence model classes as well as all LES turbulence model classes are derived from the same base class. A lot of solvers of OpenFOAM allow the user to choose between laminar simulation as well as RAS or LES turbulence modelling. Therefore, by the time of writing the source code, nobody could have known, which turbulence exactly will handle the momentum diffusion term.

To overcome such problems, modern programming languages support a technique called polymorphism. In the source code the instruction `turbulence->divDevReff(U)` is called to compute the diffusive term. This instruction means, that the method `divDevReff()` of the object `turbulence` is called.

```

1 // Solve the Momentum equation
2
3 tmp<fvVectorMatrix> UEqn
4 (
5     fvm::ddt(U)
6     + fvm::div(phi, U)
7     + turbulence->divDevReff(U)
8 );
9
10 UEqn().relax();
11
12 sources.constrain(UEqn());
13
14 volScalarField rAU(1.0/UEqn().A());
15
16 if (pimple.momentumPredictor())
17 {
18     solve(UEqn() == -fvc::grad(p) + sources(U));
19 }

```

Listing 482: The file *UEqn.H* of *pimpleFoam*

The source code of the file `createFields.H` tells us, that the object `turbulence` is of the data type `turbulenceModel`.

```

1 singlePhaseTransportModel laminarTransport(U, phi);
2
3 autoPtr<incompressible::turbulenceModel> turbulence
4 (
5     incompressible::turbulenceModel::New(U, phi, laminarTransport)
6 );

```

Listing 483: The file *createFields.H* of *pimpleFoam*

By the time of compilation, it is guaranteed that the object `turbulence` is of the data type `turbulenceModel`. However, `turbulence` will never actually be of the data type `turbulenceModel`. It will be of a data type derived from `turbulenceModel`. The decision which exact method `divDevReff()` has to be called, will be made at run-time based on the actual type of `turbulence`.

Listing 484 shows the declaration of the virtual method `divDevReff()`. See Section 57.10 for a discussion on virtual methods. Listing 485 shows how this method is actually implemented by the standard k- ϵ turbulence models of OpenFOAM.

```
// - Return the source term for the momentum equation
virtual tmp<fvVectorMatrix> divDevReff(volVectorField& U) const = 0;
```

Listing 484: Declaration of the virtual Method *divDevReff* in *turbulenceModel.H*

```
tmp<fvVectorMatrix> kEpsilon::divDevReff(volVectorField& U) const
{
    return
    (
        - fvm::laplacian(nuEff(), U)
        - fvc::div(nuEff()*dev(T(fvc::grad(U))))
    );
}
```

Listing 485: Implementation of the virtual Method *divDevReff* in *kEpsilon.H*

The calculation of `divDevReff()` is equivalent to Eq. (79).

$$\begin{aligned} \text{divDevReff} &= \nabla \cdot (\text{dev}(-\nu (\nabla \mathbf{U} + (\nabla \mathbf{U})^T))) \\ &= \underbrace{-\nabla \cdot (\nu (\nabla \mathbf{U}))}_{\text{laplacian}(\nu, \mathbf{U})} - \underbrace{\nabla \cdot (\nu (\nabla \mathbf{U})^T)}_{\text{div}(\nu * \text{dev}(\mathbf{T}(\text{grad}(\mathbf{U})))} \end{aligned}$$

The momentum diffusion term is most probably split into two parts for numerical reasons.

61 The incompressible k- ϵ turbulence model

61.1 The k- ϵ turbulence model in literature

The governing equations for the k- ϵ model for a single phase are taken from Wilcox [67].

Eddy viscosity

$$\mu_T = \rho C_\mu \frac{k^2}{\epsilon} \quad (213)$$

Turbulent kinetic energy

$$\rho \frac{\partial k}{\partial t} + \rho U_j \frac{\partial k}{\partial x_j} = \tau_{ij} \frac{\partial U_i}{\partial x_j} - \rho \epsilon + \frac{\partial}{\partial x_j} \left[\left(\mu + \frac{\mu_T}{\sigma_k} \right) \frac{\partial k}{\partial x_j} \right] \quad (214)$$

Dissipation Rate

$$\rho \frac{\partial \epsilon}{\partial t} + \rho U_j \frac{\partial \epsilon}{\partial x_j} = C_{\epsilon 1} \frac{\epsilon}{k} \tau_{ij} \frac{\partial U_i}{\partial x_j} - C_{\epsilon 2} \rho \frac{\epsilon^2}{k} + \frac{\partial}{\partial x_j} \left[\left(\mu + \frac{\mu_T}{\sigma_\epsilon} \right) \frac{\partial \epsilon}{\partial x_j} \right] \quad (215)$$

Closure coefficients

$$C_{\epsilon 1} = 1.44, \quad C_{\epsilon 2} = 1.92, \quad C_\mu = 0.09, \quad \sigma_k = 1.0, \quad \sigma_\epsilon = 1.3 \quad (216)$$

The transport equations for k and ϵ are reorganized to follow the basic structure

$$\text{local derivative} + \text{convection} + \text{diffusion} = \text{source \& sink terms}$$

Turbulent kinetic energy

$$\rho \frac{\partial k}{\partial t} + \rho U_j \frac{\partial k}{\partial x_j} - \frac{\partial}{\partial x_j} \left[\underbrace{\left(\mu + \frac{\mu_T}{\sigma_k} \right) \frac{\partial k}{\partial x_j}}_{D_k} \right] = \underbrace{\tau_{ij} \frac{\partial U_i}{\partial x_j}}_G - \rho \epsilon \quad (217)$$

Dissipation Rate

$$\rho \frac{\partial \epsilon}{\partial t} + \rho U_j \frac{\partial \epsilon}{\partial x_j} - \frac{\partial}{\partial x_j} \left[\underbrace{\left(\mu + \frac{\mu_T}{\sigma_\epsilon} \right) \frac{\partial \epsilon}{\partial x_j}}_{D_\epsilon} \right] = C_{\epsilon 1} \frac{\epsilon}{k} \underbrace{\tau_{ij} \frac{\partial U_i}{\partial x_j}}_G - C_{\epsilon 2} \rho \frac{\epsilon^2}{k} \quad (218)$$

Diffusivity constants

$$D_k = \mu + \frac{\mu_T}{\sigma_k} \quad (219)$$

$$D_\epsilon = \mu + \frac{\mu_T}{\sigma_\epsilon} \quad (220)$$

The constant expressions in the diffusive terms are combined into the diffusivity constants D_k and D_ϵ . The first term on the right hand side of the turbulent kinetic energy equation is the production of turbulent kinetic energy G .

61.2 The k- ϵ turbulence model in OpenFOAM

61.2.1 Governing equations

The governing equations of the k- ϵ model of OpenFOAM are basically the same equations as in Section 61.1. The vector notation is used in this section because the syntax OpenFOAM uses strongly resembles the vector notation. However, there are some modifications to the equations.

First, the transport equations for k and ϵ are divided by the density ρ . Therefore, all terms containing viscosity contain the kinematic viscosity ν instead of the dynamic viscosity μ .

Secondly, the standard k- ϵ model of OpenFOAM has eliminated the model constant σ_k . Since the value of this constant is one, this constant has been eliminated. This does not change the behaviour of the model. However, if the user tries to change this model constant, nothing actually happens. See Section 32.4.2 for a discussion and an example.

Finally, the convection term is converted into two term by the product rule of differentiation. See Eqn. (222).

Eddy viscosity, see Listing 486

$$\begin{aligned} \mu_T &= \rho \nu_T \\ \nu_T &= C_\mu \frac{k^2}{\epsilon} \end{aligned} \quad (221)$$

Turbulent kinetic energy, see Listing 487

$$\begin{aligned} U_j \frac{\partial k}{\partial x_j} &= \mathbf{U} \cdot \frac{\partial k}{\partial \mathbf{x}} = \mathbf{U} \cdot \nabla k \\ \mathbf{U} \cdot \frac{\partial k}{\partial \mathbf{x}} &= \nabla \cdot (\mathbf{U}k) - (\nabla \cdot \mathbf{U})k \end{aligned} \quad (222)$$

$$\frac{\partial k}{\partial t} + \nabla \cdot (\mathbf{U}k) - (\nabla \cdot \mathbf{U})k - \nabla \cdot (D_k \nabla k) = G - \epsilon \quad (223)$$

Dissipation Rate

$$\frac{\partial \epsilon}{\partial t} + \nabla \cdot (\mathbf{U}\epsilon) - (\nabla \cdot \mathbf{U})\epsilon - \nabla \cdot (D_\epsilon \nabla \epsilon) = C_1 G \frac{1}{k} - C_2 \frac{\epsilon^2}{k} \quad (224)$$

Diffusivity constants - Note that σ_k has been eliminated from the equations

$$D_k = \text{DkEff} = \nu + \nu_T \quad (225)$$

$$D_\epsilon = \text{DepsilonEff} = \nu + \frac{\nu_T}{\sigma_\epsilon} \quad (226)$$

Closure coefficients - default values

$$C_1 = 1.44, \quad C_2 = 1.92, \quad C_\mu = 0.09, \quad \sigma_\epsilon = 1.3 \quad (227)$$

The default values of the model constants can be found in the constructor of the respective turbulence model class.

61.2.2 The source code

Listing 486 shows the calculation of the eddy viscosity. A (too) short glimpse on the code may lead to confusion, as the function `sqr()` meaning taking a variable to the power of two looks similar to `sqrt()`, which is the square root.

Listing 487 shows the transport equation for the turbulent viscosity. The last term on the right hand side is expanded.

$$\epsilon = \underbrace{\frac{\epsilon}{k}}_{\text{fvm::Sp}(\text{epsilon}/k, k)} k \quad (228)$$

```
nut_ = Cmu_*sqr(k_)/epsilon_;
```

Listing 486: Calculation of the eddy viscosity

```
tmp<fvScalarMatrix> kEqn
(
    fvm::ddt(k_)
  + fvm::div(phi_, k_)
  - fvm::Sp(fvc::div(phi_), k_)
  - fvm::laplacian(DkEff(), k_)
==
    G
  - fvm::Sp(epsilon_/k_, k_)
);
```

Listing 487: Transport equation for the turbulent kinetic energy

Constructor

Listing 488 shows the first lines of the constructor of the `kEpsilon` class. The constructor receives five arguments. After the colon (in line 9), the initialisation list follows. This list contains also the default values of the model constants. See Section 56.5 for details about constructors in C++. In line 18 the default value of the model constant C_μ is defined.

```
1 kEpsilon::kEpsilon
2 (
3     const volVectorField& U,
4     const surfaceScalarField& phi,
5     transportModel& transport,
6     const word& turbulenceModelName,
7     const word& modelName
8 )
9 :
10    RASModel(modelName, U, phi, transport, turbulenceModelName),
11
12    Cmu_
13    (
14        dimensioned<scalar>::lookupOrAddToDict
```

```

15      (
16      "Cmu",
17      coeffDict_,
18      0.09
19      )
20  ),
21  /* code continues */

```

Listing 488: The constructor of the `kEpsilon` class

61.3 The k- ϵ turbulence model in *bubbleFoam* and *twoPhaseEulerFoam*

The k- ϵ turbulence model is hardcoded in *bubbleFoam* and *twoPhaseEulerFoam*. This means, that these solvers do not use the generic turbulence modelling other than most OpenFOAM solvers.

The question of turbulence modelling in dispersed two-phase flows is not fully answered yet. There are several strategies:

Per phase The turbulence is modelled for both phases individually.

Mixture The turbulence is modelled based on mixture quantities.

Liquid phase Turbulence is modelled based in the quantites of the liquid phase. The turbulence of the dispersed phase is either neglected or considered by a model constant.

61.3.1 Governing equations

The k- ϵ turbulence model of *bubbleFoam* and *twoPhaseEulerFoam* is in some aspects different than the standard k- ϵ turbulence model of OpenFOAM.

1. The diffusivity constants are calculated from the effective viscosity. Compare Eqns. (219, 220) and (234, 235)
2. The model constants σ_k and σ_ϵ are replaced by their reciprocal values.
3. Other than in the standard k- ϵ model, the model constant σ_k is not dropped. By defining a value for the constant $\alpha_{1,k} = 1/\sigma_k$, a value for σ_k is assigned.

Turbulence modelling in *bubbleFoam* and *twoPhaseEulerFoam* is based on the liquid quantities. Turbulence of the gas phase is considered by the use of the model constant C_t . This constant connects the turbulent viscosity of the liquid and the gas phase. By setting this constant to zero, turbulence is ignored in the gas phase.

Eddy viscosity

$$\nu_{2,T} = C_\mu \frac{k^2}{\epsilon} \quad (229)$$

$$\nu_{2,eff} = \nu_2 + \nu_{2,T} \quad (230)$$

$$\nu_{1,eff} = \nu_1 + C_t^2 \nu_{2,T} \quad (231)$$

Turbulent kinetic energy, see Listing 487

$$\frac{\partial k}{\partial t} + \nabla \cdot (\mathbf{U}_2 k) - (\nabla \cdot \mathbf{U}_2) k - \nabla \cdot (\alpha_{1,k} \nu_{2,eff} \nabla k) = G - \epsilon \quad (232)$$

Dissipation Rate

$$\frac{\partial \epsilon}{\partial t} + \nabla \cdot (\mathbf{U}_2 \epsilon) - (\nabla \cdot \mathbf{U}_2) \epsilon - \nabla \cdot (\alpha_{1,\epsilon} \nu_{2,eff} \nabla \epsilon) = C_1 G \frac{1}{k} - C_2 \frac{\epsilon^2}{k} \quad (233)$$

Diffusivity constants - Note the different definition

$$\alpha_{1,k} = \frac{1}{\sigma_k}$$

$$\alpha_{1,\epsilon} = \frac{1}{\sigma_\epsilon}$$

$$D_k = \alpha_{1,k} \nu_{2,eff} = \frac{\nu_{2,eff}}{\sigma_k} \quad (234)$$

$$D_\epsilon = \alpha_{1,\epsilon} \nu_{2,eff} = \frac{\nu_{2,eff}}{\sigma_\epsilon} \quad (235)$$

Closure coefficients - default values

$$C_1 = 1.44, \quad C_2 = 1.92, \quad C_\mu = 0.09, \quad \alpha_{1,k} = 1, \quad \alpha_{1,\epsilon} = 0.76923 \quad (236)$$

61.3.2 Source code

The transport equations of *bubbleFoam* and *twoPhaseEulerFoam* reside in the file `kEpsilon.H`. Listing 489 shows the most important lines of `kEpsilon.H`.

```

1  tmp<volTensorField> tgradU2 = fvc::grad(U2);
2  volScalarField G(2*nut2*(tgradU2() && dev(symm(tgradU2()))));
3
4  // Dissipation equation
5  fvScalarMatrix epsEqn
6  (
7      fvm::ddt(epsilon)
8      + fvm::div(phi2, epsilon)
9      - fvm::Sp(fvc::div(phi2), epsilon)
10     - fvm::laplacian
11     (
12         alpha1Eps*nuEff2, epsilon,
13         "laplacian(DepsilonEff,epsilon)"
14     )
15     ==
16     C1*G*epsilon/k
17     - fvm::Sp(C2*epsilon/k, epsilon)
18 );
19
20 // Turbulent kinetic energy equation
21 fvScalarMatrix kEqn
22 (
23     fvm::ddt(k)
24     + fvm::div(phi2, k)
25     - fvm::Sp(fvc::div(phi2), k)
26     - fvm::laplacian
27     (
28         alpha1k*nuEff2, k,
29         "laplacian(DkEff,k)"
30     )
31     ==
32     G
33     - fvm::Sp(epsilon/k, k)
34 );
35
36 // Re-calculate turbulence viscosity
37 nut2 = Cmu*sqr(k)/epsilon;

```

Listing 489: The turbulent transport equations of the *bubbleFoam* and *twoPhaseEulerFoam* solver

61.4 Modelling the production of turbulent kinetic energy

When comparing the turbulent equations From literature and the sources, the definition of the production of turbulent kinetic energy shows great differences.

61.4.1 Definitions from literature and source files

The production of turbulent kinetic energy seems to be differently defined.

Thesis of H. Rusche [54] - the basis of *bubbleFoam* and *twoPhaseEulerFoam*

$$P_b = 2\nu_{2,eff} (\nabla \mathbf{U}_b \cdot \text{dev} (\nabla \mathbf{U}_b + (\nabla \mathbf{U}_b)^T)) \quad (237)$$

Source code - `kEpsilon.H` of *bubbleFoam* - See Line 2 Listing 489

$$G = 2\nu_T (\nabla \mathbf{U}_2 : \text{dev}(\text{sym}(\nabla \mathbf{U}_2))) \quad (238)$$

Source code - standard k- ϵ model, `kEpsilon.C`

$$G = 2\nu_T |\text{sym}(\nabla \mathbf{U})|^2 \quad (239)$$

Ferziger Peric [35]

$$P = \mu_T \nabla \mathbf{U} : (\nabla \mathbf{U} + (\nabla \mathbf{U})^T) \quad (240)$$

Wilcox [67]

$$G = \mu_T \nabla \mathbf{U} : (\nabla \mathbf{U} + (\nabla \mathbf{U})^T) - \frac{2}{3} \rho k \mathbf{I} : \nabla \mathbf{U} \quad (241)$$

Some definitions use the dynamic viscosity and some others use the kinematic viscosity. For incompressible fluids, this is no major difference between the definitions.

61.4.2 Different use of viscosity

Eq. (237) is the only definition that makes use of the [54] effective viscosity instead of the turbulent viscosity. The reason for this is not explained.

However, the FLUENT Theory Guide [7] states that the effective viscosity is used to calculate the production term when high-Reynolds number versions of the k- ϵ model are used. It is not further specified what is meant with high-Reynolds number versions of the k- ϵ model.

61.4.3 Notation

The definitions in Section 61.4.1 are written in vector notation. However, there seems to be a minor flaw in Eq. (237). There

$$P_b = 2\nu_{2,eff} (\nabla \mathbf{U}_b \cdot \text{dev} (\nabla \mathbf{U}_b + (\nabla \mathbf{U}_b)^T)) \quad (237)$$

The dot can not denote an inner product. The result only has the correct dimension, if the dot denotes a contraction. Therefore, the equation should read

$$P_b = 2\nu_{2,eff} (\nabla \mathbf{U}_b : \text{dev} (\nabla \mathbf{U}_b + (\nabla \mathbf{U}_b)^T)) \quad (242)$$

61.4.4 Definitions from literature

The definition of the production term in Eq. (240) and (241) differ only in the last term.

$$G = \mu_T \nabla \mathbf{U} : (\nabla \mathbf{U}_b + (\nabla \mathbf{U}_b)^T) - \frac{2}{3} \rho k \mathbf{I} : \nabla \mathbf{U} \quad (241)$$

Using the following identities, the contraction can be replaced by an inner product

$$\mathbf{I} : \nabla \mathbf{U} = \text{tr}(\nabla \mathbf{U}) = \nabla \cdot \mathbf{U} \quad (243)$$

For incompressible fluids the divergence of the velocity must be zero due to the continuity equation

$$\nabla \cdot \mathbf{U} = 0 \quad (244)$$

$$G = \mu_T \nabla \mathbf{U} : (\nabla \mathbf{U}_b + (\nabla \mathbf{U}_b)^T) - \underbrace{\frac{2}{3} \rho k \mathbf{I} : \nabla \mathbf{U}}_{=0} \quad (245)$$

Therefore, Eqns. (240) and (241) are identical if the fluid is incompressible. We now can examine the differences of the definitions of the production term, using Eq. (240) as reference equation.

61.4.5 Definitions of Rusche and *bubbleFoam*

The solvers *bubbleFoam* and *twoPhaseEulerFoam* are based on the thesis of H. Rusche [54]. However, the production term is defined differently. Compare Eq. (237) and (238).

$$P_b = 2\nu_{2,eff} (\nabla \mathbf{U}_b : \text{dev} (\nabla \mathbf{U}_b + (\nabla \mathbf{U}_b)^T)) \quad (237)$$

$$G = 2\nu_T (\nabla \mathbf{U}_2 : \text{dev}(\text{sym}(\nabla \mathbf{U}_2))) \quad (238)$$

We ignore the different symbols for the velocity of the continuous phase

$$\mathbf{U}_2 = \mathbf{U}_b \quad (246)$$

The second operator of the contraction is different in both equations. We ask, if the following equation holds

$$\nabla \mathbf{U}_2 : \text{dev}(\text{sym}(\nabla \mathbf{U}_2)) \stackrel{?}{=} \nabla \mathbf{U}_b : \text{dev} (\nabla \mathbf{U}_b + (\nabla \mathbf{U}_b)^T) \quad (247)$$

With the following identities the question is easily answered

$$\text{dev}(\mathbf{T}) = \mathbf{T} - \frac{1}{3} \text{tr}(\mathbf{T}) \quad (248)$$

$$\text{sym}(\mathbf{T}) = \frac{1}{2} (\mathbf{T} + (\mathbf{T})^T) \quad (249)$$

$$\text{dev} (\text{sym}(\nabla \mathbf{U}_2)) = \text{dev} \left(\frac{1}{2} (\nabla \mathbf{U}_2 + (\nabla \mathbf{U}_2)^T) \right) \quad (250)$$

$$\text{dev} (\text{sym}(\nabla \mathbf{U}_2)) = \frac{1}{2} \text{dev} (\nabla \mathbf{U}_2 + (\nabla \mathbf{U}_2)^T) \quad (251)$$

$$\text{dev} (\text{sym}(\nabla \mathbf{U}_2)) = \frac{1}{2} \underbrace{\left((\nabla \mathbf{U}_2 + (\nabla \mathbf{U}_2)^T) - \frac{1}{3} \text{tr}(\nabla \mathbf{U}_2 + (\nabla \mathbf{U}_2)^T) \right)}_{=\text{dev}(\nabla \mathbf{U}_2 + (\nabla \mathbf{U}_2)^T)} \quad (252)$$

$$\text{dev} (\text{sym}(\nabla \mathbf{U}_2)) = \frac{1}{2} \text{dev} (\nabla \mathbf{U}_2 + (\nabla \mathbf{U}_2)^T) \quad (253)$$

This leads to the answer

$$\nabla \mathbf{U}_2 : \text{dev} (\text{sym}(\nabla \mathbf{U}_2)) = \frac{1}{2} \nabla \mathbf{U}_b : \text{dev} (\nabla \mathbf{U}_b + (\nabla \mathbf{U}_b)^T) \quad (254)$$

The definition of the production term in the source code differs in two ways from the definition in the source code

1. The use of different viscosities, see Eqns. (237) and (238).
2. A factor of 2, compare Eqns. (247) and (254)

The reason for this differences is not clear. H. Rusche refers to an article which is not available to the author.

61.4.6 Definitions of Ferziger and *bubbleFoam*

We now compare the definitions of Ferziger and *bubbleFoam*. The definition of Ferziger is – like the equations in most other book about turbulence – for single-phase systems. However, *bubbleFoam* is a two-phase solver. The question of considering turbulence in two-phase systems is not answered yet. *bubbleFoam* considers turbulence for the continuous phase by the use of a turbulence model. The turbulence of the disperse phase is linked to the continuous phase. Therefore, turbulence model equations of *bubbleFoam* are quite similar to single-phase turbulence equations.

$$G = 2\nu_T (\nabla \mathbf{U}_2 : \text{dev}(\text{sym}(\nabla \mathbf{U}_2))) \quad (238)$$

$$P = \mu_T \nabla \mathbf{U} : (\nabla \mathbf{U} + (\nabla \mathbf{U})^T) \quad (240)$$

We ignore the different viscosities and ask ourselves

$$\nabla \mathbf{U} : (\nabla \mathbf{U} + (\nabla \mathbf{U})^T) \stackrel{?}{=} 2 (\nabla \mathbf{U}_2 : \text{dev}(\text{sym}(\nabla \mathbf{U}_2))) \quad (255)$$

Inserting Eq. (253) gives

$$\nabla \mathbf{U} : (\nabla \mathbf{U} + (\nabla \mathbf{U})^T) = 2 (\nabla \mathbf{U}_2 : \underbrace{\text{dev}(\text{sym}(\nabla \mathbf{U}_2)))}_{=\frac{1}{2} \text{dev}(\nabla \mathbf{U}_2 + (\nabla \mathbf{U}_2)^T)}) \quad (256)$$

$$\nabla \mathbf{U} : (\nabla \mathbf{U} + (\nabla \mathbf{U})^T) = \nabla \mathbf{U}_2 : \text{dev}(\nabla \mathbf{U}_2 + (\nabla \mathbf{U}_2)^T) \quad (257)$$

Now we insert Eq. (248) into the *rhs* of Eq. (257)

$$\nabla \mathbf{U} : (\nabla \mathbf{U} + (\nabla \mathbf{U})^T) = \nabla \mathbf{U} : \left(\text{dev}(\nabla \mathbf{U} + (\nabla \mathbf{U})^T) + \frac{1}{3} \text{tr}(\nabla \mathbf{U} + (\nabla \mathbf{U})^T) \right) \quad (258)$$

Using the following identities and Eq. (243)

$$\text{tr}(\mathbf{A} + \mathbf{B}) = \text{tr}(\mathbf{A}) + \text{tr}(\mathbf{B}) \quad (259)$$

$$\text{tr}(\mathbf{A}^T) = \text{tr}(\mathbf{A}) \quad (260)$$

$$\mathbf{I} : \nabla \mathbf{U} = \text{tr}(\nabla \mathbf{U}) = \nabla \cdot \mathbf{U} \quad (243)$$

$$\nabla \mathbf{U} : (\nabla \mathbf{U} + (\nabla \mathbf{U})^T) = \nabla \mathbf{U} : \left(\text{dev}(\nabla \mathbf{U} + (\nabla \mathbf{U})^T) + \frac{2}{3} (\nabla \cdot \mathbf{U}) \right) \quad (261)$$

The second term of the *rhs* vanishes according to the continuity equation for an incompressible fluid

$$\nabla \mathbf{U} : (\nabla \mathbf{U} + (\nabla \mathbf{U})^T) = \nabla \mathbf{U} : \left(\text{dev}(\nabla \mathbf{U} + (\nabla \mathbf{U})^T) + \frac{2}{3} \underbrace{(\nabla \cdot \mathbf{U})}_{\nabla \cdot \mathbf{U} = 0} \right) \quad (262)$$

Eq. (263) now resembles Eq. (257). Therefore, we proofed that the definition of *bubbleFoam* is equivalent to the definition of Ferzinger

$$\nabla \mathbf{U} : (\nabla \mathbf{U} + (\nabla \mathbf{U})^T) = \nabla \mathbf{U} : \text{dev}(\nabla \mathbf{U} + (\nabla \mathbf{U})^T) \quad (263)$$

61.4.7 Definition of standard k-ε of OpenFOAM

We now compare the definition of the production term of the standard k-ε model implemented in OpenFOAM with the definition found in [35].

Source code - standard k-ε model, `kEpsilon.C`

$$G = 2\nu_T |\text{sym}(\nabla \mathbf{U})|^2 \quad (239)$$

Ferzinger Peric [35]

$$P = \nu_T \nabla \mathbf{U} : (\nabla \mathbf{U} + (\nabla \mathbf{U})^T) \quad (240)$$

Starting from Eq. (240), we will use Eq. (263) and Eq. (253)

$$\nabla \mathbf{U} : (\nabla \mathbf{U} + (\nabla \mathbf{U})^T) = \nabla \mathbf{U} : \text{dev}(\nabla \mathbf{U} + (\nabla \mathbf{U})^T) \quad (263)$$

$$\text{dev}(\nabla \mathbf{U} + (\nabla \mathbf{U})^T) = 2 \text{dev}(\text{sym}(\nabla \mathbf{U})) \quad (253)$$

to gain

$$\nabla \mathbf{U} : (\nabla \mathbf{U} + (\nabla \mathbf{U})^T) = 2 \nabla \mathbf{U} : \text{dev}(\text{sym}(\nabla \mathbf{U})) \quad (264)$$

We use definition (265) to change Eq. (239)

$$|\text{sym}(\nabla \mathbf{U})|^2 = \text{sym}(\nabla \mathbf{U}) : \text{sym}(\nabla \mathbf{U}) \quad (265)$$

Now we pose the question

$$\text{sym}(\nabla \mathbf{U}) : \text{sym}(\nabla \mathbf{U}) \stackrel{?}{=} \nabla \mathbf{U} : \text{dev}(\text{sym}(\nabla \mathbf{U})) \quad (266)$$

The *lhs* of Eq. (266) corresponds to Eq. (239). The *rhs* of Eq. (266) was derived from Eq. (240). Now, we use some identities

$$\text{dev}(\mathbf{T}) = \mathbf{T} - \frac{1}{3} \text{tr}(\mathbf{T}) \quad (248)$$

$$\text{tr}(\text{sym}(\mathbf{T})) = \text{tr}(\mathbf{T}) \quad (267)$$

to reformulate the *rhs* of Eq. (266)

$$\nabla \mathbf{U} : \text{dev}(\text{sym}(\nabla \mathbf{U})) = \nabla \mathbf{U} : \left(\text{sym}(\nabla \mathbf{U}) - \frac{1}{3} \text{tr}(\nabla \mathbf{U}) \right) \quad (268)$$

As we now concentrate on incompressible single-phase problems, we can eliminate the second term of the *rhs* of Eq. (268) by the use of Eq. (243)

$$\mathbf{I} : \nabla \mathbf{U} = \text{tr}(\nabla \mathbf{U}) = \nabla \cdot \mathbf{U} = 0 \quad (243)$$

We now have

$$\nabla \mathbf{U} : \text{dev}(\text{sym}(\nabla \mathbf{U})) = \nabla \mathbf{U} : \text{sym}(\nabla \mathbf{U}) \quad (269)$$

The following equation remains, which is easily proofed by some tensor calculus

$$\text{sym}(\nabla \mathbf{U}) : \text{sym}(\nabla \mathbf{U}) = \nabla \mathbf{U} : \text{sym}(\nabla \mathbf{U}) \quad (270)$$

Every tensor can be decomposed into a symmetric and a skew part

$$\mathbf{T} = \text{sym}(\mathbf{T}) + \text{skew}(\mathbf{T}) \quad (271)$$

$$\text{sym}(\mathbf{T}) = \frac{1}{2} (\mathbf{T} + \mathbf{T}^T) \quad (272)$$

$$\text{skew}(\mathbf{T}) = \frac{1}{2} (\mathbf{T} - \mathbf{T}^T) \quad (273)$$

Therefore, we can write

$$\mathbf{T} : \text{sym}(\mathbf{T}) = \text{sym}(\mathbf{T}) : \text{sym}(\mathbf{T}) + \text{skew}(\mathbf{T}) : \text{sym}(\mathbf{T}) \quad (274)$$

The following properties of skew tensors let the second contraction vanish

$$\underbrace{\text{skew}(\mathbf{T})}_{a_{ij}} : \underbrace{\text{sym}(\mathbf{T})}_{s_{ij}} \quad (275)$$

$$a_{ii} = 0 \quad (276)$$

$$a_{ij} = -a_{ji} \quad (277)$$

$$\text{skew}(\mathbf{T}) : \text{sym}(\mathbf{T}) = a_{ij} s_{ij} = 0 \quad (278)$$

Finally, we obtain

$$\mathbf{T} : \text{sym}(\mathbf{T}) = \text{sym}(\mathbf{T}) : \text{sym}(\mathbf{T}) \quad (279)$$

Therefore, we proofed that the definition of the standard k- ϵ model is equivalent to the definition of Ferziger.

62 Some theory behind the scenes of LES

62.1 LES model hierarchy

The large eddy simulation is based on the spatial filtering of the governing equations. Similar to the Reynolds-averaged modelling strategy (filtering with respect to time), the large eddy modelling strategy requires some

closure models. In principle, the velocity is decomposed into a grid-scale and a sub-grid scale portion. The grid-scale portion is resolved by the governing equations. The sub-grid scale portion – or the influence of the sub-grid scale portion on the resolved velocity – needs to be modelled.

Similar to the RANS approach, the closure terms appear in the stress terms of the momentum equations. There are several modelling strategies to close the equations. The class hierarchy of the LES models of OpenFOAM reflects the different approaches. Figure 136 shows the first layer of the class hierarchy of the LES models in OpenFOAM. First layer means that a class derived from the abstract class `LESModel` may be an abstract class itself and therefore be the base for other classes²²⁶²²⁷.

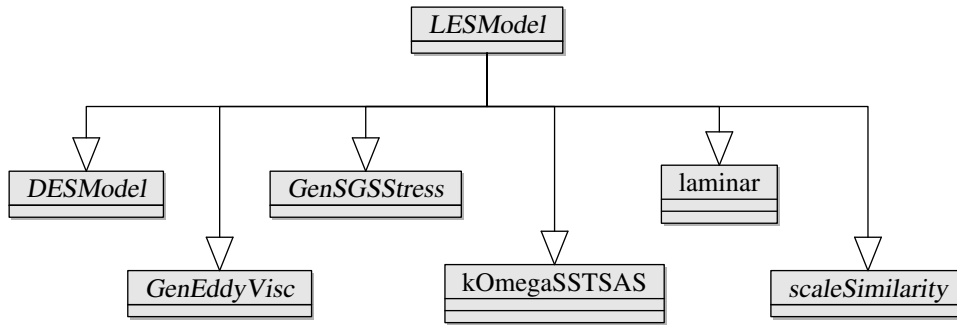


Figure 136: First layer of the class hierarchy of the LES models of OpenFOAM

The classification according to Figure 136 is not the only possible way to divide all existing LES models into categories.

62.2 Eddy viscosity models

One of the most common approaches of closing the governing equations when using an LES turbulence modelling strategy are eddy viscosity models. Like the RANS turbulence models, the eddy viscosity models make use of the Boussinesq hypothesis. The contribution of the sub-grid scale terms is modelled by an additional viscosity. The effective viscosity is the sum of the laminar viscosity and the sub-grid viscosity.

$$\nu_{eff} = \nu + \nu_{SGS} \quad (280)$$

62.2.1 Class hierarchy

The base class for all eddy viscosity models is `GenEddyVisc`. Figure 137 shows the class hierarchy with focus on `GenEddyVisc`.

62.2.2 Classification

The eddy viscosity models can be divided further based on the way the sub-grid viscosity is computed and the complexity of the model.

²²⁶In a class diagram a class with an italic written name is an abstract class. A class with an upright written name is an actual class.

²²⁷This shows the great advantage of object oriented programming. The class hierarchy of the code reflects the relation between the objects in reality, e.g. every eddy viscosity model is an LES model, but not every LES model is an eddy viscosity model.

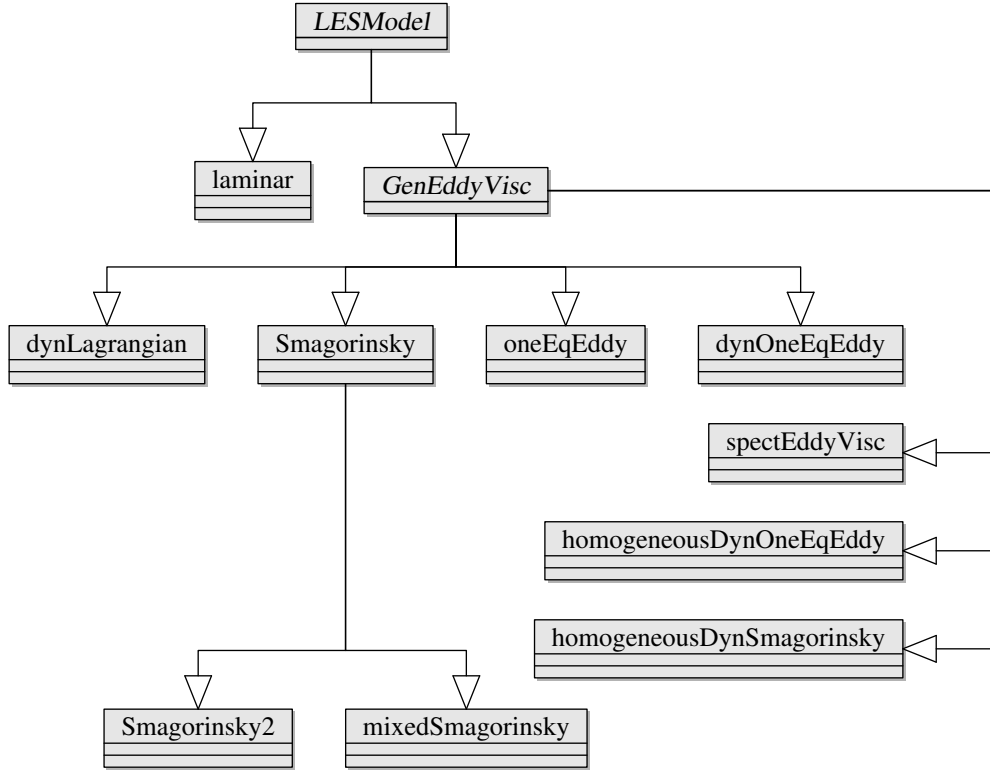


Figure 137: Class hierarchy of the eddy viscosity models in OpenFOAM

	constant coefficient	dynamic coefficient
algebraic model	Smagorinsky	homogeneousDynSmagorinsky
	Smagorinsky2	spectEddyVisc
one equation model	oneEqEddy	dynOneEqEddy
		homogeneousDynOneEqEddy
		dynLagrangian

Table 7: Comparison of the eddy viscosity models of OpenFOAM

62.2.3 Eddy viscosity

For dimensional reasons, the eddy viscosity must be a product of a length and a velocity scale [24]. Eq. (282) shows the generic equation for the sub-grid viscosity. An additional model constant is the third term in the product. The way the model constant is computed as well as the choice for the length and velocity scales is determined by the model.

$$[\nu_{SGS}] = \frac{\text{m}^2}{\text{s}} = \frac{\text{m}}{\text{s}} \cdot \text{m} \quad (281)$$

$$\nu_{SGS} = C_{SGS} l_{SGS} q_{SGS} \quad (282)$$

A choice that is common to a number of eddy viscosity models in OpenFOAM is to choose the filter width as the length scale and the square root of the sub-grid kinetic energy as the velocity scale. Algebraic models usually calculate the sub-grid kinetic energy from known quantities, e.g. based on the velocity gradient. One equation models typically solve a transport equation for the sub-grid scale kinetic energy.

$$l_{SGS} = \Delta \quad (283)$$

$$[l_{SGS}] = \text{m} \quad (284)$$

$$q_{SGS} = \sqrt{k_{SGS}} \quad (285)$$

$$[q_{SGS}] = \sqrt{\frac{\text{m}^2}{\text{s}^2}} = \frac{\text{m}}{\text{s}} \quad (286)$$

62.2.4 The Smagorinsky LES model

The Smagorinsky eddy viscosity is one of the simplest LES models. From Table 7 we see that this is an algebraic model with a constant model coefficient. This model was published 1963 [56].

Eq. (287) shows the definition of the sub-grid scale viscosity according to the Smagorinsky model as it can be found in literature [24].

$$\nu_{SGS} = (C_S \Delta)^2 |\mathbf{S}| \quad (287)$$

with

$$\mathbf{S} = \text{sym}(\nabla \mathbf{u}) = \text{sym}(\text{grad}(\mathbf{u}))$$

$$|\mathbf{T}| = \sqrt{\mathbf{T} : \mathbf{T}}$$

Some rearrangement of Eq. (287) is necessary to match the form of Definition (282) and (285). Eqns. (288) to (290) show the necessary steps to match the generic definition of ν_{SGS} .

$$\nu_{SGS} = C_S^2 \underbrace{\Delta}_{l_{SGS}} \underbrace{\Delta \sqrt{\mathbf{S} : \mathbf{S}}}_{q_{SGS}} \quad (288)$$

$$q_{SGS} = \sqrt{k_{SGS}} = \Delta \sqrt{\mathbf{S} : \mathbf{S}} \quad (289)$$

$$\Rightarrow k_{SGS} = \Delta^2 \mathbf{S} : \mathbf{S} \quad (290)$$

Implementation

The implementation in the source code differs a little from the equations above.

```

1 void Smagorinsky::updateSubGridScaleFields(const volTensorField& gradU)
2 {
3     nuSgs_ = ck_*delta()*sqrt(k(gradU));
4     nuSgs_.correctBoundaryConditions();
5 }

```

Listing 490: The function `updateSubGridScaleFields()` in the file `Smagorinsky.C`

```

1 tmp<volScalarField> k(const tmp<volTensorField>& gradU) const
2 {
3     return (2.0*ck_/ce_)*sqr(delta())*magSqr(dev(symm(gradU)));
4 }

```

Listing 491: The function `k()` in the file `Smagorinsky.H`

Listing 490 shows the implementation of how the sub-grid viscosity is computed by the Smagorinsky model in OpenFOAM. Listing 491 shows how the model calculates the sub-grid kinetic energy.

$$\text{nuSgs} = ck \Delta \sqrt{k} \quad (291)$$

$$k = 2 \frac{ck}{ce} \Delta^2 |\text{dev} \mathbf{S}|^2 \quad (292)$$

with

$$\mathbf{S} = \text{sym grad}(\mathbf{u}) \quad (293)$$

it follows

$$\text{nuSgs} = ck\Delta\sqrt{2\frac{ck}{ce}\Delta^2|\text{dev } \mathbf{S}|^2} \quad (294)$$

$$\text{nuSgs} = ck\sqrt{2\frac{ck}{ce}\Delta^2|\text{dev } \mathbf{S}|} \quad (295)$$

the comparison with Eq. 287 shows

$$\nu_{SGS} = (C_S\Delta)^2|\mathbf{S}| \quad (287)$$

$$\Rightarrow C_S^2 = ck\sqrt{2\frac{ck}{ce}} \quad (296)$$

Eq. (296) shows how the Smagorinsky constant can be calculated from the model constants. The Smagorinsky constant is often stated in publications using or investigating the Smagorinsky model, because it is the only degree of freedom of the Smagorinsky model.

In OpenFOAM the Smagorinsky model has two model constants. ce is inherited from the class `GenEddyVisc`. This constant is used in the definition of the sub-grid dissipation rate. The default value of ce is 1.048 and is defined in the constructor of the class `GenEddyVisc` in the file `GenEddyVisc.C`.

Therefore, the model constant ck is the only degree of freedom of the Smagorinsky model of OpenFOAM. The default value of ck is 0.094. This results in a default value for C_S of $0.1995 \approx 0.2$. The value of C_S varies in literature depending on the publication from 0.07 to 0.33 [9, 45].

```

1  //- Return sub-grid dissipation rate
2  virtual tmp<volScalarField> epsilon() const
3  {
4      return tmp<volScalarField>
5      (
6          new volScalarField
7          (
8              IOobject
9              (
10                 "epsilon",
11                 runTime_.timeName(),
12                 mesh_,
13                 IOobject::NO_READ,
14                 IOobject::NO_WRITE
15             ),
16             ce_*k()*sqrt(k())/delta()
17         )
18     );
19 }
```

Listing 492: The function `epsilon()` in the file `GenEddyVisc.H`

62.2.5 The oneEqEddy LES model

The `oneEqEddy` model is one of the standard LES models of OpenFOAM. This model is an one equation eddy viscosity model with a constant model coefficient. Eq. 297 shows how the sub-grid viscosity is calculated by the `oneEqEddy` model. The constant ck has a default value of 0.094.

$$\nu_{SGS} = ck\Delta\sqrt{k_{SGS}} \quad (297)$$

The transport equation for k_{SGS}

As this model is an one equation model, it introduces an additional equation to the set of equations. This additional equation is a transport equation for the sub-grid kinetic energy k_{SGS} . k_{SGS} is the kinetic energy of

the unresolved portion of the velocity. Thus, k_{SGS} is called sub-grid kinetic energy.

$$\frac{\partial k_{SGS}}{\partial t} + \nabla \cdot (k_{SGS} \mathbf{u}) - \nabla \cdot (D_k \nabla k_{SGS}) = G - \epsilon_{SGS} \quad (298)$$

with

$$\begin{aligned} D_k &= \nu + \nu_{SGS} \\ G &= \nu_{SGS} |\text{sym}(\nabla \mathbf{u})|^2 \\ \epsilon_{SGS} &= ce \frac{\sqrt{k_{SGS}}}{\Delta} k_{SGS} \end{aligned}$$

Eq. 298 is similar to the transport equation for k of the k- ϵ model. Also the definition of the sub-grid viscosity is similar to the definition of the turbulent viscosity of the k- ϵ model. This is not very obvious. Therefore, we shall explore this matter further.

$$\nu_{SGS} = ck \Delta \sqrt{k_{SGS}} \quad (297)$$

$$\nu_{SGS} = ck \frac{ce}{ce} \frac{k_{SGS}}{k_{SGS}} \frac{\sqrt{k_{SGS}}}{\sqrt{k_{SGS}}} \Delta \sqrt{k_{SGS}} \quad (299)$$

$$\nu_{SGS} = ck ce \frac{k_{SGS} \sqrt{k_{SGS}} \sqrt{k_{SGS}}}{ce \frac{k_{SGS} \sqrt{k_{SGS}}}{\Delta}} \quad (300)$$

$$\nu_{SGS} = ck ce \frac{k_{SGS}^2}{\epsilon_{SGS}} \quad (301)$$

Eq. 301 is similar to Eq. 221 – the definition of the turbulent viscosity of the k- ϵ model

$$\nu_T = C_\mu \frac{k^2}{\epsilon} \quad (221)$$

The product of ck and ce when using their default values gives $ck \cdot ce = 0.0985$ which is approximately the default value of C_μ of the k- ϵ model, which is $C_\mu = 0.09$.

63 The use of phi

63.1 The question

The governing equations of the solvers of OpenFOAM are written in a special notation that makes it easy to compare the source codes with equations from a fluid dynamics textbook. In Section 42.1 the governing equations of the solver *pimpleFoam* are examined. There, the terms of Eq. 79 are compared with the source code, see Listing 263. Here, we repeat the comparison of how the convective term is written in the sources and how this term is expressed mathematically.

$$\underbrace{\nabla(\mathbf{u}\mathbf{u})}_{\text{div}(\mathbf{u}\mathbf{u})} \Leftrightarrow \text{fvm}::\text{div}(\mathbf{phi}, \mathbf{U})$$

We now examine how **phi** is defined and how we can find **phi** in the math.

63.2 Implementation

63.2.1 The origin of fields

One way to learn more about **phi** is to look for its definition in the source code of OpenFOAM.

Listing 493 shows the first lines of the **main** function of the solver *pimpleFoam*. The **main** function of any C or C++ program is entered, when this program is executed. So, the instructions of Listing 493 are the first instructions that are executed, when the solver is called.

In line 6 of Listing 493 the file **createFields.H** is included. This file contains instructions that create the data structures of all fields that are necessary for the solver (e.g. the pressure or the velocity field).

```

1 int main(int argc, char *argv[])
2 {
3     #include "setRootCase.H"
4     #include "createTime.H"
5     #include "createMesh.H"
6     #include "createFields.H"
7     #include "initContinuityErrs.H"
8
9     /* the rest of the solver */

```

Listing 493: The first few line of the main function of *pimpleFoam* in *pimpleFoam.C*

The file `createFields.H` contains the content of Listing 494. There, the velocity field `U` is created. In line 15 the file `createPhi.H` is included. There, the field `phi` is created.

```

1 Info<< "Reading field U\n" << endl;
2 volVectorField U
3 (
4     IOobject
5     (
6         "U",
7         runTime.timeName(),
8         mesh,
9         IOobject::MUST_READ,
10        IOobject::AUTO_WRITE
11    ),
12    mesh
13 );
14
15 #include "createPhi.H"

```

Listing 494: The creation of `U` and `phi` in the file `createFields.H`

63.2.2 How `phi` is defined

Listing 495 shows the content of the file `createPhi.H`. From this Listing we see the data type of `phi`, it is `surfaceScalarField`. This tells us, that `phi` is a scalar, that is defined on the faces of the control volumes (cells) of the mesh.

Line 13 tells us how `phi` is defined. There, we find out, that `phi` is the inner product of the velocity – we forget for the moment about the function `linearInterpolate` – and the face surface area vector. In Listing 496 we see the declaration of the function `Sf()`. In Listing 497 we see, that the variable `mesh` of Listing 495 is of the type `fvMesh`.

```

1 Info<< "Reading/calculating face flux field phi\n" << endl;
2
3 surfaceScalarField phi
4 (
5     IOobject
6     (
7         "phi",
8         runTime.timeName(),
9         mesh,
10        IOobject::READ_IF_PRESENT,
11        IOobject::AUTO_WRITE
12    ),
13    linearInterpolate(U) & mesh.Sf()
14 );

```

Listing 495: The creation of `phi` in the file `createPhi.H`

```

1 //- Return cell face area vectors
2 const surfaceVectorField& Sf() const;

```

Listing 496: The declaration of the method `Sf()` of the class `fvMesh` in the file `fvMesh.H`

```

1 Foam::Info
2   << "Create mesh for time = "
3   << runTime.timeName() << Foam::nl << Foam::endl;
4
5 Foam::fvMesh mesh
6 (
7   Foam::IObject
8   (
9     Foam::fvMesh::defaultRegion,
10    runTime.timeName(),
11    runTime,
12    Foam::IObject::MUST_READ
13  )
14 );

```

Listing 497: The creation of the mesh in the file `createMesh.H`

63.3 The math

Now, let us examine the origin of `phi` from the mathematical point of view. We start with the governing equations of a solver for incompressible fluids. Therefore, Eq. 79 is repeated below.

$$\frac{\partial \mathbf{u}}{\partial t} + \nabla(\mathbf{u}\mathbf{u}) + \nabla \cdot \text{dev}(-\nu^{eff} (\nabla \mathbf{u} + (\nabla \mathbf{u})^T)) = -\nabla p + \mathbf{Q} \quad (79)$$

This equation is written in differential form and is valid everywhere in the fluid. In order to use the finite volume method, we need the governing equations in the integral form. Integrating Eq. (79) over a control volume yields:

$$\int_V \frac{\partial \mathbf{u}}{\partial t} + \nabla(\mathbf{u}\mathbf{u}) + \nabla \cdot \text{dev}(-\nu^{eff} (\nabla \mathbf{u} + (\nabla \mathbf{u})^T)) dV = \int_V -\nabla p + \mathbf{Q} dV \quad (302)$$

Now we will have a closer look on the second term of Eq. (302). That is the convective term we already saw at the beginning of this section.

Using Gauss' theorem, we replace the integration over the volume of our control volume with the integration over the surface of the control volume.

$$\int_V \nabla(\mathbf{u}\mathbf{u}) dV = \oint_{\partial V} (\mathbf{u}\mathbf{u}) \cdot d\mathbf{S} \quad (303)$$

Because our control volume is a polyhedron (in most cases a hexahedron or a tetrahedron), the surface integral reduces to a sum of integrals over the faces S_f of the polyhedron.

$$\oint_{\partial V} (\mathbf{u}\mathbf{u}) \cdot d\mathbf{S} = \sum_f \int_{S_f} (\mathbf{u}\mathbf{u}) \cdot d\mathbf{S}_f \quad (304)$$

$$\|\mathbf{S}_f\| = S_f \quad (305)$$

With $\mathbf{S}_f$ being the surface normal vector of the face f . The norm of this vector is equal to the area of the face f . We denote with the subscript f the mean face-value of a quantity.

$$\sum_f \int_{S_f} (\mathbf{u}\mathbf{u}) \cdot d\mathbf{S}_f = \sum_f (\mathbf{u}\mathbf{u})_f \cdot \mathbf{S}_f \quad (306)$$

$$(\mathbf{u}\mathbf{u})_f = \frac{1}{S_f} \int_{S_f} (\mathbf{u}\mathbf{u}) d\mathbf{S}_f \quad (307)$$

$$(\mathbf{u}\mathbf{u})_f \approx (\mathbf{u}_f \mathbf{u}_f) \quad (308)$$

$$\sum_f (\mathbf{u}\mathbf{u})_f \cdot \mathbf{S}_f \approx \sum_f (\mathbf{u}_f \mathbf{u}_f) \cdot \mathbf{S}_f \quad (309)$$

Eq. (309) contains the fundamental assumption or approximation of the finite volume method. It is assumed, that the mean face-value of the product of the velocities is (approximately) equal to the product of the mean face-values of the velocity, see Eq. (308). In general, the operations averaging and multiplication are not commutative.

We are now nearly finished. The *rhs* of Eq. (309) contains all ingredients we need for **phi**. A surface area vector, a velocity and an inner vector product. See Listing 495. However, this ingredients are not in the order we need. Therefore, there is need for some more math to do.

A general rule of tensor calculus states:

$$\mathbf{a} \otimes \mathbf{b} \cdot \mathbf{c} = \mathbf{a}(\mathbf{b} \cdot \mathbf{c}) \quad (310)$$

In this document, we omit the symbol $\otimes$ for the sake of brevity.

$$\mathbf{a} \otimes \mathbf{b} \cdot \mathbf{c} = (\mathbf{a}\mathbf{b}) \cdot \mathbf{c} \quad (311)$$

Eq. (311) looks like the *rhs* of Eq. (309).

$$(\mathbf{u}_f \mathbf{u}_f) \cdot \mathbf{S}_f = \mathbf{u}_f \underbrace{(\mathbf{u}_f \cdot \mathbf{S}_f)}_{=\phi_f} \quad (312)$$

$$\mathbf{u}_f(\mathbf{u}_f \cdot \mathbf{S}_f) = \mathbf{u}_f \phi_f \quad (313)$$

63.4 Summary

Now, after having dug deep into the sources and after having done some math, we can summarize all thoughts so far. We want to understand this equivalency.

$$\underbrace{\nabla(\mathbf{u}\mathbf{u})}_{\text{div}(\mathbf{u}\mathbf{u})} \Leftrightarrow \text{fvm::div}(\mathbf{phi}, \mathbf{U})$$

The math tells use the following identities.

$$\int_V \nabla(\mathbf{u}\mathbf{u}) \, dV = \oint_{\partial V} (\mathbf{u}\mathbf{u}) \cdot d\mathbf{S} \quad (314)$$

$$\oint_{\partial V} (\mathbf{u}\mathbf{u}) \cdot d\mathbf{S} = \sum_f (\mathbf{u}\mathbf{u})_f \cdot \mathbf{S}_f \quad (315)$$

$$\sum_f (\mathbf{u}\mathbf{u})_f \cdot \mathbf{S}_f \approx \sum_f (\mathbf{u}_f \mathbf{u}_f) \cdot \mathbf{S}_f \quad (316)$$

$$\sum_f (\mathbf{u}_f \mathbf{u}_f) \cdot \mathbf{S}_f = \sum_f \mathbf{u}_f (\mathbf{u}_f \cdot \mathbf{S}_f) \quad (317)$$

$$\sum_f \mathbf{u}_f (\mathbf{u}_f \cdot \mathbf{S}_f) = \sum_f \mathbf{u}_f \phi_f \quad (318)$$

We have shown, that the integral formulation of the convective term can be reformulated to incorporate ϕ and $\mathbf{u}$ instead of $\mathbf{u}\mathbf{u}$.

64 Derivation of the IATE diameter model

In this section we cover the derivation of OpenFOAMs IATE diameter model from [33].

64.1 Number density transport equation

We start with the transport equation for the bubble number density distribution $f = f(V, \mathbf{x}, t)$, e.g. from [33]. For sake of readability in most cases we refer to $f(V, \mathbf{x}, t)$ simply as f .

The first term of Eqn. (319) is the local rate of change of the bubble number density distribution. The second term represents convective transport. The third term represents the rate of change due to change of bubble volume. On the right hand side of the equation are source terms due to bubble interactions S_j and phase change S_{ph} .

$$\frac{\partial f}{\partial t} + \nabla \cdot (f\mathbf{u}) + \frac{\partial}{\partial V} \left(f \frac{dV}{dt} \right) = \sum_j S_j + S_{ph} \quad (319)$$

The equation for the bubble number density distribution is much too detailed for most flow studies [39]. Thus, we derive a transport equation for the area concentration a_i . The area concentration is a moment of the bubble number density distribution. Besides the area concentration we can define further quantities based on the moments of the number density distribution.

Eqn. (320) lists the general definition of the i -th moment m_i of the probability density function $f(x)$.

$$m_i = \int_a^b f(x) x^i dx \quad (320)$$

We now define some moments of the bubble number density distribution.

$$\text{Total number of bubbles per unit volume} \quad n(x, t) = \int_{V_{min}}^{V_{max}} f(V, \mathbf{x}, t) dV \quad (321)$$

$$\text{Volume fraction of bubbles} \quad \alpha(x, t) = \int_{V_{min}}^{V_{max}} f(V, \mathbf{x}, t) V dV \quad (322)$$

$$\text{Area concentration of bubbles} \quad a_i(x, t) = \int_{V_{min}}^{V_{max}} f(V, \mathbf{x}, t) A_i(V) dV \quad (323)$$

64.2 Interfacial area transport equation

64.2.1 Deriving the governing equations

We will use Eqn. (323) to derive a transport equation for the area concentration from Eqn. (319). First we multiply Eqn. (319) by the average interfacial area $A_i(V)$ of bubbles with the volume V .

$$A_i \frac{\partial f}{\partial t} + A_i \nabla \cdot (f\mathbf{u}) + A_i \frac{\partial}{\partial V} \left(f \frac{dV}{dt} \right) = A_i \left(\sum_j S_j + S_{ph} \right) \quad (324)$$

Then, we integrate Eqn. (324) over all bubble sizes

$$\int_{V_{min}}^{V_{max}} \left[A_i \frac{\partial f}{\partial t} + A_i \nabla \cdot (f\mathbf{u}) + A_i \frac{\partial}{\partial V} \left(f \frac{dV}{dt} \right) \right] dV = \int_{V_{min}}^{V_{max}} A_i \left(\sum_j S_j + S_{ph} \right) dV \quad (325)$$

Now we will take a closer look on the single terms of Eqn. (325). For the first term, we simply apply Leibnitz rule. Here it is important to note, that A_i is constant in space and time. With Eqn. (323), we gain the local derivative of the interfacial area concentration a_i .

$$\int_{V_{min}}^{V_{max}} A_i \frac{\partial f}{\partial t} dV = \frac{\partial}{\partial t} \int_{V_{min}}^{V_{max}} A_i f dV \quad (326)$$

$$\int_{V_{min}}^{V_{max}} A_i \frac{\partial f}{\partial t} dV = \frac{\partial}{\partial t} a_i \quad (327)$$

The convective term of Eqn. (325) can be treated in a similar fashion. If the velocity is independent of the bubble size, we can put the $\mathbf{u}$ in front of the integral over all bubble sizes. Thus, we gain the convective term for the interfacial area concentration.

$$\int_{V_{min}}^{V_{max}} A_i \nabla \cdot (f \mathbf{u}) dV = \int_{V_{min}}^{V_{max}} \nabla \cdot (A_i f \mathbf{u}) dV \quad (328)$$

$$\int_{V_{min}}^{V_{max}} A_i \nabla \cdot (f \mathbf{u}) dV = \nabla \cdot \left(\mathbf{u} \int_{V_{min}}^{V_{max}} A_i f dV \right) \quad (329)$$

$$\int_{V_{min}}^{V_{max}} A_i \nabla \cdot (f \mathbf{u}) dV = \nabla \cdot (\mathbf{u} a_i) \quad (330)$$

If the velocity is not independent of the bubble size we can follow a similar strategy to derive a convective term which is formulated in terms of the interfacial area concentration.

$$\int_{V_{min}}^{V_{max}} A_i \nabla \cdot (f \mathbf{u}) dV = \int_{V_{min}}^{V_{max}} \nabla \cdot \left(\frac{a_i}{a_i} A_i f \mathbf{u} \right) dV \quad (331)$$

$$\int_{V_{min}}^{V_{max}} A_i \nabla \cdot (f \mathbf{u}) dV = \nabla \cdot \left(a_i \frac{\int_{V_{min}}^{V_{max}} A_i f \mathbf{u} dV}{a_i} \right) \quad (332)$$

$$\int_{V_{min}}^{V_{max}} A_i \nabla \cdot (f \mathbf{u}) dV = \nabla \cdot (a_i \mathbf{u}_i) \quad (333)$$

With the average local bubble velocity weighted by the bubble number $\mathbf{u}_i$ [18]

$$\mathbf{u}_i = \frac{\int_{V_{min}}^{V_{max}} A_i f \mathbf{u} dV}{\int_{V_{min}}^{V_{max}} A_i f dV} \quad (334)$$

The third term of Eqn. (325) needs more special treatment. In Section 64.5.1 we show the proof for (335). This term relates to the gas expansion.

$$\int_{V_{min}}^{V_{max}} A_i \frac{\partial}{\partial V} \left(f \frac{dV}{dt} \right) dV = -\frac{2}{3} \frac{\dot{\alpha}}{\alpha} a_i \quad (335)$$

The RHS of Eqn. 325 contains the terms due to bubble-bubble interaction and due to phase change.

$$\int_{V_{min}}^{V_{max}} \left[A_i \frac{\partial f}{\partial t} + A_i \nabla \cdot (f \mathbf{u}) + A_i \frac{\partial}{\partial V} \left(f \frac{dV}{dt} \right) \right] dV = \int_{V_{min}}^{V_{max}} A_i \left(\sum_j S_j + S_{ph} \right) dV \quad (325)$$

There are two approaches to model the source terms due to bubble interaction [41]. One can solve the integral equation for these source terms (336) or solve algebraic equations using mean parameters (337).

The latter approach assumes monosized bubble, i.e. a bubble breaks up into two equalized daughter bubbles [41]. In this approach each bubble interaction results in a change of interfacial area $\Delta A_i = \frac{1}{3} A_i$.

$$\int_{V_{min}}^{V_{max}} A_i \sum_j S_j dV = \Phi_j \quad (336)$$

$$\Phi_j = S_j \Delta A_i \quad (337)$$

with the interfacial area A_i

$$A_i = \frac{a_i}{n} \quad (338)$$

and bubble number density n , $\Psi = \frac{1}{36\pi}$ for spherical bubbles

$$n = \Psi \frac{a_i^3}{\alpha^2} \quad (339)$$

$$\Phi_j = \frac{1}{3} \frac{1}{\Psi} \left(\frac{\alpha}{a_i} \right)^2 S_j \quad (340)$$

The phase change term can be modelled directly, but within the framework of this manual we will not consider phase change. Thus we gained a transport equation for the interfacial area concentration a_i .

$$\frac{\partial a_i}{\partial t} + \nabla \cdot (\mathbf{u} a_i) = \frac{2}{3} \frac{\dot{\alpha}}{\alpha} a_i + \sum_j \frac{1}{3} \frac{1}{\Psi} \left(\frac{\alpha}{a_i} \right)^2 S_j \quad (341)$$

64.3 Interfacial curvature transport equation

64.3.1 Basic definitions

The IATE diameter model solves a transport equation for the interfacial curvature `kappai_`.

Solves for the interfacial curvature per unit volume of the phase rather than interfacial area per unit volume to avoid stability issues relating to the consistency requirements between the phase fraction and interfacial area per unit volume.

Class description in `IATE.H`

By looking into the sources, we find the following relations

$$a_i = \alpha \kappa \quad (342)$$

$$d_{sm} = \frac{6}{\kappa} \quad (343)$$

Thus, the Sauter mean diameter d_{sm} equals

$$d_{sm} = \frac{6\alpha}{a_i} \quad (344)$$

Which corresponds with the definition given in literature [31, 32].

$$d_{sm} = \frac{6\alpha}{a_i} \quad (345)$$

Listing 498 and 499 show the relevant source code of the IATE diameter model. This source code is the basis for Eqns. (342) and (343).

```

1  //- Return the interfacial area
2  tmp<volScalarField> a() const
3  {
4      return phase_*kappai_;
5  }
```

Listing 498: Definition of the method `a()` of the IATE diameter model class in the file `IATE.H`.

```

1  Foam::tmp<Foam::volScalarField> Foam::diameterModels::IATE::dsm() const
2  {
3      return max(6/max(kappai_, 6/dMax_), dMin_);
4  }
```

Listing 499: Definition of the method `dsm()` of the IATE diameter model class in the file `IATE.C`.

The definition of `kappai_` as the interfacial curvature seems a bit counter-intuitive, as the curvature of a sphere is the inverse of its radius.

64.3.2 Derivation of the governing equations

We will now derive the governing equations for the interfacial curvature κ from the equations for the interfacial area concentration a_i which we derived from the transport equations for the bubble size distribution.

Here we will make no further assumptions, as we are simply rearranging the equations. We start from the transport equation for the interfacial area concentration a_i and OpenFOAMs definition of a_i .

$$\frac{\partial a_i}{\partial t} + \nabla \cdot (\mathbf{u} a_i) = \frac{2}{3} \frac{\dot{\alpha}}{\alpha} a_i + \sum_j \frac{1}{3} \frac{1}{\Psi} \left(\frac{\alpha}{a_i} \right)^2 S_j \quad (341)$$

$$a_i = \alpha \kappa \quad (342)$$

Inserting (342) into (341) yields

$$\frac{\partial \alpha \kappa}{\partial t} + \nabla \cdot (\mathbf{u} \alpha \kappa) = \frac{2}{3} \frac{\dot{\alpha}}{\alpha} \alpha \kappa + \sum_j \frac{1}{3} \frac{1}{\Psi} \left(\frac{\alpha}{\alpha \kappa} \right)^2 S_j \quad (346)$$

Next, we partial derivation of all terms containing κ

$$\alpha \frac{\partial \kappa}{\partial t} + \kappa \frac{\partial \alpha}{\partial t} + \kappa \nabla \cdot (\mathbf{u} \alpha) + \alpha \mathbf{u} \cdot \nabla \kappa = \frac{2}{3} \dot{\alpha} \kappa + \sum_j \frac{1}{3} \frac{1}{\Psi} \left(\frac{1}{\kappa} \right)^2 S_j \quad (347)$$

$$\underbrace{\kappa \left[\frac{\partial \alpha}{\partial t} + \nabla \cdot (\mathbf{u} \alpha) \right]}_{\dot{\alpha}} + \alpha \left[\frac{\partial \kappa}{\partial t} + \mathbf{u} \cdot \nabla \kappa \right] = \frac{2}{3} \dot{\alpha} \kappa + \sum_j \frac{1}{3} \frac{1}{\Psi} \left(\frac{1}{\kappa} \right)^2 S_j \quad (348)$$

$$\alpha \left[\frac{\partial \kappa}{\partial t} + \mathbf{u} \cdot \nabla \kappa \right] = -\frac{1}{3} \dot{\alpha} \kappa + \sum_j \frac{1}{3} \frac{1}{\Psi} \left(\frac{1}{\kappa} \right)^2 S_j \quad (349)$$

$$\frac{\partial \kappa}{\partial t} + \mathbf{u} \cdot \nabla \kappa = -\frac{1}{3} \dot{\alpha} \frac{\kappa}{\alpha} + \sum_j \frac{1}{3} \frac{1}{\Psi} \frac{1}{\alpha} \left(\frac{1}{\kappa} \right)^2 S_j \quad (350)$$

With $\frac{1}{\kappa} = \frac{\alpha}{a_i}$

$$\underbrace{\frac{\partial \kappa}{\partial t} + \nabla \cdot (\kappa \mathbf{u})}_I - \underbrace{\kappa \nabla \cdot \mathbf{u}}_{II} = \underbrace{-\frac{1}{3} \frac{\dot{\alpha}}{\alpha} \kappa}_{III} + \frac{1}{3\Psi} \left(\frac{\alpha}{a_i} \right)^2 \sum_j \frac{S_j}{\alpha} \quad (351)$$

The form of Eqn. (351) is chosen to match the equations given in [33]. The second term of the RHS has exactly the same form as the equivalent terms in [33].

64.3.3 Implemented equations

Thus, we have derived a transport equation for κ . However, we still need to check the equations that are implemented in OpenFOAM. Therefore, we take a look at the source code.

In Listing 500 we see the main code for the transport equation. In Line 4 we see the terms marked with I of Eqn. (351). In Line 5 term II of Eqn. (351) is implemented.

```

1 // Construct the interfacial curvature equation
2 fvScalarMatrix kappaIEqn
3 (
4     fvm::ddt(kappaI_) + fvm::div(phase_.phi(), kappaI_)
5     - fvm::Sp(fvc::div(phase_.phi()), kappaI_)
6     ==
7     - fvm::SuSp(R, kappaI_)
8 //+ Rph() // Omit the nucleation/condensation term
9 );

```

Listing 500: Construction of the transport equation in the file IATE.C.

The right hand side of the equation in Listing 500 combines all term of the RHS of Eqn. (351) into the term `fvm::SuSp(R, kappai_)`. The method `fvm::SuSp()` implements a source term for a matrix equation. The arguments translate to $R * kappai_$.

The first term on the RHS of Eqn. (351) is due to the change of bubble volume (dilatation effect). The code in Listing 501 translates to Eqn. (352).

```

1 // Initialise the accumulated source term to the dilatation effect
2 volScalarField R
3 (
4     (
5         (1.0/3.0)
6         /max
7         (
8             fvc::average(phase_ + phase_.oldTime()),
9             residualAlpha_
10        )
11    )
12    *(fvc::ddt(phase_) + fvc::div(phase_.alphaPhi()))
13 );

```

Listing 501: The first term of the RHS of Eqn. (351) of the transport equation in the file `IATE.C`.

$$R = \frac{1}{3} \frac{\frac{\partial \alpha}{\partial t} + \nabla \cdot (\alpha \mathbf{u})}{\alpha} \quad (352)$$

The method call `fvm::SuSp(R, kappai_)` multiplies R with $kappai_$. Thus we recognize the first term of the RHS of Eq. (351).

$$\begin{array}{ll} \text{Eqn. (351)} & III = -\frac{1}{3} \frac{\dot{\alpha}}{\alpha} \kappa \end{array} \quad (353)$$

$$\begin{array}{ll} \text{OpenFOAM} & III = -R\kappa \end{array} \quad (354)$$

$$R = \frac{1}{3} \frac{\dot{\alpha}}{\alpha} \quad (355)$$

$$III = -\frac{1}{3} \frac{\dot{\alpha}}{\alpha} \kappa \quad (356)$$

The other source terms related to the models for bubble-bubble interaction are added to R . Listing 502 shows the loop over all sources, note the minus sign.

```

1 // Accumulate the run-time selectable sources
2 forAll(sources_, j)
3 {
4     R -= sources_[j].R();
5 }

```

Listing 502: The second term of the RMS of Eqn. (351) of the transport equation in the file `IATE.C`.

For the interaction models the minus of Listing 502 cancels the minus of Listing 500.

64.4 Interaction models

OpenFOAM provides a base class for all models related to bubble-bubble interaction. There are several interaction mechanisms implemented.

1. Breakage due to impact of turbulent eddies (TI - *turbulent impact*)
2. Coalescence through random collision driven by turbulent eddies (RC - *random collision*)
3. Coalescence due to acceleration of the following bubble in the wake of preceding bubble (WE - *wake entrainment*)

The base class is named `IATEsource` and it defines a pure virtual function named `R()`. This means that every derived class has to provide its implementation of `R()`. Besides `R()`, the base class provides a number of helper methods that are used in the derived classes, e.g. bubble Reynolds number `Re()` or the Weber number `We()`.

64.4.1 Turbulent impact - TI

In [32, 33] the source term due to turbulent impact is stated as:

$$n = \Psi \frac{a_i^3}{\alpha^2} \quad (357)$$

$$u_t = \sqrt{2k} \quad (358)$$

$$R_{TI} = C_{TI} \left(\frac{nu_t}{D_b} \right) \exp \left(-\frac{We_{cr}}{We} \right) \sqrt{1 - \frac{We_{cr}}{We}} \quad \text{where } We_{cr} > We \quad (359)$$

The Weber number We can be seen as the ratio of inertia forces and surface tension forces and is defined as:

$$We = \frac{\rho u^2 d}{\sigma} \quad (360)$$

with

ρ	density
u	characteristic velocity
d	characteristic length scale
σ	surface tension

The Weber number is provided by the class `IATSource` as the base class for all interaction models. See Section 64.4.4 for implementation details.

The critical Weber number We_{cr} and the model constant C_{TI} must be provided by the user in the appropriate dictionary.

64.4.2 Random collision - RC

In [32, 33] the source term due to random collision is stated as:

$$u_t = \sqrt{2k} \quad (361)$$

$$R_{RC} = C_{RC} \left[\frac{n^2 u_t D_b^2}{\alpha_{max}^{1/3} (\alpha_{max}^{1/3} - \alpha^{1/3})} \right] \left[1 - \exp \left(-C \frac{\alpha_{max}^{1/3} \alpha^{1/3}}{\alpha_{max}^{1/3} - \alpha^{1/3}} \right) \right] \quad (362)$$

The model constants C_{RC} , C and α_{max} need to be provided by the user.

64.4.3 Wake entrainment - WE

In [32, 33] the source term due to wake entrainment is stated as:

$$R_{WE} = C_{WE} C_D^{1/3} n^2 D_b^2 u_r \quad (363)$$

The model constant C_{WE} needs to be provided by the user.

64.4.4 Implementation details of the `IATSource` class

Weber number

The Weber number is implemented in the class `IATSource`, see Listing 503. This definition makes use the method `Ur()`, which is also provided by `IATSource`.

```

1 Foam::tmp<Foam::volScalarField> Foam::diameterModels::IATSource::We()
2 const
3 {
4     return otherPhase().rho()*sqr(Ur())*phase().d()/fluid().sigma();
5 }

```

Listing 503: The definition of the Weber number We in `IATSource.C`

Relative velocity

The relative velocity between the bubbles and the surrounding fluid is given by [30, 38]. Compare Eqn. (364) and Listing 504.

$$u_r = \sqrt{2} \left[\frac{\sigma g \Delta \rho}{\rho_L^2} \right]^{1/4} (1 - \alpha)^{1.75} \quad (364)$$

```
1 Foam::tmp<Foam::volScalarField> Foam::diameterModels::IATeSource::Ur() const
2 {
3     const uniformDimensionedVectorField& g =
4         phase().U().db().lookupObject<uniformDimensionedVectorField>("g");
5
6     return
7         sqrt(2.0)
8         *pow(0.25
9             (
10                 fluid().sigma()*mag(g)
11                 *(otherPhase().rho() - phase().rho())
12                 /sqr(otherPhase().rho())
13             )
14         *pow(max(1 - phase(), scalar(0)), 1.75);
15 }
```

Listing 504: The definition of the relative velocity between bubbles and surrounding liquid in `IATeSource.C`

The IATE implicitly applies only to bubbly systems, i.e. gas-liquid systems. If the IATE model is applied to the denser phase, then Line 11 of Listing 504 leads to a floating-point exception (FPE). If `phase` refers to the liquid phase, then Line 11 evaluates to a negative number. Raising a negative number to a non-integer power is not possible within the domain of the real numbers. Thus, OpenFOAM will issue an error message due to an floating-point exception.

Comparing the formulations

Here we take a closer look on the implementation of the source terms. Listing 505 shows the method `R()` of the `IATeSource` class.

```
1 Foam::tmp<Foam::volScalarField>
2 Foam::diameterModels::IATeSources::turbulentBreakUp::R() const
3 {
4     tmp<volScalarField> tR
5     (
6         new volScalarField
7         (
8             IOobject
9             (
10                 "R",
11                 iate_.phase().U().time().timeName(),
12                 iate_.phase().mesh()
13             ),
14             iate_.phase().U().mesh(),
15             dimensionedScalar("R", dimless/dimTime, 0)
16         )
17     );
18
19     volScalarField R = tR();
20     scalar Cti = Cti_.value();
21     scalar WeCr = WeCr_.value();
22     volScalarField Ut(this->Ut());
23     volScalarField We(this->We());
24     const volScalarField& d(iate_.d());
25
26     forAll(R, celli)
27     {
28         if (We[celli] > WeCr)
29         {
```

```

30         R[celli] =
31             (1.0/3.0)
32             *Cti/d[celli]
33             *Ut[celli]
34             *sqrt(1 - WeCr/We[celli])
35             *exp(-WeCr/We[celli]);
36     }
37 }
38
39 return tR;
40 }

```

Listing 505: The definition of the method `R()` in `turbulentBreakUp.C`

Listing 505 translates to the following mathematical expression:

$$R_{TI} = \frac{1}{3} \frac{C_{TI}}{d_{sm}} u_t \sqrt{1 - \frac{We_{cr}}{We}} \exp\left(-\frac{We_{cr}}{We}\right) \quad \text{where } We > We_{cr} \quad (365)$$

Comparing Eqns. (359) and (365) reveals some differences in formulation. This is due to the fact, that Eq. (359) is a source term for the transport equation for the interfacial area concentration a_i and Eq. (365) is a source term for the transport equation for the interfacial curvature κ .

In the derivation of the curvature equation from the area concentration equation we divided by the volume fraction. Otherwise, only rearrangement and variable substitution was performed.

For this term we now have a look on the RHS of the equations for a_i and κ and compare the implementation of OpenFOAM with the equations stated in literature.

We begin with repeating the equations for a_i and κ . The interaction source term S_j can be found in this form in [33].

$$\frac{\partial a_i}{\partial t} + \nabla \cdot (\mathbf{u} a_i) = \frac{2}{3} \frac{\dot{\alpha}}{\alpha} a_i + \sum_j \frac{1}{3} \frac{1}{\Psi} \left(\frac{\alpha}{a_i}\right)^2 S_j \quad (341)$$

$$\frac{\partial \kappa}{\partial t} + \nabla (\kappa \mathbf{u}) - \kappa \nabla \cdot \mathbf{u} = -\frac{1}{3} \frac{\dot{\alpha}}{\alpha} \kappa + \underbrace{\frac{1}{3\Psi} \left(\frac{\alpha}{a_i}\right)^2 \sum_j \frac{S_j}{\alpha}}_{IV} \quad (351)$$

The interaction model source terms in the curvature equation of OpenFOAM takes the following form:

$$\frac{\partial \kappa}{\partial t} + \nabla (\kappa \mathbf{u}) - \kappa \nabla \cdot \mathbf{u} = -\frac{1}{3} \frac{\dot{\alpha}}{\alpha} \kappa + \underbrace{\sum_j R_j \kappa}_{IV} \quad (366)$$

We now compare the terms marked with IV of Eqns. (351) and (366). As these terms must be equal, we can form the following equation.

$$\frac{1}{3\Psi} \left(\frac{\alpha}{a_i}\right)^2 \sum_j \frac{S_j}{\alpha} = \sum_j R_j \kappa \quad (367)$$

We now demand, that the summands need to be equal, and we focus on the term for turbulent break-up (TI)

$$\frac{1}{3\Psi} \left(\frac{\alpha}{a_i}\right)^2 \frac{1}{\alpha} \underbrace{C_{TI} \left(\frac{nu_t}{D_b}\right) \exp\left(-\frac{We_{cr}}{We}\right) \sqrt{1 - \frac{We_{cr}}{We}}}_{S_j} = \frac{1}{3} \frac{C_{TI}}{d_{sm}} u_t \underbrace{\sqrt{1 - \frac{We_{cr}}{We}} \exp\left(-\frac{We_{cr}}{We}\right) \kappa}_{R_j} \quad (368)$$

Next, we cancel all common symbols and expressions, note the different symbols for the bubble diameter ($D_b = d_{sm}$)

$$\frac{1}{\Psi} \left(\frac{\alpha}{a_i}\right)^2 \frac{1}{\alpha} n = \kappa \quad (369)$$

We now insert the definition of n , see Eq. (357)

$$\frac{1}{\Psi} \left(\frac{\alpha}{a_i} \right)^2 \frac{1}{\alpha} \Psi \frac{a_i^3}{\alpha^2} = \kappa \quad (370)$$

$$\frac{a_i}{\alpha} = \kappa \quad (371)$$

We now end up with an equation that is fulfilled, when we look at the definition of κ , see Eqn. (342)

$$a_i = \alpha \kappa \quad (342)$$

Thus, we have demonstrated on the example of the source term for turbulent break-up of bubbles, that the implementation of OpenFOAM follows exactly the model published in [33].

64.5 Appendix

64.5.1 The proof for Eqn. (335)

We use the following symbols.

$$x = V \quad (372)$$

$$f(x) = f(V, \mathbf{x}, t) \quad (373)$$

$$g(x) = A_i(V) \quad (374)$$

$$a = V_{min} \quad (375)$$

$$b = V_{max} \quad (376)$$

Thus, the LHS of Eqn. (335) becomes

$$\int_{V_{min}}^{V_{max}} A_i \frac{\partial}{\partial V} \left(f \frac{dV}{dt} \right) dV = \int_a^b g(x) \frac{\partial}{\partial x} \left(f(x) \frac{dx}{dt} \right) dx \quad (377)$$

Now, we apply partial integration

$$\int_a^b g(x) \frac{\partial}{\partial x} \left(f(x) \frac{dx}{dt} \right) dx = \left[f(x) \frac{dx}{dt} g(x) \right]_a^b - \int_a^b \frac{\partial g(x)}{\partial x} \left(f(x) \frac{dx}{dt} \right) dx \quad (378)$$

As $f(x)$ is a probability density distribution it has the following properties

$$f(a) = f(b) = 0 \quad (379)$$

Thus, the first term of the RHS of Eqn. (378) vanishes

$$\int_a^b g(x) \frac{\partial}{\partial x} \left(f(x) \frac{dx}{dt} \right) dx = - \int_a^b \frac{\partial g(x)}{\partial x} \left(f(x) \frac{dx}{dt} \right) dx \quad (380)$$

We now take a closer look on the relation between the average interfacial area of a bubble A_i and the volume of a bubble V .

$$A_i = d^2 \pi \quad (381)$$

$$V = \frac{d^3 \pi}{6} \quad (382)$$

$$\Rightarrow d = \sqrt[3]{\frac{6V}{\pi}} \quad (383)$$

$$A_i = \left(\frac{6V}{\pi} \right)^{2/3} \pi \quad (384)$$

Returning to our simplified notation for this proof

$$g(x) = \left(\frac{6x}{\pi}\right)^{2/3} \pi \quad (385)$$

For Eqn. (380) we also need the derivative of $g(x)$

$$\frac{\partial g(x)}{\partial x} = \frac{2}{3} \left(\frac{6}{\pi}\right)^{2/3} (x)^{-1/3} \pi \quad (386)$$

$$\frac{\partial g(x)}{\partial x} = \frac{2}{3} \left(\frac{6}{\pi}\right)^{2/3} \frac{x^{2/3}}{x} \pi \quad (387)$$

$$\frac{\partial g(x)}{\partial x} = \frac{2}{3} \frac{1}{x} \left(\frac{6x}{\pi}\right)^{2/3} \pi \quad (388)$$

$$\frac{\partial g(x)}{\partial x} = \frac{2}{3} \frac{g(x)}{x} \quad (389)$$

We now insert Eqn. (380) into Eqn. (380).

$$\int_a^b g(x) \frac{\partial}{\partial x} \left(f(x) \frac{dx}{dt} \right) dx = - \int_a^b \frac{2}{3} \frac{g(x)}{x} \left(f(x) \frac{dx}{dt} \right) dx \quad (390)$$

$$\int_a^b g(x) \frac{\partial}{\partial x} \left(f(x) \frac{dx}{dt} \right) dx = - \frac{2}{3} \int_a^b \frac{\dot{x}}{x} f(x) g(x) dx \quad (391)$$

Next, we take a closer look on the term $\frac{\dot{x}}{x}$. Since x is the volume of the bubbles V , we can relate V to the void fraction or gas phase volume fraction α . For any control volume V_{CV} we can state, that the volume of the bubbles V is equal to the volume fraction times the control volume. Here we neglect mass transfer by evaporation, see [32] for a derivation considering evaporation.

$$V = \alpha V_{CV} \quad (392)$$

$$\dot{V} = \dot{\alpha} V_{CV} \quad (393)$$

$$\frac{\dot{V}}{V} = \frac{\dot{\alpha} V_{CV}}{\alpha V_{CV}} = \frac{\dot{\alpha}}{\alpha} \quad (394)$$

$$\frac{\dot{x}}{x} = \frac{\dot{\alpha}}{\alpha} \quad (395)$$

We further assume that the rate of change of volume is independent of the volume itself [32, 38].

$$\frac{\dot{V}}{V} \neq f(V) \quad (396)$$

By using relation (391) and (395) on Eqn. (391), we gain

$$\int_a^b g(x) \frac{\partial}{\partial x} \left(f(x) \frac{dx}{dt} \right) dx = - \frac{2}{3} \frac{\dot{\alpha}}{\alpha} \int_a^b f(x) g(x) dx \quad (397)$$

or by using the other notation, Eqns. (372)-(372)

$$\int_a^b g(x) \frac{\partial}{\partial x} \left(f(x) \frac{dx}{dt} \right) dx = - \frac{2}{3} \frac{\dot{\alpha}}{\alpha} \int_{V_{min}}^{V_{max}} f A_i dV \quad (398)$$

$$\int_a^b g(x) \frac{\partial}{\partial x} \left(f(x) \frac{dx}{dt} \right) dx = - \frac{2}{3} \frac{\dot{\alpha}}{\alpha} a_i \quad (399)$$

And by using (323) on (398) we have proofed (335).

65 Derivation of the governing equations for the MRF approach

65.1 Preliminary observations

In order to use the MRF approach the mesh has to be divided into different regions. As the MRF approach in OpenFOAM covers only rotating reference frames²²⁸ only rotation can be imposed on a region. A region for which a non-zero rotation is specified has to be axis-symmetric with respect to the rotational axis. Furthermore, older versions of OpenFOAM²²⁹ only support steady rotation, i.e. the angular velocity ω is constant.

65.1.1 Pitfalls

Axi-symmetric region for MRF rotation

As stated above, the MRF region for applying reference frame rotations has to be axis-symmetric. However, OpenFOAM performs no checks to test whether this is the case. Thus, on a region axis-symmetric with respect to the z -axis, may very well be used to prescribe a reference frame rotation around the x -axis. The results will be blatantly wrong, and the solver might crash. However, if you are unlucky enough, the solver will not crash and compute woefully wrong results, wasting time and computational resources.



Always check whether the **axis** of the **MRFProperties** coincides with the symmetry axis of your MRF zone. Furthermore, make sure that **origin** of the **MRFProperties** lies on the symmetry axis of the MRF zone.

65.2 Mass conservation equation

The mass conservation equation for incompressible flows

$$\nabla \cdot \mathbf{u} = 0 \quad (400)$$

is valid in all inertial frames of reference. An inertial frame of reference is either fixed in space and time or moving with a constant translational velocity.

Due to the constraints listed in the previous section we consider only rotating reference frames in the MRF method. To translate a vector from its representation in the inertial frame of reference to the rotating frame of reference a rotation matrix $\mathbf{Q}$ is used. A rotation matrix has the property that the inverse is also the transposed.

$${}_R\mathbf{u} = \mathbf{Q}\mathbf{u} \quad (401)$$

$$\mathbf{u} = \mathbf{Q}^T {}_R\mathbf{u} \quad (402)$$

$$\mathbf{Q}^{-1} = \mathbf{Q}^T \quad (403)$$

The index R before the symbol $\mathbf{u}$ denotes that the vector ${}_R\mathbf{u}$ is given in the rotating frame of reference. If there is no index before the symbol the vector is given in the inertial coordinate system. The index R is put before the symbol to prevent the vector ${}_R\mathbf{u}$ to be mistaken as a relative velocity.

Beginning with the mass conservation equation in the inertial coordinate system we derive the mass conservation in the rotating coordinate system.

$$\nabla \cdot \mathbf{u} = 0 \quad (404)$$

$$\nabla \cdot \left(\underbrace{\mathbf{Q}^T \mathbf{Q}}_{=\mathbf{I}} \mathbf{u} \right) = 0 \quad (405)$$

$$\nabla \cdot (\mathbf{Q}^T {}_R\mathbf{u}) = 0 \quad (406)$$

²²⁸E.g. in Fluent it is possible to prescribe frame motion with unsteady translational and rotational speeds [7]. This leads essentially to more additional terms in the governing equations. OpenFOAM, however, limits the frame motion to steady rotation.

²²⁹Prior to OpenFOAM-3.0, see Section 65.4.2.

We use the relation $\nabla \cdot (\mathbf{A} \cdot \mathbf{a}) = (\nabla \cdot \mathbf{A}) \cdot \mathbf{a} + \mathbf{A} : (\nabla \mathbf{a})$ and the note that the rotation matrix is constant in space.

$$\nabla \cdot (\mathbf{Q}^T {}_R \mathbf{u}) = \underbrace{(\nabla \cdot \mathbf{Q}^T)}_{=0} \cdot {}_R \mathbf{u} + \mathbf{Q}^T : (\nabla {}_R \mathbf{u}) \quad (407)$$

$$\nabla \cdot (\mathbf{Q}^T {}_R \mathbf{u}) = \mathbf{Q}^T : (\nabla {}_R \mathbf{u}) \quad (408)$$

$$\mathbf{Q}^T : (\nabla {}_R \mathbf{u}) = 0 \quad (409)$$

Next we multiply the equation from the left with the rotation matrix.

$$\mathbf{Q} \mathbf{Q}^T : (\nabla {}_R \mathbf{u}) = 0 \quad (410)$$

$$\mathbf{I} : (\nabla {}_R \mathbf{u}) = 0 \quad (411)$$

We remember that the contraction of the unit tensor and a velocity gradient is equal to the divergence of the velocity.

$$\mathbf{I} : (\nabla {}_R \mathbf{u}) = \nabla \cdot {}_R \mathbf{u} \quad (412)$$

$$\nabla \cdot {}_R \mathbf{u} = 0 \quad (413)$$

Thus, we showed that the mass conservation equation with the velocity expressed in the rotating coordinate system has the same formulation as the mass conservation equation in inertial coordinates.

65.3 Momentum conservation equation

When we use a rotating coordinate system, we can decompose the flow velocity in two components. The first is due to the rotation of the frame of reference and the second is the relative motion between the particle or fluid parcel under consideration and the rotating reference frame.

$$\mathbf{u} = \boldsymbol{\omega} \times \mathbf{r} + \mathbf{u}_R \quad (414)$$

Eq. (414) can also be written in this form using the spin tensor $\boldsymbol{\Omega}$

$$\mathbf{u} = \boldsymbol{\Omega} \mathbf{r} + \mathbf{u}_R \quad (415)$$

The spin tensor is a skew-symmetric tensor that contains the components of $\boldsymbol{\omega}$, the angular velocity vector.

$$\boldsymbol{\Omega} = \begin{pmatrix} 0 & -\omega_z & \omega_y \\ \omega_z & 0 & -\omega_x \\ -\omega_y & \omega_x & 0 \end{pmatrix} \quad (416)$$

We now derive the momentum equation for the velocity for the rotating zone starting from the momentum conservation equation for incompressible Newtonian fluids.

$$\frac{\partial \mathbf{u}}{\partial t} + (\nabla \mathbf{u}) \cdot \mathbf{u} = -\frac{\nabla p}{\rho} + \nabla \cdot (\nu \nabla \mathbf{u}) \quad (417)$$

The terms on the LHS are the total time derivate of $\mathbf{u}$. Thus, we can write

$$\frac{d\mathbf{u}}{dt} = -\frac{\nabla p}{\rho} + \nabla \cdot (\nu \nabla \mathbf{u}) \quad (418)$$

With Eq. (414) we yield

$$\frac{d}{dt} (\boldsymbol{\Omega} \mathbf{r} + \mathbf{u}_R) = -\frac{\nabla p}{\rho} + \nabla \cdot (\nu \nabla \mathbf{u}) \quad (419)$$

We consider only steady rotation; thus, the temporal derivative of the spin vanishes

$$\frac{d\mathbf{u}_R}{dt} + \underbrace{\frac{d\boldsymbol{\Omega}}{dt} \mathbf{r}}_{=0} + \boldsymbol{\Omega} \frac{d\mathbf{r}}{dt} = -\frac{\nabla p}{\rho} + \nabla \cdot (\nu \nabla \mathbf{u}) \quad (420)$$

$$\frac{d\mathbf{u}_R}{dt} + \boldsymbol{\Omega} \mathbf{u} = -\frac{\nabla p}{\rho} + \nabla \cdot (\nu \nabla \mathbf{u}) \quad (421)$$

We now evaluate the total time derivative of $\mathbf{u}_R$

$$\frac{\partial \mathbf{u}_R}{\partial t} + \underbrace{\frac{\partial \mathbf{u}_R}{\partial \mathbf{x}}}_{=\nabla \mathbf{u}_R} \underbrace{\frac{d\mathbf{x}}{dt}}_{=\mathbf{u}} + \boldsymbol{\Omega} \mathbf{u} = -\frac{\nabla p}{\rho} + \nabla \cdot (\nu \nabla \mathbf{u}) \quad (422)$$

$$\frac{\partial \mathbf{u}_R}{\partial t} + (\nabla \mathbf{u}_R) \cdot \mathbf{u} + \boldsymbol{\Omega} \mathbf{u} = -\frac{\nabla p}{\rho} + \nabla \cdot (\nu \nabla \mathbf{u}) \quad (423)$$

Now we insert $\mathbf{u}_R$ from Eq. (414) into the local derivative

$$\frac{\partial}{\partial t} (\mathbf{u} - \boldsymbol{\Omega} \mathbf{r}) + (\nabla \mathbf{u}_R) \cdot \mathbf{u} + \boldsymbol{\Omega} \mathbf{u} = -\frac{\nabla p}{\rho} + \nabla \cdot (\nu \nabla \mathbf{u}) \quad (424)$$

As the velocity component due to the steady rotation of the reference frame is constant, the term $\frac{\partial}{\partial t} (\boldsymbol{\Omega} \mathbf{r})$ will vanish

$$\frac{\partial \mathbf{u}}{\partial t} + (\nabla \mathbf{u}_R) \cdot \mathbf{u} + \boldsymbol{\Omega} \mathbf{u} = -\frac{\nabla p}{\rho} + \nabla \cdot (\nu \nabla \mathbf{u}) \quad (425)$$

The second term on the LHS can be rewritten using the following identity

$$\nabla (\mathbf{a} \mathbf{b}) = (\nabla \cdot \mathbf{b}) \mathbf{a} + (\nabla \mathbf{a}) \cdot \mathbf{b} \quad (426)$$

$$\frac{\partial \mathbf{u}}{\partial t} + \nabla \cdot (\mathbf{u}_R \mathbf{u}) + \boldsymbol{\Omega} \mathbf{u} = -\frac{\nabla p}{\rho} + \nabla \cdot (\nu \nabla \mathbf{u}) \quad (427)$$

Thus, we derived the governing equation for the absolute velocity using flux relative to the local frame of reference.

The contribution from the rotation of the domain is limited to two terms in the governing equation. The LHS of Eq. (428) contains the relative velocity in the second term. The RHS of Eq. (428) contains the Coriolis force in the last term.

$$\frac{\partial \mathbf{u}}{\partial t} + \nabla \cdot (\mathbf{u}_R \mathbf{u}) = -\frac{\nabla p}{\rho} + \nabla \cdot (\nu \nabla \mathbf{u}) - \boldsymbol{\Omega} \mathbf{u} \quad (428)$$

Eq. (428) corresponds to the momentum equation in absolute velocity formulation that can be found in the Fluent Theory Manual [7]. Another resource for the MRF approach in OpenFOAM is [46].

65.4 Notes on the implementation of the MRF Approach

Adding Coriolis forces in the cells of the moving zone is not the only operation necessary for the MRF approach. The boundary conditions of the rotor have to be adjusted. As the rotor is moving the fluid velocity at the rotor walls is not zero. The velocity at the walls has to equal the solid body rotational velocity of the rotor.

65.4.1 OpenFOAM-2.*

In OpenFOAM-2.2.x the MRF method is part of the *fvOptions* mechanism²³⁰. This is a general mechanism that allows for run-time selectable physics. The *fvOptions* framework is a generalization for the source terms in the momentum equation. Via this framework the MRF approach can be selected along with momentum (e.g. wind turbine rotors), porosity (i.e. for modelling porous zones) and energy sources (e.g. for regions with heat transfer).

To provide this flexibility the *fvOptions* framework is implemented using an abstract class to define the behaviour of the general source. Derived class implement the actual physics, e.g. the MRF method. Thus the class `MRFSource` is derived from `option`.

The constructor of the class `MRFSource` calls the method `initialise()`. This method is defined in the class `MRFSource` and calls the method `correctBoundaryVelocity` of the class `MRFZone`. In the method `correctBoundaryVelocity` the velocity values of boundaries within an MRF-zone are set to the solid body rotational velocity. Otherwise the no-slip boundary condition would enforce a zero absolute velocity which would be clearly wrong.

²³⁰See <http://www.openfoam.org/version2.2.0/fvOptions.php>

In Listing 506 we see the prescription of the solid body velocity for all faces that lie within the MRF-zone. On these faces the solid body velocity is prescribed.

$$\mathbf{u}_{rot} = \boldsymbol{\omega} \times (\mathbf{r}_{face} - \mathbf{r}_{origin}) \quad (429)$$

```

1 void Foam::MRFZone::correctBoundaryVelocity(volVectorField& U) const
2 {
3     const vector Omega = this->Omega();
4     // Included patches
5     forAll(includedFaces_, patchi)
6     {
7         const vectorField& patchC = mesh_.Cf().boundaryField()[patchi];
8         vectorField pfld(U.boundaryField()[patchi]);
9         forAll(includedFaces_[patchi], i)
10        {
11            label patchFacei = includedFaces_[patchi][i];
12            pfld[patchFacei] = (Omega ^ (patchC[patchFacei] - origin_));
13        }
14        U.boundaryField()[patchi] == pfld;
15    }
16 }

```

Listing 506: The method `correctBoundaryVelocity` in the file `MRFZone.C`

Capabilities and limitations of the MRF approach

The MRF method in OpenFOAM deals only with rotations other than FLUENT, which is also capable of accounting for translational movement [7]. In both CFD softwares the velocity of the moving reference frame needs to be constant. This means OpenFOAM is capable of dealing with rotating reference frames that move with a constant angular velocity.

The boundary of the zone, in which the rotation of the frame of reference is active, must be oriented in a way, so that the velocity component of the reference frame's velocity normal to the boundary is zero. This means for a rotating frame of reference the zone in which this movement is acting needs to be a cylinder²³¹ with its axis parallel to the axis of rotation of the reference frame.

The FLUENT theory manual says that the MRF method is strictly speaking only valid for steady state cases [7]. However, FLUENT offers this method for unsteady simulations too [7].

The sliding mesh technique gives more accurate results than the MRF method especially when it comes to transient simulations. However, the main advantage of the MRF method is its low impact on computational cost, compared to moving mesh techniques.

65.4.2 OpenFOAM-3.*

With OpenFOAM-3.0.0²³² the MRF method was taken out from the *fvOptions* framework. The developers of OpenFOAM give the following reason for this move:

fvOptions does not have the appropriate structure to support MRF as it is based on option selection by user-specified fields whereas MRF MUST be applied to all velocity fields in the particular solver. A consequence of the particular design choices in *fvOptions* made it difficult to support MRF for multiphase and it is easier to support frame-related and field related options separately.

Currently the MRF functionality provided supports only rotations but the structure will be generalized to support other frame motions including linear acceleration, SRF rotation and 6DoF which will be run-time selectable.

As noted in the cited message above, the MRF framework was also generalized. Thus, rotating reference frames are not limited to steady rotations anymore. Listing 507 shows how this change allows to spin-up reference frame rotation. This might improve numerical behaviour in the initial stages of the simulation.

²³¹In fact the zone can be any volume defined by any surface of revolution of the rotational axis of the reference frame. However, the cylinder is the easiest and most convenient choice.

²³²<http://www.openfoam.org/version3.0.0/>

```

MRF1
{
    cellZone    rotor;

    active      yes;

    nonRotatingPatches ();

    origin      (0 0 0);
    axis        (0 0 1);

    omega        table
                2(
                    (0 0.01)
                    (0.5 104.72)
                );
}

```

Listing 507: Passing a table as angular velocity to the MRF framework.

Part XI

Appendix

66 Useful Linux commands

66.1 Getting help

66.1.1 Display `-help`

Virtually all Linux commands display a summary of the programs purpose and usage. To display this message the command has to be invoked with one of those parameters: `-h`, `-help`, `--help`. If the wrong parameter is used the help message is displayed anyway or an error message naming the correct parameter to display the usage information, see Listing 508.

```
user@host:~$ ls -help
ls: invalid option -- e
Try 'ls --help' for more information.
user@host:~$
```

Listing 508: Displaying the help message

Apparently all of the tools and solvers of OpenFOAM²³³ display such help messages. New Linux and OpenFOAM users are strongly encouraged to study the help messages to deepen their understanding and insight.

66.1.2 *man* pages

Many Linux commands have an additional, more detailed documentation²³⁴. This is written in the *man* pages (*man* is short for manual). To display the *man* pages of a certain command, simply put the name of the command or program behind the command `man`. Listing 509 shows how to display the *man* pages of the Linux command `cp`.

```
man cp
```

Listing 509: Displaying the *man* pages

The *man* pages cover general commands of Linux, system call, library function of the C standard library and much more. On some systems the *man* pages are only partially or not at all installed by default.

66.2 Finding files

66.2.1 Searching files system wide

Searching for a file on the whole file system can be done by `locate`. Listing 510 shows the result of the search for the source file of *icoFoam*.

```
user@host:~/OpenFOAM/user-2.1.x/run/icoTurb$ locate icoFoam.C
/home/user/OpenFOAM/OpenFOAM-2.0.x/applications/solvers/incompressible/icoFoam/icoFoam.C
/home/user/OpenFOAM/OpenFOAM-2.1.x/applications/solvers/incompressible/icoFoam/icoFoam.C
```

Listing 510: Looking for *icoFoam.C*

66.2.2 In a certain directory

To find a file in a certain directory and its sub-directories `find` can be used. Listing 511 shows the command to search the file `LESProperties` in the OpenFOAM tutorials.

²³³No exception is known to the author.

²³⁴As an example: the *man* pages of `gcc` are longer than 10000 lines.

```
find $FOAM_TUTORIALS -name LESProperties
```

Listing 511: Search *LESProperties* in the tutorials

66.3 Find files and scan them

How do I define probes? I have seen this already, but where?

To answer this question one has to find all files in which *probes* can be defined – the *controlDict* in this case. Additionally, all of the files returned by the search have to be scanned for the definition of *probes*. As an OpenFOAM case consists of a number of text files, it is easy to scan these files for certain keywords. So, the answer to the question above is: find all *controlDict*s and scan them for the word *probe*.

Instead of performing this task manually, a single one-liner in the Terminal does the magic. Listing 512 shows how all files named *controlDict* in the tutorials are located and scanned for the word *probes*.

```
find $FOAM_TUTORIALS -name controlDict | xargs grep 'probes' -sl
```

Listing 512: Find and scan files

find looks for respectively finds all files with the name passed with the option *-name* in the specified folder and its folders. *xargs* executes the passed command line. The output of *find* is passed to *grep* as input by a pipe. *grep* then scans all files for the word *probes*.

git it done the other way

If OpenFOAM is installed from a *git* source code repository, we can also use *git* to search for patterns. Here, all (tracked) files are searched for the provided pattern. The reason for using *git*, is that *git* tracks file content in a very efficient way. Thus searching is generally faster, especially if we scan a lot of files. E.g. if we want to know where in the sources *VGREAT* is defined.

In the case of the example above, we would need to go to the tutorials directory and then tell *git* to search for the pattern “*probes*”. This causes an overall search, which is not limited to *controlDict*s. In fact, we find also some clean-up scripts.

```
cd $FOAM_TUTORIALS
git grep probes
```

Listing 513: Scan files using *git grep*

66.4 Scan a log file

grep can scan a text file for a certain pattern. In this example we want to scan the solver output for a certain pattern. The solver *twoPhaseEulerFoam* displays after every time step the minimum and maximum value of the volume fraction α . For α to be physically meaningful, its value has to be of the range $0 \leq \alpha \leq 1$.

In this example a simulation crashed and the main suspicion is, that there were values of α greater than one. Listing 514 shows two lines of solver output. The first line has a maximum value of one. In some cases, when regions evolve where the continuous phase vanishes, e.g. above a water surface, this value is perfectly reasonable. The second line comprises a maximum value of α greater than unity. This value is unphysical, because a phase can not occupy a certain amount of space – a cell – to more than 100%.

Due to the fact that simulations often do not crash immediately the log file containing the solver output is hundreds of thousands of lines long. To look for maximum values of α greater than unity manually is not an option. We need an one-liner that does that automatically for us. That’s where *grep* comes in.

```
Dispersed phase volume fraction = 0.194351  Min(alpha) = 7.52826e-42  Max(alpha) = 1
Dispersed phase volume fraction = 0.060562  Min(alpha) = 2.30261e-52  Max(alpha) = 1.00003
```

Listing 514: Example: solver output regarding volume fraction

Listing 515 shows how the user can scan the log file for the appropriate pattern. `grep` expects as first argument the pattern to look for. The second argument is optional, it specifies the file from which to read. If no file was specified, `grep` would read from standard input. The option `-c` makes `grep` display only the number of number of matches. Otherwise, `grep` would display all lines in which a match was found. In a situation in which the number of hits could reach hundreds or thousands, displaying all lines with a match could be unwise.

The first command in Listing 515 would detect a match for both lines of Listings 514. So this pattern `'Max(alpha) = 1'` is not useful to find out whether α exceeded unity or not.

The second command in Listing 515 will only detect lines in which α is larger than unity. So, of the two lines of Listings 514, only the second one would result in a match.

```
grep 'Max(alpha) = 1' foamRun.log -c
grep 'Max(alpha) = 1.' foamRun.log -c
```

Listing 515: Scan the log using *grep*

66.5 Running in scripts

66.5.1 Starting a batch of jobs

To use the computing power of a computing cluster it is a good idea to let the cluster do the work in batches. To be able to do this, this section explains how to use a script to run a number of simulations sequentially. So, the cluster can calculate a great number of cases without the need for the user to start each job separately. This would be unacceptable when simulating overnights.

The script in Listing 516 starts two parallel simulations including domain decomposition and reconstruction. The script assumes to start from a directory which contains all two cases. The first group of commands changes into a subdirectory of the current directory (`cd './fullColumn_fineV01'`). The next commands perform all tasks of a parallel simulation. Then the script changes to the second case (`cd './fullColumn_fineV02'`).

This is a very basic script. It contains no checks if a simulation has terminated prematurely or any other useful features.

```
#!/bin/bash
# fine 01

echo 'fine01'
cd './fullColumn_fineV01'

echo 'decomposing'
decomposePar > foamDecompose.log

mpirun -np 2 twoPhaseEulerFoam -parallel > foamRun.log

echo 'reconstructing'
reconstructPar > foamReconstruct.log

# fine 02
echo 'fine02'
cd './fullColumn_fineV02'

echo 'decomposing'
decomposePar > foamDecompose.log

mpirun -np 2 twoPhaseEulerFoam -parallel > foamRun.log

echo 'reconstructing'
reconstructPar > foamReconstruct.log
```

Listing 516: Using a shell script to start several simulations.

66.5.2 Terminating a running script

There may be need to stop a script from any further execution without terminating the currently running simulation. This example assumes that a script with name `runCalculations` is to be terminated. First the PID

of *runCalculations* has to be known. In Section 12.3.2 explains this bit in detail. Listing 516 shows how to look for the PID. The command in Listing 516 outputs two lines. The first line comes from the running script and the second line stems from the running parallel calculation. This is because all running processes matching the pattern *run* were searched for. Therefore, also the running instance of *mpirun* was found.

```
user@host:~$ ps -el | grep run
0 S  8553 14913 14517  0  80   0 -  2687 wait    pts/11    00:00:00 runCalculations
0 S  8553 14917 14913  0  80   0 -  2687 wait    pts/11    00:00:00 mpirun
user@host:~$
```

Listing 517: Search for PIDs using *ps* and *grep*

Terminate the script

If the script was terminated using *kill*, then the simulation would continue unaffected. Listing 518 shows how the script is terminated and *mpirun* continues to be running.

```
user@host:~$ ps -e | grep run
14913 pts/11    00:00:00 runCalculations
14917 pts/11    00:00:00 mpirun
user@host:~$ kill -KILL 14913
user@host:~$ ps -e | grep run
14917 pts/11    00:00:00 mpirun
```

Listing 518: Use *kill* to stop a shell script.

Terminate the script and the simulation

To terminate both the script and the simulation – in this example – the running simulation has to be terminated also. Terminating only the running simulation only, will cause the script to execute the next command. So, first the script and then the simulation need to be terminated.

66.6 diff

diff is a command line tool that analyses two files and prints a summary of the differences of those files. Further information on *diff* can be found in the man-pages or the help-message.

66.6.1 Meld

Meld is a graphical front-end to *diff*. This allows for a side-by-side comparison of both files under investigation. Parts of the file that differ are highlighted by colors. For more information about *Meld* see <http://meldmerge.org/>.

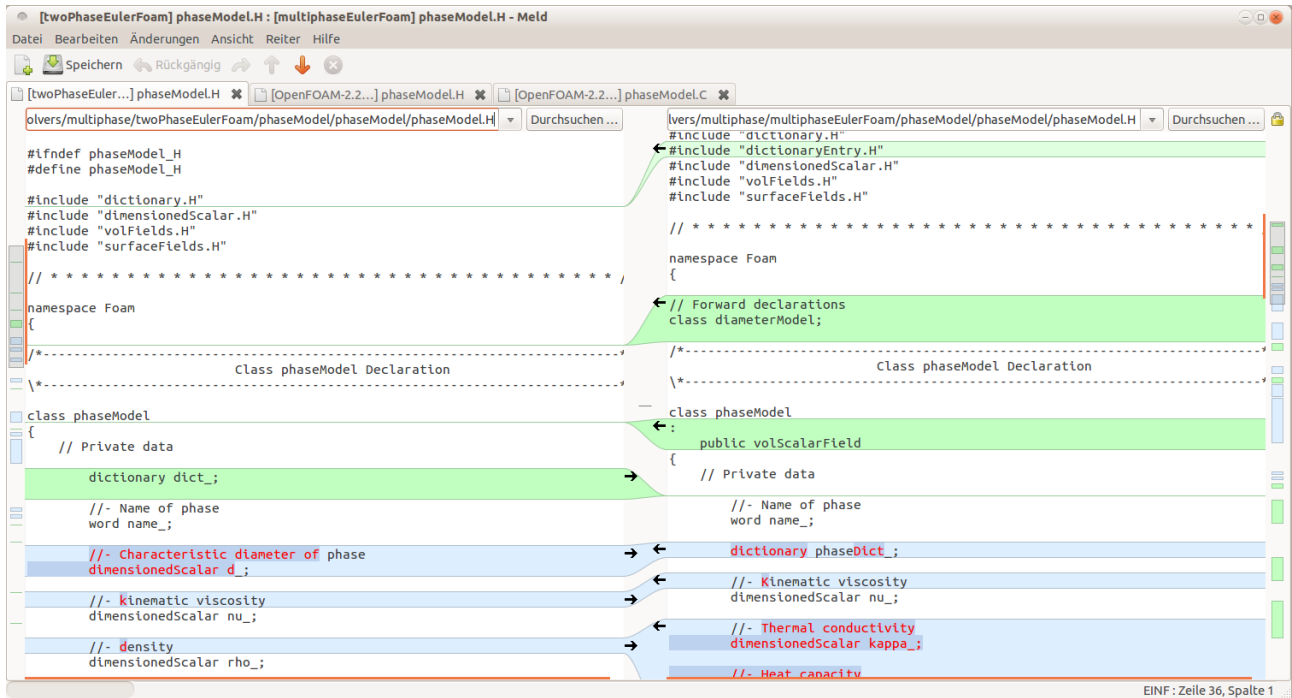


Figure 138: A screenshot of *Meld*

66.7 Case setup

There are a lot of tasks when setting up a case. Even though we might use a tutorial case as a starting point, a lot of tedious work might lie ahead of us. Computers are better at certain tasks than we humans [\[Citation needed\]](#).

Thus, we might be better off automating boring tasks. The pyFoam library is a good collection of useful stuff. However, certain tasks are done by mere one-liners in Linux. This is what this section is about.

66.7.1 Renaming files

The current multi-phase solvers use a naming scheme in which the name of the phase determines the file's extension. Thus, the thermophysical properties of the phase air are stored in the file `thermophysicalProperties.air`. This causes in certain cases the need to rename a number of files, because we use argon as gaseous phase and we want to comply with the naming scheme. The dirty hack would be to just exchange the properties of air with those of argon without changing the naming of the phase.

In Linux there are many ways to perform this sort of task. For mere text substitution a solution based on regular expressions is the way to go. Listing 519 shows how the command `rename` is used to rename all files containing a certain text.

```
rename 's/air/argon/g' ./*
```

Listing 519: Change the extension of all files having the extension `air` to `argon`.

66.8 Miscellaneous

This section contains references to useful scripts or commands explained elsewhere in this document.

Terminate a background process

See Section [12.3.2](#).

Delete the *processor** directories

If one or several simulations have been conducted on a computing cluster, it makes sense to reconstruct the domain on the cluster. Otherwise the workstation of the user would be blocked for the time needed to complete reconstruction. After reconstructing the domain the *processor** directories still contain all the time step data. If the *processor** folders are deleted on the cluster, the user can afterwards copy the whole case directory to the workstation without transmitting the solution data twice.

See Section 12.5.2 for how to deal with *processor** directories.

Redirect output

Redirecting the output of a program is explained in Section 12.1.1.

67 Archive data

Parametric studies generate a great deal of data. After the post-processing is done all files could be compressed to save disk space. On Linux systems the *tar* archiving utility may be the agent of choice. The name *tar* comes from *tape archive*, which is pretty descriptive in terms of the origins of this archiving program. A *tar* archive is a single file which contains all archived files and folders. This step alone is only a reorganisation of the data, fit for the usage of sequential data storage devices like magnetic tapes.

In a second step the *tar* archive needs to be compressed. For this task there are many possible choices. Linux systems usually provide programs like *gzip*, *bzip2* or *xz*. The distinction between archiving and compressing is probably for historical as well as practical reasons. There is also one paradigm of the UNIX philosophy (*Make each program do one thing well*) which supports the segregation in archiving and compression. The compression programs usually differ in the utilised compression algorithms. There is one rule of thumb stating: The more data is to be compressed, the longer compression takes.

Table 8 lists the achieved compression of a parametric studies with 21 cases totalling in 50 GB of data. The data was written in *ascii* format. Compressing the data resulted in a 70+ % reduction of used disk space. If space consuming cases are to be archived, slow algorithms that result in good compression rates should be preferred.

	used disk space	reduction
21 cases uncompressed	50 GB	
compressed: *.tar.bz2	13.7 GB	36.3 GB - 72.6 %

Table 8: Comparison of disk space reduction

Archive log files

In this example log files are archived. In this case the same algorithm achieves an even greater reduction of disk space usage. This example shows that the achieved compression rate strongly depends on the input data.

	used disk space	reduction
16 log files uncompressed	2.0 GB	
compressed: *.tar.bz2	154.7 MB	1.85 GB - 92.3 %

Table 9: Comparison of disk space reduction

Yet another compression comparison

When archiving OpenFOAM cases, the mesh generation may be very convoluted or time consuming. So it might be preferable to archive the mesh generation separately from the actual case. Thus, for the actual case we might want to include the final mesh and do not bother with mesh creation.

This section compares how three compression tools perform on a Linux system when compressing a mesh. The mesh in question was created by *blockMesh* and certain files are in binary format.

Listing 520 shows the actual command that were used to compress the mesh.

```
tar -cv polyMesh | gzip --best > polyMesh.tar.gz
tar -cv polyMesh | bzip2 --best > polyMesh.tar.bz
tar -cv polyMesh | lzma -9 > polyMesh.tar.xz
```

Listing 520: Compressing the mesh with various tools; note the setting for maximum compression

Listing 521 shows the sizes of the original mesh and its compressed forms. The used compression algorithms differ considerably. Compression was performed with the setting for best compression, the time it took for each tool to compress the mesh was not recorded. However, the stronger the compression the more time it takes to compress. Since for archiving purposes disk space is the limiting factor, we chose maximum compression.

```
user@host:~$ du -sh polyMesh*
23M polyMesh
5,3M polyMesh.tar.bz
6,2M polyMesh.tar.gz
2,6M polyMesh.tar.xz
```

Listing 521: Comparing the disk usage of a mesh and its various compressions

From the comparison in Listing 521 we see that the LZMA compression algorithm achieves the best compression, followed by BZIP2 and GZIP. All three compression algorithms are widely available in the UNIX world in the form of open source implementations.

The effects of renumbering and/or writing binary

When playing around with various ways to archive an existing mesh, something quite interesting has been observed: the archive file-size differs whether the mesh had been renumbered or not. Note that this comparison only deals with archiving the mesh, no solution data (i.e. fields) have been archived alongside the mesh.

In Table 10 we compare the resulting file size of an archive of a reasonable complex, all-hex mesh with 147262 cells. The required disk space of the mesh itself only differs between writing in ascii or binary format. Renumbering the mesh does not have an effect, which is not surprising, since renumbering does not add or remove information. Thus, the required disk space of the mesh should not change due to renumbering.

When it comes to archiving the mesh, however, renumbering certainly has an effect on the resulting file size of the archive. Compressing the *as-is*²³⁵ mesh yields only a slightly smaller archive, when the mesh was written in binary format to disk. The big surprise, however, is when we compare the resulting archive file size for the renumbered mesh. In both cases, ascii and binary, the compressed, renumbered mesh results in a larger archive file size than the compressed, *as-is* mesh.

The mesh	ascii	binary	The archive	ascii	binary
as-is	22M	16M	as-is	1,7M	1,6M
renumbered	22M	16M	renumbered	2,5M	2,0M

Table 10: Comparing the resulting file size of the mesh archive file for various conditions/treatments. All file or folder sizes were determined with the Linux command `du -sh FILE`. The mesh was compressed using the LZMA algorithm at maximum compression: `tar -cv constant/polyMesh | lzma -9 > polyMesh.tar.xz`.

We can draw two conclusions from this comparison. The most obvious is, that writing data in binary format is always favourable over writing in ascii format, especially for uncompressed data. Writing in ascii should only be used for trouble-shooting. In production, binary is definitely the format of choice. Writing in binary not only saves disk space in storage, it also provides the maximum read/write precision, since all relevant bits of each numerical value are stored. In contrast, writing in ascii format only writes a finite number of significant bits.

²³⁵We denote the state of the mesh after mesh-creation by the term *as-is* in order to distinguish it from the renumbered state. Mesh creation consists of a call to *blockMesh*, extruding a patch and finally removing parts of the mesh with *subsetMesh*. After these steps, any sets of points, faces or cells are deleted. Also zones of any kind have not been defined. Thus, *as-is* denotes the final, minimal mesh.

The second conclusion we can draw from the comparison above, is not to store the renumbered mesh. Even if renumbering would/does²³⁶ not increase archive file size, renumbering is an operation we can easily perform prior to our simulation. The time it takes to renumber the mesh is negligible compared to the overall simulation time. Thus, we advise to archive meshes in an un-renumbered state. This, in addition to potential savings on storage, gives future users the choice whether to run the simulation on a renumbered mesh or not. On an already renumbered mesh it is impossible to determine the benefits of renumbering the mesh with respect to simulation time or convergence. An operation which can always be performed at demand does not need to be archived.

²³⁶One case of renumbering causing larger archive file sizes does not necessarily mean that this will always cause this effect.

References

- [1] *Intel 64 and IA-32 Architectures Optimization Reference Manual*.
- [2] Viscosity of gases. In David R. Lide, editor, *CRC Handbook of Chemistry and Physics*. CRC Press, 2005.
- [3] The International System of Units, 2006. URL www.bipm.org/en/si/si_brochure.
- [4] The International System of Units (SI), 2008. URL <http://physics.nist.gov/Pubs/SP330/sp330.pdf>.
- [5] A. Alexandrescu. *Modern C++ Design: Generic Programming and Design Patterns Applied*. Addison Wesley, 2001.
- [6] J. D. Anderson. *Computational Fluid Dynamics*. McGraw-Hill International Editions, 1995.
- [7] Inc. ANSYS. *FLUENT Theory Guide*, 14.5 edition, 2012.
- [8] ANSYS, Inc. *ANSYS CFX-Solver Theory Guide*, 14.0 edition, November 2011.
- [9] N. G. Deen B. Niceno, M. T. Dhotre. One.equation sub-grid scale (sgs) modelling for euler-euler large eddy simulation (eeles) of dispersed bubbly flow. *Chemical Engineering Science*, 63:3923–3931, 2008.
- [10] A. Behzadi, R. I. Issa, and H. Rusche. Modelling of dispersed bubble and droplet flow at high phase fractions. *Chemical Engineering Science*, 59:759–770, 2004.
- [11] J. Boussinesq. Théorie de l’Écoulement tourbillant. *Mem. Présentés par Drivers Savants Acad. Sci. Inst. Fr.*, 23:46–50, 1877.
- [12] Daniel Brennan. *The Numerical Simulation of Two-Phase Flows in Settling Tanks*. PhD thesis, Imperial College of Science, Technology & Medicine, 2001.
- [13] A. Burcat and B. Ruscic. Third millennium ideal gas and condensed phase thermochemical database for combustion and updates from active thermochemical tables. Technical report, Argonne National Laboratory, 2005.
- [14] N. Böcker, M. Grahl, A. Tota, P. Häussinger, P. Leitgeb, and B. Schmücker. *Ullmann’s Encyclopedia of Industrial Chemistry*, chapter Nitrogen. Wiley-VCH Verlag GmbH & Co. KGaA, 2013.
- [15] Yunus A. Çengel. *Heat Transfer - a practical approach*. McGraw-Hill, second edition, 2003.
- [16] S. Chapman and T. G. Cowling. *Mathematical theory of non-uniform gases*. Cambridge University Press, 1952.
- [17] C. P. Dahl. *Numerical modelling of flow and settling in secondary settling tanks*. PhD thesis, Aalborg University, Denmark, 1993.
- [18] J.-M. Delhay. Some issues related to the modeling of interfacial areas in gas-liquid flows I. the conceptual issues. *Comptes Rendus de l’Académie des Sciences - Series {IIB} - Mechanics*, 329(5):397–410, 2001.
- [19] M. T. Dhotre and J. B. Joshi. CFD simulation of heat transfer in turbulent pipe flow. *Industrial & Engineering Chemistry Research*, 43:2816–2829, 2004.
- [20] Eric S. Raymond. *The Art of UNIX Programming*. Addison-Wesley, 2003.
- [21] A. Eucken. Über die temperaturabhängigkeit der wärmeleitfähigkeit einiger gase. *Physikalische Zeitschrift*, 12(24):1101–1107, 1911.
- [22] A. Eucken. Über das wärmeleitvermögen, die spezifische wärme und die innere reibung der gase. *Physikalische Zeitschrift*, 14(8):324–331, 1913.
- [23] Agner Fog. Optimizing software in c++. Technical report, Technical University of Denmark, 2014.
- [24] J. Fröhlich. *Large Eddy Simulationen turbulenter Strömungen*. Teubner, 2006.
- [25] E. Peirano & A.-E. Almstedt H. Enwald. Eulerian two-phase flow theory applied to fluidization. *Int. J. Multiphase Flow*, 22:21–66, 1996.

- [26] David P. Hill. *The computer simulation of dispersed two-phase flows*. PhD thesis, Imperial College of Science, Technology and Medicine, 1998.
- [27] C. L. Hogan and R. B. Sawyer. The thermal conductivity of metal at high temperature. *Journal of Applied Physics*, 23(2):177–180, 1952.
- [28] B. Holenda, I. Pásztor, Á. Kárpáti, and Á Rédey. Comparison of one-dimensional secondary settling tank models. Technical report, European Water Association (EWA), 2006.
- [29] J. R. Howell, R. Siegel, and Mengüç. *Thermal Radiation Heat Transfer*. CRC Press, fifth edition, 2010.
- [30] M. Ishii. One-dimensional drift-flux-model and constitutive equations for relative motion between phase in various two-phase flow regimes. Technical report, Argonne National Laboratory, 1977.
- [31] M. Ishii and T. Hibiki. *Thermo-Fluid Dynamics of Two-Phase Flow*. Springer, 2nd edition, 2011.
- [32] M. Ishii, S. Kim, and J. Uhle. Interfacial area transport equation: model development and benchmark experiments. *International Journal of Heat and Mass Transfer*, 45:3111–3123, 2002.
- [33] M. Ishii, S. Kim, and J. Kelly. Development of interfacial area transport equation. *Nuclear Engineering and Technology*, 37(6):525–536, 2005.
- [34] R. I. Issa. A simple model for c_t . Private Communications, 1992. see Hill [26].
- [35] M. Peric J. H. Ferziger. *Computational Methods for Fluid Dynamics*. Springer, 2002.
- [36] Hrvoje Jasak. *Error Analysis and Estimation for the Finite Volume Method with Applications to Fluid Flows*. PhD thesis, Imperial College of Science, Technology & Medicine, 1996.
- [37] Brian W. Kernighan and Dennis M. Ritchie. *The C Programming Language*. Prentice Hall, Inc., 2nd edition, 1988.
- [38] S. Kim, X. Sun, M. Ishii, S. G. Beus, and F. Lincoln. Interfacial area transport and evaluation of source and sink terms for confined air-water bubbly flow. *Nuclear Engineering and Design*, 219:61–75, 2002.
- [39] G. Kocamustafaogullari and M. Ishii. Foundation of the interfacial area transport equation and its closure relations. *Int. J. Heat Mass Transfer*, 38(3):481–493, 1995.
- [40] Fabian Peng Kärrholm. *Numerical Modelling of Diesel Spray Injection, Turbulence Interaction and Combustion*. PhD thesis, Chalmers University of Technology, Göteborg, Sweden, 2008.
- [41] Y. Liu, T. Hibiki, and M. Ishii. Modeling of interfacial area transport in two-phase flows. In *Advances in Multiphase Flow and Heat Transfer*, volume 4, chapter 1, pages 3–27. Bentham Science Publishers, 2012.
- [42] Alejandro López. Lpt for erosion modeling in openfoam – differences between solidparticle and kinematicparcel, and how to add erosion modeling. Technical report, Chalmers University of Technology, 2014. URL http://www.tfd.chalmers.se/~hani/kurser/OS_CFD_2013/AlejandroLopez/LPT_for_erosionModelling_report.pdf.
- [43] G. B. Macpherson, N. Nordin, and H. G. Weller. Particle tracking in unstructured, arbitrary polyhedral meshes for use in cfd and molecular dynamics. *Communications in Numerical Methods in Engineering*, 25: 263–273, 2009.
- [44] Holger Marschall. *Towards the Numerical Simulation of Multi-Scale Two-Phase Flows*. PhD thesis, Technische Universität München, 2011.
- [45] M. Milelli. *A numerical analysis of confined turbulent bubble plumes*. PhD thesis, Swiss Federal Institute of Technology Zurich, 2002.
- [46] Hakan Nilsson. Turbomachinery training at ofw8. Technical report, Chalmers University of Technology, Gothenburg, Sweden, 2013.
- [47] Niklas Nordin. *Complex Chemistry Modeling of Diesel Spray Combustion*. PhD thesis, Chalmers University of Technology, 2009.

- [48] International Union of Pure and Applied Chemistry. Iupac compendium of chemical terminology – the gold book, 2014. URL <http://goldbook.iupac.org/>.
- [49] *OpenFOAM - Programmer's Guide*. OpenFOAM Foundation, 2.1.0 edition, 2011.
- [50] *OpenFOAM - User Guide*. OpenFOAM Foundation, 2.1.0 edition, 2011.
- [51] D. Pfleger and S. Becker. Modelling and simulation of the dynamic flow behaviour in a bubble column. *Chemical Engineering Science*, 56:1737–1747, 2001.
- [52] B. E. Poling, J. M. Prausnitz, and J. P. O'Connell. *The Properties of Gases and Liquids*. McGraw-Hill, fifth edition edition, 2000.
- [53] S. P. Pope. *Turbulent Flows*. Cambridge University Press, 2000.
- [54] Henrik Rusche. *Computational Fluid Dynamics of dispersed two-phase flows at high phase fractions*. PhD thesis, Imperial College of Science, Technology & Medicine, 2002.
- [55] Y. Sato and K. Sekoguchi. Liquid velocity distribution in two-phase flow. *International Journal of Multiphase Flow*, 2:79–95, 1975.
- [56] J. Smagorinsky. General circulation experiments with the primitive equations; i. the basic experiment. *Monthly Weather Review*, 91:99, 1963.
- [57] Bjarne Stroustrup. *The C++ Programming language*. Addison-Wesley, 4th edition, 2013.
- [58] William Sutherland. Lii. the viscosity of gases and molecular force. *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, 36(223):507–531, 1893.
- [59] Imre Takács. *Experiments in Activated Sludge Modelling*. PhD thesis, Ghent University, Belgium, 2008.
- [60] D. G. Thomas. Transport characteristics of suspension: VIII. a note on the viscosity of Newtonian suspensions of uniform spherical particles. *Journal of Colloid Science*, 1965.
- [61] A. Tomiyama, I. Kataoka, T. Fukuda, and T. Sakaguchi. Drag coefficients of bubbles. 2nd report. drag coefficient for a swarm of bubbles and its applicability to transient flow. *Nippon Kikai Gakkai Ronbunshu*, 61(588):2810–2817, 1995.
- [62] VDI-Gesellschaft Verfahrenstechnik und Chemieingenieurwesen (GVC). Thermophysical properties. In *VDI Heat Atlas*. Springer, 2010.
- [63] Berend van Wachem. *Derivation, implementation and validation of computer simulation models for gas-solid fluidized beds*. PhD thesis, Delft University of Technology, 2000.
- [64] H. K. Versteeg and W. Malalasekera. *An introduction to computational fluid dynamics – the finite volume method*. Longman Scientific & Technical, 1995.
- [65] A. Vesilind. Design of prototype thickeners from batch settling tests. *Water Sewage Works*, 115(5):302–307, 1968.
- [66] H. G. Weller, G. Tabor, H. Jasak, and C. Fureby. A tensorial approach to computational continuum mechanics using object-oriented techniques. *Computers in Physics*, 12:620–631, 1998.
- [67] David C. Wilcox. *Turbulence Modelling for CFD*. DCW Industries, Inc., 1994.
- [68] D. Zhang, N. G. Deen, and J. A. M. Kuipers. Numerical simulation of the dynamic flow behaviour in a bubble column: A study of closures for turbulence and interface forces. *Chemical Engineering Science*, 61:7593–7608, 2006.

Nomenclature

ACMI	Arbitrary Coupled Mesh Interface	MPI	message passing interface
AMI	Arbitrary Mesh Interface	MRF	multiple reference frame
ASCII	American Standard Code for Information Interchange	OO	object-oriented
BC	boundary condition	OOD	object-oriented design
BIT	Bubble induced turbulence	OOP	object oriented programming
CAD	computer aided design	OS	operating system
CFD	Computational fluid dynamics	PDE	Partial differential equation
CG	Conjugate gradient	Perl	An interpreted programming language
DO	discrete ordinates	PID	process identifier
DPE	Dispersed phase element	PIMPLE	An algorithm based on PISO and SIMPLE algorithm
EDF	Électricité de France	PISO	Pressure Implicit with Split Operator
FIFO	A data structure follwing the First In, First Out principle	POSIX	Portable Operating System Interface
FPE	Floating-point exception	RAS	Reynolds averaged simulation
FVM	Finite volume method	RHS	Right hand side
GAMG	Geometric algebraic multi-grid	RNG	random number generator
gcc	GNU compiler collection	RTD	residence time distribution
GNU	GNU is not Unix	SAT	Standard ACIS Text
GUI	graphical user interface	SI	Le Système Internationale d'Unités
I/O	input and output	SIMPLE	Semi-Implicit Method for Pressure-Linked Equations
IATE	Interfacial area transport equation	SOI	start of injection
IGES	Initial Graphics Exchange Specification	STL	Surface Tessellation Language
IT	Information technology	UNIX	an operating system; ancestor of many modern operating systems, e.g. all kinds of Linux, Mac OS X.
LES	Large eddy simulation	VOF	Volume of fluid
LPT	Lagrangian Particle Tracking		
LZMA	Lempel-Ziv-Markov chain algorithm		